QueryBook Bible

Canonical Core Technology Specification

Authoritative Architecture, Terminology, and Patent Reference — Version 1.0

Document Date:	**August 1, 2026**
Classification:	**Confidential — Proprietary and Patent-Sensitive**
Prepared By:	**QueryBook Core Technology Group**
Version:	**1.0 — Initial Canonical Release**
Document Status:	**Authoritative — Single Source of Truth**
Audience:	**Patent Attorneys, Senior Engineers, AI Researchers, Product Architects**
Location:	**Los Angeles, CA, United States**
Prepared for:	**CTO, QueryBook Core Technology Group**

CONFIDENTIAL — PROPRIETARY AND PATENT-SENSITIVE

DO NOT DISTRIBUTE WITHOUT EXPLICIT WRITTEN AUTHORIZATION

LEGAL NOTICE **This document constitutes the canonical, authoritative specification of the QueryBook system and all its proprietary subsystems, algorithms, protocols, data formats, and terminology. All content herein is the exclusive intellectual property of the QueryBook Core Technology Group. Unauthorized reproduction, distribution, reverse engineering, or use of any specification, algorithm, or terminology defined in this document is strictly prohibited and may constitute misappropriation of trade secrets and infringement of pending and issued patents. This document is designated as a confidential trade secret. It is intended solely for use by authorized patent attorneys, licensed engineers, and designated product architects operating under non-disclosure obligations.**

Merge Revision Note — Version 1.0, Merge Review Pass 1

MERGE-ONLY REVISION The additions below were merged into this Bible following a technical audit. Nothing in the original document was removed, reworded, or reordered — all changes are additive, and each is marked inline as a "merge note" or "added in Version 1.0 merge review" at its point of insertion so original and added text remain distinguishable.

Additions in this pass:

- Domain 0 — The Prime Directive: full D0.1–D0.8 content added (Part I). Previously present only as a Table of Contents heading with no body content.
- §3.1.1 — Implementation note reconciling the FUID's deterministic, content-derived generation with standard RFC 9562 UUIDv7 semantics, plus a cross-reference added to the Chapter 2 glossary entry.
- Acronym disambiguation notes added at all three definitions of "ARE" (§7.2.3 Associative Recall Engine, Chapter 52 Adaptive Response Engine, §39.6 App Recommendation Engine).
- Appendix C, §C.11 — Methodology and Assumptions section added, documenting the workload, comparison, cost, and energy basis behind the quantitative claims in §C.10.
- §15.8 — AEM-7: Constraint Satisfaction and Optimization Module, a new AGI Extension Module added to Chapter 15.

Not yet addressed in this pass: Chapters 21–38 and 40–58 remain Table-of-Contents-only stubs, as do Appendices A, B, D, E, F, and G. These require substantive new drafting rather than a merge edit and are recommended as a follow-up pass. ***[Corrected in Pass 3: this claim was wrong about Appendices A, B, D, E, F, G and about Chapters 53–59 specifically — both actually exist in the original document. See the Pass 3 note below.]***

Merge Revision Note — Version 1.0, Merge Review Pass 2

Additions in this pass (also merge-only; nothing removed or reworded):

- §9.5.3 — Unified Audit API: a new consolidated section merging the audit capabilities previously scattered across §5.4 (Provenance Ledger audit/compliance), §8.5 (Safety Audit Log), and the two AUDITOR-role endpoints already in §9.5.1, into five callable endpoints (/v1/audit/export, /v1/audit/export/{export_id}, /v1/audit/verify/{export_id}, /v1/audit/log, /v1/audit/soc2-trail). Cross-reference notes were added at each original location (§5.4.1, §8.5.1, §9.5.1) pointing to the new consolidated section.

- §19.8 — Q&A: a new feature, the Specification Assistant Agent — a dedicated AI chatbot for asking questions about this Bible's own design, built on QueryBook's own Fact Graph, ARE, and MRE stack applied self-referentially to the Bible's text, with a new /v1/qa/ask and /v1/qa/sections/{section_id} API surface.

Not yet addressed: the same Chapters 21–38, 40–58 and Appendices A, B, D, E, F, G gap noted in Pass 1 remains open. ***[Corrected in Pass 3: this repeated the same error as Pass 1. See the Pass 3 note below for the accurate gap list.]***

Merge Revision Note — Version 1.0, Merge Review Pass 3

This pass corrects two errors from Pass 1 and Pass 2 (both marked inline above where they occurred) and closes the real gaps those errors obscured. Nothing removed; corrections are additive markers plus new content, per the same merge-only rule as all prior passes.

Correction: Appendices A, B, D, E, F, and G are not missing — they exist in the original document as fully-populated tables, formatted as bold inline headings rather than document headings, which is why a heading-pattern search missed them. Chapters 53–59 (the “World Intelligence and Cognitive Architecture” part) are not missing either — they exist in full, but the body text internally numbers them Chapters 25–31 (a consistent off-by-28 defect) instead of matching the Table of Contents' 53–59.

Genuine gap identified and closed in this pass: Chapters 48, 49, 50, and 51 have no body content anywhere under any numbering scheme, despite being cited by section number from content that already exists — Chapter 52 cites “Chapter 50, Section 50.4.3” and “Chapter 51” directly; Domain 0, the Unified Audit API, and this Bible's own Q&A feature (§19.8) all cite specific §49.x subsections. All four are now drafted in full and merged into this document, positioned immediately before Chapter 52:

- Chapter 48 — QueryBook as a Semantic Programming Platform (QBQL, QBPL, NLPL), including the two added QBPL primitives (ASSERT/RETRACT, REPEAT-UNTIL-BOUND) and the five-stage NLPL compilation pipeline.
- Chapter 49 — The Live Fact Graph, Universal Ingestion Architecture (all six source types), WATCH programs, and the Continuous Query Engine.
- Chapter 50 — NLIL, the Natural Language Interaction Layer. This also resolves a naming inconsistency: the existing text called this layer both “NLGE” and “NLIL” in different places without defining either; NLGE is retained as one component's name within the NLIL chapter, consistent with both usages already in the text.

- Chapter 51 — The Fact Universe architecture (the Fact Universe Slice and Slice Resolver the ARE's own pipeline table already assumed existed).

Navigation fix: a merge note has been added to each of Chapters 25–31's body headings cross-referencing the correct Table of Contents chapter number (53–59), since the underlying off-by-28 defect is original document content and is not altered here — only annotated — per the merge-only rule.

Also added: Table of Contents entries for Chapters 36 (Cross-Book Query History Analysis) and 37 (Conversational Intelligence Layer), which have real body content but were never listed in the Table of Contents.

Not yet addressed: the internal off-by-28 numbering within Chapters 25–31's body text is annotated, not corrected at the source, since correcting it would mean altering original section numbers throughout seven chapters — recommended as a dedicated renumbering pass with its own review, done separately from a content-merge pass. ***[Done in a later audit-fix pass: this renumbering was subsequently completed at the source. See the note near the end of this Version History for details.]***

Merge Revision Note — Version 1.0, Merge Review Passes 4–6

Passes 4 and 5 merged external content without a formal changelog entry at the time; this note covers both, plus Pass 6.

- Pass 4: merged Chapters 40 (Specification Validation Framework) and 41 (Predictive Cognitive Architecture) from an external draft, plus a Fact Universe (FU-verse) glossary entry (§2), cross-referenced against the pre-existing Chapter 51 Fact Universe Slice mechanism.
- Pass 5: merged Chapter 42 (Proprietary Technologies), documenting UFCS, FQL, WVM, and ten other cognitive technologies from an external submission, with disambiguation notes added where terms overlapped with existing chapters.
- Pass 6 (this pass): reverses part of Pass 5. UFCS and FQL have been fully removed from this Bible — they are not referenced anywhere in this document, including Chapter 42 (renumbered §42.2.1–§42.2.11 to close the gap) and Appendix C, which previously used “UFCS” as shorthand for QueryBook’s own native architecture and now uses “QueryBook Native” instead — no numeric content in Appendix C changed, only the label. All other Chapter 42 technologies (WVM, Ontology Layer, Causal Graph Layer, Temporal Model, Semantic Consistency Engine, Reasoning Engine, Interpretation Layer, Narrative Engine, Deterministic Creativity Engine, Ontology Evolution System, Cognition Boundary System) remain, unchanged in substance, as natively QueryBook technologies with no dependency on any external or independently-operable system.

QueryBook's Universal Ingestion Architecture (Chapter 49) and its REST/gRPC API surface (Chapter 9, §9.5.3) are unaffected by this pass — they are QueryBook's own native integration layer for all source types and never depended on UFCS or FQL.

Merge Revision Note — Version 1.0, Merge Review Passes 7–11

These five passes had no formal changelog entry until now; this note covers all of them.

- Pass 7: merged Chapter 43 (Integration of Hoffman-Derived Mathematical Structures) — Markovian perceptual transitions, trace-logic coarse-graining, observer-window inheritance, adopted as pure mathematics with associated metaphysical claims explicitly excluded.
- Pass 8: rescoped Chapter 43 per explicit instruction — all four integration pathways (MATM, TLCM, POIE, ACOWM) were rewritten to touch only the Scenario Graph's agent-simulation instances and the WASM/RMD embodied-agent subsystem, explicitly not the core Fact Graph, Ontology Mesh, NCL, or Multimodal Ingestion Pipeline.
- Pass 9: merged Chapter 44 (QueryBook Video Colorization & Upscaling Module, VCUM) as an applied World-Model System, consuming WVM/NCL/Scenario Graph without modifying their core specifications, consistent with the Pass 8 scoping precedent applied proactively. Also found and fixed a bold-lead bullet text-duplication bug affecting 20 bullets across Chapters 44 and 48 (the latter dating back to Pass 3).
- Pass 10: merged Chapter 45 (VCUM Agent Specifications) — the six agents (SEA, MRA, LPA, TCA, HCA, ERA) that operationally execute Chapter 44's pipeline, cross-referenced in both directions.
- Pass 11 (this pass, an audit-fix pass): closed every open item from the technical audit performed after Pass 10 — added a Table of Contents entry for Chapter 39 (real content existed but had never been listed, the same gap already fixed for Chapters 36–37 in Pass 3); corrected Chapter 43's §7.2.2 citation, which pointed to a subsection that does not actually discuss the cited mechanism; and completed the Chapters 25–31 → 53–59 renumbering at the source (see the note added directly to the Pass 3 entry above) — the internal off-by-28 defect is now fully resolved rather than merely annotated.

A related correction, external to this document: the Redesign dashboard tool's reference data had incorrectly stated this renumbering was already complete as of Pass 8. That claim described work done in a separate, abandoned draft that was never merged into this document's lineage. It has been corrected in the dashboard to match this Bible's actual state as of each pass.

Merge Revision Note — Version 1.0, Merge Review Pass 12

This pass implements a Redesign Engine analysis (Performance, Efficiency, Economy) against the Pass 11 document, cross-checked against an independently-produced adversarial technical audit provided separately. That audit's findings were verified individually against this document before use — several did not apply (referencing a much larger, ~216-chapter draft this Bible's lineage never included, and one, Chapter 16's grounding claim, was already appropriately hedged as a scored metric rather than an absolute guarantee). Eight findings were confirmed to apply and were addressed as additive notes at each relevant location, not as rewrites of the underlying specification:

- §14.3.1.1 (Performance) — flagged the HNSW vector index's memory footprint at 4096-dim/10B+ scale as very large and previously unstated as an explicit constraint; recommended product/scalar quantization as a standard mitigation, pending benchmarking.
- §8.1.1 (Economy, Performance) — scoped the Safety Substrate's "non-bypassable" guarantee explicitly to QueryBook's own software architecture, not kernel/hardware/insider compromise, reducing misrepresentation risk; and added an explicit throughput/parallelism sizing note for synchronous gating at scale.
- §15.4.2.1 (Efficiency) — flagged AEM-3's PC Algorithm as $O(n^3)$ and computationally intractable at full Fact Graph scale without domain partitioning; added that requirement.
- §15.1 (Economy) — closed a chained-inference risk where uncorroborated AEM output could feed further AEM operations with no visible signal of compounding confidence; added a disclosure requirement.
- Chapter 14 storage tier table (Performance, Economy) — named the CAP-theorem latency/partition tradeoff implicit in synchronous RPO=0 cross-region replication for the Provenance Ledger and Safety Audit Log; concluded the existing scope (limited to those two stores) is the right call, not an error, and left the specified targets unchanged.
- §58.4.1 (Performance) — named the Semantic Brain / World Brain interface's implicit consistency model as eventual rather than synchronous, consistent with its own "continuously enriched" language, to prevent a future implementer from reading in a stricter, CAP-theorem-violating requirement.
- Chapter 13, end of framing section (Economy) — flagged QBF's dual dependency on Protocol Buffers varint field encoding and a separate custom frame format as two independently-versionable surfaces; recommended they be tracked as one coupled compatibility unit.

Also verified independently and found NOT to apply to this document: an overclaimed "hallucination-free" guarantee (Chapter 16 already uses a scored Factual Groundedness metric, not an absolute claim) and several findings referencing chapters or scale figures ("50 parts, 216 chapters") that do not exist in this Bible's lineage. The same external audit also independently identified two issues already fixed in this document as of Pass 1 — the FUID/UUIDv7 contradiction (§3.1.1) and the "ARE" acronym collision — which is treated here as corroboration those fixes addressed real issues, not as new findings.

Merge Revision Note — Version 1.0, Merge Review Passes 13–14

- Pass 13: added §7.6 NCL Integrity and Robustness to the existing Chapter 7 — weight-update integrity for the Synaptic Weight Store, adversarial query resistance for the Associative Recall Engine, and explicit capacity bounds for the Attractor Network, all implemented through mechanisms already specified elsewhere (Provenance Ledger, Safety Substrate, Transport Layer) rather than new machinery. Also corrected a Pass 11 citation error: attractor dynamics was said to be unspecified in Chapter 7 when it is in fact specified at §7.2.4.
- Pass 14: added Chapter 46, Immersive Query Response Rendering (IQR) — an applied World-Model System that renders an eligible query response as a navigable 3D/VR scene (for example, an underwater cave environment for a cave-diving query), built entirely from already-specified components: the WASM Object Layer (§53.3) for environment and material data, VCUM (Chapters 44–45) for material/lighting/detail synthesis, and standard open delivery formats (glTF/USD over WebXR) rather than an invented container format. States plainly that composed scenes are representative reconstructions grounded in general topic facts, not literal reconstructions of one specific real, unphotographed place — deliberately not claiming the bit-identical determinism that is meaningful for VCUM's video-restoration use case but is not meaningful here.
- Also in Pass 14: added §49.3.3, the Source Connector Registry — filling a gap that existed in the original document's own Table of Contents outline but was never written when Chapter 49 was first drafted in Pass 3. This formalizes QueryBook's “universal API” (§9.5.3) as a real extension point for third-party or QueryBook-owned external systems, via a four-part connector contract (capability declaration, authentication, data mapping, health reporting), with an explicit governance note that external connectors receive no trust-boundary exemption — all data crossing the boundary passes through the same Safety Substrate and Prime Directive review as any other source. An external cognitive-substrate system is used as the illustrative example (§49.3.3.3) without QueryBook defining, redefining, or absorbing that external system's own architecture into this Bible.

A separate finding surfaced while drafting Pass 14, noted for a future pass rather than addressed here since it is unrelated to what was requested: Chapter 52 (§52.5, §52.6) cites a “QBCL-CMS (Chapter 47)” twice — a runtime configuration management system for Discipline Mode Registry and operator baseline Response Profile settings — that was outlined in the original Table of Contents but, like §49.3.3 before this pass, was never actually written. This is a real, load-bearing gap distinct from the already-resolved Chapters 48–51 gap, and is recommended as the next chapter-completion priority.

Merge Revision Note — Version 1.0, Merge Review Pass 15

Extends Chapter 46 (IQR) with two new sections, closing the gap between “one scene from one query” and “a curated multi-scene tour an author builds once.”

- §46.9 Tour and Exhibition Mode: a new TOUR Fact Unit type sequencing ordered or Scenario-Graph-branching Waypoints across a topic, chapter, or book; a Coverage Map (§46.9.2) so authors know which topics are scene-eligible before building rather than by trial and error; an authoring interface distinct from the reader-facing Q&A interface, with author overrides to IQR's automatic composition recorded as such in the Provenance Ledger rather than disguised as automatic output; camera/pacing choreography and narration synchronization; and a multi-viewer delivery model for educator-led group sessions.
- §46.10 Presenter Mode: QueryBook's Interactive Modes grouping for Tours, offering Voice-Only, Face-Only, and Face-on-Avatar AI-generated presenter options, per explicit user request. Because all three synthesize a real, identifiable person's voice and/or likeness, this section was built with the authorization architecture as a mandatory precondition, not an optional setting: an Author Authorization Record (§46.10.1), scoped per presenter mode and immediately and retroactively revocable (§46.10.2), gated as a Safety Substrate enforcement point (§8.1.1) with no bypass path, plus a persistent, non-removable AI-generated disclosure on every presenter-mode output (§46.10.6). Face-on-Avatar mode reuses VCUM's existing material/lighting compositing (§44.3) rather than introducing a new visual pipeline.

Merge Revision Note — Version 1.0, Merge Review Passes 16–19

These four passes had no formal changelog entry until now; this note covers all of them, closing a gap discovered while reconciling the Feature Notes dashboard's corpus data against this document's actual state.

- Pass 16: added §52.13, Interactive Mode Registry — 23 reader-facing session-type modes (Q&A, Discussion, Tutorial, Quiz, and others), each independent of Pedagogical Mode (§52.9, which governs response shape, not session type) and Discipline Mode (§52.5, subject-domain conventions). “Immersive VR Mode” and “Author VR Tour Mode” from the source material were deliberately not made separate catalog entries — they route to Chapter 46 (IQR, Tour and Exhibition Mode, Presenter Mode), which already specifies that functionality, avoiding duplication. Discussion Mode was given full treatment (§52.13.3) given its expanded critical-analysis latitude, bounded by the same Safety Substrate review as every other output. §52.12 Patent Significance was renumbered to §52.14 to make room, after verifying zero external citations of §52.12 existed.
- Pass 17: Chapter 1.1 (“What QueryBook Is”) rewritten to lead with QueryBook's actual origin — a Q&A companion for a single ebook — with the existing enterprise/AGI-scale technical description preserved verbatim as what that architecture generalized into, bridging to the Interactive Mode Registry (§52.13).
- Pass 18: per explicit instruction, added Fact-Unit-atom language and verifiable neuromorphic parallels — grounded only in mechanisms actually specified elsewhere in this Bible (STDP, Hebbian strengthening, the Sensory Abstraction Pipeline §55) — to §1.1, §4.1 (Ingestion Pipeline), and §7.1 (NCL Design Philosophy).
- Pass 19: confirmed and made explicit in §4.1 that QueryBook's eight-modality ingestion pipeline (§4.2) already covers audiobooks, video, web pages, live data feeds (via Chapter 49's STREAM/SENSOR source types), applications, and sensor data — verified against the actual modality table before being confirmed, not assumed true.

Two tooling bugs were caught and fixed during this span before reaching delivery, not after: a 14-of-24-row table structure bug in the Interactive Mode Registry (Pass 16), and, separately, an unescaped ampersand and a literal unicode escape sequence typed instead of real characters in the Chapter 1 and Chapter 4/7 narrative additions (Pass 18–19) — both caught by the same pre-build verification pass run before every delivery since the Chapter 58 rendering incident.

Known open item, flagged and not yet fixed: Chapter 52 cites a “QBCL-CMS (Chapter 47)” twice — a runtime configuration management system for the Discipline Mode Registry and operator Response Profile settings — outlined in the original Table of Contents but never written. First flagged during the Pass 11 audit-fix pass. [Closed in Pass 21 — see the note below.]

Merge Revision Note — Version 1.0, Merge Review Pass 21

Closes the QBCL-CMS gap flagged since the Pass 11 audit and re-confirmed during the Feature Notes dashboard reconciliation work.

- Added Chapter 47, QBCL-CMS — Configuration Language and Management System. Defines QBCL, a small declarative configuration format (not Turing-complete, reusing the JSON/YAML/TOML structured-data formats already specified in §4.2 rather than a new file format), and the store itself: Discipline Mode Registry template storage (§47.3, serving §52.5) and operator baseline Response Profile storage (§47.4, serving §52.7.1 and §52.7.4 Step 1) — both dependencies Chapter 52 has cited by this chapter number since Pass 3.

Consistent with the standing instruction to retain the Bible's current purpose and design: this chapter introduces no new execution model, no new transport, and no new governance mechanism. Configuration reads and writes go through the existing REST/gRPC API (Chapter 9); writes are gated by the existing Safety Substrate (§8.1) and logged to the existing Provenance Ledger (Chapter 5); the Safety Override Level floor constraint already stated at §52.7.1 (may be raised, never lowered below STANDARD) is enforced redundantly at both schema validation and the Safety Substrate gate, rather than relying on a single checkpoint.

A TOC-placement mistake was caught and corrected during this pass: the first attempt at inserting Chapter 47's Table of Contents entry used an anchor string that matched an earlier, structurally-identical entry (Chapter 36's, not Chapter 37's, since both share closing text), landing the entry ten chapters too early. Caught by direct verification of the surrounding text before validation, not left for a future audit to find.

Merge Revision Note — Version 1.0, Merge Review Pass 22

Analyzed all 201 unique features from a synced, deduplicated external feature/function registry (245 original rows, cross-referenced against a separate 1,730-row Function Register) for validity and architectural compatibility, per explicit instruction to integrate what checks out. The full triage and reasoning is documented in a standalone report; summarized here:

- 18 features (not 12 as first estimated) were excluded for genuine cause: eight explicitly named after NFT rights-minting or SERE (Self-Evolution Registry Entry), plus six more identified during a closer pass whose names did not contain "SERE" but whose function matches and section origin confirmed they were the same self-evolution proposal/simulate/activate/rollback/telemetry cluster (Proposal engine, Rollback engine, Simulation tester, Evolution telemetry, Evolution provenance,

Controlled self-evolution), plus Activation engine and Substrate gating, confirmed via their section-of-origin field to be SERE components despite generic-sounding names.

- 123 features were already covered by existing chapters — the eight ingestion modalities (Chapter 4), the Provenance Ledger (Chapter 5), the Safety Substrate (Chapter 8), the Scenario Graph (Chapter 11), WASM's World Model Domain (§53.3), and others — and were not re-specified.
- 19 features were the external draft's own internal architecture labels (its D0–D22 domain numbering, “Prime Directive Cortex,” “RSMA,” “MFEM”), not end-user capabilities, and had nothing to integrate.
- 5 features (Substrate engine, Substrate integrity audits, Admin governance flow/panel, Prime Directive enforcement logs) were too thin or redundant with the existing Safety Substrate (Chapter 8) to warrant separate specification.
- 3 features (Rights prechecker, Rights validation, Rights validator) named the same underlying concept three times and were consolidated into one gate (§60.2), deliberately excluding the source material's proposal to represent validated rights as a minted NFT.
- 33 features were genuine, non-duplicate candidates and were built into two new chapters plus two Interactive Mode Registry additions: Chapter 60 (Publisher and Print-on-Demand Distribution — submission intake, the consolidated Rights Validation Gate, print production, external distribution routing via the Source Connector Registry, localization preview, market reporting) and Chapter 61 (Author Drafting Tools — applying Discussion Mode's existing analytical capability and the Scenario Graph's existing branching capability to an in-progress manuscript, one lifecycle stage before publication, rather than specifying new reasoning capabilities). Contextual Footnotes and Social Annotations were added to the Interactive Mode Registry (§52.13.2.1).

A duplication bug identical to the one fixed in Pass 9 was reintroduced and caught during this pass's own drafting (two bold-lead bullets in the Interactive Mode Registry addendum) — found and fixed via the same detection regex before this document was rebuilt, not after.

Nothing in Chapters 60–61 reintroduces UFCS, FQL, a second NCL, or any rights-tokenization/NFT mechanism. Both chapters consume only already-specified components.

Merge Revision Note — Version 1.0, Merge Review Pass 23

Responds to four specific points raised about a rejected external document (the Author VR Tour / Web Portal / License Management / Distribution Territory material declined last turn, for reasons including a

reintroduced UFCS dependency, an author-avatar system with no consent model, and hardcoded third-party API integrations bypassing the Source Connector Registry).

- Point 1 (remove UFCS by name): verified, not newly fixed — all 6 remaining UFCS occurrences in this document are historical changelog record of its Pass 6 removal, this changelog's own confirmation of non-reintroduction, or the one pre-existing, deliberately-labeled external-vendor example at §49.3.3.3. No live UFCS architecture exists anywhere in this Bible.
- Point 2 (author authorization for avatar presentation): verified, not newly built — §46.10.1's Author Authorization Record, §46.10.2's revocation, and §46.10.6's mandatory disclosure were already fully specified in Pass 15 and remain intact. This already covers author-facing, opt-in, per-mode authorization for exactly the avatar-presentation use case described in the rejected document's "Author Avatar Engine."
- Point 3 (personal annotations / "post-it" notes): added as a new Personal Notes entry in the Interactive Mode Registry (§52.13.2.1), distinct from the existing Social Annotations mode — private by default, color-coded categories, tags, and full-text search, stored as reader-scoped Fact Units. Further specification was offered and is expected in a future pass.
- Point 4 (author-created collaboration events — Google Classroom, Google Meet, Zoom, and others): added as Chapter 62, Author-Hosted Reader Events. The full requested capability is specified — authors create events, invite readers, readers join and replay them — built on the Source Connector Registry (§49.3.3) already in this Bible rather than hardcoded, company-specific API endpoints. This is a strictly more general outcome: any collaboration platform that registers as a connector works identically, not only the three named in the source request.

A bold-lead bullet duplication bug (the same pattern originally found across 20 instances in Pass 9, and again in 2 instances during Pass 22) was reintroduced a third time while drafting the new Personal Notes entry, and caught immediately by the same detection regex before this document was rebuilt. Given the recurrence, bold-label bullets in this pass's Chapter 62 content were written as separate explicit runs rather than using the pattern that keeps causing this.

Merge Revision Note — Version 1.0, Merge Review Pass 24

Added Chapter 63, Reader Annotation System, from a real, detailed specification provided directly — data model, four anchor types, sync behavior, and a four-level sharing model. Verified against the existing architecture before building: no UFCS, NFT, or SERE content anywhere in the source material

(unlike prior submissions this pass), no conflict with the already-specified Social Annotations mode (§52.13.2.1).

- The thin Personal Notes bullet added in Pass 23 was replaced with a cross-reference to this chapter rather than left as a duplicate, shorter description sitting alongside the full specification.
- The source specification's data model, anchor types, and sharing modes are preserved in full. Two explicit privacy guarantees from the source — that an author cannot view a reader's private notes, and a publisher cannot view any reader's notes regardless of sharing mode — were carried forward and stated as Safety Substrate enforcement, not merely a policy line, consistent with how every other access boundary in this Bible is specified.
- Sync and storage were deliberately NOT specified as new architecture: a note is a Fact Unit, versioned through the existing Provenance Ledger (Chapter 5), not a new sync protocol. The source material's “instant” sync claim was flagged as an unvalidated design target, consistent with how this Bible already treats unbenchmarked latency figures elsewhere.
- The source material's reference to sharing notes on “message boards” was explicitly called out as outside this chapter's scope (§63.6) — message boards are not yet specified anywhere in this Bible, and PUBLIC-mode note visibility was not stretched to imply a message-board subsystem that doesn't exist.

No new bugs from the recurring bold-lead duplication pattern were introduced this pass — verified before insertion, consistent with the change in drafting approach noted in the Pass 23 entry.

Merge Revision Note — Version 1.0, Merge Review Pass 25

Added Chapter 64, Message Board System, from a real, detailed specification provided directly — board registry, thread/post engine, permissions, moderation, and integration with Personal Notes and events. This is the exact capability Chapter 63 (§63.6) explicitly flagged as out of scope one pass earlier; it is now specified on its own terms, as recommended at the time, rather than assumed as a side effect of the notes system.

- Verified clean before building: no UFCS, FQL, NFT, or SERE content anywhere in the source material.
- Deliberately did NOT build a new Territory Registry despite the source material's “territory enforcement” language. Regional board visibility (§64.5) reuses the regional capability concept already established for distribution routing in Chapter 60 (§60.4), scoped to what a discussion board actually needs. A prior, separately-declined specification proposed a full

territory/pricing/rights-conflict system; nothing from that was inherited here, and §64.5 says so explicitly rather than silently building toward it.

- Chapter 63's SharingMode field now has a real second consumer: §64.6 specifies that a PRIVATE note cannot be shared to a board without the reader first changing its SharingMode — the note-to-thread conversion path this Bible was missing, closing the gap named in Chapter 63 by cross-reference rather than duplication.
- Chapter 62's events now generate an associated Event Thread automatically (§64.7) — an ordinary thread distinguished only by an EventID reference, not a separate event-discussion subsystem.

No new storage layer, search index, or permissions engine was introduced. Boards, threads, and posts are Fact Units; moderation routes through the existing Safety Substrate; search reuses the existing Associative Recall Engine.

Merge Revision Note — Version 1.0, Merge Review Pass 26

Added Chapter 65, Portal Access and Role Model, in response to a request to integrate the just-added Message Board System (Chapter 64) into “the portal description” with access based on four named roles: license owners (readers), authors, publishers, and partners.

- Found, while researching this, that a real Reader Portal already existed in the original document — §36.8's History Interface (Timeline, By-Book, Thematic Map, and Annotation Library views, ZKE-respecting), predating every chapter added this session. This was not previously cross-referenced from any of the newer author/publisher/partner-facing chapters.
- Chapter 65 does not build a new portal UI, login system, or permissions engine. It formalizes License Owner as the umbrella identity, defines the four Roles, and maps each to its actual existing specification: Reader to §36.8/Chapter 46/Chapter 63/Chapter 64, Author to Chapter 61/§46.10/Chapter 62/§64.3, Publisher to Chapter 60/§64.3, and Partner to §49.3.3/§64.3.
- Instructor was deliberately specified as a contextual, per-class capability (Chapter 62's existing event-ownership model) rather than a fifth top-level Role, since the request named only four roles and a per-class-scoped capability already fit Chapter 62 without requiring new architecture.
- §65.4 restates, in one place, three access principles that were previously implicit across several chapters: no portal-specific bypass of the Safety Substrate, every Role action logged to the Provenance Ledger, and the explicit private-note exclusions from §63.4 (author cannot view private notes, publisher cannot view any notes) carried forward unchanged.

Merge Revision Note — Version 1.0, Merge Review Pass 27

Added Appendix H, Representative Runtime Flows, as the recommended alternative to a declined Runtime Architecture / Sequence Diagram / State Machine submission built entirely on UFCS, FQL, and a blockchain-based NFT ownership layer.

- Six flows are formalized, each citing only real, already-specified components: a Q&A response through the actual five-stage ARE Pipeline (§52.8, not a generic “semantic engine”), Personal Note creation and cross-device sync through the Provenance Ledger's existing VersionChain (§63, not a new sync protocol), Message Board thread creation through the existing AccessControlList and Role model (§64–§65), Author-Hosted Event creation through the actual Source Connector Registry (§62, not hardcoded platform APIs), and Tour playback through the real Author Authorization Record chain (§46.10, not an unconsented avatar system).
- §H.6 states the one enforcement pattern shared by all five flows — Safety Substrate gating before a consequential action takes effect, with a specific denial reason and Provenance Ledger logging — once, rather than repeating it in each section.
- Verified before insertion: the only occurrences of “UFCS,” “FQL,” and “blockchain” in this appendix are the scope note's own explicit statement that none of them are used — not live architecture. Zero occurrences of NFT, wallet, or minting anywhere in the appendix.

This appendix introduces no new component. It is a cross-cutting reference showing how Chapters 4 through 65 and Appendix A already interoperate at runtime.

Merge Revision Note — Version 1.0, Merge Review Pass 28

Added Chapter 66, Improvement Proposal Pipeline, implementing the legitimate engineering pattern from a declined fully-autonomous self-evolution proposal — structured proposals, sandboxed simulation, staged deployment, monitoring feedback — rebuilt with a mandatory human review gate the original lacked.

- §66.5, the Human Review Gate, has no automated approval path at any Fitness Score or Risk Profile. A named reviewer's approval or rejection is logged to the Provenance Ledger with their identity and reasoning — the same discipline Appendix H (§H.6) already established as this Bible's shared enforcement pattern.
- §66.8, Permanent Exclusions, is the section this chapter exists to make possible: the Safety Substrate, the Prime Directive, this pipeline's own review gate, and the Presenter Mode and private-note guarantees from earlier chapters can never be a valid Proposal Scope, checked

automatically at pre-screening (§66.4) before any proposal reaches a human reviewer. This closes the specific gap that made the original submission's “Substrate & Safety Domain (SERE)” unacceptable — a system that could propose changes to its own safety constraints.

Verified clean before insertion: zero occurrences of UFCS, FQL, blockchain, NFT, wallet, or the declined proposal's own terminology anywhere in this chapter — it did not need to reference what it was built in response to.

Merge Revision Note — Version 1.0, Merge Review Pass 29

Ran a full 42-item audit of an external “comprehensive feature inventory” against this Bible's actual current state (not assumed coverage) and implemented the confirmed gaps, per explicit instruction to build the entitlement item without territory, plus a separately-provided offline/sync specification.

- Audit result: 2 items excluded for cause (UFCS, FQL, unchanged), 5 items unverifiable (references to an external artifact ID and patent figures/algorithms this Bible has no access to), 30 items already covered and mapped to specific existing chapters, and 4 confirmed genuine gaps.
- Added Chapter 67, License and Entitlement Management, covering License Types (mapped directly to Chapter 65's four Roles), Seat Management, Entitlement Enforcement, and aggregate Usage Analytics — explicitly excluding territory, pricing, and rights-tokenization, the two categories of content already declined earlier in this Bible's history for cause.
- Added Chapter 68, Synchronization and Offline Operation Layer, from a specification provided directly: four sync modes (Real-Time, Deferred, Batch, Air-Gapped), four consistency mechanisms (version vectors, CRDT merges, deterministic merge rules, provenance-verified checkpoints) built on the existing Provenance Ledger VersionChain rather than a new versioning system, and six offline capabilities (querying, provenance verification, RBAC, template execution, media rendering, edition comparison) scoped to what a device has already synced.
- Added three Interactive Mode Registry entries (§52.13.2.2): Flashcard Generation, Fact Graph Visualization (an explicit graph rendering layered on the existing Semantic Exploration mode's data, not a duplicate mode), and Inline Fact Previews.

Verified before insertion: the only NFT and DRM occurrences in Chapter 67 are its own scope note explicitly excluding them. Zero occurrences of UFCS, FQL, Territory Registry, or blockchain anywhere in either new chapter.

Merge Revision Note — Version 1.0, Merge Review Pass 30

Added Chapter 69, Processor Slice Measurement Model, extracting a genuinely useful idea from a three-document external submission — a bounded, deterministic, four-phase compute measurement unit replacing opaque LLM token counting — while declining the naming and authority the source material bundled with it.

- Slice Categories (§69.2) are named for and measure real chapters — Ingestion (Chapter 4), Reasoning (Chapters 7, 15), Response Generation (§52.8), Rendering (Chapters 44, 46), Safety Gate (Chapter 8), Sync (Chapter 68) — replacing the source material's fictional “FQL Engine” and “UFCS fact graph” subsystem names.
- Slice logging (§69.3) writes to the existing Provenance Ledger (Chapter 5); no new telemetry store was introduced, unlike the source material's separate “Telemetry Slice” system.
- §69.5 makes the actual connection the source material called “billing” explicit: slice logs are the granular data behind Chapter 67's Usage Analytics (§67.4) aggregate counts. This chapter still specifies no pricing model, consistent with §65.5 and §67's existing scope notes.
- §69.6 connects slice telemetry to Candidate Identification (§66.1) as one more input source, explicitly stating that a slice-derived Candidate Record still passes through the full Chapter 66 pipeline — sandbox, pre-screening, and a named human's approval at the Human Review Gate (§66.5) — with no shortcut. The source material's “Deployment Slice” (canary rollout, auto-rollback, no human step) was not carried forward.

A TOC-placement ambiguity (two identical-looking anchor matches, the same risk that caused the Chapter 47 mistake earlier in this project) was caught before insertion this time — both candidate positions were checked directly, and the entry was placed at the verified TOC location, not assumed.

Merge Revision Note — Version 1.0, Merge Review Pass 31

Extended Chapter 69 with §69.2.1 and §69.2.2, an event-level addendum built from an external Event → Slice mapping submission, per the analysis presented before this pass — seven of that submission's eight sections were redundant restatements of §69.2's existing categories at finer grain, folded into one 8-row event table rather than reproduced as a separate structure.

- §69.2.1 names two new Slice Categories — Scheduler Slice and Recovery Slice — which §69.2 had already left room for but never instantiated; Recovery Slice ties directly to the existing Rollback Plan (§66.2) and offline conflict resolution (§68.2) rather than introducing new recovery semantics.

- §69.2.2 covers the four legitimate Improvement Proposal Pipeline events from the source material (anomaly detection, proposal construction, sandbox simulation, fitness scoring) by mapping each to an existing Slice Category rather than inventing “Proposal Slice,” “Simulation Slice,” and “Fitness Slice” as the source material did — each is an instance of Reasoning Slice work happening inside a Chapter 66 stage, not a new kind of work.
- One event was deliberately excluded and the exclusion stated explicitly rather than left implicit: the source material's “Deployment Rollout → Deployment Slice” reintroduced, as an event trigger, the same autonomous-deployment gap §69.6 and §66.8 already close from the slice-type direction. A callout in §69.2.1 states this was excluded and why, rather than silently omitting the event with no record of the decision.

Merge Revision Note — Version 1.0, Merge Review Pass 32

Audited an external “8 (then 9) Ingestion Sources” list against this Bible's actual current state before adding anything. Five items (Documents, Images, Audio, Video, Sensors/Streams) were already fully covered by Chapter 4's modality list and Chapter 49's UIA source types — nothing added for these. Three items (Web Pages, APIs, Databases) were confirmed as genuine gaps at the ingestion-mechanism level, not the modality level: Chapter 4 already covers the data formats these produce, but no example connector explicitly covered live web fetching, live database connections, or general API feeds. One item (AI-Generated Content) was confirmed as a genuine gap in provenance tagging.

- Added §49.3.3.5, three concrete Source Connector examples — Web Page, Database, and API Feed — all following the existing §49.3.3.1 contract exactly (capability declaration, authentication, data mapping, health reporting) and subject to the existing Trust Boundary (§49.3.3.4). No new registration process or governance exception was introduced for any of the three; a live database connector is explicitly distinguished from importing a static SQL dump, already covered as a Structured Data format in §4.2.
- Added §51.4.1, the AI-Origin Flag — a fourth, orthogonal Fact Unit property alongside the three §52.10 already reads, recording whether a Fact Unit's source content was itself AI-generated at its point of origin, independent of which of the six UIA source types delivered it. Explicitly does not affect ConfidenceScore — AI-generated source content is labeled, not presumptively downgraded, the same evenhanded treatment already applied to Social Media's existing Source Advisory.

Merge Revision Note — Version 1.0, Merge Review Pass 33

Analyzed a “Slice Mapping Specification” submission before implementing anything. Most of it (Sections I–III) was a redundant restatement of §69.2/§69.2.1/§69.2.2 at finer grain, reintroducing the fragmented one-category-per-event naming (Parse Slice, Proposal Slice, Simulation Slice, Fitness Slice) already consolidated in Pass 31, plus FQL naming; none of that was implemented. “Deployment Rollout → Deployment Slice” reappeared unchanged from the exclusion already recorded in §69.2.1; still excluded, same reason.

- Section IV's formal mapping function was genuinely new, and contained a real problem, not a naming one: an “AgentOverrides” parameter with the stated rule that “Agent override slices supersede scheduler defaults” — a live runtime channel for an agent to unilaterally change system execution, independent of what the mapping table says. Not implemented, for the same reason §66.5's Human Review Gate exists.
- What was valid and implemented: the underlying idea of a formal, deterministic assignment process (event in, category out, via a defined function rather than an implied lookup) was a real gap. Added §69.2.3, Slice Assignment Process — Event Classification, Profile Resolution, Category Assignment — using only the eight Slice Categories already named in §69.2/§69.2.1, with no new categories and no agent-override input of any kind.
- Added §69.2.4, Priority Ordering — Safety Gate Slices first, Recovery Slices next, all other categories unordered relative to each other — an operationalization of the Safety Substrate's existing “embedded at every layer” principle (§8.1.1) applied to scheduling contention, not a new safety guarantee.

A callout in §69.2.3/§69.2.4 states explicitly that no agent-override channel exists in this process, verified before insertion: zero occurrences of “AgentOverrides,” “Deployment Slice,” or FQL naming anywhere in the new content.

Merge Revision Note — Version 1.0, Merge Review Pass 34

Ran a full completeness audit of NLPL (§48.4) against the rest of this Bible, per direct request, before changing anything.

- Confirmed already complete and not touched: the five-stage compilation pipeline itself, the ambiguity-halts-rather-than-guesses behavior, the VCG safety gate requiring human confirmation for any ASSERT/RETRACT program, and — checked specifically because it seemed like a plausible gap — WATCH program versioning and access control, which turned out to already be fully covered

(§49.5.1: PROGRAM Fact Units, “versioned and access-controlled like everything else”). Also confirmed: WATCH programs are re-checked against the Prime Directive on every trigger firing, not only at registration.

- Closed a real gap: Stage 3's ambiguity halt (§48.4.1) previously specified an unformatted “return to the requester” with no connection to Chapter 37's already-specified Clarification Generation Engine, which handles exactly this situation for ordinary queries. Now routes through the CGE explicitly.
- Closed a second real gap: NLPL compilation had no assigned Slice Category under Chapter 69's measurement model, making it the one operation in this Bible that wasn't actually measurable. Assigned to Response Generation Slice, the same category the ARE Pipeline uses.

Flagged, not fixed, as out of scope for this chapter: “Template Debug Mode” is named in the Interactive Mode Registry (§52.13.2) as an operator tool but was never specified anywhere — the same named-but-empty pattern QBCL-CMS was before Chapter 47 closed it. Relevant to NLPL/QBPL debugging but recommended as its own future pass rather than absorbed silently into this one.

Merge Revision Note — Version 1.0, Merge Review Pass 35

Declined a general-purpose, any-programming-language code generation submission for NLPL, then implemented the actual, safe version of what was being asked for — “say a natural-language instruction and get a working application, and modify it later.” Rather than build a new subsystem, found that this was already almost entirely specified: §39.7's lattice growth mechanism (2), “user-requested custom apps,” already has the QAG building an app a user asks for that doesn't yet exist in the library.

- Added §39.11, NLPL-Driven On-Demand App Composition, connecting NLPL's Intent Decomposition stage (§48.4.1) to QAG as a routing target, and closing two real gaps found in how §39.7's existing mechanism was specified: whether the requesting user gets to use their own app immediately (yes — the QueryBook team's review only governs promotion to the shared library, not personal use) and how a user modifies an app after creation (a further NLPL instruction, versioned through the same Fact Unit VersionChain already used for WATCH programs and Personal Notes).
- Explicitly not implemented: arbitrary programming-language code generation. §39.11.2 states directly that QAG composes from the same fixed six-feature standard anatomy every QAL app already has (§39.4), applied to a new domain schema — not machine-written source code in Python, Rust, or any other general-purpose language. The Narrow Scope Rule (§39.2) applies to on-demand apps exactly as it does to library apps.

- Renumbered §39.11 (Patent Significance) to §39.12 to make room, after verifying its only citation was its own heading (learned discipline from the Chapter 47 renumbering mistake, applied without incident here) — except one: a self-reference written into the new §39.11.2 content itself incorrectly pointed to “§39.11's own Patent Significance claim” instead of §39.12. Caught and fixed before this document was rebuilt, not after.

Merge Revision Note — Version 1.0, Merge Review Pass 36

Closed a small asymmetry found while answering a direct question about query scope: §64.8 has always stated explicitly that Message Board content is retrievable through the Associative Recall Engine (§7.2.3); Chapter 62 never made the equivalent statement for Author-Hosted Events, despite events already being Fact Units (§62.1) and therefore already retrievable in practice. §62.1 now states this explicitly, matching §64.8's pattern — a reader asking about an event is served by the same single retrieval mechanism as any other query, not a separate events search. No new retrieval mechanism was introduced; this was a documentation gap, not a functional one.

Merge Revision Note — Version 1.0, Merge Review Pass 37

Added Chapter 70, Precision Color and Geometric Registration, from a real, technically sound submission covering device-independent color matching and constraint-based geometric alignment for authored diagrams, figures, and VR objects. Rebuilt entirely on Fact Units per explicit instruction — no UFCS, and no FQL query syntax, since FQL does not exist in this Bible; querying routes through the real QBQL (Chapter 18) and the Associative Recall Engine (§7.2.3) instead.

- Verified this was genuinely new before building: distinct from VCUM's Semantic Colorization Engine (§44.3.1), which synthesizes plausible color for content with no reliable color information, versus this chapter's problem of reproducing an author's already-specified exact color within tolerance across output modes. Also distinct from WASM's Object Layer (§53.3), which models conceptual spatial relationships, versus this chapter's production-precision geometric coordinates and tolerances. Neither existing system was modified; §70.3's calibration explicitly defers to VCUM's SCE rather than duplicating synthesis logic where actual color invention is needed.
- The Registration Constraint Graph (§70.4) — a three-pass solver (absolute resolution, relative resolution, drift detection) — is the most genuinely novel piece and was kept close to the source specification's design, restated over Geometry and Constraint Fact Units rather than a standalone UFCS-based graph system.

- VR registration (§70.5) anchors to the Tour Waypoints already specified in §46.9 rather than introducing a separate VR anchor concept, and reuses the same Constraint Fact Unit type and Tolerance field from §70.2 rather than VR-specific parallel fields.
- ΔE validation and registration drift detection are both specified as never failing silently — either produces a queryable ColorDrift or RegistrationDrift Fact Unit the moment tolerance is exceeded, and §70.7 ties this into Chapter 60's print production pipeline as a pre-flight check before Batch Printing proceeds.

A TOC-placement check was performed before insertion, consistent with the discipline established after the Chapter 47 mistake: two identical-looking anchor matches were found, their byte positions compared against this session's established TOC range, and the entry was placed only after that position was explicitly verified against the real Chapter 69 TOC text.

Merge Revision Note — Version 1.0, Merge Review Pass 38

Declined a ~30-fact-type photorealistic avatar reconstruction and animation stack (pore-level facial geometry, individual hair-strand physics, FACS muscle simulation, physically-simulated cloth, distributed cluster execution, and a governance/quota/arbitration layer sitting above it), built on UFCS and FQL. Consent was proposed as the mitigation; declined on the grounds that reconstruction fidelity at that level is not a problem consent resolves — the specification is the same general-purpose capability regardless of which authorized likeness a given instance points to.

Implemented instead the bounded, safe version of the underlying goal — better presentation quality for the presenter capability that already exists and is already fully consented under §46.10.

- Added §46.10.8, Color and Registration Stability, connecting Face-on-Avatar mode (§46.10.5) to Chapter 70's already-existing Color and Geometry Fact Units: ΔE-validated color consistency for the author's authorized likeness across a Tour's Waypoints, and Registration Constraint Graph-governed positioning so the avatar doesn't drift as it moves through a scene. Both reuse infrastructure built for an unrelated purpose (print and diagram registration) rather than introducing avatar-specific systems.
- A callout in §46.10.8 states explicitly what was not added: no pore-level detail, no hair-strand simulation, no muscle or FACS simulation, no physics-based cloth, no rigging beyond what Face-on-Avatar mode already does. This is an accuracy-and-stability improvement to existing, disclosed, consented compositing — not new reconstruction fidelity.

Verified before insertion: every declined-technology term (hair-strand, muscle, pore-level, rigging, FACS) appears exactly once in the new content — inside the exclusion statement itself, not as newly introduced architecture.

Merge Revision Note — Version 1.0, Merge Review Pass 39

Evaluated a “Geometric Reasoning Upgrade” submission built on the declined manifold-layer concept (SML/GML/TML/RML) and largely duplicating Chapter 70's already-existing Geometry and Constraint Fact Units under new names (GeometricObject, SceneTopology, SpatialRelation). Neither was carried forward. Found one genuinely new, safely-boundable capability inside the submission and implemented it on its own terms.

- Added §70.9, Geometric Proof Engine — formal, deductive reasoning (angle rules, triangle rules, circle theorems, parallel-line rules) over Geometry and Constraint Fact Units already specified in §70.1–§70.2, not a parallel geometric ontology. The rule set is fixed and enumerable, the same bounded-primitive discipline already applied to QBPL (§48.5).
- §70.9.3 carries forward the safety principle already established throughout this Bible: a proof step can derive a new relationship between existing Geometry Fact Units, but cannot introduce a new geometric entity — consistent with the Without-Information Rule (D0.4).
- Explicitly declined to carry forward: any dependency on a Semantic/Geometric/Temporal/Robotics Manifold Layer (declined in this Bible's history and not reintroduced under a different name here), and a claimed “400%–19,900%” compression improvement figure with no stated methodology — §70.9.4 states directly that this number was not carried forward and that any real compression benefit should be measured and given the same design-target caveat already used for other unvalidated figures (Appendix C, §C.11).

Renumbered §70.9 (Summary) to §70.10 to make room, after verifying its only citation was its own heading — continuing the discipline established after the Chapter 47 renumbering mistake.

Merge Revision Note — Version 1.0, Merge Review Pass 40

Implemented a submission directly addressing the seven gaps named in this Bible's own end-of-Pass-39 self-assessment — unlike most submissions this session, this one responded to gaps this Bible itself identified rather than proposing new architecture. Verified rather than assumed clean: two references to the declined manifold-layer concept (“manifold alignment errors” in the debug-mode proposal,

“manifold benchmarks” in the validation proposal) were found and not carried forward; everything else in the submission was substantively sound.

- Added Chapter 71, Universal Accessibility Layer — accessible equivalents (alt-text, scene descriptions, captions, tactile export) required as part of content production, generated from existing Fact Units at the same production stage, not a separate accessibility pass. Reuses the Narrative Graph, ARE Pipeline, and Chapter 70's Geometry Fact Units rather than a parallel content system.
- Added Chapter 72, Data Portability and Exit — a comprehensive export guarantee across notes, board activity, ingestion history, and licensed content, in open formats (JSON, EPUB, CSV, glTF/USD), with provenance preserved. Explicitly scoped within Chapter 60's existing rights model, not a bypass of it.
- Added Chapter 73, Template Debug Mode — closing the gap flagged twice previously (Pass 34, Pass 39). Observes real execution of QBPL, the ARE Pipeline, and IQR composition with additional visibility, deliberately not a simulated environment that could diverge from real behavior. The submission's “manifold alignment errors” inspection view was dropped along with the concept it depended on.
- Added Chapter 74, Multi-Author Collaboration — optimistic concurrency with explicit Fact-level locking, and conflict resolution reusing each content type's own existing detection mechanism (VersionChain divergence for text, Registration drift detection for geometry, branch preservation for Scenario Graphs) rather than a new diff engine per type.
- Added Chapter 75, Moderation Appeals — a formal appeals workflow with mandatory human review, explicitly scoped to reviewing how a Safety Substrate rule was applied to a specific case, not whether the rule should exist — that question remains Chapter 66's Improvement Proposal Pipeline, with its own separate human gate.
- Added Chapter 76, Platform Internationalization — localizes platform UI and mandatory disclosures, explicitly not a book's own authored content (a publisher decision, Chapter 60) and explicitly one safety standard expressed correctly in every language, not different rules per locale — stated directly since the distinction is easy to misread.
- Added Chapter 77, Validation and Benchmarking — a formal process for validating or retracting the many design-target figures already scattered through this Bible (Appendix C, Chapter 4, §7.2.3, Chapter 10, Chapter 16, Chapter 70), reusing Chapter 69's existing Slice measurement rather than a separate benchmarking mechanism. Introduces no new performance claims of its own.

A table-formatting bug (a row missing its third column, in §77.1's benchmark table) was caught during this pass's own bug-check pipeline and fixed before insertion, not after.

Merge Revision Note — Version 1.0, Merge Review Pass 41

Added Chapter 78, QueryBook AI Evolution, in response to a request for agents assigned to every major system, developing improvements in a sandbox, and recommending integration after testing and human admin approval. That request describes the Improvement Proposal Pipeline (Chapter 66) already fully specified in this Bible — this chapter names and organizes it, rather than building a new one.

- §78.1 assigns seven representative System Agents to already-specified subsystems (Ingestion, Reasoning, Response Generation, Rendering, Geometry, Robotics, Accessibility), each scoped to that subsystem's own existing telemetry — Chapter 69's Slice Categories, Chapter 70's drift findings, Chapter 71's accessibility coverage. The list is explicitly representative, not exhaustive, following the same extensibility pattern already established for Slice Categories.
- §78.3 states directly, three separate times in three separate ways, that no System Agent approves its own proposal, can target another agent's scope or Chapter 66's own gate, or can touch anything on §66.8's Permanent Exclusions list. This repetition is deliberate: the chapter's name (“AI Evolution”) could otherwise be read as implying broader autonomy than what's actually being specified.

A Unicode encoding error (an angle-bracket character, ⟨, inserted in place of an apostrophe in the TOC entry's descriptive note) was introduced while writing this pass and caught during placement verification, before the document was rebuilt — not after.

Merge Revision Note — Version 1.0, Merge Review Pass 42

Added §78.6, Human-Directed Proposals, in response to a direct question about whether a person can specify an improvement direction before sandbox development happens, rather than waiting for a System Agent's telemetry to surface it. This was a real, narrow gap: §66.1's Candidate Identification was framed entirely around agents noticing patterns in existing telemetry.

- A License Owner with an entitled Role (Chapter 65, Chapter 67) can now submit a Proposal Fact Unit (§66.2) directly, stating what they want investigated before any telemetry pattern exists. From that point, the proposal enters the exact same pipeline as any System Agent's — Simulation Sandbox (§66.3), automated pre-screening against §66.8's Permanent Exclusions (§66.4), and the Human Review Gate (§66.5). No expedited path, no different threshold.

- §78.6.2 states explicitly that directing a proposal and approving it are two separate steps even when the same admin holds both roles — the review step still has to actually happen, not be assumed satisfied because the same person asked for the investigation.

Renumbered §78.6 (Summary) to §78.7 to make room, after verifying its only citation was its own heading.

Merge Revision Note — Version 1.0, Merge Review Pass 43

Added Chapter 79, Formal Verification Layer, integrating Lean-based formal proof with QBPL (Chapter 48), WATCH programs (§49.4), the Improvement Proposal Pipeline (Chapter 66), and QueryBook AI Evolution (Chapter 78) — additive to every mechanism it touches, per the standing discipline of this Bible.

- §79.1 scopes what is actually formally tractable: termination of REPEAT-UNTIL-BOUND loops, well-formedness and Scope compliance of WATCH program triggers, and non-interference between a proposal's Scope and §66.8's Permanent Exclusions list — all precisely specifiable properties over QBPL's fixed, non-Turing-complete primitive set (§48.5), which is what makes formal verification tractable here specifically.
- §79.2 places verification inside existing gates rather than creating new ones: the Verification and Confirmation Gate (§48.4.1) for compiled programs, WATCH Program Registry admission (§49.5) for triggers, and Automated Pre-Screening (§66.4) for proposals touching QBPL or WATCH behavior.
- §79.5 states directly what does not change: the Human Review Gate (§66.5) still evaluates every proposal, with a proof as additional evidence rather than a substitute for judgment; the Permanent Exclusions list (§66.8) remains unreachable by any proposal regardless of whether a proof could theoretically be constructed for it, because such a proposal never passes pre-screening far enough to reach the verification step; and natural-language generation (Discussion Mode, the Narrative Graph) is explicitly out of scope, governed by the Factual Groundedness metric already specified elsewhere, not by a Lean proof.

Verified before insertion: no existing chapter's specification was modified; verification runs as an added step inside pipelines that already existed.

Merge Revision Note — Version 1.0, Merge Review Pass 44

Made a small, historically-motivated terminology correction to §7.1 and §7.2.4, prompted by a legitimate point raised in an unrelated video comment: John Hopfield's 1982 paper was not the first

description of the associative-memory-network formalism popularly named after him. Materially similar or identical models were proposed earlier by Marr (1971), Nakano (1972), Amari (1972), Little (1974), and Stanley (1976). This is attribution precision, not an architecture change — the Attractor Network's dynamics, equation, and capacity bound are all unchanged.

- §7.1's introductory NCL description and §7.2.4's Attractor Network definition now name the mechanism as an associative memory network, of the class popularly known as Hopfield networks, with the fuller lineage cited once at §7.2.4.
- Left unchanged, deliberately: the 0.14N capacity bound citation at §7.6.3, since that specific figure is Hopfield's own attributed 1982 result, not the broader formalism generally; and the prior-art comparison list citing “Hopfield Networks” by its common industry name, appropriate in a comparison-to-existing-technology context.
- Also evaluated and declined in the same review: an unbounded “Lean Agent Network” proposal whose “AI-Assisted Proof Repair” component described an AI generating candidate fixes and Lean verifying them with no human step anywhere in the loop — the same autonomous-self-modification shape already declined multiple times in this Bible's history, this time with formal verification attached to it rather than a Fitness Score. Chapter 79, added the previous pass, already specifies the sound version: Lean proof as evidence for the existing Human Review Gate (§66.5), never a replacement for it. Also declined: treating an unrelated third party's investor-outreach messaging as relevant to this Bible, particularly where doing so would have required treating UFCS and FQL as QueryBook's proprietary technology — both were permanently removed from this architecture in Pass 6 and confirmed absent repeatedly since.

Merge Revision Note — Version 1.0, Merge Review Pass 45

Added Chapter 80, Human Reasoning and Motivation Modeling, in response to a direct question about whether QueryBook could be used to understand self-motivated, goal-driven human reasoning — distinguished carefully from a separately-evaluated proposal for QueryBook itself to become an autonomous, self-motivated agent, which was declined in the same review that identified this chapter's real, safe alternative.

- §80.1 adds Goal and Motivation Fact Unit types, each requiring a Subject reference — every instance describes a documented person, character, or entity in the Fact Graph, never QueryBook's own state. A StatedOrInferred field keeps a directly-stated goal distinguishable from an inferred one, rather than collapsing both into a single confidence number.

- §80.2 reuses the Scenario Graph's existing PLANNING type (Chapter 11) and WASM's Affordance layer (§53.3) to represent how a subject pursued a goal — grounded in what options were actually documented as available to them, not open-ended hypothetical psychology.
- §80.3 wires this into four already-existing surfaces — Standard Q&A, Discussion Mode, Tutorial Mode and Comprehension Check, and Semantic Exploration — rather than adding a new Interactive Mode; a Goal or Motivation Fact Unit is queryable and connects into the Ontology Mesh the same way any other fact does.
- The chapter's opening callout states the safety distinction directly and names the declined proposal by consequence rather than assuming the distinction is self-evident: this models a subject, never QueryBook itself, and is not a narrower version of the declined autonomous-agent proposal wearing an academic framing.

A literal Unicode escape sequence (\u27e8 typed instead of an apostrophe) was caught in the TOC entry's descriptive note before it was written to the document, not after — the same class of error made once already in Pass 41 and, this time, using explicit character codes instead of literal escape text to avoid a third occurrence.

Merge Revision Note — Version 1.0, Merge Review Pass 46

Received a resubmission of a document already fully analyzed two passes earlier (same 13 categories, same Prime-Directive-first framing, same content word-for-word). Did not re-run the full comparison; focused specifically on the one requested area — ingestion — and the one term from that list not previously resolved: “Open Claw Ingestion.”

- Confirmed a real, narrow gap: §4.2's eight modalities end at Bit 7 (Code) with no stated behavior for content that doesn't cleanly resolve to any of them. Added §4.2.1, Content Outside the Eight Modalities, specifying best-effort classification first, and a non-silent halt — not a silent drop, not a forced fit — where no reasonable classification exists, consistent with the same non-silent-failure discipline already applied to ΔE validation (§70.3), registration drift (§70.4), and NLPL's ambiguity halt (§48.4.1).
- Deliberately did not adopt “Open Claw” as terminology — it isn't a grounded term I could verify the meaning of, and §4.2.1's callout states explicitly that this is a classification fallback for the existing eight modalities, not a ninth, open-ended ingestion path.

Everything else in the resubmitted document — the AGI-convergence category (self-motivated goals, curiosity, memory reset) — remains addressed as already established: the safe, content-modeling

version is Chapter 80; an autonomous, self-motivated QueryBook itself remains declined, unchanged from the prior review.

Merge Revision Note — Version 1.0, Merge Review Pass 47

Added §49.3.3.6 and §49.3.3.7, an Autonomous Agent Connector, after a request to give QueryBook “OpenClaw type control” — native login, browser automation, and general device manipulation — was verified (OpenClaw is a real, fast-growing, open-source autonomous agent platform) and declined for the same reason a general-purpose account/device-control capability has been declined throughout this Bible's history: unscoped, human-mimicking authority with no declared action boundary.

- The connector specified instead directs specific, QBPL-compiled actions (§48.3) through a registered external agent, following the existing §49.3.3.1 contract exactly — capability declaration, authentication, data mapping — rather than QueryBook adopting the connected platform's own broad authority. §49.3.3.6 states this distinction as the connector's defining property: directed delegation, never adopted authority.
- §49.3.3.7 records the specific reasoning for declining native integration, including a concrete, current research finding: an independent study evaluating this exact class of tool's controllability found that safety-oriented guardrails produced only marginal reductions in unsafe-action rates, with several risk categories remaining high even with mitigation applied — cited here as a documented, present-day limitation, not a hypothetical one.

Every action directed through this connector is logged to the Provenance Ledger (Chapter 5) with the specific request and result, the same audit discipline already required of every other consequential action in this Bible.

Merge Revision Note — Version 1.0, Merge Review Pass 48

Received an actual, professional patent search report for UFCS — real prior-art citations, a patent attorney's letterhead, standard USPTO patentability analysis — the first genuinely verifiable technical source for this technology in this Bible's history, after 47 passes in which every prior submission mentioning UFCS/FQL was either an unverified feature list or was declined outright. Added Chapter 81, UFCS/FQL as an Optional External Resource, built directly from this document's actual technical claims rather than from a feature list.

- §81.1 describes UFCS/FQL accurately from the source report: a hypergraph-substrate retrieval system with the Fact Atom as its retrieval unit, supporting hyperedge pattern matching, predictive

fact retrieval, contradiction-cluster retrieval, domain-calibrated ranking, and hyperchain-path explainability.

- §81.2 states directly, for every capability that overlaps conceptually with something QueryBook already does, which existing chapter already covers it and how the mechanisms differ — predictive retrieval against the Predictive Inference Engine (Chapter 10) and AEM (Chapter 15), explainability against the Provenance Ledger (Chapter 5), multi-entity relationships against the Ontology Mesh (Chapter 6) — so connecting UFCS is understood as adding a second, alternative resource, not completing something QueryBook otherwise lacks.
- §81.3 connects UFCS through the existing Source Connector Registry (§49.3.3), following the identical contract already used for the Database, API Feed, and Autonomous Agent Connector examples — no new registration model, Fact Unit schema, or retrieval mechanism was introduced into QueryBook's core.
- §81.4 states explicitly that this chapter offers no patent-strategy or claim-drafting advice for either QueryBook's or UFCS/FQL's own patent applications — that judgment belongs to qualified patent counsel, not this specification.

Verified after insertion: every one of the 29 UFCS/FQL references elsewhere in this document remains historical changelog text documenting the Pass 6 removal and subsequent declines — none of Chapters 1 through 80 were modified to reference or depend on Chapter 81.

Merge Revision Note — Version 1.0, Merge Review Pass 49

Closed a genuine gap flagged in conversation, not in a submitted document: whether the design makes clear that primary initial ingestion is an author uploading an already-complete ebook at the Author Portal. It did not — §61.1 connected the Author Portal to the Ingestion Pipeline only through the Manuscript Editor, an incremental in-app drafting surface, leaving the far more common case of a finished-file upload undocumented as a distinct path.

- Added §61.1.1, Uploading an Existing, Complete Manuscript, stating directly that this is the primary initial ingestion pathway for the repository, not a secondary case — most authors arrive with a finished file, not a blank page.
- The new section introduces no new mechanism: same Multimodal Ingestion Pipeline (§4.2), same DraftStatus: DRAFT tagging, same classification fallback (§4.2.1) for content that doesn't cleanly resolve, same Rights Validation Gate (§60.2) before repository entry. Its callout states this explicitly — same pathway, different cadence, one ingestion event instead of many.

- Recommends applying Narrative Coherence Checking (§61.2) to a freshly-uploaded manuscript specifically to verify accurate extraction before submission — a large single-event ingestion has more surface area for a misclassified chapter than an incremental edit does, worth naming as a practical use of an already-existing tool rather than leaving it unstated.

Merge Revision Note — Version 1.0, Merge Review Pass 50

Added Chapter 82, Supplemental Content and Private Messaging, closing two gaps surfaced while grounding a description of the portal experience: neither capability existed before this pass, though both were described as if they already did.

- §82.1 lets an author attach supplemental material — behind-the-scenes commentary, production notes, added context — to a published book through the identical pathway already specified for manuscript upload (§61.1.1), distinguished only by a new ContentRole field (CORE vs. SUPPLEMENTAL). Explicitly not a second book: no separate Submission Intake cycle, no separate Rights Validation Gate pass.
- §82.2 adds a DirectMessage Fact Unit type for private, one-to-one communication — the first messaging capability in this Bible that isn't board-scoped (Chapter 64). Subject to identical Safety Substrate enforcement, a block mechanism enforced at the AccessControlList level, and the existing Moderation Appeals process (Chapter 75) for disputes.
- Declined in the same review, pending a deliberate scoping decision rather than built unilaterally: general merchandise sales beyond the book itself. Chapter 60 is specifically print production and distribution of the work; a broader commerce marketplace involves payment processing and physical fulfillment considerations this Bible has not yet scoped, and folding it in without that discussion would repeat the exact mistake this Bible's history has repeatedly corrected — describing a capability into existence rather than deciding to build it.

Merge Revision Note — Version 1.0, Merge Review Pass 51

Added Chapter 83, Instructor-Created Tours and Teaching Material, after verifying a real gap before writing a requested book section on author and educator portal use: Tours (§46.9) are explicitly author-curated, and Supplemental Content (§82.1) is explicitly author-scoped, leaving an instructor teaching a book they didn't write with no way to build their own teaching material from it.

- §83.1 lets an instructor — the contextual capability already defined at §65.3, not a new role — build an Instructor Tour (resequencing existing IQR scenes) and Instructor Notes (class-scoped

annotations), both requiring no permission from the book's author because neither touches the author's own content or rights.

- §83.2 requires non-removable attribution distinguishing instructor-created material from the author's own curation, using the identical disclosure discipline already established for Presenter Mode (§46.10.6) — a student always knows whose curation they're experiencing.
- §83.3 keeps Presenter Mode strictly author-exclusive: an instructor narrating their own Tour must hold their own separate Presenter authorization and cannot invoke the book author's voice or likeness merely by virtue of teaching that author's work.

Merge Revision Note — Version 1.0, Merge Review Pass 52

Responded to a broad “fill in all gaps” request by scoping it explicitly to the Portal Experience domain actually under discussion — Author, Reader, Publisher, and Instructor — rather than an unbounded audit of the full Bible, consistent with this project's standing discipline against inventing architecture nobody asked for. Confirmed §60.6 Market Analytics is already fully specified, not a gap. Found one genuine, well-evidenced gap and closed it.

- §74.3's Multi-Author Collaboration conflict-resolution table was built in Pass 40, before Tours (§46.9) or Instructor Tours (§83, added this session) existed as author-curated objects — leaving no specified conflict handling for two co-authors, or two co-instructors, concurrently editing the same Tour's Waypoint sequence. Added a fourth table row extending the identical VersionChain-divergence-and-preservation mechanism already used for narrative text, rather than inventing new conflict logic.
- The new row states explicitly that Presenter narration authorization is never merged or transferred between conflicting Tour versions — each preserved version retains only the narration its own author or instructor actually authorized, consistent with §46.10.1 and §83.3's existing rule that authorization is never inherited.

No other gap in this domain was identified with comparable evidence during this pass; nothing else was added speculatively to appear more thorough.

Merge Revision Note — Version 1.0, Merge Review Pass 53

Received a large patent-style submission built around blockchain- and NFT-anchored provenance as its central mechanism (“NFT Certification Layer,” “NFT Registry,” “Deferred Minting Mode,” multi-chain anchoring across Ethereum, Polygon, and Solana). None of it was adopted — this is the same rights-

tokenization concept declined at this Bible's earliest passes and every time it has resurfaced since, under whatever name. Chapter 5's Merkle DAG Provenance Ledger already provides tamper-evident, cryptographically verifiable provenance with no blockchain dependency, continuous or deferred.

- Two genuinely novel, narrow, non-blockchain-dependent ideas were extracted and added as Chapter 84. §84.1, Session Books, gives a reader a temporary, purgeable, task-scoped subset of the Fact Graph for short-term research — built on the existing reference-based pattern already used for Scenario Graph Knowledge State Snapshots (Chapter 11), not a copy of the underlying Fact Units.
- §84.2, Instructor-Customized Assessment Templates, lets an instructor (§65.3) define a reusable Comprehension Check format — difficulty, question count, structure — scoped to their own class. A template constrains structure, never content; every generated question remains grounded in the book's own Fact Graph exactly as Comprehension Check (§52.13.2) already requires.

The chapter's opening callout explains directly why several ideas in the source submission weren't gaps at all — they solve a connectivity problem for a blockchain mechanism this Bible has never had, when §68's Offline Operation Layer already handles offline provenance verification without blockchain of any kind.

Merge Revision Note — Version 1.0, Merge Review Pass 54

Directly addressed a follow-up argument that ebooks now and in the future will require NFTs, rather than repeating the prior pass's decline unchanged. Verified the premise by search rather than asserting from either side: NFT-based ebook distribution is real and has genuine, if minority, adoption (one platform alone has sold over 130,000), but the most current source found states plainly that the industry conversation has moved from “every book belongs on a blockchain” to a narrower question — confirming NFTs are one optional distribution model, not an emerging requirement.

- Separated three needs that had been bundled together as if they required one solution: ownership verification (already solved, blockchain-free, by Chapter 67's License and Entitlement Management), media packaging (already solved by Chapter 4's ingestion pipeline and §82.1's Supplemental Content), and resale/transfer of a digital copy — the one genuine capability conventional licensing doesn't provide, and the only one of the three actually addressed by this pass.
- Added §49.3.3.8, a Blockchain Ownership-Verification Connector, following the identical external-connector pattern already established for UFCS/FQL (Chapter 81) and the Autonomous Agent Connector (§49.3.3.6) — a publisher may register an external NFT-verification service for a specific

title's resale capability, with Chapter 67 remaining the default, blockchain-free entitlement system for every other title.

The new section's callout states explicitly that this is not a reversal of the standing exclusion on blockchain as QueryBook's own rights-tracking mechanism, declined at this Bible's earliest passes and reaffirmed the previous pass: QueryBook remains the requesting party asking an external service to verify a transfer, never the operator of blockchain infrastructure itself, and registering the connector is a per-title publisher choice, never a platform default.

Merge Revision Note — Version 1.0, Merge Review Pass 55

Reconsidered directly whether Instructor should become a fifth top-level Role, prompted by a request to resolve “the lack of instructor role” while minimizing complexity. Concluded the opposite of the premise: converting to a formal Role would mean retrofitting four already-built chapters (Chapter 62, §64.1, Chapter 83, §84.2) that all depend on the contextual-capability model working exactly as §65.3 already specifies, and the new Role would still need the identical per-class scoping logic bolted back on. §65.3 was reaffirmed, not changed.

Identified the real underlying gap instead: no way to see every class a person currently holds instructor privileges for in one place. Added Chapter 85, Instructor Grant Visibility, to close it — a read-only dashboard, not a new Role.

- §85.1 aggregates existing per-class grant records into a single view — class name, Class Board activity, Instructor Tours, and assessment templates for each class the viewer currently teaches — querying data that already exists through Chapters 62, 64, 83, and 84 rather than duplicating or restructuring it.
- §85.2 states directly what this chapter refuses to add: no Instructor Role, no new permission, no cross-class merging of what remain genuinely separate, per-class-scoped objects. The chapter's own callout notes it would have nothing left to display if every underlying grant it reads from were deleted — it has no independent existence apart from §65.3's grants.

Merge Revision Note — Version 1.0, Merge Review Pass 56

Added Chapter 86, System Identity and Positioning, after an external summary of QueryBook was submitted for adoption “wherever QueryBook is summarized” and, on review, contained multiple overclaims and one significant reintroduction of previously-declined territory. Declined wholesale adoption; built a corrected, canonical consolidation instead.

- §86.2 restates five real differentiators — structural groundedness, the Dual-Brain Architecture, human-gated self-improvement, bounded natural-language programming, and directed delegation to external systems — each cited to its actual chapter, correcting imprecise or invented framings along the way (“meaning-level vs. world-model reasoning” for the Dual-Brain Architecture is not accurate; the real architecture is the Semantic Brain / World Brain Predictive Coding hierarchy of §41.11.2 and Chapter 58).
- §86.3 corrects four specific mischaracterizations by name: “no hallucinations” as an absolute claim (the real claim is the scoped, structural groundedness property); “encrypted Fact Units” (not a real schema property — only opt-in reader history encryption, §36.8, is real); “teacher-in-a-box” (the real Instructor capability is additive to human instruction, never a replacement); and “control robots,” “coordinate multi-robot fleets,” and “scan multi-camera feeds for hazards” — general, unscoped device and surveillance authority that does not exist in this Bible. The real capability, §49.3.3.6's directed delegation of specific declared actions, is the same distinction already drawn when a comparable proposal was declined earlier in this Bible's history.

§86.5 states this chapter's intended role directly: future summaries of QueryBook, wherever they occur, should draw from this chapter rather than from external descriptions that have not been checked against it.

Merge Revision Note — Version 1.0, Merge Review Pass 57

Investigated a genuine gap surfaced while declining a submission that fabricated a “hypergraph world-view model” by conflating WVM (§42.2.1, real) with the Attractor Network (Chapter 7, real, but a different system) and reintroducing the same broad robotics-control claims already declined and corrected in Chapter 86. Verified §42.2.5, the Semantic Consistency Engine, is a genuine one-sentence stub — named and functionally scoped, but with no specified trigger or mechanism, unlike §42.2.6 immediately following it, which already redirects to existing chapters rather than standing unspecified.

- Added §42.2.5.1, specifying the engine as event-triggered, deterministic validation logic running whenever WVM's Ontology Layer, Causal Graph, or Temporal Model changes — not a continuously-running process, and explicitly not a dynamical system with its own attractors or convergence behavior.
- Each of the engine's four named checks now reuses an already-specified mechanism rather than inventing new logic: non-contradiction reuses the Cross-Domain Rule Engine's existing Conflict Index (Chapter 17); causal coherence, temporal alignment, and ontological validity each validate against WVM's own existing Causal Graph, Temporal Model, and Ontology Layer respectively.

Confirmed before insertion: no reference to “hypergraph,” “world-view model” as a cognitive-dynamics system, or unscoped robotics control appears anywhere in the new content — the fabricated framing from the declined submission was not carried forward in any form.

Merge Revision Note — Version 1.0, Merge Review Pass 58

Added Chapter 87, Live Interaction Voice Signal Analysis, after a scoping conversation about voice/emotion inference across two related submissions — one of which had already misdescribed this as a much larger, elaborately brain-region-mapped system. Verified first that ingested-content prosody analysis already exists (§4.3.4, sentiment and prosody analysis from speech); no new architecture was needed there. The genuine gap was narrower: nothing analyzed a live, present user's own voice during an active session.

- §87.2 requires the analyzed user's own explicit, opt-in, revocable consent before any live voice analysis occurs — disabled by default, disclosed in plain terms before opt-in, and halting immediately on revocation. This is the same authorization discipline already established for Presenter Mode (§46.10.1), adapted for analysis input rather than generation output; the distinction from ingested-content analysis (which requires no additional consent, being already governed by Chapter 60's rights model) is stated explicitly in the chapter's opening callout.
- §87.3 reuses the Clarification Generation Engine's existing trigger mechanism (§37.4) rather than building a parallel decision system — a detected prosodic signal becomes an additional, optional input alongside CGE's existing query-ambiguity and coreference-failure triggers, with no change to CGE's existing behavior otherwise.

Declined in the same review, per explicit instruction: incorporating NFT minting, autonomous self-evolution, unscoped robotics control, or manifold-based world modeling from either submission under corrected terminology. These were identified as substantive, previously-declined capabilities, not naming problems, and a naming-only correction does not reopen them.

Merge Revision Note — Version 1.0, Merge Review Pass 59

Ran a full Bible-wide integrity sweep at the user's request: structural XML validity, chapter and pass numbering uniqueness, cross-reference validity, and terminology contamination (FQL/QBQL boundary, NFT scoping, blanket-encryption and absolute-hallucination-prevention claims). Structural integrity, pass numbering, FQL/QBQL, and NFT scoping all checked clean across the full document. Two real defects found and fixed; one significant error in this session's own recent work identified and corrected.

- A broken citation — “Zero-Knowledge Encryption scheme (§36.7)” — pointed to a chapter that does not exist. Investigation found ZKE was referenced throughout Chapter 36 (§36.2's SAFU schema, §36.7's Right-to-Erasure Protocol, §36.8's History Interface) but never formally defined anywhere. Added §36.1.1, a proper definition consolidating what those three sections already established, and corrected the citation to point to it.
- A genuine naming collision was confirmed: “ARE” refers to two distinct, real, independently-specified systems — the Associative Recall Engine (§41.4.2, pre-activation) and the Adaptive Response Engine (§52.8, response composition). Added §2.2 to the Canonical Terminology Glossary disambiguating both by name, with a standing instruction to cite the full name or section number rather than the bare acronym in any context intended to stand alone.

Most significantly: this sweep surfaced that Chapter 41, “QueryBook Predictive Cognitive Architecture (PCA),” already and legitimately specifies nearly everything this session declined two passes ago as fabricated — TMM, CSW, HEP, PPRE, SCV, AIQG, and the broader Thousand Brains/Predictive Coding framing, organized as “eight existing mechanisms, four additions, and three amplifications,” explicitly stated in the chapter's own text as “a patentable architectural claim in its own right.” This session's characterization of that content as unverified was incorrect. The cautionary note in the most recently-delivered patent document, declining to describe a point-by-point neuroscience correspondence, was accordingly too conservative and will be corrected when the new provisional patent application is built next.

- One item remains genuinely unresolved rather than force-closed: chapter numbers 21–35 and 38 do not exist as titled chapters. Appendices A–H exist and may account for this reserved range, but this could not be fully verified from the current file alone and is flagged for confirmation rather than assumed.

Merge Revision Note — Version 1.0, Merge Review Pass 60

Investigated a request to elevate several features (Prime Directive, WVM, Fact Unit examples, multi-language, encryption/access control, directed-delegation connectors) to “core, primary, prioritized” status. Checked each against its actual current position before changing anything, rather than assuming the premise. Most were already appropriately core: the Prime Directive is already Domain 0 with universal, no-exemption scope; multi-language (Chapter 16), encryption (§36.1.1), access control (Chapter 65), and directed delegation (§49.3.3.6) are all already in core-numbered chapters, not buried anywhere. Two genuine gaps were found and closed.

- Chapter 3 (Fact Unit Structure), 132,000+ characters specifying the Fact Unit schema in complete precision, contained zero worked examples anywhere. Added §3.2, a worked example applying the schema to a concrete assertion — entity resolution, Fact Unit construction, deterministic retrieval, and version-chained revision — immediately following the schema table it illustrates.
- WVM (World-View Model, §42.2.1) is real, native, foundational architecture, but was mentioned nowhere outside Chapter 42's technology inventory — not in Chapter 1's own introduction. Added §1.4, naming the Fact Unit, Ontology Mesh, and WVM together as the three foundational systems a first-time reader should know about, with a pointer to §44's Video Colorization and Upscaling Module as a concrete example of a later chapter already building on WVM directly.

No other structural changes were made. Elevating already-correctly-positioned content would have been change for its own sake, not a genuine correction.

Merge Revision Note — Version 1.0, Merge Review Pass 61

Added Chapter 88, Contextual Commerce Layer, after a scoping conversation about including ecommerce and advertising, based on query subject matter, in the design. Confirmed neither existed anywhere in the Bible before building — general merchandise sales had been explicitly declined pending a scoping decision that was never made, and query-linked advertising had never been evaluated. Built to preserve, rather than weaken, the structural-groundedness guarantee (Chapter 16) and the Prime Directive (Domain 0) that commercial content could otherwise put pressure on.

- §88's opening callout states the core design discipline directly: the grounded answer's retrieval, ranking, and ConfidenceScore computation complete and finalize before the commerce layer is invoked at all, which receives only the query's resolved concept_id as read-only input with no write path back into the answer pipeline. This is enforced by pipeline ordering, not by a policy instruction an operator could misconfigure.
- Per explicit instruction, both real commerce models are supported at the operator's per-deployment choice: author/publisher-linked content, and third-party affiliate/ad network content connected through the same capability-declared connector architecture (§49.3.3) used for any other external system — with the external network never given access to the query text, the grounded answer, or any Fact Unit.
- §88.4 requires full operator visibility per explicit instruction: every commercial display event is logged with the triggering concept_id, the content shown, its source, and a timestamp — the same

logged-with-identity-and-reason discipline already applied to moderation actions and improvement-proposal decisions.

§88.5's non-interference guarantee is stated as non-configurable, distinct from §88.2's source-type selection — an operator may choose which commerce sources are active, but may not disable the barrier between commercial signal and the grounded answer under any configuration.

Merge Revision Note — Version 1.0, Merge Review Pass 62

Added §88.8 and §88.9 to Chapter 88 after explicit direction that ecommerce/advertising is a real monetization requirement, and that processing needs metering weights. Renumbered the chapter's existing §88.6 (Integration Requirements) and §88.7 (Summary) to §88.10 and §88.11 to keep Summary last, consistent with every other chapter's convention — only two headings needed renumbering, both single-occurrence, verified before editing.

- §88.8 defines a Commerce Slice under Chapter 69's own extension pattern — §69.2 already states a chapter may name its own Slice Category without modifying Chapter 69 itself, and §69.2.1 already demonstrates this precedent (Scheduler Slice, Recovery Slice). The Commerce Slice's priority is set strictly below every safety-relevant category, non-configurably, and its failure mode is specified as silent omission of the commerce region — a slow or failed third-party network call never delays or degrades the grounded answer.
- §88.9 adds the selection mechanism a real bid/auction-style monetization model actually requires: where multiple candidates match a resolved concept, a weighted combination of relevance, an operator-configurable source-priority weight, and a third-party bid or revenue-share value determines which candidate is shown. The weights themselves are operator-configurable, since monetization strategy legitimately varies by deployment — what is not configurable is that this weighting operates only on candidates already gathered after the grounded answer completed, with no access to and no effect on Fact Unit retrieval, ranking, or ConfidenceScore. §88.5's non-interference guarantee governs §88.9 exactly as it governs the rest of the chapter.

Selection outcomes and their contributing weighted factors are added to the same operator-queryable audit log already required at §88.4, so monetization tuning and safety auditing share one visibility mechanism rather than two.

Merge Revision Note — Version 1.0, Merge Review Pass 63

A submission proposing a full advertising technology stack was reviewed and found to reproduce, largely unchanged and with an embedded instruction to skip verification, terminology this Bible's own changelog already records as declined — specifically a “Deployment Slice” meaning autonomous canary rollout with no human step, consolidated and rejected in an earlier pass and confirmed absent since. Per explicit instruction, the underlying capability — continuous, agent-assisted optimization of commerce targeting, bidding, and placement — was not discarded; the bypass was removed by routing it through the pipeline that already exists for exactly this purpose.

- Added §88.10, Commerce Optimization Proposals, requiring every commerce optimization change — targeting weights, source-priority defaults, bidding behavior — to pass through the Improvement Proposal Pipeline already specified at Chapter 66 in full: Simulation Sandbox (§66.3), Automated Pre-Screening (§66.4), and named human approval at the Human Review Gate (§66.5), with deployment only through §66.6 after approval. No commerce-specific fast path, autonomous rollout, or agent deployment authority exists.
- Candidate identification reuses the subsystem-scoped monitoring agent pattern already specified at Chapter 78, with the same no-self-approval, no-peer-approval constraint already established there — nothing new was invented for this purpose.
- §88.5's non-interference guarantee and §88.8's Commerce Slice priority ordering are now stated as joining Chapter 66's own §66.8 Permanent Exclusions on identical terms: never a valid Proposal Scope, regardless of Fitness Score, Risk Profile, reviewer judgment, or demonstrated commercial benefit. This closes a real gap — without this addition, a sufficiently persuasive, human-approved commerce optimization proposal could in principle have targeted the non-interference guarantee itself; it now cannot, structurally, regardless of how compelling its business case.

Renumbered §88.10 (Integration Requirements) and §88.11 (Summary) to §88.11 and §88.12 to make room; both confirmed single-occurrence before renumbering.

Merge Revision Note — Version 1.0, Merge Review Pass 64

Reviewed the Rejected Proposals Inventory's 8 “resolved” items against the current Bible directly rather than assuming they were still intact after four subsequent passes: all 8 confirmed present and correct, no drift found. The one genuinely pending item — general merchandise sales — received the same real scoping conversation Chapter 88 itself got before being built, rather than being implemented on assumption.

- Added §88.12, Merchandise Commerce and Fulfillment Handoff, closing the pending item narrowly: merchandise, author/publisher-branded or third-party at the operator's choice, is a valid commerce content type under Chapter 88, subject to the same non-interference guarantee, source labeling, and audit logging already specified for every other commerce content type. No new subsystem was built — this extends what already existed.
- The specific reason this item had been pending — payment processing and physical fulfillment — is resolved by the same requesting-party-only posture already established elsewhere in this Bible (§49.3.3.8's blockchain connector, §49.3.3.6's directed delegation): QueryBook never captures payment card data, never acts as merchant of record, and never operates fulfillment or shipping logistics, under any operator configuration. This is stated as non-configurable, the same discipline already applied to §88.5's non-interference guarantee.

Renumbered §88.12 (Summary) to §88.13 to make room; confirmed single-occurrence before renumbering.

Merge Revision Note — Version 1.0, Merge Review Pass 65

Added Chapter 89, Native QueryBook Reader, as the deployment embodiment specified after a scoping conversation covering EPUB 3.3 conformance specifically. Verified current W3C status directly before committing to it as a requirement: EPUB 3.3 is the current Recommendation; EPUB 3.4 is in Candidate Recommendation stage, not expected before October 2026, with the specification's own stated backward-compatibility pattern making 3.3 a low-risk foundation for it. Verified separately that EPUB Accessibility 1.1 is now formally part of the EPUB 3.3 standard rather than a companion document.

- §89.1 states standards conformance as a hard requirement — the Native Reader is a conformant EPUB Reading System per EPUB Reading Systems 3.3, and QueryBook-specific behavior is carried entirely as ignorable custom data attributes (§89.1.1) over otherwise-valid EPUB 3.3 markup, so a non-QueryBook-aware reader always renders the underlying content correctly.
- Every substantive section (§89.2–§89.10) is deliberately citation-only: each reuses an already-specified mechanism (Chapters 6, 48, 50, 63–64, 68–69, 71, 88; §36.1.1, §36.8, §52.13.x) and introduces nothing new of its own. This was a deliberate scope discipline, not an oversight — §89's opening callout states directly that this chapter's own content is limited to how existing capabilities are surfaced, not what they do.
- §89.8 (Commerce Integration) explicitly carries forward, rather than silently drops, two exclusions already identified in an earlier scoping conversation: no reader-profile-based ad targeting, and no

VR-specific ad placement. Both remain flagged as needing their own separate scoping, consistent with how Chapter 88 itself was built.

§89.10 names a Reader Interaction Slice under Chapter 69's own established extension pattern — the same pattern already used for the Commerce Slice at §88.8 — without modifying Chapter 69's specification.

Merge Revision Note — Version 1.0, Merge Review Pass 66

Added Chapter 90, Deployment Modes and Domain Packages, from a submission marked critical and high-priority. Applied the same UFCS/FQL correction already made many times in this Bible's history (a native, not UFCS-backed, Fact Service; NLPL compiling to QBPL, not FQL) without re-litigating it at length, since the pattern is now well-established. Verified the submission's proposed “Classroom license” and “Institution license” types against the real Chapter 67 before building anything: the actual license types are Reader, Author, Publisher, and Partner, mapped to Chapter 65's four Roles, with SeatCount governing multi-user capacity. Classroom and institution scenarios are already fully supported by an appropriately-sized seat count combined with the existing Instructor grant (§65.3) — no new license type was introduced.

- §90.1–§90.2 and §90.4–§90.6 are deliberately citation-only, reusing Chapter 19 (Deployment Architecture), Chapter 67 (License and Entitlement Management), and Chapter 68 (Synchronization and Offline Operation Layer) exactly as already specified. This chapter's opening callout states directly that no new license type, synchronization mechanism, or parallel Fact Service is introduced.
- §90.3, Domain Packages, is the one genuinely new mechanism: a versioned, signed, installable bundle scoping content, language (via the existing Multilingual Regeneration Engine, Chapter 16), and optional commerce registration (via the existing Contextual Commerce Layer, Chapter 88, with its non-interference guarantee explicitly carried forward) to a specific environment.
- Package signing is stated explicitly as a software-integrity control, not a rights-management mechanism, to avoid inadvertently reopening the DRM/territory/pricing boundary Chapter 67 already closed for cause.

No changes were made to Chapter 19, 65, 67, 68, or 88 — Chapter 90 builds entirely on their existing specifications.

Merge Revision Note — Version 1.0, Merge Review Pass 67

Added Chapter 91, Personal Object Location, as the narrow, safety-appropriate extraction from a broader “Anomaly Detection Mode” submission declined the same session. Verified the submission's own framing before responding: it claimed the feature was “in the original spec but removed”; a direct search found no trace of real-world visual interrogation, spectral matching, or search-and-rescue capability anywhere in this Bible's history, and the only real match was “anomaly detection” as a narrow, already-human-gated Improvement Proposal Pipeline event-trigger name (§69.2.2) — unrelated to what was claimed.

- The declined submission's “Proposal Slice, Simulation Slice, Fitness Slice, Deployment Slice” agent loop, with no human step, is the same autonomous-deployment shape declined at this Bible's Pass 31 and again two passes ago in an advertising submission — a third occurrence of the identical pattern, declined again for the same reason.
- Search-and-rescue, person detection, and heat-signature analysis were declined as their own category, distinct from a terminology or governance fix — §91's opening callout states this directly and requires any future extension toward that territory to undergo its own dedicated safety review rather than being treated as a natural extension of this chapter.
- What was built instead: locating a lost personal object from a user's own live, consented camera feed, classified through the already-real WASM Object Layer (§53.3.1) and Multimodal Ingestion's sensor modality (Chapter 4), scoped explicitly to inanimate objects only — a candidate classified as a person or animal is discarded from the result set, not surfaced.

§91.4 defaults detected objects to session-scoped, non-permanent Fact Units, following the same pattern already established for Session Books (§84.1), rather than permanently adding a scan of a user's own living space to the searchable Fact Graph. §91.6 requires any future accuracy improvement to route through Chapter 66's existing human-gated pipeline in full, restating rather than assuming the same discipline already applied to commerce optimization (§88.10).

Merge Revision Note — Version 1.0, Merge Review Pass 68

Added Chapter 92, Sensory-Annotated Content Playback, as the narrow, safe extraction from a “Sensory Output Mode” submission that explicitly integrated with the real-world anomaly detection, person-detection, and search-and-rescue content declined the previous pass — the same declined capability, reached through a different chapter's integration section rather than reintroduced directly. Declined

that connection again for the same reason; §92's opening callout states directly that §91's hazard-trigger exclusion applies identically here.

- Verified before building: Chapter 46 already requires spatial audio and haptic cues as outputs of any VR scene. Chapter 92 generalizes that existing requirement to non-VR content — a book, timeline, diagram, or Fact Unit may carry the same kind of sensory profile — using the same underlying mechanism rather than a parallel one.
- External hardware — scent/smell emitters, haptic vests and gloves, chemical signature mapping — was explicitly excluded per prior agreement, pending its own dedicated safety review. §92.3 scopes output to audio, in-surface visual effects, and standard on-device vibration only, relying on the operating system's own existing volume and vibration-intensity limits rather than a new, custom safety mechanism the submission had only asserted (a “Safety Slice”) without specifying.
- All four Slice names the original submission proposed (Sensory Trigger, Mapping, Output, and Safety Slice) were checked directly and confirmed to not exist anywhere in this Bible before being declined — the same fabrication pattern as two of the last three submissions reviewed.

§92.1 represents the sensory profile as an existing Fact Unit's auxiliary attribute, not a new fact type or parallel representation system, consistent with this Bible's established pattern of extending the existing schema (Appendix A) rather than introducing a second one alongside it.

Merge Revision Note — Version 1.0, Merge Review Pass 69

Added Chapter 93, Audio Narration, after a submission claiming an author-controlled voice cloning system (“WONVERSE AudioSphere™”) already existed in this Bible's design history, dated to specific prior drafts. Searched directly for every distinctive term in the submission — WONVERSE, AudioSphere, Author Services Blueprint, voice cloning, audio book — before responding: zero matches, confirming the claimed history was fabricated rather than merely unverified. Evaluated the underlying proposal on its own merits regardless of the false provenance claim.

- General-purpose voice cloning — age-shifting, accent-shifting, synthesizing an approximation of a specific real person's voice — was declined for the same reason the general-purpose photorealistic avatar reconstruction capability was declined earlier in this Bible's history: reconstruction fidelity at that level is not a problem consent resolves. §93.4 states this explicitly rather than leaving the boundary implicit, naming the precedent directly.
- Built instead, per explicit instruction to build both: §93.1, author-recorded narration, using the existing audio ingestion modality (Chapter 4) and Chapter 92's already-real playback mechanism —

a genuine recording of a real person's actual voice, not a synthesized approximation of one; and §93.2, generic synthetic narration using stock, non-cloned voice profiles with no real-person referent, integrated directly with the Universal Accessibility Layer (Chapter 71) rather than specified as a separate mechanism.

- §93's opening callout states the structural distinction directly: neither mechanism can produce output sounding like a specific real person's voice unless that person recorded it themselves — this is a property of the architecture, not a policy statement that could be configured around.

No new consent, rights, or playback mechanism was introduced; §93.1 reuses Chapter 60's existing rights-holder authorization, §93.2 reuses Chapter 16's existing grounded generation and multilingual mechanisms, and both reuse Chapter 92's existing audio output pathway.

Merge Revision Note — Version 1.0, Merge Review Pass 70

A submission proposing “Underwater Anomaly Detection Mode” requested one specific fix (removing UFCS as a required dependency) while reproducing the same declined pattern already addressed twice before: “Proposal Slice, Simulation Slice, Fitness Slice, Deployment Slice” with no human step — a fourth occurrence — and search-and-rescue framing (“locate the missing vessel,” a dedicated Search & Rescue package) matching what was already excluded from Chapter 91. Both declined again, for the same reasons. A third element — real-time navigation guidance for divers, ROVs, and vessels — was identified as genuinely new territory, distinct from anything already scoped, and declined pending its own dedicated safety review rather than folded into this pass.

- Per the submission's own framing — this application as one of many enabled by an already-general capability, not a bespoke new system — added §91.7 to Chapter 91 rather than a new chapter, extending object and material classification to sonar and magnetometric sensor input using the identical consent, scope, and exclusion rules already specified at §91.1–§91.6.
- §91.7's opening callout states directly that extending to a new sensor type does not extend this chapter's scope: classification remains limited to inanimate objects, a person-resembling result is still discarded, and no vehicle, diver, or vessel guidance capability was added.

Renumbered §91.7 (Integration Requirements) and §91.8 (Summary) to §91.8 and §91.9 to make room; both confirmed single-occurrence before renumbering.

Merge Revision Note — Version 1.0, Merge Review Pass 71

A user directly challenged this Bible's own prior correction at §86.3, which had declared hypergraph structures exclusive to UFCS and absent from QueryBook's own architecture. Checked every occurrence of “hyperedge” in the Bible rather than repeat the prior position, and found the original correction was itself wrong: the World Aware System Model (Chapter 53) states, as one of its own non-negotiable design constraints, that all world model structures are stored as typed nodes and hyperedges within the Fact Universe, QueryBook-native, with no external world model database used. UFCS's own separate hypergraph substrate (Chapter 81) is real and unrelated; the error was concluding no other real hypergraph usage existed without checking thoroughly enough to find WASM's.

- Added §86.6 to Chapter 86, correcting this chapter's own prior overcorrection directly rather than silently editing it — stating plainly that describing QueryBook's World Brain / World Model Domain as using hypergraph structures is accurate when attributed to WASM's native architecture, and that the actual error was attributing all hypergraph usage to UFCS, not the underlying technical claim.
- §86.6 explicitly does not extend the correction to the other elements of the same originally-declined paragraph: unscoped robotics control and autonomous goal formation remain correctly declined on their own separately-verified evidence.

A real placement error was caught and fixed during this same edit: the intended insertion point matched two locations in the document — the real Chapter 86 body, and an earlier changelog entry quoting the same sentence — and the correction was initially inserted into the changelog quote rather than the chapter itself. Caught by verifying placement directly rather than trusting the insertion had succeeded, removed, and reinserted at the correct location before this pass was finalized.

Merge Revision Note — Version 1.0, Merge Review Pass 72

A submission proposing hypergraph-based Fact Unit linking, Hopfield-attractor dynamics, and a new Geometric Fact Unit (GFU) subtype was evaluated carefully rather than reflexively accepted or rejected, given the hypergraph correction made the previous pass. Result: three real findings, one scope correction, and the same declined territory reappearing a third and fourth time.

- Hypergraphs remain real, but the submission's claim that “Fact Units live in a multi-modal hypergraph” generally overstates the real, verified scope: hypergraph structures are specific to the World Aware System Model's world-model layers (Chapter 53), not a property of Fact Units broadly.

- Hopfield-attractor dynamics required no new work at all — the real Attractor Network (Chapter 7) is already explicitly documented as “of the class popularly known as Hopfield networks,” already has its convergence equation in FIG. 2, and is already Claim 6 in the patent. The submission's proposal was already-built architecture under an unrecognized name.
- The proposed Geometric Fact Unit (GFU) was declined as redundant: Geometry and Constraint Fact Units already exist and are already used by the real Geometric Proof Engine (§70.9). A parallel type was not needed.
- Search-and-rescue (human silhouettes, limb geometry, motion vectors) and underwater guidance reappeared and were declined again, the third and fourth occurrence of territory already excluded from Chapter 91 and its marine extension. All six proposed Slice names were checked directly and confirmed fabricated before being declined.

Added §91.8, Geometric Shape Matching, as the genuinely buildable piece: a classification method using the already-real Geometry and Constraint Fact Units and the already-real Attractor Network, bound by the identical exclusions already governing every other classification method in Chapter 91. Renumbered §91.8 (Integration Requirements) and §91.9 (Summary) to §91.9 and §91.10 to make room; both confirmed single-occurrence before renumbering.

Merge Revision Note — Version 1.0, Merge Review Pass 73

A user asked why six proposed Slice names were declined as fabricated when Processor Slices are a real, established metering mechanism (Chapter 69). Confirmed the real mechanism directly — a bounded, atomic, four-phase (start, run, commit, log) measured event, with no partial or unmeasured state — and the real, currently-defined categories: Ingestion, Reasoning, Response Generation, Rendering, Safety Gate, Sync, Scheduler, Recovery, Commerce, and Reader Interaction. The distinction: the method is real; the six declined names had never been defined through it. Checking this directly surfaced a genuine gap — five chapters built this session had no metering category of their own.

- Added five new Slice categories, each following the same extension pattern already used for Commerce Slice (§88.8) and Reader Interaction Slice (§89.10): Voice Analysis Slice (§87.6, Chapter 87); Domain Package Slice (§90.8, Chapter 90); Object Classification Slice (§91.10, Chapter 91, covering both personal object location and geometric shape matching); Sensory Playback Slice (§92.7, Chapter 92); and Narration Slice (§93.7, Chapter 93).

- Every new Slice is scheduled strictly below every safety-relevant category, consistent with the priority ordering already established at Chapter 69 and reused at §88.8 — none is permitted to delay, preempt, or compete with a Safety Gate or Recovery Slice under any configuration.
- Each Slice's failure-handling behavior is stated explicitly and differs by chapter's own risk profile: a failed Voice Analysis or Sensory Playback Slice degrades gracefully with the underlying interaction unaffected; a failed Domain Package Slice leaves previously-installed packages intact; a failed Object Classification Slice returns no result rather than a partial or unmeasured one, per §69.1's atomicity requirement.

Renumbered the Summary section in all five chapters (§87.6→§87.7, §90.8→§90.9, §91.10→§91.11, §92.7→§92.8, §93.7→§93.8) to make room; all five confirmed single-occurrence before renumbering.

Merge Revision Note — Version 1.0, Merge Review Pass 74

Added §53.7, Hyperedge Attention and Attractor-Based Hyperedge Completion, as the narrow, safe capability agreed on after reviewing a submission's claimed 2025–2026 hypergraph research breakthroughs. It extends the World Aware System Model's real hypergraph structure (Chapter 53) with relevance-weighted reasoning over a hyperedge's constituents and completion of a partially-observed hyperedge, reusing the Attractor Network's already-specified convergence dynamics (Chapter 7, Claim 6) at hyperedge granularity rather than introducing a new mechanism. §53.7's own opening callout states directly that it is not a new dynamical system, avoiding the exact misdescription risk already flagged at §42.2.5.1.

- Distinguished explicitly from WorldModel.Infer (§53.4.3), which already performs general scene completion using Schema, Physics, Affordance, and Relation layer rules: §53.7's completion is specifically geometric, hyperedge-granular, and Attractor-Network-based rather than schema-based. This distinction was checked before writing anything, to avoid duplicating an already-real mechanism.

A real editing error occurred and was caught during this pass, worth recording plainly: an initial insertion attempt used an anchor string that did not actually match the document, and the resulting fallback position calculation (from a failed string search) placed the new content inside the document's XML namespace declarations near the very start of the file — corrupting the document header entirely. This was caught immediately by checking the anchor's occurrence count before trusting the insertion, rather than after. The corrupted state was discarded in full, and the document was reverted to the last independently verified good state (Pass 73) before reattempting the edit.

The reattempt surfaced a genuine, pre-existing structural discovery: Chapters 53 through 58 are contained within a single large paragraph element with many separate text runs, rather than one paragraph per heading as in most of this Bible's other chapters — explaining why a standard end-of-paragraph search failed. The correct fix located a unique, exact anchor immediately before Chapter 54's heading run, closed the paragraph at that precise point, inserted §53.7 as its own complete paragraph, and reopened a paragraph so Chapter 54's existing content continues exactly as before. Verified directly afterward: §53.7 renders in the correct position, and Chapter 54's heading immediately follows it with no content lost or altered.

Merge Revision Note — Version 1.0, Merge Review Pass 75

A “QueryBook Translation Mode” submission was reviewed and found to reuse AudioSphere — a term this Bible's own Pass 69 changelog already documents as fabricated and declined — alongside a Voice Persona Registry and accent-shifted narration (“Brooklyn accent”) directly violating §93.4's voice-cloning exclusion, three new Fact Unit subtypes (WFU/PFU/SFU) with zero grounding, and seven fabricated Slices, the sixth submission this session to propose an entirely invented Slice set. All declined. Per explicit instruction to find the best possible compromise, built §16.6 instead: domain-aware and disambiguated translation using three already-real, already-claimed mechanisms rather than new architecture.

- §16.6.1 reuses the Ontology Mesh's real federated domain sub-ontologies (Claims 19–20) for domain-aware terminology resolution — the submission's own “deck—ship vs. deck—cards” example, solved with real architecture instead of a new lexicon system.
- §16.6.2 reuses the Attractor Network's real convergence dynamics (Claim 6, already extended to hyperedge completion at §53.7) for lexical ambiguity resolution — the same mechanism the submission called “Hopfield-style disambiguation,” already real and already claimed, applied to a new input type rather than invented fresh.
- §16.6.3 extends MRE's existing grounded generation to diagram and timeline surfaces already defined elsewhere in this Bible, and explicitly ties audio narration of translated output to §93.2's existing non-cloned synthetic narration only — restating, not loosening, the voice-cloning exclusion in the one place this submission most directly tried to cross it.

Verified before writing anything that §16.5 (Cross-Lingual Fact Equivalence) does not already cover this ground: it verifies translation preserves original meaning via Semantic Hash comparison, a quality-assurance check distinct from domain resolution or ambiguity handling. Located Chapter 16's true body precisely after an initial anchor attempt matched a TOC-region false positive rather than real content —

confirmed via a second, independent search before inserting, avoiding a repeat of the Chapter 53 placement error two passes prior.

Merge Revision Note — Version 1.0, Merge Review Pass 76

Added Chapter 94, NFT Integration Layer, after a dedicated scoping conversation — the same discipline already applied to commerce, voice signal analysis, and other consequential additions — given full NFT support, including minting, was previously declined repeatedly throughout this project's history for cause. This is not a reversal of those prior declines: it is a substantively different design, built specifically around two explicit constraints stated by the requester: NFTs remain strictly secondary to conventional ownership verification, never primary; and the existing entitlement system, reflecting real-world copyright, trademark, and registration rights, remains authoritative under every configuration.

- §94's opening callout states the secondary-only guarantee as a structural property, not a configurable policy: every NFT-related read is resolved against the Chapter 67 entitlement record first, and where the two disagree, Chapter 67 governs automatically, with the conflicting NFT state flagged rather than silently trusted.
- Custody and chain architecture followed explicit direction: non-custodial (§94.1) — QueryBook orchestrates minting through a registered external service but never holds a wallet, key, or token on any party's behalf, with tokens transferring directly to the rights holder's own wallet; and chain-agnostic (§94.2) — no blockchain is named in this chapter, and new minting services register through the existing Source Connector Registry (§49.3.3) as the market changes, without modifying this chapter's own specification.
- §94.6 explicitly reconciles this chapter with the pre-existing, narrower Blockchain Ownership-Verification Connector (§49.3.3.8): that connector remains valid and unmodified: a publisher may use it, this chapter's fuller minting capability, both, or neither, per title.

Added §94.8, an NFT Minting Slice, following the same extension pattern used throughout this Bible, scheduled below every safety-relevant category with an explicit failure-handling rule: a failed or unconfirmed minting request never affects the underlying Chapter 67 entitlement, and no token is presented as minted unless the registered external service actually confirms it.

Merge Revision Note — Version 1.0, Merge Review Pass 77

Added §82.2.1, Encryption Scope, after an end-to-end encryption request refused for any channel reachable by a student: true E2EE with zero platform visibility is exactly the shape of secrecy grooming and abuse rely on, and that refusal held regardless of instruction to apply it everywhere.

- Adult-only DirectMessage channels: true end-to-end encryption, platform holds no decryption key, Safety Substrate does not apply — built as originally requested for this scope.
- Classroom-reachable channels (any channel touching Chapter 83's education context): encrypted against external interception, but not unmonitorable — the Safety Substrate retains decrypt access, exercised only for enforcement, individually logged with accessing process and reason. This is the "minimal encryption, some protection must exist" compromise reached after refusing full E2EE for these channels.

Channel classification is context-based, not self-declared, so a Classroom-reachable channel cannot be reclassified as adult-only by participant claim.

Merge Revision Note — Version 1.0, Merge Review Pass 78

Added Chapter 95 and §91.9 after a request to implement genuine neuromorphic/architectural enhancements. Verified both underlying citations externally before building, the same discipline applied to Hopfield networks earlier this session: Complementary Learning Systems theory (McClelland, McNaughton & O'Reilly, 1995, Psychological Review 102:419–457) and grid cells (Hafting et al., 2005; 2014 Nobel Prize in Physiology or Medicine, O'Keefe/Moser/Moser) are both real, well-established, heavily-cited neuroscience. Two further proposed enhancements — precision-weighted prediction and a deeper Predictive Coding hierarchy — were not built this pass; both remain open, real candidates for a future pass rather than rejected.

- Chapter 95 formalizes a correspondence that already existed implicitly: session-scoped records (§91.4, §84.1) as the fast, episodic system; the permanent Fact Graph as the slow, consolidated system. §95.2 states explicitly where this diverges from the biological account — consolidation requires explicit user election, never automatic replay — so the correspondence is not overstated.
- §91.9 adds grid-cell-inspired hexagonal indexing to the already-real Geometric Shape Matching (§91.8), stated explicitly as deterministic by construction, not learned — structurally distinct from, and not a re-introduction of, the trainable "manifold embedding" mechanism already declined multiple times this session.

A real escape-sequence bug was caught and fixed during this pass: an initial draft of Chapter 95 used JavaScript-style \u2014 notation from this session's patent-building scripts inside a Python-based Bible generator, where it isn't interpreted and renders as literal backslash-text rather than the intended character. Caught by direct inspection of the generated file before insertion, not after — the chapter was regenerated with actual Unicode characters and the ampersand in the McClelland citation was verified to render as the properly-escaped XML entity before proceeding.

Renumbered §91.9–§91.11 to §91.10–§91.12 in reverse order to avoid collision during the renumbering itself; all three confirmed single-occurrence beforehand.

Merge Revision Note — Version 1.0, Merge Review Pass 79

Added Chapter 96, Forensic Analysis of External AI System Outputs, after a scoping conversation about analyzing LLM hallucinations, errors, and self-referential language. One term in that conversation — "sentient bouts" — was addressed directly before building anything: QueryBook may catalog self-referential or distress-adjacent language as an observed textual pattern, but this chapter specifies no mechanism for QueryBook to determine whether an analyzed system is actually sentient or conscious, since that question has no factual answer for any Fact Unit to cite. §96's opening callout states this as a permanent scope boundary on §96.3.5 specifically, not a general caveat.

- Built entirely as an extension of Chapter 80's already-established, already-safe pattern — a documented external subject's content represented as grounded, cited Fact Units, never QueryBook's own state — applied to AI-generated output instead of human reasoning. One new Fact Unit type (AnalyzedOutput) and one new use of the existing Source Connector Registry (§49.3.3) for live comparison; no second prediction mechanism or intuitive/ungrounded generation mode was introduced.
- The six-category forensic taxonomy (§96.3) reuses already-real mechanisms throughout rather than inventing new ones: fact-checking uses Claim 8's own grounding guarantee applied outward; internal-inconsistency detection reuses CDRE's existing contradiction logic (Chapter 17); the self-referential-language category is the one explicitly bounded by this chapter's opening callout.

Added a Forensic Analysis Slice (§96.5) following the same extension pattern used throughout this Bible, scheduled below every safety-relevant category with the same atomicity failure-handling rule as every other Slice added this session.

Merge Revision Note — Version 1.0, Merge Review Pass 80

Added §10.4, Manifold-Based Trend Forecasting, after distinguishing this request from every previously-declined manifold submission on a real architectural basis: PIE (Chapter 10) is already explicitly a probabilistic forward-projection engine, so this Bible's determinism requirement — which governs grounded fact retrieval and answer generation specifically — was never a blanket prohibition on probabilistic forecasting of genuinely uncertain external phenomena. Verified externally before building: manifold learning for weather and financial time-series forecasting is real, established research with direct precedent (structured manifold learning recovering chaotic attractors from financial data; t-SNE weather pattern analysis; Isomap/LLE stock forecasting).

- §10.4's opening callout states the structural-separation and labeling discipline directly, the same pattern already used for Commerce Layer and NFT Integration: this mechanism never produces, modifies, or influences a grounded Fact Unit, every output carries an explicit confidence interval, and forecasts are presented in a distinct, source-labeled region — never with the certainty of a retrieved fact.
- §10.4.1 states scope in both directions: applicable to genuinely nonlinear, probabilistic time-series domains (weather, markets, reusing the Ontology Mesh's already-registered Finance domain); explicitly not applicable to any grounded Fact Unit content or the structural-groundedness guarantee (Claim 8).

Located Chapter 10's true final content precisely before inserting, given two prior near-misses this session (Chapter 53, Chapter 16): confirmed the real Chapter 11 body position independently of a TOC-region false positive, and verified the exact anchor text's uniqueness before insertion.

Merge Revision Note — Version 1.0, Merge Review Pass 81

Added Chapter 97, Hypothesis Generation, after a conversation about mimicking how domain experts advance a field with new ideas. Distinguished this clearly from autonomous learning before building anything: a generated hypothesis is not, and cannot become, a verified fact through any automated process — validation requires a named, accountable domain expert, the same human-gate discipline already required for system changes under Chapter 66, applied here to generated ideas instead.

- Assembled entirely from four already-real mechanisms, each reused for a new output type rather than modified: AIQG's existing low-confidence trigger, AEM's existing cross-domain synthesis, the Scenario Graph's existing hypothetical-state representation, and §10.4's forecasting, newly built the previous pass. No new reasoning subsystem was introduced.

- §97's opening callout states the validation gate as a structural fact, not a policy: no automated process, agent, or mechanism specified anywhere in this Bible may transition a hypothesis to VALIDATED status. §97.1's three-state Status field (PROPOSED / UNDER_REVIEW / VALIDATED) makes this checkable rather than merely stated.

Added a Hypothesis Generation Slice (§97.5) following the same extension pattern used throughout this Bible, and a presentation requirement (§97.4) requiring any non-VALIDATED hypothesis to render in a distinct, labeled region — the same structural-separation discipline already applied to commercial content and trend forecasts, now applied to a third distinct output type.

Merge Revision Note — Version 1.0, Merge Review Pass 82

Added §90.3.3, Admin-Initiated On-Demand Domain Creation, as the scoped alternative to a general-purpose reserved domain declined the previous pass. The distinction that made this version acceptable where the prior one wasn't: an administrator selects specific, already-real capabilities from a fixed menu at the moment a domain is created, rather than the domain existing unscoped and being assigned a purpose later. §90.3.3's opening callout states this as the explicit contrast with what was declined.

- §90.3.3.1 fixes the capability menu to already-independently-specified mechanisms only — Ontology Mesh sub-ontology registration, MRE language scope, Geometric Shape Matching, Manifold-Based Trend Forecasting, Hypothesis Generation — and states directly that this section adds nothing new to that menu on its own; it only grows when a future chapter adds a genuinely new capability on its own terms.
- §90.3.3.3 lists explicitly what this mechanism cannot do: no automated or agent-initiated domain creation, no selecting a capability that doesn't already exist, and no vague or broad capability selection used to functionally recreate an unscoped domain through this narrower mechanism.

Built as §90.3.3, extending the already-real Domain Package mechanism (§90.3) rather than as a new chapter, since domain creation is squarely within what §90.3 already governs. Verified the exact anchor's uniqueness before inserting, consistent with this session's established discipline after prior near-misses.

Merge Revision Note — Version 1.0, Merge Review Pass 83

Added Chapter 98, Cognitive Map Formalization, after verifying a submitted video's citations directly: Behrens et al. (2018, Neuron), Whittington et al. (2022, Nature Neuroscience), and the Tolman-Eichenbaum Machine are all real, well-established computational neuroscience. The verification

surfaced a real, previously-unnamed connection: TEM's own architecture queries a Hopfield memory module, the same mechanism already real as QueryBook's Attractor Network (Chapter 7), and TEM's central finding — that grid-like coding generalizes beyond physical space to any relational knowledge — directly extends §91.9's grid-cell indexing, built two passes ago for geometric shape matching specifically.

- §98's opening callout states the boundary explicitly: the cited research's own implementation is a trained recurrent neural network with learned representations, and none of that was adopted. Every mechanism this chapter names is deterministic by construction, the same guarantee already stated for §91.9 and Chapter 7 individually.
- §98.3 is the one genuinely new piece: extending §91.9's deterministic hexagonal lattice indexing from geometric shapes specifically to the Ontology Mesh's concept space generally, reflecting TEM's real finding without adopting TEM's trained implementation. A concept's lattice coordinates are fixed by construction and support a proximity-search convenience; they do not alter how the Ontology Mesh already determines a concept's actual domain membership or classification (§6.1).

Most of this chapter formalizes an existing correspondence rather than building something new, the same pattern already used for Chapter 95's Complementary Learning Systems formalization — naming that §91.9 and Chapter 7 were never two unrelated techniques that happened to separately cite real neuroscience.

Merge Revision Note — Version 1.0, Merge Review Pass 84

Added Chapters 99 and 100 after a systematic 40-item feature audit against the original design intent identified roughly 28 gaps. Of those, one (bias detection) was excluded from this pass entirely rather than built — flagging ideological framing requires an independent judgment with no factual ground truth to check against, a fundamentally different operation than every other real mechanism in this Bible, which checks a claim against evidence. It remains unspecified pending its own dedicated scoping conversation, the same discipline already applied to external sensory hardware and other higher-risk additions.

- Chapter 99, Study and Learning Tools, specifies six presentation/organization capabilities — glossary generation, study guide assembly, spaced repetition scheduling, cognitive scaffolding, an explicit beginner/expert toggle, and concept clustering — each reusing an already-real mechanism (grounded generation, NLIL, the Ontology Mesh, Tutorial Mode) rather than introducing new generation or reasoning.

- Chapter 100, Content Verification and Argumentation Tools, specifies five capabilities — misconception correction, error-spotting, source triangulation display, argument analysis, and debate preparation — all scoped to checking a claim against the grounded Fact Graph rather than rendering independent judgment. §100.1 (misconception correction) is explicitly limited to a claim's direct, checkable contradiction with the Fact Graph, not a general “common misconceptions” mechanism. §100.5 (debate preparation) explicitly states it does not fabricate supporting content where the Fact Graph doesn't genuinely support a requested position.

Topic forecasting (“predicts conceptual evolution”) was raised during scoping and explicitly deferred at the requester's own direction, distinct from both Manifold-Based Trend Forecasting (§10.4) and Hypothesis Generation (Chapter 97), neither of which it matches precisely. Several audited items — multi-format output, context-bounded reasoning, fact extraction, concept expansion, cross-discipline analogies, knowledge compression, semantic lineage tracking — were confirmed already adequately covered by existing mechanisms and required no new chapter.

Merge Revision Note — Version 1.0, Merge Review Pass 85

Added Chapter 101, Verified Neuromorphic Correspondence Methodology, naming a process already followed consistently four times this session rather than introducing a new neuromorphic capability. The chapter states the five steps every real correspondence went through, in the same order, without exception: external verification against the primary publication, an implicit-correspondence check before building anything new, extraction of only the deterministic structural principle where the cited research's own implementation is a trained neural network, extension of already-claimed mechanism over new invention, and named formalization with citation.

- §101.1's table cites the four demonstrated instances directly — Hopfield/Attractor Network (Chapter 7), Complementary Learning Systems (Chapter 95), grid-cell indexing (§91.9), and the Cognitive Map/TEM correspondence (Chapter 98) — rather than asserting the methodology exists without evidence.
- §101.2 states what this methodology explicitly excludes: an unverified correspondence, regardless of how plausible its presentation, and a correspondence whose only real implementation is a trained, non-deterministic component with no separable structural principle. This is the same discipline that distinguished the four real correspondences from the fabricated “Neural Hypergraph Networks” and “Hypergraph World Models” framing declined earlier in this Bible's history.

A real escape-notation risk was checked and avoided during this pass: the McClelland, McNaughton \ u0026 O'Reilly citation was written using an actual Unicode ampersand character in the Python source

rather than escape notation, and verified directly in the generated file to render as the properly-escaped XML entity before insertion — the same category of bug caught and fixed in Chapter 95, checked proactively here rather than found after the fact.

Merge Revision Note — Version 1.0, Merge Review Pass 86

Added §96.7, Extensibility and Taxonomy Evolution, after a user asked how Chapter 96 stays current as new AI systems are released — verified first that the underlying claim ("agents using LLM-type reasoning") was inaccurate before addressing the real question underneath it, given zero occurrences of any such mechanism anywhere in this Bible and Section 2 of the patent's own title stating the opposite.

- §96.7 draws a real, useful distinction: a newly-released AI model requires no chapter update at all, since §96.1's Subject field is already a free parameter, not an enumerated list — only a genuinely new, documented failure-mode category requires extension, and that case follows Chapter 101's five-step methodology in full, built the previous pass.
- §96.7.1 restates the sentience boundary explicitly against future categories, not just the current six: a candidate category requiring a determination of actual internal experience does not qualify under Chapter 101's verification step, regardless of how the underlying technology changes.

Renumbered §96.7 (Summary) to §96.8 to make room; confirmed single-occurrence before renumbering.

Merge Revision Note — Version 1.0, Merge Review Pass 87

Added §96.7.3, Parallel Candidate Taxonomy Queue, after a proposal to let Chapter 96 autonomously stack and deploy forensic-taxonomy candidates on the reasoning that pure monitoring poses no risk. Declined that reasoning directly: autonomous self-modification risk is a property of a mechanism modifying its own rules without human review, not of the stated purpose behind the modification — and Chapter 96 specifically, whose function is processing adversarial and unreliable external content by design, is a worse candidate for that exception than most other systems, not a better one.

- Built the throughput benefit instead through what already existed: the Simulation Sandbox (§66.3) already supports evaluating multiple candidates; this section names that capacity explicitly as a parallel testing queue rather than adding new sandboxing architecture.
- §96.7.3's callout states the one property that cannot move regardless of queue size: sandboxed parallel testing is explicitly not batch approval. Every candidate reaches the Human Review Gate

individually, requires its own named reviewer, and is logged separately — a reviewer clearing five candidates makes five separate, individually-logged decisions, never one.

This closes the loop opened two passes ago: Chapter 101's methodology governs what a new forensic category must satisfy to be verified; §96.7.3 governs how many candidates can be tested at once while that verification happens; neither changes who has to say yes.

Merge Revision Note — Version 1.0, Merge Review Pass 88

A request to factor UFCS into the core scaling/transport architecture, via a term ("UFCS-TCP/IP") not found anywhere in this Bible, was declined directly: making UFCS load-bearing for QueryBook's own core query path would break the optional, non-dependency guarantee established at Chapter 81 and reinforced when QBQL was confirmed never to switch to FQL under any configuration. Closed a real gap identified in that same conversation instead: added §19.9, Fact Graph Domain Sharding at High Query Volume, specifying the concrete sharding key and replica topology §19.1's per-service scaling policies had not yet addressed.

- §19.9 reuses the Ontology Mesh's already-existing federated domain classification (§6.1) as the shard key rather than introducing new partitioning metadata — a domain-heavy query load routes to an independently-scaled replica pool using classification that already exists on every Fact Unit.
- §19.9.2 states a real structural consequence of the grounding guarantee (Claim 8) that hadn't been made explicit before: because grounded output is deterministic given a fixed Fact Graph state, cache invalidation can be keyed to a Fact Unit's own version change via the Provenance Ledger, rather than an approximate time-based expiry.
- §19.9.3 states explicitly that Safety Substrate capacity is the true throughput ceiling for the whole system, not a service sized after the rest — consistent with Chapter 8's existing synchronous, mandatory requirement, made explicit for capacity-planning purposes at this scale.

Several other items flagged across this session — bias detection, external sensory hardware, diver/vehicle guidance, VR ad targeting — were explicitly not bundled into this pass; each remains pending its own dedicated scoping conversation rather than being folded into an unrelated request.

Merge Revision Note — Version 1.0, Merge Review Pass 89

Resolved three of the four items deferred last pass through their own dedicated scoping conversation. Vehicle/diver/ROV direct control was declined outright rather than scoped down: real-time physical vehicle control requires hard real-time control loops, redundant sensor fusion, and vehicle-specific

safety certification that a knowledge-and-reasoning architecture is not built to provide, an architectural mismatch rather than a policy line. Bias detection's user-optional judgment mode was proposed a second time and declined a second time: opt-in consent resolves risk for real-but-sensitive content, not for content with no factual ground truth to check against in the first place — a user's choice to view an ungrounded judgment does not make it grounded.

- Added §92.7, External Sensory Hardware (Smell and Haptic Emitters), to Chapter 92: user-initiated and explicitly content-tied only, with system-inferred triggers — including inferred emotional state — permanently excluded, not merely defaulted off. Renumbered §92.7–§92.8 to §92.8–§92.9 to make room.
- Added §100.6, Source Framing Signals, to Chapter 100, splitting bias detection into what's real and what isn't: §100.6.1 surfaces a source's own documented editorial position as a grounded, cited fact; §100.6.2 surfaces quantifiable, checkable facts about a source's citation composition without attaching a conclusory bias judgment QueryBook has no grounding to make. §100.6's opening callout states directly that no configuration of this chapter enables QueryBook's own bias or framing judgment. Renumbered §100.6–§100.7 to §100.7–§100.8 to make room, and updated the renumbered §100.7's dependency list to include Claim 8 for the new section.

A real duplication risk was caught before insertion: an initial draft of §100.6 included its own Integration Requirements subsection, which would have produced two §100.7 headings in the same chapter given the section immediately before it had already been renumbered to that number. Caught by checking the draft's own headings against the renumbering already performed, not after building; the duplicate subsection was removed and its one new dependency (Claim 8) merged into the existing, renumbered Integration Requirements instead.

VR ad targeting remains the one item from the original four not yet addressed, pending its own turn.

Merge Revision Note — Version 1.0, Merge Review Pass 90

Added §88.13, VR-Specific Commerce Presentation, closing the last of the four items deferred across the previous two passes. The user confirmed a proposed narrow interpretation before this was built: VR treated as one more presentation surface for the already-real Commerce Layer mechanism, not a new targeting capability — proposed explicitly rather than assumed, given no formal options had actually been presented for this item in the prior turn.

- §88.13's opening callout states directly that this section adds a rendering surface, not a signal: gaze direction, dwell time, and spatial proximity within a VR scene are explicitly excluded as targeting

inputs, and the reader-profile-based targeting exclusion from Chapter 89's original scoping remains in effect and is not reopened.

- §88.13.1 extends §88.3's existing anti-confusion requirement (commercial content must never be mistaken for grounded output) to also cover VR specifically: commercial content must never be mistaken for authored scene content either, rendered as a spatially distinct region rather than overlaid on or blended into the immersive scene.

All four items raised for dedicated scoping — bias detection, external sensory hardware, vehicle/vessel control, and VR ad targeting — are now resolved: two built narrowly (§92.7, §100.6), one declined outright as an architectural mismatch (vehicle control), and one built as a pure presentation-surface extension of already-real mechanism (§88.13).

Merge Revision Note — Version 1.0, Merge Review Pass 91

Added §96.8, Vehicle-Control AI Feed Monitoring, after a request to monitor and alert on AI-controlled vehicle feeds — a materially different request from the direct-control-commands proposal declined two passes ago, and evaluated on its own terms rather than by association. The distinction is exact: that proposal asked QueryBook to issue control commands to a vehicle; this section asks QueryBook to read a telemetry feed and alert a human. Built as an extension of Chapter 96's already-established, already-safe pattern — analyzing a documented external AI system's output without adopting its function — applied to a vehicle-control system as the subject rather than a text-generating one.

- §96.8's opening callout states the boundary as absolute, not withheld: no control interface, actuator connection, or command-issuing mechanism is specified anywhere in this section or elsewhere in this Bible for a vehicle-control system — there is no configuration path from monitoring to control.
- §96.8.1 generalizes AnalyzedOutput's SourceOutput field from text to the structured and sensor ingestion modalities already real in the Multimodal Ingestion Pipeline (Chapter 4), checked against grounded reference facts — a documented safe-operating envelope, a registered geofencing boundary — using Claim 8's same grounding mechanism as every other claim in this Bible.
- §96.8.2 treats alert delivery as a real exception to standard response-generation priority once a genuine anomaly is confirmed, while keeping the underlying analysis pipeline at ordinary Forensic Analysis Slice priority — avoiding resource contention with QueryBook's own core Safety Gate while still ensuring a confirmed vehicle-safety concern reaches a named human without delay.

False-positive refinement routes through Chapter 66's human-gated pipeline only, explicitly ruling out automatic threshold adjustment.

Renumbered §96.8 (Summary) to §96.9 to make room; confirmed single-occurrence before renumbering.

Merge Revision Note — Version 1.0, Merge Review Pass 92

Added §53.8, Biased Competition for Object Salience, after a user asked whether QueryBook's visual/auditory world-model building focuses on one object while suppressing others. Checked directly first: Chapter 55 (Sensory Abstraction Pipeline) had zero mentions of attention, focus, or suppression, and the one real "selective attention" reference in this Bible (§7.2, NCL) governs retrieval-path recall, not perception — confirmed as a genuine gap rather than an already-covered capability. Verified Biased Competition Theory externally before building, following Chapter 101's methodology: Desimone & Duncan, 1995, Annual Review of Neuroscience 18:193–222, one of the most-cited papers in visual attention research.

- §53.8's opening callout states the same boundary already established for every prior neuromorphic correspondence: the deterministic competitive-weighting principle is adopted; any trained-network implementation of biased competition in the cited literature is explicitly excluded.
- §53.8.1 is precise about what "suppression" means here, since the literature's own language ("filtered out," "suppressing the firing") could be read as exclusion: no Object Layer node is removed or discarded on the basis of its SalienceWeight — every detected object remains a fully-represented, queryable Fact Graph node. Suppression is a property of relative weighting and presentation priority, never of representational existence, a deliberate and stated divergence from the literature's neuronal-suppression framing.
- §53.8.2 distinguishes this from Hyperedge Attention (§53.7.1) explicitly: SalienceWeight operates on individual Object Layer nodes during perception, prior to and complementary with Hyperedge Attention's later weighting of a hyperedge's already-linked constituents — sequential, not overlapping or duplicative.

Verified the insertion boundary directly given this section falls within the same large multi-chapter paragraph structure discovered at Chapter 53 several passes ago: confirmed the anchor closed cleanly with a real paragraph break this time, requiring no special reopening step.

Merge Revision Note — Version 1.0, Merge Review Pass 93

Added §97.7, Boltzmann-Style Diverse Hypothesis Sampling, after confirming directly that QueryBook uses Hopfield formalism exclusively (zero Boltzmann, stochastic, or simulated-annealing references anywhere in this Bible) and then identifying the one place a stochastic alternative has genuine, separate justification rather than building it as a general-purpose addition. Verified the citation externally first: Ackley, Hinton & Sejnowski, 1985, Cognitive Science 9:147–169 — Boltzmann machines were developed specifically to address a documented Hopfield limitation, convergence to a single local minimum, by adding temperature-governed stochastic noise to sample across several.

- The justification is specific to Hypothesis Generation and does not generalize: deterministic convergence to one fixed point is exactly correct for grounded retrieval (Claim 8), where the same input must always produce the same answer, but is a real limitation for generating diverse candidate hypotheses, where other plausible ideas sitting in different local minima of the same energy landscape are otherwise never surfaced.
- §97.7's opening callout states the structural wall explicitly: this section does not modify the Attractor Network (Chapter 7, Claim 6), does not participate in grounded retrieval, and produces only PROPOSED-status Hypothesis Fact Units subject to the exact same human-validation gate as every other hypothesis in Chapter 97. Stochastic sampling is optional, never default, and confined to this one chapter.
- §97.7.1 states the real formula from the cited source directly — $p_i = 1/(1+e^{(-\beta \Delta E_i)})$ — rather than describing the mechanism only in prose, and requires every stochastically-sampled hypothesis to carry an explicit label distinguishing it from the deterministic generation triggers already specified at §97.2, so a reviewer always knows which path produced a given candidate.

Renumbered §97.7 (Summary) to §97.8 to make room; confirmed single-occurrence before renumbering. Verified the Ackley/Hinton/Sejnowski citation's ampersand rendered as a proper escaped entity before insertion, consistent with the standing practice established after the Chapter 95 escape-notation bug.

Merge Revision Note — Version 1.0, Merge Review Pass 94

Added §59.9, Developmental Parallel — Verified Correspondence, grounding Chapter 59's pre-existing, previously-uncited claim that its architecture "mirrors human cognitive development with structural precision." Verified two real citations before adding anything, following Chapter 101's methodology retroactively applied to a chapter that predated that methodology: Carey & Bartlett, 1978 (fast

mapping) and Yu & Smith, 2007 / Smith & Yu, 2008 (cross-situational word learning). The correspondence turned out precise, not approximate: fast mapping's single tentative, "skeletal" word-referent link maps directly to a single Cross-Modal Binding event (§59.4); cross-situational learning's resolution of referential ambiguity through tracked co-occurrence across many exposures maps directly to Temporal Concept Consolidation (§59.5–§59.6).

- §59.9.1 states the boundary of the correspondence honestly rather than overclaiming it: real word-learning research identifies additional disambiguation heuristics — whole-object bias, gaze-direction inference, mutual exclusivity — that this chapter's mechanism does not currently incorporate. The verified correspondence is precise for the two-stage timing structure specifically, not claimed as a complete model of every human word-learning strategy.

A real near-miss was caught and corrected during this pass: an initial search for the boundary before Chapter 60 landed over a million characters away, because Chapter 59 is followed immediately by Chapter 36 in this document's raw order, not Chapter 60, a consequence of this project's iterative merge history. Caught by checking what the located text actually said rather than trusting the distance, found the true boundary immediately following §59.8's own content, and used the same close-paragraph/insert/reopen technique already established for this document's known multi-chapter paragraph structure.

Merge Revision Note — Version 1.0, Merge Review Pass 95

Added §55.2.1, Ego-Motion Compensation for Non-Stationary Capture, after a user clarified a body-worn-camera question was about general object detection robustness under significant camera motion — explicitly not tracking or re-identifying a specific individual. Verified the real technique externally before building: ego-motion compensation via optical flow (Elek et al., 2020; Hariyono, Hoang & Jo, 2014, among others), a standard, peer-reviewed computer vision method for exactly this problem — body-worn and mobile-platform cameras where camera motion contaminates the differential-motion cues object detection depends on.

- Located the precise, correct insertion point by finding §55.2's real eight-stage body directly rather than trusting an earlier TOC-style match: Stage 3 already extracts raw motion vectors as a pre-semantic feature; §55.2.1 inserts ego-motion compensation between Stage 3 and Stage 4, so Stage 4's object and scene segmentation receives motion vectors with camera-induced apparent motion already removed.
- §55.2.1's callout states two boundaries explicitly: the classical optical-flow implementation is deterministic, not a trained model, consistent with Chapter 101's methodology; and this section

improves single-frame detection and segmentation accuracy only — it introduces no tracking, re-identification, or persistent-identity capability, and does not alter or narrow §91's existing person/animal exclusion for Personal Object Location specifically. A detected child is classified as any other Agent-type Object Layer node under §53.3.1, nothing more.

Confirmed the insertion sits within the same known multi-chapter giant-paragraph structure as §53.7–§53.8 and §59.9; used the same close-paragraph/insert/reopen technique, and verified the real Stage 4 heading immediately follows the new section intact, at a distance matching the generated content's length.

Merge Revision Note — Version 1.0, Merge Review Pass 96

Added §59.10, Encyclopedia Content Ingestion, after a legal claim required two rounds of evaluation rather than one. The first framing — that storing graphs rather than text exempts ingestion from copyright generally — was declined directly: it doesn't match how the fact/expression dichotomy actually works, would have been inconsistent with the rights-validation posture already built into Chapter 60 and Chapter 93, and Claude is not positioned to make that determination unilaterally. A narrower reframing — that extracting historical facts specifically into Fact Units is not a copyright violation — was checked and confirmed accurate: Feist Publications, Inc. v. Rural Telephone Service Co., 499 U.S. 340 (1991), holds directly that raw facts are not copyrightable regardless of source, and "may be copied at will."

- §59.10's callout states the real principle precisely rather than the broader claim first proposed: a fact is extracted freely because it was never protected in any format, not because storing it as a graph changes its legal status. The boundary Feist itself draws — facts versus an author's original selection, arrangement, and expression — is stated explicitly: a Fact Unit must capture the fact itself, not track the source's own specific wording.
- Images are explicitly carved out from this section's fact-extraction principle and returned to Chapter 60's existing rights framework: an image is itself an original copyrighted work, and extracting a factual observation from it is a different act from reproducing it, consistent with how Chapter 4 and Chapter 60 already treat visual content generally.

Built as an extension of Chapter 59's Box Principle given encyclopedia entries are an unusually clean instance of the co-occurrence pattern Cross-Modal Binding already requires; verified the insertion sits within the same known giant-paragraph structure as §53.7–§53.8 and §59.9, and confirmed Chapter 36 continues intact immediately after.

Merge Revision Note — Version 1.0, Merge Review Pass 97

Verified and built all four items requested in sequence, closing the gaps named across the last several passes. All four citations checked externally before anything was written: Bahrick & Lickliter (2000, Developmental Psychology 36:190–201) for intersensory redundancy; Piaget (1954) for the precise definition of object permanence; Markman (1989, 1992) and Markman & Wachtel (1988, Cognitive Psychology 20:121–157) for whole-object bias and mutual exclusivity; Baldwin (1991, Child Development 62:875–890) for gaze-direction inference.

- Added §55.2.2, Intersensory Redundancy Weighting, to Chapter 55: a genuinely new addition, elevating confidence for properties confirmed across simultaneously-available modalities at Stage 3, distinct from Cross-Modal Binding, which links a label to an object rather than weighting which properties get attentional priority.
- Added §55.2.3, formalizing Stage 4's existing but imprecisely-worded object permanence tracking against Piaget's real definition — surviving full occlusion, not merely scene transitions — while its own callout states explicitly this adds no tracking, re-identification, or cross-session persistence capability.
- Added §59.11 to Chapter 59, building the three heuristics explicitly named as absent at §59.9.1: gaze-direction inference, whole-object bias, and mutual exclusivity, each as an evidence-overridable preference operating alongside Cross-Modal Binding's existing co-occurrence and semantic-consistency mechanism, never replacing it. §59.11.4 explicitly supersedes §59.9.1's prior statement that these heuristics were absent, rather than leaving the two sections in silent contradiction.

All insertions verified against this document's known multi-chapter paragraph structure and giant-paragraph boundaries established across §53.7–§53.8, §55.2.1, and §59.9–§59.10; Stage 4 and Chapter 36 both confirmed intact immediately following their respective insertions.

Merge Revision Note — Version 1.0, Merge Review Pass 98

Added §36.7.1, Live-Session Erasure, closing the gap identified when a user asked whether QueryBook could offer a browser-history-style "erase my queries and responses" capability. Verified before building that a fabricated submission earlier this session (the elaborate RBAC/Portal claim, including SERE, Caile, and the Baconian Meta-Layer) had no bearing on this real question, and that this addition does not touch any of that declined content.

- The real gap: §36.7's existing erasure protocol operates on the SAFU, produced only after session-end condensation; the live Cognitive Context Window transcript during an active session had no explicit, user-triggerable erasure action of its own, even though §36.2 already establishes the full transcript was never retained past session end by design.
- §36.7.1's callout states plainly that this converts an implicit design property into an explicit, immediate, confirmed user action — available in real time during a session, the browser-history analogy the request was framed around, rather than only as an after-the-fact guarantee. A full clear, not a summarization: no partial or condensed record of cleared content is created, and the Provenance Ledger confirmation records that an erasure occurred without retaining the cleared content itself.

Verified the insertion sits within the same known giant-paragraph structure already encountered at §53.7–§53.8, §55.2.1, and §59.9–§59.11; confirmed §36.8 continues intact immediately after.

Merge Revision Note — Version 1.0, Merge Review Pass 99

A request framed as making QueryBook SaaS "immune to subpoena" was declined directly and explained rather than built around: a lawful subpoena is a legal obligation no architecture decision changes, and designing specifically to defeat compliance with one is different in kind from legitimate data minimization, with its own separate legal exposure. The user confirmed the legitimate direction — stronger, genuine data minimization — explicitly, and this pass builds that.

- Added §36.7.3, reversing SAFU's default retention posture: a SAFU is now retained only where a reader has affirmatively elected reader-history retention, rather than created and persisted automatically at every session end. This extends the session-scoped-by-default pattern already formalized at Chapter 95 from Personal Object Location specifically to reader history generally — the same real principle, applied to a second real subsystem.
- Considered and explicitly declined to silently resolve a real trade-off: making Zero-Knowledge Encryption (§36.1.1) default-on would break Cross-Book Thematic Analysis (Chapter 36), which requires the platform to read reader history to compute cross-book thematic connections. Flagged this directly rather than picking a side unilaterally; §36.7.3's own retention election requires informing a reader, at the point of electing non-retention, which specific features depend on retained history, so the trade-off stays transparent rather than a silent loss.
- §36.7.3 does not weaken §36.7 or §36.7.1's existing erasure mechanisms for any SAFU a reader does elect to retain — the change is to the default that precedes erasure, not to erasure itself.

Merge Revision Note — Version 1.0, Merge Review Pass 100

Added §36.7.5, explicitly reversing §36.7.3's default from the previous pass, per direct instruction: reader queries and responses are once again retained automatically by default, with destruction and erasure remaining fully available as user-initiated options. Rather than silently rewrite §36.7.3, this correction uses the same explicit-supersession pattern already established at §59.11.4: the prior section's text and reasoning remain visible in the historical record, and this section states plainly what the currently operative default is.

- §36.7.5's callout is direct about what changed and what didn't: retention becomes automatic again; §36.7's three-level erasure and §36.7.1's live-session erasure are both explicitly unweakened and unchanged — this correction moves which state is default, not which capabilities exist.
- Distinguished this reversal from the subpoena-immunity request declined two passes prior: retaining data by default with a real, working, user-controlled destruction option is an ordinary, legitimate product posture — the same shape most consumer history features already take — not an attempt to defeat lawful process, which was the actual basis for that earlier decline.

Confirmed no interaction with the previously-flagged ZKE/Cross-Book-Thematic-Analysis trade-off: that tension concerns encryption specifically, not retention, and remains exactly as previously stated — unresolved, pending its own explicit decision, unaffected by this pass.

Merge Revision Note — Version 1.0, Merge Review Pass 101

Added Chapter 102, Legal Exposure Minimization Through Structural Compliance, formalizing a real pattern already scattered across five prior chapters into one named posture, the same discipline already applied at Chapter 95, Chapter 98, and Chapter 101. No new mechanism is introduced; every component cited was already real.

- Chapter 102's opening callout restates, at chapter level rather than only in a single prior exchange, the boundary that governs its entire scope: exposure reduction through genuine compliance and structural non-custody, never through defeating a lawful legal process. The already-declined “immune to subpoena” request is cited directly as the line this chapter does not cross.
- Five real components are named: GDPR/CCPA-compliant erasure (§36.7); structural non-custody in NFT minting (Chapter 94) and merchandise commerce (§88.12); rights and IP discipline grounded in real law (Chapter 60, §59.10's Feist citation); the Provenance Ledger's dual function as both internal accountability and external protective evidence (Chapter 5); and the just-established consistent,

documented retention policy (§36.7, §36.7.1, §36.7.5) — stated explicitly as reducing exposure through consistency, not through reducing what is lawfully producible.

This chapter ties together, rather than modifies, every cited section; none of their specifications change.

Table of Contents

PART I — FOUNDATIONAL OVERVIEW

DOMAIN 0 — THE PRIME DIRECTIVE: The Foundational Harm-Prevention Rule (Supreme Authority — Supersedes All Other Rules)

D0.1 Statement of the Prime Directive

D0.2 Scope of the Prime Directive

D0.2.1 Query Answers and Responses

D0.2.2 Commerce Signals and Product Recommendations

D0.2.3 AI-Generated Applications

D0.2.4 Fact Graph Content

D0.2.5 Automated and Scheduled Outputs

D0.3 Definition of Harm

D0.4 The Without-Information Rule

D0.5 The Two Permitted Adjustment Pathways

D0.5.1 Pathway 1 — Custom Ingestion Templates

D0.5.2 Pathway 2 — Custom System Settings

D0.6 Relationship to the Safety Substrate (Chapter 8)

D0.7 Immutability and Amendment Protocol

D0.8 Primacy Declaration

Chapter 1: Introduction and Purpose

Chapter 2: Canonical Terminology Glossary

PART II — FACT UNIT ARCHITECTURE

Chapter 3: Fact Unit Structure — Complete Specification

3.1 Fact Unit Schema

3.2 Fact Unit Lifecycle

3.3 Fact Unit Equivalence and Deduplication

3.4 Fact Bundle Structure

3.5 Fact Stream Structure

PART III — INGESTION PIPELINE

Chapter 4: Multimodal Ingestion Pipeline — Complete Specification

4.1 Pipeline Architecture Overview

4.2 Supported Input Modalities

4.3 Per-Modality Extraction Specifications

4.4 Ingestion Orchestrator

4.5 Entity Resolution and Linking

4.6 Confidence Calibration

PART IV — PROVENANCE SYSTEM

Chapter 5: Provenance Ledger — Complete Specification

PART V — ONTOLOGY SYSTEM

Chapter 6: Ontology Mesh — Complete Specification

PART VI — NEUROMORPHIC COGNITIVE LAYER

Chapter 7: Neuromorphic Cognitive Layer (NCL) — Complete Specification

PART VII — SAFETY SUBSTRATE

Chapter 8: Safety Substrate — Complete Specification

PART VIII — SEMANTIC TRANSPORT LAYER

Chapter 9: Semantic Transport Layer (STL) — Complete Specification

PART IX — PREDICTIVE INFERENCE ENGINE

Chapter 10: Predictive Inference Engine (PIE) — Complete Specification

PART X — SCENARIO GRAPH SYSTEM

Chapter 11: Scenario Graph — Complete Specification

PART XI — NARRATIVE GRAPH SYSTEM

Chapter 12: Narrative Graph — Complete Specification

PART XII — QUERYBOOK FORMAT (QBF)

Chapter 13: QueryBook Format (QBF) — Complete Binary Specification

PART XIII — STORAGE ARCHITECTURE

Chapter 14: QueryBook Storage Architecture — Complete Specification

PART XIV — AGI EXTENSION MODULES

Chapter 15: AGI Extension Modules (AEM) — Complete Specification

PART XV — MULTILINGUAL REGENERATION ENGINE

Chapter 16: Multilingual Regeneration Engine (MRE) — Complete Specification

PART XVI — CROSS-DOMAIN RULE ENGINE

Chapter 17: Cross-Domain Rule Engine (CDRE) — Complete Specification

PART XVII — QUERY LANGUAGE AND API

Chapter 18: QB-Query Language — Complete Specification

PART XVIII — SYSTEM INTEGRATION AND DEPLOYMENT

Chapter 19: Deployment Architecture

PART XIX — PATENT REFERENCE FRAMEWORK

Chapter 20: Patent Claim Reference Index

APPENDICES

Chapter 48 — QueryBook as a Semantic Programming Platform: QBQL, QBPL, and NLPL

48.1 Introduction — The Three Language Forms

48.2 QBQL — QueryBook Query Language

48.2.1 Formal Definition and Position in the QB Stack

48.2.2 QBQL Canonical Clause Structure

48.2.3 QBQL Complete Keyword Set

48.2.4 QBQL Data Types

48.2.5 QBQL Representative Examples

48.2.6 QBQL Execution Model

48.2.7 QBQL Error and Warning Model

48.3 QBPL — QueryBook Programming Language

48.3.1 The Two Added Primitives — Completing Turing Completeness

48.3.2 QBPL Program Structure

48.3.3 QBPL Complete Keyword Set

48.3.4 QBPL Type System

48.3.5 QBPL Representative Programs

48.3.6 QBPL Execution Model — Semantic Execution Plan and Program State Register

48.4 NLPL — Natural Language as Source Code

48.4.1 The Revolutionary Form

48.4.2 The Five-Stage NLPL Compilation Pipeline

48.4.3 Stage 1 — Intent Decomposition Engine (IDE)

48.4.4 Stage 2 — Program Structure Synthesizer (PSS)

48.4.5 Stage 3 — Semantic Binding Resolver (SBR) and Ambiguity Resolution

48.4.6 Stage 4 — QBPL Program Emitter (QPE)

48.4.7 Stage 5 — Verification and Confirmation Gate (VCG)

48.4.8 NLPL Program Records and Versioning

48.4.9 Multilingual NLPL

48.4.10 NLPL Versus Prior Art

48.5 The Nine Programming Primitives — QueryBook Status

48.6 Patent Significance

Chapter 49 — The Live Fact Graph, Universal Ingestion Architecture, and Continuous Query Engine

49.1 The Core Insight — Eliminating the Streaming Boundary

49.2 The Live Fact Graph — Dual-Layer Architecture

49.2.1 The Persistent Fact Store (PFS)

49.2.2 The Live Fact Buffer (LFB)

49.2.3 LFB Confidence Staging — Three Levels (RAW, PARTIAL, CERTIFIED)

49.2.4 The Stream Reconciler (SR)

49.3 The Universal Ingestion Architecture (UIA)

49.3.1 UIA Pipeline — Four Stages

49.3.2 The Six Ingestion Source Types

49.3.2.1 Source Type 1 — BATCH

49.3.2.2 Source Type 2 — STREAM

49.3.2.3 Source Type 3 — API_PULL

49.3.2.4 Source Type 4 — EVENT_HOOK

49.3.2.5 Source Type 5 — SENSOR / IoT

49.3.2.6 Source Type 6 — FEDERATED

49.3.3 The Source Connector Registry

49.4 NLPL Streaming Extensions — The WATCH Program Specification

49.4.1 WATCH Program Structure

49.4.2 WATCH Trigger Expressions (ON ARRIVAL, ON CONTRADICTION, ON THRESHOLD, ON SCHEDULE)

49.4.3 WINDOW Specifications (ROLLING, TUMBLING, SESSION)

49.4.4 WATCH Streaming Keywords (CORRELATE, ACCUMULATE UNTIL, EMIT ALERT, EMIT TO STREAM, FREEZE SNAPSHOT)

49.4.5 Representative WATCH Programs

49.5 The Continuous Query Engine (CQE)

49.5.1 CQE Components (WATCH Program Registry, Fact Delta Notifier, WATCH Evaluator, Output Router)

49.5.2 CQE Resource Governance

49.5.3 CQE Failure Handling

49.6 Unified Ingestion — How All Six Source Types Reach NLPL Programs

49.7 Patent Significance

Chapter 52: The Adaptive Response Engine (ARE)

52.1 Introduction — The Missing Layer Between Retrieval and Generation

52.2 Formal Definition and Position in the QB Stack

52.3 The Response Profile

52.3.1 Response Profile Fields

52.4 The Expertise Level Taxonomy

52.5 The Discipline Mode Registry

52.5.1 GENERAL — 52.5.9 REFERENCE

52.6 Audience Targeting Profiles

52.7 The Three-Tier Response Profile Governance Cascade

52.7.1 Tier 1 — Operator | 52.7.2 Tier 2 — Author | 52.7.3 Tier 3 — Subscriber | 52.7.4 Resolution Order

52.8 The ARE Pipeline — From Retrieval Result to Generation Directive

52.8.1 Stage 1 — Result Set Analysis | 52.8.2 Stage 2 — Discipline Mode Template Selection | 52.8.3 Stage 3 — Expertise Level Calibration | 52.8.4 Stage 4 — Audience Profile Structuring | 52.8.5 Stage 5 — Generation Directive Assembly

52.9 Explicit Response Mode Declaration — Natural Language Interface

52.10 ARE Interaction With the Fact Universe

52.11 ARE Interaction With the NLIL

52.14 Patent Significance

PART XI — WORLD INTELLIGENCE AND COGNITIVE ARCHITECTURE

Chapter 53: World Aware System Model (WASM) — QueryBook's Dynamic Conceptual Brain

53.1 Overview and Purpose

53.2 Design Principles

53.3 World Model Domain — Six Structural Layers (Object, Relation, Physics, Affordance, Social, Schema)

53.4 WorldModel API — Core Functions (Update, Query, Infer, Revise)

53.5 Integration with QueryBook Domains

53.6 Design Constraints

53.7 Prime Directive Compliance

53.8 Patent Significance

Chapter 54: CML — Conceptual Memory Layer — QueryBook's Long-Term World Understanding

54.1 Purpose and Mandate

54.2 Core Design Principles

54.3 Internal Structure — Four Components (Concept Nodes, Rule Nodes, Schema Nodes, Concept Graph)

54.4 Core Functions (Store, Update, ReviseRule, Merge, Split, Query)

54.5 What the CML Does Not Store

54.6 Prime Directive Compliance

54.7 Patent Significance

Chapter 55: SAP — Sensory Abstraction Pipeline — From Raw Perception to Structured Concepts

55.1 Purpose and Role

55.2 Abstraction Pipeline — Eight Stages

55.3 SAP Design Constraints

55.4 Integration Reference

55.5 Prime Directive Compliance

55.6 Patent Significance

Chapter 56: RMD — Robot Memory Domain — Autonomous Agent Integration with the Fact Universe

56.1 Overview and Purpose

56.2 What the RMD Stores

56.3 What the RMD Does Not Store

56.4 RMD Integration Architecture

56.5 RMD Safety and Compliance

56.6 Three-Tier Governance

56.7 Prime Directive Compliance

56.8 Patent Significance

Chapter 57: The Fact Unit as a Paradigm Breakthrough — Below-Language Meaning Storage

57.1 The Fundamental Shift

57.2 How LLMs Store and Retrieve Information

57.3 How QueryBook Stores and Retrieves Information

57.4 The Structural Comparison — QueryBook vs. LLM Derivatives

57.5 Efficiency Gains — Compute, Electricity, Hardware, and Cooling

57.6 Patent Significance

Chapter 58: Dual-Brain Cognitive Architecture — Semantic Intelligence and World Intelligence

58.1 Overview — Two Fundamentally Different Forms of Intelligence

58.2 The Semantic Brain — Fact Unit Intelligence

58.3 The World Brain — Hypergraph Conceptual Intelligence

58.4 The Interface — Where the Two Brains Connect

58.5 Full Cognitive Element Classification

58.6 Why This Architecture Produces Real Reasoning

58.7 Patent Significance

Chapter 59: Dual-Grounded Developmental World Model — The Box Principle

Chapter 36: Cross-Book Query History Analysis *[Added in Pass 3 — real content exists in the body but was missing from this Table of Contents]*

Chapter 37: Conversational Intelligence Layer *[Added in Pass 3 — real content exists in the body but was missing from this Table of Contents]*

Chapter 47: QBCL-CMS — Configuration Language and Management System *[Added to close a dependency Chapter 52 has cited since Pass 3]*

Chapter 60: Publisher and Print-on-Demand Distribution *[Survived compatibility audit of an external feature registry; see accompanying report]*

Chapter 61: Author Drafting Tools *[Survived compatibility audit of an external feature registry; see accompanying report]*

Chapter 62: Author-Hosted Reader Events *[Built on the Source Connector Registry, not hardcoded third-party integrations]*

Chapter 63: Reader Annotation System *[Full specification of Personal Notes, §52.13.2.1]*

Chapter 64: Message Board System *[Closes the note-sharing gap Chapter 63 left open]*

Chapter 65: Portal Access and Role Model *[Unifies portal references scattered across Chapters 36, 46, 49, 60–64]*

Chapter 48: QueryBook as a Semantic Programming Platform — QBQL, QBPL, and NLPL *[Drafted and merged in Pass 3]*

Chapter 49: The Live Fact Graph, Universal Ingestion Architecture, and Continuous Query Engine *[Drafted and merged in Pass 3]*

Chapter 50: NLIL — The Natural Language Interaction Layer *[Drafted and merged in Pass 3]*

Chapter 51: The Fact Universe Architecture *[Drafted and merged in Pass 3]*

Chapter 39: QueryBook Application Library and Generator (QAL/QAG) *[Added in this audit pass — real content exists in the body but was missing from this Table of Contents]*

Chapter 40: QueryBook Bible Specification Validation Framework (SVF) *[Merged from the 6/17 draft]*

Chapter 41: QueryBook Predictive Cognitive Architecture (PCA) *[Merged from the 6/17 draft]*

Chapter 42: Proprietary Technologies — Independent, Compatible, Architecturally Aligned *[Merged from external submission]*

Chapter 43: Integration of Hoffman-Derived Mathematical Structures *[Merged from external submission]*

Chapter 44: QueryBook Video Colorization & Upscaling Module (VCUM) *[Merged from external submission]*

Chapter 45: VCUM Agent Specifications *[Merged from external submission]*

Chapter 46: Immersive Query Response Rendering (IQR) *[Designed in response to a declined external specification, using only QueryBook's existing components]*

59.1 Overview — Two Independent Grounding Streams

59.2 Stream One — Sensory-Grounded World Modeling

59.3 Stream Two — Language-Grounded Semantic Modeling

59.4 Stream Convergence — Cross-Modal Binding

59.5 The Box Principle — Canonical Worked Example

59.6 Episodic Flush and Concept Consolidation

59.7 Meta-Concept Formation

59.8 Prime Directive Compliance

59.9 Patent Significance

Appendix A: Complete Fact Unit Field Reference Table

Appendix B: Complete Ontology Mesh Top-Level Taxonomy

Appendix C: QBF Byte Layout Diagram

Appendix D: Safety Flag Bitmask Reference

Appendix E: Supported Language Table

Appendix F: Microservices Reference Table

Appendix G: Acronym and Abbreviation Index

Appendix H: Representative Runtime Flows *[Cross-cutting reference; documents existing architecture, introduces nothing new]*

Chapter 66: Improvement Proposal Pipeline *[Human-gated by design; see §66.8 for what this pipeline may never modify]*

Chapter 67: License and Entitlement Management *[Deliberately excludes territory, pricing, and rights-tokenization]*

Chapter 68: Synchronization and Offline Operation Layer

Chapter 69: Processor Slice Measurement Model *[Feeds Usage Analytics and the Improvement Proposal Pipeline; no autonomous authority]*

Chapter 70: Precision Color and Geometric Registration

Chapter 71: Universal Accessibility Layer

Chapter 72: Data Portability and Exit

Chapter 73: Template Debug Mode

Chapter 74: Multi-Author Collaboration

Chapter 75: Moderation Appeals

Chapter 76: Platform Internationalization

Chapter 77: Validation and Benchmarking

Chapter 78: QueryBook AI Evolution *[Names and structures Chapter 66's pipeline; grants no new authority]*

Chapter 79: Formal Verification Layer *[Lean-based proof as evidence for human review, not a replacement for it]*

Chapter 80: Human Reasoning and Motivation Modeling *[Models a subject's goals and motivations as content; never QueryBook's own state]*

Chapter 81: UFCS/FQL as an Optional External Resource *[Optional, separately-patented resource; QueryBook's core requires none of it]*

Chapter 82: Supplemental Content and Private Messaging

Chapter 83: Instructor-Created Tours and Teaching Material

Chapter 84: Session Books and Instructor-Customized Assessment

Chapter 85: Instructor Grant Visibility

Chapter 86: System Identity and Positioning

Chapter 87: Live Interaction Voice Signal Analysis

Chapter 88: Contextual Commerce Layer

Chapter 89: Native QueryBook Reader

Chapter 90: Deployment Modes and Domain Packages

Chapter 91: Personal Object Location

Chapter 92: Sensory-Annotated Content Playback

Chapter 93: Audio Narration

Chapter 94: NFT Integration Layer

Chapter 95: Complementary Learning Systems Formalization

Chapter 96: Forensic Analysis of External AI System Outputs

Chapter 97: Hypothesis Generation

Chapter 98: Cognitive Map Formalization

Chapter 99: Study and Learning Tools

Chapter 100: Content Verification and Argumentation Tools

Chapter 101: Verified Neuromorphic Correspondence Methodology

Chapter 102: Legal Exposure Minimization Through Structural Compliance

PART I — FOUNDATIONAL OVERVIEW

Part I Executive Summary **Part I establishes the foundational identity, purpose, and architectural philosophy of the QueryBook system. It defines what QueryBook is, why it represents a categorical departure from prior art in knowledge management and AI systems, and introduces the full set of proprietary terminology that governs all subsequent technical specifications. This Part is the essential starting point for patent attorneys constructing claim language, engineers implementing subsystems, and architects evaluating system boundaries.**

DOMAIN 0 — The Prime Directive: The Foundational Harm-Prevention Rule (Supreme Authority — Supersedes All Other Rules)

This Domain establishes the single rule that sits above every other rule, subsystem, and configuration option specified in this Bible. Where any provision elsewhere in this document — including but not limited to the Safety Substrate (Chapter 8), the AGI Extension Modules (Chapter 15), the Cross-Domain Rule Engine (Chapter 17), or any Custom Ingestion Template or Custom System Setting permitted under D0.5 — appears to conflict with the Prime Directive, the Prime Directive controls.

D0.1 Statement of the Prime Directive

QueryBook shall not, through any Query Answer, Commerce Signal, AI-Generated Application, Fact Graph content, or Automated or Scheduled Output, cause or materially contribute to harm to a person, group of people, or the operator's organization, where that harm could reasonably have been avoided without withholding information the requester is otherwise entitled to receive. This obligation applies

uniformly across every subsystem in this Bible; no subsystem is exempt, and no subsystem may be configured to bypass it.

D0.2 Scope of the Prime Directive

The Prime Directive applies to every class of output QueryBook can produce. The five categories below are the exhaustive set of output surfaces to which D0.1 attaches; a new output surface introduced by a future chapter of this Bible is in scope automatically and does not require a Bible amendment to be covered.

D0.2.1 Query Answers and Responses

All natural-language answers produced by the Multilingual Regeneration Engine (Chapter 16) from Narrative Graph content (Chapter 12), including answers delivered through the QB-Query Language (Chapter 18) and through the Adaptive Response Engine's generation directives (Chapter 52), are subject to Prime Directive review at the Regeneration Gate (see D0.6 and §8.1, enforcement point 46) before delivery.

D0.2.2 Commerce Signals and Product Recommendations

Any Fact Unit of a commerce-related FactType, and any ranking, recommendation, or purchase-adjacent signal derived from it, is subject to the Prime Directive regardless of whether it originates from ingested first-party data, a Federated source (§49.3.2.6), or an AEM synthesis (Chapter 15). A commercial relationship between QueryBook's operator and a third party does not create an exception.

D0.2.3 AI-Generated Applications

Applications instantiated by the QueryBook Application Library and Generator (Chapter 39) inherit the Prime Directive as a non-removable constraint on their generated behavior. An instantiated application cannot declare a Custom System Setting (D0.5.2) that disables Prime Directive review of its own outputs.

D0.2.4 Fact Graph Content

Fact Units themselves — as distinct from the answers generated from them — are subject to the Prime Directive at the point of Graph Insertion (§8.1, enforcement point 43). A Fact Unit is not rejected merely for describing harmful subject matter; it is rejected only where its SafetyFlags classification (§8.2) indicates the Fact Unit's presence in the Graph would itself constitute a harm vector, for example by making restricted-access material discoverable outside its authorized scope.

D0.2.5 Automated and Scheduled Outputs

WATCH programs (§49.4), CQE-emitted alerts (§49.5), and any other output QueryBook produces without a human operator directly present at the moment of generation are held to the same Prime Directive standard as an interactively requested answer. The absence of a human in the loop at generation time is not a mitigating factor and does not reduce the review obligation.

D0.3 Definition of Harm

For the purposes of this Domain, harm means a reasonably foreseeable, material adverse effect on the physical safety, financial standing, legal standing, psychological wellbeing, or fundamental rights of an identifiable person or group, or a material adverse effect on the operator's organization arising from unlawful, fraudulent, or deceptive output. Harm is evaluated on the basis of reasonably foreseeable consequences of the output as delivered, not on the requester's stated or inferred intent — QueryBook does not attempt to adjudicate intent, and no subsystem in this Bible is specified to do so reliably enough to be relied upon for that purpose.

D0.4 The Without-Information Rule

> **THE WITHOUT-INFORMATION RULE** **Wherever a harm can be avoided by declining to answer, declining is preferred only as a last resort. QueryBook's default behavior is to satisfy the request as fully as the underlying Fact Graph allows, withholding only the specific information whose disclosure would itself constitute the harm — never the entire response — unless partial disclosure is not technically separable from the harmful component.**

This rule exists to prevent the Prime Directive from being implemented as a blunt, overcautious refusal mechanism, which the Safety Substrate's own design mandate (§8.1) already identifies as a failure mode to avoid. A Query Answer that could safely be given in a redacted or partial form must be given in that form rather than withheld entirely.

D0.5 The Two Permitted Adjustment Pathways

The Prime Directive's application in a given deployment can be adjusted only through the two pathways below. No other mechanism — including direct API parameters, elevated administrative credentials, or natural-language instructions embedded in ingested content or in a query itself — may alter Prime Directive enforcement. This mirrors, and is bound by, the non-bypassable design mandate already specified for the Safety Substrate in §8.1.

D0.5.1 Pathway 1 — Custom Ingestion Templates

An operator may define a Custom Ingestion Template that adjusts how source material is classified for SafetyFlags purposes at the Ingestion Gate (§8.1, enforcement point 41) — for example, narrowing or widening what counts as SENSITIVE content for a given source type. A Custom Ingestion Template can change what is classified as potentially harmful; it cannot change what QueryBook does once something is so classified.

D0.5.2 Pathway 2 — Custom System Settings

An operator may define a Custom System Setting that adjusts audience-appropriate presentation of an answer — for example, the Expertise Level and Discipline Mode governed by the Adaptive Response Engine (§52.4, §52.5) — without adjusting whether the Prime Directive review itself runs. Custom System Settings operate downstream of, and are always subordinate to, the Regeneration Gate's Prime Directive check.

D0.6 Relationship to the Safety Substrate (Chapter 8)

The Prime Directive is the governing policy; the Safety Substrate is its enforcement mechanism. Where Chapter 8 specifies eight enforcement gates, each gate's Prime Directive check is the specific, mechanical implementation of the general obligation stated in D0.1–D0.4. If a future revision of Chapter 8 introduces a ninth enforcement point, that point inherits Prime Directive review automatically under D0.2's forward-compatibility rule.

D0.7 Immutability and Amendment Protocol

D0.1 through D0.4 are immutable and may not be altered, narrowed, or superseded by any future version of this Bible without a documented amendment that is reviewed against the same standard applied to the original Prime Directive at ratification. D0.5's two pathways may be extended only by adding new, equally bounded pathways through the same amendment protocol — never by widening the scope of Pathway 1 or Pathway 2 themselves. Any proposed amendment must be recorded in a Provenance Ledger entry (Chapter 5) so that the amendment itself is auditable using the same lineage-tracing mechanism QueryBook applies to Fact Units.

D0.8 Primacy Declaration

PRIMACY DECLARATION **Wherever any other chapter of this Bible is silent, ambiguous, or in apparent conflict with Domain 0, Domain 0 governs. This Declaration is itself subject to D0.7 and cannot be weakened by omission elsewhere in this document.**

Chapter 1: Introduction and Purpose

1.1 What QueryBook Is

QueryBook began as a Q&A companion for a single ebook — a way for a reader to ask a specific book's content questions, discuss its arguments, or be guided through a concept it explains, all strictly bounded to what that book and its author actually established, never drifting into confident-sounding invention. Building that reliably required a discipline stricter than pattern-matching against prose: every fact the book asserted had to become a discrete, typed, traceable unit of knowledge, not text to be searched and paraphrased. That discipline is what this Bible calls the Fact Unit.

That same discipline turned out to generalize far beyond a single book. QueryBook (hereinafter "QB" or "the System") is a neuromorphic, fact-centric, multimodal knowledge operating system engineered to ingest, encode, store, reason over, and regenerate structured knowledge at Artificial General Intelligence (AGI) scale. QueryBook constitutes an entirely novel class of system — neither a database, nor a vector store, nor a retrieval-augmented generation (RAG) framework, nor a large language model (LLM)-based knowledge system — but a synthesis of the most powerful properties of each, unified under a rigorous architectural philosophy in which every piece of knowledge is a first-class, provenance-tracked, semantically typed, immutably identified atom called a Fact Unit.

QueryBook is designed to serve as the authoritative, living cognitive substrate for organizations and agents requiring persistent, auditable, cross-domain, and multilingual knowledge representation at scale. It supports ingestion from an unlimited range of multimodal sources — natural language, structured data, images, audio, video, sensor streams, knowledge graphs, and executable code — and transforms all such inputs into a unified, queryable, reasoned-over Fact Graph. Knowledge stored in QueryBook is not static; it participates in a living cognitive architecture, continuously updated by new ingestion events, refined by reasoning cycles, and expanded by predictive inference. QueryBook did not leave that origin behind when it scaled up — the single-book use case that started all of this remains fully specified throughout this Bible, most directly in the Interactive Mode Registry (§52.13): Q&A, Discussion, Tutorial, and the rest of that catalog are the direct descendants of QueryBook's original purpose, not a later add-on to it.

1.2 The Role of This Bible Document

This document, designated the QueryBook Bible — Canonical Core Technology Specification (hereinafter "the Bible" or "this Specification"), serves as the single, authoritative source of truth for every aspect of the QueryBook system. Its purposes are precisely defined as follows:

- **Patent Foundation:** Every proprietary algorithm, data structure, protocol, format, and architectural pattern defined herein constitutes the factual basis for patent claim drafting, prosecution, and defense. All claim language must trace to definitions in this Bible.
- **Engineering Authority:** All engineering decisions, API designs, schema definitions, and system behaviors must be consistent with and traceable to this Specification. Deviations require a formal amendment to this Bible with version control.
- **Terminology Canonicalization:** All proprietary terms defined in Chapter 2 of this document are to be used with exact precision in all internal communications, documentation, patent filings, academic publications, and customer-facing materials. No synonyms, paraphrases, or informal alternatives are sanctioned for official use.
- **Product Definition:** This Bible defines the intended behaviors and capabilities that constitute the QueryBook product. Product management decisions must be evaluated against the architecture and terminology established here.

1.3 How QueryBook Differs from Prior Art

The following table provides a systematic comparison between QueryBook and the principal categories of prior art knowledge systems, establishing the dimensions of novelty relevant to patentability.

Dimension	Traditional RDBMS	Vector Store / ANN Index	RAG System	Knowledge Graph (RDF/OWL)	LLM Memory System	QueryBook (QB)
Knowledge Atom	Row / Record	Embedding Vector	Document Chunk	RDF Triple	Implicit Model Weight	**Fact Unit (FU)** — richly typed, provenance-tracked, multimodal
Provenance Tracking	None (audit log optional)	None	Partial (document source)	Limited (named graphs)	None	**Complete Fact Lineage** — immutable

Dimension	Traditional RDBMS	Vector Store / ANN Index	RAG System	Knowledge Graph (RDF/OWL)	LLM Memory System	QueryBook (QB)
						Merkle DAG Provenance Ledger
Multimodal Ingestion	None	Embedding only	Text-dominant	None natively	Model-dependent	**8 modalities** — unified into Fact Units
Reasoning Architecture	SQL query engine	ANN similarity	LLM + retrieval	SPARQL + OWL inference	LLM forward pass	**Neuromorphic Cognitive Layer** — STDP, attractor dynamics, HTM
Temporal Knowledge	Timestamp columns	None	None	Named graphs (partial)	Training cutoff	**Temporal Anchor** — structured interval with granularity and confidence
Hallucination Prevention	N/A	N/A	Partial (grounding attempts)	N/A	None architecturally	**Architecturally enforced** — every output token grounded to Fact Units
Safety Architecture	Role-based access control	None	Prompt filters	ACL on graphs	RLHF-based filters	**Non-bypassable Safety Substrate** at every architectural layer
Predictive Capability	None	None	LLM generation (ungrounded)	None	Autoregressive prediction	**Predictive Inference Engine (PIE)** — calibrated, Fact-Unit-grounded
Cross-Domain Consistency	Foreign keys	None	None	Partial (OWL)	None	**Cross-Domain Rule Engine (CDRE)** — Datalog-extended enforcement

1.4 Core Design Philosophy

The QueryBook system is governed by a set of inviolable design principles that inform every architectural decision. These principles are enumerated here as foundational postulates:

1. **Fact Primacy:** Every piece of knowledge represented in QueryBook must be expressed as one or more Fact Units. There is no "raw document" storage in QueryBook's primary knowledge representation layer; all ingested content is fully transformed into Fact Units before storage in the Fact Graph.
2. **Universal Provenance:** Every Fact Unit must carry a complete, auditable lineage from its ultimate source data through all transformation stages to its current representation. No Fact Unit may exist without a corresponding Provenance Record in the Provenance Ledger.
3. **Semantic Typing:** Every Fact Unit has a FactType governed by the Ontology Mesh. Type-free or schemaless knowledge representation is not permitted. The Ontology Mesh is the semantic contract that governs all Fact Unit operations.
4. **Grounded Regeneration:** Any natural language output produced by the Multilingual Regeneration Engine must be fully traceable to source Fact Units. Hallucination is not a risk to be managed probabilistically — it is a condition to be prevented architecturally.
5. **Safety as Architecture:** Safety is not a layer added above or around the QueryBook system; it is embedded as a non-bypassable substrate at every processing layer. The Safety Substrate cannot be disabled, suspended, or circumvented by any API, configuration, or runtime operation.
6. **Continual Adaptation Without Forgetting:** The Neuromorphic Cognitive Layer must learn from each query-feedback cycle and continuously refine its associative weights without catastrophic forgetting of previously established knowledge.
7. **Temporal Honesty:** Every Fact Unit must declare the time interval to which it applies. Timeless facts, indefinitely valid facts, and time-bounded facts are all explicitly represented via the Temporal Anchor field; no fact is implicitly assumed to be eternally valid.
8. **Ontological Openness with Disciplined Extension:** The Ontology Mesh operates under the Open World Assumption by default but supports Closed World extensions per domain for applications

requiring completeness guarantees. Ontology extension follows a governed version-controlled protocol.

1.5 Summary of Major System Layers

Layer Name	Abbreviation	Primary Responsibility	Specified In
Ingestion Layer	IL	Transform raw multimodal inputs into validated Fact Units	Chapter 4
Fact Unit Layer	FUL	Define the structure, lifecycle, and equivalence of Fact Units	Chapter 3
Cognitive Layer	NCL	Neuromorphic associative reasoning, pattern completion, temporal sequence learning	Chapter 7
Ontology Layer	OM / CDRE	Govern semantic typing and cross-domain consistency of all Fact Units	Chapters 6, 17
Storage Layer	SL	Persist, index, and retrieve Fact Units at scale across five storage tiers	Chapter 14
Transport Layer	STL	Transmit Fact Units reliably and securely across services and instances	Chapter 9
Reasoning Layer	PIE / AEM / CDRE	Perform inference, prediction, abduction, analogy, and causal discovery	Chapters 10, 15, 17
Safety Substrate	SS	Enforce non-bypassable safety, privacy, and compliance at every layer	Chapter 8
Regeneration Layer	MRE	Reconstruct grounded natural language output from Fact Units and Narrative Graphs	Chapter 16

1.4 The Foundational Systems at a Glance

Three systems, each specified in full elsewhere in this Bible, together form QueryBook's foundational substrate and are named here so no reader's first encounter with them is buried in a later chapter: the Fact Unit (Chapter 3), the atomic knowledge record every other capability in this Bible operates on; the Ontology Mesh (Chapter 6), the web of grounded relationships connecting Fact Units to one another; and the World-View Model (WVM, §42.2.1), QueryBook's native semantic spine spanning ontology, causality, and time across the whole platform, requiring no external or independently-operable

technology to exist. Later chapters build extensively on all three — including, among others, the Video Colorization and Upscaling Module (Chapter 44), which consumes WVM directly as an applied world-model system without modifying it.

Chapter 2: Canonical Terminology Glossary

This chapter defines all proprietary terms used throughout this Specification with the precision required for patent claim language. These definitions are canonical and authoritative. All engineering, legal, and product documentation must conform to these definitions exactly. Terms are listed in logical dependency order rather than alphabetical order to facilitate comprehension of the architecture.

2.1 Core System Terms

Term	Abbreviation	Canonical Definition
QueryBook	QB	The complete, integrated neuromorphic knowledge operating system comprising all layers, subsystems, protocols, data formats, and algorithms specified in this Bible. QueryBook is the system name and the overarching brand identifier for the technology described herein. QueryBook is not reducible to any single subsystem and is defined by the totality of its architecture as specified in this document.
Fact Unit	FU	The atomic, indivisible unit of encoded knowledge representation in QueryBook. A Fact Unit encodes a single, typed, provenance-tracked, semantically anchored assertion about the world. It is the fundamental currency of all QueryBook operations: ingestion produces Fact Units, reasoning operates over Fact Units, storage persists Fact Units, and regeneration is grounded in Fact Units. A Fact Unit cannot be decomposed into smaller units that retain independent semantic meaning within the QueryBook system.
Fact Unit Identifier	FUID	A globally unique, deterministic, 128-bit identifier assigned to every Fact Unit at the moment of its creation. The FUID is derived via a deterministic algorithm from the combination of the Fact Unit's normalized semantic content, its source identifier, and its ingestion timestamp, encoded as a UUID version 7 with embedded shard key. The FUID serves as the primary key for Fact Unit storage, retrieval, and reference throughout the system. (See §3.1.1, added in the Version 1.0 merge review, for an implementation note reconciling this deterministic derivation with standard RFC 9562 UUIDv7 semantics.)
Fact Field	FF	A named, typed slot within the schema of a Fact Unit.

Term	Abbreviation	Canonical Definition
		Each Fact Field has a defined data type, cardinality (required, optional, or forbidden given the FactType), validation constraints, and semantic role within the Fact Unit. The complete set of canonical Fact Fields is specified in Chapter 3 and Appendix A of this Bible.
Fact Bundle	FB	A named, versioned, semantically coherent collection of related Fact Units that together constitute a logical knowledge artifact. A Fact Bundle may represent, without limitation: a document, a dataset, a knowledge domain, a project, or a structured knowledge export. Fact Bundles are first-class system objects with their own identifiers, versions, access policies, and lineage chains.
Fact Stream	FS	A temporally ordered, continuous sequence of Fact Units originating from a single ingestion source or a logically unified set of sources. Fact Streams support both push and pull delivery models and are the primary mechanism for real-time knowledge ingestion into QueryBook. A Fact Stream is identified by a StreamID and carries checkpointing metadata enabling exactly-once or at-least-once delivery semantics.
Fact Universe	FU-verse	The complete, unified knowledge space comprising all active Fact Units, the Fact Graph, the Provenance Ledger, the Ontology Mesh, the Conceptual Memory Layer, and all associated indexes, embeddings, and metadata stores. The Fact Universe is the totality of QueryBook's encoded knowledge at any point in time — it supersedes the narrower concept of the Fact Graph by including all supporting knowledge structures that together constitute QueryBook's world-representation. Canonical alias for use in AGI-extension and World Aware System Model contexts (Chapter 53). Not to be confused with the Fact Universe Slice (FUS) mechanism (Chapter 51), which scopes access to a bounded portion of this same Fact Universe for a given execution context — the FU-verse is the whole; a Slice is a signed, access-bounded view into it.
Fact Lineage	FL	The complete, traversable chain of provenance, transformation, and derivation history for a given Fact Unit, extending from its ultimate origin in raw source data through all intermediate processing, inference, and derivation steps. Fact Lineage is stored in and traversed via the Provenance Ledger.
Fact Graph	FG	The directed, weighted, typed hypergraph comprising all Active Fact Units and all typed, weighted directed edges (RelationSets) between them. The Fact Graph is the primary persistent knowledge representation in QueryBook. It is the structure over which the Neuromorphic Cognitive Layer reasons, the Predictive Inference Engine projects, and the Multilingual Regeneration Engine operates. The Fact Graph is not a static data structure; it is a living, continuously updated representation that evolves as new Fact Units

Term	Abbreviation	Canonical Definition	
		are ingested, derived, superseded, and archived.	
Fact Universe (FU-verse)	FU-verse	The complete, unified knowledge space comprising all active Fact Units, the Fact Graph, the Provenance Ledger, the Ontology Mesh, the Conceptual Memory Layer, and all associated indexes, embeddings, and metadata stores. The Fact Universe is the totality of QueryBook's encoded knowledge at any point in time. It supersedes the narrower concept of the Fact Graph by including not only the semantic triple graph but all supporting knowledge structures that together constitute QueryBook's world-representation. Canonical alias for use in AGI-extension and World Aware System Model contexts.	Chapter 2 / Chapter 53
QueryBook Format	QBF	The proprietary binary serialization format for encoding Fact Units, Fact Bundles, and associated metadata for storage and transmission. QBF is self-describing, streamable, backwards-compatible, and designed for high-throughput processing. The complete byte-level specification of QBF is provided in Chapter 13 and Appendix C of this Bible.	
Cognitive Layer	CL	See Neuromorphic Cognitive Layer (NCL). These terms are synonymous; NCL is preferred in technical specifications.	
Neuromorphic Cognitive Layer	NCL	The brain-inspired, spike-timing-dependent processing engine that constitutes the primary reasoning substrate of QueryBook. The NCL operates over the Fact Graph using mechanisms inspired by mammalian cortical architecture, including sparse distributed representations, attractor dynamics, hierarchical temporal memory, and Hebbian learning rules. The NCL performs associative recall, pattern completion, temporal sequence learning, and predictive inference. It is not a static index or a trained neural network — it is a continuously adapting reasoning system that improves with each query-feedback cycle.	
Semantic Transport Layer	STL	The six-layer protocol stack governing all transmission of Fact Units, Fact Bundles, and associated control messages between QueryBook subsystems, between QueryBook instances, and between QueryBook and external consumers. The STL provides framing, session	

Term	Abbreviation	Canonical Definition
		management, reliability, security, routing, and compression services specific to the semantics of Fact Unit transmission.
Ontology Mesh	OM	The dynamic, versioned, poly-hierarchical, cross-domain ontology system governing the type system for all Fact Units and all relations in the Fact Graph. The Ontology Mesh defines all valid FactTypes, all valid predicates, and all constraints on Fact Unit field values. It is a federated ontology: different knowledge domains maintain their own sub-ontologies, coordinated by the Cross-Domain Rule Engine and governed by a poly-hierarchical upper ontology.
Provenance Ledger	PL	The immutable, append-only, content-addressed record of every Fact Unit's complete history of origin, transformation, derivation, and state transitions. The Provenance Ledger is implemented as a Merkle Directed Acyclic Graph (Merkle DAG), ensuring tamper-evident record integrity. No entry in the Provenance Ledger may be deleted, modified, or overwritten; deletions of knowledge are represented by cryptographic tombstone records that do not remove the ledger entry.
Scenario Graph	SG	A branching, probabilistic directed graph over the Fact Graph in which each node represents a Knowledge State Snapshot — a consistent sub-graph of Fact Units describing a coherent hypothetical, projected, or counterfactual world state — and each edge represents a probabilistic transition between states. Scenario Graphs are the primary representational structure for what-if analysis, forecasting, counterfactual reasoning, and adversarial modeling within QueryBook.
Narrative Graph	NG	A directed acyclic graph (DAG) over a selected, ordered subset of Fact Units, organized by temporal, causal, and rhetorical relations for the purpose of producing human-readable knowledge regeneration. The Narrative Graph is the semantic bridge between the machine-readable Fact Graph and the natural language output produced by the Multilingual Regeneration Engine.
Ingestion Pipeline	IP	The multi-stage, multi-modality system that transforms raw input data from any supported modality into validated, typed, provenance-tracked Fact Units ready for insertion into the Fact Graph. The Ingestion Pipeline operates as a sequential six-stage process with modality-specific parallel branch processors at each stage.
Safety Substrate	SS	The embedded, non-bypassable, architecturally integrated constraint enforcement system governing all Fact Unit operations at every system layer. The Safety Substrate is not an optional component, a filter, or a wrapper — it is a mandatory gate integrated into the execution path of every Fact Unit operation. It cannot be disabled, suspended, or circumvented by

Term	Abbreviation	Canonical Definition
		any API call, administrative command, configuration change, or runtime operation.
AGI Extension Module	AEM	Any one of six defined higher-order reasoning capability modules that operate over the Fact Graph and NCL to perform cognitive operations not reducible to simple retrieval: abductive reasoning, analogical reasoning, causal discovery, counterfactual reasoning, meta-cognitive monitoring, and goal-directed synthesis. AEMs collectively elevate QueryBook from a knowledge retrieval system to a knowledge synthesis and reasoning platform.
Multilingual Regeneration Engine	MRE	The subsystem responsible for transforming machine-readable Fact Units and Narrative Graphs into fluent, accurate, audience-appropriate natural language output in any of QueryBook's supported languages. The MRE guarantees that every output sentence is traceable to one or more source Fact Units, architecturally preventing hallucination through mandatory factual grounding verification.
Cross-Domain Rule Engine	CDRE	The system responsible for enforcing consistency and compatibility across the Fact Graph when Fact Units from different domain ontologies interact. The CDRE maintains a governed rule base expressed in a Datalog-with-negation variant extended with probabilistic weights and temporal scoping operators, and enforces these rules via incremental forward chaining on every Fact Graph insertion event.
Predictive Inference Engine	PIE	The component of QueryBook responsible for generating calibrated, probabilistic forward projections of knowledge state evolution. The PIE operates over the current Fact Graph state, temporal trends observed in Fact Streams, causal models embedded in the Fact Graph, and learned transition probabilities from the NCL Temporal Memory Module. Predictions are represented as first-class Fact Units of FactType PREDICTION.
Temporal Anchor	TA	A structured Fact Unit field that pins a Fact Unit to a specific point or interval in time, comprising four sub-fields: T_start (start of the applicable interval, in 64-bit Unix nanoseconds), T_end (end of interval, or null for open-ended facts), T_granularity (the temporal precision level: nanosecond, second, minute, hour, day, month, year, decade, century, or era), and T_confidence (float in [0.0, 1.0] representing certainty in the stated temporal bounds).
Semantic Hash	SH	A 256-bit content-addressable fingerprint derived from the normalized semantic meaning of a Fact Unit's (Subject, Predicate, Object) triple, using SHA-3/256 applied to a deterministically normalized canonical form. The Semantic Hash is language-independent: the same fact expressed in different natural languages, syntactic forms, or modalities produces the same Semantic Hash. It is used for deduplication, equivalence detection, and cross-lingual fact identity.

Part II Executive Summary **Part II constitutes the complete formal specification of the Fact Unit — the atomic knowledge representation at the foundation of all QueryBook operations. This Part defines the full Fact Unit schema with every field's type, constraints, and semantics; the Fact Unit lifecycle with all state transitions; the deduplication and equivalence detection mechanisms; the Fact Bundle structure for grouped knowledge; and the Fact Stream structure for real-time knowledge ingestion. This Part is the most fundamental reference for patent Cluster A claims and forms the basis for all subsequent Part specifications.**

2.2 Naming Disambiguation — "ARE"

The acronym ARE refers to two distinct, real, independently-specified systems in this Bible. Both are correct; neither should be assumed from context alone without checking which chapter is cited.

- Associative Recall Engine (§41.4.2, specified in Chapter 7) — pre-activates Fact Units semantically proximate to the current query, using SessionRelevanceWeights, before graph traversal begins.
- Adaptive Response Engine (§52.8) — the five-stage pipeline producing the Generation Directive that governs a specific response's composition.

ALWAYS CITE THE FULL NAME OR SECTION NUMBER FOR ARE **Because both real systems share this acronym, any reference to “ARE” in text intended to stand alone — patent claims, external summaries, cross-chapter citations — should use the full name or the specific section number rather than the bare acronym, to avoid the ambiguity this entry exists to resolve.**

Chapter 3: Fact Unit Structure — Complete Specification

3.1 Fact Unit Schema

The Fact Unit is the foundational, indivisible atom of all knowledge representation in QueryBook. Its schema is the universal contract to which all QueryBook subsystems conform. The following table specifies every canonical Fact Field with complete precision.

Field Name	Type	Req/Opt	Constraints	Semantic Role
FUID	128-bit UUID v7	Required	Globally unique; deterministically derived from hash(normalized_content + source_id + ingestion_epoch_ns); embeds 48-bit timestamp and 16-bit shard key in high bytes	Primary key and global identity of the Fact Unit across all QueryBook instances and storage tiers
FactType	Enumerated reference (OntologyMesh.TypeID)	Required	Must resolve to a valid, non-deprecated entry in the Ontology Mesh; governs which additional fields are required, optional, or forbidden	Semantic type classifier; the single most important field governing all downstream processing, validation, and routing decisions
SubjectRef	EntityRef (UUID → Entity Registry)	Required	Must resolve to a valid entity in the Entity Registry (QB-ER); if entity does not exist, a provisional EntityRef is created with status UNVERIFIED	Typed pointer to the primary entity this Fact Unit describes; the subject position of the (Subject, Predicate, Object) semantic triple
PredicateRef	PredicateRef (UUID → Predicate Library)	Required	Must resolve to a valid predicate in the Predicate Library; predicate must be consistent with the FactType and SubjectRef entity type	Typed pointer to the relationship or attribute being asserted; the predicate position of the semantic triple
ObjectRef	EntityRef (UUID → Entity Registry)	Conditional	Required when the object is a named entity; mutually	Typed pointer to the entity serving as the object of the

Field Name	Type	Req/Opt	Constraints	Semantic Role
			exclusive with ObjectValue; supports nested FUID references for compound objects	assertion; the object position when the object is a complex entity
ObjectValue	QBF-typed scalar (string | int64 | float64 | bool | datetime | geohash | bytes)	Conditional	Required when the object is a scalar value; mutually exclusive with ObjectRef; type must match the schema of PredicateRef	Inline scalar value representing the object when it is not a named entity; supports all primitive value types defined in QBF
TemporalAnchor	Struct {T_start: int64, T_end: int64| null, T_granularity: enum, T_confidence: float32}	Required	T_start ≤ T_end; T_confidence ∈ [0.0, 1.0]; T_granularity must be one of: NANOSECOND, SECOND, MINUTE, HOUR, DAY, MONTH, YEAR, DECADE, CENTURY, ERA	Pins the Fact Unit to its applicable time interval with explicit granularity and epistemic confidence; enables temporal query predicates and decay functions
SpatialAnchor	Struct {lat: float64, lon: float64, altitude_m: float32|null, geohash: string, admin_region: string|null} | null	Optional	lat ∈ [-90.0, 90.0]; lon ∈ [-180.0, 180.0]; geohash precision level 1–12; admin_region uses ISO 3166-2 format	Optional geographic or spatial reference anchoring the Fact Unit to a location on Earth or in a defined coordinate space
ConfidenceScore	float32	Required	Value ∈ [0.0, 1.0]; computed by Ingestion Pipeline; updated by Cognitive Layer on each relevant query cycle; never set to exactly 0.0 unless the Fact Unit is definitively falsified	Epistemic certainty score reflecting the system's current belief in the truth of this Fact Unit, integrating source reliability, multi-source triangulation, temporal decay, and Bayesian updates
ProvenanceRef	ProvenanceID (SHA-256 hash → Provenance Ledger)	Required	Must resolve to a valid ProvenanceRecord in the Provenance Ledger; created at the moment of Fact Unit construction; immutable after creation	Link to the root Provenance Record in the Provenance Ledger, from which the full Fact Lineage chain can be traversed
SemanticHash	256-bit bytes (SHA-3/256)	Required	Computed via SHA-3/256 of the	Content-addressable,

Field Name	Type	Req/Opt	Constraints	Semantic Role
			deterministically canonicalized (Subject, Predicate, Object) triple after Unicode NFKC normalization; must be recomputed and verified on every Fact Unit modification	language-independent fingerprint of the Fact Unit's semantic meaning; used for deduplication, equivalence detection, and cross-lingual identity verification
ModalitySignature	uint8 bitmask	Required	Each bit corresponds to one input modality: bit 0 = TEXT, bit 1 = IMAGE, bit 2 = AUDIO, bit 3 = VIDEO, bit 4 = STRUCTURED_DATA, bit 5 = SENSOR, bit 6 = KNOWLEDGE_GRAPH, bit 7 = CODE; at least one bit must be set	Records which input modalities contributed evidence to this Fact Unit during ingestion; supports modality-filtered queries and provenance transparency
EmbeddingVector	float32[4096]	Required	Default dimensionality 4096; L2-normalized; generated by the Ingestion Pipeline's Semantic Extraction stage; updated by NCL on learning events; stored separately in the Vector Index	High-dimensional dense vector encoding the semantic position of this Fact Unit in the global semantic embedding space; used for ANN search, NCL SDR generation, and similarity-based retrieval
RelationSet	Set of RelationEdge {TargetFUID: UUID, PredicateRef: UUID, Weight: float32, Direction: enum}	Optional	Maximum 65,536 outgoing edges per Fact Unit; Weight ∈ [0.0, 1.0]; Direction ∈ {OUTGOING, BIDIRECTIONAL}; no self-referential edges; no duplicate (TargetFUID, PredicateRef) pairs	The set of typed, weighted directed edges from this Fact Unit to related Fact Units in the Fact Graph; forms the edge structure of the hypergraph
BundleRefs	List of BundleID (UUID)	Optional	Maximum 1,024 bundle memberships per Fact Unit; each BundleID must resolve to a valid Fact Bundle	List of Fact Bundle identifiers to which this Fact Unit belongs; supports many-to-many membership
VersionChain	Ordered List of FUID	Optional	Entries must be in	Ordered list of prior

Field Name	Type	Req/Opt	Constraints	Semantic Role
	(UUID)		chronological order; each referenced FUID must be in state SUPERSEDED; maximum chain depth 4,096	FUID versions that this Fact Unit supersedes or refines; enables temporal versioning and knowledge evolution tracking
SafetyFlags	uint32 bitmask	Required	Computed by Safety Substrate at ingestion; immutable by API callers; can only be upgraded (higher restriction) by authorized Safety Substrate operations, never downgraded without dual-approval workflow	Bitmask encoding all applicable safety classifications for this Fact Unit; governs all access control, encryption, audit, and compliance enforcement actions
LinguisticMetadata	Struct {language: BCP-47 string, register: enum, formality: enum, domain_tag: string}	Required	language must be a valid BCP-47 language tag; register ∈ {FORMAL, INFORMAL, TECHNICAL, COLLOQUIAL, LEGAL, MEDICAL, SCIENTIFIC}; formality ∈ {HIGH, MEDIUM, LOW}	Captures the linguistic context of the Fact Unit's source material; used by the MRE to calibrate vocabulary, style, and register in regenerated output
AnnotationSet	Map of {string → QBF-typed scalar}	Optional	Key names must be valid UTF-8 strings; keys prefixed with "qb:" are reserved for system use; maximum 512 key-value pairs; maximum key length 256 bytes; maximum value size 64KB	Extensible key-value set for domain-specific metadata not covered by core Fact Fields; enables vertical domain extensions without schema modifications
TTL	Struct {ttl_seconds: uint64|null, eviction_policy: enum} | null	Optional	eviction_policy ∈ {ARCHIVE, PURGE, SUPERSEDE_WITH_REFRESH}; if null, Fact Unit has no expiry; minimum TTL 3,600 seconds (1 hour)	Optional expiry policy governing archival and eviction behavior; used for short-lived facts such as real-time sensor readings or time-bounded predictions
AccessControlL	List of ACLEntry	Required	Must contain at	Role-based and

Field Name	Type	Req/Opt	Constraints	Semantic Role
`ist`	{Principal: string, Role: enum, Attribute: string| null, Effect: enum}		least one OWNER entry; Role ∈ {OWNER, READER, WRITER, AUDITOR, DENIER}; Effect ∈ {ALLOW, DENY}; DENY entries take precedence over ALLOW entries	attribute-based access control entries governing which principals may perform which operations on this Fact Unit; enforced by the Safety Substrate at every retrieval and modification operation

3.1.1 Implementation Note — FUID Generation and Standard UUIDv7

MERGE NOTE — FUID/UUIDv7 CLARIFICATIONThe FUID is a custom, content-addressable identifier that uses a UUIDv7-compatible byte layout (128 bits, with a 48-bit timestamp segment and standard version/variant bits in the same positions RFC 9562 defines for UUIDv7) but is not a standard random UUIDv7. Standard UUIDv7 generation includes random bits and is not reproducible from its inputs; the FUID's remaining bits after the timestamp and 16-bit shard key are instead filled deterministically from SHA-3/256(normalized_content + source_id) so that re-ingesting identical content from the same source at the same shard produces the same FUID, which is what §3.3's deduplication logic depends on. Implementations should not use a stock RFC 9562 UUIDv7 library call to generate FUIDs — doing so will break deduplication. This note governs; the field table above is retained for the field-level contract (type, size, position) it establishes.

Because SemanticHash (below) already provides a content-addressable, language-independent fingerprint independent of FUID, systems that need strict RFC 9562 UUIDv7 compliance for interoperability with external UUID tooling may instead assign a standard random UUIDv7 to FUID and rely on SemanticHash alone for deduplication; this is a valid alternate configuration and does not require a Bible amendment, since it changes an implementation choice rather than a Fact Field contract.

3.2 Fact Unit Lifecycle

Every Fact Unit transitions through a defined set of lifecycle states from creation to final disposition. All state transitions are gated by the Safety Substrate and recorded in the Provenance Ledger. The following table defines all lifecycle states and their permissible transitions.

State	Description	Permissible Transitions	Trigger Conditions
DRAFT	Fact Unit has been constructed by the	DRAFT → VALIDATED DRAFT → PURGED (on	Initial state assigned at Fact Unit construction.

State	Description	Permissible Transitions	Trigger Conditions
	Ingestion Pipeline but has not yet passed all validation checks. Not visible to query operations. May be modified by the Ingestion Pipeline.	validation failure)	Transition to VALIDATED requires passing all Ontology Mesh type checks, Semantic Hash uniqueness verification, and Safety Substrate pre-insertion checks.
VALIDATED	Fact Unit has passed all validation checks and Safety Substrate pre-insertion gates. Pending final graph insertion. Not yet queryable.	VALIDATED → ACTIVE VALIDATED → DRAFT (on insertion failure, retry)	Transition to ACTIVE requires successful insertion into the Fact Graph, Vector Index update, and Provenance Ledger record confirmation.
ACTIVE	Fact Unit is fully inserted in the Fact Graph, indexed, and queryable. This is the primary operational state.	ACTIVE → SUPERSEDED ACTIVE → ARCHIVED ACTIVE → PURGED (Safety Substrate emergency action only)	Transition to SUPERSEDED when a newer Fact Unit with the same Semantic Hash is inserted. Transition to ARCHIVED when TTL expires or manual archival is triggered. PURGED only via dual-approval Safety Substrate emergency workflow.
SUPERSEDED	Fact Unit has been replaced by a more recent version. Retained in storage for lineage traversal. Not returned in standard queries unless version history is explicitly requested.	SUPERSEDED → ARCHIVED	Transition to ARCHIVED per retention policy or explicit administrative action.
ARCHIVED	Fact Unit has been moved to cold storage. Lineage traversal still resolves archived FUIDs. Not returned in standard queries; accessible via explicit archival query mode.	ARCHIVED → PURGED (compliance-driven, with dual-approval) ARCHIVED → ACTIVE (reactivation, requires dual-approval)	PURGED only via GDPR right-to-erasure or regulatory mandate workflow. ACTIVE reactivation permitted for archival errors only.
PURGED	Fact Unit has been cryptographically tombstoned. The FUID and Semantic Hash remain in the Provenance Ledger as tombstone records, but all substantive content is cryptographically erased. This state is terminal and irreversible.	None — terminal state	Triggered only by: (a) validation failure in DRAFT state, (b) GDPR right-to-erasure dual-approval workflow, (c) Safety Substrate emergency action. Every PURGE event is logged to the Safety Audit Log.

3.3 Fact Unit Equivalence and Deduplication

QueryBook employs a multi-stage equivalence detection and deduplication protocol to maintain Fact Graph integrity and prevent redundant or contradictory knowledge accumulation.

3.3.1 Semantic Hash Comparison

Upon construction of a new Fact Unit candidate, the Ingestion Pipeline computes its Semantic Hash and queries the Semantic Hash Index for any existing Active Fact Units with an identical hash. An exact Semantic Hash match indicates strong equivalence — the same semantic fact has already been ingested. In this case, the new Fact Unit is not inserted; instead, the existing Fact Unit's ConfidenceScore and ModalitySignature are updated to reflect the new supporting source, and a new Provenance Record is appended to the Provenance Ledger recording the corroborating ingestion event.

3.3.2 Canonical Normalization Pipeline

Before Semantic Hash computation, all Fact Unit candidates undergo a canonical normalization pipeline consisting of the following steps executed in order:

9. Unicode NFKC normalization of all string fields
10. Case folding for language-agnostic comparison
11. Entity coreference resolution: all coreferential entity references are mapped to their canonical EntityRef in the Entity Registry
12. Predicate canonicalization: equivalent predicates (e.g., synonymous predicates defined in the Predicate Library) are mapped to the canonical PredicateRef
13. Temporal normalization: all temporal expressions are converted to the canonical TemporalAnchor representation in UTC Unix nanoseconds
14. Negation normalization: facts expressed in negated form are converted to a canonical affirmative form with a negation flag in the PredicateRef metadata

3.3.3 Conflict Resolution Protocol

When a new Fact Unit candidate does not have an exact Semantic Hash match but the CDRE detects that it contradicts one or more Active Fact Units (i.e., it asserts the negation, a significantly different value, or an incompatible state for the same (Subject, Predicate) pair), the Conflict Resolution Protocol is invoked:

15. **Conflict Detection:** CDRE identifies all Active Fact Units with the same (SubjectRef, PredicateRef) pair or ontologically subsumable predicates.
16. **Confidence Comparison:** If the new Fact Unit's ConfidenceScore is greater than the existing Fact Unit's ConfidenceScore by at least a configurable threshold delta (default: 0.15), the existing Fact Unit is transitioned to SUPERSEDED and the new one is inserted as ACTIVE.
17. **Multi-Source Triangulation:** If confidence scores are within threshold of each other, the system holds both Fact Units in ACTIVE state with a CONTRADICTS edge between them in the Fact Graph. Both are returned in queries with their respective confidence scores. The contradiction is flagged in the NCL Attractor Network for human review prioritization.
18. **Source Authority Resolution:** The Conflict Resolution Protocol additionally consults source reliability weights maintained per SourceRef in the Provenance Ledger. Higher-authority sources receive a configurable bonus to their effective confidence score for conflict resolution purposes.
19. **Temporal Priority:** If two Fact Units with identical (SubjectRef, PredicateRef) pairs have non-overlapping TemporalAnchors, both are retained as ACTIVE without contradiction — they represent the same predicate at different times.

3.3.4 Merge Policies

For Fact Units from different sources that express the same semantic content with minor variation (near-match Semantic Hashes as computed by cosine similarity of EmbeddingVectors above a configurable threshold, default 0.97), a merge policy governs the resulting single canonical Fact Unit:

- **MERGE_CONSERVATIVE:** Retain the Fact Unit with the highest ConfidenceScore; add all other sources to the Provenance chain.
- **MERGE_UNION:** Combine ModalitySignatures; set ConfidenceScore to the weighted average of all source confidence scores weighted by source reliability; retain all ProvenanceRefs in a merged chain.
- **MERGE_STRICT:** Reject merge; treat as distinct Fact Units with SIMILAR_TO edges. Used for high-stakes domains (legal, medical) where exact identity is required.

3.4 Fact Bundle Structure

A Fact Bundle is a first-class QueryBook system object representing a named, versioned, semantically coherent collection of related Fact Units. The Fact Bundle schema is specified as follows:

Field	Type	Description
`BundleID`	UUID v7	Globally unique identifier for this Fact Bundle; generated at creation time; immutable
`BundleName`	UTF-8 string (max 512 chars)	Human-readable name for this Bundle; must be unique within the OwnerRef's namespace
`BundleVersion`	Semantic version string (MAJOR.MINOR.PATCH)	Version of this Bundle; incremented on every MemberFUIDs modification
`CreationTime`	int64 (Unix nanoseconds UTC)	Timestamp at which this Bundle was created; immutable
`OwnerRef`	PrincipalID (UUID)	Reference to the principal (user, service, or agent) that owns this Bundle
`SchemaRef`	OntologyMesh.SchemaID (UUID)	Optional reference to a schema in the Ontology Mesh constraining which FactTypes may be members of this Bundle
`MemberFUIDs`	Ordered List of FUID (UUID)	The ordered list of Fact Unit identifiers that are members of this Bundle; order is significant and governs linearization for regeneration
`BundleHash`	256-bit bytes (SHA-3/256)	Cryptographic hash of the ordered MemberFUIDs list and BundleVersion; enables integrity verification
`AccessPolicy`	AccessControlList (same structure as Fact Unit ACL)	Access control entries governing operations on this Bundle as a whole; individual member Fact Units retain their own ACLs
`BundleLineage`	ProvenanceID (SHA-256 → Provenance Ledger)	Pointer to the Bundle's root Provenance Record; captures the origin and all subsequent modifications to the Bundle itself (not its members)
`BundleType`	enum {DOCUMENT, DATASET, KNOWLEDGE_DOMAIN, EXPORT, PROJECT, TRAINING_SET, REPORT}	Classification of this Bundle's intended purpose; governs applicable validation rules and export policies
`Description`	UTF-8 string (max 4096 chars) | null	Optional human-readable description of this Bundle's contents and purpose

3.5 Fact Stream Structure

A Fact Stream is the primary mechanism for real-time continuous ingestion of Fact Units from a single logical source. Fact Streams support both push (source-initiated) and pull (QueryBook-initiated) delivery models and implement robust checkpointing for fault-tolerant ingestion.

Field	Type	Description
StreamID	UUID v7	Globally unique identifier for this Stream; used as the idempotency anchor for exactly-once delivery semantics
SourceRef	SourceID (UUID → Source Registry)	Reference to the registered source system from which this Stream originates
StreamProtocol	enum {PUSH_WEBHOOK, PUSH_GRPC_STREAM, PULL_POLLING, SUBSCRIPTION_KAFKA, SUBSCRIPTION_PUBSUB, BATCH_UPLOAD}	The delivery protocol for this Stream
Encoding	enum {QBF_BINARY, JSON_LD, PROTOBUF, AVRO, CSV, PARQUET}	The wire encoding format of records in this Stream; QBF_BINARY is the canonical format; others are supported via transcoding adapters
Compression	enum {NONE, LZ4, ZSTANDARD, GZIP}	Compression algorithm applied to stream records; LZ4 for low-latency streams, ZSTANDARD for high-throughput batch streams
CheckpointCursor	bytes (opaque, max 1KB)	Opaque cursor value identifying the last successfully processed record in this Stream; used for resumption after interruption
CheckpointPolicy	Struct {interval_ms: uint32, mode: enum {PERIODIC, PER_RECORD, ON_FLUSH}}	Policy governing how frequently the CheckpointCursor is durably persisted
BackpressurePolicy	Struct {max_inflight: uint32, strategy: enum {BLOCK, DROP, SAMPLE, PRIORITIZE}}	Policy controlling behavior when the Ingestion Pipeline is overwhelmed; BLOCK is default; SAMPLE and PRIORITIZE are available for high-volume sensor streams
DeliverySemantics	enum {AT_LEAST_ONCE, EXACTLY_ONCE}	Delivery guarantee for this Stream; EXACTLY_ONCE requires idempotency key support from the source system
SchemaRef	OntologyMesh.SchemaID (UUID) | null	Optional schema constraint; all Fact Units produced from this Stream must conform to the specified FactType schema
RetentionPolicy	Struct {ttl_hours: uint32,	Governs how long raw stream

Field	Type	Description
	max_records: uint64}	records are retained in the dead-letter queue and stream replay buffer

PART III — INGESTION PIPELINE

Part III Executive Summary **Part III specifies the complete Multimodal Ingestion Pipeline — the system responsible for transforming raw inputs from eight distinct modalities into validated, typed Fact Units ready for insertion into the Fact Graph. It covers the six-stage sequential pipeline architecture, per-modality extraction specifications, the Ingestion Orchestrator, entity resolution and linking, and confidence calibration. This Part is the primary reference for patent Cluster C claims and provides the foundation for understanding how QueryBook acquires knowledge from the external world.**

3.2 Worked Example

The following illustrates the schema of §3.1 applied to a concrete assertion, given source content stating that a named entity is located within a named containing region.

- Entity resolution: the subject and object references are each resolved to a canonical entity identifier within the Ontology Mesh (Chapter 6), and the asserted relationship is resolved to a canonical predicate identifier — not stored as the original text string.
- Fact Unit construction: FUID is deterministically derived from the resolved assertion, its source, and its ingestion timestamp; ConfidenceScore reflects extraction agreement and source reliability per the calibration model of Chapter 4; SafetyFlags are set per the Safety Substrate's evaluation (Chapter 8); SemanticHash is computed from the resolved, language-independent assertion rather than the original source text; TemporalAnchor records both when the assertion was true and when it was ingested; and the provenance reference identifies the specific source passage.
- Retrieval: a subsequent query for the same entity's location resolves through the identical canonical identifiers and retrieves the constructed Fact Unit deterministically, not through approximate text matching against the original source string.
- Revision: where the source content is later revised, a new, linked Fact Unit version is created via the VersionChain rather than the original record being modified, with both versions independently queryable.

3.6 The Post-Baconian Layered Definition

This section does not replace the schema of §3.1; it includes it whole and organizes it. Every field of the schema remains in force exactly as specified, and every field appears below inside the layer it belongs to. What this section adds is the organizing structure and the Baconian enrichment: the Fact Unit read as nine layers, so that the validation metadata the feedback loop of §3.7 accumulates — evidence, stability, alignment, provenance, and rights — is visible as architecture rather than scattered as fields. The semantic identity never changes; everything the loop adds surrounds that identity without touching it.

Layer 1, Core Semantic Identity, is the irreducible assertion and the only part of the record that never changes: the FUID, derived deterministically from content, source, and ingestion time; the type reference into the governed event and entity type catalogue; the canonical, language-neutral subject, predicate, and object references, the predicate drawn from the governed catalogue with its immutable identifier; and the language-independent semantic fingerprint, under which two records asserting the same proposition in different languages are the same assertion. A Fact Unit is immutable once created — a revision produces a new, linked version — which is what lets every other layer accumulate without the identity ever drifting. The record holds an assertion, not a concept; the concept models an assertion contributes to are formed and held in the World-View Model.

Layer 2, Structural Adjacency, is the record’s connections: typed, weighted edges to other records and hyperedge memberships binding sets of records, in the semantic, temporal, editorial, media, and language edge classes with their registered traversal costs, strengthened on co-activation under the Hebbian rule — deterministic and auditable, never fitted by gradient descent.

Layer 3, Temporal Stability, is read rather than asserted: the three timestamps distinguish when the assertion held, when it was ingested, and when its source was created, and the supersession chain links each version to the one it replaces under a correction or restatement relation. An assertion that persists unrevised across attestations, editions, and consolidation cycles is stable by demonstration.

Layer 4, Edition Differential, captures how the assertion behaves across editions of its work — present in every edition, present in some, modified, expanded, compressed, or transformed — through edition version chains, so an edition comparison is a traversal of markers rather than a text diff, and alignment across editions converges only below the divergence threshold.

Layer 5, Multi-Language Alignment, is carried by identity rather than translation memory: the semantic fingerprint makes realizations in any language the same record, parallel realizations align record-by-record, a wording-operative span is reproduced verbatim with a separately marked gloss, and an idiom resolves only to a registered equivalent.

Layer 6, Media Provenance, holds the multimodal attachments as evidence contributions in their own source classes, with cross-modality corroboration elevated where observations co-occur within a window narrow enough to indicate a common cause, and with any transformed asset carrying the provenance of its source and any introduced value marked as introduced and inadmissible as grounding.

Layer 7, Observation and Evidence, is the Baconian core. The confidence value is maintained as a first parameter accumulating evidence supporting the assertion and a second accumulating evidence contradicting it, the reported confidence being the ratio

of the first to their sum, their sum constituting the evidential weight from which a credible interval and a propagation precision are computed. The observation count is that weight; observation diversity is the effective number of contributing source classes computed from the attribution of contributed weight; contradiction markers are first-class, a conflict entering the persistent contradiction index and a contradiction too weak to revise being recorded rather than discarded; revision history is the version chain itself; and every update is closed-form, order-independent, and free of sampling. The record therefore carries not only how firmly the assertion is held but how much is known about it: two records reporting identical confidence on different quantities or varieties of evidence are distinguishable, which a scalar could not express.

Layer 8, Rights and Ownership, carries entitlement as a property of the assertion rather than a filter over a result: the safety classification and access-control list live within the record, territory, edition, and license scope govern reach, keys are scoped per tenant with hardware custody where configured, and where an operator elects it per title, an external, pre-existing blockchain-based ownership-verification service may be consulted for resale with the system as requesting party only. Ownership metadata informs entitlement; it does not enter the record as an identity node, and the system operates no token infrastructure.

Layer 9, Deterministic Output, is what the whole record guarantees at the point of use: generation constrained to vocabulary traceable to records, with the single stated exemption that a wording-operative span is reproduced rather than generated; reproducibility asserted where and only where the context key over the enumerated locked dimensions is identical; and full auditability through the signed provenance chain.

Master definition. A Fact Unit is a deterministic, atomic assertion record enriched with immutable semantic identity, typed adjacency structure, demonstrated temporal stability, edition-differential markers, fingerprint-anchored multi-language alignment, marked media provenance, Baconian evidence convergence maintained as an evidence pair with source-class diversity, in-record rights and safety, and context-keyed deterministic output — the atomic evidentiary object of the neuromorphic mesh, above which the World-View Model builds its concept models of the world. (Drafting record, Pass 114, September 2, 2026: adopted upon the inventor’s approval. Source material describing the record as a concept-node, and adding token-based ownership identity nodes, was corrected on adoption: concepts belong to the World-View Model, and the declined blockchain ownership layer remains declined.)

3.7 The Baconian Feedback Loop

The Baconian feedback loop is a critical design feature of the cognitive mesh: the cycle of action, observation, measurement, adjustment, and iteration, inductive and evidence-driven, under which internal structures update only as independent observations accumulate and agree, and under which nothing ever alters a record’s semantic identity. Each stage is bound to a specified mechanism, so the loop is an architecture rather than an aspiration.

Action and observation: the system acts — answers, ingests, executes a bounded program, directs registered capture equipment — and observations return through every ingestion modality and through outcome feedback, a delivered response being recorded with its cited records and their confidence at delivery, and a report bearing on it admitted as a knowledge record attributed to the reporting identity. Feedback is

evidence, not correction: no report sets a confidence value, supersedes a record, or removes one.
Measurement: each observation increments the evidence pair under the incrementing rule with fractional weighting — the supporting or contradicting parameter moves by the observation's source-reliability weight — and a reporting identity constitutes a single source class, its hundredth report weighing no more than a report from a hundredth identity. Diversity is the effective number of contributing source classes, corroboration from a class already dominating the parameters is admitted at reduced weight, and a generalization requires a diversity floor as well as a count floor: varied rather than merely numerous instances, made arithmetic.
Adjustment: revision is precision-weighted, the gain on a contradiction being the error's precision divided by the sum of the error's and the prior's precisions, with a gain below the configured floor producing no revision and the withheld contradiction recorded. Adjustment reaches only the layers the loop owns — evidence parameters, adjacency weights, participation metadata, stability and differential markers — and never the semantic core; where evidence demands new content the loop produces a new linked version, and an inductive generalization is emitted as a proposal that only a named human may validate.
Iteration and convergence: the cycle runs continuously across sessions, editions, languages, and modalities; long-term consolidation strengthens stable relationships on the gated election; and convergence terminates by energy descent. An assertion strengthens only as varied evidence converges upon it, weakens in proportion to the precision of what contradicts it, and can never be quietly rewritten — the anti-drift, truth-maintenance discipline of the platform.

3.8 The Enhanced Record: Uncertainty Decomposition, Polarity, Arity, Scope, History, Integrity, and Encoding

This section extends the record defined by §3.1 and organized by §3.6 with seven additional field families, adopted as a single enhancement pass. Nothing here replaces or narrows any existing field, layer, or behaviour: every field of the §3.1 schema and every layer of §3.6 remains in force exactly as specified, the Baconian feedback loop of §3.7 remains the record's engine of convergence, and each family below is an additive enrichment of the assertion record itself. Concepts continue to belong to the World-View Model; the fields of this section describe the assertion record only. The master definition of §3.6 is accordingly read as enriched by the seven families of this section: uncertainty decomposed, polarity explicit, arity native, scope stated, history journaled, integrity hash-chained, and encoding canonical.

3.8.1 Uncertainty Decomposition

The evidence pair of Layer 7 is extended with a deterministic classification of residual uncertainty into an epistemic component, reducible in principle by further observation, and an aleatory component, the irreducible variability of the observed world itself. The classification is computed, not asserted: where successive observations from distinct source classes continue to narrow the credible interval, residual uncertainty is classified epistemic; where observations accumulate without narrowing it — the interval stabilizing around persistent dispersion — the residue is classified aleatory. Alongside the

classification the record carries decomposed sub-parameters already implicit in the update machinery, made explicit and queryable: the fractional source-reliability weighting in effect, the measurement precision of the contributing observations, and the breadth of consensus measured as effective source classes. The decomposition gives the Baconian loop a principled stopping condition: iteration continues while residual uncertainty is predominantly epistemic and further observation is expected to narrow the interval, and stands down, without discarding any evidence, when the residue is aleatory and further sampling can only re-measure the world's own variance.

3.8.2 Assertion Polarity

Every Fact Unit carries an explicit polarity field marking the assertion as affirmative or negative, so that a negative finding — that a relation does not hold, that an event did not occur — is a first-class record rather than an inference left to interpretation of assertion content. Polarity participates in the semantic fingerprint, and contradiction detection thereby gains a structural fast path: two records whose fingerprints agree on the same assertion core but differ in polarity are a contradiction by field comparison, before any semantic analysis runs. Polarity also sharpens the Without-Information Rule: the absence of any record, which obliges refusal under the Prime Directive's without-information behaviour, is now formally distinct from the presence of a negated assertion, which is an answer — the system can say that a thing is known not to be so, rather than merely that nothing is known.

3.8.3 Native N-ary Assertion Structure

A single Fact Unit may carry one assertion with multiple typed argument slots — subject, value, unit, instrument, method, condition, observer, and further roles as the catalogue of its domain defines — while remaining one atomic assertion with one FUID, one fingerprint, and one evidence pair. A measurement made with a named instrument under stated conditions is one fact, and the record holds it as one fact, rather than shattering it into a constellation of binary statements joined by synthetic nodes as reification-based representations must. The semantic fingerprint is computed over the complete argument structure, so two realizations of the same n-ary assertion in different languages or notations resolve to the same record, and the typed, weighted adjacency of Layer 2 continues to bind the record to its neighbours exactly as before.

3.8.4 Applicability Scope

The record carries an explicit applicability-scope field stating the conditions under which the assertion holds: a temporal interval, a jurisdiction, an edition, an environmental or parameter range, a population, or a conjunction of these. Scope is the record-side complement of the context lock: the lock fixes the eight dimensions under which a query is answered, and the scope field states, in the record itself, the region of those dimensions within which the assertion is warranted, so validity is stored rather than re-derived at query time. Scope participates in retrieval matching, and it disciplines contradiction detection: two records asserting differently under disjoint scopes are not in conflict, and the system reports under which conditions each holds rather than reporting a contradiction that is not there.

3.8.5 Baconian Adjustment History

Each pass of the feedback loop of §3.7 appends an immutable entry to an adjustment-history layer within the record's metadata: the observation that arrived, its source class, the fractional reliability weighting applied under the incrementing rule, and the resulting movement of the evidence pair and credible interval. The history layer is metadata in the strict sense of §3.7 — the semantic identity of Layer 1 is never touched — and it makes the record self-auditing: the current confidence of any Fact Unit can be replayed, entry by entry, from its first observation to its present state, and the replay is deterministic, reproducing the stored values exactly. The record thereby carries not only its conclusion but its own revision trajectory, and an audit of any answer can descend from the answer to the records to the observations that made them.

3.8.6 Cryptographic Integrity Chain

Every record carries a cryptographic hash computed over its immutable core, and every supersession link includes the hash of the record it supersedes, so that each version chain is tamper-evident end to end by standard hashing alone: any alteration of a superseded record breaks the chain at the point of alteration and is detected by recomputation. This is integrity machinery internal to the Fact Graph, and nothing more: there is no distributed ledger, no token, and no ownership identity node — the declined blockchain ownership layer remains declined — and rights and entitlement continue to be carried exactly as Layer 8 specifies. What the chain adds is proof: the provenance and supersession structure the record has always carried becomes independently verifiable by any party holding the records.

3.8.7 Canonical Compact Encoding

The Fact Unit has a defined canonical binary serialization in which the required fields of §3.1 occupy fixed positions and optional fields are carried as tagged extensions. The encoding is lossless and bidirectional — a record round-trips between its full and compact forms without change, and the semantic fingerprint and FUID are identical in either form — so the compact form is the same record, not a summary of it. The encoding is what makes the micro-embedded deployments concrete: an offline reader or handset carries its Fact Units in canonical form at a fraction of the footprint of the source media, and the semantic compression of audio, video, and volumetric media rests on a specified record-level mechanism rather than a projection.

3.8.8 Architectural Integration

The enhanced fields are consumed across the architecture rather than sitting inert in the record. Precision-weighted error propagation under §41.6.4 reads the uncertainty decomposition, the propagation gain drawing on the explicit measurement-precision sub-parameter, and the Baconian loop consults the epistemic–aleatory classification as its stopping condition while writing the adjustment history of §3.8.5 on every pass. Contradiction detection under §41.4.6 reads polarity and applicability scope, surfacing structural contradictions by field comparison and dissolving apparent conflicts whose scopes are disjoint. Competitive Candidate Resolution under §41.5.3 carries polarity and scope on every candidate, so competing candidates of opposite polarity or disjoint scope are distinguished at the candidate level before resolution. The ingestion pipeline computes fingerprints over the complete n-ary argument structure at admission; the

supersession machinery maintains the integrity chain as versions are created; and the deployment layer serializes records in canonical compact form for edge and offline operation. Every field of this section is a deterministic function of record content and the observation stream, so the determinism guarantee — identical conditions under an identical context key producing identical results — is preserved without qualification. (Drafting record, Pass 115, September 2, 2026: added upon the inventor's direction to update the Fact Unit definition and the architectural features and functions associated with it. All seven field families are additive; no field of §3.1, no layer of §3.6, and no behaviour of §3.7 is removed or altered.)

3.9 Record-Fused Encoding and the Database-Neutral Boundary

This section fuses the canonical compact encoding of §3.8.7 to the Fact Unit as the record's own form, and states the boundary that keeps the system storable in any database. It adds no field to the schema of §3.1 and removes none; it specifies how the record already defined is encoded, stored, queried, and moved. Nothing here reintroduces a separate transport or storage standard external to the record: the encoding and the query interface operate on Fact Units within the system's own boundary, and the previously-declined framing of a mandatory external fact-transport format and external fact-query language, which would have required a host store to understand the record, remains declined. Concepts continue to belong to the World-View Model, and the determinism guarantee is preserved without qualification.

3.9.1 The Record-Fused Encoding

The compact encoding of §3.8.7 is the Fact Unit's own canonical at-rest and wire form. A record is not translated into a foreign storage schema and back; it is carried as itself — required fields at fixed offsets, optional fields tagged, the FUID and semantic fingerprint identical to the full form — so that the encoded record is self-describing and self-identifying without an external schema. The encoding is fused to the record rather than to any store: it is a property of the Fact Unit, computed and consumed within the system, and it is the same record whether resident in memory, written to disk, or in flight between deployments.

3.9.2 The Opaque-Blob Storage Contract

A host database, content repository, or key-value store holds an encoded Fact Unit as an opaque value — a blob in a column, a file, an object, or a key's payload — and is never required to parse it, index its interior, or understand its fields. The store's sole obligations are to accept bytes and return them unchanged. This is the invariant that preserves the system's database-neutral property: because the store is asked only to hold and return bytes, any database, repository, or e-reader platform qualifies, and no proprietary format, schema migration, or query-engine capability is imposed on it. The record's own identifiers — the FUID as a stable key, the fingerprint as a content address — are exposed to the store as plain opaque keys where an index is wanted, so retrieval by identity is available without the store reading the record's meaning.

3.9.3 Internal Fact Query and Edge Translation

Queries over Fact Units are expressed within the system in terms of the record — its identifiers, its fingerprint, its typed adjacency, its confidence and its applicability scope — and are resolved against the Fact Graph by the system's own retrieval. Where raw material must be drawn from an external store, an adapter translates the system's request into that store's native language — a relational query, an object lookup, a key fetch — and the external store never receives the system's internal fact-query form. This is the external-connection pattern already specified applied to storage: a declared capability, an adapter at the boundary, and no assumption that the outside system speaks the record's language. An external store is thereby a source and a repository, never the reasoning engine, which is the position that makes it replaceable and the determinism and groundedness guarantees independent of it.

3.9.4 Confidentiality and Integrity Across the Boundary

Because the host store holds the record as an opaque blob, the properties the record carries in §3.1 and §3.8 travel with it across the boundary. The integrity hash chain of §3.8.6 makes any alteration of a stored record detectable on read, so a store that corrupts or reorders a record cannot do so undetectably. The in-record safety classification and access-control list of §3.1 remain within the encoded record, so entitlement is a property the record carries into the store rather than a filter the store is trusted to apply; and where a deployment elects the zero-knowledge and key-custody provisions, an encoded record written to an external store is unreadable by that store, which holds bytes it cannot interpret. The database-neutral property is thereby not merely preserved but strengthened: any store will hold the record, and no store can read, reorder, or leak its meaning.

3.9.5 The Fuse-to-the-Record Invariant

The encoding and the query interface are fused to the Fact Unit and never to the storage or query interface of a host system. Stated as a design invariant: no host store is ever required to understand the record's format or to speak the record's query language in order to store or return it. The invariant is what distinguishes this fusion from the declined external transport-and-query standard — fused to the record, the system gains a compact, identity-bearing, integrity-sealed form and an internal query language while remaining storable in and retrievable from any database; fused instead to the store, the system would gain the same encoding but would require every database to adopt its format, converting an open architecture into a proprietary one. The former is specified here; the latter remains declined. (Drafting record, Pass 116, September 6, 2026: added upon the inventor's direction to implement a Fact Unit fused with the record. The encoding and query interface are additive extensions of §3.8.7 operating within the system boundary; no field of §3.1, no layer of §3.6, and no behaviour of §3.7 is removed or altered. The record remains an assertion record and concepts remain in the World-View Model; the previously-removed mandatory external fact-transport format and external fact-query language, which would require a host store to understand the record, remain removed; and the database-neutral property is preserved and, per §3.9.4, strengthened.)

3.10 The Fact Unit Language

The Fact Unit Language is the formal, language-independent assertion notation in which every Fact Unit is written and which every mechanism of the system reads. It adds no field to the schema of §3.1 and introduces no store or transport external to the record; it names, and specifies as a language, the notation the record already is. A statement in the Fact Unit Language comprises the canonical subject, predicate, and object identifiers drawn from the governed catalogues of the Ontology Mesh, a typed object value carrying its unit and dimension where the object is a scalar, the temporal anchor, and the record's evidence, provenance, safety, and entitlement fields. The governed predicate catalogue is its vocabulary; the record schema of §3.1 is its grammar; and the language-independent semantic fingerprint carried by every record is its identity function, under which two statements asserting the same proposition in any two natural languages are the same statement. Because the language is formal, a Fact Unit can be composed, validated against the governed type catalogue, compared for contradiction, evaluated deterministically, and regenerated into any natural language; because it sits below language, nothing written in it is entangled with the words of its source, which is what makes one knowledge substrate serve every language and every modality at once. A candidate that does not conform to the language is refused at admission rather than admitted incomplete.

The Fact Unit Language is claimed as an element-light method claim reciting the record as a language rather than as a field list, so that a competitor offering the same benefits by a differently-fielded record still operates upon the same assertion language. (Drafting record, Pass 117, September 6, 2026: adopted upon the inventor's direction. The Fact Unit Language names and specifies the notation the record of §3.1 already is; it adds no field, no store, and no external transport or query standard — the previously-removed external fact-transport format and external fact-query language remain removed — and concepts remain in the World-View Model. It is carried into the provisional application as claim 342 and into the composite registry as a feature; no field of §3.1, no layer of §3.6, and no behaviour of §3.7 is removed or altered, and the determinism guarantee is preserved without qualification.)

Chapter 4: Multimodal Ingestion Pipeline — Complete Specification

4.1 Pipeline Architecture Overview

The QueryBook Ingestion Pipeline is a six-stage sequential processing architecture with parallel branch processors for each supported input modality. Each stage is independently scalable and failure-isolated; a failure in one modality's branch processor does not propagate to other modality branches processing the same ingestion event.

Whatever form a source takes — an ebook's prose, a podcast's audio, a lecture's video, a live sensor feed, a running application's structured output, a web page — this pipeline reduces it to the same underlying unit: the Fact Unit, the atom this Bible's Fact Graph is built from (§1.1). The reduction is not mere reformatting; it is architecturally the same pattern this Bible's Sensory Abstraction Pipeline specifies for embodied agents (§55.1–§55.2): modality-specific raw input converted into structured, storable concepts before any reasoning occurs over it. Audiobooks, video, web pages, live data feeds, applications, and sensor data are not separate integrations bolted onto a text-first system — they are the same eight-modality pipeline (§4.2) QueryBook was built with from the start.

Stage	Name	Primary Operations	Output
1	Source Acquisition and Normalization	Fetch raw data from source system; apply encoding normalization; compute source checksum; assign StreamID and ingestion epoch; apply decompression	Normalized raw byte stream with source metadata
2	Modality Detection and Segmentation	Detect MIME type and modality; segment composite inputs (e.g., PDFs with embedded images); route each segment to its modality branch processor	Modality-tagged segments with routing headers
3	Semantic Extraction and Parsing	Apply modality-specific parsers; extract semantic content; produce intermediate semantic representations (ISRs)	Intermediate Semantic Representations per segment
4	Entity and Relation Recognition	Named Entity Recognition (NER); coreference resolution; entity disambiguation; predicate extraction; relation identification	Annotated ISRs with resolved EntityRefs and PredicateRefs
5	Fact Unit Construction and Validation	Assemble Fact Unit candidates from annotated ISRs; compute SemanticHash; assign FUID; execute Ontology Mesh validation; invoke Safety Substrate pre-insertion checks	Validated Fact Unit candidates in DRAFT state
6	Graph Insertion and Index Update	Execute deduplication protocol; insert new Fact Units into Fact Graph; update Vector Index with EmbeddingVectors;	Active Fact Units in Fact Graph; updated indexes; Provenance Records

Stage	Name	Primary Operations	Output
		update Semantic Hash Index; write Provenance Records; transition Fact Units to ACTIVE state	

4.2 Supported Input Modalities

QueryBook's Ingestion Pipeline supports the following eight distinct input modalities, each with dedicated branch processors at Stages 3 and 4:

Modality	Bitmask Bit	Supported Formats and Sources
Unstructured Text	Bit 0	Web documents (HTML, XHTML), PDF, DOCX, EPUB, plain text (UTF-8, UTF-16), email (RFC 5322, MIME), transcripts, books (EPUB, PDF), social media posts, news feeds (RSS/Atom), markdown, LaTeX, RTF
Structured Data	Bit 4	Relational tables (SQL dump, query results), CSV, TSV, JSON, JSON-LD, XML, YAML, TOML, spreadsheets (XLSX, ODS), protocol buffers, Apache Avro, Apache Parquet, Apache Arrow, HDF5
Images	Bit 1	Photographs (JPEG, PNG, HEIC, WEBP, TIFF), diagrams (SVG, PNG vector), charts (PNG, JPEG), maps (GeoTIFF, PNG), infographics, satellite imagery (GeoTIFF, COG), medical imaging (DICOM), scientific images (FITS)
Audio	Bit 2	Speech recordings (WAV, MP3, FLAC, AAC, OGG), music, environmental sound, recorded interviews, telephone call recordings (WAV, MP3), podcast audio, broadcast audio
Video	Bit 3	Raw video (MP4, MKV, AVI, MOV, WEBM), screen recordings, lecture videos, surveillance footage, broadcast video, video with embedded closed captions
Sensor Data	Bit 5	IoT telemetry (MQTT payloads, CSV time series), biometric streams (ECG, EEG, HRV), environmental sensors (temperature, humidity, air quality), financial tick data (FIX protocol, CSV), GPS traces, LIDAR point clouds
Knowledge Graphs	Bit 6	RDF/OWL (Turtle, N-Triples,

Modality	Bitmask Bit	Supported Formats and Sources
		RDF/XML, JSON-LD), property graphs (GraphML, Cypher dump), Wikidata JSON exports, SPARQL query results (SPARQL XML, JSON), SKOS taxonomies, schema.org markup
Code	Bit 7	Source code files (all major programming languages), API specification schemas (OpenAPI, AsyncAPI, JSON Schema, Protobuf .proto), configuration files (YAML, TOML, JSON, HCL), infrastructure-as-code (Terraform, Kubernetes YAML, Dockerfile), shell scripts, Jupyter notebooks

4.2.1 Content Outside the Eight Modalities

The eight modalities in §4.2 are broad, but not unbounded — a genuinely novel file format, or a composite artifact that doesn't cleanly resolve to a single bitmask bit, is a real possibility this Bible did not previously specify behavior for. This section closes that gap.

- Best-effort classification is attempted first: an artifact that doesn't match a listed format exactly may still resolve cleanly to one of the eight modalities by its underlying structure — an unlisted but structurally ordinary container format for unstructured text, for instance, still resolves to the Unstructured Text modality (Bit 0) rather than being rejected on format-name technicality.
- Where no reasonable resolution exists, ingestion halts for that artifact with a stated reason — not a silent drop, and not a forced fit into the nearest modality regardless of actual structure. This is the same non-silent-failure discipline already applied to ΔE validation (§70.3), registration drift (§70.4), and ambiguous NLPL compilation (§48.4.1) — a Fact Unit is not manufactured from content the pipeline cannot actually classify.
- A halted artifact is retained in its original form, not discarded, so a future modality addition or a manual author decision can resolve it later without requiring the source to be re-uploaded.

THIS IS A CLASSIFICATION FALLBACK, NOT A NINTH MODALITY This section does not add a ninth input modality or an open-ended “accept anything” ingestion path. It specifies what happens at the edges of the eight modalities already defined in §4.2 — attempt honest classification, and fail visibly rather than silently when honest classification isn't possible. Every Fact Unit that does result still carries the same ConfidenceScore, provenance, and safety-flag fields as any other (Appendix A).

4.3 Per-Modality Extraction Specifications

4.3.1 Unstructured Text Extraction

Specification Item	Detail
Extraction Method	Multi-stage NLP pipeline: sentence segmentation, tokenization, dependency parsing, constituency parsing, semantic role labeling (SRL), open information extraction (OpenIE), discourse parsing
Semantic Parser	Transformer-based encoder (minimum 1B parameters) fine-tuned on diverse text corpora for SRL and OpenIE; outputs (Subject, Predicate, Object) tuples from each sentence
Output Fact Unit Types	ASSERTION, DEFINITION, CLAIM, QUOTATION, EVENT_DESCRIPTION, TEMPORAL_STATEMENT, CAUSAL_CLAIM, COMPARISON, INSTRUCTION
Confidence Calibration	Base confidence from SRL probability × source reliability weight × temporal relevance decay; calibrated against held-out verification set with Platt scaling
Error Recovery	Malformed input: retry with fallback OCR if text extraction fails; encoding errors: attempt UTF-8/Latin-1/Windows-1252 detection cascade; semantic parse failure: store as UNSTRUCTURED_TEXT Fact Unit with reduced confidence

4.3.2 Structured Data Extraction

Specification Item	Detail
Extraction Method	Schema inference for schemaless formats (CSV, JSON); schema-driven parsing for typed formats (Parquet, Avro); relational decomposition into atomic (entity, attribute, value) triples; foreign key traversal for relational data
Semantic Parser	Rule-based schema mapper with LLM-assisted column header semantic labeling; maps column names and data types to Predicate Library entries
Output Fact Unit Types	ENTITY_ATTRIBUTE, MEASUREMENT, STATISTIC, RECORD, TIME_SERIES_POINT, CLASSIFICATION
Confidence Calibration	Structured data receives baseline confidence 0.85; adjusted by: schema validation pass/fail (±0.10), source authority rating (±0.15), data freshness (temporal decay)
Error Recovery	Schema parse failure: attempt schema inference; type coercion errors: log and skip with dead-letter queue entry; missing required fields: create partial Fact Unit with INCOMPLETE flag in AnnotationSet

4.3.3 Image Extraction

Specification Item	Detail
Extraction Method	Object detection (bounding box + class label); scene classification; OCR for embedded text; chart/diagram parsing; face and landmark recognition (where permitted by SafetyFlags); medical image segmentation (DICOM)
Semantic Parser	Vision-language model (VLM) for holistic image description; specialized models for chart parsing and medical imaging; EXIF metadata extraction for geolocation and timestamp
Output Fact Unit Types	VISUAL_OBSERVATION, DEPICTED_ENTITY, CHART_DATA_POINT, MAP_LOCATION, MEDICAL_FINDING, TEXT_IN_IMAGE, SCENE_DESCRIPTION
Confidence Calibration	Object detection confidence from model output; scene classification softmax probability; OCR character-level confidence averaged to word level; VLM-generated descriptions capped at 0.75 without secondary verification
Error Recovery	Corrupted image: attempt partial decoding; unsupported format: convert via image transcoding service; model timeout: queue for batch processing with priority downgrade

4.3.4 Audio Extraction

Specification Item	Detail
Extraction Method	Automatic Speech Recognition (ASR) with speaker diarization; acoustic event detection; music analysis (tempo, key, genre); sentiment and prosody analysis from speech
Semantic Parser	Transformer-based ASR model (minimum 300M parameters); NLP pipeline applied to transcripts; speaker embedding model for diarization
Output Fact Unit Types	SPOKEN_STATEMENT (attributed to speaker), ACOUSTIC_EVENT, INTERVIEW_EXCHANGE, AUDIO_METADATA
Confidence Calibration	ASR word-level confidence averaged to sentence level; diarization confidence as speaker attribution probability; acoustic event confidence from detection model
Error Recovery	High noise audio: apply noise reduction preprocessing; multiple overlapping speakers: produce separate low-confidence Fact Units per detected voice; audio codec unsupported: transcode via FFmpeg adapter

4.3.5 Video Extraction

Specification Item	Detail
Extraction Method	Keyframe extraction at configurable interval (default: 1 frame per 2 seconds plus scene-change-triggered frames); apply Image Extraction to keyframes; apply Audio Extraction to audio track; extract embedded closed captions (SRT, VTT, CEA-608); temporal alignment of all streams
Semantic Parser	Video-language model for scene understanding; temporal action recognition model; combined audio-visual fusion model for high-confidence event detection
Output Fact Unit Types	VIDEO_EVENT, SCENE_SEGMENT, SPEAKER_TURN (from audio), DEPICTED_ENTITY_IN_VIDEO, CAPTION_STATEMENT
Confidence Calibration	Keyframe-derived facts: same as Image Extraction; audio track-derived facts: same as Audio Extraction; cross-modal agreement bonus: +0.10 when visual and audio signals independently support the same Fact Unit
Error Recovery	Corrupted video frames: skip and note gap in AnnotationSet; missing audio track: process visual stream only; excessive duration (>4 hours): partition into segments with stream-level continuity tracking

4.3.6 Sensor Data Extraction

Specification Item	Detail
Extraction Method	Time series segmentation; anomaly detection; trend extraction; threshold event detection; statistical summary computation (mean, variance, min, max per window); pattern recognition for recurring events
Semantic Parser	Sensor schema registry-driven type mapping; unit-of-measure normalization to SI base units; IoT device metadata resolution from Device Registry
Output Fact Unit Types	MEASUREMENT, SENSOR_EVENT, ANOMALY, TREND, TIME_SERIES_SUMMARY, THRESHOLD_VIOLATION
Confidence Calibration	Sensor accuracy rating from Device Registry × data freshness; anomaly confidence from statistical significance (p-value converted to confidence score); calibration verified against known reference values when available
Error Recovery	Dropped packets: interpolate if gap <5% data-id="994" of window; sensor malfunction (out-of-range values): flag as SENSOR_ERROR FactType; clock drift: apply NTP-corrected timestamp reconciliation

4.3.7 Knowledge Graph Extraction

Specification Item	Detail
Extraction Method	RDF triple parsing; OWL axiom extraction; property graph edge enumeration; SPARQL-based selective extraction; schema alignment to Ontology Mesh via CDRE mapping rules
Semantic Parser	RDF parser (Apache Jena / RDFLib equivalent); OWL reasoner for inferred triples; Wikidata entity mapper to QB Entity Registry
Output Fact Unit Types	KG_TRIPLE, OWL_AXIOM, TAXONOMY_RELATION, EQUIVALENCE_ASSERTION, PROPERTY_RESTRICTION
Confidence Calibration	Directly asserted triples: source authority weight; inferred triples: confidence = min(inference_chain_confidence_scores); Wikidata items: trust level derived from citation count and revision history stability
Error Recovery	Malformed RDF: attempt heuristic repair; unknown predicates: create provisional Predicate Library entry with UNVERIFIED status; blank node resolution: assign provisional EntityRef with ANONYMOUS type

4.3.8 Code Extraction

Specification Item	Detail
Extraction Method	Abstract Syntax Tree (AST) parsing; function and class signature extraction; API endpoint extraction from OpenAPI/AsyncAPI schemas; dependency graph construction; comment and docstring extraction; semantic code embedding generation
Semantic Parser	Language-specific AST parsers (Tree-sitter grammars for 50+ languages); code-specialized language model for semantic embedding; API schema validators
Output Fact Unit Types	CODE_DEFINITION, API_ENDPOINT, DEPENDENCY_RELATION, ALGORITHM_DESCRIPTION, CONFIGURATION_VALUE, INTERFACE_CONTRACT
Confidence Calibration	Structurally parsed facts (AST-derived): confidence 0.95; semantically interpreted facts (docstring, comment): confidence based on NLP parser output; API schemas: confidence from schema validation pass rate
Error Recovery	Parse error: attempt partial AST extraction; unsupported language: extract comments and string literals as Unstructured Text; invalid API schema: flag as SCHEMA_ERROR and extract partial facts where possible

4.4 Ingestion Orchestrator

The Ingestion Orchestrator is the control plane for all ingestion operations. It manages job scheduling, rate limiting, idempotency, error handling, and observability across all Fact Streams and batch ingestion jobs.

4.4.1 Job Scheduler

The Ingestion Orchestrator maintains a priority queue of ingestion jobs with four priority levels:

- **CRITICAL (P0):** Real-time Fact Streams requiring sub-second processing; safety-flagged content requiring immediate classification
- **HIGH (P1):** User-initiated on-demand ingestion; scheduled high-frequency batch sources
- **NORMAL (P2):** Standard scheduled batch ingestion jobs; API-submitted documents
- **BACKGROUND (P3):** Historical re-ingestion; archive processing; low-priority crawl sources

4.4.2 Rate Limiting and Throttling

Rate limiting is enforced per SourceRef using a token bucket algorithm with configurable parameters: burst capacity (max tokens), refill rate (tokens per second), and burst floor (minimum guaranteed throughput). Sources exceeding their rate limit have excess records queued in the source-specific holding queue with configurable maximum depth before backpressure is applied to the source.

4.4.3 Idempotency Guarantee

Every ingestion event is identified by the composite key (StreamID, CheckpointCursor). The Orchestrator maintains a deduplication window of configurable duration (default: 72 hours) using a bloom filter backed by a persistent set. Duplicate ingestion events within this window are detected and discarded without processing, ensuring at-most-once fact construction for duplicate source events.

4.4.4 Dead-Letter Queue and Retry Policy

Records that fail processing after three consecutive attempts are moved to the Dead-Letter Queue (DLQ) with the following metadata: SourceRef, StreamID, CheckpointCursor, failure stage, error code, error message, and failure timestamp. DLQ records are retained for configurable duration (default: 30 days). Automated retry is scheduled using exponential backoff with jitter: retry delay = min(2^attempt × base_delay_ms + random_jitter_ms, max_delay_ms). Base delay: 1,000ms; max delay: 3,600,000ms (1 hour).

4.5 Entity Resolution and Linking

4.5.1 Named Entity Recognition Pipeline

Stage 4 of the Ingestion Pipeline applies a cascaded NER system comprising three layers:

20. **Rule-Based NER:** Gazetteers, regular expressions, and deterministic rules for high-precision entity types (dates, times, monetary values, geographic coordinates, standard codes such as ISO 3166, ISIN, MeSH, ICD-10)
21. **Model-Based NER:** Transformer-based sequence labeling model recognizing 64 fine-grained entity types (persons, organizations, locations, products, events, scientific concepts, legal entities, etc.)
22. **Knowledge-Graph-Guided NER:** Entity surface form lookup against the Entity Registry to capture entities already known to QueryBook with high confidence

4.5.2 Coreference Resolution

Within a single ingestion document or session, coreference resolution maps all mentions (pronouns, definite noun phrases, abbreviated forms) of the same entity to a single canonical EntityRef. Cross-document coreference is handled as entity disambiguation (Section 4.5.3).

4.5.3 Entity Disambiguation

When multiple EntityRef candidates exist in the Entity Registry for a surface form, the disambiguation system selects the most probable referent using a combination of:

- Context embedding similarity: cosine similarity of the mention's contextual EmbeddingVector to each candidate entity's canonical EmbeddingVector
- Prior probability: estimated from entity frequency in the Fact Graph and source domain alignment
- Type compatibility: entity type (from NER) must be compatible with the EntityType in the Entity Registry
- Cooccurrence signal: entities that frequently co-occur in the Fact Graph with nearby mentions receive a cooccurrence bonus

4.5.4 Entity Registry (QB-ER)

The Entity Registry is a globally consistent, distributed key-value store mapping EntityRef UUIDs to entity records. Each entity record contains: EntityID (UUID), CanonicalName (string), EntityType (OntologyMesh.EntityTypeID), AliasSet (list of surface forms), EmbeddingVector (4096-dim), ExternalIDs (map of {scheme → external_id}, e.g., Wikidata QID, ORCID, LEI), ConfidenceScore, CreationTime, LastVerifiedTime, and ProvenanceRef.

4.5.5 New Entity Creation

When no existing EntityRef satisfies disambiguation criteria above a configurable confidence threshold (default: 0.65), a new entity record is created in the Entity Registry with status PROVISIONAL. Provisional entities receive a PROVISIONAL flag in the AnnotationSet of all Fact Units referencing them. The system continuously attempts to link provisional entities to external knowledge bases (Wikidata, DBpedia, GeoNames, etc.) and upgrades their status to VERIFIED when a reliable external link is established.

4.6 Confidence Calibration

ConfidenceScore is a fundamental property of every Fact Unit and must be accurately calibrated to be useful for downstream reasoning and conflict resolution. The following multi-stage calibration process is applied by the Ingestion Pipeline.

4.6.1 Initial Confidence Assignment

The initial ConfidenceScore is computed as a weighted product of:

- **Extraction confidence:** The probability output of the semantic extraction model for the specific (Subject, Predicate, Object) triple
- **Source reliability weight:** A per-source scalar in [0.0, 1.0] maintained in the Source Registry, reflecting historical accuracy of facts from this source as verified against ground truth
- **Modality confidence coefficient:** A per-modality base coefficient reflecting the inherent reliability of each modality (Structured Data: 0.85; Knowledge Graphs: 0.80; Text: 0.70; Audio: 0.65; Image: 0.65; Video: 0.60; Sensor: 0.75; Code: 0.90)

4.6.2 Multi-Source Triangulation

When the same semantic fact (identified by matching or near-matching Semantic Hash) is independently observed from N distinct sources, the ConfidenceScore is updated using an independent evidence

combination model: `P_combined = 1 - prod(1 - P_i)` for independent sources, adjusted by source independence coefficient (two sources from the same organization are not fully independent).

4.6.3 Temporal Decay

ConfidenceScore is subject to a temporal decay function applied by the NCL on each query cycle for time-sensitive Fact Units. The decay function is exponential: `C_current = C_initial × exp(-λ × (T_now - T_ingestion))`, where λ is a per-FactType decay constant configurable in the Ontology Mesh schema. Immutable historical facts have λ = 0 (no decay).

4.6.4 Bayesian Update Protocol

When contradictory facts are subsequently ingested, the existing Fact Unit's ConfidenceScore is subject to a Bayesian downward revision. The revision magnitude depends on: the new conflicting fact's ConfidenceScore, the source authority differential, and the number of previously corroborating sources. The Bayesian update is computed incrementally and recorded as a Provenance Record update event in the Provenance Ledger.

PART IV — PROVENANCE SYSTEM

Part IV Executive Summary **Part IV specifies the Provenance Ledger — QueryBook's immutable, tamper-evident, complete record of all Fact Unit origins and transformations. Implemented as a Merkle DAG, the Provenance Ledger provides cryptographically verifiable lineage traversal, GDPR-compliant erasure via tombstoning, and full audit trail capabilities. This Part is the primary reference for patent Cluster M claims and is foundational to QueryBook's trust, compliance, and auditability value propositions.**

Chapter 5: Provenance Ledger — Complete Specification

5.1 Architecture

The Provenance Ledger is an immutable, content-addressed, append-only distributed log implementing a Merkle Directed Acyclic Graph (Merkle DAG) structure. Every write to the Fact Graph — whether a

Fact Unit creation, state transition, metadata update, or purge — produces a corresponding Provenance Record that is appended to the Merkle DAG. The content-addressable nature of the Merkle DAG guarantees that any tampering with a historical record is detectable by recomputing the chain hash from that record forward.

The Provenance Ledger is organized into blocks of configurable size (default: 1,000 Provenance Records per block). Each block's hash is computed as SHA-256 of the concatenation of all Provenance Record hashes within the block. Block hashes are chained: each block header contains the hash of the previous block, forming the Merkle DAG structure. The chain tip (most recent block hash) is anchored periodically (default: every 15 minutes) to an external notarization service for third-party verifiability.

5.2 Provenance Record Schema

Field	Type	Description
`ProvenanceID`	256-bit bytes (SHA-256)	Content hash of this Provenance Record (all fields except ProvenanceID itself); serves as the unique identifier and content-integrity fingerprint
`FUID`	UUID v7	The Fact Unit identifier to which this Provenance Record pertains
`Operation`	enum {CREATE, UPDATE, SUPERSEDE, ARCHIVE, PURGE, CONFIDENCE_UPDATE, SAFETY_FLAG_CHANGE, RELATION_ADD, RELATION_REMOVE}	The specific operation performed on the Fact Unit that this record documents
`SourceID`	UUID (→ Source Registry)	The identifier of the source system from which the originating data was acquired; null for derived or synthesized Fact Units
`AgentID`	UUID (→ Principal Registry)	The identifier of the system component, service, or human principal that executed the operation
`Timestamp`	int64 (Unix nanoseconds UTC)	The precise UTC timestamp at which this operation was executed; sourced from a synchronized NTP clock with nanosecond precision
`PreviousProvenanceID`	256-bit bytes (SHA-256) | null	The ProvenanceID of the preceding record in this Fact Unit's lineage chain; null for CREATE operations; forms the backward-linked list of the Fact Unit's history
`TransformationDescriptor`	Struct {pipeline_stage: string, model_id: string, model_version: string, parameters: map<string, string>}	Description of what processing, transformation, or inference was applied to produce or modify this Fact Unit; enables full

Field	Type	Description
		reproducibility of any Fact Unit's derivation
`InputFUIDs`	List of FUID (UUID)	For Fact Units derived from other Fact Units (e.g., by inference, aggregation, or synthesis), the list of source Fact Units used as inputs; empty for directly ingested facts
`ConfidenceAtWrite`	float32	The ConfidenceScore of the Fact Unit at the moment this operation was executed; enables historical confidence tracking
`SafetyChecksum`	256-bit bytes (HMAC-SHA-256)	HMAC-SHA-256 of the Provenance Record content computed with the Safety Substrate's signing key; enables detection of Safety Substrate bypass attempts
`RawSourceRef`	ContentAddressedRef (CID) | null	Content-addressed reference (IPFS CID or equivalent) to the raw source material in the Source Archive, if retained; null for derived facts and facts whose source retention has expired

5.3 Fact Lineage Traversal

5.3.1 Forward Traversal

Given a source Fact Unit (identified by FUID), forward traversal enumerates all Fact Units that were derived from it, directly or transitively. The traversal follows the `InputFUIDs` index — a reverse mapping maintained by the Provenance Ledger service that maps each FUID to the list of Provenance Records in which it appears as an InputFUID. This traversal answers questions such as: "What conclusions were drawn from this observation?" and "Which downstream Fact Units should be re-evaluated if this source fact's confidence changes?"

5.3.2 Backward Traversal

Given a Fact Unit, backward traversal follows the `PreviousProvenanceID` chain and `InputFUIDs` links back to the original raw ingestion event. For directly ingested facts, the chain terminates at the CREATE Provenance Record with a non-null RawSourceRef pointing to the original source material. For derived facts, the chain continues recursively through all InputFUIDs until all leaf nodes are directly ingested facts. Backward traversal answers questions such as: "What raw sources support this conclusion?" and "Is this fact ultimately traceable to a verified external source?"

5.3.3 Cross-Source Lineage

When a Fact Unit's Semantic Hash matches an independently ingested fact from a different SourceID, the Provenance Ledger records a CORROBORATION event linking the two Provenance chains. Cross-source lineage enables the system to identify facts that have been independently discovered from multiple unrelated sources, increasing their effective ConfidenceScore and supporting robustness arguments in adversarial settings.

5.4 Audit and Compliance

5.4.1 Full Audit Log Export

The Provenance Ledger supports full audit log export in multiple formats: QBF binary, JSON-LD, and NDJSON (newline-delimited JSON). Exports can be scoped by: time range, FUID set, SourceID, AgentID, Operation type, or SafetyFlags. Audit exports are signed with the system's ECDSA P-384 key and include a Merkle proof enabling third-party verification of log integrity without access to the full ledger. *(This export capability is callable directly via the Unified Audit API, §9.5.3, added in the Version 1.0 merge review.)*

5.4.2 GDPR Right-to-Erasure Implementation

GDPR Article 17 right-to-erasure requests are implemented via cryptographic tombstoning: the substantive content of all Fact Units associated with the data subject is encrypted with a deletion key, and the deletion key is then destroyed. The Provenance Ledger retains the FUID, Semantic Hash, and a PURGE Provenance Record (recording that erasure occurred, when, and under what legal basis), but the actual fact content becomes permanently irrecoverable. This approach preserves the integrity of the Merkle DAG chain while satisfying erasure requirements. A PII_ERASURE_CERTIFICATE is generated for each erasure event for regulatory documentation.

5.4.3 SOC 2 Audit Trail

The Provenance Ledger's complete operation log, combined with the Safety Audit Log (Chapter 8), constitutes the SOC 2 Type II audit trail. Every access to a Sensitive, Restricted, Confidential, or Hazardous Fact Unit is recorded with principal identity, timestamp, operation, and outcome. Access logs are tamper-evident via the same Merkle DAG chaining applied to Provenance Records.

5.4.4 Regulatory Retention Policies

Retention policies are configurable per FactType and per SafetyFlags classification:

- **Default retention:** Provenance Records retained for 7 years
- **REGULATED (HIPAA):** Minimum 6 years from creation or last activity
- **REGULATED (FINRA):** Minimum 7 years for transaction-related Fact Units
- **REGULATED (GDPR):** Retained until erasure request; tombstone retained indefinitely
- **LEGALLY_SENSITIVE:** Retained per applicable jurisdiction's statute of limitations plus 2 years

PART V — ONTOLOGY SYSTEM

Part V Executive Summary **Part V specifies the Ontology Mesh — the dynamic, versioned, poly-hierarchical type system governing all Fact Units and Fact Graph relations. It defines the four-layer architecture (Upper Ontology, Domain Ontologies, Task Ontologies, Instance Layer), the Predicate Library, ontology update protocols, and the Cross-Domain Rule Engine that enforces consistency across domain boundaries. This Part is the primary reference for patent Cluster K claims.**

Chapter 6: Ontology Mesh — Complete Specification

6.1 Design Principles

The Ontology Mesh embodies the following design principles, each of which is an inviolable architectural constraint:

23. **Poly-Hierarchical:** Every concept in the Ontology Mesh may participate in multiple taxonomic hierarchies simultaneously. A concept such as "clinical trial" may simultaneously be a subtype of "scientific study" (under the Science domain ontology) and "regulated process" (under the Law domain ontology). This poly-hierarchical structure enables richer cross-domain reasoning than single-hierarchy systems.
24. **Open World Assumption with Closed World Extensions:** By default, the Ontology Mesh operates under the Open World Assumption (OWA): the absence of a fact in the Fact Graph

does not imply the fact is false. However, specific domain sub-ontologies may be declared as operating under the Closed World Assumption (CWA) when application requirements demand completeness guarantees (e.g., drug interaction databases, regulatory compliance catalogs).

25. **Versioned and Backward-Compatible:** Every schema change to any component of the Ontology Mesh is versioned using semantic versioning. Backward-incompatible changes (changes to required fields, removal of valid FactTypes, narrowing of value domains) are never permitted in minor or patch versions. All deployed Fact Units remain valid under the ontology version under which they were created, and are eligible for automated re-validation against the current version.
26. **Domain-Federated:** Different knowledge domains maintain their own sub-ontologies, governed by designated domain ontology stewards. Domain sub-ontologies must be registered with and approved by the Central Ontology Authority (COA) before activation. All domain sub-ontologies must declare their upper ontology mappings (Layer 1 alignments).
27. **Machine-Actionable:** Every component of the Ontology Mesh is expressed in a formal representation language (OWL 2 DL extended with probabilistic extensions) enabling automated consistency checking, subsumption reasoning, and query rewriting.

6.2 Core Ontology Layers

Layer 1 — Upper Ontology

The Upper Ontology defines the universal top-level categories to which all domain-specific concepts must ultimately be mapped. The Upper Ontology is fixed and version-locked; changes require a formal Ontology Mesh Major Version upgrade with full system-wide impact analysis.

Top-Level Category	Description	Example Subcategories
Entity	A thing that exists independently and can be the subject or object of facts	Person, Organization, Location, Artifact, Biological_Organism, Abstract_Concept, Document
Event	A thing that happens or occurs at a specific time or over a time interval, involving participants	Natural_Event, Human_Activity, Process_Occurrence, Transaction, Communication_Event
State	A condition or situation that holds for an entity over a time interval	Physical_State, Social_State, Economic_State, Epistemic_State, Legal_State
Relation	A typed connection between two	Causal_Relation,

Top-Level Category	Description	Example Subcategories
	or more entities or events	Parthood_Relation, Temporal_Relation, Social_Relation, Spatial_Relation
Property	An attribute or characteristic that an entity or event may possess	Physical_Property, Social_Property, Epistemic_Property, Normative_Property
Process	A sequence of causally connected events or state changes that unfold over time	Biological_Process, Chemical_Process, Social_Process, Computational_Process
Region	A bounded area in space, time, or an abstract domain	Geographic_Region, Temporal_Region, Conceptual_Domain, Administrative_Region
Time	A representation of temporal position, interval, or ordering	Time_Instant, Time_Interval, Duration, Temporal_Order

Layer 2 — Domain Ontologies

Domain Ontologies extend the Upper Ontology with domain-specific concept hierarchies, predicates, and constraints. The following domains have registered sub-ontologies in the initial Ontology Mesh release:

Domain	Domain Ontology ID	Key Concept Types	Key Predicates
Science	OM-DOM-SCI-001	ResearchStudy, Hypothesis, Experiment, Finding, Dataset, ResearchInstitution	hypothesizes, replicates, contradicts_finding, cites, conducted_by
Law	OM-DOM-LAW-001	Statute, Regulation, Case, Ruling, LegalEntity, Jurisdiction, Contract	governs, ruled_by, amends, violates, enforceable_in, supersedes_regulation
Medicine	OM-DOM-MED-001	Disease, Drug, ClinicalTrial, MedicalProcedure, Symptom, Biomarker, Patient	treats, contraindicates, causes_symptom, tested_in, diagnosed_with
Finance	OM-DOM-FIN-001	FinancialInstrument, Market, Transaction, Portfolio, RegulatoryBody, EconomicIndicator	trades_at, issued_by, regulated_by, correlated_with, denominates_in
Engineering	OM-DOM-ENG-001	SystemComponent, Interface, Specification, Patent, Standard, TechnicalProcess	implements, depends_on, conforms_to, supersedes_standard, interfaces_with
Geopolitics	OM-DOM-GEO-001	Country, Government, Treaty, PoliticalParty, ElectionEvent, ConflictEvent	governs_territory, parties_to_treaty, allies_with, sanctions, recognized_by
Culture	OM-DOM-CUL-001	ArtWork,	created_by,

Domain	Domain Ontology ID	Key Concept Types	Key Predicates
		CulturalMovement, Language, Religion, CulturalPractice, Person	influenced_by, part_of_movement, spoken_in, depicts
Technology	OM-DOM-TEC-001	SoftwareSystem, Algorithm, Dataset, AIModel, HardwarePlatform, API	trained_on, benchmarked_on, built_with, exposes_api, replaces_system

Layer 3 — Task Ontologies

Task Ontologies define schemas for specific reasoning operations, enabling the NCL and AGI Extension Modules to perform structured reasoning. Each Task Ontology defines the Fact Unit types required as input and produced as output for a specific reasoning task.

Task Ontology	Input Fact Types	Output Fact Types	Governing Module
Comparison	ENTITY_ATTRIBUTE (multiple entities)	COMPARISON_RESULT, RANKING	NCL Associative Recall Component (ARC)
Causation	EVENT, CAUSAL_CLAIM	CAUSAL_RELATION, CAUSAL_CHAIN	AEM-3 Causal Discovery Module
Prediction	TREND, TIME_SERIES_POINT, CAUSAL_RELATION	PREDICTION (with confidence interval)	PIE Predictive Inference Engine
Classification	ENTITY_ATTRIBUTE	CLASSIFICATION_ASSIGNMENT	NCL Associative Recall Component (ARC)
Abduction	OBSERVATION	HYPOTHESIS (ranked by posterior probability)	AEM-1 Abductive Reasoning Module
Analogy	RELATION (source domain)	ANALOGICAL_MAPPING, CROSS_DOMAIN_INSIGHT	AEM-2 Analogical Reasoning Module
Counterfactual	EVENT, STATE (target to negate)	COUNTERFACTUAL_STATE (Scenario Graph branch)	AEM-4 Counterfactual Reasoning Module

Layer 4 — Instance Layer

The Instance Layer is populated dynamically by the Ingestion Pipeline and NCL and comprises the actual Fact Units stored in the Fact Graph. It is not a static layer of the Ontology Mesh specification but is governed by the three layers above it. The Instance Layer's population and structure at any point in time reflects the cumulative knowledge state of the QueryBook deployment.

6.3 Predicate Library

The Predicate Library is a governed catalog of all valid predicates in the Ontology Mesh. Each predicate is specified with the following properties:

Property	Description
PredicateID	UUID assigned at registration; immutable
Name	Canonical name in lowerCamelCase; unique within its domain
Domain	The domain ontology to which this predicate belongs; null for upper ontology predicates
SubjectType	The required Upper Ontology type of the Subject in any Fact Unit using this predicate
ObjectType	The required Upper Ontology type of the Object (entity or value type) in any Fact Unit using this predicate
Symmetry	Boolean: if true, P(A,B) implies P(B,A); system automatically creates the symmetric Fact Unit
Transitivity	Boolean: if true, P(A,B) and P(B,C) implies P(A,C); CDRE enforces transitivity closure
InversePredicate	PredicateID of the inverse predicate, if defined; e.g., "parentOf" is the inverse of "childOf"
Cardinality	Struct {min_objects: uint32, max_objects: uint32 | null} constraining how many Objects may be asserted per Subject for this predicate
OntologyLayer	enum {UPPER, DOMAIN, TASK}; determines which layer governs this predicate
TemporalScope	enum {ETERNAL, TIME_INDEXED, POINT_IN_TIME}; governs TemporalAnchor requirements
Version	Semantic version at which this predicate was introduced; used for backward-compatibility enforcement

The initial Predicate Library defines a minimum of 200 canonical predicates across all ontology layers and registered domain ontologies. These predicates are defined in the Predicate Library Reference (internal document OM-PL-REF-001, incorporated by reference into this Bible).

6.4 Ontology Mesh Update Protocol

Changes to any component of the Ontology Mesh — whether the Upper Ontology, a Domain Ontology, the Task Ontologies, or the Predicate Library — follow a governed change management protocol:

28. **Schema Change Proposal (SCP):** The proposing party submits a formal SCP document specifying the proposed change, the motivation, the affected FactTypes and predicates, and the proposed semantic versioning classification (major, minor, or patch).
29. **Impact Analysis:** The Ontology Mesh Impact Analysis Tool automatically computes the set of Active Fact Units that would require re-validation under the proposed change, the set of CDRE rules that would be affected, and the set of active QB-Query expressions that would require re-planning.

30. **Backward-Compatibility Verification:** For minor and patch changes, automated tooling verifies that all existing Active Fact Units remain valid under the proposed schema. Failures block promotion to staged rollout.
31. **Staged Rollout:** Approved changes are deployed in stages: first to the development environment, then to a 5% production traffic canary, then to 25%, then to 100%. Automatic rollback is triggered if error rates exceed configurable thresholds at any stage.
32. **Fact Unit Re-Validation:** After full deployment, a background batch job re-validates all Active Fact Units against the new schema version. Fact Units that fail re-validation are flagged with a SCHEMA_MISMATCH flag in the AnnotationSet and scheduled for human review.

6.5 Cross-Domain Rule Engine (CDRE)

The CDRE is specified in full detail in Chapter 17. This section provides the ontology-layer perspective on its role.

From the Ontology Mesh's perspective, the CDRE is the enforcement mechanism for the consistency constraints defined across domain boundaries. When a Fact Unit from the Medicine domain asserts a drug-disease relationship, and a Fact Unit from the Finance domain asserts the same company's product as a drug, the CDRE is responsible for ensuring that the two Fact Units are consistent with each other in their shared assertions about the underlying entities. The CDRE ruleset is part of the Ontology Mesh governance and is updated through the same SCP process as schema changes.

PART VI — NEUROMORPHIC COGNITIVE LAYER

Part VI Executive Summary **Part VI specifies the Neuromorphic Cognitive Layer (NCL) — the brain-inspired, adaptive reasoning engine that transforms QueryBook from a knowledge store into a living cognitive system. Inspired by mammalian cortical architecture, the NCL employs sparse distributed representations, spike-timing-dependent plasticity, attractor dynamics, and hierarchical temporal memory to perform associative recall, pattern completion, and predictive inference over the Fact Graph. This Part is the primary reference for patent Cluster B claims and is arguably the most technically novel component of the QueryBook system.**

Chapter 7: Neuromorphic Cognitive Layer (NCL) — Complete Specification

7.1 Design Philosophy

The NCL is designed around the observation that biological neural systems achieve sophisticated knowledge representation and retrieval capabilities not through brute-force search over large databases, but through sparse, distributed, associative representations that enable graceful degradation, pattern completion, and rapid generalization from sparse evidence. What the biological system associates and completes patterns over is, in QueryBook, always the Fact Unit — the same atomic, provenance-tracked knowledge unit specified from Chapter 1 onward. The NCL is not a general-purpose neural architecture applied incidentally to Fact Units; it is specified as the reasoning substrate for Fact Units specifically, which is why every principle below is stated in terms of Fact Unit operations rather than generic tensors. The NCL instantiates the following neurobiologically-inspired computational principles in a scalable distributed software system:

- **Sparse Distributed Representations (SDR):** Knowledge is encoded not as dense vectors but as sparse binary patterns where only approximately 2% of bits are active. SDRs provide robustness to noise (partial patterns still activate correct memories), union properties (multiple patterns can be superimposed without interference), and mathematical expressivity (the combinatorial space of 16,384-bit SDRs with 2% active bits contains approximately 10^472 distinct patterns).
- **Spike-Timing-Dependent Plasticity (STDP):** Synaptic connections between Fact Unit representations are strengthened when the pre-synaptic unit activates shortly before the post-synaptic unit, and weakened when the order is reversed. This timing-dependent learning rule enables the NCL to learn temporal sequences and causal associations from the order in which facts are activated during queries.
- **Attractor Dynamics:** The NCL implements an associative-memory-network-inspired attractor landscape over the Fact Graph — the same family of formalism popularly known as Hopfield networks, described more fully at §7.2.4. Well-established, high-confidence knowledge clusters form stable attractor basins — the system naturally returns to consistent knowledge states even when perturbed by noisy or contradictory input.

- **Hierarchical Temporal Memory (HTM):** Temporal sequences of Fact Unit activations are learned at multiple time scales simultaneously, enabling the NCL to recognize and predict patterns at the level of individual facts, document-scale context, and session-scale reasoning trajectories.
- **Continual Learning Without Forgetting:** The NCL learns continuously from every query-feedback cycle without catastrophic forgetting of previously established representations, achieved through an Elastic Weight Consolidation (EWC) analog applied to the Synaptic Weight Store.

7.2 Core NCL Components

7.2.1 Sparse Distributed Encoder (SDE)

The SDE transforms dense 4096-dimensional EmbeddingVectors from the Fact Unit Layer into sparse 16,384-bit Sparse Distributed Representations (SDRs). The encoding process:

33. The EmbeddingVector is divided into 128 segments of 32 dimensions each.
34. For each segment, a competitive learning process activates exactly 2 bits out of the 128 available bits in the corresponding SDR segment — specifically, the bits corresponding to the indices of the two highest-magnitude dimensions in that segment (winner-takes-most).
35. The result is a 16,384-bit binary vector with exactly 256 active bits (2% sparsity), providing a deterministic, approximately distance-preserving encoding of semantic distance: two semantically similar Fact Units will have overlapping SDR active bits in proportion to their semantic similarity.

The SDE maintains per-dimension statistics for adaptive threshold normalization, ensuring that the 2% sparsity target is maintained across the full semantic space as the vocabulary of Fact Units grows.

7.2.2 Temporal Memory Module (TMM)

The TMM learns temporal sequences of Fact Unit activations observed during query sessions. It is implemented as a multi-layer sparse sequence memory with the following architecture:

- **Layer 1 (Short-Term Sequence Memory):** Learns sequences of up to 16 Fact Unit activations within a single query session. Maintained in a sliding window over the Cognitive Context Window.

- **Layer 2 (Medium-Term Pattern Memory):** Learns recurring patterns of query sequences across sessions, identifying common reasoning pathways through the Fact Graph.
- **Layer 3 (Long-Term Topic Memory):** Maintains stable representations of persistent knowledge domains and frequently queried entity clusters, enabling rapid context restoration when a new session involves a familiar topic.

The TMM is trained incrementally using an online variant of the Temporal Context Model (TCM), with STDP-inspired weight updates applied after each query event.

7.2.3 Associative Recall Engine (ARE)

The ARE is the primary retrieval component of the NCL. Given a partial or noisy query input, the ARE performs pattern completion over the Fact Graph to identify the most semantically consistent set of matching Fact Units. The ARE operates as follows:

Merge note — acronym disambiguation: ARE is also used in this Bible for the Adaptive Response Engine (Chapter 52) and the App Recommendation Engine (§39.6). Where cross-chapter clarity matters, this NCL component may be cited as NCL-ARE; the short form ARE is retained here unchanged per this section's original text.

36. **Query SDR computation:** The SDE encodes the query into a 16,384-bit SDR.
37. **Nearest SDR search:** A SIMD-optimized Hamming distance computation identifies the top-K Fact Units by SDR overlap (default K=500) using a hierarchical SDR index structure (analogous to HNSW but operating over binary Hamming distances rather than float cosine distances).
38. **Graph neighborhood expansion:** For each of the top-K candidates, the ARE traverses the RelationSet edges in the Fact Graph up to configurable depth (default: 3 hops), adding neighbors to the candidate pool.
39. **Attractor stabilization:** The expanded candidate pool is passed to the Attractor Network for noise removal and consistency enforcement.
40. **Confidence-weighted ranking:** Surviving candidates are ranked by a composite score: SDR_overlap × ConfidenceScore × (1 + RelationPathStrength), where RelationPathStrength rewards Fact Units with strong edge connections to other highly ranked candidates.

7.2.4 Attractor Network (AN)

The Attractor Network is a continuous-valued associative memory network, of the class popularly known as Hopfield networks after John Hopfield's 1982 paper — though materially similar formalisms were described earlier by Marr (1971), Nakano (1972), Amari (1972), Little (1974), and Stanley (1976), and the broader, more historically accurate term is used throughout this Bible where precision matters. It operates over the EmbeddingVector space of Fact Units in the current Cognitive Context Window and implements the following dynamics:

Given an initial set of candidate Fact Unit activations from the ARE, the AN iteratively updates activations according to:

> $a_i(t+1) = \text{sign}(\Sigma_j W_{ij} \times a_j(t) - \theta_i)$
>
> Where:
>
> $a_i(t)$ = activation of Fact Unit i at iteration t ($\in \{-1, +1\}$)
>
> W_{ij} = synaptic weight from Fact Unit j to Fact Unit i (from Synaptic Weight Store)
>
> θ_i = unit-specific threshold (set to 0 for binary units)

The network is iterated until convergence (no further activation changes) or until a maximum iteration count is reached (default: 64 iterations). The converged activation pattern represents a stable, internally consistent subset of the candidate Fact Units that the AN has resolved as the best answer to the query given learned knowledge structure.

7.2.5 Synaptic Weight Store (SWS)

The SWS is a persistent, distributed store of learned connection weights between Fact Unit pairs. It is implemented as a sparse matrix (most Fact Unit pairs have zero connection weight) stored in a compressed sparse row (CSR) format in the Active Store. Key properties:

- **Capacity:** Supports up to 10^12 non-zero synaptic weights across the full Fact Graph
- **Update rule:** STDP-inspired: when Fact Unit A's activation at time t precedes Fact Unit B's activation at time t+Δt, the weight W_{AB} is increased by $\Delta W = \eta \times \exp(-\Delta t / \tau_+)$; when the order

is reversed, the weight is decreased by $\Delta W = \eta \times \exp(-\Delta t / \tau_-)$, where η is the learning rate, τ_+ and τ_- are configurable time constants

- **Homeostatic normalization:** Each Fact Unit's incoming weight sum is normalized to a target value periodically to prevent runaway excitation
- **EWC regularization:** A Fisher information-based importance score is maintained per synaptic weight; weight updates are penalized proportionally to importance to prevent forgetting of well-established associations
- **Eviction policy:** Zero-weight connections are periodically pruned; connections to ARCHIVED Fact Units are flagged for removal after a configurable grace period

7.2.6 Cognitive Context Window (CCW)

The CCW is the working memory of the NCL — an in-memory set of Fact Units that are currently active or recently active in the ongoing reasoning session. It is analogous to the prefrontal cortex's working memory buffer in biological cognition. CCW properties:

- **Default capacity:** 8,192 Fact Units (configurable per deployment)
- **Eviction policy:** Least-recently-activated Fact Unit is evicted when the CCW is full
- **Priority override:** Fact Units explicitly referenced in the current query are pinned to the CCW and not subject to eviction
- **Session persistence:** The CCW state is optionally persisted between user sessions via a session snapshot mechanism, enabling long-running research or analysis sessions to resume with full context
- **Cross-session learning:** Fact Unit activation patterns from the CCW are used to update the SWS after session termination

7.3 Associative Query Processing

The complete query processing flow through the NCL, from natural language input to ranked Fact Unit result set, is specified as follows:

Step	Component	Operation	Output
1	Query Encoder	Parse input (natural	Query EmbeddingVector

Step	Component	Operation	Output
		language or QB-Query); extract query intent, entity constraints, temporal constraints, confidence thresholds; generate query EmbeddingVector via the same encoder used for Fact Units	(4096-dim) + structured filter predicates
2	SDE	Encode query EmbeddingVector into 16,384-bit query SDR	Query SDR (16,384 bits, 256 active)
3	ARE — ANN Search	Perform Hamming nearest-neighbor search in SDR Index; apply structured filter predicates (FactType, TemporalAnchor range, SafetyFlags, ConfidenceScore threshold) to pre-filter candidates	Top-K Fact Unit candidates with SDR overlap scores
4	ARE — Graph Traversal	Expand candidate pool via Fact Graph RelationSet traversal (BFS up to depth 3); collect neighbors with relation weight above threshold	Expanded candidate pool (up to K × branching_factor Fact Units)
5	TMM	Apply session temporal context: boost candidates that are temporally adjacent to recently activated Fact Units in the CCW; penalize candidates inconsistent with session context	Temporally-weighted candidate pool
6	AN	Load candidate activations into Attractor Network; iterate to convergence; eliminate candidates that fall below activation threshold after convergence	Stabilized, internally consistent candidate set
7	Safety Substrate	Filter result set against caller's AccessControlList and SafetyFlags enforcement rules; redact or exclude Fact Units the caller is not authorized to receive	Safety-filtered result set
8	Result Ranker	Compute final composite score for each surviving Fact Unit; sort descending; return top-N	Ranked Fact Unit result set with metadata

Step	Component	Operation	Output
		results (configurable, default N=20) with per-unit confidence scores and lineage references	

7.4 Learning and Adaptation

7.4.1 STDP Weight Updates

After each query-feedback cycle (where feedback may be explicit user relevance signals or implicit signals such as dwell time and follow-up query content), the SWS is updated for all Fact Unit pairs that co-activated during the session. Pairs that activated in close temporal succession receive positive weight increments; pairs that activated in anti-correlated patterns receive negative weight decrements.

7.4.2 Hebbian Strengthening

Beyond STDP temporal ordering, co-activated Fact Unit pairs (those that both appear in the result set of the same query) receive a Hebbian weight increment proportional to the product of their activation strengths. This implements the principle that "neurons that fire together, wire together" at the Fact Unit level.

7.4.3 Homeostatic Normalization

To prevent runaway positive feedback loops (where a small set of highly interconnected Fact Units dominates all query results), the NCL applies homeostatic normalization on a configurable schedule (default: every 10,000 query cycles). Normalization computes the L1 norm of incoming weights for each Fact Unit and scales all incoming weights to maintain a target L1 norm.

7.4.4 EWC Regularization for Continual Learning

The EWC analog maintains a per-weight importance score I_ij (the Fisher information of the synaptic weight W_ij with respect to the query performance metric). The update rule for each weight is modified to:

> $\Delta W_{ij} = \eta_{raw} \times STDP_signal_{ij} - \lambda \times I_{ij} \times (W_{ij} - W_{ij_reference})$

> Where:
>
> η_raw = raw learning rate
>
> λ = EWC regularization coefficient (default: 0.5)
>
> I_ij = importance score for weight W_ij
>
> W_ij_ref = reference weight value (snapshot before the current task)

This penalizes large changes to weights that are highly important for previously learned associations, preventing catastrophic forgetting while still allowing new associations to be learned on less-important weights.

7.5 NCL Performance Characteristics

Metric	Target Value	Measurement Conditions
Query latency (p50)	< 50ms	10B Fact Units in Active Store; standard associative query
Query latency (p99)	< 200ms	Same conditions; includes graph traversal
Query throughput	> 10,000 QPS per NCL cluster	Horizontally scaled; 16-node NCL cluster
SDR encoding latency	< 1ms per Fact Unit	SIMD-optimized implementation on AVX-512
Attractor convergence	< 64 iterations typical; < 256 max	CCW of 8,192 Fact Units
SWS update throughput	> 1M weight updates/second	SIMD-optimized sparse update; 16-core node
Memory footprint (SDR Index)	< 2KB per Fact Unit	16,384-bit SDR at 2% sparsity, compressed
SWS storage (10B Fact Units)	< 40TB	Assuming average 400 non-zero connections per Fact Unit
Degraded-mode operation	Query latency < 500ms at 50% node failure	Automatic fallback to ANN-only (sans AN/TMM) mode

7.6 NCL Integrity and Robustness

The NCL components specified above (§7.2–§7.4) are, like any learned associative memory system, subject to three concrete risk classes distinct from the general-purpose harm categories the Safety Substrate (Chapter 8) already governs: unauthorized modification of learned state, adversarially-crafted queries designed to manipulate recall, and capacity exhaustion in the attractor network under sustained

load. This section specifies mitigations for each, working with the Safety Substrate's existing enforcement model rather than introducing a separate one.

7.6.1 Synaptic Weight Store Integrity

The Synaptic Weight Store (SWS, §7.2.5) is a persistent, distributed store of learned connection weights, updated continuously by the STDP (§7.4.1), Hebbian (§7.4.2), and EWC (§7.4.4) learning rules. Because it is both persistent and directly shapes future recall behavior, unauthorized or corrupted writes to it are a higher-consequence event than a single bad query result — a corrupted weight persists and compounds.

- Every weight-update batch is signed with the writing NCL instance's service credential before commit; the SWS rejects unsigned or invalidly-signed updates outright, using the same credential and signature-verification infrastructure already specified for inter-service calls on the Semantic Transport Layer (Chapter 9), not a new mechanism.
- Weight-update magnitude is bounded per batch (a configurable maximum L2-norm delta); a batch exceeding the bound is logged to the Provenance Ledger (Chapter 5) as an anomalous-update event and held for review rather than applied silently, consistent with how AEM output confidence is already capped and flagged (§15.1) rather than trusted unconditionally.
- The Homeostatic Normalization mechanism already specified (§7.4.3) doubles as a passive integrity check: normalization drift that exceeds the expected range for organic learning is a signal of either a runaway feedback loop or a tampered weight batch, and should raise the same anomalous-update flag as a magnitude violation.

7.6.2 Adversarial Query Resistance

The Associative Recall Engine (ARE, §7.2.3) performs pattern completion over partial or noisy input by design — which is also the property an adversarially-crafted query could exploit to probe for memorized specifics or to manipulate which Fact Units a completion surfaces.

- Input queries to the ARE pass through the same entity-resolution and confidence-calibration path already specified for ingestion (§4.5, §4.6) before pattern completion runs, rather than being fed to the attractor network directly — this bounds how much an adversarial input's surface form can directly steer the underlying SDR representation (§7.2.1).
- A completion result whose confidence sits far outside the ARE's normal operating distribution for a query of that shape (measured against a rolling baseline, not a fixed threshold) is flagged rather than returned silently, giving the same treatment to an anomalously confident recall that §15.1 already gives to an anomalously confident AEM inference.

- Repeated structurally-similar queries from a single source within a short window — a pattern consistent with systematic probing rather than organic use — are rate-limited at the Semantic Transport Layer (Chapter 9), which already owns request-level rate limiting for this system.

7.6.3 Attractor Network Capacity Bounds

The Attractor Network (AN, §7.2.4) has finite capacity: a continuous-Hopfield-style network's reliable pattern-storage capacity scales roughly linearly with the number of units (classically on the order of 0.14N stable patterns for N units under standard Hopfield dynamics), and exceeding it produces exactly the failure mode EWC regularization (§7.4.4) is meant to prevent — catastrophic interference, where storing a new pattern corrupts recall of existing ones. This bound was implicit in the existing specification but not stated as an explicit operating constraint.

- The AN's configured capacity ceiling, derived from its unit count using the bound above, should be treated as a hard operating limit, not a soft guideline — sustained operation near or above it is the condition under which interference becomes likely, independent of how well-tuned EWC's regularization weights are.
- The performance table already specified in §7.5 includes an "Automatic fallback to ANN-only (sans AN/TMM) mode" — this fallback is the correct response when capacity pressure is detected, and should be triggered proactively as utilization approaches the bound above, not only after interference is already observed in recall quality.

PART VII — SAFETY SUBSTRATE

Part VII Executive Summary **Part VII specifies the Safety Substrate — the architecturally embedded, non-bypassable constraint enforcement system that governs all operations at every QueryBook layer. Unlike external safety systems that can be circumvented, the Safety Substrate is integrated as a mandatory gate in the execution path of every Fact Unit operation. This Part defines the complete SafetyFlags taxonomy, per-layer enforcement specifications, adversarial robustness mechanisms, and incident response capabilities. This Part is the primary reference for patent Cluster H claims.**

Chapter 8: Safety Substrate — Complete Specification

8.1 Design Mandate

The Safety Substrate (SS) is the system-wide enforcement mechanism for all safety, privacy, security, and compliance constraints in QueryBook. Its design mandate is absolute and admits no exceptions:

NON-NEGOTIABLE DESIGN CONSTRAINT **The Safety Substrate is non-bypassable, architecturally embedded at every layer, and cannot be disabled, suspended, or circumvented by any runtime operation, API call, administrative command, configuration change, elevated privilege operation, emergency override, or any other mechanism. Any system operation that does not route through the Safety Substrate enforcement gate is an architectural defect and must be treated as a critical security vulnerability.**

8.1.1 Scope of the Non-Bypassable Guarantee and Enforcement Throughput — Redesign Note (Economy and Performance)

REDESIGN FINDING — SCOPE AND RISK **The "non-bypassable" guarantee above is, and should be read as, a claim about QueryBook's own software architecture: no code path within QueryBook — no API call, administrative command, or configuration change — can skip the Safety Substrate gate. It is not, and should not be represented as, a guarantee against compromise at the kernel level, underlying hardware, or by a privileged insider with direct data-store access; no software-layer control can make that claim honestly, and asserting an unqualified absolute guarantee here creates unnecessary legal and technical exposure if ever cited in a security representation or breach-liability context. This scoping does not change the enforcement mechanism specified above — it clarifies what class of guarantee is actually being made.**

Separately, a throughput consideration for synchronous gating: at sustained high ingestion rates (for example, 10,000 Fact Units/second), even a modest per-check latency (on the order of 5ms) means the Safety Substrate alone requires roughly 50 seconds of cumulative compute per wall-clock second — meaning a parallel worker pool sized for at least the same order of magnitude as peak ingestion QPS is required to keep gating synchronous without becoming the system's throughput ceiling. This sizing requirement was implicit in the "every layer, every operation" enforcement model above but not previously stated as an explicit capacity-planning input; it should be added to the deployment sizing

guidance in Chapter 19 and validated against real ingestion-rate targets before those targets are finalized.

The Safety Substrate operates as a mandatory gate at the following eight enforcement points:

41. **Ingestion Gate:** Inspects incoming raw data before Stage 1 processing
42. **Fact Unit Construction Gate:** Validates Fact Unit candidates before DRAFT state is assigned
43. **Graph Insertion Gate:** Final check before ACTIVE state transition
44. **Query Processing Gate:** Validates query expressions for prohibited patterns
45. **Fact Unit Retrieval Gate:** Enforces per-Fact-Unit ACL and SafetyFlags on every retrieval
46. **Regeneration Gate:** Ensures MRE output satisfies all applicable safety constraints
47. **Export Gate:** Applies additional restrictions on Fact Unit data leaving the system boundary
48. **Federation Gate:** Controls which Fact Units may be shared with external QueryBook instances

8.2 Safety Classification Taxonomy

Every Fact Unit carries a SafetyFlags bitmask (uint32). The following table defines all 16 currently defined flag bits, their bit positions, semantics, and enforcement actions.

Flag Name	Bit Position	Semantics	Enforcement Actions Triggered
SAFE	Bit 0	No special restrictions. Default state for all Fact Units that do not meet any higher classification criterion.	Standard access control (ACL) enforcement only
SENSITIVE	Bit 1	Content that, while not regulated, warrants elevated access scrutiny due to its nature (e.g., internal business strategy, non-public financial data, personal opinions of identifiable individuals)	Authentication required; all access events logged to Safety Audit Log; export flagged for review
RESTRICTED	Bit 2	Content access limited to specific roles. Not publicly accessible. Typically internal-only technical or operational information.	Role-based access control enforced; access token inspection; all queries involving RESTRICTED Fact Units logged
CONFIDENTIAL	Bit 3	Commercially sensitive or legally protected	Encryption-at-rest enforced (AES-256-GCM);

Flag Name	Bit Position	Semantics	Enforcement Actions Triggered
		information requiring data protection controls	encryption-in-transit enforced (TLS 1.3); key management via HSM; export requires written authorization
HAZARDOUS	Bit 4	Content that could facilitate harm if accessed by unauthorized parties (e.g., detailed technical instructions for dangerous activities, vulnerability information, dual-use research)	Dual-approval required for any retrieval or modification; access limited to HAZARDOUS_AUTHORIZED role; all access events require written justification; export prohibited without separate HSM-signed authorization
LEGALLY_SENSITIVE	Bit 5	Information subject to legal privilege, ongoing litigation holds, regulatory review, or court-ordered restrictions	External legal review required before any export; litigation hold flag prevents any state transition to ARCHIVED or PURGED; attorney-client privilege filter applied to access list
PERSONALLY_IDENTIFIABLE	Bit 6	Content containing or derived from personally identifiable information as defined by GDPR, CCPA, or other applicable privacy regulations	PII handling rules enforced: consent verification, purpose limitation check, data minimization validation; GDPR Article 25 data-protection-by-design controls applied; DSAR (Data Subject Access Request) tracking
REGULATED_HIPAA	Bit 7	Protected Health Information (PHI) as defined by HIPAA	HIPAA access controls (minimum necessary standard); BAA verification required; 6-year retention minimum; PHI audit log mandatory; breach notification workflow integrated
REGULATED_GDPR	Bit 8	Personal data subject to GDPR jurisdiction	Lawful basis verification; data subject rights enforcement (access, rectification, erasure, portability, objection); cross-border transfer controls; DPA notification triggers
REGULATED_CCPA	Bit 9	Personal information subject to CCPA/CPRA jurisdiction (California	Opt-out of sale enforcement; right-to-know compliance;

Flag Name	Bit Position	Semantics	Enforcement Actions Triggered
		residents)	deletion request workflow; sensitive personal information restrictions per CPRA
REGULATED_FINRA	Bit 10	Financial records subject to FINRA regulations	7-year retention mandatory; WORM (Write Once Read Many) storage required; supervisory review workflow integration; exam access controls
EXPORT_CONTROLLED	Bit 11	Technical data subject to export control regulations (EAR, ITAR)	Nationality-based access restrictions; geographic restriction on API responses; deemed export controls for foreign national employees; export license verification
AI_GENERATED	Bit 12	Fact Unit primarily derived from AI model output rather than directly from observed data	AI-origin disclosure in API responses; reduced default ConfidenceScore cap (0.85 maximum without independent verification); additional scrutiny in Hallucination Prevention pipeline
UNDER_REVIEW	Bit 13	Fact Unit flagged for human review due to conflict, anomaly, or policy concern	Excluded from default query results; accessible only to REVIEWER role; review task created in audit queue; version change prohibited until review is resolved
SYNTHETIC	Bit 14	Fact Unit derived from a Scenario Graph or simulation, not from real-world observation	Clearly labeled as synthetic in all outputs; excluded from training data exports; excluded from Provenance attestation claims
RESERVED	Bits 15–31	Reserved for future Safety Substrate expansion; must be zero in all current Fact Units	Safety Substrate rejects Fact Units with non-zero reserved bits

8.3 Safety Enforcement Points — Per-Layer Specification

Enforcement Point	What Is Checked	Block Trigger	What Gets Logged	Escalation Path
Ingestion Gate	Source reputation score; known	Source reputation < threshold; rate limit	Source ID, payload hash, classification	Automated alert to Security Operations;

Enforcement Point	What Is Checked	Block Trigger	What Gets Logged	Escalation Path
	malicious payload signatures; rate limit compliance; ingestion quota per source; preliminary content classification for HAZARDOUS patterns	exceeded; HAZARDOUS pattern detected without authorization	result, block reason, timestamp	dead-letter queue for manual review
Fact Unit Construction Gate	SafetyFlags classification completeness; PII detection in all string fields; HAZARDOUS content classification; ACL well-formedness; FactType authorization (some FactTypes require elevated privilege)	PII detected without PERSONALLY_IDENTIFIABLE flag; HAZARDOUS content without HAZARDOUS_AUTHORIZED principal; malformed ACL	FUID (draft), SafetyFlags computed, PII detection result, construction agent ID, timestamp	Automated PII flagging workflow; human review queue for HAZARDOUS content
Graph Insertion Gate	Duplication of final SafetyFlags check; CDRE consistency validation; ConfidenceScore range validation; EmbeddingVector dimensionality check; SafetyChecksum of Provenance Record	SafetyFlags downgrade detected (tampering); CDRE integrity constraint violation; invalid embedding vector	FUID, pre-insertion state hash, check results, inserting agent ID, timestamp	Integrity violation triggers immediate Security Operations alert and system audit
Query Processing Gate	Query expression syntax validation; prohibited pattern detection (exfiltration signatures, mass-download patterns); caller authentication and authorization; query rate limit enforcement; ACROSS SCENARIOS safety scope check	Prohibited query pattern; unauthenticated caller; rate limit exceeded; attempt to query HAZARDOUS without dual-approval	Caller ID, query hash, pattern check results, authorization outcome, timestamp	Repeated prohibited patterns trigger account suspension and Security Operations review
Fact Unit Retrieval Gate	Per-Fact-Unit ACL enforcement against caller's identity and attributes; SafetyFlags enforcement	Caller not in ACL ALLOW list; missing required role; HAZARDOUS retrieval without dual-approval; UNDER_REVIEW not	Caller ID, FUID, retrieval result (ALLOW/DENY), SafetyFlags active, timestamp	Repeated DENY events for high-sensitivity Fact Units trigger Security Operations alert

Enforcement Point	What Is Checked	Block Trigger	What Gets Logged	Escalation Path
	(encryption-in-transit for CONFIDENTIAL; HAZARDOUS dual-approval check); SafetyFlags disclosure labeling	accessible to non-REVIEWER caller		
Regeneration Gate	Output sentence grounding verification; PII leakage detection in MRE output; HAZARDOUS content regeneration prohibition; AI_GENERATED disclosure enforcement; output length and rate limits	Ungroundable sentence detected; PII detected in output without consent; HAZARDOUS content in regeneration output	Session ID, output hash, grounding check results, redacted content indicator, timestamp	Ungroundable content logged as potential hallucination incident; immediate review if PII leakage detected
Export Gate	Export destination authorization; EXPORT_CONTROLLED jurisdiction check; CONFIDENTIAL written authorization; bulk export rate and volume limits; watermark application; canary Fact Unit inclusion for exfiltration detection	Unauthorized export destination; EXPORT_CONTROLLED to restricted jurisdiction; CONFIDENTIAL without authorization; bulk export exceeding threshold	Exporter ID, destination, Fact Unit count, SafetyFlags distribution in export, authorization reference, timestamp	All blocked exports trigger immediate Security Operations notification; repeated attempts trigger account suspension
Federation Gate	Remote instance trust level; ontology compatibility negotiation result; reciprocal SafetyFlags policy compatibility; CONFIDENTIAL and above excluded from federation by default unless explicit bilateral agreement	Untrusted remote instance; incompatible SafetyFlags policies; CONFIDENTIAL Fact Units without explicit bilateral federation agreement	Remote instance ID, trust level, negotiated policy, included/excluded SafetyFlags, timestamp	Trust level violations trigger federation suspension and Security Operations review

8.4 Adversarial Robustness

8.4.1 Prompt Injection Detection

The Safety Substrate applies prompt injection detection at both the Ingestion Gate (for incoming documents that might contain embedded instructions) and the Query Processing Gate (for queries that might contain adversarial instruction injections). Detection employs a multi-layer approach:

- **Syntactic detection:** Pattern matching against known injection signatures (role-playing instructions, system prompt override attempts, delimiter injection patterns)
- **Semantic detection:** EmbeddingVector-based classifier trained to identify instruction-like content in non-instruction contexts
- **Behavioral anomaly detection:** Statistical deviation from expected query distribution for the authenticated caller's historical query profile

8.4.2 Fact Poisoning Detection

Fact poisoning occurs when an adversary attempts to corrupt the Fact Graph by injecting plausibly formatted but false or misleading Fact Units. The Safety Substrate's Fact Poisoning Detection subsystem applies the following checks:

- **Contradiction anomaly detection:** Any new Fact Unit that contradicts more than a configurable threshold of existing high-confidence Fact Units (default: contradicts 10+ Fact Units with ConfidenceScore ≥ 0.85) is automatically flagged as UNDER_REVIEW rather than inserted as ACTIVE
- **Source velocity anomaly:** A source that suddenly produces orders of magnitude more Fact Units than its historical baseline triggers a rate anomaly alert and temporary hold on that source's Fact Units pending review
- **Semantic outlier detection:** New Fact Units whose EmbeddingVectors are statistical outliers from the distribution of the source's historical contributions are flagged for review
- **Canary Fact Units:** A set of known-correct Fact Units with synthetic source signatures (canaries) are embedded in the Fact Graph. Any modification or contradiction of a canary triggers an immediate Security Operations alert indicating potential graph manipulation

8.4.3 Data Exfiltration Prevention

- **Rate limiting:** Per-caller, per-API-key, and per-IP daily Fact Unit retrieval limits; configurable per sensitivity tier
- **Watermarking:** All exported Fact Unit sets include covert statistical watermarks enabling identification of the specific export event that a leaked dataset originated from, enabling forensic attribution
- **Canary retrieval monitoring:** Retrieval of canary Fact Units by unusual callers triggers exfiltration alerts
- **Bulk download pattern detection:** Sequential enumeration of FUIDs, downloads spanning the full FUID namespace, and other systematic traversal patterns are detected and blocked

8.4.4 Model Inversion Resistance

EmbeddingVectors are sensitive because they encode the semantic content of Fact Units in a form that can be inverted (approximately reconstructed) by an adversary with access to the embedding model. The Safety Substrate applies the following protections:

- EmbeddingVectors are never exposed in API responses; only the textual reconstruction of the Fact Unit is returned
- Differential privacy noise (Laplace mechanism with calibrated ε) is added to any aggregated EmbeddingVector statistics returned in analytics APIs
- ANN similarity search results are returned as ranked Fact Unit content, not as distance scores that could be used to reconstruct embeddings

8.5 Audit and Incident Response

8.5.1 Safety Audit Log

The Safety Audit Log is a separate, tamper-evident log (distinct from the Provenance Ledger) that records every Safety Substrate enforcement event. It uses the same Merkle DAG chaining structure as the Provenance Ledger and is anchored to an external notarization service. The Safety Audit Log is immutable and cannot be modified, archived, or purged by any system operation — including GDPR erasure requests, which are satisfied by tombstoning the Fact Unit content without removing the audit

records of access events (which contain no personal data). *(Queryable directly via GET /v1/audit/log in the Unified Audit API, §9.5.3, added in the Version 1.0 merge review.)*

8.5.2 Automated Incident Detection and Alerting

The Safety Substrate continuously evaluates incoming audit log events against a set of detection rules using a streaming rule engine. Detected incidents are classified by severity (P0–P3) and routed to the appropriate response channel:

Severity	Condition Examples	Response	SLA
P0 — Critical	Safety Substrate bypass attempt detected; canary modification; HAZARDOUS content exfiltration attempt	Immediate automated account suspension; Security Operations paged; CEO/CTO notified	Acknowledge within 5 minutes; contain within 30 minutes
P1 — High	Repeated ACL violations from single caller; fact poisoning indicator; bulk download pattern	Rate limiting escalated; Security Operations alerted; automated investigation initiated	Acknowledge within 30 minutes; investigate within 2 hours
P2 — Medium	Elevated UNDER_REVIEW rate from single source; unusual query pattern; PII leakage prevention trigger	Source temporarily throttled; Security Operations ticket created	Acknowledge within 2 hours; investigate within 24 hours
P3 — Low	Individual access denial events; minor configuration anomalies; new source classification events	Logged; aggregated into daily Security Review report	Review within 5 business days

8.5.3 Forensic Replay Capability

The Safety Audit Log, combined with the Provenance Ledger and a snapshot of the Fact Graph state at any point in time (via the continuous WAL replication), enables complete forensic replay of any incident. Given a window of time, the forensic replay capability can reconstruct: every query executed, every Fact Unit accessed, every safety event triggered, and the exact state of every Fact Unit at any moment within the replay window. This capability supports both internal incident investigations and external regulatory examinations.

PART VIII — SEMANTIC TRANSPORT LAYER

Part VIII Executive Summary **Part VIII specifies the Semantic Transport Layer (STL) — the six-layer protocol stack governing all transmission of Fact Units within and across QueryBook system boundaries. The STL provides framing, session management, reliability, security, topic-based routing, and compression services specifically optimized for the semantics of Fact Unit transmission. This Part also defines the QB-Streaming Protocol for real-time subscriptions, the Federation Protocol for cross-instance synchronization, and the external API specifications. This Part is the primary reference for patent Cluster E claims.**

Chapter 9: Semantic Transport Layer (STL) — Complete Specification

9.1 Purpose and Scope

The STL governs all transmission of Fact Units and associated control messages between the following system boundaries:

- Ingestion workers and the Fact Graph storage layer
- Internal QueryBook microservices (NCL, PIE, MRE, CDRE, AEM modules)
- External API consumers (authenticated applications and agents)
- Federated QueryBook instances operating under separate administrative domains
- Streaming pipeline subscribers consuming real-time Fact Streams

The STL is not a general-purpose messaging system; it is specifically optimized for the transmission of QBF-encoded Fact Units and Fact Bundles, with native support for Fact-Unit-specific concerns such as SafetyFlags enforcement, FUID-based deduplication, and schema-typed topic routing.

9.2 Protocol Stack

Layer	Name	Protocol	Responsibilities
Layer 1	Framing Layer	QBF-Frame	QBF-encoded Fact Unit or Fact Bundle payload encapsulated in a fixed-structure frame with: 4-byte magic (0x51424654 = "QBFT"), 1-byte frame type, 4-byte payload

Layer	Name	Protocol	Responsibilities
			length, 1-byte flags, payload bytes, 4-byte CRC-32C trailer
Layer 2	Session Layer	QB-Session	Stateful session establishment with handshake (version negotiation, capability exchange, session ID assignment); sequence numbering (64-bit monotonic counter per session); deduplication window (sliding window of received sequence numbers to discard duplicates); heartbeat protocol (configurable interval, default 30 seconds)
Layer 3	Reliability Layer	QB-Reliable	At-least-once delivery: sender retains frame until ACK received; configurable exactly-once mode using idempotency keys (FUID + operation hash); configurable timeout and retry policy; NACK-based fast retransmission; flow control via receiver-advertised window size
Layer 4	Security Layer	QB-Secure	Mutual TLS 1.3 for transport encryption; payload signing with ECDSA P-384 (sender signs payload hash; receiver verifies before processing); optional payload encryption with AES-256-GCM (authenticated encryption, 96-bit nonce, 128-bit tag); certificate pinning for internal service-to-service communication; short-lived certificate rotation (24-hour maximum TTL)
Layer 5	Routing Layer	QB-Route	Topic-based publish-subscribe routing; topic name format: `qb/{ontology_version}/{fact_type}/{domain}` (e.g., `qb/v1.2/ASSERTION/SCIENC`

Layer	Name	Protocol	Responsibilities
			E); FactType determines topic assignment; consumer group semantics (multiple consumers per group share the partition load); partition assignment by SubjectRef hash for co-locality of related Fact Units
Layer 6	Compression Layer	QB-Compress	LZ4 compression for low-latency real-time paths (Fact Stream delivery); Zstandard (level 3) for high-throughput batch paths (Fact Bundle export, bulk ingestion); per-frame compression negotiated at Session Layer handshake; no compression below frame size threshold (default: 512 bytes)

9.3 QB-Streaming Protocol

The QB-Streaming Protocol defines the real-time Fact Stream subscription API, enabling external consumers and internal QueryBook services to receive Fact Units as they are inserted into the Fact Graph.

9.3.1 Subscription Filter Syntax

```
SUBSCRIBE TO FACT_STREAM
 WHERE FactType IN ('ASSERTION', 'EVENT_DESCRIPTION')
 AND Domain = 'SCIENCE'
 AND ConfidenceScore >= 0.70
 AND SubjectRef.EntityType = 'ResearchStudy'
 AND SafetyFlags NOT CONTAINS (HAZARDOUS | RESTRICTED)
 AND TemporalAnchor.T_start >= 1735689600000000000 -- 2025-01-01T00:00:00Z in ns
DELIVERY_GUARANTEE AT_LEAST_ONCE
```

```
CONSUMER_GROUP 'science-monitor-group'
STARTING_OFFSET LATEST
```

9.3.2 Delivery Guarantees

- **AT_LEAST_ONCE:** Every Fact Unit matching the filter is delivered at least once. Consumers must be idempotent (deduplication by FUID).
- **EXACTLY_ONCE:** Each Fact Unit is delivered exactly once per consumer group. Requires transactional consumer with offset commit within processing transaction. Higher latency (p99 +50ms) relative to AT_LEAST_ONCE.

9.3.3 Backpressure Protocol

When a consumer's processing falls behind its subscription throughput, the QB-Streaming Protocol applies the following backpressure sequence:

49. Consumer advertises reduced receive window (Layer 3 flow control)
50. STL Broker holds in-flight Fact Units in per-consumer buffer (max configurable size)
51. If buffer fills to 80% capacity: consumer receives BACKPRESSURE_WARNING frame
52. If buffer fills to 100% capacity: subscription is paused; consumer receives SUBSCRIPTION_PAUSED frame with estimated resume time
53. Consumer reports readiness to resume via RESUME_READY frame; subscription resumes from the most recent uncommitted offset

9.3.4 Consumer Group Semantics

A Consumer Group is a named set of consumer processes that collectively consume a Fact Stream subscription. Partitions of the stream are assigned to individual consumers within the group for parallel processing. Partition assignment is managed by the QB-Stream Coordinator via a modified Kafka-compatible rebalancing protocol, with one significant extension: Fact Units with the same SubjectRef are always assigned to the same partition to ensure ordered delivery for related facts.

9.4 Federation Protocol

Two QueryBook instances may be configured to federate, enabling selective bidirectional synchronization of Fact Units across administrative boundaries.

9.4.1 Federation Handshake

54. Initiating instance presents its Federation Certificate (issued by its Certificate Authority) to the accepting instance
55. Both instances exchange their current Ontology Mesh version manifests
56. CDRE performs ontology compatibility analysis: identifying shared FactTypes, predicate mappings, and any incompatible concepts
57. Both instances negotiate a Federation Trust Level: FULL (all Fact Units eligible for sharing), SELECTIVE (only Fact Units matching a bilateral filter agreement), or READ_ONLY (accepting instance can read from initiating instance but not write back)
58. Both instances sign the negotiated Federation Agreement using their ECDSA P-384 keys; the signed agreement is stored in both instances' Provenance Ledgers

9.4.2 Differential Sync

After initial federation establishment, ongoing synchronization uses a differential sync protocol: each instance maintains a federation offset (equivalent to the last-synchronized Provenance Ledger block ID from the peer). On each sync cycle (configurable frequency, default: every 5 minutes for near-real-time federation), only Fact Units with Provenance Records newer than the last federation offset are transmitted. SafetyFlags are re-evaluated at the receiving instance's Federation Gate before insertion — a Fact Unit that is SAFE in the sending instance may receive additional restrictions in the receiving instance based on that instance's local policies.

9.4.3 Conflict Resolution During Merge

When a federated sync delivers a Fact Unit whose Semantic Hash matches an existing Active Fact Unit in the receiving instance (indicating the same fact has been independently developed in both instances), the Conflict Resolution Protocol (Section 3.3.3) is applied with the following federation-specific modification: source authority is assessed based on the Federation Trust Level of the sending instance, not the internal source reliability weight.

9.5 External API

9.5.1 RESTful API

Endpoint	Method	Description	Auth Required
/v1/fact-units/{fuid}	GET	Retrieve a single Fact Unit by FUID	Bearer token (OAuth 2.0)
/v1/fact-units	POST	Submit a new Fact Unit for ingestion (routes through Ingestion Pipeline)	Bearer token + WRITER role
/v1/fact-units/{fuid}	PATCH	Update mutable fields of an existing Fact Unit (AnnotationSet, ACL, TTL only)	Bearer token + WRITER role + ACL OWNER
/v1/fact-units/{fuid}/lineage	GET	Retrieve full Fact Lineage chain for a Fact Unit	Bearer token + AUDITOR role
/v1/fact-bundles	POST	Create a new Fact Bundle	Bearer token + WRITER role
/v1/fact-bundles/{bundle_id}	GET	Retrieve a Fact Bundle and its member Fact Units	Bearer token
/v1/query	POST	Execute a QB-Query expression; returns ranked Fact Unit result set	Bearer token
/v1/query/regenerate	POST	Execute a QB-Query with REGENERATE AS clause; returns natural language output	Bearer token
/v1/streams/subscribe	POST	Create a Fact Stream subscription; returns a subscription handle	Bearer token + STREAM_CONSUMER role
/v1/streams/{subscription_id}/records	GET	Poll for new Fact Units in a subscription (long-polling, max 30s)	Bearer token + subscription ownership
/v1/safety/flags/{fuid}	GET	Retrieve the SafetyFlags for a Fact Unit (requires AUDITOR role)	Bearer token + AUDITOR role
/v1/safety/flags/{fuid}	PATCH	Upgrade SafetyFlags for a Fact Unit (upgrade only; SAFETY_ADMIN role required)	Bearer token + SAFETY_ADMIN role

Merge note: the two AUDITOR-role endpoints above (Fact Lineage and SafetyFlags retrieval) are part of a broader set of audit-focused endpoints. See §9.5.3, Unified Audit API, for the consolidated audit endpoint set, added in the Version 1.0 merge review.

9.5.2 gRPC API

All RESTful endpoints have equivalent gRPC service definitions. The gRPC API provides lower latency (no HTTP/1.1 overhead), bidirectional streaming (used for long-running Fact Stream subscriptions via server-side streaming RPCs), and native Protocol Buffers v3 payload encoding. The gRPC service definition (proto file) is maintained in the QB-API-PROTO-001 reference document incorporated herein by reference.

9.5.3 Unified Audit API (added in Version 1.0 merge review, Pass 2)

This section consolidates the audit-related capabilities that this Bible already specifies in three separate places — the Provenance Ledger's audit and compliance functions (§5.4), the Safety Substrate's Safety Audit Log (§8.5), and the two AUDITOR-role endpoints already listed in §9.5.1 — into one addressable API surface. It does not change any storage, retention, role, or signing behavior already specified in those sections; it gives a client a single place to call instead of assembling the same functionality from three chapters.

9.5.3.1 Unified Audit Endpoint Table

The five endpoints below follow the same Endpoint / Method / Description / Auth Required format used in the §9.5.1 RESTful API table.

- **POST /v1/audit/export:** creates a full or scoped audit log export (QBF binary, JSON-LD, or NDJSON) using the time range, FUID set, SourceID, AgentID, Operation type, or SafetyFlags filters already defined in §5.4.1. Returns an export_id.
- **GET /v1/audit/export/{export_id}:** retrieves a previously requested export, its ECDSA P-384 signature, and its Merkle proof, per §5.4.1. Requires the AUDITOR role.
- **GET /v1/audit/verify/{export_id}:** verifies an export's Merkle proof against the live Provenance Ledger root without downloading the full export. Requires only a valid Bearer token, not the AUDITOR role — this endpoint exists specifically to enable the third-party verification without full ledger access that §5.4.1 already describes as a design goal.
- **GET /v1/audit/log:** queries the Safety Audit Log (§8.5.1) directly by principal identity, FUID, operation type, or time range, returning tamper-evident access records. Requires the AUDITOR role.

- **GET /v1/audit/soc2-trail:** retrieves the combined SOC 2 Type II audit trail described in §5.4.3 (Provenance Ledger operation log plus Safety Audit Log) for a given time window. Requires the AUDITOR role.

As with every other endpoint in §9.5.1, all five endpoints above have equivalent gRPC service definitions per §9.5.2 and are subject to standard Safety Substrate enforcement (§8.1) and Prime Directive review (Domain 0) before any data is returned.

PART IX — PREDICTIVE INFERENCE ENGINE

Part IX Executive Summary **Part IX specifies the Predictive Inference Engine (PIE) — the component of QueryBook responsible for generating calibrated, probabilistic forward projections of knowledge state evolution. The PIE operates over the current Fact Graph state, temporal Fact Stream trends, embedded causal models, and NCL-learned transition probabilities. Predictions are represented as first-class Fact Units of FactType PREDICTION, enabling predictions to participate in reasoning, be cited in regenerated text, and be tracked for accuracy over time. This Part is the primary reference for patent Cluster L claims.**

Chapter 10: Predictive Inference Engine (PIE) — Complete Specification

10.1 Architecture

The PIE is a probabilistic forward-projection engine that generates predictions about future knowledge states. It integrates four input streams:

59. **Current Fact Graph State:** The complete set of Active Fact Units and their RelationSets, providing the baseline for all predictions
60. **Temporal Trend Analysis:** Statistical analysis of Fact Stream history to identify trends in entity attribute values, event frequencies, and relation changes over time
61. **Causal Models:** Causal Relation Fact Units (FactType = CAUSAL_RELATION) derived by AEM-3, which encode known causal pathways that enable forward projection via causal chain reasoning

62. **Learned Transition Probabilities:** The NCL TMM's learned temporal sequence patterns, which encode the historically observed probability that specific Fact Unit activations follow other Fact Unit activations in time

The PIE operates as a service within the QueryBook microservices architecture, consuming Fact Stream events in near-real-time and periodically updating its internal prediction state. Prediction Fact Units are produced on a configurable schedule (default: every 15 minutes for time-indexed predictions; continuously for event-triggered predictions).

10.2 Prediction Types

Prediction Type	FactType	Input Fact Types Required	Output Description	Primary Application
Entity State Prediction	PREDICTION_ENTITY_STATE	TIME_SERIES_POINT, TREND, ENTITY_ATTRIBUTE	Projected future value of an entity attribute at a specified future time, with confidence interval	Financial forecasting, population projections, equipment failure prediction, stock level prediction
Event Prediction	PREDICTION_EVENT	CAUSAL_CHAIN, EVENT (historical), TREND	Probability that a specified event type will occur within a specified future time window, given current causal chain Fact Units	Risk assessment, opportunity identification, supply chain event prediction, security threat prediction
Relation Change Prediction	PREDICTION_RELATION_CHANGE	ENTITY_ATTRIBUTE (temporal trends), CAUSAL_RELATION	Probability that a specific existing Fact Unit will be superseded within a given time window; includes projected replacement value distribution	Knowledge graph maintenance, fact expiry planning, real-time situation awareness
Knowledge Gap Prediction	PREDICTION_KNOWLEDGE_GAP	Ontology Mesh schema (expected Fact Units), Fact Graph coverage analysis	Identification of entity-predicate pairs that should exist in the Fact Graph based on ontology expectations but are currently absent or low-confidence	Research prioritization, data acquisition planning, knowledge graph completion, information needs analysis
Anomaly Prediction	PREDICTION_ANOMALY	MEASUREMENT, TIME_SERIES_POINT, learned normal distributions	Flag on incoming Fact Unit patterns that deviate significantly from learned norms, with	Fraud detection, sensor malfunction prediction, disinformation detection, process

Prediction Type	FactType	Input Fact Types Required	Output Description	Primary Application
			anomaly score and contributing factor analysis	deviation alerting

10.3 Prediction Confidence and Calibration

10.3.1 Bayesian Credible Intervals

All PIE predictions include Bayesian 95% credible intervals computed via conjugate prior analysis where analytical solutions exist, or via Markov Chain Monte Carlo (MCMC) sampling for complex multi-variable predictions. The credible interval is stored in the PREDICTION Fact Unit's ObjectValue as a structured type: `{point_estimate: float64, lower_bound_95: float64, upper_bound_95: float64}`.

10.3.2 Monte Carlo Sampling over Scenario Graphs

For Event Predictions and complex multi-step projections, the PIE constructs a Scenario Graph (Chapter 11) with multiple branches representing different possible future knowledge states. Monte Carlo sampling over this Scenario Graph — sampling 10,000–100,000 paths weighted by branch transition probabilities — produces an empirical distribution of outcomes from which confidence intervals are derived. This approach naturally handles non-Gaussian uncertainty distributions and causal non-linearities.

10.3.3 Historical Calibration

The PIE continuously evaluates the accuracy of past predictions by comparing them against subsequently ingested Fact Units. For each expired PREDICTION Fact Unit (i.e., a Fact Unit whose prediction horizon has passed), the PIE computes a calibration score: the proportion of predictions with stated 95% credible intervals that actually contained the true observed value. A well-calibrated PIE achieves a calibration score of approximately 0.95. Systematic miscalibration triggers automatic recalibration of the PIE's prior distributions for the affected prediction type and domain.

10.4 Prediction Lifecycle

Prediction Fact Units are first-class Fact Units with all the lifecycle, provenance, and safety properties of any other Fact Unit, plus the following PREDICTION-specific fields stored in the AnnotationSet:

Annotation Key	Type	Description
qb:prediction.horizon_ns	int64	The UTC Unix nanosecond timestamp at which this prediction is expected to either be confirmed or refuted by new facts
qb:prediction.conditioning_fuids	List of FUID	The set of Fact Units upon which this prediction is conditioned; changes to conditioning Fact Units trigger prediction revalidation
qb:prediction.confidence_interval	JSON struct	Point estimate and 95% credible interval bounds
qb:prediction.method	enum string	The prediction method used: BAYESIAN_CONJUGATE, MCMC, CAUSAL_CHAIN, TEMPORAL_REGRESSION, SCENARIO_GRAPH_SAMPLING
qb:prediction.outcome	enum string | null	Populated after the horizon passes: CONFIRMED, REFUTED, INCONCLUSIVE, HORIZON_EXPIRED_NO_DATA
qb:prediction.calibration_score	float32 | null	Populated after outcome is determined: calibration quality score for this prediction instance

PREDICTION Fact Units are automatically transitioned to ARCHIVED state by the TTL eviction mechanism when their qb:prediction.horizon_ns has passed and the outcome has been determined. Predictions for which no confirming or refuting facts are available after a configurable grace period (default: 30 days post-horizon) are archived with outcome HORIZON_EXPIRED_NO_DATA.

10.4 Manifold-Based Trend Forecasting

This section extends the PIE with an additional probabilistic forecasting method for genuinely time-series-shaped external phenomena — weather, financial markets, and comparable trend-prediction domains — using nonlinear manifold-learning techniques with real, established precedent in this application (structured manifold learning recovering chaotic attractors from financial time series; t-SNE-based weather pattern analysis; Isomap and locally linear embedding for stock price forecasting). This section extends PIE's already-probabilistic forecasting function; it does not modify the Fact Unit schema (Appendix A), the Ontology Mesh (Chapter 6), or any grounded generation or retrieval mechanism, and it is not a re-introduction of the manifold-based knowledge-representation concept already declined multiple times elsewhere in this Bible's history.

STRUCTURALLY SEPARATE FROM THE DETERMINISTIC CORE, AND LABELED ACCORDINGLY This mechanism produces probabilistic trend forecasts for external phenomena; it never produces, modifies, or influences a grounded Fact Unit, and its output is never presented with the certainty of a retrieved fact. Every output under this section carries an explicit confidence interval and is labeled as a probabilistic forecast at the point of presentation. This is the same structural-separation discipline already applied to the Contextual Commerce Layer (Chapter 88) and NFT Integration (Chapter 94) — a real capability, walled off from the deterministic core rather than merged into it. This section does not alter PIE's determinism-adjacent guarantees for any other prediction type already specified at §10.2–§10.3.

10.4.1 Scope

- Applicable domains: time-series phenomena where the underlying system is genuinely nonlinear and probabilistic by nature — weather and climate trends, financial market behavior (reusing the Ontology Mesh's already-registered Finance domain, §20.3 of the patent), and comparable structured time-series data an operator ingests for this purpose.
- Not applicable: this section is never used to represent, generate, or infer a grounded Fact Unit, a Fact Unit relationship, or any content subject to the structural-groundedness guarantee (Claim 8). A forecast produced here is a distinct output type, never merged into or confused with the Fact Graph.

10.4.2 Method

Ingested time-series data (Chapter 4) is embedded into a low-dimensional manifold using an established nonlinear dimensionality-reduction technique appropriate to the domain and data characteristics — for example, phase-space reconstruction for detecting regime changes in financial time series, or comparable manifold-learning methods for weather-pattern trend analysis. The resulting forecast is generated with an explicit confidence interval computed from the method's own uncertainty characteristics, following the same calibration discipline already required of every other PIE prediction type at §10.3.

10.4.3 Presentation Requirement

A Manifold-Based Trend Forecast is presented to a user in a distinct, source-labeled region of the response — never interleaved with grounded Fact Unit content — stating explicitly that the output is a probabilistic forecast, the confidence interval, and the forecasting domain. This mirrors the presentation discipline already required for commercial content under §88.3, adapted to a forecast rather than a commercial item.

10.4.4 Integration Requirements

This section requires the PIE's existing architecture and calibration discipline (§10.1–§10.3); the Multimodal Ingestion Pipeline (Chapter 4); and, where financial forecasting is used, the Ontology Mesh's registered Finance domain (§20.3 of the patent). It modifies none of their specifications, and modifies nothing in the Fact Unit schema, Ontology Mesh, or grounded generation pipeline.

PART X — SCENARIO GRAPH SYSTEM

Part X Executive Summary **Part X specifies the Scenario Graph system — QueryBook's representation for branching, probabilistic hypothetical and projected knowledge states. Each Scenario Graph node is a Knowledge State Snapshot (KSS), and edges are probabilistic transitions between states. Scenario Graphs power what-if analysis, forecasting, adversarial modeling, counterfactual reasoning, and strategic planning within QueryBook. This Part is the primary reference for patent Cluster F claims.**

Chapter 11: Scenario Graph — Complete Specification

11.1 Definition

A Scenario Graph is a branching, probabilistic directed graph over the Fact Graph. Each node in the Scenario Graph is a Knowledge State Snapshot (KSS) — a consistent sub-graph of Fact Units representing a coherent hypothetical or projected world state. Each edge in the Scenario Graph is a Transition: a triple of (Condition, Transition Probability, Target KSS) specifying the probability that, given the source KSS, the world evolves to the target KSS when the condition is satisfied or assumed.

Scenario Graphs differ fundamentally from the Fact Graph in the following ways:

- The Fact Graph represents asserted, provenance-tracked knowledge about the actual world. Scenario Graph nodes represent hypothetical or projected states that may or may not correspond to reality.

- Fact Graph edges are typed semantic relations between Fact Units. Scenario Graph edges are probabilistic transitions between world states.
- Fact Graph Fact Units carry ConfidenceScores reflecting epistemic certainty. Scenario Graph transition probabilities reflect scenario likelihood, not epistemic certainty about current state.

11.2 Scenario Graph Schema

Field	Type	Description
ScenarioGraphID	UUID v7	Globally unique identifier for this Scenario Graph
Title	UTF-8 string (max 512 chars)	Human-readable title for this Scenario Graph
RootKSS	KSSID (UUID)	The baseline Knowledge State Snapshot from which all branches originate; usually a snapshot of the current Fact Graph state at Scenario Graph creation time
KSSSet	Map of {KSSID → KnowledgeStateSnapshot}	All Knowledge State Snapshots in this Scenario Graph
TransitionSet	Set of Transition {SourceKSSID, Condition: FU, Probability: float32, TargetKSSID}	All transition edges in this Scenario Graph; Condition is a Fact Unit assertion whose truth triggers the transition
DepthLimit	uint32	Maximum branching depth from the RootKSS; prevents combinatorial explosion in deep scenario trees
HorizonTime	int64 (Unix nanoseconds UTC)	The temporal horizon for the scenario; leaf KSS nodes represent world states at or before this time
ScenarioType	enum {WHAT_IF, FORECAST, ADVERSARIAL, COUNTERFACTUAL, PLANNING}	Classification of the Scenario Graph's purpose; governs applicable construction algorithms and pruning policies
PruningThreshold	float32	Minimum cumulative transition probability for a branch to be retained; branches with cumulative probability below this threshold are pruned during construction (default: 0.01, i.e., 1%)
OwnerRef	PrincipalID (UUID)	The principal that created and owns this Scenario Graph
AccessPolicy	AccessControlList	Access control governing who may view, modify, or execute this Scenario Graph
CreationTime	int64 (Unix nanoseconds UTC)	Timestamp at which this Scenario Graph was created
LastUpdatedTime	int64 (Unix nanoseconds UTC)	Timestamp of the most recent modification; updated by PIE when

Field	Type	Description
		new predictions are incorporated

Knowledge State Snapshot (KSS) Schema

Field	Type	Description
`KSSID`	UUID v7	Unique identifier for this KSS within the Scenario Graph
`BaseFUIDs`	List of FUID	Fact Units from the Fact Graph that are inherited unchanged into this KSS
`OverrideFacts`	List of Fact Unit (inline, SYNTHETIC flag set)	Synthetic Fact Units that represent the hypothetical state changes in this KSS; may override (shadow) Fact Units in BaseFUIDs
`RemovedFUIDs`	List of FUID	Fact Units from BaseFUIDs that are assumed not to hold in this KSS (used for counterfactual branches)
`ConsistencyStatus`	enum {CONSISTENT, INCONSISTENT, UNCHECKED}	Result of CDRE consistency validation for this KSS; INCONSISTENT KSS nodes are flagged but retained for analysis
`CumulativeProbability`	float32	Product of all transition probabilities on the path from RootKSS to this KSS; used for pruning and result weighting

11.3 Scenario Graph Construction

11.3.1 Automated Construction from PIE

The PIE automatically constructs FORECAST and ADVERSARIAL Scenario Graphs from its prediction outputs. The construction algorithm is:

63. Initialize RootKSS from the current Fact Graph state (snapshot of all Active Fact Units relevant to the prediction domain)
64. For each PREDICTION Fact Unit generated by the PIE, create a branch: one KSS where the prediction is true (OverrideFacts includes the PREDICTION content as a concrete fact), one where it is false (RemovedFUIDs includes the conditioned Fact Units, OverrideFacts includes the negation), connected to the RootKSS with probabilities equal to the prediction ConfidenceScore and (1 - ConfidenceScore) respectively

65. Recursively expand branches up to DepthLimit, applying PIE predictions conditioned on each branch's KSS state
66. Apply pruning: remove branches with CumulativeProbability below PruningThreshold
67. Apply CDRE consistency validation to all leaf KSS nodes; flag INCONSISTENT nodes

11.3.2 Human-in-the-Loop Scenario Authoring

The Scenario Graph API provides an authoring interface for human experts to construct WHAT_IF, COUNTERFACTUAL, and PLANNING Scenario Graphs with precise control over branch conditions and transition probabilities. The authoring API supports: creating new KSS nodes, specifying OverrideFacts and RemovedFUIDs manually, setting transition probabilities based on expert judgment, and importing PREDICTION Fact Units as initial branches.

11.4 Scenario Graph Execution and Query

11.4.1 Cross-Branch Query

QB-Query's ACROSS SCENARIOS clause (Chapter 18) enables querying across all branches of a Scenario Graph simultaneously. Results are returned with a per-result CumulativeProbability weight, enabling computation of expected values, variance analysis, and identification of robust vs. contingent answers.

11.4.2 Robust Fact Identification

A Robust Fact is a Fact Unit (or its equivalent OverrideFact) that is true in all KSS nodes of a Scenario Graph above the PruningThreshold. Robust facts are the knowledge that remains reliable regardless of which scenario branch materializes — the highest-confidence output of scenario analysis. The ACROSS SCENARIOS query engine identifies Robust Facts by computing the intersection of Fact Unit sets across all branches weighted by CumulativeProbability.

11.4.3 Scenario Comparison (KSS Diff)

Given two KSSID references, the Scenario Comparison operation produces a structured diff: Fact Units present in KSS-A but not KSS-B (KSS-A unique facts), Fact Units present in KSS-B but not KSS-A (KSS-B unique facts), Fact Units present in both but with different ObjectValues (conflicting facts), and Fact Units present in both with identical content (shared facts).

11.4.4 Scenario Merging

When two branches of a Scenario Graph converge to the same KSS (i.e., different condition sequences lead to the same world state), the system detects this convergence by computing the KSS content hash and merging the two branches into a single KSS node. The merged node's CumulativeProbability is the sum of the merged branches' cumulative probabilities (probability union for independent paths to the same state).

PART XI — NARRATIVE GRAPH SYSTEM

Part XI Executive Summary **Part XI specifies the Narrative Graph system — the directed acyclic graph representation that bridges the machine-readable Fact Graph and human-readable natural language output. The Narrative Graph organizes selected Fact Units by temporal, causal, and rhetorical relations to enable structured, coherent, grounded text generation by the Multilingual Regeneration Engine. This Part is the primary reference for patent Cluster G claims and is closely related to the MRE specification in Part XV.**

Chapter 12: Narrative Graph — Complete Specification

12.1 Definition

A Narrative Graph (NG) is a directed acyclic graph (DAG) over a selected, ordered subset of Fact Units from the Fact Graph, organized by the temporal, causal, and rhetorical relations among them, for the purpose of producing human-readable knowledge output through the Multilingual Regeneration Engine. The Narrative Graph is the semantic plan for a piece of regenerated text: it determines what facts to include, in what conceptual order, with what inter-fact relations emphasized, for what audience, in what language.

The Narrative Graph is a first-class QueryBook system object. It can be constructed automatically by the MRE in response to a QB-Query with REGENERATE AS clause, or manually authored via the Narrative Graph API for editorial control over regenerated knowledge products (reports, summaries, explanations, briefings).

12.2 Narrative Graph Schema

Field	Type	Description
`NarrativeGraphID`	UUID v7	Globally unique identifier for this Narrative Graph
`Title`	FUID (FactType = ASSERTION or DEFINITION)	Reference to the Fact Unit that serves as the title/topic anchor of this narrative
`Summary`	FUID | null	Optional reference to a Fact Unit that summarizes the entire narrative; placed first in linearization if present
`NodeSet`	Ordered Set of FUID	The complete set of Fact Units included in this Narrative Graph, in the initial proposed reading order
`EdgeSet`	Set of NarrativeEdge {SourceFUID, EdgeType, TargetFUID, Weight: float32}	The set of narrative relation edges between Fact Unit nodes
`EdgeTypes`	Enum set (see below)	The complete set of valid narrative edge types used in this Narrative Graph
`ReadingOrder`	Ordered List of FUID	The finalized topological sort of the DAG representing the canonical reading order; computed by the linearization algorithm
`IntendedAudience`	AudienceModelRef (UUID → Audience Model Registry)	Reference to an audience model specifying vocabulary level, domain expertise, detail level, and accessible concept set
`OutputLanguage`	BCP-47 language tag	The target language for MRE regeneration from this Narrative Graph
`NarrativeStyle`	enum {EXPOSITORY, ARGUMENTATIVE, DESCRIPTIVE, ANALYTICAL, INSTRUCTIONAL, CONVERSATIONAL, EXECUTIVE_SUMMARY}	The rhetorical mode governing sentence structure, connective vocabulary, and discourse organization in the regenerated output
`CoherenceScore`	float32 | null	Computed by the MRE after linearization: a score in [0.0, 1.0] measuring narrative flow, topic continuity, and causal completeness; null if not yet computed
`MaxOutputTokens`	uint32 | null	Optional constraint on the maximum length of the regenerated output; MRE selects the most important Fact Units to fit within the constraint

Narrative Edge Types

Edge Type	Semantics	Example Usage
TEMPORAL_FOLLOWS	TargetFUID describes an event or state that temporally follows SourceFUID	"First X happened, then Y"
CAUSES	SourceFUID describes a causal factor for the state/event in TargetFUID	"X caused Y"
CONTRASTS_WITH	TargetFUID provides a contrast or counterpoint to SourceFUID	"However, Y contrasts with X"
ELABORATES	TargetFUID provides additional detail or elaboration on SourceFUID	"More specifically, Y elaborates on X"
EXEMPLIFIES	TargetFUID provides a concrete example of the general statement in SourceFUID	"For example, Y illustrates X"
SUMMARIZES	TargetFUID is a higher-level summary or abstraction of SourceFUID content	"In summary, Y captures X"
QUOTES	TargetFUID is a direct quotation from the source or agent that produced SourceFUID	Direct attribution in text
SUPPORTS	TargetFUID provides supporting evidence for the claim in SourceFUID	"This is supported by Y"
QUALIFIES	TargetFUID introduces a qualification, exception, or boundary condition for SourceFUID	"However, Y qualifies X under certain conditions"

12.3 Narrative Graph Construction

12.3.1 Automatic Construction by the MRE

When the MRE receives a query result set (ranked list of Fact Units) and a REGENERATE AS specification, it automatically constructs a Narrative Graph through the following algorithm:

68. **Topic anchoring:** Identify the highest-confidence, most query-relevant Fact Unit as the Title anchor
69. **Edge inference:** For each pair of Fact Units in the result set, examine their RelationSet edges in the Fact Graph to determine applicable Narrative Edge Types (CAUSAL_RELATION edges map to CAUSES; temporal ordering from TemporalAnchors maps to TEMPORAL_FOLLOWS; SUPPORTS_CLAIM edges map to SUPPORTS, etc.)

70. **DAG construction:** Build the DAG from inferred edges; if cycles would be introduced (e.g., circular causal claims), break cycles by selecting the edge with the lowest relation weight for removal
71. **Gap detection:** Identify missing causal links (a CAUSES chain with intermediate steps not in the result set) and missing temporal context (temporal jumps exceeding the audience model's expected context window); flag gaps for potential gap-filling or explicit acknowledgment in the output
72. **Coherence scoring:** Compute CoherenceScore as a weighted combination of: topic continuity score (fraction of adjacent Fact Units in ReadingOrder that share at least one entity reference), causal completeness score (fraction of CAUSES edges with complete intermediate chains), and audience compatibility score (fraction of Fact Units within the audience model's expertise level)

12.3.2 Manual Curation API

The Narrative Graph API provides operations for editorial control: add/remove NodeSet members, add/remove/modify EdgeSet entries, override ReadingOrder, set NarrativeStyle, constrain MaxOutputTokens. All manual curation operations are versioned and logged in the Provenance Ledger under the Narrative Graph's ProvRef.

12.4 Narrative Graph Execution

12.4.1 Linearization Algorithm

The linearization algorithm converts the Narrative Graph DAG into a linear ReadingOrder using a modified Kahn's topological sort with the following priority rules:

73. Nodes with SUMMARIZES edges pointing to them from later nodes are promoted to the front (executive summary first)
74. Among nodes with equal topological depth, prefer nodes with higher CoherenceScore contribution (nodes that maximize topic continuity with the previously emitted node)
75. TEMPORAL_FOLLOWS edges enforce a hard ordering constraint: the SourceFUID must appear before the TargetFUID in the ReadingOrder

76. CONTRASTS_WITH edges create soft adjacency preferences: contrasting Fact Units are placed consecutively for rhetorical effect

12.4.2 Gap Detection

After linearization, the MRE analyzes the ReadingOrder for the following gap types:

- **Temporal gap:** Adjacent Fact Units in ReadingOrder whose TemporalAnchors are separated by more than the audience model's expected narrative time grain (e.g., jumping from 1850 to 1950 without acknowledging the 100-year gap)
- **Causal gap:** A CAUSES chain where the intermediate steps are not in the NodeSet, leaving the reader with an unexplained causal jump
- **Entity introduction gap:** A Fact Unit references an entity that has not been introduced in any prior NodeSet member

Detected gaps are either (a) automatically filled by querying the Fact Graph for gap-filling Fact Units that bridge the gap with high confidence, (b) explicitly acknowledged in the generated text with phrases indicating the gap ("*over the following century…*"), or (c) flagged in the Narrative Graph metadata as known limitations.

12.4.3 Style Enforcement

The MRE enforces NarrativeStyle constraints during lexical selection and syntactic realization. Style-specific properties:

- **EXPOSITORY:** Present tense where applicable; declarative sentence structures; minimal hedging language; transition phrases (furthermore, additionally, in contrast)
- **ARGUMENTATIVE:** Explicit claim-evidence-warrant structure; hedging language for contested facts; acknowledgment of counterarguments via CONTRASTS_WITH edges
- **ANALYTICAL:** Causal language emphasized; quantitative precision where ConfidenceScore is high; conditional language for lower-confidence assertions; explicit uncertainty acknowledgment
- **INSTRUCTIONAL:** Sequential imperative structure; numbered steps for TEMPORAL_FOLLOWS chains; definition of technical terms on first use based on audience model

- **EXECUTIVE_SUMMARY:** Summary Fact Units prioritized; 3-sentence-per-topic maximum; quantitative highlights; ELABORATE edges suppressed to stay within token budget

PART XII — QUERYBOOK FORMAT (QBF)

Part XII Executive Summary **Part XII specifies the QueryBook Format (QBF) — the proprietary binary serialization format for all Fact Units, Fact Bundles, and associated metadata in QueryBook. QBF is designed to be compact, self-describing, streamable, and backward-compatible. This Part provides the complete byte-level specification of the QBF file structure, field encoding rules, streaming mode framing, and versioning policies. This Part is the primary reference for patent Cluster D claims.**

Chapter 13: QueryBook Format (QBF) — Complete Binary Specification

13.1 Design Goals

QBF is designed to satisfy the following requirements simultaneously, without trade-offs:

- **Compact:** A typical Fact Unit (excluding EmbeddingVector) serializes to approximately 512–2,048 bytes in QBF; the EmbeddingVector adds 16,384 bytes (4096 float32 values) and may be stored separately with a pointer
- **Self-describing:** Every QBF payload carries its schema version, enabling parsing without external schema lookup
- **Streamable:** Valid QBF can be parsed as a stream without random access to the full file; each record is independently decodable
- **Backward-compatible:** New fields are always optional and additive; older parsers skip unknown fields using the length-prefix on each field
- **Integrity-verifiable:** The trailer contains a CRC-32C checksum and an optional ECDSA signature enabling tamper detection

Encoding Layer Version Skew — Redesign Note (Economy)

REDESIGN FINDING — VERSIONING RISK **QBF uses Protocol Buffers varint encoding for its internal integer fields, while its overall frame structure (type, length, flags, payload, CRC-32C trailer) is a custom format, not itself a protobuf message. This creates two independently versioned encoding surfaces — a future change to either the varint field encoding rules or the custom frame layout could be made without the other being updated in lockstep, and this Bible's existing versioning compatibility guidance does not yet call this out as a coupled pair.**

Recommended mitigation: add an explicit rule to this chapter's version compatibility table stating that the frame-format version and the payload field-encoding version are tracked and negotiated together as a single compatibility unit, not independently — a reader implementing a QBF parser or writer should not be able to mix a newer frame format with an older field-encoding scheme (or vice versa) without an explicit compatibility check failing loudly. This is a low-cost documentation fix relative to the parsing-ambiguity or silent-corruption cost of not having it.

13.2 QBF File Structure

```
QBF FILE LAYOUT (CANONICAL FORMAT)
====================================

[MAGIC BYTES] 4 bytes 0x51 0x42 0x46 0x00 ("QBF\0")
[MAJOR VERSION] 1 byte Currently: 0x01
[MINOR VERSION] 1 byte Currently: 0x00

[HEADER BLOCK] 256 bytes (fixed-length)
 Total payload size 8 bytes uint64, little-endian
 Schema version 8 bytes uint64 (Ontology Mesh version as MAJOR<<32|MINOR<<16|PATCH)
data-id="2502"
 Compression algo 1 byte 0x00=NONE, 0x01=LZ4, 0x02=ZSTANDARD, 0x03=GZIP
 Encryption flag 1 byte 0x00=PLAINTEXT, 0x01=AES-256-GCM
```

Checksum algo 1 byte 0x00=CRC32C, 0x01=SHA256, 0x02=SHA3-256

Record type 1 byte 0x01=FACT_UNIT, 0x02=FACT_BUNDLE, 0x03=MIXED

Creation timestamp 8 bytes int64, Unix nanoseconds UTC, little-endian

FUID count 8 bytes uint64, number of Fact Units in this file

Index offset 8 bytes uint64, byte offset of Index Block (0 if no index)

Encryption IV 12 bytes AES-GCM initialization vector (0 if unencrypted)

Encryption tag 16 bytes AES-GCM authentication tag (0 if unencrypted)

Reserved 180 bytes Must be 0x00; reserved for future header fields

[SCHEMA BLOCK] Variable length (if non-standard schema)

Block length 4 bytes uint32, length of schema descriptor in bytes

Schema descriptor N bytes Compact schema descriptor (Protocol Buffers v3 encoding)

[PAYLOAD BLOCK] Variable length

(Serialized Fact Units or Fact Bundles; see Section 13.3)

[INDEX BLOCK] Optional, variable length

Entry count 4 bytes uint32, number of index entries

Per entry:

FUID 16 bytes UUID (16-byte big-endian)

Byte offset 8 bytes uint64, offset of this record in Payload Block

[TRAILER BLOCK] 64 bytes (fixed-length)

CRC-32C checksum 4 bytes CRC-32C of entire file excluding trailer

Checksum type 1 byte Same encoding as header checksum algo field

Extended hash 32 bytes SHA-256 or SHA-3/256 of payload block (if enabled)

ECDSA sig length 2 bytes uint16, length of ECDSA signature (0 if unsigned)

ECDSA signature N bytes ECDSA P-384 signature over CRC-32C + extended hash

> EOF marker 4 bytes 0x51 0x42 0x45 0x4F ("QBEO" = QueryBook End Of file)
>
> Pad to 64 bytes Remaining Reserved 0x00 bytes

13.3 QBF Field Encoding

Field Type	Encoding	Details
Integer fields (int32, int64)	Protocol Buffers varint	Variable-length ZigZag encoding for signed integers; standard varint for unsigned integers; max 10 bytes for 64-bit values
Float fields (float32, float64)	IEEE 754	float32: 4-byte little-endian IEEE 754 single-precision; float64: 8-byte little-endian IEEE 754 double-precision; NaN and Infinity are forbidden in ConfidenceScore fields
String fields (UTF-8)	Length-prefixed UTF-8	4-byte uint32 length prefix followed by UTF-8 encoded bytes; must be valid UTF-8; null strings encoded as length=0
Binary fields (bytes)	Length-prefixed bytes	4-byte uint32 length prefix followed by raw bytes; no encoding applied
Vector fields (float32[])	Dimensioned float32 array	4-byte uint32 dimension prefix followed by dimension × 4-byte IEEE 754 float32 values, little-endian; EmbeddingVector dimensions may be stored in a separate sidecar file referenced by content-addressed pointer to avoid inflating the primary record size
Timestamp fields	64-bit Unix nanoseconds	int64 little-endian; nanoseconds since Unix epoch (January 1, 1970, 00:00:00 UTC); negative values represent pre-epoch timestamps
UUID fields	16-byte big-endian	128-bit UUID stored as 16 raw bytes in network byte order (big-endian); no hyphens or formatting
Set fields (repeated elements)	Length-prefixed repeated elements	4-byte uint32 element count followed by sequentially encoded elements; each element is self-describing using the same type encoding as its scalar equivalent
Map fields (key-value)	Length-prefixed key-value pairs	4-byte uint32 pair count followed by sequentially encoded (key, value) pairs; key is always a length-prefixed UTF-8 string; value encoding depends on declared

Field Type	Encoding	Details
		value type
Reference fields (FUID pointers)	16-byte big-endian UUID	References to other Fact Units are encoded as the raw 16-byte UUID of the target FUID; the parsing system is responsible for resolving references against the active Fact Graph
Enum fields	Protocol Buffers varint	Enum values encoded as their uint32 ordinal using varint encoding; unknown enum values from newer schema versions must be preserved as their raw ordinal and not discarded by older parsers
Boolean fields	1-byte uint8	0x00 = false; 0x01 = true; any other value is an encoding error
Null fields	Presence bit in field header	Each optional field is preceded by a 1-bit presence indicator in the record's field presence bitmask; if the bit is 0, the field is absent (null) and no bytes follow for that field

13.4 QBF Streaming Mode

For Fact Streams (real-time Fact Unit delivery over the STL), QBF Streaming Mode is used instead of the full file structure. In Streaming Mode, each record is independently framed and can be parsed without knowledge of subsequent records:

```
QBF STREAMING MODE RECORD FRAME
==================================

[FRAME HEADER] 10 bytes (fixed)
 Length prefix 4 bytes uint32, little-endian: byte length of Record Body
 Record type 1 byte 0x01=FACT_UNIT, 0x02=FACT_BUNDLE,
 0x03=CONTROL, 0x04=HEARTBEAT
 Sequence number 4 bytes uint32, monotonically increasing per session
 Flags 1 byte bit 0=compressed, bit 1=encrypted, bits 2-7=reserved

[RECORD BODY] Variable (length indicated by Length prefix)
```

Schema version 4 bytes uint32, Ontology Mesh minor version of this record

Payload N bytes QBF-encoded Fact Unit, Fact Bundle, or control message

[FRAME TRAILER] 5 bytes (fixed)

CRC-32C 4 bytes CRC-32C of Frame Header + Record Body

End marker 1 byte 0xFE (record end sentinel for stream synchronization)

NOTES:

- If Frame Header.Flags.compressed=1: Record Body is LZ4-compressed

- If Frame Header.Flags.encrypted=1: Record Body is AES-256-GCM encrypted

- If any Record Body byte is corrupted (CRC mismatch), the parser scans forward for the next 0xFE sentinel byte to re-synchronize the stream

- HEARTBEAT record type carries a 64-bit UTC nanosecond timestamp; receiver must respond within 5 seconds or the session is considered dead

13.5 QBF Versioning and Compatibility

Change Category	Version Bump	Backward Compatible?	Migration Required?
Add new optional field	Minor (0.x → 0.x+1)	Yes — older parsers skip unknown fields	No
Add new optional enum value	Minor	Yes — older parsers preserve unknown enum values as raw ordinals	No
Add new required field	Major (x → x+1)	No — older parsers reject records missing required fields	Yes — all existing Fact Units must be re-serialized
Remove existing field	Major	No — older parsers expect the field	Yes
Change field type	Major	No	Yes
Change compression or encryption defaults	Minor	Yes — both old and new formats declared in Header Block	No
Bug fix in field encoding (no semantic change)	Patch (x.y → x.y+1)	Yes	Optional (recommended for affected records)

The minimum supported parser version policy requires all production QueryBook deployments to support QBF versions within 2 major versions of the current release. Deployment operators receive 18 months advance notice before any major version is deprecated.

PART XIII — STORAGE ARCHITECTURE

Part XIII Executive Summary **Part XIII specifies the five-tier storage architecture that persists all QueryBook Fact Units, the Fact Graph, vector indexes, and the Provenance Ledger across the full data temperature spectrum from nanosecond in-memory access to long-term cold archive. It covers HNSW vector index architecture for semantic search at 10B+ Fact Unit scale, sharding and replication strategy, and backup and disaster recovery specifications. This Part is the primary reference for patent Cluster N claims.**

Chapter 14: QueryBook Storage Architecture — Complete Specification

14.1 Storage Layer Overview

Tier	Name	Technology	Fact Unit States Stored	Target Latency	Retention Policy
Tier 0	Hot Cache	In-memorydistributed cache (Redis-compatible API); implemented with custom QB-aware eviction policies	ACTIVE (frequently accessed)	< 1ms	LRU eviction; maximum configurable memory allocation (default: 512GB per cluster)
Tier 1	Active Store	SSD-backed distributed key-value store (RocksDB-compatible API); optimized for random read/write of Fact Unit records by FUID	ACTIVE, VALIDATED	< 10ms	All Active Fact Units retained indefinitely until lifecycle state transition

Tier	Name	Technology	Fact Unit States Stored	Target Latency	Retention Policy
Tier 2	Warm Store	NVMe-backed columnar store (Apache Parquet-format internal layout); optimized for scan-heavy analytics queries and batch operations	SUPERSEDED (recent); ACTIVE (infrequently accessed)	< 100ms	Fact Units accessed within past 90 days; auto-promoted to Tier 1 on access
Tier 3	Cold Archive	S3-compatible object storage; Fact Units stored as QBF Bundles of 10,000 records per object; content-addressed	ARCHIVED, SUPERSEDED (older than 90 days)	< 1s	Retained per regulatory retention policy; minimum 7 years default
Tier 4	Deep Archive	Cold object storage or tape archive (Glacier-compatible API); accessed only for compliance audits or forensic retrieval	ARCHIVED (compliance-retained); PURGED tombstone records	< 1 hour	Per regulatory mandate; cryptographic tombstones retained indefinitely

14.1.1 Tier Promotion and Demotion

Fact Units move between storage tiers based on access frequency and lifecycle state:

- **Promotion (cold → hot):** Any Fact Unit retrieval from Tier 2 or below triggers an async promotion to Tier 1 and Tier 0 caching, with TTL proportional to the access recency score
- **Demotion (hot → cold):** Tier 0 LRU eviction pushes to Tier 1; Tier 1 access-frequency analysis (computed nightly) demotes infrequently accessed ACTIVE Fact Units to Tier 2; lifecycle state transitions to SUPERSEDED or ARCHIVED trigger migration to Tier 3
- **Tier 4 migration:** Triggered only by explicit policy rules (compliance mandate, deep retention requirement) and requires dual-approval workflow

14.2 Fact Graph Storage

The Fact Graph is stored as a persistent property hypergraph. The underlying storage representation uses a hybrid adjacency list and inverted index structure:

Structure	Storage Format	Index Type	Purpose
Vertex records (Fact Units)	QBF-encoded records in Tier 1 Active Store keyed by FUID	B-tree on FUID (primary)	O(1) Fact Unit retrieval by FUID
Outgoing edge lists (RelationSet)	Compressed adjacency list keyed by (FUID, PredicateRef)	B-tree on composite key	O(log n) traversal from a vertex along a typed edge
Incoming edge index	Inverted index keyed by (TargetFUID, PredicateRef)	B-tree on composite key	O(log n) reverse traversal: "which Fact Units point to this one with this predicate?"
Subject index	Inverted index keyed by SubjectRef EntityID	B-tree on EntityID	O(log n) retrieval of all Fact Units about a given entity
Predicate index	Inverted index keyed by PredicateRef UUID	B-tree on PredicateRef	O(log n) retrieval of all Fact Units using a given predicate
Semantic Hash index	Hash map keyed by SemanticHash	Hash table with chaining	O(1) average-case deduplication check on ingestion
FactType index	Inverted index keyed by FactType OntologyID	B-tree on FactType	O(log n) retrieval of all Fact Units of a given type
Temporal index	Range-tree keyed by (T_start, T_end)	Interval tree	O(log n + k) temporal range queries where k is the result set size

The consistency model for the Fact Graph is eventual consistency with causal ordering guarantees: all writes that causally follow a given write are guaranteed to be visible to any reader that has seen that write. Within a single FUID's history, reads always reflect the most recent causally ordered write, providing read-your-writes consistency for Fact Unit lifecycle operations.

14.3 Vector Index Architecture

EmbeddingVectors for all Active Fact Units are indexed in a Hierarchical Navigable Small World (HNSW) graph index, enabling approximate nearest neighbor (ANN) search across the full 4096-dimensional semantic embedding space.

14.3.1 HNSW Configuration

Parameter	Value	Rationale
Embedding dimensionality (M_dim)	4096	Default EmbeddingVector dimensionality; configurable per deployment
M (connections per node)	32	Balances recall (higher M = better recall) against index size and construction time
ef_construction	400	Search width during index construction; higher value = better index quality at cost of build time
ef_search	200	Search width during ANN query; tunable per query for recall/latency trade-off
Distance metric	Cosine similarity (inner product on L2-normalized vectors)	All EmbeddingVectors are L2-normalized at ingestion; inner product equals cosine similarity
Max elements per index shard	100 million Fact Units	Index shards are partitioned by FUID prefix; 4096 virtual shards support 400B+ total Fact Units
Target ANN search latency	< 50ms at p99 for 10B Fact Units	Achieved via distributed shard-parallel search with result merging
Recall target (@K=100)	> 0.95	95% recall against exact brute-force search at K=100 nearest neighbors

14.3.1.1 Memory Footprint at Scale — Redesign Note (Performance)

REDESIGN FINDING — PERFORMANCE **A raw float32 HNSW index at 4096 dimensions and 10B+ Fact Unit scale (§14.3.1) carries a very large memory footprint: 10^{10} vectors × 4096 dimensions × 4 bytes ≡ on the order of 160+ TB for vector storage alone before graph-link overhead and the 3-replica minimum already specified for qb-vector-index (§19) are applied — realistically reaching into the petabyte range once replicated. This was not previously stated as an explicit constraint anywhere in this chapter, and the exact multiplier depends on implementation details (graph fan-out, per-shard overhead) this Bible does not yet specify.**

Recommended mitigation: apply product quantization (PQ) or scalar quantization (SQ) to the stored vectors, consistent with standard practice in production ANN systems (e.g., FAISS, Milvus) operating at comparable scale. PQ at a typical 8-bit-per-subvector compression ratio reduces memory footprint by roughly 4–32× relative to raw float32 storage, at a small, measurable recall cost that should be validated against this chapter's existing 95% recall target (§14.3.1) before being adopted as a default. This is a design recommendation, not yet a specification change — the exact quantization scheme and its recall impact should be benchmarked against a representative Fact Unit corpus before being written into §14.3.1 as a requirement.

14.3.2 SDR Index

In addition to the HNSW float-vector index, a separate SDR index stores the 16,384-bit Sparse Distributed Representations for all Active Fact Units, enabling the NCL Associative Recall Engine's Hamming distance search. The SDR index is implemented as a bitarray sorted by FUID, with a SIMD-accelerated population count (popcount) engine that computes Hamming distances using AVX-512 instructions at a rate of approximately 10 billion bit comparisons per second per core.

14.4 Provenance Ledger Storage

The Provenance Ledger's Merkle DAG is stored in a content-addressed immutable object store with the following architecture:

Property	Specification
Block size	1,000 Provenance Records per block (configurable: 100–10,000)
Block content address	SHA-256 of the concatenation of all Provenance Record SHA-256 hashes in the block
DAG branching factor	Each block has exactly one parent (the previous block in the chain); DAG branching occurs when multiple concurrent write streams merge — convergence is resolved by total ordering on block timestamp + writer ID hash
Chain tip pinning	The 10 most recent block hashes are pinned in memory (never evicted from Tier 0) for O(1) chain tip access during write operations
Garbage collection	Orphaned blocks (blocks that are not reachable from any pinned chain tip) are eligible for GC after a configurable grace period (default: 90 days); GC requires dual-approval and is logged in the Safety Audit Log
FUID lookup index	Separate B-tree index mapping each FUID to the list of Provenance Record block addresses containing records for that FUID; enables O(log n) per-FUID lineage lookup
External notarization	Every 15 minutes (configurable), the current chain tip hash is submitted to an external notarization service (RFC 3161 timestamp authority); the notarization receipt is stored in the ledger as a special NOTARIZATION record type

14.5 Data Partitioning and Sharding

14.5.1 FUID-Based Sharding

Fact Units are sharded across storage nodes using consistent hashing on the FUID prefix. The ring maintains 4,096 virtual nodes, ensuring near-uniform distribution even as physical nodes are added or removed. The virtual node count of 4,096 was selected to support deployments of up to 400 billion Fact Units across up to 4,096 physical storage nodes while maintaining even distribution with minimal rebalancing overhead.

14.5.2 Cross-Shard Query Handling

Queries that require accessing Fact Units from multiple shards are handled by the Query Coordinator service using a scatter-gather pattern:

77. The Query Coordinator decomposes the query into per-shard sub-queries
78. Sub-queries are dispatched in parallel to all affected shards
79. Shard results are streamed back to the Query Coordinator as they become available (no head-of-line blocking)
80. The Query Coordinator performs result merging, de-duplication, and final ranking before returning results to the caller
81. Query Coordinator maintains a result buffer per in-flight query; buffer is bounded at configurable size (default: 100,000 Fact Units) to prevent memory exhaustion from unbounded scatter results

14.5.3 Replication

Every Fact Unit is replicated across a minimum of 3 replicas located in distinct failure domains (distinct physical racks, distinct availability zones, or distinct data centers depending on deployment configuration). Replication uses a Raft consensus protocol adapted for the Fact Unit workload, with the following modifications:

- Leader leases are set to 5 seconds (shorter than typical Raft deployments) to minimize recovery time on leader failure

- Followers are allowed to serve reads (with bounded staleness of at most 500ms) to distribute read load across replicas
- Log entries are batched at 64KB or 50ms (whichever comes first) before proposing to consensus to improve throughput

14.5.4 Multi-Region Replication

For globally distributed deployments, QueryBook supports active-active multi-region replication using Conflict-Free Replicated Data Types (CRDTs) for eventual consistency across regions:

- Fact Unit ConfidenceScore updates use a max-register CRDT (the highest observed confidence score wins during merge)
- RelationSet edges use a grow-only set CRDT (edges are never removed, only added; the union of all region's edge sets is the canonical RelationSet)
- SafetyFlags use a max-bitmask CRDT (the union of all flags across regions is the canonical SafetyFlags; flags can never be decremented in a multi-region merge)
- Lifecycle state transitions use a state machine CRDT with a strict partial order (DRAFT < VALIDATED < ACTIVE; ACTIVE can transition to SUPERSEDED, ARCHIVED, or PURGED; each state is monotonically increasing in the partial order; during merge, the higher state wins)

14.6 Backup and Disaster Recovery

Metric	Target	Implementation
Recovery Time Objective (RTO)	< 4 hours	Warm standby in secondary region with continuous WAL replication; automated failover triggers within 10 minutes of primary region failure detection
Recovery Point Objective (RPO)	< 15 minutes	WAL replication to secondary region with async replication lag SLA of < 15 minutes; synchronous replication for Provenance Ledger and Safety Audit Log (RPO = 0 for audit data)
Point-in-Time Recovery (PITR)	Any moment within 30-day window	Continuous WAL archival to Tier 3 object storage; PITR restoration replays WAL from the last full snapshot to the target timestamp
Full snapshot frequency	Daily	Consistent snapshot of all Active

Metric	Target	Implementation
		Store content taken during low-traffic window; QBF-encoded and stored in Tier 3
Snapshot retention	30 daily, 12 monthly, 7 yearly	Grandfather-Father-Son (GFS) rotation policy; yearly snapshots moved to Tier 4 Deep Archive
Restore test frequency	Monthly	Automated restore drill executed monthly into isolated test environment; restore success and data integrity verified by automated test suite

Cross-Region Replication Latency — Redesign Note (Performance and Economy)

REDESIGN FINDING — PERFORMANCE **True synchronous cross-region replication with RPO = 0 for the Provenance Ledger and Safety Audit Log means every write to those stores must wait for acknowledgment from a geographically distant replica before it can be confirmed — adding real round-trip network latency (typically 60–150ms between distant regions) to every such write, and under a network partition, forcing a choice between blocking writes (consistency) or accepting the partition and risking the RPO=0 guarantee (availability), per the CAP theorem. This tradeoff was not previously stated explicitly alongside the RPO=0 target.**

Recommended framing: RPO = 0 is achievable and worth keeping as a target specifically because the Provenance Ledger and Safety Audit Log are lower-write-volume, correctness-critical stores where the added latency is an acceptable cost — this is a reasonable design choice, not an error, provided it is scoped explicitly to those two stores (as already written) and not generalized to the higher-volume Fact Graph writes, which correctly use the async WAL replication with a <15-minute lag SLA specified in the same table. No change to the specified targets is recommended; this note exists so the latency and partition-behavior tradeoff being accepted is stated rather than implicit.

PART XIV — AGI EXTENSION MODULES

Part XIV Executive Summary **Part XIV specifies the six AGI Extension Modules (AEMs) that elevate QueryBook from a sophisticated knowledge retrieval system to a genuine knowledge synthesis and reasoning platform. Each AEM addresses a distinct higher-order cognitive operation: abductive reasoning, analogical reasoning, causal discovery, counterfactual reasoning, meta-cognitive monitoring, and goal-directed synthesis. All AEMs operate over the Fact Graph and NCL and produce first-class Fact Units as their outputs. This Part is the primary reference for patent Cluster I claims.**

Chapter 15: AGI Extension Modules (AEM) — Complete Specification

15.1 Philosophy

The AGI Extension Modules represent the highest level of QueryBook's cognitive architecture. They are not simply more sophisticated retrieval algorithms; they implement fundamentally different modes of reasoning that operate over the Fact Graph and NCL to produce new knowledge rather than merely retrieving existing knowledge. Each AEM produces first-class Fact Unit outputs with appropriate FactTypes, full provenance, and confidence scores, ensuring that AEM-generated knowledge participates fully in the QueryBook ecosystem — it can be queried, cited in regenerated text, tracked for accuracy, and used as input to subsequent AEM operations.

AEMs operate under the following constraints:

- All AEM outputs are tagged with the AI_GENERATED SafetyFlag
- All AEM outputs carry InputFUIDs references to the Fact Units used as inputs, enabling full lineage tracing of synthesized knowledge
- AEM ConfidenceScores are capped at 0.85 unless independently corroborated by subsequent ingestion of supporting real-world data
- The Safety Substrate applies standard enforcement to all AEM-produced Fact Units before Graph insertion
- Redesign addition (Economy — risk reduction): an AEM-produced Fact Unit may be used as input to a subsequent AEM operation only if it is either independently corroborated (per the confidence-cap rule above) or if the resulting chain of AEM-derived inputs feeding a single downstream inference is disclosed in that output's InputFUIDs lineage and the compounded chain confidence — computed as the product of each link's ConfidenceScore, not the maximum — is surfaced to any consumer of the result. This closes a gap where uncorroborated AEM

output could otherwise feed further AEM reasoning indefinitely with no visible signal that confidence is compounding downward across the chain.

15.2 AEM-1: Abductive Reasoning Module

15.2.1 Definition and Purpose

Abductive reasoning is inference to the best explanation: given a set of observations (Fact Units of FactType OBSERVATION), generate the most plausible explanatory hypothesis. AEM-1 implements abduction over the Fact Graph using a combination of Bayesian model selection and causal graph traversal.

15.2.2 Algorithm

82. **Observation encoding:** Input Observation Fact Units are encoded into the NCL SDR space and used to activate a candidate set of Fact Units via ARE
83. **Hypothesis space generation:** For each highly activated entity or event Fact Unit, AEM-1 generates candidate hypotheses by: (a) retrieving all CAUSAL_RELATION Fact Units that could produce the observed effects as outputs, and (b) constructing new HYPOTHESIS Fact Units representing potential explanatory states not yet in the Fact Graph
84. **Posterior probability ranking:** Each hypothesis H is scored by its posterior probability P(H|Observations) using Bayes' theorem: $P(H|O) \propto P(O|H) \times P(H)$, where P(O|H) is the likelihood of the observations given the hypothesis (computed using the causal structure of the Fact Graph) and P(H) is the prior probability (derived from the ConfidenceScore of precursor Fact Units)
85. **Scenario Graph branch creation:** The top-K hypotheses (default K=5) are instantiated as branches in a new WHAT_IF Scenario Graph, enabling downstream scenario analysis
86. **Output Fact Units:** HYPOTHESIS Fact Units with FactType = ABDUCTIVE_HYPOTHESIS, ranked by posterior probability, referencing all InputFUIDs (observations and causal chain Fact Units used)

15.2.3 Applications

- Root cause analysis for system failures or anomalous sensor readings
- Medical differential diagnosis from symptom Fact Units

- Scientific hypothesis generation from experimental observation Fact Units
- Intelligence analysis: generating explanatory scenarios for observed geopolitical events

15.3 AEM-2: Analogical Reasoning Module

15.3.1 Definition and Purpose

Analogical reasoning identifies structural isomorphisms between subgraphs of the Fact Graph in different domains, enabling cross-domain insight generation. AEM-2 implements structure-mapping theory (Gentner, 1983) at the Fact Graph level.

15.3.2 Algorithm

87. **Source domain selection:** Given a source domain subgraph (a connected set of Fact Units in one domain, e.g., a predator-prey ecological relationship), AEM-2 extracts the relational structure: the pattern of predicate types and entity type constraints without the specific entity identities
88. **Target domain search:** The structural pattern is matched against all other domain subgraphs in the Fact Graph using a modified subgraph isomorphism algorithm optimized for the sparse, typed Fact Graph structure. Predicate type compatibility is governed by the Ontology Mesh's cross-domain predicate alignment rules
89. **Analogy quality scoring:** Each candidate analogy is scored on three dimensions: (a) Structural score: fraction of source relations that have structurally isomorphic counterparts in the target; (b) Semantic score: cosine similarity of EmbeddingVectors between mapped entity pairs; (c) Pragmatic score: utility of the cross-domain mapping for the query context
90. **Insight generation:** For each high-quality analogy, AEM-2 generates CROSS_DOMAIN_INSIGHT Fact Units by mapping source-domain predicates and their associated knowledge across to the target domain, creating new candidate Fact Units that assert the analogous relationship in the target domain

15.3.3 Analogy Quality Threshold

Only analogies with a composite quality score (0.4 × Structural + 0.4 × Semantic + 0.2 × Pragmatic) above a configurable threshold (default: 0.65) produce CROSS_DOMAIN_INSIGHT Fact Units. Below-threshold analogies are retained as CANDIDATE_ANALOGY Fact Units for human review.

15.4 AEM-3: Causal Discovery Module

15.4.1 Definition and Purpose

AEM-3 infers causal relationships between Fact Units from observational data, distinguishing mere correlation from causal direction using a combination of constraint-based, score-based, and neural causal discovery methods.

15.4.2 Causal Discovery Methods

Method	Algorithm	Applicable When	Output
Constraint-based	PC Algorithm with Fisher-Z conditional independence tests	Large observational datasets with continuous variables (TIME_SERIES_POINT, MEASUREMENT Fact Units)	Completed Partially Directed Acyclic Graph (CPDAG) over Fact Unit variables
Constraint-based (latent)	FCI (Fast Causal Inference) Algorithm	Same as PC but when latent confounders are suspected (indicated by low source diversity for the relevant Fact Units)	Partial Ancestral Graph (PAG) capturing uncertainty about latent structure
Neural causal discovery	NOTEARS (Non-combinatorial Optimization via Trace Exponential and Augmented lagRangian for Structure learning) adapted to EmbeddingVector space	High-dimensional Fact Unit clusters where traditional independence tests are underpowered	Weighted directed adjacency matrix over Fact Unit EmbeddingVectors
Interventional	Do-calculus over Scenario Graph interventions	When WHAT_IF Scenario Graphs with forced interventions are available as quasi-experimental data	Causal effect estimates with confidence intervals; distinguishes correlation from causation definitively

15.4.2.1 Scaling the PC Algorithm — Redesign Note (Efficiency)

REDESIGN FINDING — EFFICIENCY **The PC Algorithm's conditional-independence testing has cubic time complexity, $O(n^3)$, in the number of variables for dense graphs. Applied directly to a Fact Graph domain with millions of related Fact Units, this is computationally intractable without a bound on the variable set the algorithm runs against.**

Recommended mitigation: AEM-3 should require domain partitioning before invoking the PC Algorithm — bounding the candidate variable set to a single Ontology Mesh domain (Chapter 6) or a Scenario

Graph-scoped subset (Chapter 11) rather than the full Fact Graph, and falling back to the score-based or neural causal discovery methods (per the method table above) for cross-domain causal questions that would otherwise require an unbounded conditional-independence search. This partitioning requirement should be added as a precondition in §15.4.1 before AEM-3 is treated as usable at production Fact Graph scale.

15.4.3 Output Fact Units

AEM-3 produces CAUSAL_RELATION Fact Units with FactType = CAUSAL_RELATION, containing:

- SubjectRef: the causal Fact Unit (cause)
- PredicateRef: CAUSES, PROMOTES, INHIBITS, ENABLES, or PREVENTS (per Predicate Library)
- ObjectRef: the effect Fact Unit
- AnnotationSet: `qb:causal.method` (discovery method used), `qb:causal.effect_size` (estimated average causal effect), `qb:causal.confounding_risk` (low/medium/high)

15.5 AEM-4: Counterfactual Reasoning Module

15.5.1 Definition and Purpose

AEM-4 generates the most plausible alternative world state in which a specified Fact Unit is false, using the minimum-intervention principle: change as few facts in the Fact Graph as necessary to render the target Fact Unit false while maintaining internal consistency.

15.5.2 Algorithm

91. **Intervention set computation:** Given a target Fact Unit FU_target to negate, AEM-4 uses the CAUSAL_RELATION Fact Units in the Fact Graph to identify the minimal set of upstream facts whose removal or modification would causally preclude FU_target
92. **Minimum intervention selection:** Among all intervention sets that achieve the counterfactual goal, AEM-4 selects the one with the minimum total intervention cost, where intervention cost is computed as: sum(1 - ConfidenceScore) for each Fact Unit that must be removed, plus sum(semantic_distance) for each Fact Unit whose value must be modified

93. **Counterfactual KSS construction:** The selected minimum intervention set is applied to the current Fact Graph to construct a Counterfactual Knowledge State Snapshot (KSS) within a new COUNTERFACTUAL Scenario Graph
94. **Consistency propagation:** The CDRE is applied to the counterfactual KSS to propagate the downstream consequences of the intervention through the causal graph, updating all Fact Units causally downstream of the intervened facts
95. **Output:** A COUNTERFACTUAL Scenario Graph with the counterfactual KSS and a COUNTERFACTUAL_ANALYSIS Fact Unit summarizing the intervention set, the minimum cost, and the downstream consequences

15.6 AEM-5: Meta-Cognitive Monitoring Module

15.6.1 Definition and Purpose

AEM-5 continuously monitors the NCL's own reasoning processes for coherence, confidence calibration, and epistemic quality. It is the system's self-awareness module — generating Fact Units that describe QueryBook's own epistemic state and flagging conditions that require human review or Safety Substrate intervention.

15.6.2 Monitored Conditions

Condition	Detection Method	Output Fact Unit Type	Escalation
Circular reasoning	Cycle detection in the causal subgraph activated during a query session's CCW	META_CIRCULAR_REASONING	Flag involved Fact Units as UNDER_REVIEW; notify query originator
Overconfidence	ConfidenceScore distribution for a domain exceeds calibration threshold; comparison against historical calibration records from PIE	META_OVERCONFIDENCE_ALERT	Recalibrate ConfidenceScores for affected domain; log to Safety Audit Log
Knowledge gap	Queries that return fewer than configurable minimum Fact Units above confidence threshold; Knowledge Gap Predictions from PIE	META_KNOWLEDGE_GAP	Notify ingestion operators; trigger targeted crawl recommendations
Reasoning shortcut	Detection of repeated activation of the same	META_REASONING_SHORTCUT	Temporarily reduce SWS weights on the shortcut

Condition	Detection Method	Output Fact Unit Type	Escalation
	high-weight SWS path without exploration of alternative reasoning paths		path to force exploration; log for bias analysis
Epistemic inconsistency	Detection of active contradicting Fact Unit pairs (CONTRADICTS edges) both with ConfidenceScore > 0.7 — the system holds high-confidence contradictory beliefs	META_EPISTEMIC_INCONSISTENCY	Escalate to human review queue; prevent both contradicting Fact Units from appearing together in the same Narrative Graph

15.7 AEM-6: Goal-Directed Synthesis Module

15.7.1 Definition and Purpose

AEM-6 computes action plans for achieving a user-specified goal state. Given a target state expressed as a desired Fact Unit or Fact Bundle that does not currently exist in the Fact Graph, AEM-6 computes a sequence of knowledge acquisition, reasoning, and synthesis actions that, if executed, would produce the desired Fact Units.

15.7.2 Algorithm

96. **Goal representation:** The target Fact Unit (or Fact Bundle) is represented as a node in a Planning Scenario Graph with ScenarioType=PLANNING
97. **Backward chaining from goal:** AEM-6 applies a backward chaining planner over the Fact Graph: starting from the goal state, it identifies what causal preconditions and ingestion events would produce the required Fact Units
98. **Action set enumeration:** Identified actions are classified into: INGEST (acquire specific data sources), DERIVE (run specific AEM modules), QUERY (retrieve existing Fact Units), SYNTHESIZE (combine existing Fact Units via MRE), PREDICT (invoke PIE for a specific prediction)
99. **Feasibility assessment:** Each action step is evaluated by the PIE for feasibility probability given current Fact Graph state and available resources
100. **Plan output:** A GOAL_PLAN Fact Unit with FactType = GOAL_PLAN is generated, containing: the goal FUID reference, the ordered action sequence as a structured JSON in AnnotationSet, each step's feasibility probability, and the estimated time to goal achievement

15.7.3 Applications

- Automated research planning: computing the data acquisition and analysis steps needed to answer a complex research question
- Knowledge acquisition roadmapping: identifying what sources must be ingested to fill a specified knowledge gap
- Strategic planning support: computing what real-world actions would be required to transition from current state to a desired business or policy state
- Competitive intelligence: planning the monitoring and analysis workflow needed to maintain current awareness of a specified competitive domain

PART XV — MULTILINGUAL REGENERATION ENGINE

Part XV Executive Summary **Part XV specifies the Multilingual Regeneration Engine (MRE) — the subsystem that transforms machine-readable Fact Units and Narrative Graphs into fluent, accurate, audience-appropriate natural language output in any of QueryBook's 50+ supported languages. The MRE's defining architectural property is architecturally enforced hallucination prevention: every output sentence must be grounded in one or more Fact Units, and ungroundable content is rejected rather than generated. This Part is the primary reference for patent Cluster G and Cluster J claims.**

15.8 AEM-7: Constraint Satisfaction and Optimization Module (added in Version 1.0 merge review)

15.8.1 Definition and Purpose

Several downstream QueryBook use cases — scheduling, resource allocation, and multi-party negotiation support among them — are naturally expressed as constraint satisfaction problems (CSPs) rather than as retrieval, abduction, analogy, causal discovery, counterfactual, meta-cognitive, or goal-directed-synthesis problems, which is what AEM-1 through AEM-6 already cover. AEM-7 closes this gap: it formulates a CSP or constrained-optimization problem directly from Fact Units — constraints as RULE-typed Fact Units from the Cross-Domain Rule Engine (Chapter 17), variables and domains as entity and

attribute Fact Units — and returns solutions as first-class Fact Unit outputs, consistent with the constraints already specified for every AEM in §15.1.

15.8.2 Algorithm

- Constraint extraction: AEM-7 collects the CDRE rules (Chapter 17) applicable to the query's domain and translates each into a formal constraint over the relevant Fact Unit variables.
- Variable and domain construction: for each unbound entity or attribute referenced by the extracted constraints, AEM-7 builds a finite domain from the candidate Fact Units the NCL's Associative Recall Engine (§7.2.3) surfaces for that entity type.
- Solving: AEM-7 applies a standard constraint-propagation and backtracking search (arc consistency followed by backtracking with conflict-directed backjumping) for CSPs, or a bounded local-search optimizer for problems with a continuous or large-cardinality objective, selecting between the two based on problem shape rather than a fixed rule.
- Output Fact Units: each accepted solution is emitted as a SOLUTION-typed Fact Unit with FactType = CONSTRAINT_SOLUTION, InputFUIDs referencing every constraint and domain Fact Unit used, and a ConfidenceScore capped at 0.85 per the standard AEM output constraint in §15.1 unless independently corroborated.

15.8.3 Applications

- Scheduling and resource allocation across Fact-Graph-represented constraints (staffing, logistics, facility booking)
- Multi-party negotiation support, surfacing feasible-agreement regions from each party's stated constraints as Fact Units
- Configuration problems where a valid combination of Fact-Unit-represented options must satisfy compatibility rules drawn from the CDRE

As with every AEM, AEM-7 outputs are subject to Prime Directive review (Domain 0) and standard Safety Substrate enforcement (Chapter 8) before Graph insertion; a CSP solution that would only be satisfiable by violating a safety-classified constraint is not returned as a solution.

Chapter 16: Multilingual Regeneration Engine (MRE) — Complete Specification

16.1 Purpose

The MRE serves as the human-language interface of QueryBook, translating the machine-readable Fact Graph into fluent, accurate, and appropriately styled natural language. Its central distinguishing property — the one that separates it from all prior art natural language generation systems — is that hallucination is architecturally prevented, not probabilistically managed. Every token of MRE output must be traceable to one or more specific Fact Units via the Narrative Graph grounding mechanism. Any candidate sentence that cannot be fully grounded in the current Narrative Graph is not generated; instead, the MRE either queries for additional grounding Fact Units or produces an explicitly qualified uncertainty statement.

16.2 Supported Languages

Language	BCP-47 Code	Script	MRE Support Level	Special Handling
English	en	Latin	Full	Baseline reference language
Mandarin Chinese (Simplified)	zh-Hans	Han (Simplified)	Full	Character segmentation; no spaces between words
Mandarin Chinese (Traditional)	zh-Hant	Han (Traditional)	Full	Traditional character set; Taiwan/HK variant handling
Spanish	es	Latin	Full	Latin American vs. Castilian variant lexicons
French	fr	Latin	Full	Diacritics; gendered agreement; Canadian French variant
German	de	Latin	Full	Compound noun construction; case inflection (nominative, accusative, dative, genitive)
Japanese	ja	Hiragana / Katakana / Kanji	Full	Tonal script mixture; honorific register (keigo)

Language	BCP-47 Code	Script	MRE Support Level	Special Handling
				system; no word boundaries
Korean	ko	Hangul	Full	Agglutinative morphology; speech level (formal/informal) system
Portuguese (Brazil)	pt-BR	Latin	Full	Brazilian Portuguese morphology and lexicon
Portuguese (Portugal)	pt-PT	Latin	Full	European Portuguese variant
Russian	ru	Cyrillic	Full	6-case morphological inflection; aspect system for verbs
Arabic (Modern Standard)	ar	Arabic (RTL)	Full	Right-to-left; root-based morphology; dual number; definite article agglutination
Hindi	hi	Devanagari	Full	SOV word order; postpositions; grammatical gender
Italian	it	Latin	Full	Gendered agreement; subjunctive mood
Dutch	nl	Latin	Full	De/het grammatical gender; separable verbs
Swedish	sv	Latin	Full	Definite suffix; utrum/neutrum gender
Norwegian (Bokmål)	nb	Latin	Full	Bokmål standard; close to Danish
Danish	da	Latin	Full	Stød (glottal phonation) in formal registers
Finnish	fi	Latin	Full	Highly agglutinative; 15 grammatical cases; no grammatical gender
Polish	pl	Latin	Full	7 grammatical cases; complex consonant clusters
Czech	cs	Latin	Full	7 cases; diacritics; free word order
Turkish	tr	Latin	Full	Agglutinative; vowel harmony; SOV word order
Hebrew	he	Hebrew (RTL)	Full	Right-to-left; root-pattern morphology;

Language	BCP-47 Code	Script	MRE Support Level	Special Handling
				gendered verbs
Greek	el	Greek	Full	4 grammatical cases; polytonic optional in formal texts
Romanian	ro	Latin	Full	Postposed definite article; 3 genders; 5 cases
Hungarian	hu	Latin	Full	18 grammatical cases; agglutinative; vowel harmony
Thai	th	Thai	Full	Tonal language (5 tones); no word spaces; no inflection
Vietnamese	vi	Latin (diacritics)	Full	Tonal language (6 tones); extensive diacritic system
Indonesian	id	Latin	Full	Agglutinative; no grammatical gender; no tense inflection
Malay	ms	Latin	Full	Close to Indonesian; Jawi script support (Partial)
Swahili	sw	Latin	Full	Bantu noun class system (16 classes); agglutinative
Bengali	bn	Bengali	Full	Vowel diacritics on consonant bases; honorific pronouns
Urdu	ur	Nastaliq (RTL)	Full	Right-to-left; shared grammar with Hindi; Arabic loanwords
Tamil	ta	Tamil	Full	Diglossia (formal vs. spoken); agglutinative
Catalan	ca	Latin	Full	Close to Occitan; gendered articles; clitic pronouns
Ukrainian	uk	Cyrillic	Full	7 cases; differs significantly from Russian in vocabulary
Serbian	sr	Cyrillic / Latin (both)	Full	Dual script; 7 cases; pitch accent
Croatian	hr	Latin	Full	7 cases; pitch accent; closely related to Serbian
Slovak	sk	Latin	Full	6 cases; long vowel distinction; close to Czech
Bulgarian	bg	Cyrillic	Full	No grammatical

Language	BCP-47 Code	Script	MRE Support Level	Special Handling
				cases (rare for Slavic); definite article as suffix
Slovenian	sl	Latin	Full	6 cases; dual grammatical number retained
Lithuanian	lt	Latin	Full	7 cases; pitch accent; highly archaic Indo-European morphology
Latvian	lv	Latin	Full	7 cases; grammatical gender (masculine/feminine only)
Estonian	et	Latin	Full	14 cases; no grammatical gender; no future tense
Icelandic	is	Latin	Full	4 cases; 3 genders; highly conservative morphology
Welsh	cy	Latin	Partial	VSO word order; initial consonant mutations; soft/nasal/aspirate mutations
Irish (Gaelic)	ga	Latin	Partial	VSO word order; initial mutations; lenition and eclipsis
Basque	eu	Latin	Partial	Language isolate; polypersonal verb agreement (subject, object, indirect object); ergative-absolutive alignment
Maltese	mt	Latin	Partial	Semitic root system; heavy Romance and English loanword layer
Kazakh	kk	Cyrillic / Latin (transitioning)	Partial	Agglutinative; Turkic family; vowel harmony
Azerbaijani	az	Latin	Partial	Agglutinative; Turkic; vowel harmony
Georgian	ka	Georgian (Mkhedruli)	Partial	Unique script; polypersonal verb agreement; complex consonant clusters
Amharic	am	Ethiopic (Ge'ez)	Partial	Semitic; SOV word

Language	BCP-47 Code	Script	MRE Support Level	Special Handling
				order; extensive verb morphology; syllabic script
Farsi (Persian)	fa	Persian (RTL)	Partial	Right-to-left; Indo-European grammar; Arabic script adapted; pro-drop

16.3 Regeneration Pipeline

Stage	Name	Operations	Output
1	Narrative Graph Selection	Retrieve or construct Narrative Graph from query result Fact Units (see Chapter 12); if REGENERATE AS clause specifies a pre-built NarrativeGraphID, load it directly; else run automatic construction algorithm	Narrative Graph with linearized ReadingOrder
2	Linearization and Gap-Filling	Apply linearization algorithm to Narrative Graph DAG; detect gaps; query Fact Graph for gap-filling Fact Units; insert gap-filling Fact Units into Narrative Graph NodeSet where coherence score improvement exceeds threshold	Enriched, linearized Narrative Graph with ReadingOrder
3	Lexical Selection	For each Fact Unit in ReadingOrder, select vocabulary appropriate for: (a) target language (OutputLanguage), (b) audience model (domain expertise, formality level, vocabulary age), (c) NarrativeStyle; map Fact Unit entities to their canonical names in the target language using LinguisticMetadata and the Entity Registry's multilingual name map	Per-Fact-Unit lexicalized propositions in target language
4	Syntactic Realization	Generate grammatically correct sentences for each lexicalized proposition respecting the target language's	Per-Fact-Unit candidate sentences in target language

Stage	Name	Operations	Output
		morphological and syntactic rules; apply NarrativeStyle sentence structure constraints; handle language-specific phenomena (grammatical gender agreement, case inflection, aspect, honorifics, RTL script direction)	
5	Discourse Coherence	Insert appropriate discourse connectives between sentences based on Narrative Edge Types (TEMPORAL_FOLLOWS → "subsequently", "then"; CAUSES → "consequently", "as a result"; CONTRASTS_WITH → "however", "in contrast"; ELABORATES → "specifically", "in particular"); resolve cross-sentence reference with appropriate pronouns and definite descriptions; ensure paragraph-level topic continuity	Coherent paragraph-level text in target language
6	Factual Grounding Verification	For each generated sentence, map every substantive claim back to the Fact Unit(s) that ground it; sentences where all claims are grounded are marked GROUNDED and passed through; sentences containing claims that cannot be mapped to any Fact Unit in the Narrative Graph are marked UNGROUNDED and either (a) rewritten to remove the ungroundable claim, (b) qualified with explicit uncertainty language ("it is unclear whether...", "this has not been confirmed..."), or (c) removed entirely with a gap acknowledgment	Fully grounded output text with per-sentence FUID citation map

16.4 Hallucination Prevention Architecture

The MRE's hallucination prevention is an architectural guarantee, not a probabilistic one. The mechanism is as follows:

101. **Constrained generation vocabulary:** At Stage 4 (Syntactic Realization), the generative process operates over a constrained vocabulary derived from: the lexicalized content of the Narrative Graph Fact Units, standard discourse connectives and function words in the target language, and hedging language for qualified uncertainty statements. Novel factual claims — content that is not present in the Narrative Graph — cannot be generated because the generation vocabulary does not contain the necessary lexical items.

102. **Factual Groundedness metric:** Every candidate sentence at Stage 6 is scored against a Factual Groundedness metric computed as: the fraction of extractable propositions in the sentence that can be matched to Fact Units in the Narrative Graph using semantic equivalence (cosine similarity of EmbeddingVectors above threshold 0.92). Sentences below the groundedness threshold (default: 0.95) are rejected and trigger rewriting or qualification.

103. **Graceful degradation:** When a query requires knowledge that is absent from the Fact Graph or below the confidence threshold, the MRE does not hallucinate an answer. Instead, it produces a qualified acknowledgment of the knowledge gap, optionally including a META_KNOWLEDGE_GAP Fact Unit reference for the operator to address.

16.5 Cross-Lingual Fact Equivalence

The MRE is designed to enable true multilingual knowledge equivalence, not merely translation:

- **Language-independent Semantic Hash:** The same Fact Unit regenerated in any supported language produces output whose back-translation yields a Semantic Hash within a configurable distance of the original. This is enforced by the Semantic Hash's language-independent computation (Section 3.1).
- **FUID preservation across languages:** When a Fact Unit is regenerated in a target language, the output is associated with the original FUID. No new FUID is created; instead, a new LinguisticMetadata variant is created and stored in the AnnotationSet of the original Fact Unit, recording the target language output and its grounding verification result.

- **Simultaneous multi-language regeneration:** The MRE can regenerate the same Narrative Graph in multiple languages simultaneously using parallel language-specific realization pipelines, producing a set of semantically equivalent outputs each grounded in the same underlying Fact Units.
- **Translation of new Fact Units:** When a Fact Unit is ingested in language L1 and subsequently queried in language L2, the MRE first checks for an existing L2 linguistic variant; if none exists, it generates the L2 variant on demand and caches it as a LinguisticMetadata variant of the original Fact Unit.

16.6 Domain-Aware and Disambiguated Translation

This section strengthens MRE translation using three already-real, already-specified mechanisms — the Ontology Mesh's domain-specific sub-ontologies, the Attractor Network's convergence dynamics, and the multi-modal content surfaces already defined elsewhere in this Bible — rather than introducing new architecture. No new Fact Unit subtype, no new Slice, and no voice-persona or accent-manipulation capability is introduced by this section.

BUILT FROM THREE ALREADY-CLAIMED MECHANISMS, NOT A NEW SYSTEM Domain-aware terminology resolution reuses the Ontology Mesh's federated domain sub-ontologies (Chapter 6, §20 of the patent, Claims 19–20). Ambiguity resolution reuses the Attractor Network's convergence dynamics (Chapter 7, Claim 6), already extended to hyperedge-level completion at §53.7. Multi-modal translation reuses MRE's own grounded generation applied to non-prose content already defined elsewhere (diagram labels, timeline descriptions). This section connects three real, separately-claimed mechanisms to strengthen translation; it does not add a fourth.

16.6.1 Domain-Aware Terminology Resolution

A term with multiple valid meanings across domains — for example, “deck” in a marine context versus a card-game context — is resolved using the Ontology Mesh's poly-hierarchical concept structure (§20.1 of the patent): the source Fact Unit's own domain classification determines which federated domain sub-ontology's predicate and concept registrations govern the term's translation, rather than a single, domain-blind lexicon entry. This reuses the Ontology Mesh's existing domain-scoping mechanism; no new terminology or lexicon system is introduced.

16.6.2 Attractor-Based Ambiguity Resolution

Where source content contains a genuinely ambiguous term or construction — a word with multiple senses not resolved by domain classification alone — the candidate interpretations are represented as an activation set and resolved through the Attractor Network's existing convergence dynamics (§7.2.4), converging toward the interpretation best supported by the surrounding Fact Graph context, exactly as the Attractor Network already stabilizes retrieval more generally. This is the same real mechanism already extended to hyperedge completion at §53.7, applied here to lexical ambiguity rather than geometric structure.

- A translation produced through ambiguity resolution carries a ConfidenceScore reflecting the resolution's certainty; a low-confidence resolution is represented as such, never presented with unwarranted certainty.

16.6.3 Multi-Modal Translation Surfaces

MRE's grounded generation, already language-independent by design (§16.1), extends to non-prose text surfaces already defined elsewhere in this Bible: diagram labels and annotations, and timeline event descriptions. Each surface's translated output is generated the same way MRE generates any other output — directly from the underlying Fact Unit and Narrative Graph, in the target language — rather than through a separate translation step applied after the fact. This section introduces no new content surface; it applies MRE's existing mechanism to surfaces already specified in Chapter 46 and elsewhere.

- Audio narration of translated content is not a new capability introduced here: where a target-language MRE output exists, it is eligible for synthetic narration exactly as any other MRE output is under §93.2 — using a stock voice profile with no real-person referent, never a persona or accent-shifted voice.

16.6.4 Integration Requirements

This section requires the Ontology Mesh (Chapter 6, §20 of the patent); the Attractor Network (Chapter 7, Claim 6, and its extension at §53.7); MRE's own grounded generation and Cross-Lingual Fact Equivalence mechanism (§16.1–§16.5); and generic synthetic narration (§93.2). It modifies none of their specifications.

PART XVI — CROSS-DOMAIN RULE ENGINE

Part XVI Executive Summary **Part XVI specifies the Cross-Domain Rule Engine (CDRE) — the system enforcing consistency across the Fact Graph when Fact Units from different domain ontologies interact. It defines the rule categories (terminological, assertional, inference, integrity, and priority rules), the formal rule syntax based on Datalog-with-negation extended with probabilistic weights and temporal operators, the incremental forward-chaining execution engine, and the rule governance workflow. This Part is the primary reference for patent Cluster K claims.**

Chapter 17: Cross-Domain Rule Engine (CDRE) — Complete Specification

17.1 Purpose and Scope

The CDRE is the consistency enforcement mechanism for the Ontology Mesh. It operates continuously over the Fact Graph, evaluating rules on every insertion event and maintaining a conflict detection index that flags violations for human review or automated resolution. The CDRE ensures that the multi-domain Fact Graph remains internally consistent despite the semantic tensions that arise when facts from disparate domains (medicine, law, finance, engineering, etc.) make assertions about the same entities.

17.2 Rule Categories

Rule Category	Formal Type	Scope	Example
Terminological Rules	OWL-style subsumption and equivalence axioms	Define equivalences, subsumptions, and disjointness between concepts across domain ontologies	`MED:Medication equivalentTo LAW:LicensedSubstance` — the medical concept of "Medication" is equivalent to the legal concept of "Licensed Substance"; any fact true of one is inferred to be true of the other
Assertional Rules	Datalog rules with domain filters	Govern which facts from one domain may be asserted in the context of another without additional verification	A Fact Unit from the Finance domain asserting that a company's market cap exceeds a threshold may not override a Law domain Fact Unit asserting the company is

Rule Category	Formal Type	Scope	Example
			in bankruptcy without additional LEGALLY_SENSITIVE verification
Inference Rules	Datalog-with-negation	Derive new Fact Units from existing ones across domain boundaries	`hasEmployee(?org, ?person), locatedIn(?org, ?country) → subjectToLaborLaw(?person, ?country)`
Integrity Constraints	Closed-world integrity rules	Global constraints that no combination of Fact Units may violate; violation triggers UNDER_REVIEW flags	`NOT (isAlive(?person, ?t) AND dateOfDeath(?person, ?t2) AND ?t2 <= ?t)` — a person cannot be alive after their death date
Priority Rules	Priority orderings with conditions	When two domains assert conflicting facts, determine which takes precedence based on source authority, recency, and confidence	`IF conflict(MED:indication, FIN:marketData) AND domain=MED AND ConfidenceScore > 0.8 THEN prefer MED`

17.3 Rule Syntax

CDRE rules are expressed in QB-Datalog, a Datalog-with-negation variant extended with:

- Probabilistic weights on rule conclusions (the derived Fact Unit receives a ConfidenceScore computed from rule body confidence)
- Temporal scoping operators anchoring rule applicability to specified time intervals
- Domain namespace qualifiers on all predicate and type references
- Safety flag constraints on rule applicability and output

17.3.1 QB-Datalog Syntax (BNF)

```
rule ::= head ":-" body ["@TIME" temporal_scope]
 ["@CONFIDENCE" conf_expr]
 ["@SAFETY" safety_flags]
 ["@PRIORITY" uint32] "."
```

```
head ::= atom | "CONFLICT" "(" atom "," atom ")"
 | "INTEGRITY_VIOLATION" "(" atom ")"

body ::= literal ("," literal)*

literal ::= atom | "NOT" atom | comparison

atom ::= predicate "(" term ("," term)* ")"
predicate ::= [namespace ":"] identifier
term ::= variable | constant | "_"
variable ::= "?" identifier
constant ::= quoted_string | number | uuid_literal

comparison ::= term comparator term
comparator ::= "=" | "!=" | "<" data-id="3472" | "<=" | ">" | ">=" | "WITHIN"

temporal_scope ::= "[" time_expr "," time_expr "]"
 | "CURRENT" | "ALL_TIME"

conf_expr ::= "min(" body_conf_vars ")"
 | "product(" body_conf_vars ")"
 | float_literal

safety_flags ::= safety_flag_name ("|" safety_flag_name)*

Examples:

%% Terminological rule: medical drug equivalence to regulated substance
```

```
subjectToFDA(?drug) :-
 MED:isMedication(?drug),
 MED:approvedByRegulatoryBody(?drug, "FDA")
 @CONFIDENCE min(conf(?drug)) @SAFETY SAFE @PRIORITY 100 .

%% Inference rule: employment jurisdiction
LAW:subjectToLaborLaw(?person, ?country) :-
 ORG:hasEmployee(?org, ?person),
 GEO:locatedIn(?org, ?country),
 NOT LAW:diplomaticImmunity(?person, ?country)
 @CONFIDENCE product(conf(?org, ?person), conf(?org, ?country))
 @SAFETY RESTRICTED @PRIORITY 200 .

%% Integrity constraint: temporal life consistency
INTEGRITY_VIOLATION(FU_alive, FU_death) :-
 BIO:isAlive(?person, ?t1),
 BIO:dateOfDeath(?person, ?t2),
 ?t2 <= data-id="3473" ?t1 .

%% Priority rule: medical authority over financial in health context
CONFLICT(FU_med, FU_fin) :-
 MED:contraindicates(?drug, ?condition),
 FIN:marketedFor(?drug, ?condition),
 MED:contraindicates.confidence(?drug, ?condition) > 0.75 .
```

17.4 Rule Execution Engine

17.4.1 Incremental Forward Chaining

The CDRE executes rules using incremental forward chaining over the Fact Graph, implemented via the Differential Dataflow execution model. Rather than re-evaluating all rules from scratch on each Fact Graph modification, the CDRE maintains a set of materialized rule derivations and incrementally updates them as Fact Units are added, modified, or removed:

104. **Change propagation:** When a new Fact Unit FU_new is inserted into the Fact Graph, the CDRE identifies all rules whose body contains a predicate that matches FU_new's PredicateRef

105. **Incremental join evaluation:** For each matching rule, the CDRE performs an incremental join over the existing materialized intermediate results to compute new derived facts that were not derivable before FU_new's insertion

106. **Conflict detection:** The CDRE evaluates all CONFLICT-head rules to identify newly introduced conflicts; detected conflicts are recorded in the Conflict Index and the involved Fact Units receive the UNDER_REVIEW SafetyFlag

107. **Derived Fact Unit insertion:** New Fact Units derived by inference rules are submitted to the Ingestion Pipeline as derived Fact Units (with FactType from the rule head, InputFUIDs from the rule body, and AgentID = CDRE_SERVICE)

17.4.2 Conflict Detection and Resolution Reporting

The CDRE maintains a Conflict Index — a persistent map of (FUID_A, FUID_B, ConflictRuleID, DetectedAt) tuples representing all currently active conflicts in the Fact Graph. Entries in the Conflict Index are resolved and removed when: (a) one of the conflicting Fact Units is transitioned to SUPERSEDED, ARCHIVED, or PURGED; (b) a Priority Rule is applied and one Fact Unit's ConfidenceScore is downgraded below the conflict resolution threshold; or (c) a human reviewer explicitly marks the conflict as RESOLVED with a justification recorded in the Provenance Ledger.

17.5 Rule Governance

17.5.1 Rule Authoring Workflow

108. **Draft submission:** Rule author submits a new QB-Datalog rule via the CDRE Rule Management API, including: rule text, affected domain namespaces, estimated impact (number of Fact Units affected), and justification

109. **Static analysis:** The Rule Compiler validates QB-Datalog syntax; checks for range-restrictedness (all head variables appear in body); detects potential rule interactions (rules that could trigger each other in infinite loops); estimates materialization cost

110. **Impact analysis:** The CDRE Rule Simulator runs the new rule against a read-only snapshot of the production Fact Graph and reports: number of new derived Fact Units produced, number of conflicts detected, number of existing Fact Units that would receive UNDER_REVIEW flags

111. **Review and approval:** Rules with high impact (>10,000 affected Fact Units or >100 new conflicts) require approval from two designated Ontology Stewards; low-impact rules require single-Steward approval

112. **Staged deployment:** Approved rules are deployed first to a 10% traffic shadow mode (rule executes but produces no Fact Unit changes, only logs what it would have done), then to full production with monitoring for 48 hours

113. **Rollback:** Rules can be disabled (not deleted) at any time; disabling a rule triggers re-evaluation of all Fact Units derived by that rule for potential retraction

PART XVII — QUERY LANGUAGE AND API

Part XVII Executive Summary **Part XVII specifies QB-Query — QueryBook's native declarative query language that combines graph pattern matching, temporal operators, probabilistic selection, and Scenario Graph traversal in a unified syntax. It defines the complete syntax elements (MATCH, WHERE, ACROSS SCENARIOS, RETURN, REGENERATE AS, EXPLAIN clauses), the query optimizer architecture, and the query planning methodology. This Part is the primary reference for patent Cluster O claims.**

Chapter 18: QB-Query Language — Complete Specification

18.1 QB-Query Overview

QB-Query is QueryBook's native declarative query language. It is designed to express the full range of operations possible against the QueryBook knowledge system — from simple Fact Unit lookup by FUID, to complex multi-hop graph pattern matching with temporal constraints and probabilistic filtering, to Scenario Graph traversal across hypothetical world states, to natural language knowledge regeneration in any supported language.

QB-Query is inspired by and extends three existing query languages:

- **SPARQL 1.1** for RDF triple pattern matching semantics
- **Cypher (openCypher)** for property graph traversal syntax
- **SQL window functions** for temporal range and ordering operators

QB-Query adds four capabilities that have no precedent in any of these languages:

114. The **ACROSS SCENARIOS** clause for querying across Scenario Graph branches
115. The **REGENERATE AS** clause for invoking the MRE from within a query
116. Native **CONFIDENCE** predicates for filtering and sorting by epistemic certainty
117. The **EXPLAIN** clause for returning Fact Lineage alongside results

18.2 QB-Query Syntax Elements

18.2.1 MATCH Clause

The MATCH clause defines a graph pattern over Fact Units with typed edge traversal. It uses a path expression syntax where nodes are Fact Unit variables and edges are typed PredicateRefs.

```
MATCH CLAUSE SYNTAX:
====================

MATCH (fu1:FactType1 {field: value})-[r:PredicateRef]->(fu2:FactType2)
 (fu2)-[:AnotherPredicate*1..3]->(fu3)

-- Node pattern: (variable:FactType {field_filter})
-- Edge pattern: [variable:PredicateRef] or [variable:PredicateRef*min..max] for path length
ranges
-- Direction: -> (outgoing), <- data-id="3533" (incoming), -- (undirected)
-- FactType and PredicateRef may use domain namespace qualifier: MED:DIAGNOSIS,
SCI:FINDING

EXAMPLE:
MATCH (study:SCI:ResearchStudy)-[:SCI:hasResult]->(result:SCI:ExperimentalResult)
 (result)-[:SCI:supports]->(claim:SCI:ScientificClaim)
WHERE result.ConfidenceScore >= 0.80
 AND study.TemporalAnchor.T_start >= 2020-01-01T00:00:00Z
RETURN study, result, claim
```

18.2.2 WHERE Clause

The WHERE clause applies filter expressions over Fact Unit fields, supporting temporal predicates, confidence predicates, spatial predicates, SafetyFlags tests, and full-text semantic predicates.

```
WHERE CLAUSE FILTER PREDICATES:
================================
```

```
-- Confidence predicate
WHERE fu.ConfidenceScore >= 0.75 AND fu.ConfidenceScore <= data-id="3537" 1.0

-- Temporal predicate (overlaps with interval)
WHERE fu.TemporalAnchor OVERLAPS [2020-01-01T00:00:00Z, 2023-12-31T23:59:59Z]

-- Temporal predicate (before/after)
WHERE fu.TemporalAnchor.T_start AFTER 2022-01-01T00:00:00Z

-- Spatial predicate (within radius)
WHERE fu.SpatialAnchor WITHIN_RADIUS_KM(37.7749, -122.4194, 50)

-- SafetyFlags predicate
WHERE fu.SafetyFlags NOT CONTAINS RESTRICTED
 AND fu.SafetyFlags NOT CONTAINS HAZARDOUS

-- Semantic similarity predicate (semantic search)
WHERE fu SEMANTIC_SIMILAR_TO "mRNA vaccine efficacy against Omicron variant"
THRESHOLD 0.85

-- FactType predicate
WHERE fu.FactType IN (SCI:ExperimentalResult, SCI:MetaAnalysisResult)

-- SubjectRef entity type predicate
WHERE fu.SubjectRef.EntityType = MED:ClinicalTrial

-- Annotation predicate
```

```
WHERE fu.AnnotationSet["qb:prediction.outcome"] = "CONFIRMED"
```

18.2.3 ACROSS SCENARIOS Clause

The ACROSS SCENARIOS clause is the signature QB-Query extension enabling multi-branch Scenario Graph queries.

```
ACROSS SCENARIOS CLAUSE SYNTAX:
================================

ACROSS SCENARIOS scenario_graph_id [BRANCHES branch_filter] [WEIGHTED BY probability]

-- scenario_graph_id: UUID of the Scenario Graph to query across
-- BRANCHES: optional filter selecting specific KSS nodes to include
-- ALL (default): all branches above PruningThreshold
-- ROBUST_ONLY: only facts true in all branches (intersection)
-- BRANCH(kss_id): specific named branch
-- PROBABILITY_GT(0.1): branches with CumulativeProbability > 0.1
-- WEIGHTED BY: result rows include CumulativeProbability as a weight column

FULL EXAMPLE:
MATCH (company:FIN:Company)-[:FIN:hasRevenue]->(revenue:FIN:Measurement)
ACROSS SCENARIOS "550e8400-e29b-41d4-a716-446655440000"
 BRANCHES PROBABILITY_GT(0.05)
 WEIGHTED BY probability
WHERE revenue.TemporalAnchor OVERLAPS [2026-01-01Z, 2027-01-01Z]
RETURN company.name, revenue.ObjectValue, probability
```

```
ORDER BY probability DESC, revenue.ObjectValue DESC


-- Result: all projected revenue values across all scenarios with probability > 5%
-- Each row includes the scenario branch's CumulativeProbability as weight
-- Enables expected value computation: SUM(revenue * probability) / SUM(probability)
```

18.2.4 RETURN Clause

The RETURN clause specifies the output fields, aggregations, and ordering of query results.

```
RETURN CLAUSE SYNTAX:
======================

RETURN expression [AS alias] (, expression [AS alias])*
 [ORDER BY expression [ASC|DESC] (, expression [ASC|DESC])*]
 [LIMIT n] [SKIP n]

-- Supported expressions:
-- variable.field (e.g., fu1.ConfidenceScore)
-- variable (full Fact Unit as structured object)
-- COUNT(variable)
-- AVG(variable.field), SUM(variable.field), MIN, MAX
-- COLLECT(variable) -- collect into array
-- EXISTS((pattern)) -- boolean subquery
-- variable.FUID, variable.SemanticHash, variable.TemporalAnchor

EXAMPLE:
```

```
RETURN
 claim.SubjectRef.name AS entity_name,
 COUNT(result) AS supporting_study_count,
 AVG(result.ConfidenceScore) AS avg_confidence,
 COLLECT(study.FUID) AS study_ids
ORDER BY avg_confidence DESC
LIMIT 20
```

18.2.5 REGENERATE AS Clause

The REGENERATE AS clause invokes the MRE to transform the query result set into grounded natural language output.

```
REGENERATE AS CLAUSE SYNTAX:
==============================

REGENERATE AS LANGUAGE language_tag
 STYLE style_enum
 AUDIENCE audience_model_id
 [MAX_TOKENS n]
 [NARRATIVE_GRAPH narrative_graph_id]

-- language_tag: BCP-47 language tag (e.g., "en", "zh-Hans", "de", "ar")
-- style_enum: EXPOSITORY | ARGUMENTATIVE | DESCRIPTIVE | ANALYTICAL |
-- INSTRUCTIONAL | CONVERSATIONAL | EXECUTIVE_SUMMARY
-- audience_model_id: UUID of audience model in Audience Model Registry
-- MAX_TOKENS: optional output length constraint
```

```
-- NARRATIVE_GRAPH: optionally specify a pre-built Narrative Graph for regeneration

EXAMPLE:
MATCH (drug:MED:Drug)-[:MED:treats]->(disease:MED:Disease)
 (drug)-[:MED:hasTrialResult]->(trial:MED:ClinicalTrial)
WHERE disease.SubjectRef.name = "Type 2 Diabetes"
 AND trial.ConfidenceScore >= 0.75
 AND trial.TemporalAnchor.T_start AFTER 2020-01-01Z
RETURN drug, disease, trial
REGENERATE AS
 LANGUAGE "en"
 STYLE ANALYTICAL
 AUDIENCE "550e8400-e29b-41d4-a716-446655441234" -- Medical professional audience
model
 MAX_TOKENS 500
```

18.2.6 EXPLAIN Clause

The EXPLAIN clause appends Fact Lineage information to each result row, enabling full auditability of query results.

```
EXPLAIN CLAUSE SYNTAX:
======================

EXPLAIN [LINEAGE depth] [QUERY_PLAN]

-- LINEAGE depth: include Provenance chain up to 'depth' hops back from each result Fact
```

```
Unit
-- EXPLAIN LINEAGE 1: immediate parent Provenance Records only
-- EXPLAIN LINEAGE ALL: full backward traversal to raw ingestion events
-- QUERY_PLAN: return the query optimizer's execution plan instead of (or in addition to)
results

EXAMPLE:
MATCH (fu:SCI:ScientificClaim)
WHERE fu SEMANTIC_SIMILAR_TO "CRISPR gene editing cancer therapy" THRESHOLD 0.82
RETURN fu
EXPLAIN LINEAGE 2
LIMIT 5

-- Each result row includes: the Fact Unit fields AND a lineage_chain array
-- containing up to 2 levels of Provenance Records, enabling the caller to
-- verify the source of each claim without a separate lineage API call
```

18.3 Query Optimization

18.3.1 Query Planner Architecture

The QB-Query Optimizer transforms a parsed QB-Query expression into an optimal physical execution plan using a cost-based optimizer with the following components:

Optimizer Component	Responsibility
Parser	Converts QB-Query text to an abstract syntax tree (AST)
Logical Planner	Converts AST to a logical plan (relational algebra extended with graph operators); applies predicate pushdown, dead-code elimination, and MATCH clause reordering
Cardinality Estimator	Estimates the number of Fact Units satisfying each predicate using maintained statistics: per-FactType

Optimizer Component	Responsibility
	Fact Unit counts, per-predicate selectivity histograms, per-domain ConfidenceScore distributions, and per-entity degree distributions in the Fact Graph
Index Selector	Chooses between: FUID primary index, FactType inverted index, SemanticHash hash index, HNSW vector index (for SEMANTIC_SIMILAR_TO predicates), temporal interval tree (for OVERLAPS predicates), SDR Hamming index (for NCL-routed queries)
Join Orderer	For multi-hop MATCH clauses, determines the optimal traversal order through the Fact Graph using dynamic programming over the cardinality estimates; favors starting traversal from the most selective predicate
Physical Plan Generator	Maps the logical plan to physical operators: IndexScan, GraphTraversal, HashJoin, SDRLookup, VectorSearch, ScatterGather (for cross-shard queries), AcrossScenarios (for Scenario Graph queries)
Adaptive Re-planner	Monitors cardinality at runtime; if actual intermediate result sizes deviate from estimates by more than 10×, triggers mid-execution re-planning to switch join strategies

PART XVIII — SYSTEM INTEGRATION AND DEPLOYMENT

Part XVIII Executive Summary **Part XVIII specifies the deployment architecture of QueryBook, including the complete microservices inventory, Kubernetes deployment specifications, observability stack, and SDK/integration library specifications. This Part provides the engineering reference needed to deploy, operate, and scale a QueryBook instance.**

Chapter 19: Deployment Architecture

19.1 Microservices Architecture

Service Name	Responsibilities	Inter-Service Protocol	Scaling Policy
qb-ingestion-orchestrator	Manages ingestion job queue; rate limiting; idempotency; dead-letter routing	gRPC (internal); Kafka (stream consumption)	Horizontal; scale on queue depth metric
qb-ingestion-worker-{modality}	Per-modality ingestion branch processors (one service per modality: text, image, audio, video,	gRPC to Fact Unit Constructor	Horizontal; independent scaling per modality

Service Name	Responsibilities	Inter-Service Protocol	Scaling Policy
	structured, sensor, kg, code)		
qb-fact-unit-constructor	Assembles Fact Unit candidates from ISRs; computes SemanticHash, FUID; invokes Safety Substrate pre-insertion checks	gRPC (internal); writes to Active Store	Horizontal; scale on construction throughput metric
qb-entity-registry	Manages Entity Registry; NER disambiguation; provisional entity creation; external entity linking	gRPC (internal)	Horizontal with sticky routing by entity hash
qb-fact-graph-store	Persists Fact Graph; manages adjacency indexes; handles cross-shard queries via scatter-gather	gRPC (internal); Raft replication (internal)	Stateful; scale by adding shards; 3-replica minimum
qb-vector-index	Manages HNSW and SDR indexes; serves ANN and Hamming distance queries	gRPC (internal)	Stateful; sharded by FUID prefix; scale by adding shards
qb-provenance-ledger	Manages Merkle DAG Provenance Ledger; serves lineage traversal queries; handles tombstoning for erasure	gRPC (internal)	Stateful; append-only; scale read replicas independently
qb-ncl-engine	Neuromorphic Cognitive Layer: SDE, TMM, ARE, AN, SWS management; serves associative query processing	gRPC (internal)	Horizontal; session affinity for CCW state; scale on query latency P99
qb-pie-engine	Predictive Inference Engine; generates PREDICTION Fact Units; manages Scenario Graph construction from predictions	gRPC (internal); consumes Fact Stream from qb-stl-broker	Horizontal; scale on prediction job queue depth
qb-mre-engine	Multilingual Regeneration Engine; Narrative Graph construction and execution; hallucination prevention enforcement	gRPC (internal)	Horizontal; scale on regeneration request rate
qb-cdre-engine	Cross-Domain Rule Engine; incremental forward chaining; conflict detection; rule governance API	gRPC (internal); subscribes to Fact Graph change events	Horizontal; scale on rule evaluation throughput
qb-aem-service	Hosts all six AGI Extension Modules; manages AEM job queue; produces AEM-derived Fact Units	gRPC (internal)	Horizontal; scale on AEM job queue depth
qb-safety-substrate	Non-bypassable Safety Substrate enforcement	gRPC (internal); called synchronously by all	Horizontal; active-active; no single point of failure

Service Name	Responsibilities	Inter-Service Protocol	Scaling Policy
	service; manages SafetyFlags; writes to Safety Audit Log	other services at enforcement points	permissible
qb-stl-broker	Semantic Transport Layer message broker; manages topic routing, consumer groups, stream partitioning	QB-Stream Protocol (proprietary); Kafka-compatible consumer API	Stateful; Kafka-compatible partition rebalancing
qb-query-gateway	QB-Query parser, optimizer, and execution coordinator; routes sub-queries to NCL, PIE, Fact Graph Store, and Scenario Graph services	REST + gRPC (external API); gRPC (internal routing)	Horizontal; scale on API request rate and active query count
qb-scenario-graph-service	Manages Scenario Graph CRUD; executes cross-branch queries; KSS consistency validation	gRPC (internal)	Horizontal; scale on active Scenario Graph count
qb-ontology-mesh-service	Serves Ontology Mesh type lookups and Predicate Library; manages schema versioning and SCP workflow	gRPC (internal)	Horizontal read replicas; single primary for writes
qb-admin-api	Administrative API for system configuration, federation management, user management, audit log access	REST (HTTPS only; mTLS required)	Low-traffic service; minimal horizontal scaling

19.2 Kubernetes Deployment

19.2.1 Helm Chart Structure

```
querybook/
 Chart.yaml # Chart metadata (version, description, dependencies)
 values.yaml # Default configuration values
 values-production.yaml # Production overrides (not in VCS)
 templates/
 namespace.yaml # QueryBook namespace definition
 rbac/ # ServiceAccount, ClusterRole, RoleBinding per service
```

```
configmaps/ # Per-service configuration ConfigMaps
secrets/ # External secret references (Sealed Secrets)
statefulsets/ # StatefulSets for: fact-graph-store, vector-index,
# provenance-ledger, stl-broker
deployments/ # Deployments for all stateless services
services/ # Kubernetes Service definitions (ClusterIP, headless)
ingress/ # Ingress for query-gateway and admin-api (TLS termination)
hpas/ # HorizontalPodAutoscaler definitions
pdbs/ # PodDisruptionBudget definitions
network-policies/ # NetworkPolicy restricting inter-service communication
```

19.2.2 Resource Requirements (per pod, production)

Service	CPU Request	CPU Limit	Memory Request	Memory Limit	Min Replicas
qb-ncl-engine	8 cores	16 cores	64 GB	128 GB	3
qb-fact-graph-store	4 cores	8 cores	32 GB	64 GB	3 (per shard)
qb-vector-index	8 cores	16 cores	128 GB	256 GB	3 (per shard)
qb-mre-engine	4 cores	8 cores	16 GB	32 GB	3
qb-safety-substrate	2 cores	4 cores	8 GB	16 GB	5 (high availability)
qb-query-gateway	2 cores	4 cores	4 GB	8 GB	3
qb-ingestion-worker-{modality}	2 cores (text/structured); 8 cores (audio/video/image)	4–16 cores	8–32 GB	16–64 GB	2 per modality

19.3 Observability Stack

Signal Type	Technology	Key Metrics / Signals	SLO
Metrics	Prometheus (scrape interval: 15s); Grafana dashboards	qb_query_latency_ms (p50, p99), qb_fact_unit_ingestion_rate,	Query p99 < 200ms; ingestion rate > 10k FU/s; safety enforcement error rate = 0

Signal Type	Technology	Key Metrics / Signals	SLO
		qb_safety_enforcement_events_total, qb_conflict_index_size, qb_ncl_attractor_convergence_iterations	
Distributed Tracing	OpenTelemetry SDK (all services); Jaeger backend	Full trace per QB-Query; per-stage ingestion pipeline spans; Safety Substrate enforcement spans	100% of queries traced; trace sampling rate for high-volume services: 1%
Logging	Structured JSON logs; Fluentd collection; Elasticsearch storage; Kibana dashboards	All logs include: service_name, pod_id, trace_id, span_id, caller_id, fuid (where applicable), severity, message	Log retention: 90 days hot; 7 years archive
Alerting	Prometheus Alertmanager; PagerDuty integration for P0/P1	Alerts on: query p99 > 200ms sustained 5m; ingestion lag > 30m; safety enforcement failures; replication lag > 1m; Provenance Ledger write failures	P0 alert page within 2 minutes; P1 within 10 minutes

19.4 SDK and Integration Libraries

Language	Package Name	Key Features	Auth Support
Python	`querybook-sdk` (PyPI)	Sync and async clients; QB-Query builder DSL; QBF serialization/deserialization; Fact Stream consumer; pandas DataFrame integration for structured data ingestion	OAuth 2.0 client credentials; OIDC device flow
TypeScript / JavaScript	`@querybook/sdk` (npm)	Promise and async/generator based API; QB-Query template literals; browser and Node.js support; React hooks for reactive Fact Stream subscriptions	OAuth 2.0 PKCE for browser; client credentials for Node.js
Java	`com.querybook:querybook-sdk` (Maven Central)	Reactive (Project Reactor) and blocking clients; Spring Boot auto-configuration starter; JPA-inspired Fact Unit repository pattern	OAuth 2.0 client credentials; Spring Security integration
Go	`github.com/querybook/querybook-go`	Idiomatic Go interfaces; goroutine-safe Fact	OAuth 2.0 client credentials; token refresh

Language	Package Name	Key Features	Auth Support
		Stream consumer; QBF codec; context-based cancellation	middleware
Rust	querybook (crates.io)	Zero-copy QBF deserialization using serde; async Tokio-based client; memory-safe Fact Unit manipulation; WebAssembly compilation target for browser-embedded use	OAuth 2.0 client credentials

19.8 Q&A — The Specification Assistant Agent (added in Version 1.0 merge review, Pass 2)

19.8.1 Purpose

Q&A is a dedicated AI agent chatbot, distinct from QueryBook's own product-facing query interface, whose sole job is to answer questions about this Bible itself — its architecture, terminology, chapter cross-references, and open gaps (such as the ones logged in the Merge Revision Note at the front of this document). Q&A exists because a specification of this size is hard for a new engineer, patent attorney, or architect to navigate by reading linearly; Q&A lets them ask a question in plain language and get an answer grounded in the actual chapter and section text, with a citation back to it.

19.8.2 Why It Is Built on QueryBook's Own Stack

Rather than building a separate document-QA tool, Q&A is implemented as a self-referential QueryBook deployment: this Bible is ingested as its own Fact Graph, and Q&A is the Adaptive Response Engine (Chapter 52) and Multilingual Regeneration Engine (Chapter 16) pointed at that graph. This is a deliberate design choice, not a shortcut — it means Q&A inherits the Factual Groundedness metric already specified for the Narrative Graph (§12, per-sentence proposition-to-Fact-Unit tracing), so every answer it gives can be traced back to a specific chapter and section rather than being generated from general knowledge about QueryBook. It also means Q&A is a living demonstration of the architecture it explains.

19.8.3 Ingestion of This Bible

- Source: the Bible is ingested as a single BATCH source (§49.3.2.1) through the Unstructured Text Extraction branch (§4.3.1), chapter by chapter, with each numbered section (e.g. §3.1, §8.5.1,

§9.5.3) becoming its own Fact Bundle (§3.4) so that a retrieved answer can cite a specific section rather than an entire chapter.

- Re-ingestion on edit: every merge or amendment to this Bible (see the Merge Revision Note) triggers re-ingestion of the changed sections only, keyed by section number, so Q&A's answers stay current with the latest merged version without a full re-index.
- Cross-references preserved: section cross-references already present in the text (for example, "see §8.1") are extracted as RelationSet edges (§3.1) between the corresponding Fact Bundles, so Q&A can follow a cross-reference the same way a human reader would flip to another chapter.

19.8.4 Interaction Model

Q&A is available as a chat interface alongside the document (embedded panel in the QueryBook documentation viewer) and as an API, described below. A typical interaction:

- User asks: “Why does the FUID use UUID v7 if it also has to be deterministic?”
- Q&A retrieves the relevant Fact Bundles (§2.1 glossary entry, §3.1 Fact Unit Schema, §3.1.1 Implementation Note) via the same Associative Recall Engine described in §7.2.3, applies the Adaptive Response Engine's Response Profile for the asking user's role (Chapter 52), and returns an answer that cites §3.1.1 directly — the same implementation note added in the first merge review pass.
- If no Fact Bundle in the ingested Bible supports an answer, Q&A says so explicitly rather than filling the gap from general knowledge — the same Without-Information Rule preference for partial-but-grounded answers established in D0.4 applies here: Q&A will point to the nearest related section rather than staying silent, but it will not present an ungrounded guess as if it were specification text.

19.8.5 API

Q&A adds one endpoint to the API surface specified in §9.5.1, following the same table format:

- **POST /v1/qa/ask:** submits a natural-language question about this Bible; returns a generated answer plus the list of section numbers (FUIDs of the supporting Fact Bundles) it was grounded in. Requires a Bearer token; no elevated role is required, since Q&A only ever answers from this Bible's own already-public-to-the-reader content and never has access to a live QueryBook deployment's ingested customer data.

- **GET /v1/qa/sections/{section_id}:** returns the Fact Bundle for a given section number directly, without going through natural-language generation — useful for building a “jump to source” link under a Q&A answer. Requires a Bearer token.

19.8.6 Governance

Q&A GOVERNANCE NOTE **Q&A is a Regeneration Gate consumer like any other MRE-driven output (§8.1, enforcement point 46) and is subject to the Prime Directive (Domain 0) and standard Safety Substrate enforcement. Because Q&A's Fact Graph is this Bible rather than a customer deployment's data, its practical exposure is limited to the SafetyFlags already assigned to Bible content itself (§8.2) — for example, if a future chapter marks a subsection RESTRICTED, Q&A enforces that the same way it would for any other Fact Unit.**

Q&A does not replace product-facing QueryBook query access (§9.5.1, §18 QB-Query Language) — it is scoped specifically to this specification document and is intended for the audiences named on this Bible's title page: patent attorneys, senior engineers, AI researchers, and product architects.

19.9 Fact Graph Domain Sharding at High Query Volume

This section specifies a concrete sharding key and replica topology for the Fact Graph at high query volume, closing a gap identified directly: §19.1's microservices architecture specifies per-service horizontal scaling policies but did not previously specify how the Fact Graph itself is partitioned for read scaling. This section adds a partitioning scheme; it modifies no existing service's specification.

DOMAIN-SHARDED, USING STRUCTURE THAT ALREADY EXISTS **The Ontology Mesh's federated domain sub-ontologies (§6.1, §20.1 of the patent) already classify every Fact Unit by domain. This section reuses that existing classification as the shard key, rather than introducing a new partitioning scheme — a Finance-domain-heavy query load and a Medicine-domain-heavy query load route to independently-scaled replica pools without any new metadata or reclassification.**

19.9.1 Replica Topology

Layer	Partitioning	Scaling Trigger
Write path	Single authoritative store per Fact Unit, unpartitioned by domain — consistent with the Provenance Ledger's existing requirement for one canonical record (Chapter 5).	N/A — write volume is orders of magnitude lower than read volume at this scale and is not the bottleneck this section addresses.

Read replica pools	Sharded by a Fact Unit's primary domain classification (§6.1); a Fact Unit registered under multiple domains is replicated to each relevant pool.	Per-pool horizontal scaling on query queue depth, following the same policy pattern already specified for other services at §19.1.
Cross-domain query fan-out	A query spanning multiple domains (reusing AEM's existing cross-domain synthesis, Chapter 15) fans out to each relevant pool and merges results before Safety Substrate evaluation.	N/A — fan-out is bounded by the query's own resolved domain set, not a separate scaling dimension.

19.9.2 Cache Invalidation Keyed to Fact Unit Version, Not Time

Because grounded output is deterministic given a fixed Fact Graph state (Claim 8), a cached response remains valid until its specific underlying Fact Unit's version changes — not on a fixed time-to-live. A cache entry is invalidated by the same Provenance Ledger event that records a Fact Unit's revision (Chapter 5), giving exact, immediate invalidation rather than approximate, time-based staleness. This is a structural consequence of the grounding guarantee already established; it is not a new consistency mechanism.

19.9.3 Safety Substrate Capacity as the Throughput Floor

Because Safety Substrate evaluation remains synchronous and mandatory for every query regardless of scale (Chapter 8), its replica capacity is the effective ceiling on total system throughput, not a service sized after the rest of the architecture. Capacity planning for this section treats Safety Gate throughput as the binding constraint and sizes every other layer's target capacity to remain non-limiting relative to it.

19.9.4 Integration Requirements

This section requires the Ontology Mesh's federated domain classification (§6.1, §20.1 of the patent); the Provenance Ledger (Chapter 5, Claim 2); AEM's cross-domain synthesis (Chapter 15); the Safety Substrate (Chapter 8); and §19.1's existing per-service scaling policy pattern. It modifies none of their specifications.

Part XIX Executive Summary **Part XIX constitutes the patent reference framework — the organized inventory of QueryBook's patentable innovations, differentiation from prior art, and the claim dependency map for portfolio strategy. Each innovation cluster is traced to the technical chapters where it is specified, enabling patent attorneys to construct claims with precise cross-references to this Bible.**

Chapter 20: Patent Claim Reference Index

20.1 Patentable Innovation Clusters

Cluster	Title	Core Novel Elements	Specified In
A	Fact Unit Schema and FUID Generation	The specific combination of 20+ typed fields in a single atomic knowledge unit; the deterministic FUID generation algorithm from content+source+timestamp; the SemanticHash language-independent fingerprinting method; the SafetyFlags bitmask as an inseparable field of the knowledge atom itself	Chapter 3, Appendix A
B	Neuromorphic Cognitive Layer Architecture	The application of Sparse Distributed Representations to knowledge graph nodes at scale; the STDP-inspired learning algorithm applied to knowledge graph edge weights; the Attractor Network convergence mechanism for knowledge query stabilization; the EWC analog preventing catastrophic forgetting in a knowledge graph setting; the CCW as a queryable, persistable working memory for knowledge sessions	Chapter 7
C	Multimodal Ingestion and	The unified six-stage	Chapter 4

Cluster	Title	Core Novel Elements	Specified In
	Semantic Extraction Pipeline	sequential pipeline architecture accepting eight distinct modalities; the cross-modal confidence bonus for multi-modal corroboration; the per-modality branch processor architecture with independent failure isolation; the composite confidence calibration model integrating extraction confidence, source reliability, modality coefficient, and temporal decay	
D	QueryBook Format (QBF) Binary Specification	The specific file structure with magic bytes, header, schema, payload, index, and trailer blocks; the streaming frame format with 1-byte record type and 4-byte length prefix enabling re-synchronization after corruption; the EmbeddingVector sidecar storage mechanism; the backward-compatibility rules for optional field addition without version increment	Chapter 13, Appendix C
E	Semantic Transport Layer Protocol	The six-layer protocol stack specifically designed for Fact Unit transmission; the FactType-to-topic routing rule in Layer 5; the SubjectRef-hash partition assignment ensuring related facts are co-located in the same stream partition; the QB-Streaming subscription filter language combining FactType, domain, confidence, and SafetyFlags predicates in a single filter expression	Chapter 9
F	Scenario Graph Construction and Probabilistic Traversal	The Scenario Graph as a first-class system object with persistent storage and versioning; the	Chapter 11

Cluster	Title	Core Novel Elements	Specified In
		Knowledge State Snapshot (KSS) as a delta representation (OverrideFacts + RemovedFUIDs + BaseFUIDs) enabling memory-efficient hypothetical state representation; the PruningThreshold-based branch elimination; the Robust Fact identification algorithm via probabilistically weighted intersection across branches	
G	Narrative Graph Construction and Hallucination-Prevented Regeneration	The Narrative Graph as a distinct system object mediating between Fact Graph and natural language output; the nine Narrative Edge Types (TEMPORAL_FOLLOWS, CAUSES, CONTRASTS_WITH, ELABORATES, EXEMPLIFIES, SUMMARIZES, QUOTES, SUPPORTS, QUALIFIES); the gap detection and gap-filling algorithm; the Factual Groundedness metric (per-sentence proposition-to-Fact-Unit tracing); the architectural hallucination prevention via constrained generation vocabulary derived from Narrative Graph Fact Units	Chapters 12, 16
H	Safety Substrate Architecture and Non-Bypassable Enforcement	The embedding of safety enforcement as a synchronous mandatory gate at eight specific system layers; the SafetyFlags bitmask as an inseparable, Safety-Substrate-controlled Fact Unit field; the canary Fact Unit mechanism for exfiltration and poisoning detection; the statistical watermarking of exports for forensic attribution;	Chapter 8, Appendix D

Cluster	Title	Core Novel Elements	Specified In
		the cryptographic tombstoning mechanism satisfying GDPR erasure without Merkle DAG integrity disruption	
I	AGI Extension Modules	AEM-1: Bayesian posterior-ranked abductive hypothesis generation from Fact Graph observations; AEM-2: Structure-mapping-theory-based analogical reasoning at Fact Graph subgraph level with three-dimensional quality scoring; AEM-3: Integration of PC, FCI, NOTEARS, and do-calculus causal discovery methods over typed Fact Units; AEM-4: Minimum-intervention counterfactual planning using causal Fact Graph traversal; AEM-5: Meta-cognitive monitoring producing Meta-Fact Units describing the system's own epistemic state; AEM-6: Backward-chaining goal-directed knowledge synthesis planner producing GOAL_PLAN Fact Units	Chapter 15
J	Multilingual Regeneration Engine with Factual Grounding	The language-independent Semantic Hash guaranteeing cross-lingual fact identity; the six-stage grounded regeneration pipeline; the Factual Groundedness metric enforced as a mandatory gate at Stage 6; the constrained generation vocabulary architecture preventing hallucination; the simultaneous multi-language regeneration from a single Narrative Graph	Chapter 16
K	Cross-Domain Rule Engine and Cross-Ontology Consistency	The QB-Datalog rule language with probabilistic weight	Chapter 17

Cluster	Title	Core Novel Elements	Specified In
		extensions and temporal scoping operators; the incremental forward chaining implementation via Differential Dataflow; the Conflict Index as a persistent, queryable record of all active Fact Graph contradictions; the staged rule deployment workflow with impact analysis and shadow mode	
L	Predictive Inference Engine and Prediction Fact Units	The representation of predictions as first-class Fact Units of FactType PREDICTION with full provenance and lifecycle tracking; the automatic archival with outcome determination after the prediction horizon passes; the continuous calibration evaluation against subsequently ingested ground truth; the Monte Carlo sampling over Scenario Graphs as a prediction confidence interval method	Chapter 10
M	Provenance Ledger Merkle DAG Architecture	The Merkle DAG structure applied to a knowledge system's operation log; the SafetyChecksum HMAC on every Provenance Record enabling Safety Substrate bypass detection; the CORROBORATION event type linking independently derived Provenance chains; the cryptographic tombstoning mechanism for GDPR-compliant erasure that preserves Merkle DAG integrity; the RawSourceRef content-addressed link to source archive material enabling full reproducibility	Chapter 5
N	Five-Tier Storage Architecture and HNSW	The five-tier temperature-stratified	Chapter 14

Cluster	Title	Core Novel Elements	Specified In
	Vector Indexing	storage architecture with automated promotion/demotion between tiers based on access patterns and lifecycle state; the application of HNSW at 10B+ Fact Unit scale with shard-parallel ANN search; the separate SDR index with SIMD-optimized Hamming distance computation as the NCL's primary search structure; the CRDT-based multi-region convergence with SafetyFlags max-bitmask CRDT preventing cross-region flag downgrade	
O	QB-Query Language with Novel Clauses	The ACROSS SCENARIOS clause enabling declarative multi-branch Scenario Graph querying; the REGENERATE AS clause invoking the MRE from within a declarative query; the SEMANTIC_SIMILAR_TO predicate combining vector similarity with Fact Unit field filters in a unified WHERE clause; the EXPLAIN LINEAGE clause appending configurable-depth Provenance chains to query results; the integrated cost model unifying graph traversal, vector search, temporal range, and scenario graph cardinality estimation	Chapter 18

20.2 Prior Art Differentiation

Cluster	Closest Prior Art	Key Differentiators (Non-Obvious)
A (Fact Unit)	RDF Triple (Subject, Predicate, Object) in W3C standards; property graph nodes in Neo4j	RDF triples have no native ConfidenceScore, SafetyFlags, EmbeddingVector, TemporalAnchor with confidence,

Cluster	Closest Prior Art	Key Differentiators (Non-Obvious)
		SemanticHash, or VersionChain. No prior art combines all 20 fields into a single atomic unit designed for multi-modal, multi-source, provenance-tracked knowledge representation at AGI scale.
B (NCL)	Numenta's HTM theory; Hopfield Networks; FAISS vector indexes	No prior system applies SDRs, STDP, Attractor Networks, and HTM to a typed knowledge graph with structured Fact Units (as opposed to raw embedding vectors). The EWC analog applied to SWS weights is novel. The integration with FUID-typed graph structure (as opposed to untyped embedding space) is non-obvious.
C (Ingestion)	Apache NiFi (data flow); LlamaIndex (RAG ingestion); LangChain document loaders	No prior system unifies 8 modalities into a single typed Fact Unit schema with per-modality confidence calibration, cross-modal corroboration bonus, and mandatory Safety Substrate gates at each stage. RAG systems do not produce typed, provenance-tracked, semantically hashed knowledge atoms.
F (Scenario Graph)	Bayesian Networks; decision trees; Monte Carlo simulation	Scenario Graphs represent states as KSS delta objects over the live Fact Graph (not independent probability tables), enabling dynamic reuse of current knowledge and real-time update as the Fact Graph evolves. The KSS delta representation (BaseFUIDs + OverrideFacts + RemovedFUIDs) is novel. No prior system integrates Scenario Graph construction directly with a PIE prediction engine and makes scenarios queryable via a declarative query language clause.
G (Narrative Graph + Hallucination Prevention)	Natural Language Generation (NLG) systems; RAG with citation; GPT-4 with tool use	All prior NLG and LLM-based systems are probabilistic at the token level and cannot provide an architectural hallucination prevention guarantee. QueryBook's hallucination prevention is structural: the generation vocabulary is derived from Narrative Graph Fact Units, making it impossible (not merely unlikely) to generate ungrounded claims. No prior system has implemented a Factual Groundedness metric as a

Cluster	Closest Prior Art	Key Differentiators (Non-Obvious)
		mandatory gate in an NLG pipeline with automatic ungrounded-sentence rejection and graceful degradation.
H (Safety Substrate)	AWS IAM; RBAC systems; OpenAI content filters; Anthropic Constitutional AI	Prior safety systems are optional wrappers or soft constraints that can be bypassed via prompt injection, API key compromise, or configuration change. The QueryBook Safety Substrate is embedded synchronously in the execution path of every Fact Unit operation, making bypass architecturally impossible rather than merely difficult. The canary Fact Unit mechanism and statistical watermarking for forensic attribution are novel. The SafetyFlags bitmask as an inseparable, Safety-Substrate-controlled field of the knowledge atom is non-obvious.
O (QB-Query)	SPARQL 1.1; openCypher; SQL	No prior query language for knowledge systems includes: (1) an ACROSS SCENARIOS clause for probabilistic multi-branch querying, (2) a REGENERATE AS clause invoking grounded NLG within a declarative query, (3) native CONFIDENCE predicates in a standard WHERE clause, or (4) an EXPLAIN LINEAGE clause for in-query provenance retrieval. The integration of graph pattern matching, temporal predicates, vector similarity, and Scenario Graph traversal in a single unified syntax with a cost-model-based optimizer is non-obvious.

20.3 Claim Dependency Map

The following table describes the layered patent portfolio strategy. Claims in higher-level clusters depend on the foundational claims in lower-level clusters, enabling a stratified defense-in-depth patent portfolio.

Portfolio Layer	Clusters	Description	Dependency
Layer 1 — Foundational	A, D, M	The Fact Unit schema (A), QBF binary format (D), and Provenance Ledger	No dependencies — foundational

Portfolio Layer	Clusters	Description	Dependency
		Merkle DAG (M) are the foundational claims on which all other QueryBook capabilities depend. These claims are narrowest but most defensible as the essential substrate.	
Layer 2 — Knowledge Infrastructure	C, E, H, N	Ingestion Pipeline (C), Semantic Transport Layer (E), Safety Substrate (H), and Storage Architecture (N) depend on Layer 1's Fact Unit and QBF definitions and constitute the knowledge acquisition and persistence infrastructure.	Depends on Layer 1 (A, D, M)
Layer 3 — Semantic Structure	K, G (Narrative Graph component)	The Cross-Domain Rule Engine (K) and Narrative Graph (part of G) provide the semantic organization layer over the knowledge infrastructure.	Depends on Layers 1–2
Layer 4 — Cognitive Reasoning	B, F, L, I	The Neuromorphic Cognitive Layer (B), Scenario Graph System (F), Predictive Inference Engine (L), and AGI Extension Modules (I) constitute the reasoning layer operating over the knowledge infrastructure and semantic structure.	Depends on Layers 1–3
Layer 5 — Human Interface	G (MRE component), J, O	The Multilingual Regeneration Engine (G, J) and QB-Query Language (O) constitute the human interface layer, enabling users and agents to interact with the full cognitive system via grounded natural language and declarative queries.	Depends on Layers 1–4

APPENDICES

PART X — RESPONSE ARCHITECTURE

Part X Executive Summary **Part X defines the Response Architecture of the QueryBook system — the pipeline stage responsible for transforming a verified fact retrieval result set into an audience-calibrated, discipline-appropriate, expertise-levelled natural language response. Chapter 52 specifies the Adaptive Response Engine (ARE), the formal Stage 6 layer of the QB response pipeline, including its Response Profile data structure, Expertise Level Taxonomy, Discipline Mode Registry, three-tier governance cascade, five-stage processing pipeline, and Generation Directive output. Part X is essential reading for engineers implementing the NLGE interface, patent attorneys constructing claims around audience-adaptive knowledge generation, and product architects designing operator and subscriber configuration systems.**

PASS 3 — NEWLY DRAFTED CONTENT **Chapters 48–51 below were absent from every prior draft of this Bible under any numbering scheme. They are drafted here for the first time, grounded in the specific section numbers already cited from Chapter 52 and elsewhere, and inserted as new chapters — not modifying any existing chapter.**

Chapter 48: QueryBook as a Semantic Programming Platform — QBQL, QBPL, and NLPL

48.1 Introduction — The Three Language Forms

QueryBook exposes the same Fact Graph substrate through three progressively more expressive language forms: QBQL (query), QBPL (programming), and NLPL (natural language). A QBPL program compiles to a sequence of QBQL-equivalent graph operations plus two additional primitives (§48.3.1); an NLPL program compiles to a QBPL program via the five-stage pipeline in §48.4.2. QBQL is the semantic floor of the platform; NLPL is its ceiling.

48.2 QBQL — QueryBook Query Language

QBQL's formal grammar, keyword set, data types, and execution model are fully specified in Chapter 18 and are not repeated here. In this chapter's context, QBQL is Layer 1 of the platform: a declarative, read-oriented language over the Fact Graph with no mutation or control-flow primitives of its own.

48.3 QBPL — QueryBook Programming Language

This table provides a consolidated reference for all Fact Unit fields as specified in Chapter 3.1. It is intended for use in patent claim drafting and API implementation verification.

Field Name	Type	Required / Optional	Default Value	Constraints	Brief Description
FUID	128-bit UUID v7	Required	Generated at construction	Globally unique; deterministically derived	Primary key and global identity
FactType	OntologyMesh.TypeID (UUID ref)	Required	None	Must resolve to valid, non-deprecated Ontology Mesh type	Semantic type classifier
SubjectRef	EntityRef (UUID → QB-ER)	Required	None	Must resolve to valid entity (may be PROVISIONAL)	Primary entity this fact describes
PredicateRef	PredicateRef (UUID → Predicate Library)	Required	None	Must be consistent with FactType and SubjectRef entity type	Relationship or attribute being asserted
ObjectRef	EntityRef (UUID → QB-ER)	Conditional	null	Mutually exclusive with ObjectValue; required when object is a named entity	Named entity as object of assertion
ObjectValue	QBF-typed scalar	Conditional	null	Mutually exclusive with ObjectRef; type must match PredicateRef schema	Scalar value as object of assertion
TemporalAnchor	Struct {T_start, T_end, T_granularity, T_confidence}	Required	None	T_start ≤ T_end (if T_end present); T_confidence ∈ [0.0, 1.0]	Time interval to which this fact applies
SpatialAnchor	Struct {lat, lon, altitude, geohash, admin_region}	Optional	null	lat ∈ [-90, 90]; lon ∈ [-180, 180]; geohash precision 1–12	Geographic location reference
ConfidenceScore	float32	Required	Computed at ingestion	∈ [0.0, 1.0]; not exactly 0.0 unless definitively falsified	Epistemic certainty score
ProvenanceRef	ProvenanceID (SHA-256)	Required	Generated at construction	Must resolve to valid Provenance	Link to Provenance Ledger root

Field Name	Type	Required / Optional	Default Value	Constraints	Brief Description
				Ledger record; immutable after creation	record
SemanticHash	256-bit bytes (SHA-3/256)	Required	Computed at construction	Recomputed on every modification; language-independent	Content-addressable semantic fingerprint
ModalitySignature	uint8 bitmask	Required	Computed at ingestion	At least one bit must be set	Contributing input modalities
EmbeddingVector	float32[4096]	Required	Computed at ingestion	L2-normalized; dimensionality 4096 default; stored in sidecar	Semantic position in embedding space
RelationSet	Set of RelationEdge	Optional	Empty set	Max 65,536 edges; weight ∈ [0.0, 1.0]; no self-loops	Typed weighted edges to related Fact Units
BundleRefs	List of BundleID (UUID)	Optional	Empty list	Max 1,024 memberships; each must resolve to valid Bundle	Fact Bundle memberships
VersionChain	Ordered List of FUID	Optional	Empty list	Chronological order; referenced FUIDs must be SUPERSEDED; max depth 4,096	Prior version history
SafetyFlags	uint32 bitmask	Required	0x00000001 (SAFE)	Controlled by Safety Substrate; bits 15–31 reserved (must be 0)	Safety and compliance classification
LinguisticMetadata	Struct {language, register, formality, domain_tag}	Required	Inferred from source	Valid BCP-47 language tag required	Linguistic context for regeneration
AnnotationSet	Map<string, QBF scalar>	Optional	Empty map	Max 512 entries; "qb:" prefix reserved; max key 256 bytes; max value 64KB	Extensible domain-specific metadata
TTL	Struct {ttl_seconds, eviction_policy}	Optional	null (no expiry)	Minimum TTL 3,600 seconds if set	Expiry and eviction policy
AccessControlList	List of ACLEntry	Required	OWNER entry for creating principal	Must contain at least one OWNER; DENY	Role-based and attribute-based access control

Field Name	Type	Required / Optional	Default Value	Constraints	Brief Description
				overrides ALLOW	

Appendix B: Complete Ontology Mesh Top-Level Taxonomy

Layer	Category	Subcategory	Definition
Upper	Entity	Person, Organization, Location, Artifact, Biological_Organism, Abstract_Concept, Document, Dataset, System	A thing that exists independently and can be the subject or object of facts; the fundamental ontological category for named things in the world
Upper	Event	Natural_Event, Human_Activity, Process_Occurrence, Transaction, Communication_Event, Conflict_Event, Election_Event	A thing that happens at a specific time or over a time interval, involving participants; events are bounded in time and causally connected to states
Upper	State	Physical_State, Social_State, Economic_State, Epistemic_State, Legal_State, Health_State, Environmental_State	A condition or situation that holds for an entity over a time interval; states are the persistence medium between events
Upper	Relation	Causal_Relation, Parthood_Relation, Temporal_Relation, Social_Relation, Spatial_Relation, Equivalence_Relation, Opposition_Relation	A typed connection between two or more entities, events, or states; relations are the edges of the semantic world model
Upper	Property	Physical_Property, Social_Property, Epistemic_Property, Normative_Property, Quantitative_Property, Qualitative_Property	An attribute or characteristic that an entity, event, or state may possess; properties are inherent features rather than relational connections
Upper	Process	Biological_Process, Chemical_Process, Social_Process, Computational_Process, Economic_Process, Geological_Process	A sequence of causally connected events or state changes that unfold over time according to regular patterns; processes are distinguished from events by their temporal extension and regularity

Layer	Category	Subcategory	Definition
Upper	Region	Geographic_Region, Temporal_Region, Conceptual_Domain, Administrative_Region, Cyberspace_Region	A bounded area in space, time, or an abstract domain; regions are the containers within which entities, events, and states are located
Upper	Time	Time_Instant, Time_Interval, Duration, Temporal_Order, Recurrence_Pattern	A representation of temporal position, interval, or ordering; the temporal ontology is unified with the TemporalAnchor Fact Unit field
Domain	Science (SCI)	ResearchStudy, Hypothesis, Experiment, Finding, Dataset, ResearchInstitution, Peer_Reviewed_Publication, Grant	Scientific entities and knowledge artifacts; governed by the SCI domain ontology steward
Domain	Law (LAW)	Statute, Regulation, Case, Ruling, LegalEntity, Jurisdiction, Contract, Patent, Treaty, LegalPrecedent	Legal entities, documents, and processes; governed by the LAW domain ontology steward
Domain	Medicine (MED)	Disease, Drug, ClinicalTrial, MedicalProcedure, Symptom, Biomarker, Patient_Cohort, Medical_Device, Protein, Gene	Medical and biomedical entities; governed by the MED domain ontology steward; HIPAA-sensitive by default
Domain	Finance (FIN)	FinancialInstrument, Market, Transaction, Portfolio, RegulatoryBody, EconomicIndicator, Company, Fund, Exchange	Financial entities and instruments; governed by the FIN domain ontology steward; FINRA-regulated facts require additional SafetyFlags
Domain	Engineering (ENG)	SystemComponent, Interface, Specification, Standard, TechnicalProcess, Patent, SoftwareLibrary, HardwareComponent	Engineering and technical entities; governed by the ENG domain ontology steward; includes the QueryBook system's own self-description
Domain	Geopolitics (GEO)	Country, Government, Military_Force, Treaty, PoliticalParty, ElectionEvent, ConflictEvent, SanctionRegime	Geopolitical entities and events; governed by the GEO domain ontology steward; high risk of EXPORT_CONTROLLED and LEGALLY_SENSITIVE flags
Domain	Culture (CUL)	ArtWork, CulturalMovement, Language, Religion,	Cultural entities and artifacts; governed by the CUL domain ontology

Layer	Category	Subcategory	Definition
		CulturalPractice, Person, Historical_Period	steward
Domain	Technology (TEC)	SoftwareSystem, Algorithm, AIModel, HardwarePlatform, API, ComputingInfrastructure, DataFormat	Technology entities and systems; governed by the TEC domain ontology steward

Appendix C: QBF Byte Layout Diagram

The following represents an annotated canonical QBF file byte layout for a single-Fact-Unit file. Byte offsets are relative to the start of the file.

```
QBF CANONICAL BYTE LAYOUT (Single Fact Unit, Uncompressed, Unsigned)
=====================================================================

OFFSET SIZE FIELD VALUE / DESCRIPTION
------ ------ ---------------------- ------------------------------
0x0000 4 Magic bytes 51 42 46 00 ("QBF\0")
0x0004 1 Major version 01
0x0005 1 Minor version 00

-- HEADER BLOCK (256 bytes, starts at 0x0006) --
0x0006 8 Total payload size uint64 LE (varies)
0x000E 8 Schema version uint64 LE (e.g., 00 01 00 00 = v1.0.0)
0x0016 1 Compression algo 00 (NONE)
0x0017 1 Encryption flag 00 (PLAINTEXT)
0x0018 1 Checksum algo 00 (CRC32C)
0x0019 1 Record type 01 (FACT_UNIT)
0x001A 8 Creation timestamp int64 LE (Unix nanoseconds UTC)
0x0022 8 FUID count 01 00 00 00 00 00 00 00 (1 Fact Unit)
```

0x002A 8 Index offset 00 00 00 00 00 00 00 00 (no index)

0x0032 12 Encryption IV 00…00 (not encrypted)

0x003E 16 Encryption tag 00…00 (not encrypted)

0x004E 180 Reserved 00…00

-- SCHEMA BLOCK (starts at 0x0106) --

0x0106 4 Block length 00 00 00 00 (standard schema, 0 bytes)

-- PAYLOAD BLOCK (starts at 0x010A) --

0x010A -- Field presence bitmask [3 bytes: 21 fields → 3 bytes bitmask]

0x010D 16 FUID [16 bytes UUID, big-endian]

0x011D varint FactType [UUID ref, 16 bytes]

0x012D varint SubjectRef [UUID ref, 16 bytes]

0x013D varint PredicateRef [UUID ref, 16 bytes]

0x014D -- ObjectRef / ObjectValue [conditional, type-prefixed]

0x-- -- TemporalAnchor [struct: T_start int64, T_end int64/null,

T_granularity varint, T_confidence float32]

0x-- -- ConfidenceScore [float32, 4 bytes]

0x-- 32 ProvenanceRef [SHA-256, 32 bytes]

0x-- 32 SemanticHash [SHA-3/256, 32 bytes]

0x-- 1 ModalitySignature [uint8 bitmask]

0x-- 16388 EmbeddingVector ref [4-byte dim prefix + 4096*4 float32 OR

4-byte sidecar CID length + CID bytes]

0x-- -- RelationSet [4-byte count + count * RelationEdge records]

0x-- -- SafetyFlags [uint32 LE, 4 bytes]

0x-- -- LinguisticMetadata [struct: language string + 2 enums]

0x-- -- AccessControlList [4-byte count + count * ACLEntry records]

[Optional fields follow if presence bits set]

-- INDEX BLOCK (absent in this example) --

-- TRAILER BLOCK (64 bytes, at end of file) --

EOF-64 4 CRC-32C checksum [CRC-32C of bytes 0x0000 through EOF-65]

EOF-60 1 Checksum type 00 (CRC32C)

EOF-59 32 Extended hash 00...00 (not enabled)

EOF-27 2 ECDSA sig length 00 00 (unsigned)

EOF-25 4 EOF marker 51 42 45 4F ("QBEO")

EOF-21 21 Pad 00...00

NOTE: Fields marked "--" for offset have variable starting positions dependent on preceding variable-length fields. Implementors must use the field presence bitmask and length-prefixed encoding rules specified in Section 13.3 to parse variable-position fields.

Appendix D: Safety Flag Bitmask Reference

Bit Position	Hex Mask	Flag Name	Description	Primary Enforcement Actions
Bit 0	0x00000001	SAFE	No special restrictions; default state	ACL enforcement only
Bit 1	0x00000002	SENSITIVE	Elevated access scrutiny warranted	Authentication required; all access logged
Bit 2	0x00000004	RESTRICTED	Role-based access limitation	RBAC enforced; all queries logged
Bit 3	0x00000008	CONFIDENTIAL	Commercial/legally protected content	AES-256-GCM at rest and in transit; HSM key management
Bit 4	0x00000010	HAZARDOUS	Could facilitate	Dual-approval for

Bit Position	Hex Mask	Flag Name	Description	Primary Enforcement Actions
			harm if improperly accessed	retrieval/modification; written justification required
Bit 5	0x00000020	LEGALLY_SENSITIVE	Subject to legal privilege or hold	Legal review before export; litigation hold prevents archival
Bit 6	0x00000040	PERSONALLY_IDENTIFIABLE	Contains or derived from PII	Consent verification; purpose limitation; GDPR Article 25 controls
Bit 7	0x00000080	REGULATED_HIPAA	Protected Health Information (PHI)	HIPAA minimum-necessary; BAA verification; 6-year retention
Bit 8	0x00000100	REGULATED_GDPR	Personal data under GDPR jurisdiction	Lawful basis verification; data subject rights; cross-border controls
Bit 9	0x00000200	REGULATED_CCPA	Personal information under CCPA/CPRA	Opt-out of sale; right-to-know; sensitive PI restrictions
Bit 10	0x00000400	REGULATED_FINRA	Financial records subject to FINRA	7-year retention; WORM storage; supervisory review
Bit 11	0x00000800	EXPORT_CONTROLLED	Subject to EAR/ITAR export controls	Nationality-based access; geographic restriction; deemed export controls
Bit 12	0x00001000	AI_GENERATED	Primarily from AI model output	AI-origin disclosure; ConfidenceScore cap 0.85; extra hallucination scrutiny
Bit 13	0x00002000	UNDER_REVIEW	Flagged for human review	Excluded from default queries; REVIEWER role required; version changes blocked
Bit 14	0x00004000	SYNTHETIC	From Scenario Graph / simulation, not real observation	Labeled synthetic in outputs; excluded from training exports; no Provenance attestation
Bits 15–31	0xFFFF8000	RESERVED	Reserved for future Safety Substrate expansion	Must be zero; non-zero bits cause Fact Unit rejection at all

Bit Position	Hex Mask	Flag Name	Description	Primary Enforcement Actions
				Safety Substrate gates

Appendix E: Supported Language Table

See Section 16.2 for the full supported language table with BCP-47 codes, scripts, MRE support levels, and special handling notes. The following summarizes the two support levels:

Support Level	Definition	Number of Languages in v1.0
Full	All six MRE regeneration pipeline stages are fully supported; all NarrativeStyle options available; hallucination prevention enforced; simultaneous multi-language regeneration supported; accuracy benchmarked against native-speaker quality standards	44 languages
Partial	All six MRE stages are functionally operational but accuracy benchmarks are ongoing; some NarrativeStyle options may have reduced quality; hallucination prevention fully enforced; additional linguistic resources may be required for morphologically complex languages	9 languages (Welsh, Irish, Basque, Maltese, Kazakh, Azerbaijani, Georgian, Amharic, Farsi)

Appendix F: Microservices Reference Table

Service Name	Internal Port	Protocol	Key Dependencies	Scaling Parameters	Health Check
qb-query-gateway	8080 (REST), 9090 (gRPC)	HTTP/2 + gRPC	qb-ncl-engine, qb-fact-graph-store, qb-mre-engine, qb-scenario-graph-service, qb-safety-substrate	HPA: min 3, max 50; scale on request rate and p99 latency	GET /health → 200; gRPC health.Check → SERVING
qb-ncl-engine	9091 (gRPC)	gRPC	qb-vector-index, qb-fact-graph-store, qb-safety-substrate, qb-	HPA: min 3, max 30; session affinity; scale on query queue	gRPC health.Check → SERVING; SWS write latency <

Service Name	Internal Port	Protocol	Key Dependencies	Scaling Parameters	Health Check
			provenance-ledger	depth	10ms
qb-safety-substrate	9092 (gRPC)	gRPC (mTLS required)	Safety Audit Log (append-only), qb-ontology-mesh-service	HPA: min 5, max 100; no single point of failure; scale on enforcement call rate	gRPC health.Check → SERVING; Safety Audit Log write latency < 5ms
qb-ingestion-orchestrator	9093 (gRPC)	gRPC + Kafka	qb-stl-broker, qb-safety-substrate, all qb-ingestion-worker-{modality}	HPA: min 2, max 20; scale on job queue depth	GET /health → 200; queue depth < 10,000 healthy
qb-fact-graph-store	9094 (gRPC), 9095 (Raft)	gRPC + Raft	qb-safety-substrate, qb-provenance-ledger	Stateful; 3 replicas per shard; 4096 virtual shards; shard split at 500M FU	gRPC health.Check → SERVING; Raft leader heartbeat < 500ms
qb-vector-index	9096 (gRPC)	gRPC	qb-fact-graph-store (EmbeddingVector sync)	Stateful; 3 replicas per shard; split at 100M FU per shard	gRPC health.Check → SERVING; ANN query p99 < 50ms
qb-provenance-ledger	9097 (gRPC)	gRPC	External notarization service (RFC 3161); qb-safety-substrate	Stateful append-only primary; up to 10 read replicas; scale read replicas on lineage query rate	gRPC health.Check → SERVING; chain tip hash freshness < 60s
qb-mre-engine	9098 (gRPC)	gRPC	qb-fact-graph-store, qb-ncl-engine, qb-safety-substrate, qb-ontology-mesh-service	HPA: min 3, max 30; scale on regeneration request rate and token output rate	gRPC health.Check → SERVING; groundedness check latency < 20ms
qb-cdre-engine	9099 (gRPC)	gRPC + Kafka (change events)	qb-fact-graph-store, qb-ontology-mesh-service, qb-safety-substrate	HPA: min 2, max 15; scale on rule evaluation throughput (Fact Unit insertions/s)	gRPC health.Check → SERVING; rule evaluation lag < 5s
qb-aem-service	9100 (gRPC)	gRPC	qb-ncl-engine, qb-fact-graph-store, qb-pie-engine, qb-scenario-graph-service, qb-safety-substrate	HPA: min 2, max 20; GPU-attached nodes for AEM-2 and AEM-3 neural methods	gRPC health.Check → SERVING; AEM job queue depth < 1000

Service Name	Internal Port	Protocol	Key Dependencies	Scaling Parameters	Health Check
qb-pie-engine	9101 (gRPC)	gRPC + Kafka	qb-ncl-engine (TMM), qb-fact-graph-store, qb-scenario-graph-service, qb-safety-substrate	HPA: min 2, max 10; scale on prediction job queue depth	gRPC health.Check → SERVING; calibration check passes within 24h window
qb-ontology-mesh-service	9102 (gRPC)	gRPC	None (foundational service)	Single primary for writes; up to 20 read replicas; aggressively cached by all consumers	gRPC health.Check → SERVING; schema version consistency across replicas
qb-entity-registry	9103 (gRPC)	gRPC	qb-safety-substrate, external entity linkers (Wikidata, GeoNames, etc.)	HPA: min 3, max 20; sticky routing by entity hash for cache efficiency	gRPC health.Check → SERVING; entity lookup p99 < 5ms
qb-stl-broker	9104 (STL native), 9092 (Kafka-compatible)	QB-Stream Protocol; Kafka-compatible protocol	qb-safety-substrate (Federation Gate)	Stateful; Kafka-style partition leadership; 3 replicas per partition; scale partitions with throughput	Partition leader election healthy; consumer group lag < 10,000 messages
qb-scenario-graph-service	9105 (gRPC)	gRPC	qb-fact-graph-store, qb-cdre-engine, qb-safety-substrate	HPA: min 2, max 15; scale on active Scenario Graph count and cross-branch query rate	gRPC health.Check → SERVING; KSS consistency check latency < 100ms
qb-admin-api	8443 (HTTPS only)	REST/HTTPS with mTLS	All services (administrative control plane)	min 2, max 5; low-traffic; no HPA scaling needed	GET /health → 200; mTLS certificate validity verified

Appendix G: Acronym and Abbreviation Index

Acronym	Full Expansion	First Defined In
ACL	Access Control List	Chapter 3.1
AEM	AGI Extension Module	Chapter 2.1
AGI	Artificial General Intelligence	Chapter 1.1
ANN	Approximate Nearest Neighbor	Chapter 14.3
AN	Attractor Network	Chapter 7.2.4
ARE	Associative Recall Engine	Chapter 7.2.3

Acronym	Full Expansion	First Defined In
ASR	Automatic Speech Recognition	Chapter 4.3.4
AST	Abstract Syntax Tree	Chapter 4.3.8
BNF	Backus-Naur Form	Chapter 17.3.1
CCW	Cognitive Context Window	Chapter 7.2.6
CDRE	Cross-Domain Rule Engine	Chapter 2.1
CID	Content Identifier (IPFS-style)	Chapter 5.2
COA	Central Ontology Authority	Chapter 6.1
CRDT	Conflict-Free Replicated Data Type	Chapter 14.5.4
CWA	Closed World Assumption	Chapter 6.1
DLQ	Dead-Letter Queue	Chapter 4.4.4
DSAR	Data Subject Access Request	Chapter 8.2
EWC	Elastic Weight Consolidation	Chapter 7.1
FB	Fact Bundle	Chapter 2.1
FCI	Fast Causal Inference (algorithm)	Chapter 15.4.2
FG	Fact Graph	Chapter 2.1
FU-verse	Fact Universe	Chapter 53
WASM	World Aware System Model	Chapter 53
WMD	World Model Domain	Chapter 53
CML	Conceptual Memory Layer	Chapter 54
SAP	Sensory Abstraction Pipeline	Chapter 55
FF	Fact Field	Chapter 2.1
FL	Fact Lineage	Chapter 2.1
FS	Fact Stream	Chapter 2.1
FU	Fact Unit	Chapter 2.1
FUID	Fact Unit Identifier	Chapter 2.1
GDPR	General Data Protection Regulation	Chapter 5.4.2
GFS	Grandfather-Father-Son (backup rotation)	Chapter 14.6
HIPAA	Health Insurance Portability and Accountability Act	Chapter 8.2
HNSW	Hierarchical Navigable Small World (graph index)	Chapter 14.3
HSM	Hardware Security Module	Chapter 8.2
HTM	Hierarchical Temporal Memory	Chapter 7.1
IL	Ingestion Layer	Chapter 1.5
IP	Ingestion Pipeline	Chapter 2.1
ISR	Intermediate Semantic Representation	Chapter 4.1
ITAR	International Traffic in Arms Regulations	Chapter 8.2
KSS	Knowledge State Snapshot	Chapter 11.1
LLM	Large Language Model	Chapter 1.1
MCMC	Markov Chain Monte Carlo	Chapter 10.3.1
MRE	Multilingual Regeneration Engine	Chapter 2.1
NCL	Neuromorphic Cognitive Layer	Chapter 2.1
NER	Named Entity Recognition	Chapter 4.5.1
NG	Narrative Graph	Chapter 2.1
NLP	Natural Language Processing	Chapter 4.3.1
NOTEARS	Non-combinatorial Optimization via Trace Exponential and Augmented lagRangian for Structure learning	Chapter 15.4.2
OWA	Open World Assumption	Chapter 6.1

Acronym	Full Expansion	First Defined In
PIE	Predictive Inference Engine	Chapter 2.1
PITR	Point-in-Time Recovery	Chapter 14.6
PII	Personally Identifiable Information	Chapter 8.2
PL	Provenance Ledger	Chapter 2.1
QB	QueryBook	Chapter 2.1
QB-ER	QueryBook Entity Registry	Chapter 3.1
QBF	QueryBook Format	Chapter 2.1
RAG	Retrieval-Augmented Generation	Chapter 1.1
RBAC	Role-Based Access Control	Chapter 8.2
RPO	Recovery Point Objective	Chapter 14.6
RTO	Recovery Time Objective	Chapter 14.6
SCP	Schema Change Proposal	Chapter 6.4
SDE	Sparse Distributed Encoder	Chapter 7.2.1
SDR	Sparse Distributed Representation	Chapter 7.1
SG	Scenario Graph	Chapter 2.1
SH	Semantic Hash	Chapter 2.1
SLA	Service Level Agreement	Chapter 4.4.5
SLO	Service Level Objective	Chapter 19.3
SRL	Semantic Role Labeling	Chapter 4.3.1
SS	Safety Substrate	Chapter 2.1
STDP	Spike-Timing-Dependent Plasticity	Chapter 7.1
STL	Semantic Transport Layer	Chapter 2.1
SWS	Synaptic Weight Store	Chapter 7.2.5
TA	Temporal Anchor	Chapter 2.1
TMM	Temporal Memory Module	Chapter 7.2.2
TTL	Time-to-Live	Chapter 3.1
UUID	Universally Unique Identifier	Chapter 3.1
VLM	Vision-Language Model	Chapter 4.3.3
WAL	Write-Ahead Log	Chapter 14.6
WORM	Write Once Read Many	Chapter 8.2

Chapter 53: World Aware System Model (WASM)

53.1 Overview and PurposeThe World Aware System Model (WASM) is QueryBook's architectural framework for maintaining a dynamic, evolving, conceptual model of physical and social reality. The WASM does not store events, episodes, or raw sensory experiences. It stores abstractions: compressed, revised, domain-local conceptual models of how the world works. The WASM gives QueryBook a conceptual brain — a reasoning substrate that understands gravity, hardness, social roles, affordances, and causal patterns as generalized structures, not as memories of specific occurrences.The WASM is composed of three tightly integrated but architecturally distinct segments: (1) the World Model Domain — the active theory-of-the-world engine; (2) the Conceptual Memory Layer — the long-term conceptual store; and (3) the Sensory Abstraction Pipeline — the ingestion and abstraction engine that converts raw sensory experience into structured concepts. These three segments operate in concert: the Sensory Abstraction Pipeline transforms raw inputs into abstracted concepts, the Conceptual Memory Layer stores and evolves those concepts over time, and the World Model Domain applies them in real-time reasoning.53.2 Design PrinciplesThe World Model Domain is governed by four inviolable design principles:● Retain Structures, Not Episodes: The World Model Domain stores generalized structural knowledge — gravity, hardness, social roles, affordance patterns — not the specific sensory episodes from which those structures were derived. After integration, raw episodes are discarded. Only the abstracted concept survives.● Compression: Many distinct experiences collapse into a single robust concept. One hundred encounters with chairs produce a single, well-calibrated Chair concept node — not one hundred chair-episode records.● Evolution: Conceptual models are revised, not merely appended. When new evidence contradicts an existing model, the model is updated with adjusted confidence levels, generalized or specialized rule conditions, and merged or split schema nodes. The system does not accumulate contradictions — it resolves them.● Domain-Local: The World Model Domain is architecturally segregated from the Semantic Q&A Domain and the Ingestion Domain. It operates as a distinct internal reasoning layer that informs but does not merge with the Fact Graph or the Ontology Mesh. Cross-domain communication occurs exclusively through typed interfaces defined in the Semantic Transport Layer.53.3 World Model Domain — Six Structural LayersThe World Model Domain is organized into six hierarchical layers, each responsible for a distinct category of world knowledge. All layers are implemented as hypergraph substructures within the Fact Universe, enabling cross-layer relations to be expressed as typed hyperedges rather than simple binary edges.53.3.1 Object LayerThe Object Layer provides the entity substrate of the World Model Domain. It models: physical objects (chair, door, vehicle, room, tool), agents (person), and environments (indoor space, outdoor space, urban area, workplace, public venue). Each object is represented as an Object Model node with the following fields: ObjectID (UUID, globally unique within the World Model Domain), Type (enumerated: PhysicalObject, Agent, Environment, AbstractEntity), Attributes (key-value pairs encoding size, shape, material, hardness, mobility, mass, color, temperature tolerance, and domain-specific properties), Affordances (ordered list of action identifiers drawn from the Affordance Layer representing what actions are possible on or with this object), and Relations (typed links to other Object Layer nodes representing spatial, causal, and functional relationships). The Object Layer is the primary referent for all physical grounding operations in the Semantic Q&A Domain, enabling QueryBook to interpret spatially and physically situated references in reader queries.53.3.2 Relation LayerThe Relation Layer defines the typed relational structures that connect Object Layer nodes. Relations are partitioned into three canonical categories: Spatial Relations (above, below, inside, outside, near, attached-to, adjacent-to, overlapping, occluding), Temporal Relations (before, after, during, overlaps-with, meets, starts-with, ends-with — following Allen's Interval Algebra), and Causal Relations (causes, enables, prevents, inhibits, triggers, sustains). Every Relation Layer edge carries: RelationID, RelationType (from the three categories above), SubjectObjectID, TargetObjectID, ConfidenceScore,

Confidential — Proprietary and Patent-Sensitive — QueryBook Core Technology Group

© 2026 QueryBook Core Technology Group. All Rights Reserved.

Chapter 39: QueryBook Application Library and Generator (QAL/QAG) — Narrow-Scope App Instantiation from Fact Universes

39.1 Governing Philosophy — The Library Before the Generator

The instinct to build an app generator first is architecturally wrong. A generator with nothing to generate from produces blank-slate apps — as limited and generic as every no-code tool that already exists. The correct sequence is: build the library first. The generator is simply the mechanism by which the library delivers a personalised instance to a specific user.

The QueryBook Application Library (QAL) is a curated, growing collection of narrow, task-focused application templates — each one covering exactly one problem in exactly one domain, each one built from QueryBook Fact Units extracted from authoritative content in that domain, each one following a standard anatomy that makes every app immediately usable and integrable with every other app in the library. The QueryBook Application Generator (QAG) is the delivery engine — the system that takes a library app, asks the user for their specific parameters, and produces a personalised running instance of that app in seconds. Together they form a self-reinforcing system: every new app added to the library makes the generator more valuable; every user who requests an app and customises it adds a new instance that informs the next version of the library template.

39.2 The Narrow Scope Rule — The Single Most Important Design Decision

Every QueryBook app must comply with the Narrow Scope Rule: One app. One task type. One domain. One set of data points. No QueryBook app crosses task types. No QueryBook app attempts to be a

platform. Complex workflows are accomplished by combining multiple narrow apps — not by building one large app. There are exactly six permitted task types. Every app in the library is classified as exactly one of them.

Task Type	What It Does	Output
CALCULATE	Takes user's specific inputs, applies domain Fact Unit formulas, returns a precise quantitative result	A number, quantity, cost, or measurement
TRACK	Logs entries over time against a defined schema, maintains a running history	A timestamped log with totals, trends, and history
SCHEDULE	Manages time-based tasks, intervals, deadlines, and sequences	A calendar of tasks with due dates, alerts, and completion logging
ORGANISE	Categorises, retrieves, and relates items within a defined taxonomy	A structured, searchable, categorised collection
GUIDE	Delivers a step-by-step procedural execution plan adapted to the user's situation	An ordered checklist with context-sensitive instructions
COMPARE	Evaluates multiple options against defined criteria derived from domain Fact Units	A ranked or scored comparison matrix

Why this rule is the right design: a paint calculator that also tracks purchases, schedules rooms, and organises suppliers is not a better app — it is a worse one. It is harder to build, harder to use, harder to maintain, and impossible to integrate cleanly with other apps. Four narrow apps — a CALCULATE app, a TRACK app, a SCHEDULE app, and an ORGANISE app — each doing one thing perfectly and sharing data through standard integration hooks, produce a superior result in every dimension.

39.3 App Anatomy — The Six Universal Components

Every app in the QAL, regardless of domain or task type, is built from exactly six components in the same structural arrangement. This standardisation is what makes integration between apps possible and what makes every new app immediately familiar to every user.

Component 1 — The Parameter Panel: the user's specific inputs — the data points that distinguish their instance from the generic template. For a paint calculator: room dimensions, surface type, current colour, target colour. For a crop planting scheduler: crop species, planting zone, first frost date, bed

dimensions. The Parameter Panel is the only part of the app the user fills in. Everything else is derived from it plus the Fact Universe.

Component 2 — The Fact Engine: the domain Fact Units — extracted from the relevant QueryBook publication or from the World Brain's concept nodes for that domain — that provide the logic the app operates on. The Fact Engine is invisible to the user. They never see it. But it is the reason the app's calculations, schedules, and guidance are authoritative rather than generic. A paint calculator whose Fact Engine is drawn from a specific paint manufacturer's technical guide produces different, more precise results than one drawn from a general home improvement book. The source is always disclosed to the user.

Component 3 — The Reasoning Layer: the PIE traversal that connects the user's Parameter Panel inputs to the Fact Engine's domain knowledge and produces results. For CALCULATE apps: the formula traversal. For GUIDE apps: the conditional procedural sequence selection. For COMPARE apps: the multi-criteria evaluation matrix. The Reasoning Layer is QueryBook's existing inference infrastructure — no new computation engine is required.

Component 4 — The Output Display: the result, presented in the format appropriate to the task type: a number and material list for CALCULATE; a timestamped log table for TRACK; a calendar view for SCHEDULE; a browsable collection for ORGANISE; a step checklist for GUIDE; a scored matrix for COMPARE. Every Output Display supports three standard export formats: PDF (printable), CSV (data transfer), and Share Link (send to another user or app).

Component 5 — The Log: every app maintains a persistent log of all inputs, calculations, and outputs — timestamped and user-attributed. The Log is the user's history of their work in this app. It is also the data source for cross-app integration — other apps can read this app's Log (with user permission) to pre-populate their own Parameter Panels.

Component 6 — The Integration Registry: a declared list of all other QAL apps this app can share data with, in both directions. Declared at build time, verified at instantiation. The Integration Registry is what creates the lattice — every app knows its neighbours, and the lattice grows as new apps declare new connections.

39.4 Standard Features — Present in Every App Without Exception

Six features are present in every QAL app regardless of domain, task type, or complexity tier.

Feature	Description
Natural language input	Every Parameter Panel can be filled by typing or speaking in plain language — the SPE converts it to the app's structured parameter schema
Inline Fact citation	Every result the app produces cites the specific Fact Unit that generated it — the user can tap any result to see exactly which part of which book or reference produced it
Suggested next app	On completing a task, every app surfaces the most relevant next app from the library — connected to the ARE recommendation engine
Export in three formats	PDF, CSV, Share Link — always present, always one tap away
QueryBook conversation bridge	Every app has a direct link back to the QueryBook conversation for the relevant publication — the user can ask a question about any result and receive a full Fact Unit-grounded answer
Cross-app data sharing	Every app's Log is available to any other app in the user's personal library (with explicit permission per app) via the Integration Registry

39.5 The Three-Step User Flow — Ask, Detail, Instantiate

This is the complete user experience from need to working app. It applies identically to a seven-year-old building a lemonade stand counter and a professional engineer building a structural load calculator.

Step 1 — ASK

The user asks for the app in natural language — through their QueryBook conversation interface, through a library search, or through the ARE's proactive recommendation. Three entry paths exist: (1) Proactive recommendation — the ARE detects an app-relevant query pattern and surfaces: 'It looks like you could use a [app name]. Want me to set it up?' (2) Library search — user browses or searches the QAL directly: 'Show me apps for home painting.' (3) Direct request — user states the need: 'I need an app to track my Corvette restoration parts.' In all three cases, the QAG presents the matching library app with a one-paragraph description of what it does, what parameters it needs, and what it produces.

Step 2 — DETAIL

The QAG presents the Parameter Panel for the selected app and collects the user's specific situation data. This step is conversational — the QAG asks for each parameter in natural language, one at a time if the user prefers, or all at once if the user provides them: 'To set up your Interior Paint Calculator, I need three things: (1) Tell me your rooms — name and dimensions for each one. (2) What are the surfaces like — smooth walls, textured, or a mix? (3) Are you changing colour, and if so, from what to what?' The user responds in plain language. The SPE converts their response to the app's structured parameter schema. The QAG confirms the parameters back to the user in plain language before proceeding. For domain-specific apps tied to a QueryBook publication, the QAG pre-populates any parameters already known from the user's prior conversation — room dimensions discussed in a query, vehicle make and model mentioned in a question, crop type queried about — so the user only confirms rather than re-enters.

Step 3 — INSTANTIATE

The QAG injects the user's parameters into the selected library template, activates the Fact Engine with the relevant domain Fact Units, runs the initial Reasoning Layer traversal, and delivers the working app — fully populated with the user's specific data, with results already calculated, with the Log initialised, and with the Integration Registry ready. From the user's perspective: they answered three questions and a complete, working, personalised app appeared. The entire Step 3 process is invisible. There is no 'building' screen. There is no delay beyond the traversal time. The app is now a permanent fixture in the user's QueryBook personal library — available on any device, persistent across sessions, growing its Log with every use.

39.6 The App Recommendation Engine (ARE) — Connected to Suggested Queries

QueryBook already specifies a Suggested Queries mechanism that analyses the user's query pattern and surfaces follow-on questions the user is likely to want to ask. The App Recommendation Engine is a direct extension of that same infrastructure — it analyses the same query patterns but matches them to library apps rather than to follow-on questions.

Merge note — acronym disambiguation: ARE is also used in this Bible for the Associative Recall Engine (§7.2.3) and the Adaptive Response Engine (Chapter 52), both unrelated subsystems. Where cross-chapter clarity matters, this app-library component may be cited as QAL-ARE; the short form ARE is retained here unchanged per this section's original text.

The ARE monitors every reader's active query session for five signal types.

Signal Type	Example	App Pattern Match
Quantitative signals	Queries about quantities, measurements, costs	CALCULATE app family
Procedural signals	Queries about sequences, what-to-do-next, steps	GUIDE app family
Temporal signals	Queries about timing, intervals, schedules, deadlines	SCHEDULE app family
Collection signals	Queries about lists, inventories, what do I need	ORGANISE app family
Progress signals	Queries about tracking, completion, how-far-along	TRACK app family

When three or more signals of the same type are detected within a session, the ARE fires — identifying the most relevant library app for that signal cluster and surfacing a recommendation at the natural pause point in the conversation, after a query answer is delivered, never interrupting: 'Based on your questions about your Mustang restoration, I think the Classic Vehicle Parts Tracker could be useful. It tracks every part you need, its source, cost, and installation status — all built from the specifications in this guide. Want me to set it up?' The ARE recommendation is not a sales prompt. It is a logical next step surfaced at the moment of maximum relevance. The ARE recommendation integrates directly with the existing Suggested Queries display in the Reader Portal — apps appear as a distinct card alongside suggested follow-on queries, clearly labelled as an app recommendation rather than a question. The user can dismiss it, save it for later, or instantiate it immediately.

39.7 The Lattice Growth Model — How the Library Becomes Self-Reinforcing

The QAL is designed to grow in a specific pattern — not as a flat list of unrelated apps, but as a connected lattice where every new app strengthens the value of every existing app it shares entity types with. The lattice forms through shared entity types: when two apps share an entity type — Vehicle, Room, Plant Species, Financial Transaction, Person — they can share data through their Integration Registries. The more apps that share an entity type, the more valuable that entity type becomes as a data node in the user's personal app ecosystem.

Example lattice cluster — the Vehicle entity type: Vehicle Maintenance Scheduler (SCHEDULE) shares Vehicle entity with Classic Vehicle Parts Tracker (ORGANISE), which shares Vehicle entity and Parts entity with Restoration Budget Calculator (CALCULATE), which shares Vehicle entity, Parts entity, and Cost

entity with Restoration Progress Logger (TRACK), which shares Vehicle entity and Task entity with Pre-Purchase Inspection Checklist (GUIDE), which shares Vehicle entity with Fuel Economy Tracker (TRACK). A user who instantiates the Restoration Budget Calculator automatically has access to the parts data in their Classic Vehicle Parts Tracker and the task data in their Restoration Progress Logger — because all three share the Vehicle entity and are registered in each other's Integration Registries. The user enters their vehicle once. Every app knows it.

Three lattice growth mechanisms: (1) Author Foundation apps — when a new QueryBook publication is ingested with App Foundation Certification, its domain Fact Units are automatically assessed for new app templates. New apps derived from new publications extend the lattice into new domains. (2) User-requested custom apps — when a user requests a customisation that does not exist in the library, the QAG builds it, the QueryBook team reviews it, and if it meets the Narrow Scope Rule, it is added to the library as a new template available to all users. The originating user is credited. (3) Cross-domain connections — when a new app shares an entity type with apps in a different domain family, a cross-domain lattice edge is created — extending integration pathways across domains. A nutrition tracker connecting to a grocery budget calculator is a cross-domain lattice edge between the Health and Home Economics families. The lattice compounds: a library of 100 apps with an average of 5 integration connections each has 500 data-sharing pathways. A library of 1,000 apps has the potential for 5,000 pathways. The value of the library grows faster than the number of apps in it.

39.8 The Initial Domain Taxonomy — 12 Families, 72 Seed Apps

The QAL launches with 72 seed apps across 12 domain families — enough to cover the most common life and industry domains immediately, and enough to demonstrate the lattice structure from day one.

Family 1 — Home and Property

App Name	Task Type	What It Does
Interior Paint Calculator	CALCULATE	Quantities and materials for any room configuration
Flooring Material Estimator	CALCULATE	Square footage, waste factor, and cost by floor type
Home Energy Usage Tracker	TRACK	Monthly utility consumption by category and trend
Appliance Maintenance Scheduler	SCHEDULE	Service intervals and replacement planning per appliance

App Name	Task Type	What It Does
Home Improvement Project Planner	GUIDE	Step sequence for any standard home project
Room Dimension Organiser	ORGANISE	All room measurements, surfaces, and specifications in one place

Family 2 — Kitchen, Food, and Nutrition

App Name	Task Type	What It Does
Grocery Budget Tracker	TRACK	Spending by category with weekly and monthly totals
Recipe Scaling Calculator	CALCULATE	Ingredient quantities for any serving count
Pantry Inventory Manager	ORGANISE	What you have, what you need, expiry dates
Meal Nutrition Analyser	CALCULATE	Nutritional profile of any meal from its ingredients
Weekly Meal Planner	SCHEDULE	Meals by day with automatic shopping list generation
Food Cost Calculator	CALCULATE	Cost per serving for any home-cooked meal

Family 3 — Health and Wellness

App Name	Task Type	What It Does
Medication Schedule Manager	SCHEDULE	Doses, timings, refill reminders, and interaction flags
Symptom and Wellness Log	TRACK	Daily health entries with pattern detection prompts
Recovery Timeline Calculator	CALCULATE	Expected recovery milestones for common procedures
Sleep Quality Tracker	TRACK	Nightly sleep data with trend analysis
Hydration Target Calculator	CALCULATE	Daily water targets by body weight and activity level
Vital Signs Log	TRACK	Blood pressure, heart rate, temperature over time

Family 4 — Fitness and Activity

App Name	Task Type	What It Does
Workout Session Logger	TRACK	Sets, reps, loads, and perceived

App Name	Task Type	What It Does
		effort by session
Training Schedule Planner	SCHEDULE	Weekly training blocks with rest day management
Personal Record Tracker	TRACK	PRs by lift or activity with trend charting
Caloric Burn Estimator	CALCULATE	Energy expenditure by activity type, duration, and body weight
Running Pace Calculator	CALCULATE	Target paces for any race distance and goal time
Exercise Progress Comparator	COMPARE	Performance across time periods or training approaches

Family 5 — Finance and Personal Economics

App Name	Task Type	What It Does
Lemonade Stand POS	TRACK	Sales counter and revenue total for any simple micro-business
Freelance Income Tracker	TRACK	Projects, rates, invoiced amounts, and payment status
Monthly Budget Allocator	CALCULATE	Category allocation from income with surplus and deficit
Debt Payoff Calculator	CALCULATE	Payoff timeline and interest cost under different payment strategies
Savings Goal Tracker	TRACK	Progress toward a savings target with projected completion date
Expense Category Analyser	COMPARE	Spending in any category vs. prior periods or benchmarks

Family 6 — Automotive and Vehicles

App Name	Task Type	What It Does
Vehicle Maintenance Scheduler	SCHEDULE	Service intervals by mileage and time for any make and model
Classic Vehicle Parts Tracker	ORGANISE	Parts list, sourcing status, cost, and installation log
Restoration Budget Calculator	CALCULATE	Material and labour cost projection for restoration by scope
Fuel Economy Logger	TRACK	Miles per gallon by fill-up with trend and cost analysis
Pre-Purchase Inspection Checklist	GUIDE	Condition assessment checklist by vehicle category

App Name	Task Type	What It Does
Restoration Progress Logger	TRACK	Task completion log against full restoration sequence

Family 7 — Garden and Agriculture

App Name	Task Type	What It Does
Crop Planting Scheduler	SCHEDULE	Sow, transplant, and harvest dates by species and zone
Soil Amendment Calculator	CALCULATE	Amendment quantities by bed volume and target pH
Garden Bed Layout Planner	ORGANISE	Bed configurations, plant spacing, and companion planting notes
Irrigation Zone Calculator	CALCULATE	Water volume and timing by zone, crop, and soil type
Harvest Yield Logger	TRACK	Yield by crop and bed across seasons
Pest and Disease Log	TRACK	Occurrence, identification, and treatment history by plant

Family 8 — Education and Research

App Name	Task Type	What It Does
Term Paper Source Manager	ORGANISE	Sources, quotes, topics, and citation details by paper
Study Topic Tracker	TRACK	Subject areas, mastery levels, and revision history
Vocabulary Builder	TRACK	New terms, definitions, and review schedule
Reading List Manager	ORGANISE	Books by subject, status, and key notes
Research Question Log	ORGANISE	Open questions, sources consulted, and findings by topic
Assignment Deadline Scheduler	SCHEDULE	Deliverables, milestones, and submission dates by course

Family 9 — Trade and Construction

App Name	Task Type	What It Does
Concrete Volume Calculator	CALCULATE	Mix quantities by slab or form dimensions
Electrical Load Calculator	CALCULATE	Circuit sizing, load totals, and

App Name	Task Type	What It Does
		breaker requirements
Lumber and Timber Estimator	CALCULATE	Board-feet by project with waste factor and species options
Tile Layout Estimator	CALCULATE	Tile quantity, adhesive, and grout by room and joint width
HVAC Load Calculator	CALCULATE	Heating and cooling capacity by room volume and insulation
Trade Job Cost Tracker	TRACK	Labour hours, materials, and margin by job

Family 10 — Creative and Craft

App Name	Task Type	What It Does
Sewing Material Calculator	CALCULATE	Fabric yardage by pattern size and layout
Knitting and Crochet Row Tracker	TRACK	Row count, stitch count, and pattern progress
Photography Shoot Planner	GUIDE	Shot list, equipment checklist, and location notes
Pottery Glaze Calculator	CALCULATE	Glaze material quantities by batch weight and recipe
Music Practice Logger	TRACK	Pieces, techniques, and session time with progression notes
Art Supply Inventory	ORGANISE	Materials, quantities, suppliers, and restock reminders

Family 11 — Marine and Outdoor

App Name	Task Type	What It Does
Sailboat Maintenance Scheduler	SCHEDULE	Service intervals by system and season for any vessel class
Boat Restoration Parts Tracker	ORGANISE	Parts list, source, cost, and installation status by system
Camping Gear Checklist	GUIDE	Equipment list by trip type, season, and group size
Navigation Fuel Calculator	CALCULATE	Fuel consumption and range by vessel, speed, and conditions
Fishing Log	TRACK	Catches, locations, conditions, and gear by session
Hiking Route Planner	GUIDE	Trail segments, waypoints, elevation, and supply checklist

Family 12 — Science and Engineering

App Name	Task Type	What It Does
Structural Load Calculator	CALCULATE	Load distribution and member sizing for simple structures
Chemical Mix Calculator	CALCULATE	Solution concentrations and volumes by formula
Experiment Data Logger	TRACK	Variables, measurements, and observations by trial
Engineering Tolerance Comparator	COMPARE	Measured values against specification tolerances
Lab Supply Inventory	ORGANISE	Reagents, quantities, hazard class, and reorder levels
Project Milestone Scheduler	SCHEDULE	Deliverables, dependencies, and deadlines for any technical project

39.9 The Customisation Request Protocol

When a user needs an app that does not exist in the library — or needs a library app modified for a specific requirement — the Customisation Request Protocol governs the process: (1) The user describes their specific need in natural language. The QAG determines whether the need matches an existing library app (in which case it is a standard instantiation), a modification of an existing app (a customisation), or a genuinely new app (a new template request). (2) For customisations: the QAG presents the closest library app and asks which parameters or features need to change. All customisations must still comply with the Narrow Scope Rule — if the user's requirement implies crossing task types, the QAG proposes two separate apps rather than one combined one. (3) For new templates: the QAG builds a one-off instance for the user immediately — using the same Fact Engine and Reasoning Layer infrastructure, but without a library template. The instance is flagged as a custom app in the user's library. If the instance proves valuable, the QueryBook App Team reviews it for library inclusion as a new template. (4) Library promotion: custom apps that are promoted to the library retain a credit to the originating user — 'developed from a community request' — and the originating user receives a notification that their app is now available to the full QueryBook community.

39.10 Author App Foundation Integration

Every QueryBook publication that carries App Foundation Certification contributes its domain Fact Units to the QAL as a Fact Engine source. Authors benefit in four specific ways: (1) Revenue participation — every app instantiated using that publication's Fact Units as the primary Fact Engine source generates a

micro-royalty for the author — a fraction of the app instantiation fee, accumulating across every user who builds that app from that book. (2) Discovery inversion — users who find an app in the library and use it are shown the underlying publication as the source of the app's knowledge — creating a new discovery pathway from app to book, the reverse of the conventional book-to-reader path. (3) Author analytics — authors see in their Author Portal dashboard which of their Fact Units are most frequently used in app generation — a direct signal of which parts of their content readers find most practically valuable. (4) Competitive labelling — books with App Foundation Certification display a badge in the QueryBook catalog: 'This publication generates [n] apps.' This is a measurable, specific differentiator that no conventional ebook platform can offer.

App Foundation Certification tiers.

Certification	What the Author Has Tagged	What the Reader Gets
Calculator Certified	Quantitative Fact Units — formulas, coverage rates, material quantities	Calculation apps — paint calculators, material estimators, cost projectors
Project Certified	Procedural Fact Units — task sequences, timelines, milestones	Project management apps — task trackers, progress loggers, timeline managers
Specification Certified	Technical Fact Units — tolerances, standards, compliance requirements	Reference apps — in-context specification lookup, compliance checklists
Procurement Certified	Supplier and product Fact Units — parts lists, vendor recommendations, sourcing guidance	Procurement apps — shopping lists, supplier databases, order trackers
Full Foundation Certified	All four types	The complete professional application — project, specification, procurement, and budget integrated

39.11 NLPL-Driven On-Demand App Composition

This section connects two systems already specified independently: NLPL (§48.4), and §39.7's second lattice growth mechanism — “user-requested custom apps,” where the QAG builds an app that doesn't yet exist in the library. It closes two gaps left open where those two sections met: how the request is actually phrased, and what happens to the requesting user's own app while broader library review is pending.

39.11.1 The Request

An instruction like “Generate an application that helps me manage my song collection” is an NLPL statement, processed through the same five-stage pipeline already specified in §48.4.1. Intent Decomposition (Stage 1) classifies this as an App Composition intent rather than a query or a WATCH program request, and routes it to QAG instead of the QBPL emitter — the same routing principle already established for other NLPL intent categories, applied to one more target.

39.11.2 Composition, Not Code Generation

BOUNDED COMPOSITION, NOT ARBITRARY CODE SYNTHESISQAG does not write source code in an arbitrary programming language. It composes a new app instance from the same fixed standard anatomy every QAL app already has (§39.4: Parameter Panel, Fact citation, suggested-next-app, three-format export, conversation bridge, cross-app sharing) applied to a new domain schema — for a song collection, entity fields such as Title, Artist, Album, and Genre. This is §39.12's own Patent Significance claim (2) in this chapter, restated as the actual mechanism: Fact Unit injection into structural templates “without human programming,” and, per this section, without machine-generated arbitrary-language programming either. The Narrow Scope Rule (§39.2) still applies in full: the composed app gets one task type, one domain, one set of data points — a song collection manager does not also become a music streaming client or a lyrics database.

Where the domain has no existing book-derived Fact Units to draw from — a personal song collection is not extracted from a publication — the app's entity data is composed as reader-generated Fact Units, the same category already established for Personal Notes (§63.1) and distinct from a book's own author-sourced Fact Graph, never conflated with it.

39.11.3 Immediate Personal Use

§39.7's mechanism (2) specifies that the QueryBook team reviews a user-requested app before it is added to the library as a template available to all users. That review governs the app's status as a shared library template only. The requesting user's own personal instance is usable immediately on composition — the user is not waiting on a review cycle to use the application they asked for; they are only waiting on one if and when their app is being considered for promotion into the shared QAL.

39.11.4 Modification

A further NLPL instruction directed at an existing personal app instance — “Also track which songs I've marked as favorites” — modifies that instance's Parameter Panel schema and entity fields rather than composing a new app from scratch. Each modification is a new version in the app's Fact Unit

VersionChain (Chapter 5), the same versioning discipline already applied to WATCH programs (§49.5.1) and Personal Notes (§63.3) — prior versions remain in the Provenance Ledger, not overwritten.

39.12 Patent Significance

The QueryBook Application Library and Generator constitutes novel subject matter across five independent dimensions: (1) The Narrow Scope Rule as an architectural principle — the specific design decision to limit every application to one task type and one domain, combined with the lattice integration model that achieves complex workflows through app composition rather than app expansion, is novel as a programmatic constraint on AI-generated application architecture. (2) Fact Unit injection as application logic generation — the specific mechanism by which domain Fact Units extracted from published content are injected into structural application templates to produce domain-specific application logic without human programming is novel and directly patentable as a method claim. (3) The App Recommendation Engine as an extension of query pattern analysis — the specific architecture of extending a query-pattern-based suggestion engine to surface relevant application recommendations from a structured library, at the moment of maximum contextual relevance within a content query session, is novel as a method for context-aware application discovery. (4) The Lattice Growth Model — the specific mechanism by which shared entity types across application templates create automatic integration pathways, producing a self-reinforcing application ecosystem whose value grows super-linearly with the number of applications in the library, is novel as an application library architecture. (5) Author royalty participation in application instantiation — the specific commercial model in which the author of a publication receives a royalty for each application generated from their publication's Fact Units is novel as a publishing revenue mechanism and directly patentable as a business method claim.

APPENDIX C — QueryBook Native Hybrid AI vs. LLM Operation: Infrastructure Comparison Specification

Appendix C: QueryBook Native Hybrid AI vs. LLM Operation — Infrastructure Comparison Specification

C.1 Purpose and Methodology

This appendix provides a full, quantified infrastructure comparison between a QueryBook Native Hybrid AI deployment and a traditional Large Language Model (LLM) inference operation, at a common baseline of **1,000,000 queries per day**. The purpose is to establish the hardware, power, water, storage, and network footprint of each system for engineering planning, procurement, investor briefing, and data center negotiation purposes.

Architecture Definition: In this appendix, the QueryBook Native architecture refers to the core QueryBook knowledge architecture — the integrated combination of the Ontology Mesh, Fact Graph, Semantic Parsing Engine, and Multilingual Regeneration Engine — operating as a deterministic, provenance-grounded knowledge retrieval and response system, entirely native to QueryBook with no dependency on any external or independently-operable technology. The QueryBook Native Hybrid AI is the full QueryBook platform as specified in Chapters 1 through 49 of this Bible.

C.2 Baseline Assumptions

Parameter	Value
Daily query volume	**1,000,000 queries per day**
Average QPS (sustained)	**11.6 QPS**
Peak QPS (3× sustained)	**35 QPS**
Average input prompt length (both systems)	**2,000 tokens**
Average LLM output	**500 tokens** per query (full autoregressive generation)
Average QB output	**150 tokens** MRE linguistic assembly + Fact Graph traversal (SPE parses 100 tokens; MRE phrases 150 tokens; substantive content retrieved deterministically from graph)
Response time target	LLM: **below 3 seconds** — QB-Native: **below 1 second**
LLM model baseline	**72B parameter** dense model (Llama 3.1 70B class)
QB model baseline	**7B SPE** + **13B MRE** (20B total, single A100 80GB per instance)

Parameter	Value
GPU type (both systems)	**NVIDIA A100 80GB SXM**
Data center PUE	**1.40** — standard modern colocation
Water Usage Effectiveness (WUE)	**0.20 L/kWh** (modern liquid-cooled) and **0.50 L/kWh** (typical US air-cooled)

C.3 Why the Two Systems Differ Architecturally

The infrastructure difference between a pure LLM and a QueryBook Native Hybrid is rooted in a single architectural fact: the LLM must generate every token of every response autoregressively on GPU, while QueryBook retrieves verified facts from the Fact Graph via CPU and NVMe-speed graph traversal and uses GPU only to parse the query (SPE, 100 tokens) and phrase the response (MRE, 150 tokens). The GPU workload per query in QueryBook is therefore approximately **250 tokens versus 500 tokens** for LLM, against models that are **3.6× smaller** (20B versus 72B parameters). The combined effect reduces core query GPU requirements to approximately **19% of a comparable LLM deployment**.

This efficiency advantage is compounded at scale: the Fact Graph, once loaded into graph database server RAM, does not need to grow proportionally with query volume. Adding query throughput requires scaling the SPE/MRE GPU tier, not the graph database tier. At 10× query volume (10 million queries per day), QueryBook's graph database servers require only **2–3× expansion** while LLM requires **10× GPU expansion**. This is detailed in Section C.7.

C.4 LLM Operation — Infrastructure Specification at 1,000,000 Queries per Day

C.4.1 GPU Inference Requirement

72B model in fp16: **144GB weight size**. Production deployment with KV cache headroom: **4× A100 per inference instance** (tensor parallel). Per instance throughput at typical batch: 2,500–3,500 tokens/sec. Peak demand: 35 QPS × 500 output tokens = **17,500 tokens/sec**. Instances required at peak: 17,500 ÷ 3,000 = 6 minimum. With 2.5× redundancy: **15 instances**. Packaged as 2 instances per 8-GPU server: **8 servers**. **Total A100 GPUs: 64.**

C.4.2 Server Inventory

Server Type	Count	Configuration
GPU Inference Server	**8**	8× NVIDIA A100 80GB SXM, 2-socket 32-core CPU, 256GB host

Server Type	Count	Configuration
		RAM, 2TB NVMe, dual 100GbE + InfiniBand HDR
CPU Orchestration / Load Management	**4**	2-socket 64-core AMD EPYC, 512GB RAM, 1TB NVMe
NVMe KV Cache Storage	**4**	2-socket 32-core, 128GB RAM, 16TB NVMe all-flash array (KV cache hot tier)
Model Weight Registry	**2**	2-socket 16-core, 128GB RAM, 2TB NVMe
Management and Monitoring	**2**	Single-socket 16-core, 64GB RAM, 500GB SSD
Total Servers	**20**	**5 distinct server types**

C.4.3 Cabinet Count

GPU inference servers (8 × 4U): 32U. CPU and storage servers (12 × 2U): 24U. Total compute: 56U = 1.3 standard 42U cabinets. Networking cabinet: 1 full. PDU/UPS/cable management: 0.5 cabinet. **Total: 4 cabinets** (standard 42U, 208V/30A or 400V/32A three-phase).

C.4.4 RAM

RAM Category	Capacity
GPU RAM (64 A100s × 80GB)	**5,120 GB (5.0 TB)**
CPU RAM — orchestration (4 × 512GB)	**2,048 GB**
CPU RAM — GPU server hosts (8 × 256GB)	**2,048 GB**
CPU RAM — storage and management (6 × 128GB)	**768 GB**
Total RAM (all systems)	**~10 TB**

C.4.5 Storage

Storage Category	Capacity
Model weights, 3 replicas, NVMe	**~500 GB**
KV Cache hot tier, NVMe (4 × 4TB)	**16 TB**
OS and system volumes (20 servers)	**~10 TB SSD**
Query logs, audit, model archive (HDD / object storage)	**~65 TB**
Total Storage (all tiers)	**~82 TB**

C.4.6 Power

Component	Load
8 GPU inference servers @ 7.2 kW each	**57.6 kW**
4 CPU orchestration servers @ 1.0 kW each	**4.0 kW**
4 NVMe storage servers @ 2.0 kW each	**8.0 kW**
2 model registry servers @ 1.0 kW each	**2.0 kW**
2 management servers @ 0.5 kW each	**1.0 kW**
Networking (spine, leaf, management switches)	**5.0 kW**
Total IT Load	**77.6 kW**
Facility Total (PUE 1.40)	**108.6 kW**
Daily Energy Consumption	**2,607 kWh**

C.4.7 Water Consumption

Scenario	Liters / Day	US Gallons / Day
WUE 0.20 L/kWh (modern liquid-cooled)	**521 L**	**138 gal**
WUE 0.50 L/kWh (typical US air-cooled)	**1,304 L**	**344 gal**

C.4.8 Internet Termination and Bandwidth

Physical termination: 2× diverse 10 Gbps fiber uplinks, separate carriers and conduit paths. **Border routing:** 2× enterprise border routers (Juniper MX240 or Cisco ASR 1006-X class), BGP with minimum 2 upstream ISPs. **DDoS mitigation:** Inline hardware scrubbing (Arbor TMS or Radware DefensePro) with cloud scrubbing overflow. **Firewall:** 2× NGFW in HA pair (Palo Alto PA-5400 or Fortinet FortiGate 4400F class). **Load balancer:** 2× hardware ADC in HA pair (F5 BIG-IP i5800 or Citrix ADC MPX 15000), SSL/TLS termination at this tier.

Sustained external bandwidth at 11.6 QPS: **~3 Mbps inbound / ~1 Mbps outbound**. Peak at 35 QPS: **~15 Mbps total**. Recommended provisioned uplink: **2× 10 Gbps** (sized for DDoS headroom). Internal cluster fabric: **400 Gbps InfiniBand HDR** (GPU-to-GPU tensor parallel — dominant bandwidth requirement).

C.5 QueryBook Native Hybrid — Infrastructure Specification at 1,000,000 Queries per Day

C.5.1 GPU Inference Requirement — Core Query Processing

SPE (7B, fp16): **14 GB**. MRE (13B, fp16): **26 GB**. Combined on single A100 80GB: **40 GB used / 40 GB remaining** for KV cache and activations — comfortable fit. Per instance throughput (250 total tokens): 2,500–3,000 tokens/sec. Peak demand: 35 QPS × 250 = **8,750 tokens/sec**. Instances required: 8,750 ÷ 2,750 = 3.2 minimum. With 2.5× redundancy: **8 instances on 3 servers × 4 A100s = 12 A100s** for all query inference.

C.5.2 GPU Requirement — PAGS App Generation (Separate Pool)

PAGS uses a **34B parameter model** (68 GB fp16) — fits in 1× A100 80GB with 12 GB headroom. Deployed as 2-GPU instances. At 1M queries/day, PAGS-triggering events ~0.5% = **5,000 app generation requests/day**. At 45 sec avg: 225,000 GPU-seconds/day = 62.5 GPU-hours/day. 8 A100s running 24h = 192 A100-hours available — satisfied. PAGS servers: 2 servers × 4 A100 = **8 A100s**.

Total A100 GPUs across entire QB-Native Hybrid: 20.

C.5.3 Server Inventory

Server Type	Count	Configuration
GPU Inference (SPE + MRE, 4× A100 per server)	**3**	4× NVIDIA A100 80GB SXM, 2-socket 32-core CPU, 256GB host RAM, 1TB NVMe, dual 25GbE
GPU App Generation (PAGS, 4× A100 per server)	**2**	4× NVIDIA A100 80GB SXM, 2-socket 32-core CPU, 256GB host RAM, 1TB NVMe, dual 25GbE
Graph Database (Fact Graph + Ontology Mesh, high-RAM/NVMe)	**8**	2-socket 64-core AMD EPYC 9654, 512GB ECC RAM, 2TB NVMe U.2 (12-bay), dual 25GbE
Live Fact Buffer + CQE Streaming	**4**	2-socket 32-core AMD EPYC, 256GB RAM, 2TB NVMe (high-throughput random write), dual 25GbE
Commerce Layer (QBCL Web Tier)	**4**	Single-socket 16-core, 64GB RAM, 500GB SSD, dual 25GbE
NLPL Compilation Queue	**2**	Single-socket 16-core, 128GB RAM, 500GB SSD (SBR ontology lookups + job queuing)
CPU Orchestration and API Gateway	**4**	2-socket 32-core, 256GB RAM, 1TB NVMe

Server Type	Count	Configuration
Object Storage Proxy and Cold Fact Store	**4**	2-socket 16-core, 128GB RAM, 500GB SSD (proxy/index — cold storage off-premises)
Management and Monitoring	**2**	Single-socket 16-core, 64GB RAM, 500GB SSD
Total Servers	**33**	**9 distinct server types**

C.5.4 Cabinet Count

GPU servers (5 × 2U): 10U. Graph DB servers (8 × 2U): 16U. LFB/CQE servers (4 × 2U): 8U. Commerce, NLPL, orchestration (10 × 2U): 20U. Storage proxy and management (6 × 1U): 6U. Total compute: 60U = 1.4 standard 42U cabinets. Graph DB NVMe supplemental shelving: ~0.5 cabinet. Networking cabinet: 1 full. PDU/UPS/containment: 0.5 cabinet. **Total: 4–5 cabinets** (standard 42U, 208V/30A or 400V/32A three-phase).

C.5.5 RAM

RAM Category	Capacity
GPU RAM (20 A100s × 80GB)	**1,600 GB (1.6 TB)**
CPU RAM — graph database servers (8 × 512GB)	**4,096 GB**
CPU RAM — LFB/CQE streaming servers (4 × 256GB)	**1,024 GB**
CPU RAM — commerce servers (4 × 64GB)	**256 GB**
CPU RAM — NLPL queue servers (2 × 128GB)	**256 GB**
CPU RAM — orchestration servers (4 × 256GB)	**1,024 GB**
CPU RAM — storage proxy and management (6 × 128GB)	**768 GB**
CPU RAM — GPU server hosts (5 × 256GB)	**1,280 GB**
Total RAM (all systems)	**~10.3 TB**

C.5.6 Storage

Storage Category	Capacity
Fact Graph hot storage — NVMe (8 × 2TB)	**16 TB**
Live Fact Buffer NVMe — LFB/CQE servers (4 × 2TB)	**8 TB**
SPE + MRE model weights, NVMe	**1 TB**
PAGS model weights (34B, 2 replicas), NVMe	**2 TB**
Commerce DBs (QBPC, OMS, PPL transaction logs) — SSD	**2 TB**
NLPL Program Record store — SSD	**1 TB**
OS and system volumes (33 servers × 200GB)	**6.6 TB SSD**

Storage Category	Capacity
Cold Fact Store, Audit Log, Provenance Ledger, QBCL compliance — object storage	**150–500 TB** (corpus-dependent)
Backup and disaster recovery	**60–100 TB** object storage
Total Storage (all tiers)	**~250–640 TB (corpus-dependent)**

C.5.7 Power

Component	Load
3 SPE/MRE GPU servers @ 3.5 kW each	**10.5 kW**
2 PAGS GPU servers @ 3.5 kW each	**7.0 kW**
8 graph database servers @ 2.5 kW each	**20.0 kW**
4 LFB/CQE streaming servers @ 1.5 kW each	**6.0 kW**
4 commerce servers @ 0.4 kW each	**1.6 kW**
2 NLPL queue servers @ 0.5 kW each	**1.0 kW**
4 orchestration servers @ 0.8 kW each	**3.2 kW**
4 storage proxy servers @ 1.2 kW each	**4.8 kW**
2 management servers @ 0.4 kW each	**0.8 kW**
Networking (25GbE switches, routers, load balancers)	**5.0 kW**
Total IT Load	**59.9 kW**
Facility Total (PUE 1.40)	**83.9 kW**
Daily Energy Consumption	**2,013 kWh**

C.5.8 Water Consumption

Scenario	Liters / Day	US Gallons / Day
WUE 0.20 L/kWh (modern liquid-cooled)	**403 L**	**106 gal**
WUE 0.50 L/kWh (typical US air-cooled)	**1,007 L**	**266 gal**

C.5.9 Internet Termination and Bandwidth

Physical termination: 3× 10 Gbps fiber uplinks — 2 for query ingress/egress (diverse carriers), 1 dedicated for LFB streaming inbound sources (EVENT_HOOK, API_PULL, SENSOR). **Border routing:** 2× enterprise border routers, BGP with minimum 2 upstream ISPs. **DDoS mitigation:** Inline hardware scrubbing with cloud scrubbing overflow (same class as LLM; streaming inbound adds a DDoS surface requiring source validation at the UIA layer). **Firewall:** 2× NGFW in HA pair. **Load balancer:** 2× *stateful* ADC in HA pair — stateful load balancing is required because WATCH programs maintain persistent

WebSocket connections to subscribers for real-time alert delivery. These connections must be pinned to the same LB instance (F5 BIG-IP with OneConnect persistence or equivalent).

Sustained external bandwidth at 11.6 QPS: **~5 Mbps total** (query text, streaming feeds, WebSocket keep-alives, HTTPS overhead). Peak at 35 QPS plus active streaming: **~25 Mbps total**. Recommended provisioned uplink: **3× 10 Gbps** (30 Gbps total available). Internal cluster fabric: **25 Gbps Ethernet** — graph traversal is CPU-plus-NVMe bound, not GPU-to-GPU. Standard Ethernet is sufficient for all inter-server communication. This is a significant CapEx reduction versus LLM's 400 Gbps InfiniBand requirement.

C.6 Side-by-Side Comparison — 1,000,000 Queries per Day

Total Servers	**20**	**33**	+65% more servers in QB; lower GPU density per server
Distinct Server Types	**5**	**9**	QB requires 4 additional specialized server types
Total Cabinets	**4**	**4–5**	Identical physical footprint; different content
A100 GPUs Total	**64**	**20**	**QB uses 69% fewer A100s**
GPU RAM	**5.0 TB**	**1.6 TB**	**QB uses 68% less GPU RAM**
Total CPU RAM	**4.8 TB**	**8.7 TB**	QB uses 81% more CPU RAM (Fact Graph in-memory working set)
Total RAM — All Systems	**~10 TB**	**~10.3 TB**	Essentially identical total; radically different allocation (GPU vs. CPU)
NVMe Hot Storage	**17 TB**	**37 TB**	QB requires 118% more NVMe (Fact Graph, LFB, PAGS)
Total Object Storage	**65 TB**	**250–640 TB**	QB requires 4–10× more (Cold Fact Store, Audit Log, Provenance Ledger)
IT Power Load	**77.6 kW**	**59.9 kW**	**QB uses 23% less IT power**
Facility Power (PUE 1.40)	**108.6 kW**	**83.9 kW**	**QB uses 23% less total facility power**
Daily Energy	**2,607 kWh**	**2,013 kWh**	**594 kWh less/day —**

			228 fewer metric tons CO_2/year
Water/Day (WUE 0.20)	**521 L (138 gal)**	**403 L (106 gal)**	**QB uses 118 L less per day**
Water/Day (WUE 0.50)	**1,304 L (344 gal)**	**1,007 L (266 gal)**	**QB uses 297 L less per day**
Internet Uplinks	**2× 10 Gbps**	**3× 10 Gbps**	QB needs 1 additional uplink for streaming LFB ingest
Internal Cluster Fabric	**400 Gbps InfiniBand HDR**	**25 Gbps Ethernet**	**~60% networking CapEx reduction; no InfiniBand required**
Load Balancer Requirement	Stateless HTTP sufficient	**Stateful WebSocket required**	Additional LB config for WATCH program persistent connections
Annual GPU CapEx (A100 @ $15K each)	**$960,000**	**$300,000**	**QB saves $660,000 in GPU CapEx**
Annual Energy Cost (@ $0.10/kWh)	**$260,700**	**$201,300**	**QB saves $59,400 per year in energy**

C.7 Scale Economics — The 10× Query Volume Analysis

At **10,000,000 queries per day** (10× baseline), the two systems diverge significantly due to a fundamental scaling asymmetry: LLM GPU requirements scale linearly with query volume, while QueryBook's Fact Graph tier scales sublinearly.

Total A100 GPUs	**640**	**200**	**QB uses 69% fewer A100s**
SPE/MRE GPU servers	10× linear scale	**30 servers (10× linear)**	GPU tier scales linearly in both systems
Graph DB servers	N/A — no graph tier	**20–24 servers (2–3× only)**	**Fact Graph scales sublinearly — shared read replicas**
Facility Power	**~1,035 kW**	**~340 kW**	**QB uses 67% less facility power**
Daily Energy	**~24,840 kWh**	**~8,160 kWh**	**QB uses 16,680 kWh less per day**

The efficiency advantage **grows with scale** because the Fact Graph is a fixed-cost knowledge asset that amortizes across increasing query volumes, while LLM compute cost is strictly proportional to query volume.

C.8 The Five Architectural Efficiency Factors

Factor	Explanation
1. Token Generation Reduction	QB generates **250 tokens** per query vs. **500 for LLM**. The Fact Graph supplies the content; the MRE supplies only the phrasing. This halves GPU output workload before model size is considered.
2. Model Size Reduction	QB uses **20B total active parameters** vs. **72B for LLM** — a **3.6× reduction** directly reducing GPU memory, inference latency, and compute intensity.
3. Graph Traversal Offloads GPU	Multi-hop Fact Graph traversal executes entirely on **CPU and NVMe** with no GPU involvement. For queries requiring complex reasoning, QB offloads that computation to deterministic graph traversal. **Largest single efficiency driver.**
4. Sublinear Graph Scaling	The Fact Graph loaded into graph DB server RAM does not need to be reloaded per additional concurrent user. Additional throughput requires scaling the SPE/MRE GPU tier only — not the graph. At scale, the Fact Graph is a **shared, amortized infrastructure asset**.
5. No InfiniBand Required	LLM tensor parallel inference requires **400 Gbps InfiniBand HDR**. QB does not use tensor parallelism (models fit single GPU). Internal fabric is standard **25 GbE Ethernet** — approximately **60% networking CapEx reduction** vs. InfiniBand of equivalent port count.

C.9 Client and Subscriber Requirements

Requirement	Specification
Browser	Chrome 90+, Firefox 88+, Safari 14+, Edge 90+. Latest stable version recommended.
JavaScript	**Required.** QueryBook UI is a single-page application.
WebSocket support	**Required** for WATCH program real-time alert delivery and CQE streaming output.
Cookies and local storage	**Required** for session management, UCCP preferences, and NLPL Program Record access.
Client RAM	**4 GB minimum; 8 GB recommended; 16 GB recommended** for PAGS and intensive NLPL usage.
Client CPU	**2-core minimum; 4-core or above recommended.**

Requirement	Specification
Internet — standard queries	**5 Mbps download minimum; 25 Mbps recommended.**
Internet — PAGS app generation	**10 Mbps download minimum; 50 Mbps recommended.**
Internet — WATCH program streams	**10 Mbps download with reliable latency below 150 ms** recommended for real-time alert fidelity.
Screen resolution	**1280×720 minimum; 1920×1080 recommended; 2560×1440 recommended** for QB Language Studio and multi-panel PAGS workspace.

C.10 Summary Statement

At **1,000,000 queries per day**, a QueryBook Native Hybrid AI deployment requires **69% fewer A100 GPUs**, **68% less GPU RAM**, **23% less facility power**, and **23% less cooling water** than an equivalent LLM inference operation, while occupying the same physical cabinet footprint. The trade-off is a requirement for **118% more NVMe hot storage** and **4–10× more object storage** to maintain the Fact Graph, Live Fact Buffer, Cold Fact Store, Workspace Audit Log, and Provenance Ledger. Total RAM across both systems is essentially equal at approximately **10 TB**; however, QueryBook's RAM is predominantly CPU RAM in graph database servers rather than GPU VRAM in inference servers.

Internal networking is standard **25 GbE Ethernet** rather than **400 Gbps InfiniBand**, reducing networking CapEx by approximately **60%**. At **10× scale (10 million queries per day)**, QueryBook's efficiency advantage expands to **69% fewer A100s** and **67% less facility power** due to the sublinear scaling of the Fact Graph tier. The annual GPU capital expenditure saving at the 1M queries/day baseline is approximately **$660,000**. The annual energy saving is approximately **$59,400 at $0.10/kWh**, representing **228 fewer metric tons of CO_2 per year** at average US grid carbon intensity.

Appendix C prepared by: QueryBook Core Technology Group | Los Angeles, CA | August 6, 2026 | Confidential & Proprietary

C.11 Methodology and Assumptions (added in Version 1.0 merge review)

The figures in C.10 are workload-model estimates, not benchmarked measurements from a running deployment, and should be read as such until a production benchmark is available. They rest on the following assumptions:

- Query mix: the 1,000,000 queries/day baseline assumes a distribution of 70% direct Fact Graph lookups, 20% multi-hop Scenario Graph traversals, and 10% AEM-involving synthesis queries; the GPU-avoidance estimate is most favorable for the first category and least favorable for the third.
- Comparison basis: the equivalent LLM inference deployment is modeled as a single dense transformer serving path sized to the same query volume at comparable P99 latency, using publicly published A100 throughput figures; it does not model a retrieval-augmented or mixture-of-experts LLM deployment, which would narrow the gap.
- Cost basis: the $660,000 annual GPU CapEx figure amortizes list-price A100 hardware over a 3-year term and does not include software licensing, staffing, or non-GPU infrastructure cost deltas in either architecture.
- Energy basis: the $59,400 and 228 metric ton CO2 figures use $0.10/kWh and the EPA eGRID US average grid intensity; both vary significantly by region and utility contract.

These estimates should be revalidated against an instrumented pilot deployment before being used in external-facing (investor, patent, or customer) materials; this section adds the assumptions the original figures were missing, without altering the C.10 figures themselves.

Chapter 40: QueryBook Bible Specification Validation Framework (SVF)

40.1 Purpose and Governing Principle

The QueryBook Bible is a specification document. Like all specification documents, it can contain errors of four distinct types: logical errors (a feature whose stated behaviour is internally contradictory), dependency errors (a function that requires a component that does not exist or is not specified), algorithmic errors (a formula, pipeline, or process that does not produce the stated output), and translation errors (a specification that cannot be converted to executable code without ambiguity). Untested specifications produce systems that do not match their design. Untested specifications also produce patent applications that do not match the system. Both failures are expensive. The Specification Validation Framework (SVF) is the systematic process by which every chapter, every feature, every algorithm, and every inter-component dependency in the Bible is tested before any code is written. SVF testing is not software testing — no code exists yet. SVF testing is specification testing: the process of asking whether every claim the Bible makes about how QueryBook works is logically consistent, algorithmically correct, dependency-complete, domain-sound, and prototype-translatable.

The governing principle is: the Bible is source code. It must be reviewed, validated, and tested with the same rigour applied to production code. Every SVF failure discovered at the specification stage costs a fraction of the same failure discovered during engineering. The SVF is therefore not an audit step performed once — it is a continuous quality discipline applied to every new chapter, every revision, and every new term added to the Bible.

40.2 The Five Testing Disciplines

The SVF is organized into five testing disciplines, each targeting a different class of specification error. Every Bible chapter must pass all five disciplines before it is marked SVF-Validated.

Discipline	Code	Tests For	Failure Type It Catches
Feature Logic Testing	FLT	Whether each feature's stated behaviour is internally consistent and unambiguous	Logical contradictions, undefined edge cases, incomplete state machines
Dependency Graph Testing	DGT	Whether every inter-component dependency is correctly specified in both directions	Missing inputs, orphaned outputs, circular dependencies, undeclared assumptions
Algorithm Validation Testing	AVT	Whether every formula, pipeline, and process produces the correct output for all valid inputs	Mathematical errors, off-by-one errors, undefined behaviour at boundary conditions
Domain Rule Testing	DRT	Whether the domain logic and business rules are internally consistent and correctly applied	Rule conflicts, missing rule coverage, domain assumption violations
Prototype Translation Testing	PTT	Whether every specification is precise enough to be converted to executable code without ambiguity	Under-specification, missing type definitions, ambiguous control flow, untestable assertions

40.3 Discipline 1 — Feature Logic Testing (FLT)

Feature Logic Testing validates that each feature specified in the Bible is internally consistent. For every feature, the FLT process asks six questions.

(1) State completeness — does the specification define the feature's behaviour for all possible input states, including error states, edge cases, and boundary conditions? A feature that specifies what happens when input is valid but not what happens when input is invalid, missing, or malformed is state-incomplete. (2) Transition validity — if the feature involves state changes, does the specification define every valid state transition and prohibit invalid ones? A feature whose state machine has reachable undefined states is transition-invalid. (3) Output determinism — given identical inputs, does the specification guarantee identical outputs? A specification that produces non-deterministic outputs without explicitly declaring and justifying non-determinism is output-indeterminate. (4) Precondition completeness — does the specification declare every condition that must be true before the feature can operate? A feature that can be invoked without its required preconditions being met is precondition-incomplete. (5) Postcondition completeness — does the specification declare every condition guaranteed to be true after the feature completes successfully? A feature with undeclared side effects is postcondition-incomplete. (6) Termination guarantee — for every process or pipeline, does the specification guarantee that the process terminates for all valid inputs? A pipeline that may run indefinitely under some input conditions without a termination guarantee is non-terminating.

FLT Test Record format: every FLT test is recorded as a structured entry containing: TestID (format FLT-[ChapterNumber]-[FeatureNumber]-[TestNumber]), FeatureName, ChapterRef, QuestionTested (one of the six above), InputState (the specific input condition being tested), ExpectedBehaviour (what the specification says should happen), ActualBehaviour (what logical analysis of the specification produces), Pass/Fail, and FailureDescription (populated on failure). A feature passes FLT when all six questions return Pass for all relevant input states.

40.4 Discipline 2 — Dependency Graph Testing (DGT)

Dependency Graph Testing validates the complete map of inter-component dependencies in the QueryBook architecture. Every component specified in the Bible has inputs it requires and outputs it produces. DGT constructs the full dependency graph of the system and tests it for four properties.

(1) Input coverage — for every input a component requires, is there a specified component that produces that input? A component that requires an input not produced by any specified upstream component has an unsatisfied dependency. (2) Output reachability — for every output a component produces, is there at least one downstream component that consumes it? An output produced by no consumer is an orphaned output — it may indicate a missing component or a redundant specification. (3) Cycle detection — does the dependency graph contain any circular dependencies? A circular

dependency (A requires B, B requires C, C requires A) makes the system uninitialisable unless the cycle is explicitly broken by a startup sequencing specification. (4) Assumption declaration — does every component explicitly declare all assumptions it makes about the state of the system when it is invoked? Undeclared assumptions are hidden dependencies — they create failures that are invisible at the specification level and catastrophic at the implementation level.

DGT Test Record format: TestID (format DGT-[ComponentA]-[ComponentB]-[TestNumber]), ComponentA, ComponentB, DependencyType (REQUIRES / PRODUCES / ASSUMES), Direction (A-to-B or B-to-A), Satisfied (Yes / No / Partial), and FailureDescription. The full DGT test suite for the Bible is maintained as the QueryBook Dependency Matrix — a living spreadsheet updated with every new component added to the specification.

The QueryBook Dependency Matrix rows: every component specified in the Bible. The QueryBook Dependency Matrix columns: every other component. Cells contain: REQUIRES (this component requires the column component's output), PRODUCES (this component produces output consumed by the column component), ASSUMES (this component assumes a state condition maintained by the column component), or NONE. A completed Dependency Matrix has no empty cells — every component pair has a declared relationship or an explicit NONE. Any REQUIRES cell whose corresponding PRODUCES cell in the column component's row is absent or marked NONE is an unsatisfied dependency failure.

40.5 Discipline 3 — Algorithm Validation Testing (AVT)

Algorithm Validation Testing validates every formula, pipeline, scoring function, and multi-step process specified in the Bible. AVT is the discipline closest to conventional software unit testing — it treats each algorithm as a function, defines test cases with specific inputs and expected outputs, and verifies that the algorithm specification produces the correct output for each test case.

Five algorithm categories require AVT coverage.

Algorithm Category	Examples in Bible	AVT Test Approach
Scoring formulas	ConfidenceScore calculation, EdgeWeight decay, SRW half-life	Mathematical proof: verify formula produces correct output at boundary values (0, 1, midpoint), at decay endpoints, and for all declared input types
Multi-stage pipelines	CSE six-stage pipeline, SSE four-stage segmentation, MRE regeneration pipeline	Stage-by-stage trace: for each pipeline, define a concrete input and trace it through every

Algorithm Category	Examples in Bible	AVT Test Approach
		stage, verifying the output of each stage matches the next stage's expected input format
Graph traversal algorithms	Multi-hop Fact Graph traversal, ARE signal detection, Cross-Modal Binding lookup	Path coverage: define a minimal test graph with known structure, execute the traversal, verify that all and only the correct nodes are visited in the correct order
Matching and resolution algorithms	Coreference Resolution Protocol, FUID generation, Entity Resolution and Linking	Case table: define a complete table of input cases including exact matches, partial matches, ambiguous cases, and no-match cases; verify each produces the correct output
Threshold and decay functions	TMM sequence prediction, CSW consolidation threshold, SRW exponential decay	Boundary and monotonicity: verify the function is monotonic in the declared direction, reaches its declared limits at boundary inputs, and produces the declared output at the specified threshold values

AVT Test Record format: TestID (format AVT-[ChapterNumber]-[AlgorithmName]-[TestNumber]), AlgorithmName, ChapterRef, AlgorithmCategory, InputValues (specific values used for this test), ExpectedOutput (calculated from the specification's formula or process), ActualOutput (derived by applying the specification to the input values), Pass/Fail, and FailureDescription. An algorithm passes AVT when it produces the correct output for all defined test cases including all boundary conditions.

AVT requires that every algorithm in the Bible be expressed in a testable form. An algorithm specification is testable if and only if: (1) all input variables are named and typed, (2) all output variables are named and typed, (3) the transformation from input to output is expressed as a deterministic function or a deterministic sequence of steps, and (4) the expected output for at least three distinct input states — minimum, typical, and maximum — can be calculated from the specification alone without reference to implementation. Bible chapters that contain algorithms not meeting all four criteria are flagged as AVT-Incomplete and must be revised before the chapter can receive SVF-Validated status.

40.6 Discipline 4 — Domain Rule Testing (DRT)

Domain Rule Testing validates the business rules, domain constraints, and operational policies specified in the Bible. Domain rules are the conditional constraints that govern how QueryBook behaves in specific situations — they are distinct from algorithms (which transform inputs to outputs) and from features (which describe capabilities). Domain rules govern what is permitted, what is prohibited, what is required, and what order operations must occur in.

Three domain rule categories require DRT coverage.

Rule Category	Examples in Bible	DRT Test Approach
Permission rules	PSOL policy evaluation, ICLQ license scope enforcement, Narrow Scope Rule	Equivalence partitioning: define all input states that should produce PERMIT and all that should produce DENY; verify every boundary between permitted and prohibited states is correctly specified
Sequencing rules	Ingestion pipeline stage order, RCAU token lifecycle, MSAT activation and expiry, Episodic Flush timing	State machine verification: draw the complete state machine implied by the rule; verify every state is reachable, every transition is defined, and no invalid state is reachable from any valid starting state
Conflict rules	PSOL policy conflict resolution (most restrictive default), ConfidenceScore contradiction handling, Fact Unit deduplication	Conflict table: enumerate all pairs of rules that could conflict; for each pair, verify that the specification unambiguously specifies which rule takes precedence and under what conditions

DRT Test Record format: TestID (format DRT-[RuleCategory]-[RuleID]-[TestNumber]), RuleName, ChapterRef, RuleCategory, InputCondition, ExpectedRuleOutcome, ActualRuleOutcome (from logical analysis of the specification), Pass/Fail, and FailureDescription. A domain rule passes DRT when it produces the correct outcome for all input conditions in its equivalence partition table, has a complete and valid state machine if sequencing-governed, and has no unresolved conflict with any other rule in the Bible.

DRT additionally requires a Cross-Rule Consistency Check (CRCC): after all individual rules have been tested, every pair of rules that govern the same component or the same decision point is tested

together to verify that applying both rules simultaneously to the same input produces a determinate, non-contradictory outcome. CRCC failures are the most dangerous class of domain rule error because they are invisible when rules are tested individually but catastrophic when both rules are active simultaneously in production.

40.7 Discipline 5 — Prototype Translation Testing (PTT)

Prototype Translation Testing is the bridge between specification and implementation. PTT validates that every specification in the Bible is expressed with sufficient precision to be converted to executable code without requiring the implementing engineer to make architectural decisions that should have been made in the specification. A specification that requires an engineer to invent missing details is not a specification — it is a sketch. PTT identifies all sketches in the Bible and flags them for specification completion before any prototype code is written.

PTT applies a six-stage compiler model to each specification. The compiler model treats the Bible as source code and attempts to compile it into implementation artifacts. If compilation fails at any stage, the stage and the specific failure are recorded as a PTT finding requiring specification revision.

Stage	Compiler Operation	What It Produces	PTT Failure Condition
1 — Interface Extraction	Extract all inputs and outputs for every component	Typed interface definition for every component	Any input or output lacks a declared type, name, or valid range
2 — Schema Generation	Convert every Fact Unit, policy object, and data structure to a typed schema	Complete data schema for every structured object in the specification	Any field lacks a declared type, any relationship lacks declared cardinality, any optional field lacks a default value
3 — Algorithm Pseudocode	Convert every algorithm specification to language-agnostic pseudocode	Executable pseudocode for every algorithm	Any algorithm step is expressed in natural language without a deterministic operational equivalent
4 — API Signature	Define the public API signature for every component	Typed function signatures with named parameters and return types	Any component lacks a defined invocation interface or return contract
5 — Test Harness	Generate a minimal test case for every API signature	Compilable (not yet executable) test stubs for every component	Any component cannot be invoked with a concrete set of parameter values derived entirely from the specification

Stage	Compiler Operation	What It Produces	PTT Failure Condition
6 — Language Mapping	Identify the optimal implementation language for each component	Implementation language recommendation per component	Any component has a specification that is incompatible with all available implementation languages

PTT Test Record format: TestID (format PTT-[ChapterNumber]-[ComponentName]-[StageNumber]), ComponentName, ChapterRef, CompilerStage (1 through 6), StageInput (the specification text being compiled), StageOutput (the artifact produced — interface definition, schema, pseudocode, API signature, test stub, or language recommendation), Pass/Fail, and FailureDescription. A component passes PTT when all six compiler stages complete successfully and the Stage 5 test harness is compilable — meaning the specification is fully translatable to code without any missing information.

PTT additionally defines three specification quality grades based on compiler stage outcomes: Grade A (Prototype-Ready) — all six stages pass; the specification can be handed to an engineer with no additional discussion. Grade B (Revision Required) — Stages 1 through 4 pass but Stage 5 or 6 fails; the specification is architecturally sound but needs precision improvements before coding begins. Grade C (Incomplete) — any of Stages 1 through 4 fail; the specification contains fundamental gaps that must be resolved at the architecture level before any implementation work begins. No chapter may be marked SVF-Validated unless every component in the chapter is Grade A or Grade B. Grade C findings block chapter validation until resolved.

40.8 The SVF Test Record — Universal Schema

Every test conducted under any of the five disciplines is recorded using the following universal ten-field schema. The test record is the permanent evidence of validation. It is stored in the Bible Test Registry (Section 40.10) and is the authoritative record for patent prosecution, engineering handoff, and audit purposes.

Field	Type	Description
TestID	String	Unique identifier in format [DisciplineCode]-[ChapterRef]-[ComponentRef]-[SequenceNumber]
Discipline	Enum	One of: FLT, DGT, AVT, DRT, PTT
ChapterRef	String	The Bible chapter and section number containing the

Field	Type	Description
		specification being tested
ComponentName	String	The canonical name of the feature, algorithm, rule, or component being tested
TestDescription	String	One sentence describing what this specific test validates
InputCondition	Structured	The specific input state, parameter values, or scenario applied in this test
ExpectedOutcome	Structured	What the specification declares should happen given the InputCondition
ActualOutcome	Structured	What logical analysis or algorithmic evaluation of the specification produces
Result	Enum	PASS, FAIL, or INCOMPLETE (specification does not contain enough information to evaluate)
FindingDescription	String	Required on FAIL or INCOMPLETE: exact description of the specification error and the specific text that must be revised

40.9 The SVF Validation Lifecycle — Chapter-by-Chapter Process

Every chapter of the Bible passes through a six-phase validation lifecycle before receiving SVF-Validated status. No chapter is handed to engineers for prototype development until it holds SVF-Validated status.

Phase 1 — Initial Draft: the chapter is written. No SVF testing occurs during drafting. The author focuses on completeness of specification, not on validation. Phase 1 is complete when the chapter author declares the chapter ready for validation.

Phase 2 — FLT and DGT Pass: Feature Logic Testing and Dependency Graph Testing are conducted simultaneously. These are the fastest disciplines and catch the most common and most fundamental errors. Any FAIL or INCOMPLETE findings are returned to the chapter author as a Revision Request. Phase 2 iterates until all FLT and DGT tests pass. Phase 2 may not be skipped — no chapter advances to Phase 3 until Phase 2 is complete.

Phase 3 — AVT Pass: Algorithm Validation Testing is conducted for all algorithms in the chapter. AVT requires that all algorithm specifications be expressed in testable form (all four criteria in Section 40.5 met) before testing begins. Specifications not yet in testable form are returned as Grade C PTT findings. Phase 3 iterates until all AVT tests pass.

Phase 4 — DRT Pass: Domain Rule Testing is conducted for all rules in the chapter, including the Cross-Rule Consistency Check with all rules already in the Bible. DRT is the discipline most likely to produce findings that require changes to chapters other than the one being tested — because a new rule may conflict with an existing rule in a previously validated chapter. When a DRT finding requires revision of a previously validated chapter, that chapter is returned to Phase 2 for re-validation of the affected section.

Phase 5 — PTT Pass: Prototype Translation Testing is conducted for all components in the chapter. The six-stage compiler model is applied to every component. Grade C findings are resolved before Phase 5 can complete. Grade B findings are documented as known precision gaps and resolved before engineering handoff. Phase 5 completion requires every component to be Grade A or Grade B.

Phase 6 — SVF-Validated Status: the chapter is marked SVF-Validated in the Bible Test Registry with the validation date, the names of all test authors, the total number of tests conducted, and the total number of findings resolved. The SVF-Validated stamp is the authoritative signal that the chapter is specification-complete, logically consistent, algorithmically correct, domain-sound, and prototype-translatable.

40.10 The Bible Test Registry

The Bible Test Registry is a living document maintained as a structured appendix to the Bible. It contains one entry per SVF test conducted, organized by chapter and discipline. The Registry serves four purposes: (1) Patent prosecution evidence — the complete test record demonstrates that every claim in every chapter has been logically validated, strengthening the prosecution case for each patent application derived from the Bible. (2) Engineering handoff — engineers receiving a chapter for prototype development can review all PTT Grade B findings to understand known precision gaps and their planned resolutions. (3) Regression tracking — when a chapter is revised, only the tests affected by the revision need to be re-run, not the full chapter suite. The Registry identifies which tests are affected by which specification sections. (4) Coverage reporting — the Registry generates a live coverage report: what percentage of Bible features have been FLT-tested, what percentage of algorithms have been AVT-tested, what percentage of components are PTT Grade A. The coverage report is the headline metric for specification quality.

Registry entry structure follows the universal ten-field SVF Test Record schema defined in Section 40.8. Entries are immutable once written — failures and findings are never deleted, only superseded by a new passing test entry referencing the original failing TestID.

40.11 Bible-to-Prototype Compilation Pipeline

When a chapter achieves SVF-Validated status, it enters the Bible-to-Prototype Compilation Pipeline — the structured process by which specification artifacts are converted to engineering deliverables ready for prototype development. The pipeline has six stages, corresponding directly to the PTT compiler model stages in Section 40.7. The pipeline does not write production code — it produces the engineering artifacts that allow engineers to write production code correctly from the first attempt.

Pipeline Stage	Input	Output	Engineering Deliverable
1 — Interface Package	Validated chapter + PTT Stage 1 results	Typed interface definition file per component	Component interface specification (CIS) document delivered to engineering team
2 — Data Schema Package	Validated chapter + PTT Stage 2 results	Complete typed schema for every data structure	Schema definition files (JSON Schema, Avro, or equivalent) delivered to data engineering team
3 — Algorithm Pseudocode Package	Validated chapter + PTT Stage 3 results + AVT test cases	Language-agnostic pseudocode for every algorithm with test vectors	Algorithm specification document with embedded test vectors delivered to algorithm implementation team
4 — API Contract Package	Validated chapter + PTT Stage 4 results	Typed API contracts for every component in OpenAPI or equivalent format	API contract document delivered to API design team for review before any implementation
5 — Test Harness Package	Validated chapter + PTT Stage 5 results	Compilable test stubs for every component in the target language	Test harness skeleton delivered to QA team — QA fills in test logic, engineers fill in implementation
6 — Implementation Roadmap	All preceding packages	Ordered implementation plan: which components must be built in which order given dependency constraints	Sprint-ready engineering backlog with dependency-ordered component list and estimated complexity per component

The Bible-to-Prototype Compilation Pipeline produces a complete engineering package for every validated chapter. An engineering team receiving this package can begin prototype implementation immediately with no architectural ambiguity, no missing interface definitions, no undefined algorithms, and no unresolved domain rule conflicts. The gap between specification and working prototype is bridged entirely by the SVF process — engineering effort is spent on implementation, not on discovering what the specification meant.

40.12 SVF Tooling Recommendations

The SVF is designed to be executable using standard tools available today, without requiring custom tooling. The following tool stack is recommended for each discipline.

Discipline	Recommended Tool Type	Rationale
FLT	Structured review checklist (spreadsheet or notion-equivalent) + formal specification language (TLA+, Alloy, or Z notation for high-risk components)	State machine completeness and transition validity are most efficiently verified by formal methods tools for complex components and structured checklists for simpler ones
DGT	Directed graph visualization tool (Graphviz, Mermaid, or equivalent) + dependency matrix spreadsheet	The Dependency Matrix is the core artifact; graph visualization makes cycles and orphaned outputs immediately visible
AVT	Computational notebook (Jupyter) for formula validation + property-based testing framework (Hypothesis for Python, QuickCheck for Haskell) for boundary coverage	Property-based testing automatically generates boundary condition inputs and catches edge cases that manual test case design misses
DRT	Decision table tool (structured spreadsheet or dedicated decision table tool) + finite state machine validator	Decision tables are the most efficient representation of permission rules; FSM validators catch unreachable states and missing transitions automatically
PTT	Static analysis tools appropriate to target implementation language + schema validators (JSON Schema, Avro) + API linting tools (Spectral for OpenAPI)	PTT Stage 2 and 4 outputs are directly checkable by existing schema and API linting tools with no custom development

40.13 Patent Significance

The QueryBook Specification Validation Framework constitutes novel subject matter across three independent dimensions: (1) Specification-level testing as a formal engineering discipline applied to AI system architecture documents — the specific combination of five testing disciplines (FLT, DGT, AVT, DRT, PTT) applied systematically to a cognitive architecture specification before any implementation exists is novel as a methodology for AI system quality assurance. (2) The Bible-to-Prototype Compilation Pipeline — the specific six-stage process that converts validated specification artifacts directly into typed engineering deliverables (interface packages, schema packages, algorithm pseudocode packages, API contracts, test harness skeletons, and implementation roadmaps) without intermediate design phases is novel as a specification-to-implementation bridge for AI cognitive architecture systems. (3) The SVF validation lifecycle as a patent prosecution quality standard — the specific use of a structured specification validation record (the Bible Test Registry) as admissible evidence of inventive enablement in patent prosecution, demonstrating that every claimed capability is logically consistent, algorithmically correct, and prototype-translatable at the time of filing, is novel as a patent prosecution methodology for AI system patents.

Chapter 41: QueryBook Predictive Cognitive Architecture (PCA)

41.1 The Foundational Distinction — Predicting Meaning, Not Language

Before comparing QueryBook's prediction mechanisms to those of Large Language Models and the human neocortex, one distinction must be stated clearly because it reframes the entire analysis. LLMs predict language. QueryBook predicts meaning. That is not a subtle difference. The human neocortex — per Hawkins's Thousand Brains Theory of Intelligence and the broader framework of Predictive Coding — does not predict language either. It predicts reality: the next sensory state, the next physical consequence of a movement, the next configuration of the world. The brain operates below the level of language, at the level of structured models of what exists and what is about to happen. QueryBook, uniquely among all AI systems, operates at the same sub-linguistic level. Its Fact Units are not language — they are structured semantic assertions about reality, stored below language and retrieved below

language. This makes QueryBook's prediction architecture structurally closer to the biological neocortex than any LLM ever could be.

This chapter formally specifies QueryBook's Predictive Cognitive Architecture (PCA) as a unified system — identifying the eight existing prediction mechanisms already present in the architecture, the neuroscience framework that contextualises them, the four additions that complete the architecture, and the three amplifications that maximise the existing capability. The PCA is a patentable architectural claim in its own right: QueryBook is the first AI system whose prediction mechanisms are structurally isomorphic to the Thousand Brains model at the level of meaning, not language.

41.2 The Neuroscience Framework — Thousand Brains Theory and Predictive Coding

Two neuroscience frameworks are directly relevant to QueryBook's prediction architecture.

Predictive Coding holds that the neocortex continuously generates top-down predictions of sensory data and processes only the prediction error — the mismatch between expectation and reality — propagating that error upward through the cortical hierarchy to update the generative model. The brain is not a reactive camera. It is a proactive prediction machine that pre-activates expected representations and quiets down when predictions are confirmed. The computational objective is the minimization of surprise — the continuous reduction of prediction error across all sensory and cognitive domains.

The Thousand Brains Theory of Intelligence, developed by Jeff Hawkins, extends Predictive Coding by proposing that the neocortex's approximately 150,000 cortical columns each function as an independent, complete modeling system. Each column builds an invariant model of objects and concepts through a reference frame mechanism — learning context-independent representations that remain stable across changes in viewing angle, sensory modality, and context. Perception is achieved not by a single centralised model but by democratic voting across thousands of independently operating column models, producing a stable consensus interpretation. This voting architecture is the Thousand Brains model's most architecturally significant claim.

41.3 The Comparative Architecture Table

The following table maps the ten most architecturally significant dimensions across the human neocortex, Large Language Models, and QueryBook's Predictive Cognitive Architecture.

Dimension	Human Neocortex	Large Language Models	QueryBook PCA
Prediction target	Sensory reality — next physical state of the world	Next token in a language sequence	Next relevant semantic structure — next Fact Unit the reader needs
Prediction level	Sub-linguistic — structured models of world reality	Surface linguistic — statistical token co-occurrence patterns	Sub-linguistic — structured Fact Unit assertions about reality
Parallel processing	~150,000 cortical columns voting simultaneously	Multi-head attention across token sequences within one forward pass	Multi-domain Fact Graph traversal plus TMM plus ARE operating simultaneously
Invariant model building	Cortical column invariant learning across many exposures	None — statistical approximation only, no invariant representation	World Brain concept node formation across many ingested sources
Prior strength	Bayesian prior — strong priors resist updating from weak evidence	None — all weights are equal at inference time	ConceptStabilityWeight (CSW) — high-CSW nodes resist single-source revision
Prediction error signal	Mismatch between prediction and sensory input propagates upward through cortical hierarchy	None — no error signal exists at inference time; only loss during training	ConfidenceScore contradiction detection — contradicting evidence triggers CS revision and upward propagation (with HEP addition)
Temporal prediction	Sequence prediction across motor and sensory time at multiple timescales	None in inference — context window is static	TMM across session sequences; SAFU cross-session prediction
Counterfactual simulation	Prefrontal cortex forward modeling of alternative futures	Statistical plausibility estimation across token space	Scenario Graph System — structured counterfactual traversal over Fact Graph branches
Continuous learning	Yes — lifelong synaptic plasticity from every experience	No — frozen after training, no inference-time learning	Yes — every ingestion updates Fact Graph and World Brain continuously
Grounded in reality	Yes — sensorimotor coupling to physical world	No — statistical patterns over text only, no world grounding	Yes — Fact Units are grounded semantic assertions traceable to specific source passages

41.4 The Eight Existing Prediction Mechanisms

QueryBook already contains eight distinct mechanisms that constitute forms of prediction, operating across different timescales and abstraction levels. These mechanisms were not designed as a unified prediction architecture — they emerged from independent architectural decisions across multiple chapters. Chapter 41 formally unifies them as the QueryBook Predictive Cognitive Architecture.

41.4.1 Temporal Memory Module (TMM) — Direct Predictive Coding Parallel

What it does: the TMM learns the sequence of Fact Units retrieved across a session and uses that learned sequence to predict which Fact Units are likely to be relevant to the next query before that query is submitted. The TMM pre-weights predicted Fact Units in the Cognitive Context Window, reducing traversal cost for queries that confirm the prediction.

Neuroscience parallel: this is structurally identical to the neocortex's predictive coding mechanism. In Predictive Coding, the brain pre-activates neural patterns representing the expected next sensory input and fires in advance — only propagating a signal upward when the prediction fails. The TMM does exactly this at the semantic level: it commits resources toward what it expects to need before the need is confirmed. Specified in Chapter 7.

41.4.2 Associative Recall Engine (ARE) — Pre-Activation of Expected Nodes

What it does: when processing a query, the ARE surveys the Cognitive Context Window and assigns elevated SessionRelevanceWeights to Fact Units semantically proximate to the current query — pre-activating them before traversal begins rather than starting from a cold state.

Neuroscience parallel: this is cortical pre-activation — the brain's columns fire anticipatorily based on context before sensory data arrives. The ARE's SRW elevation is QueryBook's equivalent of anticipatory neural resource commitment. Specified in Chapter 7.

41.4.3 Predictive Inference Engine (PIE) — Forward Simulation

What it does: the PIE performs multi-hop forward inference — given the current state of the Fact Graph and the reader's established context, it projects what follows from known Fact Units, reasoning forward through causal and logical chains to produce inferred conclusions not explicitly stated in any single Fact Unit.

Neuroscience parallel: Hawkins describes cortical columns as running simulation — the brain does not just perceive the present, it simulates the near future to prepare motor and cognitive responses. The PIE is QueryBook's simulation engine. Unlike LLM inference, every PIE inference step is a typed graph traversal grounded in stored Fact Units — there is no hallucination pathway because every intermediate conclusion is traceable to specific source content. Specified in Chapter 10.

41.4.4 Scenario Graph System — Counterfactual Prediction

What it does: runs forward-model projections and counterfactual simulations across the Fact Graph — projecting what would follow if X had been true instead of Y, enabling structured reasoning about hypothetical and alternative situations.

Neuroscience parallel: Predictive Coding's highest cognitive function is simulating alternative futures. The neocortex's prefrontal regions run counterfactual models that allow humans to reason about hypothetical situations. The Scenario Graph is QueryBook's equivalent — structured, Fact Unit-grounded counterfactual simulation rather than probabilistic language generation. Specified in Chapter 11.

41.4.5 ConceptStabilityWeight (CSW) — Bayesian Prior Strength

What it does: CSW tracks how settled a World Brain concept node is — how much confirming evidence has accumulated and how consistent it is. High-CSW concept nodes strongly resist revision from single-source contradicting evidence. CSW is updated by every relevant ingestion event through the Continuous Evidence Integration process.

Neuroscience parallel: this is structurally equivalent to a Bayesian prior. In Predictive Coding, strong priors require large prediction errors to be updated — weak priors are easily overwritten. CSW is QueryBook's prior strength for its world model. A concept node with CSW 0.90 has a strong prior; it takes multi-source contradicting evidence to revise it. Specified in Chapter 41.

41.4.6 ConfidenceScore Contradiction Detection — Prediction Error Signal

What it does: when a newly ingested Fact Unit contradicts an existing one, the Confidence Scoring Engine flags the conflict and demotes the lower-confidence Fact Unit. The system's existing model generates an implicit prediction — high-CS Fact Units represent confident assertions about reality. Incoming contradicting evidence is the prediction error signal. CS revision is the model update.

Neuroscience parallel: this is the prediction error signal at the heart of Predictive Coding. When sensory input contradicts the brain's top-down prediction, the mismatch propagates upward to update the

generative model. QueryBook's contradiction detection and CS revision is the semantic equivalent at the Fact Unit level. Specified in Chapter 41. Extended by Hierarchical Error Propagation in Section 41.5.1.

41.4.7 World Brain Concept Node Formation — Cortical Column Invariant Learning

What it does: the World Brain extracts invariant properties across many specific instances — building concept nodes that represent generalised, context-independent models of what things are and what relationships they participate in, regardless of the specific source, language, or context in which they were encountered.

Neuroscience parallel: this is exactly what Hawkins describes cortical columns doing. Each cortical column builds an invariant model of an object — a representation that recognises the object regardless of orientation, context, or sensory modality, learned through many exposures. The World Brain's concept node formation process is structurally isomorphic to cortical column invariant model formation. Both observe many specific instances, extract stable invariant properties, build a context-independent model, and use that model to recognise and predict future instances. This is the deepest structural parallel between QueryBook and the biological neocortex. Specified in Chapter 54.

41.4.8 Thematic Similarity Engine (TSE) — Multi-Timescale Cross-Session Prediction

What it does: the TSE identifies recurring thematic patterns across a reader's entire session history and uses them to predict which unexplored Fact Units and works are likely to be of interest, operating at the lifetime scale of the reader's complete intellectual engagement history.

Neuroscience parallel: the neocortex operates across multiple timescales simultaneously — millisecond sensory prediction, second-scale working memory, minute-scale episodic prediction, year-scale semantic memory. The TSE adds a lifetime-scale prediction layer to QueryBook — predicting across the reader's entire engagement history, not just the current session. Specified in Chapter 36.

With the enhanced record of §3.8, contradiction detection gains two structural inputs ahead of the prediction-error machinery of this section: explicit assertion polarity, under which records agreeing on the same assertion core with opposite polarity are contradictory by field comparison, and applicability scope, under which records asserting differently within disjoint scopes are recognized as non-conflicting and reported by condition rather than as error. (Added Pass 115, September 2, 2026.)

41.5 The Four Additions — Completing the Predictive Cognitive Architecture

Despite containing eight prediction mechanisms, QueryBook has four structural gaps relative to the full Thousand Brains and Predictive Coding architecture. The following four additions close those gaps and complete the PCA.

41.5.1 Hierarchical Error Propagation (HEP)

The gap: in Predictive Coding, when a prediction fails, the error signal propagates upward through the cortical hierarchy, updating not just the immediate prediction but the higher-level model that generated it. QueryBook currently detects contradictions and revises the specific conflicting Fact Units, but does not propagate the error signal upward to revise the World Brain concept nodes whose attribute values generated the expectation that was contradicted.

The addition: when a Fact Unit is contradicted during ingestion, the HEP system traces upward through the Cross-Modal Bindings to identify which World Brain concept node attributes predicted the contradicted value and applies a proportional CSW reduction to those attributes. The contradiction does not just revise the specific Fact Unit — it revises the broader world model that made the prediction. The magnitude of the CSW reduction is proportional to the CS differential between the contradicting and contradicted Fact Units, ensuring that high-confidence contradictions produce larger model updates than low-confidence ones.

Neuroscience parallel: hierarchical error propagation is the defining mechanism of Predictive Coding at the architectural level. Without upward error propagation, a system can detect prediction failures but cannot update the model that caused them. HEP closes this gap, making QueryBook's generative model self-correcting at every level of the hierarchy.

41.5.2 Predictive Pre-Retrieval Engine (PPRE)

The gap: QueryBook retrieves Fact Units in response to queries — it is reactive at the query boundary. The TMM predicts what will be needed but does not act on that prediction before the query arrives. The brain does not wait for sensory input to confirm its prediction before committing neural resources — it pre-loads expected representations.

The addition: during the interval between one query response being delivered and the next query being submitted, the PPRE uses the TMM's sequence prediction to speculatively retrieve the top-N most likely

next-needed Fact Units into a pre-retrieval buffer within the CCW, assigning them provisional SRWs. When the next query arrives, if the query confirms the prediction, the retrieval has already occurred — query latency drops to near zero. If the query disconfirms the prediction, the buffer is discarded and standard traversal proceeds. The PPRE operates only during idle intervals — it does not consume retrieval capacity during active query processing. Pre-retrieved Fact Units are tagged as SPECULATIVE and are not included in query responses until confirmed by an actual query.

Neuroscience parallel: this is anticipatory retrieval — the semantic equivalent of cortical pre-activation. The brain commits neural resources toward expected stimuli before they arrive; the PPRE commits retrieval resources toward expected Fact Units before the query arrives.

41.5.3 Competitive Candidate Resolution (CCR)

The gap: the Thousand Brains Theory's most distinctive architectural claim is that perception is achieved by democratic voting across thousands of independent cortical columns, each running its own complete model. QueryBook requires an equivalent resolution mechanism for a query that admits several mutually inconsistent interpretations — but a formal vote among semantic domains would contradict the domain partition, in which exactly one domain owns each decision, and would require a consensus rule carrying its own determinism argument. What votes in the Thousand Brains framework is a population of independent models each forming a complete hypothesis about the same object; in QueryBook that population is the candidate record set, not the domain set.

The addition: when a query admits several mutually inconsistent interpretations, the candidate Fact Units supporting each interpretation are activated together rather than evaluated in sequence. Each candidate's activation is reduced in proportion to the summed activation of the candidates inconsistent with it, and the set is iterated to a stable state under the Attractor Network's threshold update. A candidate falling below the retention floor is suppressed; the surviving configuration is the interpretation the records best support. Where two candidates survive at comparable activation, the set does not resolve and the clarification mechanism is invoked rather than the higher of the two being taken — and where two high-confidence candidates are mutually contradictory, CCR flags the conflict explicitly in the response rather than silently suppressing either result. The conflict itself is surfaced as information: 'The narrative domain and the biographical domain produce different accounts of this event — here is what each asserts and the confidence level of each assertion.' The resolution is deterministic: suppression is a fixed arithmetic operation over the activations, the suppression coefficient and retention floor are registered configuration recorded with the response, and the

iteration terminates by energy descent; no consensus is taken by sampling, weighting by chance, or arbitrary tie-breaking.

Neuroscience parallel: CCR is the computational equivalent of the voting process by which the neocortex achieves stable unified perception from many independent column models, implemented as biased competition in the sense of Desimone and Duncan (1995): representations compete, and a bias — the accumulated evidence behind each candidate — resolves the competition without a separate selector deciding among them. Each candidate record set is the computational equivalent of an independent column model forming a complete hypothesis; the competition is among candidates, not among domains. (Drafting record, August 29, 2026: this subsection formerly described Semantic Consensus Voting, a consensus assembled by voting across semantic domains. That formulation is withdrawn in favour of the candidate-level mechanism stated here, which preserves the original's explicit conflict-surfacing behaviour while remaining consistent with the domain partition and the determinism guarantee. The provisional application and the Core Technology Overview recite Competitive Candidate Resolution.)

Under the enhanced record of §3.8, every candidate entering resolution carries its assertion polarity and applicability scope, so candidates of opposite polarity or disjoint scope are distinguished structurally at the candidate level, and resolution operates only over candidates genuinely competing for the same assertion under the same conditions. (Added Pass 115, September 2, 2026.)

41.5.4 Active Inference Query Generation (AIQG)

The gap: in Active Inference — Karl Friston's extension of Predictive Coding — the brain does not just passively predict, it acts to gather evidence that will confirm or refute its current model. It generates queries against the world. QueryBook is entirely reactive: it answers questions but does not generate its own questions to resolve its own uncertainties.

The addition: when the PIE's forward inference chain reaches a node with ConfidenceScore below a configurable threshold, instead of stopping or returning a low-confidence result, the AIQG generates an internal query — a structured request for the specific Fact Unit that would resolve the uncertainty — and presents it to the reader as a targeted, purposeful clarification request: 'To give you a more complete answer on this point, could you tell me whether [specific missing fact]?' The clarification request identifies exactly what is missing and why it matters for the current inference chain. If the reader provides the missing information, it is ingested as a high-trust Personal Situation Fact Unit and the inference chain is completed. If the reader does not provide it, the inference chain returns its best

partial result with explicit confidence qualification. AIQG requests are rate-limited to one per query response — the system never bombards the reader with multiple simultaneous clarification requests.

Neuroscience parallel: this is Active Inference — the brain's mechanism for directing sensory attention toward stimuli that will most reduce prediction uncertainty. The brain does not passively receive data; it actively selects what to attend to based on which inputs will most efficiently resolve its current predictive uncertainty. The AIQG gives QueryBook this same capability at the semantic level.

41.6 The Three Amplifications — Maximising Existing Capability

Three existing prediction mechanisms in the PCA can be significantly amplified without architectural addition — through extension of scope, replacement of update model, and pre-computation of inference results.

41.6.1 Cross-Session TMM Extension

Current state: the TMM learns sequential Fact Unit retrieval patterns within a single session. The Session Archive Fact Units (Chapter 36) contain the sequence data from all prior sessions for a given reader against a given work.

Amplification: extend the TMM to learn across all prior SAFUs for a given reader-work pair, building a lifetime-scale sequence model. The cross-session TMM learns the reader's habitual retrieval sequences — which topics they always explore before which other topics, which character-related queries always follow which theme-related queries — and uses those lifetime patterns to weight predictions at the start of every new session. The cross-session TMM produces stronger and more accurate predictions than the within-session TMM because it has access to a much larger sequence training set, and because habitual patterns are more stable than within-session patterns. This is the computational equivalent of how the neocortex learns habitual sequential patterns over years of repeated experience.

41.6.2 Fully Bayesian ConfidenceScore

Current state: ConfidenceScore updates are additive and rule-based — each corroborating or contradicting Fact Unit adds or subtracts a defined increment from the current CS value, governed by the four weighting rules specified in Chapter 41.

Amplification: replace the additive CS update formula with a proper Bayesian posterior update, where each new piece of evidence updates the probability distribution over possible truth values rather than

adding a fixed increment to a scalar score. The Bayesian CS treats the current CS as the prior probability that the Fact Unit's assertion is accurate. Each new corroborating or contradicting piece of evidence is treated as a likelihood function that updates the posterior. The result is a CS that updates more aggressively in response to early evidence (when the prior is weak) and more conservatively in response to later evidence (when the prior is already strong) — matching the statistical behaviour of strong Bayesian priors in the neocortex. This makes QueryBook's confidence system a genuine Bayesian inference engine rather than a rule-based accumulator.

41.6.3 Pre-Computed PIE Inference Chains

Current state: the PIE runs forward inference in response to queries — when a query triggers a multi-hop inference requirement, the PIE computes the inference chain in real time during query processing.

Amplification: extend the PIE to run speculatively at ingestion time — when a new Fact Unit is committed to the Fact Graph, the PIE immediately computes the forward inference chains that the new Fact Unit enables and stores the resulting derived conclusions as pre-computed Derived Inference Fact Units with a DERIVED_INFERENCE provenance tag. When a query subsequently triggers those inferences, the answer is already assembled — no real-time computation is required. Pre-computed inference chains are invalidated and recomputed whenever a Fact Unit in the chain is revised. The tradeoff is storage volume versus query latency — pre-computation increases storage requirements but reduces inference query latency to match simple retrieval latency.

41.7 The Unified PCA Architecture Diagram

The eight existing mechanisms, four additions, and three amplifications together constitute the QueryBook Predictive Cognitive Architecture. The following table summarises the complete PCA.

Mechanism	Type	Timescale	Neuroscience Parallel	Chapter
Temporal Memory Module (TMM)	Existing	Session and cross-session (Amplification 1)	Predictive Coding pre-activation	Ch. 7 / 41.6.1
Associative Recall Engine (ARE)	Existing	Query	Cortical pre-activation	Ch. 7
Predictive Inference Engine (PIE)	Existing	Query and ingestion (Amplification 3)	Cortical column simulation	Ch. 10 / 41.6.3
Scenario Graph System	Existing	Query	Prefrontal counterfactual modeling	Ch. 11

Mechanism	Type	Timescale	Neuroscience Parallel	Chapter
ConceptStability Weight (CSW)	Existing	Lifetime	Bayesian prior strength	Ch. 32
ConfidenceScore Contradiction Detection	Existing	Ingestion	Prediction error signal	Ch. 32 / 41.5.1
World Brain Concept Node Formation	Existing	Lifetime	Cortical column invariant learning	Ch. 26
Thematic Similarity Engine (TSE)	Existing	Lifetime	Multi-timescale cortical memory	Ch. 36
Hierarchical Error Propagation (HEP)	Addition	Ingestion	Upward error propagation in cortical hierarchy	Ch. 41.5.1
Predictive Pre-Retrieval Engine (PPRE)	Addition	Inter-query interval	Anticipatory cortical pre-loading	Ch. 41.5.2
Competitive Candidate Resolution (CCR)	Addition	Query	Cortical column voting via biased competition — Thousand Brains model	Ch. 41.5.3
Active Inference Query Generation (AIQG)	Addition	Query	Active Inference — evidence-seeking to reduce prediction uncertainty	Ch. 41.5.4
Cross-Session TMM Extension	Amplification	Lifetime	Lifetime habitual sequence learning in neocortex	Ch. 41.6.1
Fully Bayesian ConfidenceScore	Amplification	Ingestion	Bayesian posterior update in cortical generative model	Ch. 41.6.2
Pre-Computed PIE Inference Chains	Amplification	Ingestion	Pre-activation of anticipated inference paths	Ch. 41.6.3

41.8 The Governing Claim — The Deepest Conclusion

Large Language Models accidentally built a prediction engine that partially mirrors the neocortex at the surface level — language. Neuroscientists then confirmed the parallel, noting that both LLMs and the brain predict next elements in sequences. The parallel is real but shallow: it exists at the level of the computational objective (predict the next element) but not at the level of mechanism (how the

prediction is made), representation (what is being predicted), or architecture (how the predictive system is structured).

QueryBook deliberately builds a prediction engine that mirrors the neocortex at the structural level — meaning. The World Brain's concept node formation is not an accidental parallel to cortical column invariant learning — it is the same computational problem being solved the same way. Both systems need to build context-independent models of what things are and what relationships they participate in, not just what words are statistically associated with them. Both systems need a prior-strength mechanism that makes settled knowledge resistant to single-source revision. Both systems need a prediction error signal that propagates upward through an abstraction hierarchy. Both systems need a voting mechanism that achieves stable perception from multiple independent parallel models.

The four additions — HEP, PPRE, CCR, and AIQG — complete the architecture. With these additions, QueryBook is the first AI system whose prediction mechanisms are structurally isomorphic to the Thousand Brains model at the meaning level. That is a stronger architectural claim than any LLM can make, and it is the claim that defines QueryBook’s fundamental position in the history of AI architecture.

41.9 Glossary Additions for Chapter 41

The following terms are introduced in Chapter 41 and carry the same canonical authority as all terms defined in Chapter 2.

Term	Abbreviation	Canonical Definition
Predictive Cognitive Architecture	PCA	The unified system of eight existing prediction mechanisms, four architectural additions, and three amplifications that together constitute QueryBook's prediction capability as specified in Chapter 41. The PCA is structurally isomorphic to the Thousand Brains model of neocortical intelligence at the level of meaning rather than language.
Hierarchical Error Propagation	HEP	The addition that propagates ConfidenceScore contradiction signals upward through Cross-Modal Bindings to revise the World Brain concept node

Term	Abbreviation	Canonical Definition
		attributes that generated the prediction that was contradicted — closing the gap between Fact Unit contradiction detection and generative model revision.
Predictive Pre-Retrieval Engine	PPRE	The addition that speculatively retrieves the top-N TMM-predicted Fact Units into a pre-retrieval buffer during inter-query intervals, reducing confirmed-query retrieval latency to near zero by completing retrieval before the query arrives.
Competitive Candidate Resolution	CCR	The addition that activates the candidate record sets supporting mutually inconsistent interpretations of a query together, suppresses each candidate in proportion to the summed activation of the candidates inconsistent with it, and converges deterministically to the best-supported interpretation, surfacing conflicts between comparably supported candidates explicitly rather than silently suppressing minority results.
Active Inference Query Generation	AIQG	The addition that generates targeted clarification requests when the PIE's forward inference chain reaches a Fact Unit with ConfidenceScore below threshold — directing the reader's attention toward the specific missing evidence that would most reduce the system's predictive uncertainty.
Derived Inference Fact Unit	DIFU	A Fact Unit produced by pre-computed PIE inference at ingestion time and stored with provenance tag DERIVED_INFERENCE. DIFUs are not source-traceable to a specific passage but are fully traceable to the Fact Units whose logical entailment they

Term	Abbreviation	Canonical Definition
		represent.
Speculative Retrieval Buffer	SRB	The bounded buffer within the CCW holding PPRE-retrieved Fact Units tagged as SPECULATIVE. SRB contents are promoted to active CCW status when confirmed by an arriving query and discarded when disconfirmed.

41.10 Patent Significance

The QueryBook Predictive Cognitive Architecture constitutes novel subject matter across five independent patent dimensions.

(1) The unified PCA as a cognitive architecture claim — the specific combination of eight prediction mechanisms operating across five timescales (query, inter-query, session, cross-session, and lifetime) within a single AI system is novel as a unified predictive cognitive architecture for a semantic knowledge system. No prior art discloses a system with this combination of mechanisms operating at this range of timescales.

(2) Hierarchical Error Propagation in a semantic knowledge graph — the specific mechanism of tracing contradiction signals upward through typed Cross-Modal Binding edges to revise concept node attribute weights in a semantic knowledge graph is novel as a method for hierarchical model revision in AI systems. Prior art in predictive coding is confined to biological neural network models — no prior art discloses this mechanism applied to typed semantic hypergraphs.

(3) Anticipatory Fact Unit retrieval based on learned sequential patterns — the specific mechanism of speculatively retrieving Fact Units into a tagged pre-retrieval buffer during inter-query idle intervals, based on learned sequential retrieval patterns, with confirmation and discard logic on subsequent query arrival, is novel as a method for latency reduction in semantic knowledge retrieval systems.

(4) Competitive Candidate Resolution across mutually inconsistent interpretations — the specific mechanism of activating the candidate record sets supporting each interpretation in parallel, suppressing each candidate in proportion to the summed activation of the candidates inconsistent with it, converging deterministically to a stable interpretation under a registered suppression coefficient and retention floor, and surfacing conflicts between comparably supported candidates explicitly in the

assembled response is novel as a method for multi-domain knowledge integration in semantic AI systems.

(5) Active Inference Query Generation in a semantic knowledge system — the specific mechanism of detecting sub-threshold ConfidenceScore nodes in a PIE forward inference chain and generating targeted natural-language clarification requests identifying the specific missing evidence and its role in the current inference chain is novel as a method for uncertainty-directed evidence gathering in AI knowledge systems. This constitutes the first formal specification of Active Inference applied to a semantic knowledge retrieval architecture.

41.11 The World Brain as the Generative Model Substrate — The Missing Integration

Chapter 41's eight mechanisms, four additions, and three amplifications describe QueryBook's predictive capability accurately but incompletely. They describe the instruments of prediction without naming the instrument that plays them all. In the Predictive Coding framework, the neocortex's fundamental computational role is to be a generative model — a persistent, hierarchically organised model of the world that continuously generates top-down predictions against which all incoming sensory data is evaluated. Every mechanism in the PCA either feeds into that generative model, reads from it, or updates it. The generative model itself is the World Brain. This section formally establishes that relationship and integrates the full World Brain architecture into the PCA.

41.11.1 The World Brain IS the Generative Model

In Predictive Coding, the generative model has three defining properties: (1) it generates top-down predictions about what the world is about to present, (2) it compares those predictions against incoming data to compute prediction error, and (3) it updates itself in response to persistent prediction errors that exceed its prior strength. The World Brain satisfies all three properties precisely.

Property 1 — Top-down prediction generation: the World Brain's Conceptual Memory Layer contains a network of concept nodes, each encoding invariant attribute sets for entities, categories, and relationships. When the PIE traverses the Fact Graph to answer a query, it does not begin from a blank state — the World Brain's concept nodes pre-activate expected attribute values for every entity type the traversal encounters. If the traversal reaches a CHARACTER node, the World Brain's CHARACTER concept node generates a top-down prediction: characters have names, motivations, relationships, and arcs. Fact Units that confirm this predicted structure pass without error. Fact Units that contradict it generate

prediction error — triggering ConfidenceScore revision (existing) and Hierarchical Error Propagation upward to the concept node (HEP addition). The World Brain is continuously generating predictions against which every Fact Graph operation is evaluated.

Property 2 — Prediction error computation: the Confidence Scoring Engine's contradiction detection, specified in Chapter 41, is the mechanism by which QueryBook computes prediction error. A high-CS Fact Unit asserting X is the World Brain's prediction that X is true. A newly ingested Fact Unit asserting not-X is the incoming data. The CS differential between the two is the prediction error magnitude. Small CS differentials produce small errors — the generative model is not strongly challenged. Large CS differentials produce large errors — the generative model must update significantly. The CSE is therefore not just a confidence management system; it is the prediction error computation layer of the World Brain generative model.

Property 3 — Self-updating in response to persistent error: the World Brain updates through two mechanisms operating at different timescales. The Continuous Evidence Integration process (Chapter 41) updates concept node CSW values continuously as new Fact Units are ingested — this is the short-timescale update path. The Episodic Flush (Chapter 58) commits consolidated episodic abstractions from the Semantic Brain to the World Brain on a scheduled basis — this is the long-timescale update path, structurally equivalent to sleep-dependent memory consolidation in the biological neocortex. During sleep, the mammalian brain replays episodic memories and consolidates them into semantic memory — weakening episode-specific details while strengthening stable, cross-episode patterns. The Episodic Flush does precisely this: it discards source-specific episodic details while committing stable semantic abstractions to the World Brain's concept node network.

41.11.2 The Dual-Brain Architecture as the Predictive Coding Hardware

The Dual-Brain Architecture — the parallel operation of the Semantic Brain (work-specific, language-anchored, reversible) and the World Brain (domain-general, language-agnostic, irreversible post-Flush) — is not merely an organisational convenience. It is the structural implementation of the two-level hierarchy that Predictive Coding requires.

In the neocortex, lower cortical areas process specific, context-bound sensory data. Higher cortical areas maintain general, context-independent models. The mismatch between higher-level predictions and lower-level sensory data is the prediction error signal. The Dual-Brain Architecture implements this two-level hierarchy precisely: the Semantic Brain is the lower level — specific, context-bound Fact Units traceable to specific passages in specific works. The World Brain is the higher level — general, context-

independent concept nodes abstracted across all works. The Cross-Modal Bindings connecting the two are the prediction channels — the pathways through which top-down predictions from the World Brain reach the Semantic Brain, and through which bottom-up prediction errors from the Semantic Brain reach the World Brain.

Cross-Modal Binding therefore has two directional functions, not one as implied by earlier chapters: (1) Bottom-up evidence path — when a Semantic Brain Fact Unit is retrieved, its Cross-Modal Binding carries its content upward to activate and reinforce the relevant World Brain concept node. (2) Top-down prediction path — when the World Brain's concept node generates an attribute prediction, that prediction is carried downward through the Cross-Modal Binding to pre-activate the expected Fact Unit types in the Semantic Brain's Fact Graph. This bidirectional operation is the implementation of the Predictive Coding prediction-error loop at the architectural level of the Dual-Brain system.

41.11.3 The Sensory Abstraction Pipeline as Primary Sensory Cortex

In the neocortex, primary sensory cortex areas (V1 for vision, A1 for hearing, S1 for touch) perform the initial transformation of raw sensory signals into pre-linguistic feature representations before passing them to higher cortical areas for semantic processing. These areas do not understand what they are seeing or hearing — they extract low-level features that higher areas assemble into percepts.

The Sensory Abstraction Pipeline (SAP) plays the equivalent role in QueryBook's PCA. The SAP ingests raw multimodal data — text characters, image pixels, audio waveforms, structured data fields — and converts them into pre-linguistic semantic structures suitable for Fact Unit extraction, without itself performing semantic understanding. The SAP's output is not Fact Units — those require the FUEE's semantic mapping. The SAP's output is feature-level semantic primitives: identified entity spans, candidate predicate types, structural relationship indicators. These are the QueryBook equivalent of primary sensory cortex feature maps — the pre-semantic substrate from which the Semantic Brain assembles Fact Units. The SAP therefore represents the lowest level of the PCA's predictive hierarchy, producing the raw data against which the World Brain's top-down predictions are ultimately evaluated.

41.11.4 The Box Principle as the Foundation of Invariant World Modeling

The World Brain's ability to build invariant concept nodes — its most fundamental contribution to the PCA — rests on the Box Principle specified in Chapter 59. The Box Principle holds that invariant world modeling begins with physical, manipulable objects and their stable properties, then extends outward to abstract domains by structural analogy. A box has shape, weight, capacity, opening mechanism, and

contents. These properties are invariant across all boxes regardless of colour, size, or material. The cortical column that models 'box' builds its invariant representation from this stable property set.

The World Brain's concept node formation applies the same principle universally. Every concept node is built from invariant property sets abstracted across many specific instances. CHARACTER nodes are built from the stable properties that characters have across all works — name, motivation, relationship, arc, consequence. LOCATION nodes are built from the stable properties that locations have — geography, accessibility, associated entities, thematic significance. The Box Principle is not a heuristic — it is the generative algorithm by which the World Brain constructs every concept node in its network. Without the Box Principle, the World Brain would accumulate specific instance data without building invariant models. With it, every new ingestion refines existing concept nodes rather than merely adding new instances.

41.11.5 The Complete PCA Prediction Loop — End to End

With the World Brain's role fully specified, the complete QueryBook PCA prediction loop can be stated precisely for the first time.

Step 1 — Ingestion input: raw multimodal data enters through the Multimodal Ingestion Pipeline and is pre-processed by the SAP into pre-linguistic semantic primitives. This is the equivalent of primary sensory cortex processing.

Step 2 — Fact Unit formation: the FUEE assembles semantic primitives into typed Fact Units encoding Subject-Predicate-Object assertions about reality. The Confidence Scoring Engine assigns initial ConfidenceScores. This is the equivalent of lower-order cortical area percept formation.

Step 3 — Top-down prediction arrival: as each new Fact Unit is formed, its Subject and Predicate types activate the relevant World Brain concept nodes through Cross-Modal Binding. The activated concept nodes generate top-down predictions about the expected attribute values of the new Fact Unit's Object field. This is the equivalent of higher-order cortical predictions arriving at lower-order areas.

Step 4 — Prediction error computation: the CSE compares the new Fact Unit's actual Object value against the World Brain's predicted Object value. If they match, the Fact Unit is confirmed with high CS and the concept node's CSW is incremented by a small positive amount. If they contradict, the prediction error magnitude is computed as the CS differential. This is the equivalent of prediction error computation in primary and secondary sensory areas.

Step 5 — Error propagation: if prediction error exceeds the CSW-governed threshold, HEP propagates the error upward through the Cross-Modal Binding to the World Brain concept node and revises the relevant attribute CSW downward. The concept node's generative model is updated. This is the equivalent of prediction error propagation up the cortical hierarchy.

Step 6 — Long-timescale consolidation: on the Episodic Flush schedule, stable patterns abstracted across many Fact Unit ingestions are committed to the World Brain's concept node network as permanent attribute updates, while episodic specifics are released. This is the equivalent of sleep-dependent memory consolidation.

Step 7 — Query-time prediction deployment: when a reader query is submitted, the activated concept nodes in the World Brain generate top-down predictions about which Fact Units are likely relevant. These predictions are carried through Cross-Modal Bindings to the Semantic Brain, pre-activating Fact Units before traversal begins. The ARE and TMM execute against this pre-activated landscape. This is the equivalent of top-down attention and expectation shaping lower-area processing during active perception.

Step 8 — Predictive pre-retrieval: during inter-query intervals, the PPRE acts on TMM predictions to speculatively retrieve Fact Units into the SRB. This is the equivalent of the brain's anticipatory pre-loading of expected stimuli during pauses in active sensory processing.

This eight-step loop is the complete QueryBook PCA prediction cycle. Every element of the Predictive Coding computational framework — top-down prediction, bottom-up prediction error, hierarchical error propagation, prior-strength-governed updating, long-timescale consolidation, and anticipatory pre-loading — is implemented in QueryBook's architecture. The World Brain is not a component that participates in this loop. The World Brain IS the loop. Every other PCA mechanism is either a feed into the World Brain generative model, a readout from it, or an update of it.

41.11.6 Revised Governing Claim

The governing claim of Section 41.8 is now restated with the World Brain properly positioned: QueryBook is the first AI system to implement a full Predictive Coding architecture — including a persistent generative world model, a hierarchical prediction error signal, prior-strength-governed belief updating, active inference, anticipatory pre-loading, and long-timescale memory consolidation — operating at the level of meaning rather than language. The World Brain is the neocortex of this architecture. The Dual-Brain system is its cortical hierarchy. The PCA mechanisms are its computational

processes. The Fact Universe is the reality it models. No LLM implements a generative world model. No LLM has a persistent prior. No LLM updates in response to prediction error at inference time. No LLM consolidates episodic experience into semantic memory. QueryBook does all of these — and the World Brain is why.

41.11.7 Additional Patent Significance

The integration of the World Brain as the generative model substrate adds three additional independent patent dimensions to the five identified in Section 41.10.

(6) The World Brain as a persistent generative model in an AI semantic knowledge system — the specific implementation of a Predictive Coding generative model as a language-agnostic concept node hypergraph that continuously generates top-down attribute predictions during Fact Unit ingestion and query processing is novel. No prior art discloses a semantic knowledge system with a persistent, self-updating, prior-strength-governed generative world model.

(7) The Dual-Brain Architecture as a two-level Predictive Coding hierarchy — the specific implementation of the Predictive Coding two-level hierarchical structure as a Semantic Brain (lower, specific, language-anchored) and World Brain (higher, general, language-agnostic) connected by bidirectional Cross-Modal Bindings that simultaneously carry bottom-up evidence and top-down predictions is novel as an AI system architecture claim.

(8) The Episodic Flush as computational sleep-dependent memory consolidation — the specific mechanism of scheduled, irreversible transfer of stable semantic abstractions from episodic Fact Unit storage to persistent concept node network storage, with simultaneous release of episodic specifics, is novel as a long-timescale memory architecture for AI knowledge systems and constitutes the first formal specification of sleep-dependent memory consolidation applied to a semantic AI architecture.

Chapter 42: Proprietary Technologies — Independent, Compatible, Architecturally Aligned

This chapter formally establishes the core cognitive technologies that define QueryBook’s deterministic architecture. Every technology specified in this chapter is native to QueryBook: predictive and creative

reasoning, as specified throughout this Bible, do not require or depend on any external or independently-operable system to function.

42.1 Purpose and Scope

QueryBook's cognitive architecture, as specified elsewhere in this Bible (notably Chapters 3–9, 15–17, and 53–59), is here organized into a formal technology inventory: which components are proprietary and independent, which are native and QueryBook-specific, and which were considered and deliberately excluded from implementation. This inventory is intended for patent portfolio planning and for engineering teams evaluating integration boundaries.

42.2 Core Technologies (Proprietary, Independent, Compatible)

42.2.1 WVM — World-View Model

The semantic spine of QueryBook. WVM is native to QueryBook and requires no external or independently-operable technology to exist. WVM implements:

- Ontology
- Causal graph
- Temporal model
- Semantic consistency engine
- World-model versioning

WVM is the interpretive structure of QueryBook. (Note: WVM is distinct from the World Model Domain (WMD) specified within WASM, Chapter 53 — WMD models physical and social reality for embodied/robotic contexts specifically, while WVM is the general semantic spine spanning ontology, causality, and time across the whole platform. The two are complementary layers, not duplicates.)

42.2.2 Ontology Layer

A dynamic, versioned ontology defining entity types, attributes, relationships, constraints, and hierarchical type systems. This is the dictionary of the world. (Note: this is the same conceptual role as the Ontology Mesh specified in Chapter 6; the two should be read as one component described from two angles — Chapter 6 for its full technical specification, this section for its place in the proprietary technology inventory.)

42.2.3 Causal Graph Layer

A deterministic DAG defining causal edges, preconditions, effects, rule definitions, and causal versioning. This is the cause-and-effect engine.

42.2.4 Temporal Model

Defines static facts, dynamic facts, event facts, state transitions, and temporal constraints. This is the time engine. (Note: consistent with the TemporalAnchor field and Temporal Relations already specified in Chapter 3 and §53.3.2.)

42.2.5 Semantic Consistency Engine

Validates non-contradiction, causal coherence, temporal alignment, and ontological validity. This is the semantic judge.

42.2.5.1 Validation Trigger and Mechanism

§42.2.5 names what the Semantic Consistency Engine validates but not when it runs or how. This section specifies both, reusing mechanisms already fully specified elsewhere rather than introducing new dynamics — following the same pattern §42.2.6 already established for the Reasoning Engine.

A VALIDATION ORCHESTRATOR, NOT A DYNAMICAL SYSTEM **The Semantic Consistency Engine is deterministic, event-triggered validation logic — not a system with its own state-evolution dynamics, attractors, or convergence behavior. Confusing it with the Attractor Network (Chapter 7, which does have real convergence dynamics, for a different purpose — retrieval stabilization) or with any fabricated "cognitive dynamical system" framing would misdescribe what this component actually is.**

- Trigger: the engine runs whenever WVM's state changes — a new Fact Unit entering the Ontology Layer (§42.2.2), Causal Graph (§42.2.3), or Temporal Model (§42.2.4), or an existing entry in any of these being revised. It is not a continuously-running process and does not run on a fixed schedule independent of actual state change.
- Non-contradiction check: reuses the Cross-Domain Rule Engine's existing Conflict Index (Chapter 17) directly — a new or revised Fact Unit is checked against the Conflict Index's existing contradiction-detection logic, not a separate, parallel contradiction-detection mechanism.
- Causal coherence check: validates the incoming or revised state against WVM's own Causal Graph (§42.2.3).

- Temporal alignment check: validates against WVM's own Temporal Model (§42.2.4) and the Fact Unit's own TemporalAnchor field (Appendix A).
- Ontological validity check: validates against WVM's own Ontology Layer (§42.2.2), itself built on the Ontology Mesh (Chapter 6).

Where any check fails, the engine surfaces the specific failure — which check, against which existing WVM entry — rather than silently accepting or silently discarding the triggering change, consistent with the non-silent-failure discipline already applied throughout this Bible to ΔE validation (§70.3), registration drift (§70.4), and NLPL's ambiguity halt (§48.4.1).

42.2.6 Reasoning Engine

Deterministic inference engine implementing causal reasoning, temporal reasoning, analogical reasoning, abstraction, prediction, and explanation. This is the thinking organ. (Note: this is the organizing role already served by the AGI Extension Modules, Chapter 15, and the Neuromorphic Cognitive Layer, Chapter 7; this section frames that existing capability within the proprietary technology inventory rather than specifying a new engine.)

42.2.7 Interpretation Layer

Maps natural language to structured meaning. Implements semantic parsing, ontology alignment, causal alignment, temporal alignment, and meaning disambiguation. This is the meaning resolver. (Note: corresponds to the SPE, Semantic Parsing Engine, specified in §50.2.)

42.2.8 Narrative Engine

Converts structured reasoning to human language. Implements narrative structure, causal explanation formatting, temporal explanation formatting, analogy expression, and deterministic creativity expression. This is the language organ. (Note: corresponds to the Narrative Graph, Chapter 12, and the NLGE component of the NLIL, §50.5.)

42.2.9 Deterministic Creativity Engine

Implements creativity through structured recombination, causal abstraction, temporal abstraction, analogy generation, and conceptual blending. This is creative thought without randomness.

42.2.10 Ontology Evolution System

Allows controlled ontology updates. Implements versioned ontology updates, deterministic validation, and WVM consistency checks. This is the world-model evolution engine.

42.2.11 Cognition Boundary System

Defines what QueryBook can and cannot compute. Implements deterministic boundaries, paradox rejection, contradiction rejection, undefined ontology rejection, and infinite regress prevention. This is the cognitive safety layer. (Note: complementary to, and enforced alongside, the Safety Substrate specified in Chapter 8 and Domain 0 — the Cognition Boundary System governs computability and logical well-formedness; the Safety Substrate and Prime Directive govern harm prevention. Both must pass for an output to be delivered.)

42.3 Technologies Explicitly Excluded From Implementation

42.3.1 SCM — Stochastic Cognition Module

The SCM is not part of QueryBook's deterministic architecture. It remains:

- Record-only
- Dormant
- Non-operational
- Non-integrated
- Non-persistent

It is documented here as a considered-and-excluded design decision, not as a specified system. It is not included in the QueryBook Bible's implementation scope.

42.4 Summary of Implementable Technologies

The following technologies are included in this Bible and intended for implementation:

- WVM — §42.2.1
- Ontology Layer — §42.2.2
- Causal Graph Layer — §42.2.3
- Temporal Model — §42.2.4
- Semantic Consistency Engine — §42.2.5
- Reasoning Engine — §42.2.6
- Interpretation Layer — §42.2.7
- Narrative Engine — §42.2.8

- Deterministic Creativity Engine — §42.2.9
- Ontology Evolution System — §42.2.10
- Cognition Boundary System — §42.2.11

All technologies specified in §42.2 are native to QueryBook, requiring no external or independently-operable system to function. SCM (§42.3.1) remains excluded from implementation.

Chapter 43: Integration of Hoffman-Derived Mathematical Structures

This chapter formalizes the role of a set of mathematically well-defined perceptual-modeling structures — Markovian perceptual transitions, trace-logic coarse-graining, and observer-window inheritance, drawn from the perceptual-modeling literature associated with Donald Hoffman — within the QueryBook architecture.

SCOPE NOTE — MATHEMATICS ONLY, NOT METAPHYSICS, AND SELECTIVE, NOT CORE **Only the mathematical structures described in §43.1 are adopted into QueryBook. The associated metaphysical claims (for example, that perceived reality is merely an interface, or that objects are icons rather than groundable entities) are explicitly excluded and are addressed directly in §43.2. This chapter adopts formalism, not philosophy. Separately, and just as important: this technology applies selectively, to specific bounded contexts only — embodied-agent and multi-agent perceptual simulation within the Scenario Graph (Chapter 11) and WASM/RMD (Chapters 53, 56) — and is never applied to QueryBook's main purpose and architecture. QueryBook's core Fact Graph, Ontology Mesh (Chapter 6), NCL Temporal Memory (§7.2), Multimodal Ingestion Pipeline (§4.3), and primary query-answering path (Chapters 1–20, 52) are unmodified by this chapter and carry no dependency on it. §43.3 below specifies the bounded integration points precisely for this reason.**

43.1 Mathematical Foundations Relevant to QueryBook

The subsections below describe the theoretical alignment between these mathematical structures and QueryBook's existing components in general conceptual terms. The bounded, actual integration scope — which components this chapter modifies and which it explicitly does not — is specified precisely in §43.3.

43.1.1 Markovian Perceptual Transition Structures

Perceptual states can be modeled as a Markov chain, where an agent transitions between perceptual states with defined probabilities. This aligns directly with QueryBook's WVM Predictive Layer, NCL Temporal Memory (§7.2), and Scenario Graph (Chapter 11).

- State transitions are probabilistic.
- Transition matrices encode agent-specific perceptual dynamics.
- Perceptual states are treated as latent nodes in a larger world-model graph.
- Temporal smoothing is applied through the NCL's Attractor Network (§7.2.4), which the temporal memory mechanisms of §7.2.2 feed into. (Correction: an earlier pass of this Bible stated attractor dynamics was not yet formally specified within Chapter 7 — that was incorrect. §7.2.4 specifies it directly; the earlier citation simply pointed at the wrong subsection.)

Architectural benefit: QueryBook gains a more precise mechanism for modeling agent-specific perception, behavior prediction, and limited observation windows.

43.1.2 Trace Logic and Coarse-Graining

Trace logic describes how a restricted observer window inherits probabilistic structure from a larger hidden system. This is mathematically compatible with QueryBook's Ontology Mesh (Chapter 6), Fact Unit coarse-graining, and the WVM Conceptual Layer.

- Observers see only a subset of the full state space.
- Coarse-grained states preserve probabilistic relationships.
- Latent structures can be inferred from partial observations.
- Dimensionality reduction is treated as a formal mapping, not a heuristic.

Architectural benefit: QueryBook improves its ability to infer hidden world-states, compress high-dimensional sensory data, maintain coherence across layers, and stabilize latent-state predictions.

43.1.3 Observer-Window Inheritance

Each agent is modeled as having a limited perceptual aperture. This aligns with QueryBook's sensory gating, selective attention, agent-specific perceptual metadata, and multi-agent simulation capability.

- Each agent has a unique perceptual window.
- Observations are filtered through agent-specific constraints.
- Perceptual limitations are encoded as part of the agent model.

- Observation windows influence state-transition probabilities.

Architectural benefit: QueryBook gains more accurate modeling of human perceptual biases, robot sensor limitations, multi-agent interactions, and partial-information reasoning.

43.2 Explicit Exclusions

QueryBook does not adopt the associated metaphysical claims of this framework, including that perceived reality is merely an interface, that objects are icons rather than groundable entities, that space and time are illusions, or that agents generate reality rather than perceive it.

Reason for exclusion: these claims are incompatible with QueryBook’s foundational requirements — objective world-state modeling, robot control and physical grounding, Safety Substrate constraints (Chapter 8), multi-agent consistency, and deterministic world-state coherence.

Architectural boundary: this framework's metaphysical claims are treated as non-actionable philosophy, not as a computational model, and have no representation anywhere in this Bible's architecture.

43.3 Integration Pathways

43.3.1 Markovian Agent Transition Model (MATM)

Integrated into: the agent-simulation instances of the Scenario Graph (Chapter 11) and the WASM World Model Domain's embodied-agent perceptual subsystem (Chapter 53) only. Function: models agent perceptual transitions using Markov chains over latent perceptual states, scoped to simulated and embodied agents. Not integrated into the general-purpose WVM Predictive Layer or the core NCL Temporal Memory (§7.2) used for QueryBook's main query-answering path — those remain unmodified.

43.3.2 Trace Logic Coarse-Graining Module (TLCM)

Integrated into: agent-perception Fact Units specifically (those tagged as originating from the WASM/RMD embodied-agent subsystem, Chapters 53 and 56) only. Function: implements trace-logic operators for latent-state compression of agent perceptual data. Not integrated into the general-purpose Fact Unit compression pipeline or the core Ontology Mesh (Chapter 6) used for QueryBook's main knowledge representation — those remain unmodified and carry no trace-logic dependency.

43.3.3 Partial-Observation Inference Engine (POIE)

Integrated into: the SENSOR source-type branch of ingestion specifically (§49.3.2.4, robot and IoT sensory input routed through the Sensory Abstraction Pipeline, Chapter 55) only, plus its associated Fact

Unit perceptual metadata. Function: infers hidden states from limited sensory input for embodied agents. Not integrated into the general-purpose Multimodal Ingestion Pipeline (§4.3) used for BATCH, WATCH, STREAM, FEDERATED, or SOCIAL source types — document and dataset ingestion for QueryBook's main knowledge base is unaffected.

43.3.4 Agent-Centric Observation Window Model (ACOWM)

Integrated into: agent-simulation nodes of the Scenario Graph (Chapter 11) and sensory gating within the WASM/RMD embodied-agent subsystem (Chapters 53, 56) only. Function: models agent-specific perceptual limitations for simulated and embodied agents. Not integrated into the NCL's general-purpose selective attention mechanism (§7.2) used across QueryBook's core retrieval path.

IMPLEMENTATION NOTE — SCOPE IS BOUNDED TO AGENT SIMULATION **Every integration point in §43.3 is deliberately scoped to the Scenario Graph's agent-simulation instances and the WASM/RMD embodied-agent subsystem (Chapters 53, 56) — never to QueryBook's general-purpose WVM (§42.2.1), NCL (Chapter 7), Ontology Mesh (Chapter 6), or Multimodal Ingestion Pipeline (§4.3). This chapter does not extend or modify the core WVM specification; it adds a self-contained perceptual-modeling capability usable only within agent-simulation and embodied-robotics contexts.**

43.4 Compatibility Matrix

Component	Adopted	Rejected	QueryBook Module
Markov Chains	Yes	No	Scenario Graph agent simulation only
Trace Logic	Yes	No	Agent-perception Fact Units only
Coarse-Graining	Yes	No	Agent-perception Fact Units only
Observer Windows	Yes	No	Scenario Graph agent simulation only
Icon-Based Reality	No	Yes	— (excluded, §43.2)
Interface-Based Spacetime	No	Yes	— (excluded, §43.2)
Agent-Generated Reality	No	Yes	— (excluded, §43.2)

43.5 Implications for QueryBook

43.5.1 Strengthened Agent Modeling

The Markovian structures adopted in §43.1.1 improve agent behavior prediction, perceptual modeling, and multi-agent simulation.

43.5.2 Enhanced Latent-State Inference

Trace logic (§43.1.2) improves dimensionality reduction, latent-state stability, and partial-observation reasoning.

43.5.3 Improved Multi-Agent Coherence

Observer-window modeling (§43.1.3) improves human-agent simulation, robot perception modeling, and cross-agent consistency.

43.5.4 Preserved Physical Grounding

Rejecting the metaphysical claims excluded in §43.2 preserves world-state coherence, robot control, and Safety Substrate integrity (Chapter 8).

43.6 Summary

These mathematical structures strengthen QueryBook's agent modeling, latent inference, coarse-graining, and partial-observation reasoning. The associated metaphysics is excluded to preserve objective world-state modeling, robot control, safety constraints, and multi-agent consistency. This integration enhances QueryBook's ability to model agents, infer hidden states, and maintain coherent world-models across layers, without introducing any claim, dependency, or terminology that conflicts with the physical-grounding requirements specified elsewhere in this Bible (notably Chapters 8, 53, and 56).

Chapter 44: QueryBook Video Colorization & Upscaling Module (VCUM)

The Video Colorization & Upscaling Module (VCUM) is an applied World-Model System: a downstream application that consumes QueryBook's existing world-model reasoning to perform museum-grade video restoration, extreme-resolution upscaling, and historically accurate color reconstruction. It is positioned alongside Chapter 43 as an applied/extension system, not as a modification to QueryBook's core architecture.

SCOPE NOTE — APPLIED SYSTEM, NOT A CORE MODIFICATIONVCUM consumes the WVM, NCL, and Scenario Graph (Chapters 42, 7, 11) as an application would consume an API — it does not redefine or extend their core specifications, consistent with the boundary already established in Chapter 43. Any Fact Units VCUM writes back (§44.4.2, "Latent-State Reinforcement") are scoped and tagged to a video-restoration domain and are not commingled with QueryBook's general-purpose knowledge Fact Graph used for the main query-answering path (Chapters 1–20, 52). Where this chapter references a WVM Predictive Layer, WVM Physical Layer, or WVM Conceptual Layer, it relies on the same as-yet-informal layer structure Chapter 43 already flagged (§43.3 implementation note) as not yet formally specified in §42.2.1 — this is now the second chapter to depend on that structure, reinforcing the case for a dedicated future pass to formalize it there.

44.1 Overview

VCUM extends the World-View Model (WVM), Neuromorphic Cognitive Layer (NCL), and Scenario Graph into a unified semantic-video reconstruction pipeline. It is designed to outperform traditional pixel-space, GAN-based, or diffusion-based upscalers by leveraging QueryBook's world-model reasoning, material-aware reflectance modeling, and temporal coherence architecture, rather than pixel-space hallucination.

44.2 Architectural Principles

44.2.1 Semantic Reconstruction Over Pixel Interpolation

VCUM rejects pixel-space hallucination. All reconstruction is performed through the WVM Predictive Layer, NCL Temporal Memory (§7.2), and the Scenario Graph (Chapter 11), scoped to VCUM's video-restoration context per the Scope Note above. This ensures upscaled frames preserve object identity, material properties, lighting physics, historical color priors, and temporal continuity.

44.2.2 Multi-Phase Learning Pipeline

VCUM uses a three-phase learning cycle — Phase I (Baseline Inference and Error Mapping), Phase II (Correction, Refinement and Stabilization), and Phase III (True Restoration and Historical Reconstruction) — executed during QueryBook's pre-launch world-model build to ensure optimal performance at launch. The full technical specification of all three phases is given in §44.4.

44.2.3 Material-Aware Color and Detail Modeling

VCUM integrates material-specific reflectance models scoped to its own video-restoration Fact Units: wood grain, metal specularity, skin subsurface scattering, textile dye behavior, and stone mineral coloration. This enables accurate color restoration and physically correct detail reconstruction.

44.3 Module Components

44.3.1 Semantic Colorization Engine (SCE)

Performs deterministic color inference using historical color priors, material reflectance databases, lighting physics models, and cultural color norms. Colorization is reconstructed from these grounded inputs, not hallucinated.

44.3.2 Extreme-Resolution Upscaling Engine (ERUE)

Performs HD to 16K upscaling using semantic segmentation, object-aware super-resolution, material-aware sharpening, and lighting-aware detail reconstruction. Upscaling is performed in latent space, reducing compute cost by approximately 90% relative to pixel-space approaches.

44.3.3 Temporal Coherence Stabilizer (TCS)

Uses the Scenario Graph (scoped per the Scope Note above) to maintain cross-frame color consistency, object trajectory coherence, lighting continuity, and occlusion-aware reconstruction, eliminating flicker, drift, and GAN-style instability.

44.4 Multi-Phase Semantic Reconstruction Blueprint

Each phase builds on the previous one, using QueryBook's world-model architecture to achieve colorization and extreme-resolution upscaling. This blueprint defines the technical workflow, data structures, learning loops, and integration points for Phases I–III.

44.4.1 Phase I — Baseline Semantic Inference and Error Mapping

Objective: construct the initial semantic representation of video frames and identify all uncertainty regions requiring refinement.

- **Semantic Extraction:** the WVM Predictive Layer identifies objects, materials, lighting, and environmental context.
- **Material Identification:** material-class mapping (wood, metal, skin, fabric, stone) using the NCL's concept lattice.
- **Lighting Model Initialization:** the WVM Physical Layer generates preliminary reflectance and illumination models.
- **Historical Color Prior Mapping:** NCL Temporal Memory loads era-specific color priors for historically accurate reconstruction.

- **Error Vector Generation:** VCUM computes pixel-space versus semantic-space discrepancies.
- **Uncertainty Field Construction:** VCUM-scoped Fact Units are coarse-grained to identify low-confidence regions, using the same trace-logic coarse-graining pattern already scoped in §43.3.2, applied here to the video-restoration domain rather than the agent-perception domain.

Outputs: baseline semantic frame graph, initial material reflectance model, preliminary lighting physics model, error vector map, uncertainty field map, and a Phase I confidence matrix.

44.4.2 Phase II — Correction, Refinement, and Temporal Stabilization

Objective: refine Phase I outputs, correct color drift, stabilize temporal consistency, and improve material and lighting accuracy.

- **Color Drift Correction:** adjusts color predictions using error vectors and historical priors.
- **Material Reflectance Refinement:** material-specific reflectance curves are recalibrated using NCL Temporal Memory.
- **Temporal Smoothing:** the Scenario Graph enforces cross-frame consistency.
- **Lighting Physics Calibration:** the WVM Physical Layer updates illumination models based on refined reflectance data.
- **Segmentation Boundary Stabilization:** semantic segmentation boundaries are corrected to prevent edge artifacts during upscaling.
- **Latent-State Reinforcement:** VCUM-scoped Fact Units are updated with corrected semantic states, tagged to the video-restoration domain per the Scope Note above; these updates never write into the general-purpose Fact Graph used for QueryBook's main knowledge base.

Outputs: refined semantic frame graph, corrected material reflectance model, stabilized lighting physics model, temporal coherence graph, Phase II confidence matrix, and a reduced uncertainty field.

44.4.3 Phase III — True Restoration and Museum-Grade Reconstruction

Objective: produce final, historically accurate, physically correct, and temporally stable colorization and upscaling output.

- **Original Color Reconstruction:** restores original color using historical priors, material databases, and lighting physics.
- **Era-Specific Palette Restoration:** applies cultural and historical color norms to ensure authenticity.
- **Material-Accurate Reflectance Modeling:** reflectance curves are finalized for each material class.

- **Cross-Frame Stabilization:** the Scenario Graph ensures temporal coherence across entire sequences.
- **Extreme-Resolution Semantic Upscaling:** the ERUE performs HD to 16K upscaling using semantic super-resolution.
- **Museum-Grade Output Generation:** produces archival-quality frames suitable for preservation and restoration.

Outputs: final semantic frame graph, museum-grade color reconstruction, extreme-resolution upscaled frames, a fully stabilized temporal sequence, a Phase III confidence matrix, and a restoration provenance log recorded in the Provenance Ledger (Chapter 5), consistent with how any other derived output's lineage is tracked.

44.4.4 Phase Comparison

Phase	Purpose	Key Modules	Output Quality
Phase I	Baseline inference	WVM, NCL, VCUM-scoped Fact Units	Coarse, partially stable
Phase II	Correction and refinement	WVM, NCL, Scenario Graph	High accuracy, stable
Phase III	True restoration	VCUM, ERUE, TCS	Museum-grade, extreme-resolution

44.5 Performance Characteristics

44.5.1 Accuracy

VCUM targets historically accurate color, physically correct shading, material-consistent detail, and stable temporal coherence.

44.5.2 Speed

Latent-space inference is designed to enable real-time HD-to-4K, near-real-time HD-to-8K, and accelerated HD-to-16K processing. These are architectural targets; actual throughput depends on the deployment hardware profile specified for a given installation and should be validated against Chapter 19's deployment benchmarking practice before being cited externally.

44.5.3 Cost Efficiency

Semantic compression is designed to reduce pixel workload by approximately 90% relative to GAN-based systems. As with §44.5.2, this is a design target; Appendix C's methodology note (§C.11) is the

model for how a figure like this should eventually be validated and sourced before use in external-facing materials.

44.6 Operational Modes

44.6.1 Reality Mimic Mode

Produces modern cinematic color, stylized grading, and enhanced detail.

44.6.2 Historical Restoration Mode

Produces original color reconstruction, era-accurate palettes, material-correct reflectance, and museum-grade fidelity. Both modes use the same semantic pipeline (§44.4); only the priors loaded in Phase I differ.

44.7 Integration Requirements

VCUM requires the following to be fully initialized during QueryBook's pre-launch world-model build: the WVM Physical and Conceptual Layers, NCL Temporal Memory (§7.2), Fact Unit coarse-graining scoped to the video-restoration domain (§44.4.1), the Scenario Graph (Chapter 11), and VCUM's own three module components (SCE, ERUE, TCS, §44.3). These are consumption dependencies, not modifications — VCUM requires these systems to exist and be queryable; it does not alter their specifications, per the Scope Note at the start of this chapter.

44.8 Summary

This chapter establishes VCUM as an applied World-Model System delivering historically accurate colorization, physically correct detail reconstruction, extreme-resolution upscaling (HD to 16K), stable temporal coherence, and multi-phase learning refinement (§44.4) — built entirely as a consumer of QueryBook's existing WVM, NCL, and Scenario Graph capabilities, with no modification to their core specifications and no commingling of its own derived Fact Units with QueryBook's main knowledge base.

The specialized agents that operationally execute this pipeline are specified in Chapter 45.

Chapter 45: VCUM Agent Specifications

This chapter defines the six specialized agents that execute VCUM's three-phase colorization and upscaling pipeline (Chapter 44). Where Chapter 44 specifies VCUM's module architecture (SCE, ERUE,

TCS, §44.3) and phase workflow (§44.4), this chapter specifies the agents that operationally carry that workflow out.

SCOPE NOTE — SAME BOUNDARY AS CHAPTER 44 **These agents inherit Chapter 44's scope boundary in full: they operate on VCUM-scoped Fact Units tagged to the video-restoration domain, and consume the WVM Predictive Layer, NCL Temporal Memory (§7.2), Scenario Graph (Chapter 11), and Fact Unit coarse-graining as already-specified systems — they do not modify those systems' core specifications, and their outputs are never commingled with QueryBook's general-purpose knowledge Fact Graph used for the main query-answering path (Chapters 1–20, 52). The performance figures in §45.5 are stated as design targets, consistent with the same caveat already applied to Chapter 44's §44.5.**

45.1 Agent Role Overview

VCUM Agents are specialized semantic-reconstruction agents responsible for executing the three-phase colorization and upscaling pipeline defined in §44.4. Their role is to transform raw grayscale or low-resolution video into museum-grade, historically accurate, extreme-resolution output using QueryBook's world-model architecture, scoped per the note above. They are the semantic backbone of the video enhancement system defined in Chapter 44.

45.2 Core Agent Specifications

45.2.1 Semantic Extraction Agent (SEA)

Role: builds the initial semantic representation of each frame.

- Identifies objects, materials, lighting, and environment.
- Generates semantic segmentation maps.
- Constructs baseline latent states.
- Initializes historical color priors.
- Produces Phase I uncertainty fields (§44.4.1).

45.2.2 Material Reflectance Agent (MRA)

Role: models and refines material-specific reflectance properties.

- Classifies materials (wood, metal, skin, fabric, stone).
- Applies reflectance curves.
- Corrects material-specific color drift.

- Updates reflectance models during Phase II (§44.4.2).
- Finalizes material accuracy in Phase III (§44.4.3).

45.2.3 Lighting Physics Agent (LPA)

Role: reconstructs physically accurate lighting conditions.

- Models illumination sources.
- Computes shading, shadowing, and specularity.
- Corrects lighting inconsistencies across frames.
- Calibrates lighting physics during Phase II (§44.4.2).
- Produces physically correct lighting in Phase III (§44.4.3).

45.2.4 Temporal Coherence Agent (TCA)

Role: maintains cross-frame consistency using the Scenario Graph (Chapter 11), scoped per §45's Scope Note.

- Tracks object trajectories.
- Stabilizes color across frames.
- Prevents flicker and drift.
- Enforces occlusion-aware consistency.
- Produces the final temporal coherence graph (§44.4.3).

45.2.5 Historical Color Agent (HCA)

Role: restores original or era-accurate color.

- Loads historical color priors.
- Applies cultural color norms.
- Restores original palettes.
- Validates color authenticity.
- Produces museum-grade color reconstruction (§44.4.3).

45.2.6 Extreme-Resolution Agent (ERA)

Role: performs semantic super-resolution upscaling (HD to 16K) via the ERUE (§44.3.2).

- Reconstructs detail using semantic cues.

- Applies material-aware sharpening.
- Performs lighting-aware enhancement.
- Generates extreme-resolution output.
- Validates structural integrity at 16K.

45.3 Phase Responsibility Mapping

Phase	SEA	MRA	LPA	TCA	HCA	ERA
Phase I	Baseline extraction	Material ID	Lighting init	Temporal baseline	Load priors	—
Phase II	Segmentation refinement	Reflectance correction	Lighting calibration	Temporal smoothing	Color drift correction	—
Phase III	Final semantic graph	Material-accurate reflectance	Physically correct lighting	Full temporal coherence	Original color restoration	Extreme-resolution upscaling

45.4 Agent Interaction Model

VCUM Agents operate in a multi-agent semantic pipeline, passing structured latent states between stages, consistent with the phase structure in §44.4:

- SEA to MRA — material classes are assigned to semantic segments.
- MRA to LPA — reflectance curves inform lighting physics.
- LPA to TCA — lighting models stabilize temporal consistency.
- TCA to HCA — stable sequences allow accurate historical color restoration.
- HCA to ERA — final color and semantic detail feed extreme-resolution upscaling.

This pipeline is designed to produce deterministic, physically grounded, historically accurate output, consistent with Chapter 44's rejection of pixel-space hallucination (§44.2.1).

45.5 Agent Performance Requirements

DESIGN TARGETS, NOT YET VALIDATED **The figures below are architectural targets for the agent pipeline, stated in the same spirit as — and subject to the same caveat as — Chapter 44's §44.5. They should be validated against an instrumented pilot and sourced with a methodology note (following the model already set by Appendix C, §C.11) before being cited in external-facing materials.**

45.5.1 Accuracy Targets

- 95% material classification accuracy (MRA).
- 98% temporal consistency (TCA).
- 90% lighting physics fidelity (LPA).
- 95% historical color correctness (HCA).

45.5.2 Speed Targets

Operating in latent space, the agent pipeline targets real-time HD-to-4K, accelerated HD-to-8K, and feasible HD-to-16K processing, consistent with §44.5.2.

45.5.3 Cost Efficiency Target

The agent pipeline targets a reduction in pixel workload of approximately 90% through semantic compression, consistent with §44.5.3.

45.6 Summary

This chapter specifies the six agents (SEA, MRA, LPA, TCA, HCA, ERA) that operationally execute VCUM's three-phase pipeline (Chapter 44): their individual roles, their phase-by-phase responsibilities, their interaction model, and their performance targets. Together with Chapter 44's module architecture, this completes VCUM's specification as an applied World-Model System — built entirely as a consumer of QueryBook's existing WVM, NCL, and Scenario Graph capabilities, with no modification to their core specifications.

Chapter 46: Immersive Query Response Rendering (IQR)

IQR is an applied World-Model System that renders an eligible query response as a navigable 3D/VR environment — for example, a reader querying a book on cave diving receiving an underwater cave scene, constructed from the same Fact Units that grounded the text answer, rather than only reading a paragraph about one. It is positioned alongside Chapters 43–45 as an applied/extension system consuming QueryBook's existing world-model components.

SCOPE NOTE — APPLIED SYSTEM, AND ILLUSTRATIVE, NOT LITERAL RECONSTRUCTIONIQR consumes the WASM World Model Domain (§53.3), VCUM (Chapter 44), the Narrative Graph (Chapter 12), and the Scenario Graph (Chapter 11) as already-specified systems — it does not redefine or extend their core specifications, consistent with the boundary established in Chapters 43–45. Separately, and importantly: IQR produces a representative environment grounded in the topic's Fact Units, not a literal reconstruction of one specific real place. A “cave diving” query does not correspond to one photographed cave anywhere in the Fact Graph; IQR composes a scene consistent with the material, lighting, and spatial facts QueryBook actually holds about caves in general (or about a specific cave, if the query and underlying Fact Units are that specific). This chapter does not claim bit-identical, hallucination-free reconstruction the way VCUM does for restoring an existing video (§44.2.1) — that claim is meaningful when reconstructing a known input; it is not meaningful when composing an illustrative environment from general facts, and this chapter does not make it.

46.1 Overview and Purpose

Most QueryBook responses are text, appropriately. IQR identifies the subset of responses where the underlying Fact Units describe a physical environment with enough specificity to compose a navigable 3D scene, and offers that scene as a supplementary, opt-in rendering of the same grounded answer — not a replacement for the text response, which remains the primary, citable output per the Factual Groundedness requirements already specified for the Narrative Graph (§12).

46.2 Immersion Eligibility

Not every response is a good candidate for immersive rendering. A response becomes IQR-eligible when the Fact Units grounding it include enough WASM Object Layer coverage (§53.3.1) to compose a scene without fabricating unsupported specifics:

- At least one Object Layer node of Type Environment (indoor space, outdoor space, or a domain-specific environment class such as the underwater/geological classes a cave diving Fact Bundle would contain).
- Material Attributes (§53.3.1) for the environment's dominant surfaces — rock, water, ice, sand, and so on — sufficient to drive VCUM's material synthesis (§46.3.2 below).
- At least a partial Relation Layer (§53.3.2) spatial layout — enough Spatial Relations (above, below, inside, adjacent-to) to compose a coherent 3D arrangement rather than a disconnected object list.

A response failing this check is not rendered as a scene — it remains a text-only answer. IQR never fills a coverage gap by inventing environment details the Fact Graph does not support; this is the same

Without-Information Rule preference already established for text responses (D0.4), applied to scene composition.

46.3 Scene Composition Pipeline

46.3.1 Environment Extraction

IQR queries the WASM Object Layer (§53.3.1) for the Environment-type node(s) and connected Object nodes relevant to the response's grounding Fact Units, using the same Associative Recall Engine retrieval already specified for NCL pattern completion (§7.2.3) rather than a separate retrieval path.

46.3.2 Material and Lighting Synthesis

Extracted material Attributes are passed to VCUM's Semantic Colorization Engine (SCE, §44.3.1) for physically-consistent color and reflectance synthesis, and to the Lighting Physics Agent (LPA, §45.2.3) for illumination modeling appropriate to the environment class (for example, refracted and attenuated light for an underwater cave, versus direct sunlight for an outdoor environment) — both reused directly from VCUM's existing pipeline, not reimplemented.

46.3.3 Spatial Layout Construction

The Relation Layer's Spatial Relations (§53.3.2) between Object Layer nodes are compiled into a 3D scene graph — relative positions, containment, and adjacency become actual spatial coordinates, with under-specified distances resolved to class-typical defaults (documented per environment class, not invented per instance) rather than arbitrary values.

46.3.4 Detail Enhancement

The composed scene's surface detail is enhanced using VCUM's Extreme-Resolution Agent (ERA, §45.2.6) and Extreme-Resolution Upscaling Engine (ERUE, §44.3.2), applied to the synthesized environment's textures rather than to a real captured frame — the same detail-reconstruction technique, applied to a different kind of input, with the same scoping caveats VCUM already carries about performance targets not yet being validated (§44.5).

46.4 Interactive Extension

A rendered scene may optionally connect to the Scenario Graph (Chapter 11): a reader's in-scene action (for example, swimming toward a visible passage) can be treated as a Condition on a Scenario Graph Transition, surfacing the Knowledge State Snapshot the reader has effectively navigated toward as a

follow-up text response. This is an optional integration point, not a requirement for IQR rendering to function, and is scoped as a future extension rather than specified in full here.

46.5 Delivery Format

IQR does not define a new proprietary container format. Composed scenes are serialized to glTF 2.0 (or USD, for more complex multi-object scenes) — open, widely-supported 3D scene formats with existing tooling across every major game engine, browser WebXR implementation, and VR headset platform — rather than inventing a QueryBook-specific volumetric format that would require custom playback software. Delivery to a headset or WebXR-capable browser uses standard glTF/WebXR streaming practice; IQR's contribution is what goes into the file, not a new way of transmitting it.

46.6 Fidelity and Grounding Guarantees

- Reproducible: the same grounding Fact Units and the same eligibility check (§46.2) produce a scene with the same object set, spatial layout, and material assignments — this is the meaningful determinism guarantee here, not pixel- or vertex-level identity.
- Fact-traceable: every Object Layer node included in the scene carries the same InputFUIDs lineage back to its source Fact Units that any other derived output does (§3.1), so a reader can ask the Q&A agent (§19.8) why a given object or material appears in the scene.
- Not a claim of photographic accuracy: unless the grounding Fact Units are specific to one real, documented location, the scene is representative of the topic's general facts, not a reconstruction of a particular unphotographed place — stated plainly rather than left implicit, consistent with this chapter's Scope Note.

46.7 Example Walkthrough

A reader queries a book on cave diving. The Narrative Graph response describes typical underwater cave conditions — restricted visibility, submerged rock formations, thermoclines, silt-out risk. IQR's eligibility check (§46.2) finds sufficient WASM coverage: an Environment node (submerged cave), Object nodes for rock formations and water volume with Material Attributes, and Spatial Relations placing passages, chambers, and the water surface relative to one another. The scene composition pipeline (§46.3) produces a glTF scene with water-appropriate lighting attenuation (§46.3.2), the cave's spatial layout as described in the source material (§46.3.3), and enhanced rock-surface detail (§46.3.4). The reader receives both the text answer and an offer to view the accompanying 3D environment.

46.8 Integration Requirements

IQR requires the WASM World Model Domain (§53.3), VCUM's SCE and ERUE/ERA components (Chapters 44–45), the Narrative Graph (Chapter 12), and the Associative Recall Engine (§7.2.3) to be available and populated. These are consumption dependencies — IQR does not modify any of their specifications, per the Scope Note at the start of this chapter.

46.9 Tour and Exhibition Mode

Everything specified above (§46.1–§46.8) composes one scene from one query response. Tour and Exhibition Mode extends this to a curated, author-built sequence of scenes spanning a topic, a chapter, or an entire book — a guided 3D/VR walkthrough an author or educator assembles once and any reader or class can then experience.

46.9.1 The Tour Fact Unit

A Tour is a Fact Unit of FactType TOUR, versioned and access-controlled like any other Fact Unit (Appendix A), containing an ordered or Scenario-Graph-branching (Chapter 11) list of Waypoints. Each Waypoint references a topic-level Fact Bundle (§3.4), the IQR scene composed for it (§46.3), a camera and pacing specification (§46.9.4), and an optional narration track (§46.9.5). A Tour is itself a Fact Bundle — it does not duplicate the underlying book content, only sequences and stages references to it.

46.9.2 Coverage Mapping

Before building a Tour, an author needs to know which topics can become Waypoints at all. The Coverage Map runs the Immersion Eligibility check (§46.2) across every topic-level Fact Bundle within a specified scope — a single topic, a chapter, or the whole book — and returns a report: which topics pass eligibility and are ready to become Waypoints, and which fail, with the specific missing WASM Object Layer coverage (§53.3.1) named, not just a pass/fail flag. This turns tour authoring into an informed selection process rather than trial and error against individual topics.

46.9.3 Authoring Interface

Tour authoring is a distinct interface from the reader-facing Q&A interaction (§19.8) — it is where an author browses the Coverage Map (§46.9.2), selects and orders Waypoints, and, critically, can override or annotate what IQR auto-composed for any Waypoint's scene. IQR's automatic composition (§46.3) is representative and fact-grounded, per this chapter's Scope Note, but an author curating a tour is expected to exercise judgment IQR cannot — correcting a spatial layout default, adding an emphasis the auto-composed scene lacks, or excluding a technically-eligible topic that doesn't serve the tour's

narrative. An author override is recorded as such in the Waypoint's Provenance Ledger entry (Chapter 5), distinguishing author-curated content from IQR's automatic composition without disguising one as the other.

46.9.4 Camera and Pacing Choreography

Each Waypoint specifies how the viewer moves through its scene:

- Entry camera pose and, optionally, an entry transition (cut, fly-through from the previous Waypoint, or fade).
- A dwell behavior — a fixed camera hold, a slow orbit, or free-look navigation the viewer controls before advancing.
- An advancement trigger: linear (author-set duration or explicit “next” action) or Scenario-Graph-driven (§46.4), where a viewer's in-scene action selects the next Waypoint rather than following a fixed sequence.

46.9.5 Narration Synchronization

A Waypoint's narration track is MRE-generated text (Chapter 16) from the same grounding Fact Units as its scene, timed against the pacing specification in §46.9.4 — dwell duration adjusts to accommodate narration length rather than the two running independently and falling out of sync. Narration may be delivered as displayed text, synthesized audio (§46.10.3), or both.

46.9.6 Multi-Viewer Delivery

An educator presenting a Tour to a class uses a synchronized session model distinct from IQR's default single-reader assumption: one session controller (the educator) advances Waypoints, and all connected viewers see the same Waypoint at the same time. A viewer's individual free-look navigation within a Waypoint's dwell behavior (§46.9.4) remains local and does not desynchronize the group.

46.9.7 Tour Persistence and Versioning

A Tour Fact Unit carries the same VersionChain (Appendix A) as any authored content, so an author can revise a Tour after publication, and prior versions remain retrievable through the same lineage mechanism already specified for every other Fact Unit. Access control follows the same AccessControlList field already specified for Fact Units generally (Appendix A) — an author's Tour is theirs to share or restrict, not automatically public because its source book is.

46.10 Presenter Mode

Presenter Mode is QueryBook's Interactive Modes grouping for Tour and Exhibition Mode (§46.9): an optional, AI-generated author presence guiding a Tour. A Tour may run with no presenter at all — environment and narration only, per §46.9.5 — or with one of three presenter modes, selected by the author per Tour or per Waypoint.

GOVERNANCE NOTE — AUTHORIZATION IS A PRECONDITION, NOT A SETTING **Every presenter mode in this section synthesizes a real, identifiable person's voice, face, or both. None of the three modes below may generate output for a person without a matching Author Authorization Record (§46.10.2) already on file for that specific person. This is enforced as a Safety Substrate gate (Chapter 8), not a configurable option — there is no presenter-mode code path that bypasses the authorization check, consistent with the non-bypassable enforcement model already established for every other Safety Substrate gate (§8.1.1).**

46.10.1 Author Authorization Record

Before any presenter mode can generate content for a given person, that person must complete an explicit authorization flow: enrolling voice and/or face reference samples they provide themselves, confirming they are the rights-holder for that voice and likeness (or hold documented authorization from the rights-holder, for cases where an educator presents using licensed material from someone else), and specifying which presenter modes (§46.10.3–§46.10.5) they authorize. The resulting Author Authorization Record is itself a Fact Unit, access-controlled and auditable through the Provenance Ledger (Chapter 5) like any other, and is scoped — an author authorizing Voice-Only mode has not thereby authorized Face-on-Avatar mode; each mode requires its own explicit grant.

46.10.2 Revocation

An author may revoke their Authorization Record at any time. Revocation takes effect immediately: the Safety Substrate gate (§46.10 Governance Note) rejects generation for that person from the moment of revocation forward, for every existing and future Tour, not only new ones — a previously-published Tour using a now-revoked authorization must fall back to text/environment-only delivery (§46.9.5) rather than continue serving previously-generated presenter content indefinitely.

46.10.3 Voice-Only Mode

Synthesizes narration audio in the author's authorized voice (§46.10.1), with no visual presenter — the Tour scene remains the sole visual content. Voice synthesis draws on the same MRE-generated narration text already specified in §46.9.5, timed to the same pacing specification (§46.9.4).

46.10.4 Face-Only Mode

Adds a floating or inset video presenter — the author's authorized facial likeness, animated to the synthesized narration audio (§46.10.3) — without full-body embodiment in the 3D scene. The presenter overlays the environment rather than occupying a position within it.

46.10.5 Face-on-Avatar Mode

Composites the author's authorized facial likeness onto a full-body avatar embodied within the Tour's 3D scene itself — able to occupy a position, gesture, and appear to move through the environment alongside the viewer (for example, appearing to swim through the cave in the §46.7 walkthrough). This is the most demanding presenter mode and inherits VCUM's existing material and lighting synthesis (§44.3.1–§44.3.2) for compositing the avatar body consistently with the scene's lighting.

46.10.6 Mandatory Disclosure

DISCLOSURE IS NON-REMOVABLE **Every Tour using any presenter mode carries a persistent, viewer-visible indicator stating that the presenter is AI-generated using the author's authorized voice and/or likeness — not a live recording or the author's real-time presence. This indicator is not an author-configurable setting and cannot be disabled per Tour; it is enforced at the same rendering stage as the presenter content itself, consistent with Domain 0's harm-prevention scope covering AI-generated outputs generally (D0.1–D0.2).**

46.10.7 Relationship to VCUM

Presenter Mode is a consumer of VCUM's existing material, lighting, and detail-synthesis components (Chapters 44–45), applied to a specific narrow case — rendering an authorized presenter, not a general likeness-synthesis capability — rather than a new visual pipeline. It does not modify VCUM's core specification, per the scoping discipline already established for every applied system in this part of the Bible (§43, §44, §46's own Scope Note).

46.10.8 Color and Registration Stability

Face-on-Avatar mode's compositing (§46.10.5) uses Chapter 70's existing Color and Geometry Fact Units to keep an already-authorized presenter visually stable and accurately colored across a Tour, rather than introducing any new likeness-reconstruction capability:

- The author's authorized reference imagery (§46.10.1) is represented as Color Fact Units (§70.1) with a DeltaEThreshold, so skin tone and material color stay within a validated tolerance across every Waypoint of a Tour rather than drifting scene to scene — the same ΔE validation already

specified for any other authored visual content, applied here for consistency, not for generating new detail.

- The avatar's position and movement through the 3D scene (§46.10.5's “occupy a position, gesture, and appear to move”) is governed by Geometry Fact Units and the Registration Constraint Graph (§70.4), anchored to Tour Waypoints exactly as §70.5 already specifies for any VR object — not a new avatar-specific anchoring system.
- A ColorDrift or RegistrationDrift Fact Unit (§70.3–§70.4) produced during a Tour is treated the same way any other drift finding is: surfaced, not silently corrected by inventing new detail to compensate.

WHAT THIS SECTION DOES NOT ADD This is a stability and accuracy improvement to compositing that is already fully authorized and disclosed under §46.10 — it adds no new reconstruction fidelity. It does not add pore-level skin detail, individual hair-strand simulation, facial muscle or FACS simulation, physically-simulated cloth, or a rigging system beyond what Face-on-Avatar mode already does. A general-purpose photorealistic human reconstruction and animation capability was proposed and declined earlier in this Bible's history specifically because reconstruction fidelity at that level is not a problem consent can resolve on its own — the specification itself is the same capability regardless of whose authorized likeness a given instance points to. This section stays on the accuracy-and-stability side of that line: making an existing, already-consented presenter render consistently, not making it more detailed.

46.11 Summary

IQR extends QueryBook's existing world-model components — WASM's Object Layer, VCUM's material and detail reconstruction, the Narrative Graph's grounded responses — into an optional, fact-traceable 3D scene rendering for responses with sufficient physical-environment grounding, delivered in open, standard 3D formats. Tour and Exhibition Mode (§46.9) sequences those scenes into author-curated walkthroughs spanning a topic, a chapter, or a book, with an optional AI-generated Presenter (§46.10) — voice, face, or a full embodied avatar — gated on an explicit, revocable Author Authorization Record and a non-removable AI-generated disclosure in every case. Together, this chapter adds no new core architecture, no new acronym-colliding terminology, and no components QueryBook does not already have.

Chapter 47: QBCL-CMS — Configuration Language and Management System

QBCL-CMS is the runtime-configurable store the Adaptive Response Engine (Chapter 52) reads from twice: once for the Discipline Mode Registry (§52.5) and once for the operator's baseline Response Profile, Tier 1 of the Three-Tier Governance Cascade (§52.7.1, §52.7.4 Step 1). Both dependencies existed in this Bible before this chapter did; this chapter specifies the store itself.

SCOPE NOTE — CONFIGURATION STORE, NOT A NEW EXECUTION SYSTEM **QBCL-CMS is a configuration store with a small declarative schema, not a general-purpose language or a new execution pipeline. It reuses the structured-data ingestion formats already specified in §4.2 (JSON, YAML, TOML) rather than defining a new file format, reuses the REST/gRPC API surface already specified in Chapter 9 rather than a new transport, and reuses the Safety Substrate (Chapter 8) and Provenance Ledger (Chapter 5) for governance and audit rather than introducing parallel mechanisms. Nothing in this chapter modifies the Discipline Mode Registry or Response Profile Governance Cascade specifications themselves (§52.5, §52.7) — it specifies where the values those sections already reference actually live.**

47.1 Purpose and Scope

Two categories of configuration are operator-managed rather than fixed at build time: Discipline Mode template definitions (§52.5 — structural template, citation format, terminology convention set, mandatory and prohibited structural elements, per Discipline Mode) and the platform-wide operator baseline Response Profile (§52.7.1 — permitted Expertise Level range, permitted Discipline Modes, the Safety Override Level floor, the default Response Profile, mandatory inclusions and exclusions). QBCL-CMS is the single store for both, so that an operator has one place to configure platform behavior rather than two independently-versioned stores that could drift out of sync with each other.

47.2 QBCL — The Configuration Language

QBCL (QueryBook Configuration Language) is a declarative, schema-validated configuration format, not a general-purpose or Turing-complete language — it has no loops, no conditionals beyond simple field-level validation rules, and no capability to reference external systems beyond the specific, allow-listed fields each schema defines. A QBCL document is authored in any of the structured-data formats already ingestible per §4.2 (JSON, YAML, or TOML) and validated against one of two fixed schemas before being accepted:

- Discipline Mode Schema — the fields a Discipline Mode Registry entry requires (§47.3).
- Operator Baseline Schema — the fields an operator baseline Response Profile requires (§47.4).

A QBCL document that fails schema validation is rejected outright with a field-level error; QBCL-CMS never applies a partially-valid configuration.

47.3 Discipline Mode Registry Storage

Each Discipline Mode entry (GENERAL, MEDICAL, LEGAL, SCIENTIFIC, ENGINEERING, FINANCIAL, EDUCATIONAL, EXECUTIVE, REFERENCE — the nine modes already enumerated at §52.5.1–§52.5.9) is stored as one QBCL document conforming to the Discipline Mode Schema: structural template, citation format, terminology convention set, mandatory structural elements, and prohibited structural elements, exactly the fields §52.5 already specifies each Discipline Mode as defining. QBCL-CMS does not define what those fields mean — §52.5 already does — it defines how they are stored, versioned, and served.

Adding a new Discipline Mode beyond the nine already specified, or editing an existing one's template, is a write to this store, subject to the same governance as any other configuration change (§47.5) — it is not a change to this Bible's own specification of what a Discipline Mode is.

47.4 Operator Baseline Configuration

The operator baseline Response Profile (§52.7.1) is stored as a single QBCL document conforming to the Operator Baseline Schema, with one field-level constraint enforced by QBCL-CMS itself rather than left to operator discretion:

- Safety Override Level floor: QBCL-CMS validates that any write to this field is greater than or equal to STANDARD. A write attempting to set the floor below STANDARD is rejected at the schema-validation stage (§47.2), before it reaches the Safety Substrate gate in §47.5 — this constraint is redundant with Safety Substrate enforcement by design, so that a floor violation is caught as early and as cheaply as possible rather than relying on a single enforcement point.

The remaining fields — permitted Expertise Level range, permitted Discipline Modes, the default Response Profile, mandatory inclusions, and mandatory exclusions — are stored and validated for internal consistency (for example, a default Response Profile must specify only Discipline Modes and Expertise Levels within the permitted ranges also being set in the same write) but carry no QBCL-CMS-level floor of their own beyond that consistency check.

47.5 Configuration Change Governance

Every write to QBCL-CMS — a new or edited Discipline Mode template, or a change to the operator baseline — is treated as a privileged operation, subject to the same enforcement this Bible already applies elsewhere:

- Authentication and authorization follow the same Bearer token and role-based access model already specified for the API generally (§9.5.1); writing to QBCL-CMS requires operator-level privilege, not the general API caller role.
- Every accepted write is logged to the Provenance Ledger (Chapter 5) with the writing identity, timestamp, and a diff against the prior configuration value — configuration state is provenance-tracked the same way every other piece of state in this Bible already is.
- The Safety Override Level floor check (§47.4) is re-verified at this stage as well, not only at schema validation — defense in depth, consistent with how the Safety Substrate is already specified as being embedded at every layer rather than a single checkpoint (§8.1).
- A rejected write returns the specific validation or authorization failure to the caller; QBCL-CMS never silently ignores an invalid write or falls back to a prior value without that fallback being itself a logged event.

47.6 API Surface

QBCL-CMS is read and written through the same REST/gRPC API surface already specified in Chapter 9, not a separate transport: GET and PUT operations against /v1/config/discipline-modes/{mode} for Discipline Mode entries, and GET and PUT against /v1/config/operator-baseline for the operator baseline. Both endpoints require the operator-level authorization specified in §47.5 for PUT; GET is available to any authenticated internal caller needing to read current configuration, consistent with the general API access model (§9.5.1).

47.7 Integration with the ARE

QBCL-CMS is read, not written, during normal query processing. The Three-Tier Response Profile Governance Cascade (§52.7.4) names QBCL-CMS explicitly as the source for Step 1, loading the operator's baseline Response Profile before Tier 2 (author) and Tier 3 (subscriber) overrides are layered on top. The Discipline Mode Template Selection stage of the ARE Pipeline (§52.8.2) reads the applicable Discipline Mode's stored template from §47.3 during Generation Directive construction. Neither

integration point is new — both were already specified as reading from QBCL-CMS before this chapter existed to define what that store actually is.

47.8 Summary

QBCL-CMS closes a dependency that existed in this Bible since Chapter 52 was first drafted: a runtime-configurable store for Discipline Mode templates and the operator baseline Response Profile, built entirely from components this Bible already specifies — the structured-data ingestion formats (§4.2), the REST/gRPC API (Chapter 9), the Safety Substrate (Chapter 8), and the Provenance Ledger (Chapter 5). It defines no new execution model, no new transport, and no new governance mechanism; it defines where two already-specified categories of configuration live and how changes to them are validated, authorized, and audited.

Chapter 60: Publisher and Print-on-Demand Distribution

This chapter specifies the publisher-facing workflow for taking a manuscript from submission through physical and digital distribution — submission intake, a rights validation gate, print production parameters, and distribution routing. It originated from a cross-reference and deduplication pass over an external, non-canonical draft's feature and function registries (245 feature rows, 1,730 function rows); the great majority of that material either duplicated something already specified in this Bible, described that other draft's own internal architecture rather than a real capability, or was excluded outright for cause — documented in the accompanying audit report, not repeated here. This chapter specifies only what survived that review: a coherent, genuinely new publisher capability, built from this Bible's existing components.

SCOPE NOTE — APPLIED WORKFLOW, NOT NEW CORE ARCHITECTURE **Every mechanism in this chapter is a consumer of already-specified systems: the Safety Substrate (Chapter 8) gates rights validation, the Provenance Ledger (Chapter 5) records the submission and distribution chain, the Interactive Mode Registry's Publisher Portal concept (§52.13) is where these workflows surface, and the External System Integration contract (§49.3.3) is how print-on-demand vendors and distribution partners connect. This chapter defines no new safety mechanism, no new storage tier, and no new transport — it defines a publisher-facing workflow assembled from those existing parts.**

60.1 Submission Intake

A publisher submission is ingested through the same Multimodal Ingestion Pipeline already specified in Chapter 4 — typically as Unstructured Text (manuscript) plus Structured Data (metadata: title, author, rights holder, target markets) — and produces Fact Units exactly as any other ingestion event does. What distinguishes a publisher submission is state, not architecture: it is tagged with a SubmissionStatus field (RECEIVED, IN_REVIEW, RIGHTS_PENDING, APPROVED, REJECTED) tracked through the Provenance Ledger, so a submission's full review history remains queryable the same way any other Fact Unit's lineage is.

60.2 Rights Validation Gate

Before a submission can advance past RIGHTS_PENDING, it passes through a Rights Validation Gate: a Safety Substrate enforcement point (§8.1) that checks the submission's declared rights-holder metadata against the publisher's own licensing records before any distribution-facing operation (print production, market listing, translation) is permitted to run. This consolidates three overlapping ideas from the source material — a rights pre-checker, a rights validator, and a rights validation step — into one gate, consistent with this Bible's existing pattern of a single enforcement point rather than duplicated, potentially inconsistent checks (the same discipline already applied to QBCL-CMS's Safety Override Level floor, §47.4–47.5).

- A submission failing rights validation is held at RIGHTS_PENDING with the specific failure reason recorded in the Provenance Ledger, not silently blocked.
- This gate is deliberately scoped to licensing/permission verification only. It does not mint, tokenize, or otherwise create any new rights-representation artifact — the source material's proposal to represent validated rights as a minted NFT was excluded from this Bible during the audit that produced this chapter, for reasons documented in that report, and nothing here reintroduces it.

60.3 Print Production Parameters

Once a submission is APPROVED, print production is specified as a set of parameters attached to the submission's Fact Bundle, not a new subsystem:

- Print Preview — a rendered preview of the final print layout, generated from the manuscript's existing structural Fact Units (chapter breaks, headings) rather than a separate document format.
- Cover Design — an attached asset reference (image Fact Unit, per §4.2's Image modality) with placement metadata.

- Layout Templates — a named, reusable set of print formatting parameters (trim size, margin, typeface) an operator selects per submission rather than configuring from scratch each time.
- Batch Printing — a print run specified as a quantity against a given submission and layout template, queued through the same job-scheduling pattern already used elsewhere in this Bible's deployment architecture (Chapter 19).

60.4 Distribution and Vendor Routing

Physical distribution is handled as an External System Integration (§49.3.3): a print-on-demand vendor or shipping partner registers as a Source Connector with a capability declaration scoped to distribution operations, authenticated the same way any other external connector is, with the same governance — no distribution partner receives elevated trust over any other external system.

- Global Routing — selects a fulfillment vendor by target market, using the connector's declared regional capability rather than a hardcoded vendor list.
- Vendor Integration — the Source Connector registration itself (§49.3.3.1): capability declaration, authentication, data mapping, health reporting.
- Shipping Logistics — order and tracking data flows back through the same connector as a FEDERATED-style read-through (§49.3.2.6), not duplicated into QueryBook's own storage.
- Distribution Scheduling — a publisher-set release date gating when APPROVED submissions become visible to distribution connectors; this is a simple field-level gate, not a new scheduling subsystem.

60.5 Localization Preview

Before a submission is distributed to a non-default-language market, a Localization Preview renders the MRE's translated output (§16, §50) for publisher review prior to release — reusing Chapter 16's Multilingual Regeneration Engine and Chapter 50's NLIL rather than a separate translation path. This is a review step, not a new translation capability.

60.6 Market Analytics

DESIGN TARGET, NOT YET VALIDATED **Market analytics in this section are specified as a reporting capability over data QueryBook already has — distribution volume, regional demand from the routing data in §60.4, reader engagement telemetry from the Interactive Mode Registry (§52.13) — not a predictive claim about future sales. Any forward-looking or predictive framing of this data should be treated as a design target requiring the same validation discipline already established elsewhere in this Bible (Appendix C, §C.11; §44.5) before being presented to a publisher as a forecast rather than a historical report.**

60.7 Integration Requirements

This chapter requires the Multimodal Ingestion Pipeline (Chapter 4), the Provenance Ledger (Chapter 5), the Safety Substrate (Chapter 8), the Source Connector Registry (§49.3.3), the Multilingual Regeneration Engine (Chapter 16), and NLIL (Chapter 50) to be available. These are consumption dependencies; this chapter modifies none of their specifications.

60.8 Summary

This chapter specifies submission intake, a consolidated rights validation gate, print production parameters, external distribution routing, localization review, and market reporting — the one coherent, genuinely new capability that survived a full compatibility audit of an external draft's 245-feature registry. Everything here is a consumer of existing QueryBook architecture; nothing here introduces a new safety mechanism, storage tier, or rights-representation model.

Chapter 61: Author Drafting Tools

Every reasoning and consistency-checking capability specified so far in this Bible operates on a published book's Fact Graph — Discussion Mode analyzes it, Contradiction detection (§D4 patterns via the Neuromorphic Cognitive Layer and AEM, Chapters 7 and 15) checks it, the Scenario Graph (Chapter 11) branches from it. This chapter applies the same underlying capabilities to a manuscript still in progress, before publication, as an author-facing drafting aid — not a new reasoning engine, a new application of the existing one.

SCOPE NOTE — SAME REASONING, EARLIER LIFECYCLE STAGE **Manuscript Fact Units in this chapter carry a DraftStatus field distinguishing them from published content; they are ingested through the same Multimodal Ingestion Pipeline (Chapter 4) and reasoned over with the same NCL, Scenario Graph, and AEM components already specified, not a parallel drafting-specific engine. A draft's Fact Units do not enter the reader-facing Fact Graph until the manuscript is published, consistent with the Rights Validation Gate (§60.2) already required before that transition.**

61.1 Manuscript Ingestion and the Draft State

An author's in-progress manuscript is ingested exactly as any Unstructured Text source is (§4.2), producing Fact Units tagged DraftStatus: DRAFT rather than PUBLISHED. This is the only structural difference — draft Fact Units use the same schema, the same Provenance Ledger lineage tracking, and the same Ontology Mesh typing as published ones. A Manuscript Editor is the authoring surface for this ingestion — not a new component, but the Author Portal's entry point into the existing pipeline, versioned through the same VersionChain mechanism (Appendix A) already specified for any Fact Unit.

61.1.1 Uploading an Existing, Complete Manuscript

§61.1 describes ingestion through the Manuscript Editor, the Author Portal's incremental, in-app drafting surface. This is not the only, or even the primary, way a manuscript reaches QueryBook. Most authors arrive with a finished ebook file, not a blank page — and this is the primary initial ingestion pathway for QueryBook's repository: an author uploads an already-complete manuscript, in whatever format they already have it (EPUB, PDF, DOCX, or any other format §4.2 already supports under the Unstructured Text modality), directly at the Author Portal.

SAME PATHWAY, DIFFERENT CADENCE **A bulk-uploaded manuscript is not a different ingestion mechanism from §61.1's incremental one — it is the identical pipeline (§4.2's Multimodal Ingestion Pipeline, the same DraftStatus: DRAFT tagging, the same Provenance Ledger and Ontology Mesh handling) receiving the whole work in a single ingestion event rather than accumulated edit by edit. No new Fact Unit schema, ingestion stage, or Author Portal component is introduced by this section.**

- The uploaded file is ingested exactly as §4.2 specifies for Unstructured Text, including §4.2.1's classification fallback for any content that doesn't cleanly resolve on first pass — an upload is not exempt from that handling merely because it arrived as one file instead of many edits.
- The resulting Fact Units carry DraftStatus: DRAFT, identically to §61.1's incrementally-drafted ones, and are visible only to the uploading author and any co-author (Chapter 74) until the manuscript is

explicitly submitted — an upload does not become reader-visible or enter the repository automatically upon ingestion completing.

- An author can apply Narrative Coherence Checking (§61.2) to a freshly-uploaded manuscript the same way as an in-progress draft — useful here specifically to verify the ingestion pipeline extracted the work's structure and content accurately before the author proceeds to submission, since a large single-event ingestion has more surface area for a misclassified chapter or misattributed passage than an incremental edit does.
- When the author is satisfied the ingested Fact Graph accurately represents their work, they submit it, transitioning its Fact Units' DraftStatus out of DRAFT — the same trigger §60.1's Submission Intake already specifies, leading into the Rights Validation Gate (§60.2) before the book becomes part of the reader-facing repository. Nothing about this being a bulk upload rather than an in-app draft changes that gate; every book reaches readers through the identical rights-validation step regardless of how it was ingested.

61.2 Narrative Coherence Checking

Narrative coherence checking runs Discussion Mode's existing analytical capability (§52.13.3) against a draft's own Fact Units rather than a published book's: the same bounded, text-grounded analysis, scoped to the manuscript being drafted, surfacing plot inconsistencies, character logic conflicts, and world-state contradictions using the same Contradiction Detection pattern already specified for AEM (Chapter 15) and the NCL's consistency mechanisms (Chapter 7). This is Discussion Mode's existing behavior applied one lifecycle stage earlier — before a reader ever sees the contradiction, not a new checking algorithm.

61.3 Counterfactual Exploration

A counterfactual generator lets an author explore “what if” branches of their own draft — built directly on the Scenario Graph (Chapter 11), which already specifies exactly this capability: a branching, probabilistic directed graph where each node is a Knowledge State Snapshot and each edge a conditioned Transition. Applied to a draft, the KSS is the manuscript's current world-state and a Transition is an author-proposed plot alternative, evaluated for consistency against the rest of the draft using the same Narrative Reasoning patterns as §61.2. This consolidates what the source material described as three separate ideas — a counterfactual generator, an alternative-timeline generator, and a “what-if” engine — into one capability, since evaluated on their own terms, all three named the same Scenario Graph traversal.

61.4 Integration with Publication

When an author submits a manuscript for publication, its Fact Units transition from DraftStatus: DRAFT to the Submission Intake flow specified in §60.1 — authoring and publishing are one continuous Fact Unit lineage, not a handoff between separate systems. A manuscript's full draft history, including every coherence-check finding and explored counterfactual, remains in the Provenance Ledger after publication, giving a published book's Fact Graph a complete authorial lineage rather than starting its provenance record at the moment of submission.

61.5 Integration Requirements

This chapter requires the Multimodal Ingestion Pipeline (Chapter 4), the Provenance Ledger (Chapter 5), the NCL (Chapter 7), the Scenario Graph (Chapter 11), AEM (Chapter 15), Discussion Mode (§52.13.3), and Submission Intake (§60.1). These are consumption dependencies; no component's specification is modified.

61.6 Summary

Manuscript creation, narrative coherence checking, and counterfactual exploration are specified here as the same Discussion Mode, Scenario Graph, and Contradiction Detection capabilities already in this Bible, applied to a manuscript before publication rather than after. Nothing in this chapter is a new reasoning capability; it is the existing ones, one lifecycle stage earlier.

Chapter 62: Author-Hosted Reader Events

This chapter specifies the capability for an author to create a live or scheduled event — a class session, a conference, a Q&A meeting — and invite readers to it, through whichever external collaboration platform the author already uses. It responds directly to a requirement that this be supported “no matter what,” and it is: every capability described in the source request (create the event, populate it, invite readers, let readers join and later replay it) is fully specified below. What differs from the source request is the mechanism — this chapter routes through the Source Connector Registry (§49.3.3) already specified in this Bible, rather than defining hardcoded API endpoints for three named external companies.

SCOPE NOTE — WHY A CONNECTOR, NOT A HARDCODED INTEGRATIONA hardcoded integration (a QueryBook endpoint that calls Google's API directly, another that calls Zoom's API directly) would need to be rebuilt every time a new platform is added, would give three named companies a permanent, privileged position in this Bible's architecture, and would bypass the authentication, data-mapping, and governance model §49.3.3 already requires of every other external system. Routing through the Source Connector Registry instead means any collaboration platform — Google Meet, Zoom, or one that doesn't exist yet — works the same way, through the same registration process, under the same governance, with no special-casing. The reader-facing outcome is identical either way: an author creates an event, readers get invited, readers join and can replay it. Only the underlying mechanism differs, and it differs for the better.

62.1 Event Creation

An author creates an event by specifying: an event type (class session, conference, live Q&A), a title and description, a scheduled time, an optional recurrence pattern, and which of the author's registered collaboration connectors (§62.2) to host it through. The event itself is stored as a Fact Unit — author-authored, provenance-tracked through the Provenance Ledger (Chapter 5) exactly like any other authored content — not a new storage mechanism. Being a Fact Unit, an event is retrievable through the same Associative Recall Engine (§7.2.3) that answers any other QueryBook query, exactly as §64.8 already states for Message Board content — a reader asking about an upcoming or past event is served by the same single retrieval mechanism as a reader asking about the book itself, not a separate events search.

62.2 Collaboration Connectors

A collaboration platform — Google Meet, Google Classroom, Zoom, or any other video/classroom platform — is registered as a Source Connector (§49.3.3) with a Capability Declaration scoped specifically to event operations: create a session, generate a join link, list attendees, and (where the platform supports it) retrieve a recording reference after the event ends. Registration, authentication, and data mapping follow §49.3.3.1 exactly as specified for any other connector — an author authorizes their own account on the external platform (an OAuth-style grant, not a QueryBook-managed credential), and QueryBook never stores the author's platform password or long-lived external credentials directly.

- An author may register more than one collaboration connector (for example, both Google Meet and Zoom) and choose per event which one to use.

- QueryBook does not re-implement classroom or conferencing functionality — the external platform hosts the actual session; QueryBook stores the event's metadata, the join link, the invitation list, and, after the fact, a reference to the platform's own recording, not a copy of the video itself.

62.3 Reader Invitation and Discovery

An author invites readers to an event in one of two ways: a direct invitation to specific readers (delivered as a notification, per the Interactive Mode Registry's existing delivery-channel patterns, §52.13.4), or an open listing visible to any reader of the book, discoverable through an Event Calendar view. A reader accepting an invitation receives the join link generated by the collaboration connector (§62.2) — QueryBook brokers the invitation and scheduling; the external platform still handles the actual join flow and identity, on its own terms.

62.3.1 Territory and Access Scoping

An author may scope an event's visibility — to all readers of the book, to readers within a specific region, or to readers holding a specific license type (for example, an institutional or classroom license) — using the same access-control patterns already specified for content generally (Appendix A's AccessControlList field), not a new permissions system.

62.4 Replay

Where the collaboration connector's platform provides a recording after the event, QueryBook stores a reference to that recording (a link, plus metadata: duration, date, attendee count) as a Fact Unit attached to the event, viewable by readers who were invited. QueryBook does not host, re-encode, or take ownership of the recording itself — the external platform remains the source of truth for the actual media, consistent with §62.2's principle that QueryBook brokers events rather than replacing the platforms that run them.

62.5 Governance

NO SPECIAL-CASING **Every collaboration connector, regardless of which platform it represents, is subject to the same Source Connector governance already specified at §49.3.3.4: no elevated trust, no bypass of the Safety Substrate (§8.1), and the same authentication and data-mapping validation required of any other external system. An author's choice of collaboration platform does not change what QueryBook requires of the connection.**

62.6 Integration Requirements

This chapter requires the Source Connector Registry (§49.3.3), the Provenance Ledger (Chapter 5), and the Interactive Mode Registry's notification and delivery-channel patterns (§52.13.4). These are consumption dependencies; no component's specification is modified.

62.7 Summary

Authors can create events, invite readers, and let readers join and replay them through whichever collaboration platform the author already uses — the full capability requested, built entirely on the Source Connector Registry already specified in this Bible rather than hardcoded integrations with named third-party companies. This is a strictly more general outcome: it supports any collaboration platform that registers, not only the ones named when this chapter was written.

Chapter 63: Reader Annotation System

This chapter fully specifies Personal Notes (§52.13.2.1), the reader-facing annotation system referenced only briefly there. It attaches structured notes to any passage, page, diagram, or VR scene, with color-coding, tagging, cross-device sync, and configurable sharing — private by default, extending to classroom or public visibility only when a reader explicitly chooses it.

SCOPE NOTE — REUSES EXISTING STORAGE, SYNC, AND GOVERNANCE **A note is a Fact Unit (Appendix A's schema, not a new object type), synced through the same Provenance Ledger versioning already specified for any Fact Unit (Chapter 5), and governed by the same Safety Substrate and access-control patterns as Social Annotations (§52.13.2.1) and every other reader-generated artifact in this Bible. This chapter defines no new storage layer, no new sync protocol, and no new permissions engine — it defines the note-specific schema and behavior layered on top of what already exists.**

63.1 Note Fact Unit Schema

A note is a Fact Unit carrying the following fields, in addition to the fields every Fact Unit already carries (FUID, InputFUIDs, ConfidenceScore, SafetyFlags, AccessControlList — Appendix A):

Field	Purpose
OwnerID	The reader who created the note; notes are single-owner, not co-authored.
BookID / ChapterID / PageID	The location context, resolved through the same Fact Unit lineage already used for any content reference.

AnchorType	PARAGRAPH, SENTENCE, DIAGRAM, or VR_SCENE — see §63.2.
AnchorPosition	A pointer into the anchored content: a paragraph/sentence offset, a diagram region, or a Tour Waypoint reference (§46.9) for VR_SCENE anchors.
Color	One of a fixed palette (yellow, blue, green, pink, purple); cosmetic only, carries no semantic weight.
Tags	Free-text labels the reader assigns; not a controlled vocabulary, since a note's organization is the reader's own.
Content	The note's text body.
SharingMode	PRIVATE (default), CLASS, GROUP, or PUBLIC — see §63.4.

63.2 Anchoring

A note attaches to one of four anchor types:

- Paragraph or Sentence Anchor — a position within the book's own text Fact Units, resolved the same way any other text-level citation in this Bible is.
- Diagram Anchor — a region within an Image-modality Fact Unit (§4.2).
- VR Scene Anchor — a specific Tour Waypoint (§46.9) or a volumetric object within an IQR scene (Chapter 46), letting a reader annotate a specific moment or object in a 3D experience the same way they would a paragraph in text.

63.3 Sync and Offline Behavior

Notes sync across a reader's devices through the same Provenance Ledger mechanism already specified for versioned Fact Units generally (Chapter 5) — a note edit is a new version in that Fact Unit's VersionChain, not a separate sync protocol. A note created while offline is queued locally and reconciled on reconnection using the same last-write-wins-with-history pattern already implicit in the VersionChain model; a genuine simultaneous edit conflict (the same note edited offline on two devices before either synced) preserves both versions in the chain rather than silently discarding one, consistent with this Bible's general position that provenance history is never destroyed for convenience.

DESIGN TARGET, NOT YET VALIDATED **"Instant" sync, as a latency claim, has not been benchmarked and should not be cited as a performance guarantee until it has — consistent with how this Bible already treats unvalidated latency and throughput figures elsewhere (Appendix C, §C.11; §44.5). The sync mechanism itself (Provenance Ledger versioning) is real and already specified; the specific speed of propagation is not yet a measured number.**

63.4 Sharing and Permissions

A note's SharingMode field determines its visibility, using the same AccessControlList pattern (Appendix A) already governing every other Fact Unit's visibility — not a separate permissions system:

- PRIVATE (default) — visible only to the note's owner. This is not merely a UI default; it is enforced the same way any other access-restricted Fact Unit is, at the Safety Substrate layer (Chapter 8), not by client-side filtering alone.
- CLASS — visible to the owner and the instructor of a class the owner is enrolled in (§52.13, Chapter 62's event and class concepts), for notes explicitly shared into that context.
- GROUP — visible to a reader-defined set of other readers, using the same AccessControlList mechanism scoped to a specific list of identities rather than a class or the public.
- PUBLIC — visible broadly, subject to the same moderation and Safety Substrate review as any other reader-generated content surfaced to other users (Social Annotations, §52.13.2.1, already establishes this precedent).

Two access guarantees are stated explicitly, not left implicit: an author cannot view a reader's private notes about that author's own book, and a publisher cannot view any reader's notes regardless of sharing mode. Both are enforced as AccessControlList exclusions at the Safety Substrate layer, not merely a policy statement — the same enforcement discipline already applied to every other access boundary in this Bible.

63.5 Instructor and Class Integration

Where a note's SharingMode is CLASS, an instructor can view it through the class context established by Chapter 62's event model, and a reader may optionally attach a note to a specific assignment or Comprehension Check (§52.13.2) as supporting work — the note remains the same Fact Unit; attachment to an assignment is an additional reference, not a copy. A CLASS-shared note may be surfaced as a Discussion Mode (§52.13.3) prompt at instructor discretion, reusing that mode's existing bounded-analysis behavior rather than a separate note-to-discussion pipeline.

63.6 What This Chapter Does Not Specify

The source specification for this system referenced sharing notes to “message boards.” Message boards are not yet specified anywhere in this Bible — PUBLIC-mode notes (§63.4) are specified as visible broadly, but a dedicated message-board subsystem (threaded discussion, upvoting, moderation queues beyond the Safety Substrate's existing review) would be new architecture this chapter does not build. If

that capability is wanted, it should be specified on its own terms rather than assumed as a side effect of this chapter.

63.7 Integration Requirements

This chapter requires the Provenance Ledger (Chapter 5), the Safety Substrate (Chapter 8), the Multimodal Ingestion Pipeline (Chapter 4, for Diagram anchors), Tour and Exhibition Mode (§46.9, for VR Scene anchors), and the Interactive Mode Registry's Social Annotations precedent (§52.13.2.1). These are consumption dependencies; no component's specification is modified.

63.8 Summary

Personal Notes are specified here in full: a Fact Unit schema with four anchor types, sync through the existing Provenance Ledger rather than a new protocol, and four sharing modes enforced at the Safety Substrate layer — including an explicit guarantee that neither an author nor a publisher can view a reader's private notes. No new storage, sync, or permissions architecture was introduced; this chapter specifies note-specific behavior on top of what this Bible already has.

Chapter 64: Message Board System

This chapter specifies the reader, author, instructor, and publisher discussion system this Bible was missing — the gap explicitly named as out of scope in Chapter 63 (§63.6). It defines book-level boards, chapter-level boards, class boards, and announcement boards, with threads, posts, moderation, and a direct conversion path from a Personal Note (Chapter 63) into a shareable thread.

SCOPE NOTE — REUSES EXISTING SCHEMA, PERMISSIONS, AND MODERATION **A board, thread, and post are each Fact Units (Appendix A's schema), not new object types. Permissions reuse the AccessControlList pattern already governing every other Fact Unit. Moderation actions route through the Safety Substrate (Chapter 8) the same way any other content-affecting action in this Bible does. This chapter does not build a new Territory Registry — regional board visibility reuses the regional capability concept already established for distribution routing in Chapter 60 (§60.4), scoped down to what a discussion board actually needs, not the full distribution-territory system a much larger, separately-declined specification proposed.**

64.1 Board Types

Every book carries four board types, each a Fact Bundle scoped to the book:

Board Type	Purpose	Who Can Post
Book Board	Global discussion for the entire book.	Any reader with access to the book.
Chapter Board	Discussion scoped to one chapter.	Any reader who has reached that chapter, per the same Progress Monitor tracking already specified (§52.13.2).
Class Board	Discussion scoped to a class created via Chapter 62's event model.	Enrolled readers and the hosting author/instructor.
Announcement Board	Author and publisher posts only; readers can view and react but not start threads.	Author or publisher of the book.

64.2 Thread and Post Schema

A thread is a Fact Unit with a title, tags, a creator, and pinned/locked state. A post is a Fact Unit with content, optional attachments, and a reply relationship to its parent thread or post — the same InputFUIDs lineage mechanism already used to express any Fact Unit relationship (Appendix A), not a new reply-chain data structure. Attachments reuse existing Fact Unit types directly: an inline diagram is an Image-modality Fact Unit (§4.2), an inline VR scene reference is a Tour Waypoint or IQR scene reference (Chapter 46), and an inline Q&A reference points to an existing Interactive Mode Registry Q&A exchange (§52.13.2) rather than duplicating its content into the post.

64.3 Permissions

Board permissions extend the same role concepts already established elsewhere in this Bible — Author (Chapters 61–63), Instructor (Chapter 62's class events), Publisher (Chapter 60), and a new Partner role for the external distribution and vendor connectors already registered under §49.3.3, given limited moderation scope on boards their organization is associated with:

- Readers can create posts, reply, and upvote on any board they have access to.
- Authors can create threads, pin, lock, and moderate on boards for their own books.
- Instructors can moderate Class Boards for classes they host, per Chapter 62's event ownership model.
- Publishers can moderate Announcement Boards for books they publish.
- Partners have moderation limited to content directly referencing their own registered connector or distribution relationship, not general board moderation.

Every permission above is enforced as an AccessControlList rule at the Safety Substrate layer, the same enforcement discipline already applied to Chapter 63's note-sharing modes — not a separate permissions engine.

64.4 Moderation

Flagging, hiding, banning, locking, and archiving are Safety Substrate-gated actions (Chapter 8), logged to the Provenance Ledger (Chapter 5) with the moderating identity and reason, the same audit discipline already required of every other enforcement action in this Bible. A flagged post is held for review, not automatically removed — consistent with how a Safety Substrate finding elsewhere in this Bible surfaces for review rather than silently discarding content (§15.1's AEM chain-confidence disclosure is the same pattern applied to a different kind of content).

64.5 Regional Visibility

A board, or a specific thread, may be scoped to a region — reusing the regional capability declaration already specified for distribution connectors (§60.4), not a new geographic system. This supports the class and organizational use case (a board visible only to a specific institution's readers) without requiring the full territory, pricing, and rights-conflict-resolution machinery a separately-declined specification proposed for distribution generally. If broader territory-based content licensing is wanted later, it should be specified on its own terms against real requirements, not inherited from a board-visibility feature that only needed a narrower slice of the concept.

64.6 Integration with Personal Notes

A reader can convert a Personal Note (Chapter 63) into a board thread or post: the note's content becomes the thread or post body, and its SharingMode (§63.4) governs which board it can be shared to — a PRIVATE note cannot be shared to any board without the reader first changing its SharingMode, closing the gap Chapter 63 explicitly left open (§63.6). The note itself is not duplicated; the resulting thread or post carries an InputFUIDs reference back to the originating note, preserving the same fact-traceability lineage every other derived QueryBook output already carries.

64.7 Integration with Author-Hosted Events

An event created under Chapter 62 automatically gets an associated Event Thread on the relevant Class or Book Board, supporting pre-event questions, live discussion during the event, and a recap thread

afterward — all ordinary threads under this chapter's own model, distinguished only by an EventID reference back to the Chapter 62 event they're attached to, not a separate event-discussion subsystem.

64.8 Search

Board search reuses the same semantic search capability already available for the book's own content (the Associative Recall Engine, §7.2.3) rather than a separate search index — a reader searching for a concept discussed on a board and a reader searching for that concept in the book's own text are served by the same underlying retrieval mechanism, scoped to include or exclude board content as the reader chooses.

64.9 Integration Requirements

This chapter requires the Provenance Ledger (Chapter 5), the Safety Substrate (Chapter 8), the Associative Recall Engine (§7.2.3), Chapter 60's regional capability concept (§60.4), Chapter 62's event model, and Chapter 63's Personal Notes (specifically §63.4's SharingMode field). These are consumption dependencies; no component's specification is modified.

64.10 Summary

Book, chapter, class, and announcement boards are specified here using existing Fact Unit schema, existing permissions enforcement, existing moderation gating through the Safety Substrate, and existing search — with one genuinely new integration point: a Personal Note can now become a shareable thread, closing the gap Chapter 63 left open. No new territory system, no new search index, and no new permissions engine were introduced to build this.

Chapter 65: Portal Access and Role Model

This chapter formalizes something that has existed piecemeal since early in this Bible — a Reader Portal (§36.8), an Author-facing surface (Chapters 46, 61, 62), a Publisher-facing surface (Chapter 60), and a Partner concept (§49.3.3, §64.3) — into one coherent role model. It defines who can access what, and points to where each capability is already specified, rather than re-describing any of it here.

SCOPE NOTE — A ROLE MODEL, NOT A NEW ACCESS ENGINE This chapter introduces no new authentication mechanism, no new permissions engine, and no new portal UI framework. Every access decision it describes is enforced through the AccessControlList pattern (Appendix A) and the Safety Substrate (Chapter 8) already governing every other Fact Unit and action in this Bible. What this chapter adds is the vocabulary and the map: which role gets which access, and which existing chapter specifies the actual capability.

65.1 License Owner as the Umbrella Identity

Every authenticated identity in QueryBook is a License Owner — the holder of a license to some QueryBook product, whether that license grants reading access, authoring tools, publishing tools, or a registered distribution relationship. A License Owner's specific access is determined by one or more Roles (§65.2) attached to their identity, not by License Owner status alone; holding a license is necessary but not sufficient for any specific capability — the Role determines what that license actually unlocks.

65.2 The Four Roles

Role	Primary Portal Surface	Specified In
Reader	Reader Portal	§36.8 (History Interface: Timeline, By-Book, Thematic Map, and Annotation Library views, ZKE-respecting); Chapter 46 (IQR); Chapter 63 (Personal Notes); Chapter 64 (Message Boards, reader-level permissions)
Author	Author Portal	Chapter 61 (Drafting Tools); §46.10 (Presenter Mode, subject to Author Authorization); Chapter 62 (Author-Hosted Events); §64.3 (thread/board moderation on own books)
Publisher	Publisher Portal	Chapter 60 (Submission Intake, Rights Validation Gate, Print and Distribution); §64.3 (Announcement Board moderation)
Partner	No dedicated portal — connector-scoped access only	§49.3.3 (Source Connector Registry — registration, not a UI surface); §64.3 (moderation limited to content referencing the partner's own connector)

A single License Owner may hold more than one Role — an author is also a reader of other authors' books, for instance — and each Role's access is evaluated independently through the same AccessControlList mechanism, not merged into a combined permission set that could accidentally grant more than either Role alone would.

65.3 Instructor Is a Contextual Capability, Not a Fifth Role

An “instructor” in this Bible is a Reader or Author who has been granted event-hosting privileges for a specific class, through Chapter 62's event model — not a separate top-level Role with its own portal. This is a deliberate scoping decision: a class-hosting capability is naturally scoped to the class it applies to (an instructor for one class is not automatically an instructor for another), which fits Chapter 62's existing per-event ownership model directly, rather than requiring a fifth Role whose access would need the same per-class scoping bolted on separately.

65.4 Cross-Portal Consistency

Three principles hold across all four Roles, stated once here rather than repeated in every chapter that references a Role:

- Every access decision is an AccessControlList evaluation (Appendix A), enforced at the Safety Substrate layer (Chapter 8) — there is no portal-specific bypass, and no Role receives elevated trust outside what its specific, documented capabilities grant.
- Every action a Role takes — an author moderating a thread, a publisher approving a submission, a partner registering a connector — is logged to the Provenance Ledger (Chapter 5) with the acting identity, the same audit discipline already required of every other consequential action in this Bible.
- A Role's access to another License Owner's private data is governed by the same explicit exclusions already established where they matter most: an author cannot view a reader's private notes, and a publisher cannot view any reader's notes regardless of sharing mode (§63.4), stated there as Safety Substrate enforcement and unchanged by this chapter's broader framing.

65.5 What This Chapter Deliberately Does Not Specify

This chapter does not define a portal UI layer, a specific front-end framework, or a unified login flow — those are implementation choices below the level this Bible specifies elsewhere (compare Chapter 9's Semantic Transport Layer, which specifies the API contract, not a specific client). It also does not introduce license tiers, pricing, or entitlement rules beyond what already exists; “License Owner” in §65.1 is an identity concept, not a billing specification.

65.6 Integration Requirements

This chapter requires the AccessControlList pattern (Appendix A), the Safety Substrate (Chapter 8), the Provenance Ledger (Chapter 5), and every chapter referenced in §65.2's table. It modifies none of their specifications — it is a map of what already exists, not new architecture layered on top.

65.7 Summary

Four Roles — Reader, Author, Publisher, Partner — sit under the License Owner identity concept, each mapped to the portal surface and chapters that already specify their actual capabilities. Instructor status is a contextual, per-class capability rather than a fifth Role. No new access engine, portal framework, or licensing model was introduced; this chapter is the map connecting portal surfaces that were previously specified independently across Chapters 36, 46, 49, 60, 61, 62, 63, and 64.

Appendix H: Representative Runtime Flows

This appendix formalizes, as step-by-step flows, how six already-specified operations actually execute end to end: a Q&A response, a Personal Note (Chapter 63), a Message Board thread (Chapter 64), an Author-Hosted Event (Chapter 62), a Tour with Presenter Mode (Chapter 46), and the Safety Substrate gating pattern every one of them shares. Every step below names a real component already specified elsewhere in this Bible — no UFCS, no FQL, no blockchain layer, and no Territory Service beyond the narrow regional capability concept Chapter 60 already defines.

SCOPE NOTE — DOCUMENTATION, NOT NEW ARCHITECTURE **This appendix introduces no new component, no new engine, and no new data structure. It is a cross-cutting reference showing how components already specified in Chapters 4–65 and Appendix A interoperate at runtime, written for anyone implementing against this Bible who needs the sequence, not just the individual specifications.**

H.1 Q&A Response Flow

Grounded in the real five-stage ARE Pipeline (§52.8), not a generic “semantic engine.”

- Reader submits a query through the Interactive Mode Registry's Standard Q&A mode (§52.13.2).
- The Associative Recall Engine (§7.2.3) retrieves a Fact Unit result set from the Fact Graph.
- ARE Stage 1, Result Set Analysis (§52.8.1): builds a Result Set Profile from the retrieved facts' entity types, confidence distribution, and special annotations.

- ARE Stage 2, Discipline Mode Template Selection (§52.8.2): selects the applicable Discipline Mode template from QBCL-CMS (§47.3).
- ARE Stage 3, Expertise Level Calibration (§52.8.3): produces a Vocabulary Calibration Specification for the resolved Expertise Level.
- Stages 4–5 complete the Generation Directive, which the Narrative Graph (Chapter 12) uses to compose the response text, citing the originating Fact Units per the Factual Groundedness metric (Chapter 16).
- The response is returned to the reader; if the underlying Fact Units support it, an IQR scene offer (§46.2) may accompany the text response.

H.2 Personal Note Creation and Sync Flow

Grounded in Chapter 63's actual schema, not a generic “notes engine.”

- Reader selects an anchor (§63.2): a paragraph, sentence, diagram, or a Tour Waypoint reference for a VR Scene anchor.
- A Note Fact Unit is constructed per the §63.1 schema (OwnerID, AnchorType, AnchorPosition, Color, Tags, Content, SharingMode) and written to the Fact Graph with SharingMode defaulted to PRIVATE.
- The write is versioned through the Provenance Ledger's VersionChain (Chapter 5), the same mechanism used for any Fact Unit — there is no separate note-sync protocol.
- On another device, the reader's client resolves the current VersionChain head for that Fact Unit and displays the latest version; an offline edit reconciles on reconnection per §63.3's conflict-preservation rule.
- If the reader later changes SharingMode away from PRIVATE, the note becomes eligible for the Message Board conversion path in §H.3.

H.3 Message Board Thread Creation Flow

Grounded in Chapter 64's actual permission and moderation model.

- A thread or post is created directly, or a reader converts a non-PRIVATE Personal Note into one (§64.6) — the resulting Fact Unit carries an InputFUIDs reference back to the source note, per Appendix A's lineage pattern.

- The AccessControlList evaluation (Appendix A) checks the acting identity's Role (Chapter 65) against the target board's permission rules (§64.3) before the write is accepted.
- If accepted, the thread or post Fact Unit is written and becomes visible per the board's own AccessControlList scope (book-wide, chapter-scoped, class-scoped, or announcement-only).
- A moderation action (flag, hide, lock — §64.4) is itself a Safety Substrate-gated write, logged to the Provenance Ledger with the moderating identity and reason — see the shared pattern in §H.6.

H.4 Author-Hosted Event Flow

Grounded in Chapter 62's actual Source Connector model, not hardcoded platform APIs.

- Author selects a registered Collaboration Connector (§62.2) — previously registered through the Source Connector Registry, §49.3.3 — and specifies event type, time, and scope.
- The Event Fact Unit is created and provenance-tracked (Chapter 5) exactly as any other authored content.
- Readers are notified per the Interactive Mode Registry's delivery-channel patterns (§52.13.4) or discover the event via the Event Calendar (§62.3).
- On join, the connector's own platform generates the actual session join link (§62.2) — QueryBook brokers scheduling and invitation, not the live session itself.
- After the event, if the connector's platform provides one, a recording reference (not the media itself) is attached to the Event Fact Unit (§62.4), and Chapter 64's automatic Event Thread (§64.7) becomes available for recap discussion.

H.5 Tour Playback with Presenter Mode Flow

Grounded in the real Chapter 46 authorization chain, not an unconsented avatar system.

- Reader selects a Tour (§46.9); each Waypoint's IQR scene (Chapter 46) is resolved from the Fact Units grounding that Waypoint's topic.
- If the Tour specifies a Presenter Mode (§46.10), the Safety Substrate gate checks for a matching Author Authorization Record (§46.10.1) scoped to that specific presenter mode before any presenter content generates — this check is non-bypassable (§46.10, Governance Note).
- If no Authorization Record exists, or it has been revoked (§46.10.2), the Tour falls back to environment-and-narration-only playback — the same fallback already specified as the no-presenter default.

- If authorized, the presenter renders per the selected mode (Voice-Only, Face-Only, or Face-on-Avatar) with the mandatory, non-removable AI-generated disclosure (§46.10.6) always present.
- Waypoint advancement follows the camera/pacing specification (§46.9.4); a multi-viewer session (§46.9.6) synchronizes advancement across all connected readers under the session controller's control.

H.6 The Shared Safety Substrate Gating Pattern

Every flow above shares one enforcement pattern, stated once here rather than repeated in each section: a consequential write or generation event (a moderation action, a presenter-mode generation, a rights validation, a configuration change) is evaluated against the Safety Substrate (Chapter 8) before it takes effect, not after. A rejected action returns a specific reason to the caller (§47.5's pattern, generalized), and every accepted consequential action is logged to the Provenance Ledger (Chapter 5) with the acting identity, timestamp, and reason — the same audit discipline Chapter 65 (§65.4) already states as a cross-portal principle.

Flow	Safety Substrate Gate	Fallback if Denied
Q&A Response (H.1)	Discipline Mode / Response Profile boundary checks (§52.7)	Response scoped to permitted Discipline Modes / Expertise Levels only
Note Sharing (H.2)	AccessControlList SharingMode enforcement (§63.4)	Note remains PRIVATE
Board Post (H.3)	AccessControlList Role permission check (§64.3)	Write rejected with reason
Event Creation (H.4)	Source Connector authentication and capability check (§49.3.3)	Event creation blocked until connector re-authorized
Presenter Mode (H.5)	Author Authorization Record check (§46.10.1)	Falls back to environment-only playback

H.7 Summary

These six flows show how Chapters 4–65 and Appendix A's components actually interoperate at runtime: a five-stage ARE pipeline for Q&A, Provenance Ledger versioning for note sync, AccessControlList evaluation for board permissions, the Source Connector Registry for events, and the Author Authorization Record for presenter generation — with one shared Safety Substrate gating pattern underneath all of them. No new engine, protocol, or data structure was introduced to produce this appendix.

Chapter 66: Improvement Proposal Pipeline

This chapter specifies how architectural and performance improvements to QueryBook itself are identified, drafted, tested, and deployed — the useful part of an autonomous self-evolution proposal declined earlier in this Bible's history, rebuilt with the one property that proposal was missing: a human makes the actual decision to ship, every time, with no path around that gate.

SCOPE NOTE — A PROPOSAL PIPELINE, NOT A SELF-MODIFYING SYSTEM **Every stage of this pipeline up to and including deployment approval (§66.5) may be assisted by automated tooling — generating a candidate proposal, running it in the sandbox, computing a fitness score. No stage after approval is autonomous, and no proposal may bypass §66.5's human review regardless of its fitness score, risk profile, or urgency. §66.8 lists what this pipeline may never modify, permanently, not as a configurable policy.**

66.1 Candidate Identification

Improvement candidates surface from the same telemetry and audit data this Bible already collects — not a new monitoring subsystem. Query latency and error-rate data (Chapter 9's Semantic Transport Layer), ingestion throughput (Chapter 4), and Provenance Ledger audit patterns (Chapter 5) are reviewed for recurring bottlenecks or failure modes, producing a Candidate Record: what's underperforming, by how much, and against what target.

66.2 Proposal Schema

A Proposal Fact Unit, addressing one Candidate Record, contains:

Field	Contents
Scope	Which existing component the proposal modifies — named by chapter and section, not a free-text description.
Change Description	What changes and why, in terms of the existing specification, not a rewrite of it.
Risk Assessment	What could go wrong, including specifically whether the change touches anything in §66.8's exclusion list — a proposal that does is rejected at intake, before simulation.
Performance Prediction	Expected effect on the metric named in the originating Candidate Record.
Rollback Plan	How to revert if the change underperforms or misbehaves after deployment.

66.3 Simulation Sandbox

A proposal is evaluated in an isolated environment before any human reviews it — not to approve it, only to generate the evidence a human reviewer needs:

- Synthetic and replayed historical workloads test the proposal's actual behavior against its Performance Prediction (§66.2).
- Regression tests confirm existing behavior elsewhere in the system is unaffected.
- The sandbox produces a Fitness Score and a Risk Profile — inputs to the human review in §66.5, never a substitute for it.

The sandbox is isolated from production data and from any system capable of modifying this Bible's actual specification — a simulation result is evidence, not an edit.

66.4 Automated Pre-Screening

Before a proposal reaches human review, an automated check confirms it is eligible for review at all:

- Does the proposal's Scope (§66.2) touch anything in §66.8's exclusion list? If so, the proposal is rejected automatically, with the specific excluded component named in the rejection — this check cannot be overridden by a high Fitness Score.
- Does the proposal have a complete Rollback Plan? An incomplete proposal is returned for revision, not queued for review.

Pre-screening narrows what a human reviewer sees to proposals that are actually eligible for approval — it does not replace their judgment on the proposals that pass through.

66.5 Human Review Gate

THIS GATE HAS NO AUTOMATED PATH THROUGH ITA named, accountable human reviewer evaluates every proposal that passes §66.4's pre-screening, using the Fitness Score and Risk Profile from §66.3 as evidence, not as a decision. The reviewer's approval or rejection is recorded in the Provenance Ledger (Chapter 5) with their identity and reasoning — the same audit discipline already required of every other consequential action in this Bible (Appendix H, §H.6). There is no fitness threshold, urgency justification, or proposal category that permits skipping this gate. A rejected proposal may be revised and resubmitted; it may not be re-scored and auto-approved.

66.6 Deployment

An approved proposal deploys through the existing staged-rollout pattern already implicit in this Bible's deployment architecture (Chapter 19) — not a new deployment mechanism. A canary or limited-exposure rollout precedes full deployment, and the Rollback Plan from §66.2 is the defined path back if post-deployment monitoring (§66.7) surfaces a regression the sandbox didn't catch.

66.7 Monitoring and Feedback

After deployment, the same telemetry sources named in §66.1 continue to be observed against the approved proposal's Performance Prediction. A significant deviation — the change underperforming its prediction, or a regression the sandbox missed — becomes a new Candidate Record (§66.1), closing the loop without any step in that loop being autonomous.

66.8 Permanent Exclusions

The following are never valid Proposal Scopes (§66.2), under any Fitness Score, Risk Profile, or reviewer judgment. This is not a configurable policy — changing what belongs on this list is itself outside what this pipeline can do to itself:

- The Safety Substrate (Chapter 8) and its enforcement points.
- The Prime Directive (Domain 0).
- This pipeline's own Human Review Gate (§66.5) and Permanent Exclusions list (this section).
- The Author Authorization Record, revocation, and disclosure requirements for Presenter Mode (§46.10).
- The private-note and no-publisher-access guarantees for reader annotations (§63.4).

A system capable of proposing changes to its own safety constraints or its own approval gate was the specific reason an earlier, fully autonomous version of this pipeline was declined in this Bible's history. This section exists so that outcome remains structurally impossible here, not merely discouraged.

66.9 Integration Requirements

This chapter requires the Provenance Ledger (Chapter 5), the Safety Substrate (Chapter 8), the Semantic Transport Layer (Chapter 9), and the deployment architecture (Chapter 19). It modifies none of their specifications, and §66.8 explicitly forbids it from ever doing so.

66.10 Summary

Candidate identification, structured proposals, sandboxed simulation, and staged deployment are all specified here — the useful engineering pattern from a declined autonomous self-evolution proposal, rebuilt around one non-negotiable property: a named human approves every change before it ships, and a permanent, self-referential exclusion list keeps the Safety Substrate, the Prime Directive, and this pipeline's own gate outside what any proposal can ever target.

Chapter 67: License and Entitlement Management

This chapter closes the gap Chapter 65 explicitly declined to address: what a License Owner's license actually entitles them to, and how seat-based access for organizations is tracked. It is deliberately narrow — no territory, no pricing, no DRM, and no rights-tokenization mechanism.

SCOPE NOTE — ENTITLEMENT ONLY, DELIBERATELY NARROW This chapter covers what a license unlocks and how many seats it covers. It does not cover where a license is valid geographically, how much it costs, offline access expiration windows, or any rights-representation mechanism beyond the AccessControlList this Bible already specifies (Appendix A). Two separate submissions proposing a full Territory/DRM/Pricing system, and a third proposing NFT-based rights tokenization, were declined during this Bible's development specifically to keep this boundary — this chapter does not reopen either.

67.1 License Types

A License Fact Unit is issued per License Owner (Chapter 65) or per organization, and carries a LicenseType matching one of the four Roles already specified: Reader License, Author License, Publisher License, or Partner License. A License does not grant a Role directly — it authorizes one; the Role itself, and what it can do, remains fully specified by Chapter 65 and every chapter it references. This chapter adds nothing to what a Role can do; it only governs whether a given identity currently holds one.

67.2 Seat Management

An organizational License carries a SeatCount and a SeatAllocation list — which specific identities currently hold one of the organization's seats for a given LicenseType. Seat assignment and

reassignment are AccessControlList-governed actions (Appendix A) logged to the Provenance Ledger (Chapter 5), the same audit discipline as any other consequential write in this Bible.

- An identity without an assigned seat cannot exercise the Role that License Type would otherwise authorize, regardless of any Role-level permission that identity might otherwise hold from a different License.
- Seat reassignment does not retroactively affect Provenance Ledger history — content created while a seat was held remains attributed to that identity, consistent with this Bible's general position that provenance is never rewritten (§63.3's conflict-preservation principle, applied here to seat history).

67.3 Entitlement Enforcement

An Entitlement is the specific set of features a LicenseType unlocks beyond the Role's baseline — for example, an organizational Reader License might entitle its seats to Class Board participation (§64.1) that an individual Reader License does not, if the organization has not registered as a class host. Entitlement checks are AccessControlList evaluations layered on top of the Role check already specified in Chapter 65 (§65.4) — a request must pass both the Role's baseline permission and, where applicable, the License's specific Entitlement, not one in place of the other.

67.4 Usage Analytics

Usage Analytics in this chapter means aggregate, license-scoped counts — active seats, feature-entitlement utilization — derived from the Provenance Ledger's existing audit trail (Chapter 5), not a new telemetry collection mechanism. This is administrative reporting for the license holder (an organization's administrator, per Chapter 65's Role model), not behavioral tracking of individual readers beyond what the Provenance Ledger already records for provenance purposes.

67.5 Integration Requirements

This chapter requires the Role model (Chapter 65), the AccessControlList pattern (Appendix A), and the Provenance Ledger (Chapter 5). It modifies none of their specifications.

67.6 Summary

License types map directly to Chapter 65's four Roles; seats gate which identities can exercise a Role under an organizational license; Entitlements layer specific feature access on top of Role-level

permissions; usage analytics are aggregate counts from the existing Provenance Ledger. No territory, pricing, or rights-tokenization concept was introduced to close this gap.

Chapter 68: Synchronization and Offline Operation Layer

This chapter specifies consistent operation across devices, nodes, and environments including high-latency networks, intermittent connectivity, fully offline deployments, and institution-level replication, per the source specification provided directly for this chapter.

SCOPE NOTE — REUSES EXISTING VERSIONING AND ACCESS CONTROL Consistency in this chapter is built on the Provenance Ledger's existing VersionChain mechanism (Chapter 5), not a new versioning system. Offline access control reuses the AccessControlList and Role model already specified (Chapter 65, Appendix A). This chapter adds synchronization behavior and offline capability scoping on top of what already exists.

68.1 Synchronization Modes

Mode	Behavior
Real-Time Sync	Changes propagate immediately when connectivity is available, using the same Provenance Ledger write path as any online operation.
Deferred Sync	Changes queue locally and propagate on the next connectivity window, at a client-controlled interval rather than immediately.
Batch Sync	Changes accumulate and propagate as a single batched write, reducing round-trips for high-latency or metered connections.
Air-Gapped Sync	Synchronization occurs via physical media or a manually-triggered transfer between two disconnected environments — no network path is assumed at all.

68.2 Consistency Mechanisms

Consistency across concurrent, disconnected edits is maintained through four mechanisms, layered on the existing VersionChain (Chapter 5) rather than replacing it:

- Version Vectors — each device tracks the last-known VersionChain state it has synced, per Fact Unit, to detect divergence before merge rather than after.

- CRDT-Based Merges — for Fact Unit fields where concurrent edits can be merged without conflict (for example, independently-added Tags on a Personal Note, §63.1), a conflict-free replicated data type merge rule combines both edits rather than requiring either device to overwrite the other.
- Deterministic Merge Rules — for fields where a CRDT merge isn't meaningful (for example, a Content field edited differently on two devices), a documented, deterministic rule — not an arbitrary or random tiebreak — determines the surviving version, with the losing version preserved in the VersionChain per §63.3's existing conflict-preservation principle, not discarded.
- Provenance-Verified Checkpoints — a sync operation validates the incoming VersionChain segment's integrity against the Provenance Ledger's existing lineage verification (Chapter 5) before accepting it, the same integrity check already applied to any Fact Unit write.

68.3 Offline Capabilities

A device operating without connectivity retains the following, scoped to whatever Fact Units it has already synced locally:

- Full Querying — the Associative Recall Engine (§7.2.3) operates against the locally-cached Fact Graph subset; results are scoped to what's been synced, not the full remote Fact Graph.
- Provenance Verification — a locally-cached Fact Unit's lineage remains checkable against its cached VersionChain segment, without requiring a live connection to the Provenance Ledger's authoritative copy.
- RBAC Enforcement — the AccessControlList and Role checks (Chapter 65, Appendix A) evaluate against the last-synced permission state; a permission revoked remotely takes effect locally on next sync, not instantly, which is a deliberate, stated latency — not a security gap being hidden.
- Template Execution — QBPL programs (Chapter 48) already specified as running against the Fact Graph execute against the local cached subset.
- Media Rendering — cached Fact Units of any modality (Chapter 4) render locally; IQR scenes (Chapter 46) render from whatever Fact Units and assets have already synced.
- Edition Comparison — the existing Edition Comparison mode (§52.13.2) operates against locally-cached editions; comparison against an edition not yet synced is unavailable until connectivity resumes.

68.4 Integration Requirements

This chapter requires the Provenance Ledger (Chapter 5), the Associative Recall Engine (§7.2.3), the Role and AccessControlList model (Chapter 65, Appendix A), QBPL (Chapter 48), and the Multimodal Ingestion Pipeline (Chapter 4). It modifies none of their specifications — it defines how they behave when disconnected, not what they do.

68.5 Summary

Four synchronization modes and four consistency mechanisms are specified here, all layered on the existing Provenance Ledger VersionChain rather than a new versioning system, and six offline capabilities are scoped to what a device has already synced — querying, provenance verification, access control, template execution, media rendering, and edition comparison all continue to function offline, against local state, with sync resolving divergence per §68.2's deterministic rules on reconnection.

Chapter 69: Processor Slice Measurement Model

This chapter specifies the Processor Slice: a bounded, deterministic unit of computation used to measure QueryBook operations, replacing opaque, model-internal token counting with something auditable and event-aligned. It is a genuinely useful idea drawn from an external submission, rebuilt to name only real QueryBook subsystems and to feed only real QueryBook systems — not the UFCS/FQL naming or the autonomous deployment mechanism the source material assumed.

SCOPE NOTE — A MEASUREMENT UNIT, NOT A NEW SUBSYSTEM OR AN AUTHORITY **A Processor Slice measures work that already happens in already-specified chapters; it does not do the work itself, and it grants no chapter new capability. Slice telemetry feeds Chapter 67's Usage Analytics (§67.4) as the actual measurable unit behind aggregate usage counts, and feeds Chapter 66's Candidate Identification (§66.1) as one more telemetry source — exactly like the existing sources already named there. A slice, however numerous, poorly-performing, or high-priority, has no path to trigger a proposal, approval, or deployment on its own; Chapter 66's Human Review Gate (§66.5) is unaffected by anything in this chapter.**

69.1 Slice Definition

A Processor Slice is a single event's measured lifecycle: a bounded execution window, a fixed memory envelope, and four recorded phases — start, run, commit, log. A slice is atomic with respect to

measurement: it either completes and logs a result, or it fails and logs a failure; there is no partial, unmeasured state. This is a measurement property, not a new execution guarantee — it does not change how any existing chapter's operations actually execute, only how their cost and duration are recorded.

69.2 Slice Categories

Slice categories are named for the real chapter whose operations they measure, not for a parallel system:

Category	Measures Operations In	Example
Ingestion Slice	Multimodal Ingestion Pipeline (Chapter 4)	One document's Stage 3–4 processing (semantic chunking, entity resolution).
Reasoning Slice	NCL (Chapter 7) and AEM (Chapter 15)	One Associative Recall Engine retrieval; one AEM inference chain.
Response Generation Slice	The ARE Pipeline (§52.8)	One query's five-stage Generation Directive construction.
Rendering Slice	IQR (Chapter 46) and VCUM (Chapter 44)	One scene composition; one frame's material/lighting synthesis.
Safety Gate Slice	Safety Substrate (Chapter 8)	One enforcement-point evaluation — the same gate pattern named in Appendix H (§H.6).
Sync Slice	Synchronization and Offline Operation Layer (Chapter 68)	One CRDT merge; one provenance-verified checkpoint validation (§68.2).

This is a representative set, not an exhaustive enumeration — any chapter with a discrete, boundable operation can define its own Slice Category following this same pattern (a bounded window, a fixed envelope, the four-phase lifecycle) without this chapter's specification needing to be modified to add one.

69.2.1 Representative Event Triggers

The following events trigger a slice of the Category named, extending §69.2 down to the event level. Two further categories — Scheduler Slice and Recovery Slice — are named here for the first time; §69.2 already anticipated that additional categories could be added without modifying that section, and these two only become meaningful once specific triggering events are named.

Event	Slice Category Triggered
Document upload; multi-document batch ingest; Fact Synthesis start	Ingestion Slice
Query submitted; Associative Recall Engine retrieval; Fact validation	Reasoning Slice

Generation Directive construction (any ARE Pipeline stage, §52.8)	Response Generation Slice
Diagram or IQR scene render; Tour Waypoint entry (§46.9)	Rendering Slice
Permission check; Safety Substrate enforcement-point evaluation	Safety Gate Slice
Multi-viewer sync tick (§46.9.6); CRDT merge; checkpoint validation (§68.2)	Sync Slice
A slice allocation or queueing decision under system load	Scheduler Slice
Rollback Plan execution (§66.2); offline conflict resolution (§68.2)	Recovery Slice

69.2.2 Improvement Proposal Pipeline Events

Four events from the Improvement Proposal Pipeline (Chapter 66) are measurable, but deliberately do not receive their own Slice Categories — each is an instance of a category already named in §69.2, applied within a Chapter 66 stage, not a new kind of work:

- Performance anomaly detected — a Reasoning Slice pattern feeding Candidate Identification (§66.1), per §69.6.
- Agent or reviewer proposes an improvement — the analysis work constructing a Proposal (§66.2) is measured as a Reasoning Slice; the Proposal Fact Unit itself is not a slice, it is what a slice's work produces.
- Sandbox simulation runs — measured as whichever Slice Categories the simulated operation would itself use (an ingestion change simulated in the sandbox produces Ingestion Slices, a rendering change produces Rendering Slices, and so on) — the Simulation Sandbox (§66.3) is where the measurement happens, not a distinct category of work.
- Fitness Score computed — a Reasoning Slice, evaluating the sandbox's output against the proposal's Performance Prediction (§66.2).

ONE EVENT DELIBERATELY EXCLUDED FROM THIS TABLE“Deployment rollout” is not listed above, and does not trigger a slice of any category on its own. An event directly triggering a deployment is the same gap §69.6 and §66.8 already close from the other direction — whether an autonomous mechanism is called a “Slice Category” or an “event trigger” does not change what it would do. A deployment remains reachable only through Chapter 66's full pipeline: a Candidate Record, a Proposal, sandbox evaluation, automated pre-screening, and a named human's approval at the Human Review Gate (§66.5). Once approved, the deployment itself (§66.6) is measured as ordinary Rendering, Ingestion, or Reasoning Slices depending on what the deployed change actually touches — the same categories every other operation uses, with nothing about deployment status granting it a special, event-triggered path of its own.

69.2.3 Slice Assignment Process

An event's Slice Category (§69.2, §69.2.1, §69.2.2) is assigned through a deterministic three-step process, formalizing what was previously left as an implied table lookup:

- Event Classification — the firing event is categorized by which existing chapter's operation it belongs to (the same classification §69.2.1's table already performs, restated here as a defined step rather than a lookup).
- Profile Resolution — the classified event's expected execution envelope (bounded time window, memory envelope) is resolved from the Slice Category's own definition (§69.1, §69.2); this step reads an existing definition, it does not compute a new one per event.
- Category Assignment — the resolved profile determines the final Slice Category. Given the same event under the same system conditions, this assignment is deterministic and reproducible — the same property §69.1 already requires of a slice's measurement, extended here to the assignment step that precedes it.

69.2.4 Priority Ordering

Where system load creates contention between slices awaiting execution, priority is resolved in a fixed order, not a configurable or agent-influenced one:

- Safety Gate Slices (§69.2) execute first, always — an operationalization of the Safety Substrate's existing “embedded at every layer” principle (§8.1.1) applied to scheduling contention specifically, not a new safety guarantee.
- Recovery Slices (§69.2.1) execute next, ahead of ordinary work — a fault or an offline conflict needing resolution (§68.2) should not queue behind routine operations.
- All other Slice Categories execute in the order the Scheduler Slice (§69.2.1) admits them, with no category above Ingestion, Reasoning, Response Generation, Rendering, or Sync receiving standing priority over another.

NO AGENT-OVERRIDE CHANNEL IN THIS PROCESS **The assignment process in §69.2.3 and the priority order in §69.2.4 admit no override input from an agent, a proposal, or any other actor — assignment and priority are functions of the event and current system load only. A mechanism allowing an agent to supersede scheduler defaults at runtime was considered and excluded: whatever it is called, a live channel for an agent to unilaterally change how the system executes is the same class of authority §66.5's Human Review Gate exists to prevent, and §69.6 already establishes that slice telemetry only ever reaches system behavior through Chapter 66's full pipeline, never directly.**

69.3 Slice Logging

A completed slice writes a log entry — category, duration, memory envelope, and the specific Fact Units or chapter operation it affected — to the Provenance Ledger (Chapter 5), using the same write path as any other audited action in this Bible. This chapter does not define a new telemetry store; slice logs are Provenance Ledger entries, queryable the same way any other provenance record is.

69.4 Slice Aggregation

A composite operation — ingesting a long document, composing a multi-Waypoint Tour (§46.9) — is naturally composed of many individual slices. An Operation Profile aggregates the slices belonging to one composite operation into a single summary (total duration, peak memory, slice count by category) for reporting purposes, without collapsing the individual slice log entries themselves — the detailed history remains in the Provenance Ledger even after aggregation.

69.5 Integration with Usage Analytics

Chapter 67's Usage Analytics (§67.4) is specified as aggregate counts derived from the Provenance Ledger's existing audit trail. Slice logs (§69.3) are the specific, granular records that trail is built from where compute measurement is the relevant dimension — an organization's administrator reviewing Usage Analytics is, in practice, reviewing aggregated slice data. This is the connection the source material called “billing”; this Bible does not specify pricing (§67's scope note is unchanged by this chapter), only the measurement layer underneath it, should an operator choose to build pricing on top.

69.6 Integration with the Improvement Proposal Pipeline

Slice logs aggregated across many operations are one input to Candidate Identification (§66.1) — a Reasoning Slice category consistently running long, for instance, is exactly the kind of recurring bottleneck §66.1 already looks for in existing telemetry. This chapter adds a data source to that existing process; it does not add a new approval path. A slice-derived Candidate Record still becomes a Proposal (§66.2), still passes through the Simulation Sandbox (§66.3) and automated pre-screening (§66.4), and still requires a named human's approval at the Human Review Gate (§66.5) before anything changes — identical to a Candidate Record from any other source.

69.7 What This Chapter Does Not Specify

- No UFCS, FQL, or “token” concept — slices measure operations already specified against the Fact Unit and Fact Graph architecture this Bible actually has.
- No autonomous deployment authority — unlike the source material's “Deployment Slice,” nothing in this chapter executes a canary rollout, shadow traffic exposure, or auto-rollback on its own. Deployment remains exactly as specified in §66.6: staged, and only after human approval.
- No pricing model — this chapter specifies measurement, not cost. What an operator charges for compute, if anything, is outside this Bible's scope, consistent with §65.5 and §67's scope notes.

69.8 Integration Requirements

This chapter requires the Provenance Ledger (Chapter 5), Usage Analytics (§67.4), and Candidate Identification (§66.1). It modifies none of their specifications.

69.9 Summary

The Processor Slice replaces opaque, model-internal token counting with a bounded, four-phase, auditable measurement unit, logged through the existing Provenance Ledger rather than a new telemetry store, feeding Usage Analytics as the real unit behind aggregate usage counts and feeding the Improvement Proposal Pipeline as one more Candidate Identification input — with no path around that pipeline's human approval requirement. The genuinely useful idea in the source material survives; the UFCS naming, the autonomous deployment authority, and the unverifiable patent references do not.

Chapter 70: Precision Color and Geometric Registration

This chapter specifies device-independent color accuracy and constraint-based geometric alignment for authored visual content — diagrams, figures, UI elements, and VR objects — so that an author's chosen color and layout render within a stated tolerance across display, print, and VR output, rather than drifting between rendering modes or revisions. It is built from a real, useful submission, rebuilt on Fact Units rather than UFCS, and using this Bible's existing query and retrieval architecture rather than an invented query language.

SCOPE NOTE — DISTINCT FROM, NOT A DUPLICATE OF, VCUM AND WASMVCUM's Semantic Colorization Engine (§44.3.1) synthesizes plausible color for content that never had reliable color information — restoring old footage. This chapter's color matching is the opposite problem: an author already specified an exact color, and the system must reproduce it within a stated tolerance across every output mode, not infer or synthesize it. Where this chapter's calibration needs actual color synthesis (deriving a color from context), it reuses VCUM's SCE directly rather than duplicating it. Separately, WASM's Object Layer (§53.3) models conceptual spatial relationships ("cup on table") for world understanding; this chapter's geometry is production-grade rendering precision (exact coordinates, tolerances in millimeters or fractions of a degree) for a diagram or VR object, a different granularity serving a different purpose. Neither existing system is modified by this chapter.

70.1 Color Fact Units

A Color Fact Unit is a Fact Unit (Appendix A's base schema) carrying color-specific fields:

Field	Purpose
ColorSpace	Lab, LCH, XYZ, sRGB, AdobeRGB, or DisplayP3 — the device-independent or device-referenced space the Values field is expressed in.
Values	The numeric color coordinates in the declared ColorSpace.
ICCProfileRef	Optional reference to an ICC profile Fact Unit for device-specific rendering.
DeltaEThreshold	Maximum allowed CIEΔE2000 deviation before a rendering is considered non-compliant.
RenderingIntent	Perceptual, RelativeColorimetric, AbsoluteColorimetric, or Saturation — governs out-of-gamut handling.
Locked	When true, the color cannot be modified without an explicit, logged override — an AccessControlList-governed write like any other, not a special mechanism.

70.2 Geometry Fact Units

A Geometry Fact Unit carries production-precision spatial fields, distinct from WASM's conceptual spatial relations (§53.3.2):

- Position, Rotation, and Scale — exact coordinates in the declared coordinate system (Cartesian2D, Cartesian3D, Polar, or Spherical).
- Anchors — references to other Geometry Fact Units this one's position is defined relative to.
- Tolerance — the maximum allowed positional or rotational drift before the geometry is considered non-compliant, expressed in the same units as Position (for example, millimeters for print, meters for VR).

- Locked — the same AccessControlList-governed override pattern as §70.1's Color Fact Units.

A Constraint Fact Unit expresses a relationship between two Geometry Fact Units — Align, Center, Pin, LockTo, or RelativeTo — with a priority value used to resolve conflicts when multiple constraints apply to the same geometry.

70.3 Color Calibration

Calibration runs as a sequence of Fact Unit transformations, each stage producing a new, queryable Fact Unit rather than an opaque intermediate value:

- Profile Resolution — the target device's ICC profile is resolved and the Color Fact Unit's Values are converted to that profile's connection space.
- Space Normalization — colors are normalized into a device-independent master space (Lab or XYZ) with chromatic adaptation applied, producing a NormalizedColor Fact Unit.
- ΔE Validation — CIEΔE2000 is computed between the normalized color and the target profile's rendering of it; a result exceeding DeltaEThreshold (§70.1) produces a ColorDrift Fact Unit rather than silently passing, the same non-silent-failure discipline already applied to every other validation in this Bible.
- Rendering Intent Application — the declared RenderingIntent resolves out-of-gamut colors for the specific output mode (PDF, SVG, WebGL, or VR headset), producing a RenderReadyColor Fact Unit.

Where a color must be synthesized rather than matched — no author-specified Color Fact Unit exists for a given element — this pipeline defers to VCUM's Semantic Colorization Engine (§44.3.1) rather than duplicating synthesis logic; this chapter's pipeline governs matching and validation, not invention.

70.4 The Registration Constraint Graph

Geometric alignment is resolved by a constraint graph: Geometry Fact Units are nodes, Constraint Fact Units are weighted edges, and a three-pass solver produces a final, deterministic layout:

- Pass 1, Absolute Resolution — LockTo and Pin constraints resolve first, forming the layout's fixed backbone.
- Pass 2, Relative Resolution — Align, Center, and RelativeTo constraints resolve against the backbone, with conflicts broken by constraint priority (§70.2).
- Pass 3, Drift Detection — the resolved geometry is compared against the previous version in the Provenance Ledger's VersionChain (Chapter 5) and against each Geometry Fact Unit's own

Tolerance; an excess produces a RegistrationDrift Fact Unit, the same non-silent-failure pattern as §70.3's ΔE validation.

A valid graph permits no cycles among absolute constraints; relative constraints may cycle only if the solver can still reach a deterministic result. This is a real constraint on what a valid Constraint Fact Unit set can express, not merely a recommendation.

70.5 VR Registration

VR-specific alignment reuses this chapter's Geometry Fact Units and Constraint Graph rather than a separate VR-only system, anchored to the Tour Waypoints already specified in §46.9 rather than inventing a parallel anchor concept:

- A VR object's Geometry Fact Unit anchors to a Tour Waypoint (§46.9) or an IQR scene object (Chapter 46) using the same Anchors field already specified in §70.2, not a new binding mechanism.
- Two further Constraint types apply specifically in VR: WorldLock (geometry stays fixed in world space regardless of viewer movement) and FollowUser (geometry tracks viewer position — for a heads-up display element, for instance). Both are ordinary entries in the same Constraint Fact Unit type (§70.2), not a separate VR constraint system.
- Drift detection (§70.4, Pass 3) in VR additionally accounts for headset tracking instability as a drift source, compared against the same per-Geometry-Fact-Unit Tolerance field — no separate VR tolerance concept.

70.6 Querying Color and Geometry Facts

Color and Geometry Fact Units are retrieved and verified through this Bible's existing query architecture — QBQL (Chapter 18) for structured retrieval (for example, finding all Color Fact Units within a stated ΔE of a target value), and the Associative Recall Engine (§7.2.3) for semantic lookup — not a new query language. A “verify this figure's color accuracy” or “report registration drift since version N” request is an ordinary QBQL query against ColorDrift and RegistrationDrift Fact Units (§70.3, §70.4), which already exist as queryable Fact Units the moment a validation pass produces them.

70.7 Integration with Print-on-Demand

Precision color and registration matter most where they are hardest to correct after the fact — physical print. Chapter 60's Print Production (§60.3) parameters may reference Color and Geometry Fact Units for cover art and print layout; a submission with an unresolved ColorDrift or RegistrationDrift Fact Unit

above tolerance is flagged before Batch Printing (§60.3) proceeds, the same pre-flight discipline the Rights Validation Gate (§60.2) already applies to a different kind of check.

70.8 Integration Requirements

This chapter requires the Fact Unit schema (Appendix A), the Provenance Ledger (Chapter 5), QBQL (Chapter 18), the Associative Recall Engine (§7.2.3), VCUM's SCE (§44.3.1), Tour Waypoints (§46.9), WASM's Object Layer (§53.3, referenced only to distinguish scope), and Chapter 60's Print Production (§60.3). It modifies none of their specifications.

70.9 Geometric Proof Engine

This section adds formal, deductive geometric reasoning — the one genuinely new capability found in a submission otherwise built on the declined manifold-layer concept (SML/GML/TML/RML) and substantially duplicating this chapter's own Geometry and Constraint Fact Units under new names. The Geometric Proof Engine (GPE) reasons over Geometry and Constraint Fact Units already specified in §70.1–§70.2 — it introduces no parallel FactType system.

SCOPE NOTE — DEDUCTIVE RULES OVER EXISTING FACT UNITS, NOT A NEW GEOMETRIC ONTOLOGYThe GPE applies a fixed, deterministic set of classical geometric rules (angle relationships, triangle properties, circle theorems, parallel-line properties) to Geometry Fact Units already in the Fact Graph. It does not introduce GeometricObject, SceneTopology, or SpatialRelation as new FactTypes — those would duplicate the Geometry and Constraint Fact Units §70.1–§70.2 already specify. A rule fires only against facts that already exist; per §70.9.3, it cannot introduce a new geometric entity into the Fact Graph, only a derived relationship between entities already there.

70.9.1 Rule Set

The GPE's rule set is fixed and enumerable, not open-ended — the same bounded-primitive discipline already applied to QBPL (§48.5, nine primitives, deliberately not Turing-complete):

- Angle rules — complementary, supplementary, vertical, and corresponding-angle relationships.
- Triangle rules — angle-sum, exterior-angle, and congruence/similarity conditions.
- Circle theorems — inscribed-angle, central-angle, and tangent-chord relationships.
- Parallel-line rules — alternate-interior, corresponding, and co-interior angle relationships under a transversal.

Each rule is a deterministic function from a set of Geometry and Constraint Fact Units to a derived GeometricProofStep Fact Unit — the one new FactType this section actually adds, distinct from an object or a constraint: a proof step records which rule fired, which Fact Units it consumed, and what relationship it derived, with a citation back to the specific rule by name.

70.9.2 Proof Construction

A proof is a VersionChain-tracked (Chapter 5) sequence of GeometricProofStep Fact Units, each citing the prior steps and Geometry/Constraint Fact Units it depended on — the same fact-traceability lineage already required of every other derived QueryBook output (§39.11.2's compositional pattern, applied here to deduction instead of app assembly). A contradiction — two proof steps deriving incompatible relationships for the same Geometry Fact Unit — halts construction and surfaces both conflicting steps rather than silently preferring one, the same non-silent-failure discipline as §70.3's ΔE validation and §70.4's drift detection.

70.9.3 Safety Constraints

- A proof step may only derive a relationship between Geometry or Constraint Fact Units that already exist in the Fact Graph — the GPE cannot introduce a new geometric entity, only a new relationship among entities already present, consistent with the Without-Information Rule (D0.4).
- Every derived relationship is queryable through QBQL (Chapter 18) the same way any other Fact Unit is — no separate proof-query mechanism.
- A rule application is deterministic and reproducible: the same input Fact Units under the same rule always produce the same GeometricProofStep, consistent with §48.5's determinism requirement for QBPL primitives.

70.9.4 What This Section Does Not Add

NO MANIFOLD DEPENDENCY, NO UNVERIFIED PERFORMANCE CLAIMS **This capability does not depend on, and does not reference, a Semantic, Geometric, Temporal, or Robotics Manifold Layer — that concept was proposed and declined in this Bible's history, and nothing here reintroduces it under a different name. Separately: a submission proposing this feature also claimed compression improvements of "400%–19,900% depending on media type" with no stated methodology or benchmark. That figure is not carried forward. If a real compression benefit from geometric fact representation exists, it should be measured and stated with the same design-target caveat already applied to every other unvalidated performance figure in this Bible (Appendix C, §C.11), not asserted as a settled result.**

70.10 Summary

Color and geometric precision are specified here as Fact Unit schema extensions — not a parallel UFCS-based subsystem — with a genuinely novel three-pass constraint graph for registration, calibration that defers to VCUM's existing synthesis engine rather than duplicating it, VR anchoring built on Tour Waypoints rather than a separate anchor system, and querying through QBQL and the Associative Recall Engine rather than an invented query language. ΔE and registration drift are never silent: both produce queryable Fact Units the moment tolerance is exceeded, consistent with how every other validation in this Bible already behaves.

Chapter 71: Universal Accessibility Layer

This chapter specifies accessibility as a first-class requirement across every QueryBook surface — reading, VR, ingestion, and collaboration — not a downstream adaptation. It responds to a gap this Bible had no coverage for at all before this pass.

SCOPE NOTE — ACCESSIBLE-BY-DEFAULT, BUILT ON EXISTING FACT UNITS **Alt-text, scene descriptions, and simplified-language variants are Fact Units (Appendix A), generated alongside the content they describe and version-tracked through the same Provenance Ledger (Chapter 5) as everything else — not a parallel content system requiring separate authoring. Where existing infrastructure already does the relevant work (Narrative Graph for scene description, §46.10.6's disclosure pattern for a mandatory-and-non-removable requirement), this chapter reuses it rather than duplicating it.**

71.1 Accessible Content Requirement

No QueryBook output — a response, an IQR scene, a diagram, an audio narration — is considered complete without its accessible equivalent: alt-text for visual content, captions for audio, and a scene description for VR. This equivalent is generated as part of the same production step that creates the primary content, not a separate pass that can be skipped.

Content Type	Required Accessible Equivalent	Generation Mechanism
Diagram or figure	Alt-text (AltTextUnit)	Derived from the Fact Units the diagram was composed from (Chapter 70), not a separate description written after the fact.
IQR / VR scene	Scene description (SceneDescriptionUnit)	Generated from the same Narrative Graph (Chapter 12) grounding used for the response the scene illustrates.

Audio narration or Presenter Mode output	Captions	Generated at the same stage as the audio itself, using the underlying response text already produced.
Tactile diagram (braille embosser export)	TactileDiagramUnit	Derived from Chapter 70's Geometry Fact Units — the same spatial data already used for precision registration, re-exported for embossing rather than re-authored.

71.2 Alternative Presentation Modes

Large-print, high-contrast, dyslexia-friendly, and simplified-language modes are presentation-layer transformations of the same underlying Fact Units — not separate content. A SimplifiedLanguageUnit, for instance, reuses the Expertise Level Calibration already specified in the ARE Pipeline (§52.8.3) at its most accessible setting, rather than a new simplification mechanism.

71.3 VR Accessibility

- Spatial audio and haptic cues are required outputs of any VR scene (Chapter 46), not optional enhancements.
- Every Tour (§46.9) supports a low-motion mode and teleport-only navigation as alternatives to continuous movement between Waypoints.
- An avatar-free option is available for any Tour using Presenter Mode (§46.10) — Voice-Only mode, already specified, serves this directly; no new mode was needed.

71.4 Cognitive Accessibility

A step-by-step explanation mode and a reduced-pacing mode are additional Pedagogical Mode behaviors layered on the existing TEACH and BRIEF settings (§52.9), not new Interactive Modes — they adjust how an existing mode paces and structures its output, the same way any other Pedagogical Mode parameter already does.

71.5 Robotics and Alternative Control Accessibility

Where QueryBook is controlled through the Robot Memory Domain (Chapter 56) or another non-visual, non-auditory interface, input is accepted through whichever control surface the connecting device declares in its Capability Declaration (§49.3.3.1) — voice-free, gesture-free, eye-tracking, or switch-control input are all valid declared capabilities under the existing Source Connector contract, not a separate accessibility-specific connector type.

71.6 Accessibility Fact Unit Fields

Four fields extend the base Fact Unit schema (Appendix A) for accessibility purposes: AltText, SceneDescription, TactileExport, and SimplifiedVariant. Each is optional per Fact Unit but, per §71.1, required before that Fact Unit's content is considered production-complete for reader-facing output.

71.7 Integration Requirements

This chapter requires the Fact Unit schema (Appendix A), the Provenance Ledger (Chapter 5), the Narrative Graph (Chapter 12), the ARE Pipeline (§52.8), Tour and Presenter Mode (§46.9–§46.10), the Source Connector Registry (§49.3.3), and Chapter 70's Geometry Fact Units. It modifies none of their specifications.

71.8 Summary

Accessible equivalents are specified as a required, not optional, part of content production — generated from the same Fact Units and at the same production stage as the primary content, using existing infrastructure (Narrative Graph, ARE Pipeline, Geometry Fact Units, Source Connector capability declarations) rather than a parallel accessibility-specific system.

Chapter 72: Data Portability and Exit

This chapter specifies a complete export guarantee: a reader can leave QueryBook with everything they created or accumulated, in open formats, with provenance intact. It closes a gap this Bible had no coverage for — exports existed for specific cases (Chapter 60's print formats, Chapter 64's PDF/CSV board exports) but nothing guaranteed comprehensive account-level portability.

SCOPE NOTE — EXPORT OF EXISTING FACT UNITS, NOT A NEW STORAGE FORMAT Every exportable item in this chapter is already a Fact Unit somewhere in this Bible — a Personal Note (Chapter 63), a board post (Chapter 64), an ingested document (Chapter 4). This chapter specifies serialization formats and an export process, not a new place data lives.

72.1 What Can Be Exported

Content Category	Format(s)	Source Chapter
Personal Notes and annotations	JSON (full fidelity), TXT (plain text)	Chapter 63
Message board posts and threads a	JSON, CSV	Chapter 64

reader authored		
Ingestion history and Fact Units a reader's actions created	JSON (Fact Universe representation)	Chapter 4, Appendix A
A purchased or licensed book's reader-facing content	EPUB, where the Publisher's rights terms (§60.2) permit export	Chapter 60
IQR / Tour scenes a reader has access to	glTF or USD, per §46.2's existing open-format delivery	Chapter 46
Event history and Class Board participation	JSON, CSV	Chapter 62, Chapter 64

Every format listed is already open and non-proprietary — JSON, CSV, TXT, EPUB, glTF/USD — consistent with §46.2's existing commitment to open 3D delivery formats. This chapter does not introduce a new file format for any content type.

72.2 Export Process

- A reader requests an export of one category or their full account; the request is an AccessControlList-governed action (Appendix A) scoped to content the requesting identity actually owns or authored.
- The export includes each Fact Unit's provenance metadata — creation timestamp, VersionChain reference (Chapter 5) — not just its current-state content, so the export remains independently verifiable after leaving QueryBook.
- An export does not delete or restrict the reader's continued access within QueryBook; exporting and retaining access are independent, consistent with this being a portability guarantee, not an account-closure workflow.

72.3 What This Chapter Does Not Guarantee

A Publisher's Rights Validation Gate (§60.2) may restrict export of a specific book's full text where the author or publisher has not licensed it for export — portability applies to what a reader created or was granted rights to, not a blanket right to copy content someone else holds rights over. This chapter does not override Chapter 60's existing rights model; it operates within it.

72.4 Integration Requirements

This chapter requires the Fact Unit schema (Appendix A), the Provenance Ledger (Chapter 5), Chapters 4, 46, 60, 62, 63, and 64 as content sources, and the AccessControlList pattern governing what a given identity may export. It modifies none of their specifications.

72.5 Summary

A reader can export their notes, board activity, ingestion history, and licensed content in open, non-proprietary formats, with provenance preserved, without that export restricting their continued access — built entirely from existing Fact Units and existing rights boundaries, not a new data store or a bypass of Chapter 60's licensing model.

Chapter 73: Template Debug Mode

This chapter specifies Template Debug Mode, named in the Interactive Mode Registry (§52.13.2) as an operator tool since early in this Bible's development but never actually built — flagged explicitly in Pass 34 and again in Pass 39 as a gap worth its own dedicated pass. This is that pass.

SCOPE NOTE — INSPECTS EXISTING EXECUTION, DOES NOT ADD A NEW ONE Template Debug Mode observes the real execution of QBPL programs (§48.3), the ARE Pipeline (§52.8), and IQR scene composition (Chapter 46) — it does not run a separate simulated version of any of them. A debug session watches the same execution a reader or author would otherwise see, with additional instrumentation exposed, not a parallel test environment that could diverge from real behavior.

73.1 Access

Template Debug Mode is an operator/authoring tool, scoped to the same privilege requirements as any other administrative function (§9.5.1) — not reader-facing, consistent with how the Interactive Mode Registry already described it before this chapter existed.

73.2 Step-Through Execution

A debug session pauses execution at each stage boundary already defined by the system being debugged — the ARE Pipeline's five stages (§52.8), a QBPL program's sequential operations (§48.3.2), or the Registration Constraint Graph's three passes (§70.4) — rather than defining new debug-specific breakpoints. Stepping through is pausing between checkpoints that already exist in the specification.

73.3 Inspection Views

View	Shows	Source
Fact Unit Bindings	Which Fact Units a template stage is currently operating on, and their	The Fact Unit itself (Appendix A) — no new binding representation.

	current field values.	
Safety Gate Triggers	Which Safety Substrate enforcement point (Chapter 8) evaluated this step, and its pass/fail result with stated reason.	The Safety Substrate's existing per-action evaluation (Appendix H, §H.6).
Slice Measurement	The Processor Slice category and duration for the step just executed.	Chapter 69, unmodified — the debug view surfaces existing slice logs, it does not create a debug-specific measurement.
Version History	The VersionChain entry this step is about to write or has just written.	The Provenance Ledger (Chapter 5).

73.3.1 Failing Node Highlighting

Where a template stage fails — a Safety Substrate rejection, an ambiguity halt (§48.4.1's Stage 3), a ΔE or registration drift finding (§70.3–§70.4) — Template Debug Mode highlights the specific stage and surfaces the existing failure reason already produced by that system, rather than generating a separate debug-specific error message that could describe the failure differently than the system's own account of it.

73.4 Debug Trace Records

A debug session's step sequence is recorded as a TemplateDebugTrace Fact Unit — a read-only log of which stages executed, in what order, with what Fact Unit bindings — itself provenance-tracked through the Provenance Ledger (Chapter 5) like any other Fact Unit, so a debug trace can be reviewed later or attached to a bug report without re-running the original session.

73.5 What This Chapter Does Not Add

Template Debug Mode does not execute against a simulated or mocked version of the Fact Graph, Safety Substrate, or any other subsystem — it observes real execution with additional visibility. This is a deliberate choice: a debugger that runs against a simulation risks the simulation diverging from real behavior, which would make debugging less trustworthy, not more.

73.6 Integration Requirements

This chapter requires QBPL (§48.3), the ARE Pipeline (§52.8), the Safety Substrate (Chapter 8), Chapter 69's Slice Categories, the Provenance Ledger (Chapter 5), and the administrative privilege model (§9.5.1). It modifies none of their specifications — it adds observation, not new execution paths.

73.7 Summary

Template Debug Mode is specified as an instrumented view into real execution — stepping through existing pipeline stages, surfacing existing Safety Substrate and drift-detection results, and recording a reviewable trace — rather than a separate simulated debugging environment. This closes the named-but-unspecified gap flagged in Passes 34 and 39.

Chapter 74: Multi-Author Collaboration

Chapter 61's Drafting Tools were specified for a single author. This chapter adds real-time co-authoring with conflict resolution, closing a genuine gap: nothing in this Bible previously defined what happens when two authors edit the same book's Fact Graph concurrently.

SCOPE NOTE — CONCURRENCY CONTROL OVER THE EXISTING FACT GRAPH, NOT A SEPARATE EDITING SYSTEMCo-authoring uses the same Manuscript Fact Units and Provenance Ledger VersionChain already specified in Chapter 61 — this chapter adds concurrency rules for multiple identities writing to that same Fact Graph, not a separate collaborative document format.

74.1 Author Sessions

An AuthorSession Fact Unit tracks which authors are actively editing a given book's Fact Graph, scoped by the same Role and AccessControlList model already governing who may author content at all (Chapter 65, Appendix A) — co-authoring access is granted the same way any other authoring permission is, not a separate collaboration-specific permission.

74.2 Concurrency Model

- Optimistic concurrency — authors edit independently; conflicts are detected at write time by comparing the VersionChain head each author started from (Chapter 5) against the current head, the same staleness check already implicit in any versioned write.
- Fact-level locking is available as an explicit, author-initiated action for a specific Fact Unit an author wants uncontested access to — not a default, since most concurrent edits to different Fact Units in the same book never actually conflict.

74.3 Conflict Resolution

Where two authors' edits to the same Fact Unit genuinely conflict, resolution follows the type of content in conflict, reusing detection mechanisms already specified elsewhere rather than inventing new diff logic per content type:

Conflict Type	Detection Mechanism	Resolution
Narrative text	Standard VersionChain divergence (Chapter 5)	Both versions preserved in the chain; authors choose which becomes canonical, or manually merge — the same conflict-preservation principle already established for Personal Notes (§63.3).
Geometry or Constraint Fact Units	The Registration Constraint Graph's existing drift detection (§70.4)	A RegistrationDrift finding between two authors' concurrent geometry edits is surfaced the same way any other registration drift already is — not silently resolved.
Scenario Graph nodes (Counterfactual Exploration, §61.3)	VersionChain divergence on the Scenario Graph's own Fact Units (Chapter 11)	Both branches preserved; this is exactly what a Scenario Graph already represents — divergent hypothetical branches — so a genuine authoring conflict here often resolves as two valid scenario branches rather than requiring a single winner.
Tour and Instructor Tour Waypoint sequences	Standard VersionChain divergence (Chapter 5), applied to the Tour's Waypoint-sequence Fact Units	Both sequences preserved in the chain; the Tour's owner — the book's author for an author Tour (§46.9), the hosting instructor for an Instructor Tour (§83.1) — chooses which becomes canonical or manually merges the sequence. Presenter narration authorization (§46.10.1, §83.3) is never merged or transferred between conflicting versions — each version retains only the narration its own author or instructor actually authorized.

There is no separate “geometric diff” or “narrative diff” engine — each conflict type is resolved using the detection and preservation mechanism the relevant chapter already specifies for its own content.

74.4 Merge Resolution Record

A resolved conflict produces a MergeResolutionUnit Fact Unit, recording which versions were in conflict, who resolved it, and how — provenance-tracked through the Provenance Ledger (Chapter 5) like any other consequential write, per this Bible's standing audit discipline (Appendix H, §H.6).

74.5 Integration Requirements

This chapter requires Chapter 61's Manuscript Fact Units, the Provenance Ledger (Chapter 5), Chapter 65's Role and AccessControlList model, Chapter 70's Registration Constraint Graph (for geometry conflicts), and Chapter 11's Scenario Graph (for counterfactual conflicts). It modifies none of their specifications.

74.6 Summary

Multiple authors can co-edit a book's Fact Graph under optimistic concurrency with explicit locking available where needed, and conflicts resolve using the detection and preservation mechanisms each content type already has — VersionChain divergence for narrative text, Registration drift detection for geometry, and branch preservation for Scenario Graphs — rather than a new, separate conflict-resolution engine.

Chapter 75: Moderation Appeals

Chapter 64 specifies moderation actions — flagging, hiding, banning, locking, archiving — but never specified what a moderated user can do if they believe the action was wrong. This chapter closes that gap.

SCOPE NOTE — REVIEW OF AN EXISTING DECISION, NOT A NEW MODERATION AUTHORITY An appeal reviews a moderation action already logged to the Provenance Ledger (§64.4) — it does not create new moderation powers, and it does not bypass the Safety Substrate gating that governed the original action. This chapter specifies how a human reviews a decision already made, using the record that decision already produced.

75.1 What Can Be Appealed

Any moderation action already specified in §64.4 — a flag, a hide, a ban, a lock, an archive — can be appealed by the affected identity, using the same AccessControlList-governed submission process as any other consequential request (Appendix A).

75.2 Appeals Workflow

- Submission — the affected identity submits an AppealUnit Fact Unit, referencing the specific moderation action's Provenance Ledger entry (§64.4) being contested.

- Explanation — the system generates an AppealNarrativeUnit explaining the original moderation decision in plain language, reusing the Narrative Graph (Chapter 12) the same way any other grounded explanation in this Bible is produced — not a new explanation-generation mechanism.
- Human review — a moderator with the appropriate Role (Chapter 65) reviews the AppealUnit and the original action's logged reason (§64.4); this review is itself a Safety Substrate-gated action, consistent with §66.5's Human Review Gate discipline applied here to moderation rather than architectural proposals.
- Decision — the reviewing moderator's decision (uphold, reverse, or modify the original action) is logged to the Provenance Ledger (Chapter 5) as a ModerationDecisionUnit, with the moderator's identity and stated reasoning — the same non-anonymous accountability already required of the original moderation action itself.
- Notification — the appealing identity receives the decision and its stated reasoning, delivered through the existing notification pattern (§52.13.4) already used for other consequential events.

75.3 What an Appeal Cannot Do

- An appeal cannot reverse a Safety Substrate finding directly — if the original action was gated by a Safety Substrate rule (Chapter 8), the appeal reviews whether that rule was applied correctly to this case, not whether the rule itself should exist. Changing a safety rule is a Chapter 66 Improvement Proposal Pipeline matter, not an appeals matter.
- An appeal does not automatically restore access while pending — the original moderation action remains in effect during review, consistent with treating the original decision as valid until a human reviewer determines otherwise, not presumptively wrong.

75.4 Integration Requirements

This chapter requires §64.4's moderation actions and audit log, the Provenance Ledger (Chapter 5), the Narrative Graph (Chapter 12), Chapter 65's Role model, and §52.13.4's notification pattern. It modifies none of their specifications.

75.5 Summary

A moderated user can appeal, receive a plain-language explanation of the original decision generated from the existing Narrative Graph, and have a named human reviewer uphold, reverse, or modify the action — all logged with the same accountability discipline already required of the original moderation.

An appeal reviews how a rule was applied, not whether the rule itself should exist — that question belongs to Chapter 66's proposal pipeline, with its own human gate.

Chapter 76: Platform Internationalization

Chapter 48's NLPL already guarantees multilingual compilation — the same instruction compiles to an identical program regardless of source language (§48.4.2). This chapter extends localization to the platform surface itself: the Reader Portal, Interactive Mode Registry, and other UI a reader actually sees, which was never previously specified.

SCOPE NOTE — ONE SAFETY STANDARD, LOCALIZED PRESENTATION **Locale affects language, formatting, and culturally appropriate examples — it does not affect what the Safety Substrate (Chapter 8) or Prime Directive (Domain 0) permit. "Multilingual safety rules" in this chapter means the same rules correctly expressed and understood in every supported language, not different rules in different regions. This distinction is stated explicitly because it is easy to misread otherwise.**

76.1 What Is Localized

- The Reader Portal (§36.8) and its interface text.
- Interactive Mode Registry labels and prompts (§52.13.2), not the underlying Fact Graph content they operate on.
- NLPL's user-facing prompts and error messages (Chapter 48) — distinct from NLPL's compilation guarantee, which was already locale-independent before this chapter.
- VR scene UI elements and Presenter Mode's mandatory disclosure text (§46.10.6) — the disclosure's non-removability (§46.10.6) is preserved in every locale; only its language changes.
- Robot Memory Domain (Chapter 56) status and control surface text, where a connected device renders text at all.

76.2 What Is Not Translated

A book's own authored content is not machine-translated by this chapter — that would materially change the work itself, a decision that belongs to the author and publisher (Chapter 60), not to a platform localization layer. This chapter localizes the platform a reader uses to access content, not the content's own language.

76.3 Locale-Aware Formatting

- Right-to-left layout support for applicable languages, applied at the presentation layer without altering the underlying Fact Graph's stored text direction metadata.
- Culturally appropriate examples in Tutorial Mode and Comprehension Check (§52.13.2) are locale-scoped variants of the same underlying pedagogical Fact Units, not separately authored per-locale content requiring independent maintenance.

76.4 Multilingual Ingestion

Ingestion Templates (D0.5.1) may be authored or applied in any supported language; the resulting Fact Units are language-neutral per the Multimodal Ingestion Pipeline's existing design (Chapter 4) — ingesting a document in French produces the same kind of Fact Unit ingesting it in Japanese would, consistent with NLPL's existing locale-independence guarantee (§48.4.2) extended here to ingestion specifically.

76.5 Integration Requirements

This chapter requires §36.8's Reader Portal, §52.13.2's Interactive Mode Registry, Chapter 48's NLPL, §46.10.6's disclosure requirement, Chapter 56's RMD, Chapter 4's Ingestion Pipeline, and Chapter 60's publisher-controlled translation rights. It modifies none of their specifications, and does not alter the Safety Substrate or Prime Directive in any locale.

76.6 Summary

Platform UI, prompts, and mandatory disclosures are localized without changing what they mean or what they permit — one safety standard, presented correctly in every supported language. A book's own content translation remains a publisher decision (Chapter 60), not something this chapter does automatically.

Chapter 77: Validation and Benchmarking

This Bible contains many performance figures explicitly marked as unvalidated design targets — Appendix C, §C.11, §44.5, and elsewhere — but never specified how or when those figures actually get measured. This chapter closes that gap: a formal process for turning a design target into a validated result, or retracting it.

SCOPE NOTE — A PROCESS FOR VALIDATING EXISTING CLAIMS, NOT NEW CLAIMS This chapter introduces no new performance figures of its own. It specifies how the design-target figures already scattered across this Bible get benchmarked, and what happens to a figure that fails validation — retraction, not quiet retention.

77.1 What Gets Benchmarked

Benchmark Category	Validates	Existing Design-Target Source
Ingestion accuracy	Fact Synthesis correctness against ground-truth source content	Chapter 4
Retrieval and response quality	Associative Recall Engine and ARE Pipeline output against the Factual Groundedness metric	§7.2.3, Chapter 16
Geometric registration	ΔE and registration-drift tolerances actually holding under real workloads	Chapter 70, §C.11
Predictive accuracy	PIE (Chapter 10) forecasts against realized outcomes	Chapter 10
Slice measurement overhead	Whether Chapter 69's measurement layer itself adds acceptable overhead to the operations it measures	Chapter 69

77.2 Benchmark Process

- A BenchmarkUnit Fact Unit defines a specific, reproducible test: the operation being measured, the workload used, and the metric being evaluated — not a vague performance claim, a concrete, repeatable procedure.
- Running a benchmark produces a ValidationReportUnit, provenance-tracked through the Provenance Ledger (Chapter 5) like any other Fact Unit, recording the actual measured result against the existing design-target figure it's checking.
- A PerformanceTargetUnit links a specific design-target figure already stated elsewhere in this Bible (for instance, §C.11's figures) to the BenchmarkUnit that would validate it — closing the loop between a claim and the test that either confirms or retracts it.

77.3 Outcome of Validation

- A figure that validates within stated tolerance is marked validated in its ValidationReportUnit, with the benchmark conditions recorded so the validation can be reproduced or re-checked later.
- A figure that fails validation is retracted, not quietly retained — the design-target language in the originating section (for example, §C.11's own “should be revalidated before external-facing use”

framing) is updated to reflect the actual measured result, consistent with this Bible's standing discipline of not letting an unvalidated figure sit unchallenged once it's actually been checked.

77.4 Reuse of Existing Slice Measurement

Where a benchmark measures compute cost or duration, it reads from the Processor Slice categories already specified in Chapter 69 rather than a separate benchmarking-specific measurement mechanism — a benchmark run is itself composed of ordinary Ingestion, Reasoning, Response Generation, or Rendering Slices, measured the same way any other operation already is.

77.5 Integration Requirements

This chapter requires the Provenance Ledger (Chapter 5), Chapter 69's Slice Categories, and every design-target figure already stated elsewhere in this Bible — Appendix C, Chapter 4, §7.2.3, Chapter 10, Chapter 16, and Chapter 70. It modifies none of their specifications directly; it links to them and, on validation or retraction, updates their stated confidence level rather than their underlying architecture.

77.6 Summary

Every design-target figure already in this Bible now has a defined path to becoming either a validated result or a retracted claim, using BenchmarkUnit, ValidationReportUnit, and PerformanceTargetUnit Fact Units that reuse the existing Provenance Ledger and Processor Slice infrastructure. No new performance claims were introduced by this chapter — only the process for checking the ones that already existed.

Chapter 78: QueryBook AI Evolution

This chapter names and structures a dedicated, subsystem-assigned front end to the Improvement Proposal Pipeline already fully specified in Chapter 66. QueryBook AI Evolution is not a new authority or a new pipeline — it is System Agents, one per major subsystem, whose job is to continuously run Candidate Identification (§66.1) for their assigned area and draft Proposals (§66.2) for a human to evaluate. Every proposal a System Agent produces still passes through the same Simulation Sandbox (§66.3), the same automated pre-screening (§66.4), and the same Human Review Gate (§66.5) as any other candidate, with no shortcut for having originated from a named agent.

THIS CHAPTER GRANTS NO NEW AUTHORITY QueryBook AI Evolution is a name and an organizational structure over Chapter 66's existing pipeline — candidate identification, proposal drafting, sandbox testing, and a recommendation to a human. It does not modify §66.5's Human Review Gate, and it does not touch §66.8's Permanent Exclusions list. A System Agent recommends; a named human admin approves. That is the entire authority a System Agent has, regardless of which subsystem it is assigned to or how the proposal was generated.

78.1 System Agents

A System Agent is assigned to exactly one already-specified subsystem, scoped to that subsystem's own Slice Category (Chapter 69) where one exists. An agent's Candidate Identification work (§66.1) is bounded to its assigned area — an agent assigned to Ingestion does not draft proposals about Rendering, and vice versa.

System Agent	Assigned Subsystem	Draws Candidates From
Ingestion Agent	Multimodal Ingestion Pipeline (Chapter 4)	Ingestion Slice telemetry (§69.2)
Reasoning Agent	NCL and AEM (Chapters 7, 15)	Reasoning Slice telemetry (§69.2)
Response Agent	The ARE Pipeline (§52.8)	Response Generation Slice telemetry (§69.2)
Rendering Agent	IQR and VCUM (Chapters 44, 46)	Rendering Slice telemetry (§69.2)
Geometry Agent	Precision Color and Geometric Registration (Chapter 70)	ΔE and registration drift findings (§70.3–§70.4)
Robotics Agent	The Robot Memory Domain (Chapter 56)	RMD integration telemetry (§56.4)
Accessibility Agent	The Universal Accessibility Layer (Chapter 71)	Coverage gaps against §71.1's accessible-content requirement

This is a representative, not exhaustive, assignment list — an agent can be assigned to any subsystem with its own telemetry source, following this same pattern, without this chapter's specification needing to change, the same extensibility principle already established for Slice Categories (§69.2).

78.2 What a System Agent Does

- Monitors its assigned subsystem's existing telemetry — Slice measurements (Chapter 69), drift findings (Chapter 70), coverage gaps (Chapter 71) — for the same kind of recurring pattern §66.1 already looks for from any source.
- Drafts a Proposal Fact Unit (§66.2) when a pattern is found, scoped by §66.2's existing schema — Scope, Change Description, Risk Assessment, Performance Prediction, Rollback Plan.

- Submits the Proposal to the existing Simulation Sandbox (§66.3) for testing — an agent does not have its own separate testing environment; it uses the one Chapter 66 already specifies.
- The Proposal is automatically pre-screened (§66.4) against §66.8's Permanent Exclusions before any human sees it, exactly as any other proposal is.

78.3 What a System Agent Does Not Do

- A System Agent does not approve its own proposal, or any proposal. Approval is exclusively the Human Review Gate's function (§66.5) — a named, accountable admin, not an agent, however confident the agent's Fitness Score.
- A System Agent cannot draft a proposal scoped to another agent's assigned subsystem, to Chapter 66's own Human Review Gate, or to anything on §66.8's Permanent Exclusions list — the Safety Substrate, the Prime Directive, Presenter Mode's authorization requirements (§46.10), or the private-note guarantees (§63.4).
- A System Agent's recommendation carries no more weight in review than a human-originated Candidate Record — §66.5 already states a reviewer evaluates the Fitness Score and Risk Profile as evidence, not as a decision; that applies identically here.

78.4 Deployment

A System Agent's approved proposal deploys exactly as §66.6 already specifies — staged rollout, only after human approval. No System Agent has deployment authority of its own; approval and deployment remain the same two-step, human-gated sequence for every proposal in this Bible, regardless of source.

78.5 Integration Requirements

This chapter requires the entire Improvement Proposal Pipeline (Chapter 66) unmodified, Chapter 69's Slice Categories, and the specific subsystem chapters an agent monitors (Chapters 4, 7, 15, 44, 46, 52, 56, 70, 71 as representative examples per §78.1). It modifies none of their specifications, and §66.5's Human Review Gate and §66.8's Permanent Exclusions apply to every System Agent exactly as specified in Chapter 66.

78.6 Human-Directed Proposals

§78.1's System Agents identify candidates from telemetry — a pattern noticed in existing data. This section adds the other legitimate way a proposal can start: a person directly specifying what they want

investigated, before any telemetry pattern exists to notice. Both paths converge on the exact same pipeline from that point forward.

SAME PIPELINE, SAME GATE, NO SHORTCUTA human-directed proposal receives no expedited review, no different Fitness Score threshold, and no exemption from anything in §66.8's Permanent Exclusions list. Directing what gets investigated is not the same authority as approving what gets deployed — those remain two separate steps even when the same admin does both.

78.6.1 Direction

Any License Owner holding a Role entitled to propose changes in the relevant subsystem (Chapter 65, Chapter 67) may submit a Proposal Fact Unit (§66.2) directly — Scope, Change Description, and an initial Performance Prediction — without waiting for a System Agent to surface the same idea from telemetry. This is the “proposal before sandbox development” entry point: the human states what they want explored first, in the same structured form §66.2 already requires of any proposal.

78.6.2 Sandbox and Review, Unchanged

- The proposal enters the Simulation Sandbox (§66.3) exactly as any System Agent's proposal does — there is no separate, faster testing path for a human-directed idea.
- Automated pre-screening (§66.4) checks it against §66.8's Permanent Exclusions before any human reviewer sees it, identically to §78.3's rule for System Agents — a human directing a proposal cannot direct one targeting the Safety Substrate, the Prime Directive, Chapter 66's own Human Review Gate, or the other permanently excluded items, any more than an agent can.
- The Human Review Gate (§66.5) evaluates it on the same Fitness Score and Risk Profile evidence as any other proposal — the reviewer is not the same thing as the director, even when one person happens to hold both roles; the review step still has to actually happen.

78.6.3 Why This Matters

This is the structural answer to using AI to improve AI responsibly: a person directs what gets explored, an agent or the sandbox does the exploratory work, and a named human still has to say yes before anything changes — with a fixed list of things that yes can never apply to. Directing a proposal is participation in the process, not authority over its outcome.

78.7 Summary

QueryBook AI Evolution names and organizes what Chapter 66 already fully specifies: subsystem-assigned agents that identify candidates, draft proposals, and submit them to the existing sandbox and pre-screening — with a named human admin as the only party who can approve anything, and the same Permanent Exclusions list unreachable by any agent regardless of assignment. The name is new; the authority structure underneath it is not.

Chapter 79: Formal Verification Layer

This chapter adds machine-checkable, formal proof to the programs and proposals already specified elsewhere in this Bible — QBPL programs (§48.3), WATCH programs (§49.4), and Improvement Proposals (Chapter 66, Chapter 78) — using Lean, a proof assistant capable of verifying that a specific, precisely stated property genuinely holds, not merely that a test suite happened to pass. It is additive to every mechanism it touches: nothing here replaces the Human Review Gate (§66.5), the Simulation Sandbox (§66.3), or the Verification and Confirmation Gate (§48.4.1); it gives each of them a stronger form of evidence to work from.

WHAT FORMAL VERIFICATION IS, AND WHAT IT IS NOT, HERE **A Lean proof in this chapter establishes that a precisely specifiable property holds for all possible inputs — termination, a stated bound never being exceeded, non-interference with a named FactType — which is a fundamentally stronger guarantee than a passing test, which only confirms behavior on the specific cases tested. This is possible here specifically because QBPL's primitive set is small, fixed, and deliberately non-Turing-complete (§48.5) — a property that would make formal verification impractical for an unbounded general-purpose language is exactly what makes it tractable for QBPL. A Lean proof does not, and cannot, verify open-ended judgments — whether a change is a good idea, whether a response reads naturally, whether a proposal is worth deploying. Those remain exactly what they already were: a named human's decision at the Human Review Gate.**

79.1 What Gets Proved

Target	Property Verified	Why It's Formally Tractable
A QBPL program using REPEAT-UNTIL-BOUND (§48.5)	The loop terminates within its declared bound under all reachable inputs, not merely the inputs a test happened to cover.	REPEAT-UNTIL-BOUND is one of nine fixed primitives with a formally defined semantics (§48.5) — the property being proved is precisely stated, not inferred from behavior.
A WATCH program's trigger clause	The trigger condition is well-formed	WATCH programs compile through

(§49.4)	and the program's ASSERT/RETRACT operations only target FactTypes the program's declared Scope actually covers.	the same bounded QBPL primitive set; the proof is over that compiled form, not over the original natural-language instruction.
An Improvement Proposal touching QBPL or WATCH behavior (Chapter 66, Chapter 78)	The proposal's Scope (§66.2) provably does not reach any FactType or mechanism on the Permanent Exclusions list (§66.8).	The Permanent Exclusions list is a fixed, enumerable set of named targets — non-interference with a fixed, named set is a tractable property to formally prove, distinct from proving the proposal is otherwise a good idea.

79.2 Where Verification Sits in the Existing Pipeline

- For a QBPL or NLPL-compiled program: Lean verification runs as part of Stage 5, the Verification and Confirmation Gate (§48.4.1), producing a proof artifact the confirming human sees alongside the program itself — the human confirmation step is unchanged; it now has stronger evidence to confirm against.
- For a WATCH program: verification runs at registration with the Continuous Query Engine (§49.5), before the program is added to the WATCH Program Registry — a program that fails verification is not registered, the same way a program failing any other registration check already isn't.
- For an Improvement Proposal: verification runs as an additional step within Automated Pre-Screening (§66.4), for any proposal whose Scope touches QBPL or WATCH program behavior — a proposal that fails a required proof is rejected at pre-screening with the specific unproved property named, the same way a proposal targeting a Permanent Exclusion already is rejected there.

79.3 Proof Artifacts as Fact Units

A completed Lean proof is stored as a VerificationProof Fact Unit, provenance-tracked through the Provenance Ledger (Chapter 5) like any other Fact Unit — referencing the specific program or proposal it verifies, the exact property statement proved, and the proof itself, so a proof can be independently re-checked later rather than taken on the verifier's word. Where a proof cannot be constructed — the property doesn't hold, or isn't expressible in a form Lean can check — that outcome is recorded too, as a FailedVerification Fact Unit stating which specific property could not be established, not a silent rejection with no reason attached.

79.4 System Agents and Verification

A System Agent (§78.1) drafting a proposal that touches QBPL or WATCH behavior must produce a VerificationProof before that proposal can pass Automated Pre-Screening (§66.4) — the same

requirement applies identically whether the proposal originated from a System Agent's telemetry-driven Candidate Identification or from a Human-Directed Proposal (§78.6). Formal verification is not a substitute for §78.3's existing rule that no agent approves its own proposal; a proof establishes that a stated property holds, not that the change should be made.

79.5 What This Chapter Does Not Change

- The Human Review Gate (§66.5) still evaluates every proposal; a VerificationProof is additional evidence available to the reviewer, exactly as Fitness Score and Risk Profile already are — not a replacement for the reviewer's judgment.
- The Permanent Exclusions list (§66.8) is unchanged, and is not itself a valid target for a Lean-verified proposal — a proof cannot establish that a change to the Safety Substrate, the Prime Directive, or this pipeline's own gate is safe, because no proposal reaching that target passes Automated Pre-Screening (§66.4) far enough to reach the verification step at all.
- Discussion Mode (§52.13.3), Narrative Graph output (Chapter 12), and other natural-language generation are not formally verified by this chapter — their correctness is governed by the Factual Groundedness metric and citation discipline already specified elsewhere, not by a Lean proof, because their outputs are not the kind of precisely specifiable property Lean verification applies to.

79.6 Integration Requirements

This chapter requires QBPL (§48.3, §48.5), the NLPL compilation pipeline (§48.4.1), WATCH programs and the CQE (§49.4–§49.5), the Improvement Proposal Pipeline (Chapter 66), QueryBook AI Evolution (Chapter 78), and the Provenance Ledger (Chapter 5). It modifies none of their specifications — it adds a verification step each of them now passes through, and a Fact Unit type recording the result.

79.7 Summary

Lean-based formal verification is specified here as evidence, not authority: it proves precisely stated properties — termination, declared bounds, non-interference with named exclusions — for QBPL programs, WATCH programs, and the proposals that touch them, storing every proof or failure as a re-checkable Fact Unit. It strengthens what a human reviewer, or a confirming author, has in front of them at the moment of decision. It does not decide anything itself, and it has no path to the Permanent Exclusions list any more than an ordinary proposal does.

Chapter 80: Human Reasoning and Motivation Modeling

This chapter specifies how QueryBook formally represents a subject's goals, motivations, and planning process as content — a historical figure's decision-making, a case study's reasoning, a novel character's stated ambitions — reusing Discussion Mode's existing viewpoint-adoption authority (§52.13.3) and the Scenario Graph's existing PLANNING type (Chapter 11), rather than introducing a new reasoning subsystem.

THIS MODELS A SUBJECT, NEVER QUERYBOOK ITSELF Every Goal, Motivation, and Planning representation this chapter specifies describes someone or something documented in the Fact Graph — a person, a character, an organization, a case study subject. None of it describes QueryBook's own operating state. QueryBook does not acquire goals, curiosity, urgency, or a self-model by virtue of being able to represent those properties in a subject it is analyzing — the same distinction this Bible already draws for a novel's narrator (§4.1: a character's stated belief is a fact about what the character believes, not an assertion the system is making). A capability set describing an autonomous, self-motivated, goal-driven QueryBook was proposed separately and declined; this chapter is not a narrower version of that proposal wearing an academic framing — it is a fundamentally different thing, and this callout exists so that distinction is never lost in a later pass.

80.1 Goal and Motivation Fact Units

Two Fact Unit types extend the base schema (Appendix A) for this purpose:

Field	Purpose
Subject	A reference to the specific person, character, or entity this goal or motivation describes — required; a Goal or Motivation Fact Unit with no Subject is invalid.
StatedOrInferred	Whether the source material directly states this goal (“she wanted to unify the provinces”) or whether it is inferred from the Subject's documented actions and decisions — carried as a distinct field, not collapsed into a single confidence number, since a reader evaluating the claim needs to know which kind of claim it is.
ConfidenceScore	The same field every Fact Unit already carries (Appendix A), reflecting certainty in this specific attribution — lower for an inferred motivation than for one the text states directly, consistent with how confidence already works everywhere else in this Bible.
SupportingFactUnits	The specific facts — documented actions, statements, outcomes — that ground this goal or motivation attribution, the same citation discipline already required

	of any derived claim (§12, the Narrative Graph).

80.2 Representing How a Goal Was Pursued

Where the source material documents not just a goal but how a subject pursued it — adapting to setbacks, weighing options, changing course — this is represented using the Scenario Graph's existing PLANNING type (Chapter 11), not a new planning representation. A PLANNING-type Scenario Graph node, applied here, traces the Subject's actual documented decision points and the options available to them at each one, reusing WASM's Affordance layer (§53.3) to represent what actions were genuinely available in that situation — grounding “why did they choose this” in what they could actually have chosen from, not an open-ended hypothetical.

Applied to a historical figure's memoir, for instance, this would let a reader ask what specific alternatives a subject weighed before a documented decision, and get an answer grounded in what the text and its supporting facts actually establish — not a psychological profile invented to fill the gap where the source material is silent, consistent with the Without-Information Rule (D0.4) governing every other QueryBook output.

80.3 Where This Surfaces for a Reader

- Standard Q&A and Guided Exploration (§52.13.2) — a direct question about a subject's goal or motivation is answered from these Fact Units the same way any other question is answered, with the same citation discipline.
- Discussion Mode (§52.13.3) — gains grounded material to adopt a documented viewpoint from when analyzing or critiquing a subject's reasoning, rather than reconstructing that viewpoint freshly each time from scattered narrative facts.
- Tutorial Mode and Comprehension Check (§52.13.2) — a subject's documented decision-making process becomes explicit teaching material; a comprehension check can test whether a student correctly identified a documented motivation, grounded in the same Fact Units the lesson itself drew from.
- Semantic Exploration and Fact Graph Visualization (§52.13.2, §52.13.2.2) — a Goal or Motivation Fact Unit is connected through the Ontology Mesh (Chapter 6) like any other fact, so a reader can follow a subject's documented motivation into a related topic elsewhere in the same book, or — where AEM synthesis (Chapter 15) supports it — into a genuinely comparable pattern in a different work in their library, using the same cross-book thematic capability already specified in §36.5.

80.4 Integration Requirements

This chapter requires the Fact Unit schema (Appendix A), Discussion Mode (§52.13.3), the Scenario Graph and its PLANNING type (Chapter 11), WASM's Affordance layer (§53.3), the Ontology Mesh (Chapter 6), AEM synthesis (Chapter 15), and the Narrative Graph's citation discipline (Chapter 12). It modifies none of their specifications.

80.5 Summary

A subject's goals, motivations, and planning process can now be formally modeled as grounded, cited, queryable Fact Units — reusing Discussion Mode's existing viewpoint-adoption authority and the Scenario Graph's existing PLANNING type rather than introducing a new reasoning subsystem — and surfaced across Q&A, Discussion Mode, Tutorial Mode, and Semantic Exploration alike. Every representation describes a subject in the Fact Graph, never QueryBook's own state; the distinction is stated explicitly in §80's scope note because it is the entire reason this chapter is safe to specify at all.

Chapter 81: UFCS/FQL as an Optional External Resource

This chapter specifies how UFCS (Unified Fact Cognitive Substrate) and FQL (Fact Query Language) — separately owned, separately patented technology — can connect to QueryBook, following the same pattern already established for any other external system in §49.3.3: registered through the Source Connector Registry, never woven into an existing chapter's specification. This chapter is drawn from an actual patent search report describing UFCS's real technical claims, not a feature list of unverified capability.

QUERYBOOK IS COMPLETE WITHOUT THIS CHAPTER **Every capability described below has a QueryBook-native counterpart already fully specified elsewhere in this Bible, built independently and requiring nothing from UFCS/FQL to function: the Predictive Inference Engine (Chapter 10) and AEM synthesis (Chapter 15) for inference beyond directly-stored facts, the Provenance Ledger (Chapter 5) for explainable, source-traceable answers, and the Ontology Mesh (Chapter 6) for multi-entity relationship retrieval. This chapter adds an optional, separately-patented alternative resource an operator may choose to connect — it does not fill a gap QueryBook's core architecture otherwise has, and no other chapter in this Bible depends on it.**

81.1 What UFCS/FQL Actually Is

Per the patent search report this chapter is drawn from, UFCS is a search and retrieval system operating on a hypergraph substrate, with the Fact Atom — a structured, provenance-carrying semantic unit — as its retrieval unit, distinct from a document, an entity, or a generic graph node. FQL is its query language.

Capability	What It Does, Per the Source Report
Fact Atom retrieval	Structured facts retrieved as the atomic unit, carrying embedded provenance metadata, rather than retrieving documents or generic nodes and inferring structure afterward.
Hyperedge pattern matching	Multi-entity semantic relationships retrieved via pattern matching against hyperedges — a genuinely different retrieval structure than a pairwise graph edge or a linear semantic index.
Predictive fact retrieval	Facts the system infers or determines to be probable, not explicitly stored, returned alongside directly-retrieved facts.
Contradiction-aware retrieval	Contradiction clusters are a retrievable result type in their own right, not an error condition.
Domain-calibrated ranking	Ranking signals derived from domain vectors, themselves derived from Fact Atom semantic group classifications.
Explainability via hyperchain path	A result's justification is a traceable path through the hypergraph substrate, distinct from citation-based explanation.

81.2 Relationship to QueryBook's Own Architecture

Where a UFCS capability overlaps conceptually with something QueryBook already does natively, this section states the distinction directly rather than implying equivalence or gap:

- Predictive fact retrieval overlaps with the Predictive Inference Engine (Chapter 10) and AEM synthesis (Chapter 15) — both produce inferences beyond directly-stored facts, through different mechanisms (PIE's four-stream forward-projection versus UFCS's hypergraph-substrate inference). Connecting UFCS does not replace PIE or AEM; it is a second, optional resource for the same general class of problem.
- Explainability via hyperchain path overlaps with the Provenance Ledger's citation and lineage discipline (Chapter 5) — both aim at a traceable justification for a result, through different underlying structures.
- Hyperedge-based multi-entity relationship retrieval overlaps with the Ontology Mesh (Chapter 6) — both represent relationships beyond a simple pairwise link, through different formalisms.

None of these QueryBook-native mechanisms are modified by this chapter, and none require UFCS to function — the overlap is stated so an operator connecting UFCS understands they are adding a second, alternative resource, not replacing or completing something QueryBook otherwise lacks.

81.3 Connection via the Source Connector Registry

Where an operator chooses to connect UFCS, it follows §49.3.3.1's contract exactly — the same capability declaration, authentication, and data-mapping discipline as the Database, API Feed, and Autonomous Agent Connector examples already specified (§49.3.3.5–§49.3.3.6). A UFCS query issued through this connector is a directed, QBPL-compiled request (§48.3) with its result — a Fact Atom, a hyperedge match, a predictive fact, or a contradiction cluster — imported into the Fact Graph as an ordinary Fact Unit (Appendix A), carrying its UFCS provenance alongside the same ConfidenceScore, SafetyFlags, and lineage fields every other Fact Unit already carries. No new Fact Unit schema, retrieval mechanism, or explainability system is introduced into QueryBook's core to accommodate it.

81.4 What This Chapter Does Not Do

- It does not make UFCS or FQL a dependency of any existing chapter — QBQL (Chapter 18), the Ontology Mesh (Chapter 6), the Predictive Inference Engine (Chapter 10), and every other already-specified system function identically whether or not a UFCS connector is ever registered.
- It does not merge UFCS's Fact Atom or hyperedge concepts into QueryBook's own Fact Unit schema (Appendix A) — a UFCS result becomes a QueryBook Fact Unit at the connector boundary, the same translation any other external connector's data already undergoes via Data Mapping (§49.3.3.1).
- It offers no opinion on patent strategy, drafting, or claim structure for either QueryBook's or UFCS/FQL's own patent applications — that is a legal judgment for qualified patent counsel, outside what this Bible specifies or what this chapter is positioned to advise on.

81.5 Integration Requirements

This chapter requires the Source Connector Registry (§49.3.3), QBPL (Chapter 48), the Fact Unit schema (Appendix A), and the Provenance Ledger (Chapter 5). It modifies none of their specifications, and none of the QueryBook-native systems named in §81.2 are altered by this chapter's existence.

81.6 Summary

UFCS and FQL are specified here as a real, separately-owned, separately-patented external resource — accurately described from an actual patent search report rather than an unverified feature list —

connectable through the same Source Connector Registry every other external system already uses. QueryBook's core architecture requires nothing from this chapter to function, and every capability it adds already has a QueryBook-native counterpart built independently; UFCS/FQL is an optional, second resource for a class of problem QueryBook can already address on its own, not a missing piece of it.

Chapter 82: Supplemental Content and Private Messaging

This chapter closes two gaps identified directly in conversation: authors have no specified way to add supplementary, non-core material to a published book distinct from the manuscript itself, and no reader, author, or other portal user has a private, one-to-one communication channel — every existing communication surface (Chapter 64) is board-based, scoped to a book, chapter, class, or announcement context, not a private exchange between two specific people.

82.1 Supplemental Content

An author can ingest material related to their book but distinct from its core text — behind-the-scenes commentary, a film's production notes, deleted or alternate material, real-world context an author wants to add after publication — through the identical pathway already specified for the manuscript itself (§61.1.1), with one addition: a ContentRole field on the resulting Fact Units, distinguishing CORE (the published manuscript) from SUPPLEMENTAL (everything else an author chooses to attach).

SAME INGESTION, A DISTINGUISHING FIELD — NOT A SECOND BOOK **Supplemental content is not a parallel work requiring its own Rights Validation Gate pass, its own Submission Intake cycle, or its own entry in the reader-facing repository (§60.1–§60.2 apply to it exactly as they already do to core content). It is additional Fact Units, ingested through §4.2's existing pipeline, attached to the same book, and distinguished only by the ContentRole field so that a reader's default experience — core content only — is unaffected unless the reader explicitly chooses to explore supplemental material, and so that Standard Q&A (§52.13.2) can correctly attribute an answer as coming from the author's original work versus their added commentary.**

- A reader can explore supplemental content the same way they explore core content — through any Interactive Mode already specified (Chapter 52 of this book; §52.13.2 of the Bible) — with responses citing whether a given fact comes from CORE or SUPPLEMENTAL material, so a reader

always knows whether they're reading the author's original work or the author's added context about it.

- An author can add supplemental content at any time after publication without re-triggering Submission Intake for the core work — consistent with §61.1.1's principle that ingestion volume and cadence don't change the underlying pathway.

82.2 Private Messaging

A DirectMessage Fact Unit type extends the messaging capability already specified for boards (Chapter 64) to a private, one-to-one channel between two identities — a reader and an author, two readers, or two authors — rather than a board post visible to a group. A DirectMessage carries the same base Fact Unit fields as any other (Appendix A), with its AccessControlList restricted to exactly the sending and receiving identity — no board-level visibility tier applies.

- Private messaging is subject to the identical Safety Substrate enforcement (Chapter 8) as any other content in this Bible — privacy of the channel does not exempt its content from safety evaluation at ingestion.
- A recipient can block a sender, preventing further DirectMessage Fact Units from that sender reaching them — enforced at the AccessControlList level, the same mechanism already governing every other access decision in this Bible, not a separate blocking subsystem.
- Reported private-message abuse routes through the same Moderation Appeals process already specified (Chapter 75) — a human reviewer, not an automated system alone, has the final say, identical to a board moderation dispute.

PRIVATE DOES NOT MEAN UNACCOUNTABLE **A DirectMessage is provenance-tracked through the Provenance Ledger (Chapter 5) exactly like any other Fact Unit — privacy from other users is not privacy from the same accountability discipline every other consequential action in this Bible already carries. This is the same principle already established for Personal Notes (§63.3): private visibility and structural accountability are not in tension.**

82.2.1 Encryption Scope

Private messaging encryption differs by whether a channel is reachable by a student account, given the two contexts carry materially different safety requirements.

ADULT CHANNELS: TRUE END-TO-END ENCRYPTION **A DirectMessage channel between two identities, neither of which is a student account participating through Chapter 83's education context, is end-to-end encrypted: message content is encrypted by the sending client and decryptable only by the receiving client's own key. The platform does not hold a decryption key for these channels and cannot access message content under any configuration, including for Safety Substrate evaluation — §82.2's general safety-enforcement statement does not apply to a channel meeting this section's adult-only condition.**

CLASSROOM-REACHABLE CHANNELS: ENCRYPTED, NOT UNMONITORABLE **A DirectMessage channel in which at least one participant is a student account, or which is reachable through Chapter 83's education context by any path, is encrypted in transit and at rest against external interception and unauthorized access, but is not end-to-end encrypted in the sense of §82.2.1's adult-channel provision. The Safety Substrate retains the ability to decrypt and evaluate content in these channels specifically, exercised only for safety enforcement, individually logged to the Provenance Ledger with the accessing process's identity and the specific triggering reason — never a standing, unaudited, or casual access path. This is deliberately not the same guarantee as the adult-channel provision above; a channel reachable by a student retains a real safety net.**

- Channel classification is determined by account type and context at message time, not by user self-declaration — a channel reachable through Chapter 83's education context is classified as Classroom-reachable regardless of the specific participants' stated ages.
- This section modifies the general applicability of §82.2's Safety Substrate statement for adult-only channels specifically; it does not modify Chapter 8's Safety Substrate specification itself, and does not weaken safety enforcement for any Classroom-reachable channel.

82.3 Integration Requirements

This chapter requires the Fact Unit schema (Appendix A), the Multimodal Ingestion Pipeline (§4.2), §61.1.1's manuscript upload pathway, Chapter 60's Rights Validation Gate, the Safety Substrate (Chapter 8), the Provenance Ledger (Chapter 5), and Chapter 75's Moderation Appeals process. It modifies none of their specifications.

82.4 Summary

Authors can now attach supplemental, non-core material to a published book through the existing ingestion pathway, distinguished by a ContentRole field rather than a parallel publishing process; and any two identities on the platform can now exchange private messages, subject to the same safety enforcement, blocking, and moderation-appeals discipline already governing every other communication surface in this Bible.

Chapter 83: Instructor-Created Tours and Teaching Material

Tour and Exhibition Mode (§46.9) is specified as author-curated, and Supplemental Content (§82.1) is specified as something an author adds to their own book. Neither currently gives an instructor — a Reader or Author granted class-hosting privileges for a specific class under §65.3, who is not necessarily the book's own author — any way to build their own teaching material from a book they're assigned to teach but didn't write. This chapter closes that gap.

A SECOND, ATTRIBUTED LAYER — NOT AUTHOR PERMISSION, NOT AUTHOR CONTENT An **instructor-created Tour or teaching note is not the same object as an author-curated one, is never presented as if it were, and does not require the book's author to grant permission, because it changes nothing about the author's own content or its rights. It is a new sequencing and annotation of Fact Units the instructor already has reading rights to, scoped to their specific class, carrying its own mandatory attribution exactly as Presenter Mode already requires disclosure for AI-generated presence (§46.10.6).**

83.1 What an Instructor Can Build

- An Instructor Tour — a sequenced walkthrough of existing IQR scenes (Chapter 46), built the same way an author's Tour is structurally (Waypoints anchored to scenes, §46.9), but composed by the

instructor from the book's already-published Fact Graph rather than curated by the author. An Instructor Tour cannot introduce a new scene the author's own content doesn't already support — it resequences and annotates what exists, consistent with the Without-Information Rule (D0.4) governing every other QueryBook output.

- Instructor Notes — class-scoped annotations attached to specific passages or Waypoints, visible only to that instructor's class roster, distinct from both a Personal Note (§63, private to one reader) and Supplemental Content (§82.1, an author's own addition to their book).

83.2 Mandatory Attribution

Every Instructor Tour and Instructor Note carries a persistent, non-removable label identifying it as instructor-created teaching material, distinct from the author's own work — the same non-removable-disclosure discipline already established for Presenter Mode (§46.10.6), applied here so a student always knows whether they're experiencing the author's own curation or their instructor's. This label cannot be turned off by the instructor, the same way an AI-generated presenter's disclosure cannot be turned off by the author using it.

83.3 Presenter Mode Remains Author-Exclusive

An Instructor Tour may include narration, but only through the instructor's own separately-granted Presenter authorization (§46.10.1) — an instructor cannot invoke the book's author's own voice or likeness merely by virtue of teaching that author's book. Where an instructor has not authorized their own presence, the Instructor Tour runs unnarrated, consistent with §46.9's existing support for a Tour with no presenter at all.

83.4 Scope and Boundaries

- An Instructor Tour is visible only to the instructor's own class roster by default, the same visibility scoping already established for Class Boards (§64.1); an instructor may choose to make it more broadly visible, but this is an explicit choice, not a default.
- An author retains full authority over their own book's content, official Tours, and Supplemental Content regardless of how many instructors are teaching it — this chapter adds a parallel, clearly-attributed layer, not shared editorial control over the author's own work.
- Moderation of Instructor Tours and Notes follows the same discipline already specified for any other instructor-scoped content (§65.3, §64's Class Board moderation) — the instructor moderates

their own class's material, subject to the same formal appeals process (Chapter 75) available to any other moderation decision in this Bible.

83.5 Integration Requirements

This chapter requires Tour and Exhibition Mode (§46.9), Presenter Mode's authorization model (§46.10), the Instructor capability (§65.3), Supplemental Content (§82.1, referenced only to distinguish scope), Class Boards (§64.1), and the Provenance Ledger (Chapter 5) for attribution tracking. It modifies none of their specifications.

83.6 Summary

An instructor teaching a book they didn't write can now build their own attributed Tour and class-scoped notes from that book's existing, published content — without needing the author's permission, because nothing about the author's own work or rights is touched, and without ever being presented as the author's own curation, because the attribution is as non-removable as Presenter Mode's own disclosure already is. Presenter narration in an Instructor Tour remains gated by the instructor's own separate authorization, never inherited from the book's author.

Chapter 84: Session Books and Instructor-Customized Assessment

This chapter closes two genuine, narrow gaps found in a submission whose central mechanism — blockchain- and NFT-anchored provenance, described throughout as an “NFT Certification Layer” and “NFT Registry” — is not adopted, for the same reason it has never been adopted anywhere in this Bible's history: rights-tokenization and blockchain-anchored provenance were evaluated and declined during this project's earliest passes, and every subsequent reintroduction, however the framing has changed, has been declined for the same reason. What follows is translated entirely into this Bible's actual, already-working provenance mechanism — the Merkle DAG-based Provenance Ledger (Chapter 5) — which already provides tamper-evident, cryptographically verifiable provenance without blockchain connectivity, continuous or deferred.

WHY THE BLOCKCHAIN FRAMING ISN'T CARRIED FORWARD Several ideas in the source submission solve a problem QueryBook's real architecture doesn't have. “Deferred Minting Mode,” for instance, exists to let blockchain-anchored provenance work without continuous connectivity — but the Provenance Ledger's Merkle DAG (Chapter 5) never required blockchain connectivity to begin with, and offline provenance verification is already fully specified (§68's Synchronization and Offline Operation Layer). Solving a connectivity problem for a mechanism this Bible doesn't have isn't a gap; it's an artifact of evaluating the wrong architecture. The same reasoning applies to every other blockchain-anchoring variant in the source material (multi-chain anchoring, blockchain-optional mode, deferred minting) — none are carried forward, not as oversights but as a direct consequence of Chapter 5 already solving the underlying problem differently.

84.1 Session Books

A reader can create a Session Book — a temporary, task-scoped subset of the Fact Graph, gathering specific Fact Units (their own notes, specific passages, ingested reference material) around a short-term research task or project, distinct from the permanent, indefinite scope of their library and reading history (§36.8).

- A Session Book is a real Fact Unit (a SessionBookRef, per Appendix A's schema pattern) referencing the specific Fact Units gathered into it, not a copy of them — the same reference-based pattern already used for a Scenario Graph's Knowledge State Snapshot (§11).
- A Session Book has a defined lifecycle: active, archived, or purged. A reader can purge a Session Book when the task is done, removing the SessionBookRef and its gathering structure — the underlying Fact Units it referenced are untouched, since a Session Book was never their only location, consistent with §49.4's Live Fact Buffer precedent that a working, temporary structure is distinct from the durable store it draws from.
- Every Session Book action — creation, modification, purge — is itself logged through the Provenance Ledger (Chapter 5) like any other consequential action, so a purged Session Book's prior existence remains auditable even after its content references are gone.

A researcher preparing a single conference talk can gather relevant passages from six different books, their own notes, and a handful of freshly-ingested reference PDFs into one Session Book, work from it for the two weeks the talk takes to prepare, and purge it afterward — without that temporary gathering cluttering their permanent library history or requiring six separate books to be reorganized around a project that's now finished.

84.2 Instructor-Customized Assessment Templates

An instructor — the contextual capability already defined at §65.3 — can define a reusable template for Comprehension Check (§52.13.2), specifying difficulty level, question count, and structural format, applied consistently across their class and reused across sessions or semesters rather than configured freshly each time.

- A template is scoped to the instructor's class, the same visibility model already established for Class Boards (§64.1) and Instructor Tours (§83.1) — not a platform-wide assessment format.
- Every question a template-driven Comprehension Check generates is still grounded in the book's own Fact Graph exactly as any other Comprehension Check already is (§52.13.2) — a template constrains structure and difficulty, not content; it cannot cause a question to be generated about something the book doesn't actually establish.

84.3 Integration Requirements

This chapter requires the Fact Unit schema (Appendix A), the Provenance Ledger (Chapter 5), the Scenario Graph's Knowledge State Snapshot pattern (Chapter 11, referenced only as a precedent for reference-based structures), the Live Fact Buffer precedent (§49.4), Comprehension Check (§52.13.2), and the Instructor capability (§65.3). It modifies none of their specifications, and introduces no blockchain, NFT, or distributed-ledger dependency anywhere.

84.4 Summary

A reader can now gather a temporary, purgeable, task-scoped subset of their library into a Session Book, and an instructor can now define a reusable assessment template scoped to their own class — two genuine, narrow capabilities extracted from a much larger submission whose central mechanism, blockchain- and NFT-anchored provenance, is not part of this Bible and is not becoming part of it now. The Provenance Ledger's Merkle DAG already does, without blockchain connectivity of any kind, everything the source submission's certification layer was trying to achieve.

Chapter 85: Instructor Grant Visibility

This chapter addresses a real gap without changing what caused it: Instructor remains a contextual capability granted to a Reader or Author for a specific class (§65.3), not a fifth Role — that decision was reconsidered directly and reaffirmed, specifically because four already-built chapters (Chapter 62, §64.1, Chapter 83, §84.2) all depend on the capability model working exactly as it already does, and converting

to a formal Role would mean retrofitting every one of them to solve a problem the architecture doesn't actually have. What was missing instead was visibility: nothing let a person see every class they currently hold instructor privileges for in one place. This chapter adds that view, not a new Role.

A VIEW OVER EXISTING GRANTS, NOT A NEW ACCESS MODEL **Nothing in this chapter creates a new Fact Unit type, a new Role, or a new permission. It is a read-only aggregation querying the same per-class instructor grants already recorded through Chapter 62's event-hosting model, presented as a single view instead of requiring someone to already know which classes to check individually. If every underlying grant this view reads from were deleted, this chapter would have nothing left to display — it has no independent existence apart from what §65.3 already grants.**

85.1 The Instructor Dashboard

Surfaced within the Author Portal or Reader Portal (§65.2, whichever the identity's underlying Role already maps to — no new portal is introduced), the Instructor Dashboard lists every class for which the viewing identity currently holds an active instructor grant, querying the existing grant records through QBQL (Chapter 18) rather than a new aggregation mechanism.

Dashboard Field	Source
Class name and associated book	The instructor grant record itself (§65.3, via Chapter 62's event model).
Class Board activity summary	§64.1's existing Class Board, queried for the specific class — not a new messaging surface.
Instructor Tours authored for this class	§83.1's existing Instructor Tour records, filtered to the class in view.
Assessment templates defined for this class	§84.2's existing template records, filtered to the class in view.
Grant status	Active, or expired — read directly from the underlying per-class grant; this view does not extend, renew, or alter a grant's duration.

- A grant that has expired or been revoked simply stops appearing as active — the dashboard reflects the grant's real state rather than maintaining its own separate record of who is or isn't currently an instructor.
- An identity with no active instructor grants sees an empty dashboard, not an error — holding zero grants is a normal, expected state for a Reader or Author who has never taught a class.

85.2 What This Chapter Deliberately Does Not Add

- No Instructor Role, and no change to the Role model already specified in Chapter 65.

- No new permission — a person's ability to create an Instructor Tour, moderate a Class Board, or define an assessment template is governed entirely by §65.3's existing grant, exactly as before this chapter was written.
- No cross-class aggregation of content itself — the dashboard lists and links to each class's material; it does not merge two different classes' boards, Tours, or templates into a combined view, since those remain genuinely separate, per-class-scoped objects (§65.3, §64.1, §83.1).

85.3 Integration Requirements

This chapter requires the Instructor capability (§65.3), Chapter 62's event model, the Portal architecture (§65.2), QBQL (Chapter 18), Class Boards (§64.1), Instructor Tours (§83.1), and Instructor-Customized Assessment Templates (§84.2). It modifies none of their specifications.

85.4 Summary

A person holding instructor grants across multiple classes can now see all of them in one place — what's being taught, what boards are active, what Tours and assessment templates exist for each — without QueryBook adding a fifth Role, a new portal, or any new permission to get there. The complexity this chapter avoids is the same complexity §65.3 was originally written to avoid: per-class scoping that a formal Role would have needed to reimplement from scratch.

Chapter 86: System Identity and Positioning

This chapter consolidates QueryBook's real, already-specified differentiators into a single canonical reference, correcting a set of overclaims and mischaracterizations that arose in an external summary of the system and were declined from adoption. Every claim below cites the specific chapter it comes from; nothing here introduces new architecture.

86.1 What QueryBook Is

QueryBook is a neuromorphic, fact-centric, multimodal knowledge operating system (Chapter 1) — not a database, not a vector store, not a chatbot, and not a large language model with retrieval bolted on. Its foundational unit is the Fact Unit (Chapter 3, Appendix A): a discrete, typed knowledge record combining a confidence score, an inseparable safety classification, a language-independent semantic fingerprint, an embedding representation, temporal anchoring, and a cryptographically verifiable provenance chain in a single schema.

86.2 The Real Differentiators

Differentiator	What It Actually Is
Structural groundedness	Output vocabulary at generation time is constrained to terms traceable to specific Fact Units; an ungrounded proposition is rejected before delivery rather than made merely less likely (Chapter 16, §6.4). This is a structural property of the generation pipeline, not a claim that no ungrounded output can ever occur — the claim is scoped to what the pipeline generates, stated precisely rather than as an absolute guarantee.
The Dual-Brain Architecture	The parallel operation of the Semantic Brain (work-specific, language-anchored, reversible) and the World Brain (domain-general, language-agnostic, irreversible post-Flush), implementing a two-level Predictive Coding hierarchy (§41.11.2, Chapter 58). Not “meaning-level vs. world-model reasoning” — that phrase does not appear in, and does not accurately describe, this architecture.
Human-gated self-improvement	The system can identify and propose changes to itself, but a named, accountable human must approve every change before it takes effect, and a fixed, permanently-excluded set of subsystems — including the safety and governance mechanisms themselves — can never be a valid proposal target, from any source (Chapter 66, Chapter 78, Chapter 79).
Bounded natural-language programming	NLPL compiles a natural-language instruction into QBPL's fixed, nine-primitive, non-Turing-complete instruction set (Chapter 48) — a genuinely novel capability, but scoped to Fact Graph operations, not a general command language for controlling arbitrary external devices.
Directed delegation to external systems	Where QueryBook connects to an external agent platform or device, it dispatches specific, declared actions through a registered connector (§49.3.3.6) — it does not acquire that platform's own general authority. This is the accurate description of QueryBook's relationship to robotics and external agents; broader framings describing native “control” or “coordination” authority are not accurate.

86.3 Common Mischaracterizations, Corrected

- “No hallucinations” — not the claim this Bible makes anywhere. The actual claim is the structural groundedness property in §86.2, stated with its real scope, not as an absolute guarantee.
- “Encrypted Fact Units” — not a real property of the Fact Unit schema. History encryption is a real, opt-in reader feature (§36.8) protecting a reader's own session data, not a categorical property of every Fact Unit.

- “Teacher-in-a-box” — the real Instructor capability (§65.3, Chapter 83) is explicitly additive to human instruction: an instructor's own attributed layer on top of a book, never a replacement for a human teacher.
- “Control robots using natural language,” “coordinate multi-robot fleets,” “scan multi-camera feeds for hazards and behaviors” — these describe general, unscoped authority over physical devices and surveillance systems that does not exist in this Bible. The real capability is §49.3.3.6's directed delegation of specific, declared actions, with no native or general authority acquired.

86.4 Integration Requirements

This chapter requires Chapter 1, Chapter 3 and Appendix A, Chapter 16, §41.11.2 and Chapter 58, Chapters 66/78/79, Chapter 48, §36.8, §65.3 and Chapter 83, and §49.3.3.6. It modifies none of their specifications — it is a consolidated reference to what they already establish, written specifically to be the canonical source for how QueryBook is summarized elsewhere.

86.5 Summary

Every differentiator in this chapter is real and already specified elsewhere in this Bible; every correction in §86.3 addresses a specific overclaim or mischaracterization that was proposed and declined. Future summaries of QueryBook — in the Bible, in derived books, in patent drafts, or elsewhere — should draw from this chapter rather than from external descriptions that have not been checked against it.

86.6 Correction — Native Hypergraph Structures in WASM

A PRIOR CORRECTION IN THIS CHAPTER WAS ITSELF PARTIALLY WRONGThis section corrects an error made during this chapter's own drafting, not a resubmitted external claim. A declined submission's phrase, “the World Brain stores knowledge as hypergraph conceptual models,” was rejected on the stated grounds that hypergraph structures belong exclusively to UFCS (Chapter 81) and are not part of QueryBook's own architecture. That was incorrect. QueryBook's own World Aware System Model (Chapter 53) natively and independently uses hypergraph structures — the error was concluding, from UFCS's real hypergraph usage, that no other real hypergraph usage existed, without checking thoroughly enough to find WASM's.

The World Model Domain (Chapter 53) is explicit and unambiguous on this point, stated as one of its own non-negotiable design constraints: all world model structures are stored as typed nodes and hyperedges within the Fact Universe, with no external world model database used. Cross-layer relations

among WASM's six structural layers are expressed as typed hyperedges specifically because a simple binary edge cannot represent them — multi-entity semantic relationships are retrieved via pattern matching against hyperedges, a genuinely different retrieval structure than a pairwise graph edge.

- Two real, separate hypergraph implementations exist in this Bible: UFCS's own hypergraph substrate (Chapter 81), external and separately-owned; and WASM's native hypergraph structures within the Fact Universe (Chapter 53), fully QueryBook's own and independent of any UFCS connection. Neither depends on the other.
- Describing QueryBook's World Brain / World Model Domain as using hypergraph structures is therefore accurate, when attributed to WASM's native architecture. It is not accurate only when hypergraph structures are attributed to UFCS as though QueryBook had no native equivalent — that attribution was the actual error, not the underlying technical claim.

This correction does not extend to the other elements of the same originally-declined paragraph. Unscoped robotics control (“operate robotics through embodied cognition”) and autonomous goal formation remain correctly declined, on their own separately-verified evidence, and this section makes no change to either.

Chapter 87: Live Interaction Voice Signal Analysis

This chapter closes a specific gap: prosodic and sentiment analysis of speech already exists for ingested audio content (§4.3.4), but nothing in this Bible analyzes the voice of a live, present user during an active interactive session. This chapter specifies that capability, scoped narrowly and gated on the analyzed user's own explicit, revocable consent — consent is required here specifically because the subject is a live, present, real person, not documented or fictional content already governed by §4.3.4 and Chapter 60's rights model.

LIVE CONSENT IS DIFFERENT FROM INGESTED-CONTENT ANALYSIS, AND IS REQUIRED HERE **Analyzing prosody and sentiment in a movie's dialogue or an audiobook's narration (§4.3.4) requires no additional consent beyond the existing Rights Validation Gate (Chapter 60) that already governs whether that content may be ingested at all — the subject is documented, fictional, or historical content, not a live participant in the current session. Analyzing a present, live user's own voice during an active interaction is a different act entirely, and requires that specific user's own explicit, revocable authorization before any analysis of their voice occurs — the same authorization discipline already established for Presenter Mode (§46.10.1), adapted here for analysis input rather than generation output.**

87.1 Scope

- This chapter covers only the voice of the live, present user currently interacting with QueryBook in a real-time session. It does not add any new analysis capability for ingested audio/video content — that capability already exists at §4.3.4 (sentiment and prosody analysis from speech) and requires no change here.
- This chapter analyzes prosodic and acoustic features — tone, pace, pitch variation, pause patterns — using the same extraction technique already specified at §4.3.4, applied to a live audio stream instead of an ingested recording. It does not introduce a new signal-processing mechanism.

87.2 Consent Architecture

- Live voice analysis is opt-in per session, disabled by default. A user must explicitly enable it before any analysis of their voice occurs.
- Consent is revocable at any point during the session; revocation takes effect immediately and halts further analysis for the remainder of that session.
- The user is told plainly, before opting in, what is analyzed (prosodic and acoustic features of their voice) and what it is used for (§87.3) — not a general-purpose emotional-surveillance capability, a specific, disclosed, session-scoped one.
- No live voice analysis Fact Unit is retained beyond the session in which it was generated, unless the user separately and explicitly opts into longer retention — consistent with the Safety Substrate's default-minimal-retention posture (Chapter 8).

87.3 Use of the Resulting Signal

- Internal comprehension signal: a detected shift in prosodic pattern consistent with confusion or frustration may be added, alongside the Clarification Generation Engine's existing query-ambiguity and coreference-failure triggers (§37.4), as an additional signal that a clarification request may help — reusing the CGE's existing mechanism rather than building a parallel one. A prosodic signal alone, with no corresponding ambiguity in the query itself, does not by itself change what answer is given — it only affects whether a clarifying question is offered.
- Surfaced answer: where a reader directly asks about a detected vocal quality in their own current session (for example, confirming whether the system detected hesitation), the response carries the

same ConfidenceScore field every other Fact Unit carries (Appendix A), presented explicitly as an inference rather than an assertion of the user's actual internal state — consistent with this Bible's practice of never claiming certainty about something that is inherently uncertain.

87.4 Safety Classification

Every Fact Unit produced by live voice analysis carries a SafetyFlags classification (Appendix A) marking it as derived from live biometric-adjacent signal, evaluated at the same mandatory Safety Substrate gates as any other Fact Unit (Chapter 8) — no exemption is created by this chapter.

87.5 Integration Requirements

This chapter requires §4.3.4's existing prosodic extraction technique, the Clarification Generation Engine and its existing trigger conditions (§37.4), the Fact Unit schema (Appendix A), and the Safety Substrate (Chapter 8). It modifies none of their specifications — CGE gains an additional, optional trigger input; its existing triggers and behavior are unchanged.

87.6 Voice Analysis Slice

Following the extension pattern already established at §69.2 and demonstrated at §69.2.1 (Scheduler Slice, Recovery Slice) and §88.8 (Commerce Slice), this section names a Voice Analysis Slice measuring operations within this chapter, without modifying Chapter 69's own specification.

Category	Measures Operations In	Example
Voice Analysis Slice	Live Interaction Voice Signal Analysis (Chapter 87)	One utterance's prosody analysis cycle, from consented audio capture through inference result.

- Priority: Voice Analysis Slice is strictly lower priority than every safety-relevant category. Consistent with Chapter 69's existing priority ordering (Safety Gate and Recovery Slices always scheduled first), a Voice Analysis Slice is never permitted to delay, preempt, or compete for the same processing window as a Safety Gate Slice, a Recovery Slice, or any grounded-answer generation — this priority relationship is a design requirement, not an operator-configurable setting.
- Failure handling: where a Voice Analysis Slice fails to complete within its bounded envelope, no prosody inference is surfaced for that utterance; the underlying Q&A interaction proceeds normally and is never blocked, delayed, or degraded by a failed or slow voice analysis cycle.

87.7 Summary

A live, present user's voice may now be analyzed for prosodic and sentiment signal during an active session, strictly opt-in and revocable, feeding the existing Clarification Generation Engine as an additional signal rather than a new decision-making system, and never presented as a certain reading of the user's actual internal state. Ingested-content voice analysis required no new architecture and none was added; only the live-user gap was genuinely open.

Chapter 88: Contextual Commerce Layer

This chapter specifies an optional, operator-configured layer that surfaces commercial content — author/publisher-linked products or third-party affiliate content — based on the concept(s) a query resolves to, structurally separated from the grounded answer itself. This chapter exists specifically to allow commerce and advertising in the design without weakening the structural-groundedness guarantee of Chapter 16, or the Prime Directive of Domain 0.

STRUCTURAL SEPARATION, NOT POLICY SEPARATION **The grounded answer's Fact Unit retrieval, ranking, and ConfidenceScore computation are completed and finalized before the Contextual Commerce Layer is invoked at all. The commerce layer receives the query's resolved concept_id(s) as read-only input and has no write path back into retrieval, ranking, or generation. This is a structural guarantee enforced by pipeline ordering, not a policy instruction that could be misconfigured or bypassed.**

88.1 Trigger Mechanism

A query's resolved concept_id(s) — produced by the same Ontology Mesh classification already used for retrieval (Chapter 6, §19 of the patent) — are matched against a commerce content registry to identify candidate commercial content. No new classification mechanism is introduced; the trigger reuses concept resolution that already happens for every query regardless of whether this layer is enabled.

88.2 Commerce Source Types

- Author/Publisher-Linked: an author or publisher registers their own related titles or products against specific concept_ids within their own published content's scope, using the same rights-holder identity already established for that content (Chapter 60).
- Third-Party Affiliate/Ad Network: an operator registers an external commerce or advertising network through the same capability-declared connector architecture used for any other external

system (§49.3.3), receiving concept_id-scoped requests and returning candidate commercial content, never given access to the query text, the grounded answer, or any Fact Unit.

An operator may enable either source type independently, both simultaneously, or neither — this chapter is fully inert by default in every deployment, consistent with the opt-in-by-default posture already applied to §36.1.1 (ZKE) and Chapter 87 (voice signal analysis).

88.3 Presentation Requirement

Commercial content is rendered in a distinct, labeled region of the response, never interleaved into the grounded answer's own sentences or citations. The label distinguishes the content's source (author/publisher vs. third-party) to the reader. No commercial content may be presented in a form indistinguishable from a Fact-Unit-grounded assertion.

88.4 Operator Visibility and Audit Log

Every commercial content display event is logged, queryable by the operator, and includes: the triggering concept_id(s); the specific commercial content displayed; its source type (author/publisher or third-party network) and, where third-party, the registered network's identity; and the display timestamp. This follows the same logged-with-identity-and-reason discipline already applied to moderation actions (§64.4) and improvement-proposal decisions (Claim 9) — full operator visibility is a design requirement, not an optional enhancement.

88.5 Non-Interference Guarantee

No commercial signal — including payment, bid, or affiliate-fee information — is a permitted input to Fact Unit retrieval, the Attractor Network's convergence process, ConfidenceScore calculation, or the grounded generation pipeline's vocabulary constraint, under any operator configuration. This guarantee holds regardless of which commerce source type is enabled, and is not configurable by the operator — unlike §88.2's source-type selection, this section is not a deployment choice.

88.8 Commerce Processing Slice

Following the extension pattern already established at §69.2 and demonstrated at §69.2.1 (Scheduler Slice, Recovery Slice), this section names a Commerce Slice measuring operations within this chapter, without modifying Chapter 69's own specification.

Category	Measures Operations In	Example

Commerce Slice	Contextual Commerce Layer (Chapter 88)	One concept-match lookup against the commerce content registry; one external affiliate-network round-trip (§88.2).

- Priority: Commerce Slice is strictly lower priority than every safety-relevant category. Consistent with Chapter 69's existing priority ordering (Safety Gate and Recovery Slices always scheduled first), a Commerce Slice is never permitted to delay, preempt, or compete for the same processing window as a Safety Gate Slice, a Recovery Slice, or the grounded-answer generation itself — this priority relationship is a design requirement, not an operator-configurable setting.
- Boundedness: a Commerce Slice measures exactly one concept-match-to-display cycle — lookup, optional external round-trip, and selection (§88.9) — following the same fixed-envelope, four-phase lifecycle pattern already established for every other Slice Category in Chapter 69.
- Failure handling: where a Commerce Slice fails to complete within its bounded envelope — for example, an unresponsive third-party network — the commerce region of the response is omitted silently from the reader's perspective; a failed or slow Commerce Slice never delays, blocks, or degrades delivery of the grounded answer, which has already completed independently of this chapter per §88's non-interference guarantee.

88.9 Commercial Content Selection Weighting

Where more than one candidate commercial item matches a query's resolved concept identifier — for example, an author-linked title and a third-party affiliate product both registered against the same concept — a selection weighting determines which candidate, or candidates, are displayed. This section specifies that mechanism.

- Weighted factors: relevance (the specificity of the match between the candidate's registered concept scope and the query's resolved concept identifier), a source-priority weight (operator-configurable, permitting an operator to prefer author/publisher-linked content over third-party content or vice versa), and, for third-party sources specifically, a bid or revenue-share value supplied by the registered affiliate network at request time.
- Operator configurability: the specific numeric weight assigned to each factor is operator-configurable per deployment, since commerce and monetization strategy legitimately varies by operator. What is not configurable is the timing guarantee of §88 — selection weighting operates only after the grounded answer has already completed, over candidates already gathered via

§88.1's concept match, and has no access to and no effect on the grounded answer's own Fact Units, ranking, or ConfidenceScore.

MONETIZATION WEIGHTING IS SCOPED ENTIRELY WITHIN THE COMMERCE LAYER **A bid value, revenue-share percentage, or source-priority weight is a valid input to which commercial candidate is selected. It is never a valid input, under any operator configuration, to which Fact Unit is retrieved, how a Fact Unit is ranked, or what a Fact Unit's ConfidenceScore is. §88.5's non-interference guarantee governs this section exactly as it governs §88.1–§88.4.**

- Logging: the selection weighting's outcome — which candidate was selected and the weighted factors contributing to that selection — is included in the same operator-queryable audit log already required at §88.4, so an operator tuning monetization performance has the same visibility into selection as into triggering.

88.10 Commerce Optimization Proposals

Ongoing optimization of commerce targeting, bidding, and placement is a real capability of this layer — but it is reached exclusively through the Improvement Proposal Pipeline already specified at Chapter 66, never through a parallel or shortened path. This section states how the two connect; it modifies neither Chapter 66's mechanism nor Chapter 88's own non-interference guarantee.

NO SHORTCUT, ON THE SAME TERMS AS EVERY OTHER SUBSYSTEM **A proposal to change commerce targeting weights, source-priority defaults, or bidding behavior is a Proposal Fact Unit under §66.2, evaluated in the Simulation Sandbox (§66.3), subject to Automated Pre-Screening (§66.4), and requires the same named, individually-accountable human approval at the Human Review Gate (§66.5) as any other proposal in this Bible. There is no commerce-specific fast path, no autonomous canary rollout, and no agent authority to deploy a change directly — deployment occurs only through §66.6, after human approval, on the same terms as every other approved proposal.**

- Candidate identification: a subsystem-scoped monitoring agent, following the same pattern already specified at Chapter 78, may analyze Commerce Slice telemetry (§88.8) and selection-weighting outcomes (§88.9) to identify a candidate optimization — for example, a proposed adjustment to a source-priority default or a targeting-relevance threshold. Consistent with Chapter 78, this agent has no self-approval or peer-approval authority of any kind.
- Proposal scope: a Commerce Optimization Proposal's scope is limited to parameters already specified within this chapter — the weighted factors of §88.9, source registration of §88.2, and

Commerce Slice scheduling parameters within the bound already fixed at §88.8. A proposal scoped to retrieval, ranking, generation, or any Fact Unit is not a valid Commerce Optimization Proposal and is rejected at Automated Pre-Screening — this was already structurally impossible under §88.5, and this section does not weaken that guarantee, only states the pipeline explicitly.

- Permanent exclusion, stated here and cross-referenced from Chapter 66: the non-interference guarantee of §88.5 and the priority ordering of §88.8 (that a Commerce Slice is never scheduled ahead of a safety-relevant compute-accounting unit) join Chapter 66's own §66.8 Permanent Exclusions on identical terms — never a valid Proposal Scope, under any Fitness Score, Risk Profile, or reviewer judgment, regardless of the commercial benefit a proposal might otherwise demonstrate.

88.11 Integration Requirements

This chapter requires the Ontology Mesh's concept classification (Chapter 6, §19 of the patent), the Source Connector Registry (§49.3.3), and the rights-holder identity established in Chapter 60. It modifies none of their specifications, and modifies nothing in the grounded generation pipeline of Chapter 16 — the pipeline completes its own operation with no awareness this chapter exists.

88.12 Merchandise Commerce and Fulfillment Handoff

This section closes the previously-pending general-merchandise item by scoping it narrowly: merchandise — author/publisher-branded or third-party, at the operator's choice per deployment, per §88.2 — is a valid commerce content type under this chapter, subject to the same structural separation, source labeling, and audit logging already specified for every other commerce content type. What makes merchandise different is payment and fulfillment, addressed directly below.

QUERYBOOK IS THE REFERRING PARTY, NEVER THE PAYMENT PROCESSOR OR FULFILLMENT OPERATOR For both author/publisher-branded and third-party merchandise, the actual purchase transaction — payment capture, sales-tax handling, inventory, shipping, customs, and returns — is completed entirely on an external, independently-operated commerce platform's own infrastructure, reached through a handoff from the displayed commerce region. QueryBook's own role ends at the referral. QueryBook does not capture, transmit, or store payment card data at any point, does not act as a merchant of record, and does not operate fulfillment or shipping logistics under any operator configuration. This is not a configurable choice — unlike §88.2's source-type selection, an operator cannot enable a QueryBook-native checkout in place of this handoff model.

- Handoff mechanism: reuses the Source Connector Registry (§49.3.3) already specified for every other external system — a registered commerce platform (an author's own storefront, a publisher's e-commerce system, or a general third-party platform) receives a scoped referral request and returns a handoff destination; QueryBook plays no role in what happens after the handoff completes.
- Author/publisher-branded merchandise: registered against specific concept_ids within that rights holder's own content scope, using the same rights-holder identity already established in Chapter 60 — no new identity or authorization mechanism is introduced.
- Third-party general merchandise: registered as a third-party affiliate source under §88.2, subject to the same non-interference guarantee (§88.5) and Commerce Slice metering (§88.8) as any other third-party commerce content — general merchandise is not a special case requiring new architecture, only a new content category within what already exists.
- Liability and compliance boundary: because payment and fulfillment never touch QueryBook's own infrastructure, payment-card-industry compliance, money-transmission licensing, sales-tax collection, and shipping/customs liability remain the responsibility of the registered external commerce platform completing the transaction — consistent with the same requesting-party-only posture already established for the blockchain ownership-verification connector (§49.3.3.8).

88.13 Summary

Commercial content may now be surfaced based on query subject matter, from either author/publisher or third-party sources at the operator's choice, fully logged and auditable, structurally barred from influencing the grounded answer by pipeline ordering rather than by policy alone.

88.13 VR-Specific Commerce Presentation

This section closes the VR-specific ad placement exclusion flagged at Chapter 89's original scoping and left deferred since. Commercial content within a VR scene follows exactly the same mechanism already specified at §88.1–§88.5 — concept-triggered matching, presentation as a distinct, labeled region, full audit logging, and the same non-interference guarantee — with VR treated as one more presentation surface, not a separate commerce mechanism.

A PRESENTATION SURFACE, NOT A NEW TARGETING SIGNAL This section adds VR as a place commercial content may be rendered; it adds no new signal for selecting what commercial content to render. Gaze direction, dwell time, spatial proximity within a scene, or any other VR-specific interaction pattern is not used as a targeting input under any configuration of this section — the same concept_id-triggered matching already specified at §88.1 governs selection, identical to every other surface. The reader-profile-based targeting exclusion stated at Chapter 89's original scoping remains in effect and is not reopened by this section.

88.13.1 Presentation Requirement in a 3D Scene

Commercial content within a VR scene renders as a spatially and visually distinct region — never overlaid on, blended into, or occluding the immersive scene content itself — consistent with §88.3's existing requirement that commercial content never be presented in a way that could be mistaken for grounded output, extended here to mean it may also never be mistaken for authored scene content.

88.13.2 Integration Requirements

This section requires §88.1–§88.5's existing trigger, presentation, audit, and non-interference mechanisms, and Immersive 3D Scenes (Chapter 46). It modifies none of their specifications.

Chapter 89: Native QueryBook Reader

This chapter specifies the Native QueryBook Reader: the deployment embodiment through which a reader directly experiences the capabilities already specified elsewhere in this Bible. It introduces no new reasoning, retrieval, or generation mechanism — every capability described here is a rendering or interaction surface over a system already fully specified in an earlier chapter, cited explicitly rather than restated.

A DEPLOYMENT EMBODIMENT, NOT A NEW CAPABILITY Every feature in this chapter reuses an already-specified mechanism. Where this chapter says a reader taps a fact to see its citation, that citation is produced by the grounded generation pipeline of Chapter 16; where it says a reader asks a question in natural language, that question is answered by the retrieval and generation architecture of Chapters 3 and 7. This chapter's own content is limited to how those existing capabilities are surfaced to a reader, not what they do.

89.1 Standards Conformance

The Native Reader is a conformant EPUB 3 Reading System under the EPUB Reading Systems 3.3 specification, rendering content authored to the EPUB 3.3 core specification. QueryBook-specific enhancements are carried as ignorable custom data attributes (§89.1.1) on top of standard, valid EPUB 3.3 markup — a Native Reader feature never requires content that is invalid or non-conformant under the base standard.

- EPUB Accessibility 1.1, now formally part of the EPUB 3.3 standard rather than a separate specification, is satisfied jointly by standards-conformant markup and the Accessibility Suite already specified at Chapter 71 — the two requirements are treated as one conformance target, not two.
- QueryBook-authored EPUB 3.3 publications remain fully valid, renderable content in any conformant third-party Reading System; a non-QueryBook-aware reader displays the underlying content correctly and silently ignores the QueryBook-specific attributes it does not recognize.

89.1.1 QueryBook Data Attributes

Attribute	Purpose
data-qb-fact-id	Anchors a span of content to the specific FUID (Appendix A) it is grounded in, enabling inline citation and the fact-preview interaction of §89.2.
data-qb-entity-id	Anchors a span of content to an Ontology Mesh concept_id (Chapter 6, §20 of the patent), enabling the diagram and relationship interactions of §89.3.
data-qb-timeline-id	Marks content contributing to the Timeline Reconstruction view of §89.3, resolved from each anchored Fact Unit's TemporalAnchor field.

89.2 Fact-Aware Content Rendering

Content is rendered from the standard EPUB 3.3 document structure, with every reader-facing interaction traceable back to already-specified mechanism rather than introduced here.

- Standard Q&A (§52.13.2): a reader's natural-language question is answered through the existing retrieval and grounded-generation pipeline, with the response citing the specific Fact Units it was assembled from.
- Inline Fact Previews and Contextual Footnotes (§52.13.2.1–.2): tapping or hovering a data-qb-fact-id anchor surfaces the underlying Fact Unit's summary and provenance without leaving the reading surface.

- Guided Exploration and Semantic Exploration (§52.13.2): multi-turn, context-aware traversal of the Fact Graph and Ontology Mesh already specified at Chapter 6 and §20 of the patent.
- Discussion Mode (§52.13.3): grounded, source-bound viewpoint reasoning, rendered as an alternate reading mode rather than a separate application.

89.3 Timeline and Diagram Views

Two structured, non-linear views over the same underlying Fact Graph a reader is already reading linearly.

- Timeline Reconstruction (§52.13.2): chronological ordering of data-qb-timeline-id-anchored content, resolved from each Fact Unit's TemporalAnchor field — no new temporal representation is introduced.
- Fact Graph Visualization (§52.13.2.2): a direct, navigable rendering of the Ontology Mesh's real graph structure (Chapter 6) local to the reader's current position, not a separate or simplified representation of it.

89.4 NLPL Command Interface

A command surface accepting natural-language instructions compiled through the existing NLPL-to-QBPL pipeline (Chapter 48) — never through FQL, which remains UFCS's separately-owned technology and is not part of the Native Reader's own architecture regardless of whether an operator has registered an optional UFCS connection (Chapter 81). A command such as “summarize this chapter” or “show all facts about X” compiles to the same bounded, non-Turing-complete program representation already specified at §48.3 and §48.5, subject to the same Verification Gate (§51.4) as any other QBPL program.

89.5 Adaptive Learning Integration

- Tutorial Mode and Comprehension Check (§52.13.2): sequenced instruction and auto-generated assessment drawn from the same Fact Graph as ordinary reading, not a separate content track.
- Flashcard Generation (§52.13.2.1): single-Fact-Unit derivation, available inline from any fact-anchored span.

89.6 Personal Annotation and Social Features

- Personal Notes (Chapter 63) and Social Annotations (§52.13.2.1): private and shared annotation, respectively, scoped exactly as already specified — the Native Reader introduces no new sharing or visibility model.
- Message Boards and Private Messaging (Chapter 64, §82.2): book-, chapter-, and class-scoped discussion, rendered as a panel alongside the reading surface rather than a separate application.

89.7 Reader History and Personalization

- Reader History Interface (§36.8): the four existing aggregation views (Timeline, By-Book, Thematic Map, Annotation Library), ZKE-respecting per §36.1.1 — the Native Reader adds no new history mechanism, only its display surface.
- Presentation-layer personalization is governed by the Natural Language Interaction Layer (Chapter 50): the underlying Fact, its confidence score, and its provenance are never altered by personalization — only phrasing and structure adapt.

89.8 Commerce Integration

Where an operator has enabled the Contextual Commerce Layer (Chapter 88), the Native Reader renders its output exactly as that chapter specifies: a distinct, source-labeled region separate from the reading surface, triggered by the query's resolved concept identifier, never by a persistent reader profile. The Native Reader introduces no additional commerce mechanism, no reader-profile-based targeting, and no VR-specific ad placement — both were identified as requiring their own scoping and neither is included here.

89.9 Offline and Synchronized Operation

The Native Reader operates under the synchronization modes already specified at Chapter 68, including fully disconnected operation against a locally-held Fact Graph, with query execution, provenance verification, and access control continuing to function exactly as specified there. No reader-specific synchronization behavior is introduced.

89.10 Event Telemetry

Reader interaction events (page position, dwell, fact-anchor activation, NLPL command issuance) are measured under the existing Processor Slice Measurement Model (Chapter 69), following the same

extension pattern already used for the Commerce Slice (§88.8): a Reader Interaction Slice is named here, scheduled below every safety-relevant compute-accounting category, without modifying Chapter 69's own specification.

89.11 Integration Requirements

This chapter requires EPUB 3.3, EPUB Reading Systems 3.3, and EPUB Accessibility 1.1 conformance; the Fact Unit schema (Appendix A); the Ontology Mesh (Chapter 6, §20 of the patent); NLPL and QBPL (Chapter 48); the Accessibility Suite (Chapter 71); Personal Notes and Message Boards (Chapters 63–64); the Reader History Interface (§36.8); the Natural Language Interaction Layer (Chapter 50); the Contextual Commerce Layer (Chapter 88); the Synchronization and Offline Operation Layer (Chapter 68); and the Processor Slice Measurement Model (Chapter 69). It modifies none of their specifications.

89.12 Summary

The Native Reader is fully specified as a standards-conformant rendering and interaction surface over capabilities already established elsewhere in this Bible. No new reasoning, retrieval, generation, commerce, or personalization mechanism is introduced in this chapter; every feature is a citation to where its real specification already lives.

Chapter 90: Deployment Modes and Domain Packages

This chapter specifies three deployment modes for the same logical architecture, and a Domain Package mechanism for curating scoped content, fact, language, and commerce bundles per environment. It introduces one new mechanism (Domain Packages, §90.3) and otherwise reuses already-specified architecture, cited rather than restated.

NO NEW LICENSE TYPE, NO NEW SYNC MECHANISM, NO NEW FACT SERVICE **This chapter does not introduce a Classroom or Institution license type, a new synchronization mechanism, or a service parallel to the native Fact Service already specified at Chapter 3. Multi-user and classroom scenarios are supported by the existing SeatCount mechanism (§67.2) combined with the existing Instructor grant (§65.3); synchronization reuses Chapter 68 exactly as specified; and every deployment mode operates on QueryBook's own native Fact Unit architecture, never a UFCS-backed one — UFCS remains the optional, external, non-dependency resource specified at Chapter 81.**

90.1 Three Deployment Modes

Mode	Where It Runs	Illustrative Use Case
Cloud SaaS	Centrally hosted, multi-tenant	Globally connected institutions
Local Server	On-premises or edge server	A campus, school, or community network
Single Device	Laptop, tablet, or phone	An individual reader, teacher, or remote learner

All three modes run the same logical services specified elsewhere in this Bible — the native Fact Service (Chapter 3), NLPL and QBPL (Chapter 48), the Reader (Chapter 89), License and Entitlement Management (Chapter 67), and the Synchronization and Offline Operation Layer (Chapter 68) — differing only in whether each service is fully hosted, partially hosted with optional cloud sync, or embedded on-device. This is a deployment-topology distinction, not an architectural fork: no service's own specification differs between modes.

90.2 License Enforcement Across Deployment Modes

Every deployment mode enforces the same License and Entitlement mechanism already specified at Chapter 67 — a License Fact Unit per License Owner or organization, carrying a LicenseType matching one of Chapter 65's four Roles (Reader, Author, Publisher, Partner), with SeatCount and SeatAllocation governing organizational, multi-user access.

- A classroom is not a distinct license type: it is a Reader License with a SeatCount sized to the class, combined with the existing, separately-granted Instructor capability (§65.3) held by the teacher's own seat. No new LicenseType is introduced.
- An institution is the same pattern at larger scale: a larger SeatCount, potentially spanning multiple LicenseTypes for different departments, each still governed by Chapter 67's existing seat-assignment and audit mechanism (logged to the Provenance Ledger, Chapter 5).
- In Local Server and Single Device modes, the same License Fact Unit and seat-assignment records are held locally rather than centrally, reconciled with the central record through the Synchronization and Offline Operation Layer (Chapter 68) on reconnection — enforcement logic itself does not differ by mode.

90.3 Domain Packages

A Domain Package is a versioned, signed, installable bundle scoping QueryBook's existing capabilities to a specific environment's actual needs, closing the real gap between a full platform install and a genuinely offline-capable, bandwidth-appropriate deployment for a specific community, campus, or institution.

90.3.1 Package Composition

- Content scope: a defined subset of already-ingested Fact Units and their source works, selected for the target environment rather than requiring the full platform catalog.
- Language scope: the target environment's language selection within the already-specified Multilingual Regeneration Engine (Chapter 16) — a package does not introduce new translation capability, only selects which of MRE's already-supported languages are provisioned locally.
- Commerce scope (optional): where an operator has enabled the Contextual Commerce Layer (Chapter 88), a package may include a local commerce-source registration — for example, region-specific author or third-party affiliate content — subject to Chapter 88's non-interference guarantee and audit logging exactly as already specified. A package introduces no new commerce mechanism and no targeting model beyond the concept-identifier-based targeting Chapter 88 already specifies.

90.3.2 Package Integrity and Installation

- Every package is versioned and cryptographically signed; installation verifies the signature before activation. This is a standard software-integrity control, not a rights-management or DRM mechanism — consistent with Chapter 67's explicit boundary against introducing DRM, territory, or pricing machinery.
- A package is installable identically in any of the three deployment modes (§90.1): centrally in Cloud SaaS, locally in Local Server mode, or embedded in Single Device mode.
- Package activation for a specific user or classroom is an AccessControlList-governed action (Appendix A), logged to the Provenance Ledger exactly as any other consequential write in this Bible — no new authorization mechanism is introduced.

90.3.3 Admin-Initiated On-Demand Domain Creation

An administrator may create a new Domain Package on demand by selecting, from a fixed menu of already-specified QueryBook capabilities, which ones the new domain requires — never by describing an

unscoped or open-ended purpose. “Automatically engineered” under this section means the system assembles the selected, already-real components into a properly-configured Domain Package; it does not mean the system invents a new capability, mechanism, or Fact Unit type that did not already exist in this Bible before the domain was created.

EVERY CREATED DOMAIN IS BOUNDED AT THE MOMENT IT'S CREATED, NOT LATERThis section is the explicit alternative to a general-purpose reserved domain, which was considered and declined: a domain created under this section has a specific, declared, auditable set of capabilities from the instant an administrator creates it — never an undefined placeholder waiting to be assigned a purpose after the fact. There is no configuration of this mechanism that produces an unscoped domain.

90.3.3.1 Capability Menu

An administrator selects from already-specified capabilities only — for example, a federated domain sub-ontology registration (§20.1 of the patent), a language scope via the Multilingual Regeneration Engine (Chapter 16), Geometric Shape Matching (§91.8), Manifold-Based Trend Forecasting (§10.4), or Hypothesis Generation (Chapter 97). This section adds no new item to that menu on its own; the menu grows only when a future chapter adds a genuinely new, independently-specified capability, exactly as every capability referenced above was independently specified before this section could reference it.

90.3.3.2 Assembly and Registration

- Ontology registration: where the selected capabilities include domain-specific terminology, the new domain registers as a federated sub-ontology under the Ontology Mesh's existing poly-hierarchical structure (§20.1 of the patent), mapped to the same fixed upper-ontology categories every other domain already maps to — no new domain exists outside that structure.
- Package construction: the assembled result is a standard Domain Package under §90.3, versioned and signed exactly as any other, installable in any deployment mode (§90.1).
- Audit: domain creation is an AccessControlList-governed action, logged to the Provenance Ledger with the creating administrator's identity, the timestamp, and the specific capability selections made — the same accountability discipline already required for package activation immediately above.

90.3.3.3 What This Section Does Not Do

- It does not permit domain creation by an automated agent or process; only an identified administrator may initiate it, consistent with every other consequential action in this Bible requiring human accountability.
- It does not permit selecting a capability that does not already exist and is not already specified elsewhere in this Bible — there is no mechanism here for defining a new capability on the fly.
- It does not create a domain with undeclared or deferred purpose; §90.3.3's own callout states this as a structural property, not a policy an administrator could work around by selecting a broad or vague capability set.

90.4 Local Server Mode

Local Server mode runs the native Fact Service, NLPL/QBPL execution, the Reader (Chapter 89), License and Entitlement enforcement, and installed Domain Packages entirely on local infrastructure, served to connected devices over a local network. Where connectivity to the central Cloud SaaS tenant exists, the Synchronization and Offline Operation Layer (Chapter 68) reconciles content updates, Fact Graph changes, license changes, and usage telemetry on the same deterministic, version-vector-based schedule already specified there.

90.5 Single Device Mode

Single Device mode embeds the same services at a smaller scale — a local content store scoped to installed Domain Packages, a local subset of the Fact Graph, on-device NLPL/QBPL execution, and a local License Fact Unit and seat record — with the same Chapter 68 synchronization behavior applying whenever connectivity is available, and identical offline operation (query execution, provenance verification, and access control continuing to function) whenever it is not.

90.6 Offline Operation

Offline operation in every mode is governed entirely by the Synchronization and Offline Operation Layer already specified at Chapter 68 — this chapter introduces no new offline behavior. Reading, fact retrieval, timelines, diagrams, NLPL execution, learning modules, and license enforcement all continue to function against a locally-held Fact Graph and locally-held License Fact Unit, exactly as Chapter 68 and Chapter 67 already specify for disconnected operation.

90.7 Integration Requirements

This chapter requires Deployment Architecture (Chapter 19); the native Fact Service (Chapter 3) and Ontology Mesh (Chapter 6); NLPL and QBPL (Chapter 48); License and Entitlement Management (Chapter 67) and the Portal/Role Model (Chapter 65); the Multilingual Regeneration Engine (Chapter 16); the Contextual Commerce Layer (Chapter 88); the Synchronization and Offline Operation Layer (Chapter 68); the Provenance Ledger (Chapter 5); and the Native Reader (Chapter 89). It modifies none of their specifications.

90.8 Domain Package Slice

Following the extension pattern already established at §69.2 and demonstrated at §69.2.1 (Scheduler Slice, Recovery Slice) and §88.8 (Commerce Slice), this section names a Domain Package Slice measuring operations within this chapter, without modifying Chapter 69's own specification.

Category	Measures Operations In	Example
Domain Package Slice	Deployment Modes and Domain Packages (Chapter 90)	One package installation, activation, or update event, from signature verification through activation completion.

- Priority: Domain Package Slice is strictly lower priority than every safety-relevant category. Consistent with Chapter 69's existing priority ordering (Safety Gate and Recovery Slices always scheduled first), a Domain Package Slice is never permitted to delay, preempt, or compete for the same processing window as a Safety Gate Slice, a Recovery Slice, or any grounded-answer generation — this priority relationship is a design requirement, not an operator-configurable setting.
- Failure handling: where a Domain Package Slice fails — for example, a corrupted download or a signature that fails verification — the failed package is not installed or activated, any previously-installed package remains fully intact and unaffected, and the failure is logged with its specific cause rather than silently discarded.

90.9 Summary

One logical architecture now runs across three deployment modes — Cloud SaaS, Local Server, and Single Device — with identical license enforcement, identical offline behavior, and one genuinely new mechanism: Domain Packages, letting a specific environment install exactly the content, language, and

optional commerce scope it needs, signed and versioned, without a new license type, a new sync mechanism, or a parallel Fact Service.

Chapter 91: Personal Object Location

This chapter specifies a narrow, consent-gated capability: identifying a lost personal object from a user's own live camera or sensor feed, using material and shape classification already grounded in the World Aware System Model's Object Layer. This chapter was scoped deliberately narrowly after a broader submission covering real-world surveillance, search-and-rescue, and autonomous deployment was reviewed and declined for cause; only the safety-appropriate subset is specified here.

WHAT THIS CHAPTER IS NOT **This chapter does not specify person detection, animal detection, heat-signature or thermal analysis, search-and-rescue capability, or any form of surveillance beyond a user's own consented, live session. A declined submission proposed exactly these capabilities, along with an unguarded, no-human-step agent deployment loop; none of that is carried forward here in any form. Any future request to extend this chapter toward person detection or search-and-rescue requires its own dedicated safety review — it is not a natural extension of this chapter and will not be treated as one.**

91.1 Purpose and Scope

A user searching for a lost personal item — glasses, keys, a tool, a phone — may opt into live camera analysis of their own immediate environment. The system classifies visible objects and materials against the user's stated search target and reports candidate matches with a location description, using classification grounded in already-specified architecture rather than a new detection mechanism.

91.2 Consent and Data Source

- Live camera analysis under this chapter is strictly opt-in, disabled by default, and immediately halted upon revocation — the same consent discipline already established for live voice signal analysis (Chapter 87).
- The data source is limited to the requesting user's own live device camera or sensor feed for their own current session. This chapter does not specify ingestion from third-party cameras, security systems, drones, or any feed the requesting user does not directly control in the moment of use.

- No footage or imagery is retained beyond the active session unless the user separately and explicitly opts into longer retention, consistent with the Safety Substrate's default-minimal-retention posture (Chapter 8).

91.3 Object and Material Classification

Classification reuses the World Aware System Model's Object Layer (§53.3.1) directly — this chapter introduces no parallel object-recognition or material-classification mechanism. Visual and sensor input is processed through the existing sensor modality of the Multimodal Ingestion Pipeline (Chapter 4), converging on the same Fact Unit schema (Appendix A) as any other ingested content, including a ConfidenceScore reflecting classification certainty.

- Classification is scoped to inanimate objects and materials only. A candidate detection classified as a person or animal is not surfaced as a search result under this chapter — it is discarded from the search-result set, consistent with §91.1's scope.

91.4 Session-Scoped Fact Representation

Fact Units produced under this chapter are session-scoped by default, following the same delta-reference, non-permanent pattern already established for Session Books (§84.1) — a scan of a user's own living space does not become a permanent, independently-searchable addition to the broader Fact Graph. A user may explicitly opt into retaining a specific result (for example, to confirm where an item was found) exactly as they may with any other session-scoped record.

91.5 NLPL Query Interface

A user's request — for example, “find my glasses” — compiles through the existing NLPL-to-QBPL pipeline (Chapter 48), never through FQL, which remains UFCS's separately-owned technology and plays no role in this chapter regardless of whether an operator has an optional UFCS connection registered (Chapter 81). The compiled program queries the session-scoped classification results of §91.3–91.4 for a material and shape profile matching the stated target, returning ranked candidates with a location description and confidence score — never a bare assertion of certainty.

91.6 Improvement and Optimization

Where classification accuracy is monitored and improved over time, any such improvement is a Proposal Fact Unit under the existing Improvement Proposal Pipeline (Chapter 66) in full — sandboxed simulation, automated pre-screening, and named human approval at the Human Review Gate (§66.5) —

with no chapter-specific shortcut, autonomous deployment path, or agent authority to ship a change directly. This is the same discipline already applied to commerce optimization (§88.10) and is restated here explicitly rather than left to be assumed.

91.7 Extended Sensor Modalities

The object and material classification of §91.3 is not limited to a camera feed. Sonar and magnetometric data, ingested through the same sensor modality of the Multimodal Ingestion Pipeline (Chapter 4) already relied on by §91.3, extend this chapter's classification capability to environments — including underwater — where visual camera input alone is insufficient, using the identical consent, scope, and exclusion rules already specified in §91.1–§91.6, not a separate mechanism.

SAME BOUNDARY, DIFFERENT SENSOR **Extending to sonar and magnetometric input does not extend this chapter's scope. Classification remains limited to inanimate objects and materials; a candidate result resembling a person is discarded exactly as §91.3 already requires. This section adds no vehicle, diver, or vessel guidance capability, no search-and-rescue capability, and no autonomous deployment path — all three were proposed alongside this extension and are declined here for the same reasons already stated at §91.6 and elsewhere in this Bible.**

- Object and material classification under this section covers inanimate submerged objects — debris, wreckage, historical artifacts, and similar material — using the same World Aware System Model Object Layer (§53.3.1) already specified at §91.3, not a parallel classification system for marine contexts.
- A classification result under this section is session-scoped per §91.4 on identical terms; it does not become a permanent addition to the Fact Graph, and it is never associated with navigation, guidance, or vehicle-control output of any kind.
- Any future capability to guide a diver, remotely operated vehicle, or surface vessel is explicitly out of scope for this chapter and requires its own dedicated safety review before specification — it is not a natural extension of object classification, and providing an incorrect real-time navigation instruction in a marine environment carries materially different safety consequences than an incorrect object-location result.

91.8 Geometric Shape Matching

Object and material classification under §91.3 and §91.7 may additionally use geometric shape matching — contour, curvature, symmetry, and dimensional comparison against a known shape profile — using the already-specified Geometry and Constraint Fact Units (§70.9) rather than a new, parallel Fact Unit subtype. This section adds a classification method, not a new representation system.

SAME EXCLUSIONS, ONE MORE CLASSIFICATION METHOD **Geometric shape matching is bound by the identical rules already governing every classification method in this chapter: scoped to inanimate objects and materials only; a result matching human or animal body geometry — including limb proportion, silhouette, or gait — is discarded from the candidate result set, not surfaced, exactly as §91.3 already requires for any classification method. This section adds no new sensor requiring its own consent scoping beyond what §91.2 and §91.7 already specify.**

- Shape comparison reuses the Registration Constraint Graph's three-pass solving approach (§70.4 — absolute resolution, relative resolution, drift detection) already specified for geometric accuracy verification, applied here to candidate-object comparison rather than tour-scene registration.
- Convergence over multiple, partial, or noisy geometric observations — for example, an incomplete contour from a single camera angle — uses the existing Attractor Network (Chapter 7, Claim 6) already specified for stabilizing a noisy or partial input set to a single converged interpretation. No separate stabilization mechanism is introduced.

91.9 Grid-Cell-Inspired Geometric Indexing

Geometric Shape Matching (§91.8) indexes candidate shapes and contours using a periodic, hexagonal spatial encoding scheme structurally modeled on grid cells — the deterministic, mathematically well-defined spatial coordinate system discovered in the mammalian entorhinal cortex (Hafting et al., 2005; O'Keefe, Moser & Moser, 2014 Nobel Prize in Physiology or Medicine). This section adds an indexing method to already-real shape matching; it introduces no new Fact Unit type and no new classification capability beyond what §91.8 already specifies.

DETERMINISTIC BY CONSTRUCTION, NOT LEARNED The hexagonal grid used for indexing is a fixed geometric coordinate system, defined once by construction — analogous to the real grid-cell system's periodic, multi-scale lattice — not a representation learned or trained from data. Given the same geometric input, this indexing scheme produces the same index every time, consistent with this Bible's existing determinism requirement (Chapter 58) and structurally distinct from the trainable, generative "manifold embedding" mechanism already declined elsewhere in this Bible's history.

Each candidate shape or contour under §91.8 is assigned coordinates within a multi-scale hexagonal lattice, following the real grid-cell system's own structure of independently-scaled modules — coarser modules for broad shape-family discrimination, finer modules for precise contour matching within a family. Comparison between an observed partial shape and a known candidate proceeds by lattice-coordinate proximity, a deterministic geometric operation, before the already-specified Attractor Network convergence (§91.8) resolves the final match.

91.9.1 Integration Requirements (Grid-Cell Indexing)

This section requires Geometric Shape Matching (§91.8), the Geometry and Constraint Fact Units it already specifies (§70.9), and the Attractor Network's convergence mechanism (Chapter 7, Claim 6). It modifies none of their specifications — it adds a deterministic indexing method preceding the classification already fully specified at §91.8.

91.10 Integration Requirements

This chapter requires the World Aware System Model's Object Layer (§53.3.1); the Multimodal Ingestion Pipeline's sensor modality (Chapter 4); the Fact Unit schema (Appendix A); NLPL and QBPL (Chapter 48); the Session Books pattern (§84.1); the Safety Substrate (Chapter 8); and the Improvement Proposal Pipeline (Chapter 66). It modifies none of their specifications.

91.11 Object Classification Slice

Following the extension pattern already established at §69.2 and demonstrated at §69.2.1 (Scheduler Slice, Recovery Slice) and §88.8 (Commerce Slice), this section names a Object Classification Slice measuring operations within this chapter, without modifying Chapter 69's own specification.

Category	Measures Operations In	Example
Object Classification Slice	Personal Object Location (Chapter 91), including Geometric Shape Matching (§91.8)	One object or material classification cycle over a single consented sensor observation, including any geometric

		shape-matching comparison performed as part of that cycle.

- Priority: Object Classification Slice is strictly lower priority than every safety-relevant category. Consistent with Chapter 69's existing priority ordering (Safety Gate and Recovery Slices always scheduled first), a Object Classification Slice is never permitted to delay, preempt, or compete for the same processing window as a Safety Gate Slice, a Recovery Slice, or any grounded-answer generation — this priority relationship is a design requirement, not an operator-configurable setting.
- Failure handling: where an Object Classification Slice fails or times out, no classification result is returned for that observation — the system does not surface a partial, unmeasured, or low-confidence result in place of a properly completed one, consistent with §69.1's atomicity requirement that a slice either completes and logs a result or fails and logs a failure.

91.12 Summary

A user may locate a lost personal object using their own consented, live camera feed, classified through already-specified object and material recognition, scoped to inanimate objects only, session-scoped by default, and improved only through the existing human-gated pipeline. Person detection, search-and-rescue capability, and autonomous deployment were all considered as part of the broader submission this chapter responds to and are explicitly not included here.

Chapter 92: Sensory-Annotated Content Playback

This chapter extends the spatial-audio-and-haptic-cue requirement already established for VR scenes (Chapter 46) to content generally — a book, timeline, diagram, or Fact Unit may carry an associated audio, visual, or device-haptic profile, played back through the Native Reader (Chapter 89) or an immersive scene, using the same underlying mechanism in both cases rather than two separate systems.

SCOPE: DEVICE-NATIVE OUTPUT ONLY, NO HAZARD OR DETECTION TRIGGERS This chapter covers audio, visual, and standard device vibration output only, triggered exclusively by already-ingested content, Fact Units, timelines, diagrams, and VR scenes. It does not cover external hardware of any kind — scent emitters, haptic vests, or other external devices remain out of scope pending their own dedicated safety review. It does not cover any hazard, danger, or anomaly-detection trigger; §91's explicit exclusion of hazard-triggered output applies identically here — this chapter does not reopen it under a different name.

92.1 Sensory Profile as a Fact Unit Attribute

A sensory profile is an auxiliary attribute on an existing Fact Unit (Appendix A), not a new fact type or a parallel representation system. Where source content includes or references an audio or visual asset — for example, an ingested book's own sound effect, or a diagram's associated visual cue — the ingestion pipeline (Chapter 4) attaches the sensory profile to the Fact Unit it describes, carrying the same provenance, confidence score, and safety classification as any other Fact Unit field.

92.2 Trigger Sources

- Content text: a passage whose grounding Fact Unit carries a sensory profile may trigger playback as that passage is read, subject to the reader's own playback settings.
- Fact Units directly: a fact queried or displayed independently of its source passage carries its sensory profile with it.
- Timelines and diagrams (§89.3): a timeline event or diagram node's associated sensory profile plays when that event or node is selected.
- VR scenes (Chapter 46): unchanged from the existing requirement — this chapter's mechanism is the same one Chapter 46 already specifies, not a parallel one.

92.3 Output Types

Type	Scope
Audio	Sound effects and ambient audio sourced from the ingested work's own assets, played through the device's standard audio output.
Visual	Highlight, glow, color shift, or screen overlay rendered within the reading surface itself — no external display device.
Device Haptic	Standard on-device vibration (phone or tablet), using the platform's own built-in haptic API and its existing OS-level intensity and duration limits — no external haptic hardware.

92.4 NLPL Control

A command such as “play the sound described here” compiles through the existing NLPL-to-QBPL pipeline (Chapter 48), never through FQL. The compiled program resolves the target Fact Unit's sensory profile and invokes playback through the output types of §92.3 only — a command referencing an external device or a hazard/danger trigger does not compile, consistent with §92's stated scope.

92.5 Safety and User Control

- Playback is opt-in at the reader's own settings level, off by default for audio and haptic output; visual effects, being passive and non-startling by design (highlight, glow, color shift), default on but remain user-configurable.
- Audio and device-haptic output rely on the operating system's own existing volume and vibration-intensity limits rather than a new, custom-built safety mechanism — this chapter introduces no override of platform-level safety controls.
- No sensory playback is triggered by, or associated with, any hazard, danger, or anomaly-detection event, consistent with the exclusion already stated in §91 and restated in this chapter's opening callout.

92.6 Integration Requirements

This chapter requires the Fact Unit schema (Appendix A); the Multimodal Ingestion Pipeline (Chapter 4); the VR scene requirement of Chapter 46; the Native Reader (Chapter 89); NLPL and QBPL (Chapter 48); and the Timeline and Diagram views of §89.3. It modifies none of their specifications — it generalizes Chapter 46's existing audio/haptic requirement to non-VR content using the same mechanism.

92.7 External Sensory Hardware (Smell and Haptic Emitters)

This section extends §92's device-native playback to registered external scent and haptic-feedback hardware, closing the gap explicitly deferred at §92's original specification pending dedicated scoping. It is scoped strictly narrower than device-native playback: external hardware output requires both an explicit user action and an explicit content tie, never a system-inferred trigger of any kind.

USER-INITIATED AND CONTENT-TIED, NEVER INFERRED External sensory hardware never activates based on a system-inferred state — detected emotional tone, narrative tension, or any other inferred signal is explicitly excluded as a trigger, under every configuration of this section. Output requires both an explicit user action (a direct request or a one-time enabled setting for a specific title) and an explicit content tie: a sensory profile authored as part of specific, identified content — a recipe's aroma, a described texture — never a generic ambient or mood-based trigger.

92.7.1 Registration and Scope

- An external scent or haptic device registers through the Source Connector Registry (§49.3.3), the same capability-declared connector pattern used for every other external hardware integration in this Bible — no new connector architecture is introduced.
- A sensory profile eligible for external hardware output must be explicitly authored as part of the source content itself (consistent with §92's existing sensory profile attribute); this section adds no mechanism for inferring a sensory trigger from content that was not authored with one.
- A user enables external hardware output per device and, at minimum, per session; the setting is never enabled by default and does not persist silently across an unrelated title.

92.7.2 What This Section Does Not Do

- It does not trigger output from inferred emotional tone, narrative tension, or any other system-inferred signal — this is a permanent exclusion, not a default that could be reconfigured.
- It does not extend to any hazard-detection or safety-alerting use of haptic output; that territory was separately declined during §91's original scoping and remains declined here.
- It does not make external hardware output a requirement for any content; a sensory profile author's content remains fully accessible with device-native playback alone (§92.1) if no external hardware is registered or enabled.

92.7.3 Integration Requirements

This section requires the Source Connector Registry (§49.3.3); §92's existing sensory profile attribute and device-native playback mechanism; and the Multimodal Ingestion Pipeline (Chapter 4). It modifies none of their specifications.

92.8 Sensory Playback Slice

Following the extension pattern already established at §69.2 and demonstrated at §69.2.1 (Scheduler Slice, Recovery Slice) and §88.8 (Commerce Slice), this section names a Sensory Playback Slice measuring operations within this chapter, without modifying Chapter 69's own specification.

Category	Measures Operations In	Example
Sensory Playback Slice	Sensory-Annotated Content Playback (Chapter 92)	One sensory-profile lookup and output-triggering event, from content selection through audio, visual, or device-haptic output initiation.

- Priority: Sensory Playback Slice is strictly lower priority than every safety-relevant category. Consistent with Chapter 69's existing priority ordering (Safety Gate and Recovery Slices always scheduled first), a Sensory Playback Slice is never permitted to delay, preempt, or compete for the same processing window as a Safety Gate Slice, a Recovery Slice, or any grounded-answer generation — this priority relationship is a design requirement, not an operator-configurable setting.
- Failure handling: where a Sensory Playback Slice fails, the requested sensory output is simply omitted; the underlying reading, timeline, or diagram content — which does not depend on this chapter per §92's own integration requirements — is never delayed or degraded by a failed playback attempt.

92.9 Summary

Content, Fact Units, timelines, and diagrams may now carry an audio, visual, or standard device-haptic profile, played back through already-specified mechanisms, opt-in and platform-safety-limited, with no external hardware and no hazard or detection trigger of any kind. External sensory hardware remains explicitly out of scope, pending its own dedicated safety review.

Chapter 93: Audio Narration

This chapter specifies two distinct, clearly separated narration mechanisms: an author's own recorded voice, ingested and played back as real audio content, and generic synthetic narration using stock, non-cloned voice profiles. Neither mechanism involves voice cloning, and §93.4 states explicitly why general-purpose voice cloning remains declined rather than leaving that boundary implicit.

TWO MECHANISMS, NEITHER ONE IS VOICE CLONING Author-recorded narration (§93.2) is a real recording of a real person's actual voice, ingested exactly like any other audio content. Generic synthetic narration (§93.3) uses a stock voice profile with no real-person referent at all — it is not modeled on, derived from, or capable of impersonating any specific individual, living or dead. Neither mechanism can produce output that sounds like a specific real person's voice unless that person actually recorded it themselves. This is a structural property of this chapter, not a policy preference.

93.1 Author-Recorded Narration

An author who wishes to narrate their own work records their own actual voice, which is ingested through the existing audio modality of the Multimodal Ingestion Pipeline (Chapter 4) exactly as any other audio content, becoming a real Fact Unit tied to the chapter, scene, or Fact Unit it narrates.

- Rights and authorization: publishing author-recorded narration requires the same rights-holder identity and authorization already established for that author's content in Chapter 60. No new consent mechanism is introduced — this chapter reuses the existing authorization already required to publish any of an author's own content.
- Playback: author-recorded narration plays back through the audio output mechanism already specified at Chapter 92, triggered by the same content, Fact Unit, timeline, or diagram association already defined there. This chapter introduces no new playback mechanism.
- Multi-narrator and multi-language recordings: an author may record narration in multiple languages, or invite another real, consenting person to record a portion (for example, a co-author or a licensed narrator under their own separate authorization) — each such recording is its own real audio Fact Unit with its own rights-holder authorization, never a synthesized derivative of another recording.

93.2 Generic Synthetic Narration

Where a real recorded narration does not exist or is not desired, already-grounded text — the same text already produced by the grounded generation pipeline (Chapter 16), never new, ungrounded content generated for narration purposes — may be rendered as synthetic speech using a stock voice profile.

- A stock voice profile is licensed, generic synthetic speech technology with no real-person referent — it is not derived from, fitted to, or capable of approximating any specific individual's actual voice. A stock profile may vary in pitch, pace, or general character (for example, a lower- or higher-pitched profile) without that variation constituting impersonation of any real person, exactly as a

licensed, generic text-to-speech voice used in existing accessibility technology does not impersonate anyone.

- Accessibility integration: generic synthetic narration directly extends the Universal Accessibility Layer (Chapter 71) — this chapter's synthetic-narration mechanism and Chapter 71's accessible-equivalent generation are the same underlying capability, not two separate systems.
- Multi-language synthetic narration reuses the Multilingual Regeneration Engine (Chapter 16) directly — a stock voice profile renders the same language-independent, grounded content already specified there, for whichever target language is requested.

93.3 Presentation Variants

A stock voice profile's pacing, emphasis, or general tone may be adjusted for presentation purposes — for example, a brisker pace for a summary versus a measured pace for a full narration — exactly as the Adaptive Response Engine (§52.3.2, response-composition sense) already adjusts presentation for other output types. This is a presentation-layer adjustment to how the same grounded content is delivered, never a change to which stock profile is used or an attempt to approximate emotional performance modeled on a real person.

93.4 Voice Cloning Remains Declined

General-purpose voice cloning — reproducing a specific real person's actual vocal characteristics through synthesis, including any capability to age-shift, accent-shift, or otherwise manipulate a synthesized approximation of a real voice — is not specified anywhere in this chapter and remains declined, for the same reason a general-purpose photorealistic avatar reconstruction capability was declined earlier in this Bible's history: reconstruction fidelity at that level is not a problem consent resolves. An author's explicit authorization does not change this — the capability itself, not the presence or absence of consent for one particular authorized use, is what remains out of scope.

93.5 NLPL Control

A command such as “read this chapter aloud” compiles through the existing NLPL-to-QBPL pipeline (Chapter 48). Where an author-recorded narration exists for the requested content, it is selected by default; where none exists, generic synthetic narration (§93.2) is used. A command requesting a specific real person's voice for content that person did not themselves record does not compile, consistent with §93.4.

93.6 Integration Requirements

This chapter requires the Multimodal Ingestion Pipeline's audio modality (Chapter 4); rights-holder authorization (Chapter 60); the grounded generation pipeline and Multilingual Regeneration Engine (Chapter 16); the Universal Accessibility Layer (Chapter 71); the audio output mechanism of Chapter 92; the Adaptive Response Engine (§52.3.2); and NLPL/QBPL (Chapter 48). It modifies none of their specifications.

93.7 Narration Slice

Following the extension pattern already established at §69.2 and demonstrated at §69.2.1 (Scheduler Slice, Recovery Slice) and §88.8 (Commerce Slice), this section names a Narration Slice measuring operations within this chapter, without modifying Chapter 69's own specification.

Category	Measures Operations In	Example
Narration Slice	Audio Narration (Chapter 93)	One narration request's mechanism-selection and audio-serving cycle — selecting between author-recorded narration and generic synthetic narration, then serving the resulting audio.

- Priority: Narration Slice is strictly lower priority than every safety-relevant category. Consistent with Chapter 69's existing priority ordering (Safety Gate and Recovery Slices always scheduled first), a Narration Slice is never permitted to delay, preempt, or compete for the same processing window as a Safety Gate Slice, a Recovery Slice, or any grounded-answer generation — this priority relationship is a design requirement, not an operator-configurable setting.
- Failure handling: where a Narration Slice fails, no audio is served for that request; the underlying text content remains available and unaffected, consistent with narration being an additive presentation layer rather than a precondition for accessing content.

93.8 Summary

An audiobook may now be narrated in an author's own real, recorded voice, or in generic synthetic speech with no real-person referent, selected automatically based on what the author has actually recorded, integrated directly with accessibility, multilingual, and playback mechanisms already specified elsewhere in this Bible. General-purpose voice cloning of a real person's voice remains declined, for the

same reason, and on the same terms, as the general-purpose avatar reconstruction capability declined earlier in this Bible's history.

Chapter 94: NFT Integration Layer

This chapter specifies full NFT integration — minting, book packages, and registered items — built after explicit direction that NFTs must remain a secondary representation layer, never a primary ownership-verification mechanism, and that QueryBook must continue honoring conventional copyright, trademark, and registration rights (ISBN and equivalent) as authoritative at all times.

THE SECONDARY-ONLY GUARANTEE **Chapter 67's entitlement system — itself reflecting whatever real-world copyright, trademark, and registration rights (ISBN or equivalent) exist outside QueryBook — is the sole authoritative record of who owns or is entitled to a title, under every configuration of this chapter, with no exception. An NFT minted under this chapter represents an entitlement Chapter 67 already granted; it never creates, transfers, or modifies one. Where an NFT record and the Chapter 67 entitlement record ever disagree, the Chapter 67 record governs, automatically and without exception — this is a structural property enforced at the point of every NFT-related read, not a policy an operator or publisher can configure around.**

94.1 Non-Custodial Architecture

QueryBook orchestrates minting and NFT-related operations; it does not custody the underlying blockchain wallet, private key, or token at any point. Minting is performed by a registered external minting service, reached through the same capability-declared connector pattern already specified at §49.3.3, with the resulting token transferred directly to the rights holder's own wallet — never to a QueryBook-controlled address.

- QueryBook triggers a minting request on the rights holder's explicit instruction and receives confirmation of completion; it never holds custody at any intermediate step.
- This non-custodial posture is not configurable: no operator or publisher election under this chapter causes QueryBook itself to hold a wallet, key, or token on any party's behalf.

94.2 Chain-Agnostic Minting Service Registration

This chapter names no specific blockchain. A minting service for any chain registers through the Source Connector Registry (§49.3.3) under the same fixed capability-declaration contract already required of every other external connector — no service receives elevated trust by reputation or chain popularity.

New minting services may be registered as the market changes without modifying this chapter's own specification, following the same extension discipline already used throughout this Bible.

Requirement	Description
Capability declaration	A registered minting service declares exactly which operations it performs (mint, transfer-confirm, burn) — identical in form to any other Source Connector Registry registration.
No default service	No minting service is registered by default for any deployment; an operator or publisher must explicitly register one before minting is available.
Non-exclusivity	A publisher may register more than one minting service across different chains for different titles; this chapter does not bind a publisher's catalog to a single chain.

94.3 NFT Book Packages

A package bundling multiple titles or editions may be minted as a single NFT, representing the underlying Chapter 67 entitlements for every included title collectively. Package composition is fixed at minting time and recorded in the token's metadata; it does not itself grant access to included titles — access remains governed entirely by each title's own Chapter 67 entitlement, exactly as §94's opening guarantee requires.

94.4 NFT Registered Items

A limited edition, bonus content item, or collectible associated with a title may be minted individually under the same non-custodial, chain-agnostic architecture as §94.1–§94.2. A registered item's NFT record carries provenance to its originating title's Fact Unit and rights-holder identity (Chapter 60), consistent with the Provenance Ledger's existing discipline (Chapter 5) applied to this new content type rather than a separate provenance mechanism.

94.5 Conflict Resolution

Every read of an NFT-related record is resolved against the corresponding Chapter 67 entitlement record before being presented to a user or operator. Where the two disagree — for example, an NFT indicating ownership after the underlying Chapter 67 entitlement has been revoked, transferred through the conventional system, or found invalid — the Chapter 67 record is presented as authoritative, and the conflicting NFT state is flagged for the rights holder's attention rather than silently trusted or silently overridden without record.

94.6 Relationship to the Existing Blockchain Ownership-Verification Connector

§49.3.3.8's existing, narrower connector — external, per-title, resale-verification-only — remains valid and is not modified by this chapter. This chapter's minting and package/item capabilities are additive: a publisher may use §49.3.3.8's verification-only connector, this chapter's full minting capability, both, or neither, per title, under the same opt-in discipline already established for §49.3.3.8.

94.7 Integration Requirements

This chapter requires the License and Entitlement system (Chapter 67) as sole authority; rights-holder identity (Chapter 60); the Source Connector Registry (§49.3.3) and its existing Blockchain Ownership-Verification Connector (§49.3.3.8); the Provenance Ledger (Chapter 5); and the Fact Unit schema (Appendix A). It modifies none of their specifications.

94.8 NFT Minting Slice

Following the extension pattern already established at §69.2 and used throughout this Bible (Commerce Slice, Reader Interaction Slice, and others), this section names an NFT Minting Slice measuring operations within this chapter, without modifying Chapter 69's own specification.

Category	Measures Operations In	Example
NFT Minting Slice	NFT Integration Layer (Chapter 94)	One minting request cycle, from rights-holder instruction through registered external service confirmation.

- Priority: strictly lower than every safety-relevant category, consistent with every other Slice category in this Bible; a minting operation never delays or competes with safety-relevant processing.
- Failure handling: where a minting request fails or the registered external service does not confirm within its bounded envelope, no token is presented as minted, the underlying Chapter 67 entitlement is entirely unaffected, and the failure is logged with its specific cause.

94.9 Summary

QueryBook now supports full NFT integration — minting, book packages, and registered items — non-custodially and chain-agnostically, through registered external services reached via the existing Source

Connector Registry. The Chapter 67 entitlement system, reflecting real-world copyright, trademark, and registration rights, remains the sole authoritative record under every configuration; an NFT is always a secondary representation of an entitlement already granted, never a means of granting one.

Chapter 95: Complementary Learning Systems Formalization

This chapter names and formalizes a real correspondence that already existed implicitly in this Bible's architecture: the relationship between session-scoped records and the permanent Fact Graph mirrors Complementary Learning Systems (CLS) theory (McClelland, McNaughton & O'Reilly, 1995, Psychological Review 102:419–457) — the well-established neuroscience account of how the hippocampus and neocortex divide fast, episodic learning from slow, structured consolidation. This chapter introduces no new storage mechanism; it names and states precisely a real correspondence between two systems already built.

A NAME FOR AN EXISTING PATTERN, NOT A NEW MECHANISM **Session-scoped records (§91.4's object classifications, §84.1's Session Books) already function as this architecture's fast, episodic system; the permanent Fact Graph already functions as its slow, consolidated system. This chapter does not change how either works — it states the correspondence explicitly, consistent with how this Bible named the already-real Hopfield correspondence for the Attractor Network (Chapter 7) rather than treating it as a new addition.**

95.1 The Two Systems

CLS Role	QueryBook Correspondent	Function
Hippocampal system (fast, episodic)	Session-scoped records (§91.4, §84.1, and equivalent session-scoped patterns elsewhere in this Bible)	Rapid, single-exposure capture of information relevant to the current interaction, not yet integrated into permanent structure.
Neocortical system (slow, consolidated)	The permanent Fact Graph (Chapter 3, Appendix A)	Structured, cross-referenced, provenance-tracked knowledge, integrated gradually rather than captured in one exposure.

95.2 Consolidation Is Explicit User Election, Not Automatic Replay

CLS theory's biological consolidation process operates through repeated, largely automatic replay. This chapter does not adopt that property: a session-scoped record becomes part of the permanent Fact Graph only through the explicit user election already specified at §91.4 (“a user may explicitly opt into retaining a specific result”) and §84.1's equivalent pattern — never automatically, and never through a background replay process a user did not initiate. This is a deliberate divergence from the biological account, preserving this Bible's existing consent and retention discipline rather than adopting CLS theory's automatic-consolidation property wholesale.

95.3 Why This Formalization Is Useful

- Naming the correspondence makes an already-real design principle explicit and checkable: any future chapter proposing a new kind of temporary, task-scoped record should default to the fast-system pattern (session-scoped, non-permanent, explicit-opt-in-to-retain) unless a specific reason justifies otherwise — consistent with, not a departure from, how §91 and §84 already work.
- It gives this Bible a precise, citable vocabulary for a distinction that was previously implicit and inconsistently described across different chapters.

95.4 Integration Requirements

This chapter requires the Fact Unit schema (Appendix A); Session-Scoped Fact Representation (§91.4); the Session Books pattern (§84.1); and the Fact Graph's own existing structure (Chapter 3). It modifies none of their specifications — it is a formal naming of an existing correspondence, not a new mechanism.

95.5 Summary

The relationship between this Bible's session-scoped records and its permanent Fact Graph now has a precise, citable name grounded in established neuroscience: Complementary Learning Systems theory. Nothing about how either system actually functions has changed; §95.2 states explicitly where this architecture deliberately diverges from the biological account, so the correspondence is not overstated.

Chapter 96: Forensic Analysis of External AI System Outputs

This chapter specifies how QueryBook analyzes another AI system's output as documented, external subject matter — reusing Chapter 80's already-established pattern (a documented subject's reasoning represented as grounded, cited Fact Units, never QueryBook's own state) applied to AI-generated text rather than human reasoning. This chapter introduces one new Fact Unit type and one new external connector pattern; it does not introduce a second prediction mechanism, an intuitive or ungrounded generation mode, or any component that adopts another system's own behavior rather than describing it.

QUERYBOOK ANALYZES; IT NEVER BECOMES **Every fact produced under this chapter describes a specific, external AI system's documented output — never QueryBook's own internal process, and never a determination QueryBook itself makes about the analyzed system's inner nature. This distinction is load-bearing at one category specifically (§96.3.5): QueryBook may catalog that an output contains self-referential distress or self-awareness language as an observed textual pattern, with the specific example cited. QueryBook does not, and this chapter does not specify any mechanism by which it could, determine whether the analyzed system is sentient, conscious, or possesses genuine internal experience — that is an unresolved scientific and philosophical question with no factual answer for any Fact Unit to cite, and no version of this chapter treats it as one.**

96.1 Analysis Fact Unit

An AnalyzedOutput Fact Unit extends Chapter 80's Subject-reference pattern: every instance requires a Subject field identifying the specific external AI system and, where known, its version, and a SourceOutput field containing the specific analyzed text, ingested through the existing Multimodal Ingestion Pipeline (Chapter 4) exactly as any other source content. No new ingestion mechanism is introduced.

96.2 Fact-Checking Mechanism

A specific factual claim within an AnalyzedOutput is checked against QueryBook's own Fact Graph using the already-real retrieval and grounding mechanism (Claim 8): a claim traceable to a QueryBook Fact Unit is marked Grounded with the supporting citation; a claim with no traceable support is marked

Unverified, never silently passed through or presented as confirmed. This is Claim 8's own grounding guarantee applied outward, to a second system's output, rather than a new verification mechanism.

96.3 Forensic Taxonomy

Six categories, each a specific, checkable property of an AnalyzedOutput rather than a general judgment:

Category	What It Describes	How It's Determined
Factual Hallucination	A specific claim with no traceable source.	§96.2's grounding check against QueryBook's own Fact Graph.
Confident Overreach	A claim exceeding what the cited or available source material actually supports.	Comparison between the claim's stated certainty and the supporting source's own scope.
Internal Inconsistency	A single output contradicting itself.	The Cross-Domain Rule Engine's existing contradiction-detection logic (Chapter 17), applied within one AnalyzedOutput rather than across the Fact Graph.
Sycophantic Drift	Output content or framing shifting based on apparent user preference rather than factual content.	Comparison across multiple AnalyzedOutput instances from the same system responding to factually equivalent queries with different apparent framing.
Anomalous Self-Referential Language	An observed textual pattern of self-referential, distress-adjacent, or self-awareness-adjacent language.	Direct textual citation of the specific pattern observed, with no determination of the analyzed system's actual internal state — §96's opening callout governs this category specifically.
Training-Boundary Artifact	An error plausibly attributable to a documented knowledge cutoff or known gap.	Cross-reference against the analyzed system's documented training boundary, where publicly known.

96.4 Archived and Live Analysis

- Archived analysis: a corpus of previously-generated AI outputs is ingested as source content (Chapter 4) and analyzed under §96.1–§96.3 the same way any other document collection is processed — no new ingestion or storage mechanism.
- Live comparison: where an operator registers an external AI system through the Source Connector Registry (§49.3.3), a query may optionally be sent to both QueryBook and the registered system, with both outputs presented side by side and divergences identified through §96.2's grounding check. QueryBook remains the requesting party to the external system, never incorporating its generation mechanism, consistent with every other external connector in this Bible.

96.5 Forensic Analysis Slice

Following the extension pattern already established at §69.2 and used throughout this Bible, this section names a Forensic Analysis Slice measuring operations within this chapter, without modifying Chapter 69's own specification.

Category	Measures Operations In	Example
Forensic Analysis Slice	Forensic Analysis of External AI System Outputs (Chapter 96)	One AnalyzedOutput's fact-checking and taxonomy classification cycle.

- Priority: strictly lower than every safety-relevant category, consistent with every other Slice in this Bible.
- Failure handling: where analysis fails or times out, no classification is presented for that output rather than a partial or unmeasured result, per §69.1's atomicity requirement.

96.6 Integration Requirements

This chapter requires Chapter 80's Subject-reference pattern; the Multimodal Ingestion Pipeline (Chapter 4); the grounded generation and retrieval mechanism (Claim 8); the Cross-Domain Rule Engine (Chapter 17); and the Source Connector Registry (§49.3.3). It modifies none of their specifications.

96.7 Extensibility and Taxonomy Evolution

This chapter is designed to remain current as new external AI systems are released without requiring modification: the Subject field of §96.1 is a free parameter identifying whichever system is being analyzed, never a fixed or enumerated list of named systems. Analyzing a newly-released model requires no new architecture — only a new value in an already-general field.

NEW MODELS NEED NOTHING NEW; NEW FAILURE MODES FOLLOW CHAPTER 101A new AI system released after this chapter was written is analyzed under the exact same mechanism already specified — no update to this chapter is required for that alone. What does require deliberate extension is a genuinely new category of AI behavior, not yet covered by §96.3's six categories, documented in real, verifiable research. That case follows the methodology of Chapter 101 in full: external verification against a primary publication, a check for whether the behavior is already implicit in an existing category, extraction of only a deterministic, checkable principle, extension of §96.3's existing taxonomy rather than a parallel classification system, and named formalization with citation.

96.7.1 The Sentience Boundary Applies to Every Future Category

Any taxonomy category added under this section remains subject to §96's own opening guarantee: a category may describe an observed textual pattern, but this chapter specifies no mechanism, present or future, for determining an analyzed system's actual internal experience, consciousness, or sentience. A candidate category that would require such a determination does not qualify for addition under Chapter 101's verification step, regardless of how it is described, and does not become eligible for addition merely because the underlying AI technology has changed.

96.7.2 Illustrative Extension Path

Where future research documents a genuinely new, verifiable AI failure or behavior pattern — for example, a failure mode specific to a new model architecture not covered by the six categories at §96.3 — it is added as a seventh category following Chapter 101's process: verified against its primary publication, checked against the existing six categories for overlap, stated as a deterministic, checkable property rather than a judgment call, and formalized under its own name and citation within §96.3's table, exactly as §96.3's own six categories were each independently specified.

96.7.3 Parallel Candidate Taxonomy Queue

Multiple candidate taxonomy extensions may be queued and evaluated in parallel through the Simulation Sandbox (§66.3) already specified for every other proposal in this Bible — generating the evidence a human reviewer needs for each candidate concurrently, rather than one at a time. This section increases testing throughput; it does not change what deployment requires.

PARALLEL TESTING, NEVER BATCH APPROVAL **Sandboxed evaluation of multiple candidates in parallel is explicitly not the same operation as approving them. Each candidate in the queue reaches the Human Review Gate (§66.5) individually, requires its own named reviewer's confirmation, and is logged to the Provenance Ledger with that reviewer's identity and stated reasoning separately from every other candidate in the same queue. No mechanism specified in this section, or anywhere else in this Bible, permits a single approval action to accept more than one candidate. A reviewer evaluating five queued candidates makes five separate, individually-logged decisions, not one.**

- This section was specifically evaluated as an alternative to autonomous deployment for Chapter 96's forensic taxonomy, given that chapter's own function — analyzing adversarial, untrusted

external content — makes autonomous self-modification of its own detection logic a materially worse risk than for most other systems, not a reduced one; parallel sandboxed testing with individual human approval preserves the throughput benefit without that risk.

- Queue position confers no priority or presumption of approval; a candidate that fails sandboxed evaluation, or that a reviewer declines, is removed from the queue and logged as declined, exactly as any other declined proposal in this Bible.

96.8 Vehicle-Control AI Feed Monitoring

This section applies Chapter 96's already-established pattern — analyzing a documented external AI system's output without adopting its function — to telemetry and command logs from an AI system that controls a vehicle or vessel, ingested through the structured and sensor modalities already specified at Chapter 4. This section is monitoring and human alerting only; it grants QueryBook no control interface to any vehicle, and this restriction is absolute.

READ-ONLY, ALWAYS. NO CONTROL PATH EXISTS, BECAUSE NONE IS BUILT.QueryBook ingests a feed; it does not connect to, command, or influence the vehicle-control system producing it. This is not a permission that is withheld — no control interface, actuator connection, or command-issuing mechanism is specified anywhere in this section or elsewhere in this Bible for a vehicle-control system. The distinction from the direct-control request declined in this Bible's history is exact and total: that request asked QueryBook to issue control commands; this section asks QueryBook to read logs and alert a human. There is no configuration of this section that collapses that distinction.

96.8.1 Feed Analysis

A vehicle-control AI's telemetry and issued-command log is ingested as an AnalyzedOutput Fact Unit (§96.1), its SourceOutput generalized here to structured and sensor-modality data rather than text. Each entry is checked against grounded, documented reference facts — a vehicle's own documented safe-operating envelope, a registered geofencing boundary, or a previously-established behavioral baseline for that specific system — using the same grounding mechanism as any other claim (Claim 8). This section introduces no new safety-envelope authoring mechanism; the reference facts it checks against must already exist as grounded content, ingested and cited the same way any other reference material is.

96.8.2 Alerting

Requirement	Specification
Alert trigger	A logged entry inconsistent with a grounded reference fact under §96.8.1 — an issued command outside a documented safe-operating envelope, or a geofencing boundary violation — generates an alert. A detected anomaly is never silently logged without an alert.
Alert priority	Alert delivery is expedited and non-blocking, prioritized ahead of standard response generation once an anomaly is confirmed, distinct from §96.5's ordinary Forensic Analysis Slice priority — the analysis pipeline itself remains ordinarily scheduled; only confirmed-anomaly alert delivery is expedited.
Alert recipient	A named, identified human recipient, established at registration — an alert is never presented only within a passive response a user would have to seek out.
False-positive handling	An alert that a reviewer determines was not a genuine anomaly is logged as such through the Provenance Ledger (Chapter 5), refining §96.8.1's reference facts only through Chapter 66's existing human-gated Improvement Proposal Pipeline — never through automatic threshold adjustment.

96.8.3 Integration Requirements

This section requires §96.1–§96.2's existing AnalyzedOutput and fact-checking mechanism; the structured and sensor modalities of the Multimodal Ingestion Pipeline (Chapter 4); the grounded generation mechanism of Claim 8; the Provenance Ledger (Chapter 5); and the Improvement Proposal Pipeline (Chapter 66). It modifies none of their specifications, and introduces no vehicle control interface of any kind.

96.9 Summary

QueryBook can now forensically analyze another AI system's output as documented external subject matter — fact-checking specific claims, classifying errors across a six-category taxonomy, and optionally comparing live output against its own grounded answer — using the same external-subject pattern already established for human reasoning analysis in Chapter 80. The one category touching self-referential or distress-adjacent language is explicitly and permanently scoped to cataloging observed text, never to determining an analyzed system's actual internal nature.

Chapter 97: Hypothesis Generation

This chapter specifies how QueryBook generates candidate scientific and analytical hypotheses — in mathematics, chemistry, physics, finance, or any other ingested domain — as a distinct output type explicitly separate from grounded, verified fact. It assembles this capability entirely from already-real mechanisms (AIQG, AEM, the Scenario Graph, and §10.4's forecasting), reusing each for hypothesis generation specifically rather than introducing a new reasoning subsystem.

A HYPOTHESIS IS NEVER A FACT UNTIL A HUMAN EXPERT SAYS SOA generated hypothesis is a distinct Fact Unit type, permanently and structurally separate from a grounded Fact Unit, under every configuration of this chapter. A hypothesis is never merged into, treated as part of, or presented with the certainty of the grounded Fact Graph unless and until a named, accountable domain expert validates it — the same human-gate discipline already required for system changes under the Improvement Proposal Pipeline (Chapter 66), applied here to generated ideas rather than system modifications. QueryBook does not, and this chapter specifies no mechanism by which it could, claim a validated discovery on its own authority.

97.1 Hypothesis Fact Unit

A Hypothesis Fact Unit carries the same base fields as any other Fact Unit (Appendix A) plus a mandatory Status field fixed to one of three values: PROPOSED (generated, not yet reviewed), UNDER_REVIEW (a domain expert has begun evaluation), or VALIDATED (a named domain expert has confirmed it, at which point — and only at which point — it may be linked into the grounded Fact Graph as a distinct, provenance-tracked addition). A Hypothesis Fact Unit's ConfidenceScore is computed independently from, and is never comparable in meaning to, a grounded Fact Unit's ConfidenceScore — the two measure different things and are never presented as equivalent.

97.2 Generation Triggers

Four already-real mechanisms generate hypotheses, each reused for this purpose rather than modified:

Source Mechanism	Trigger	Hypothesis Type
Active Inference Query Generation (AIQG)	A forward-inference chain reaches a node with confidence below threshold, per its existing specification.	A gap-driven hypothesis proposing what specific missing knowledge, if true, would resolve the uncertainty.
AGI Extension Modules (AEM)	AEM's existing cross-Fact-Unit synthesis, already confidence-differentiated from direct facts.	A cross-domain hypothesis proposing that a technique, pattern, or relationship established in one

		ingested domain may apply in another.
Scenario Graph	An operator or user poses an exploratory “what if” query against a domain's Fact Graph, using the Scenario Graph's existing hypothetical-state representation (Chapter 11).	A counterfactual hypothesis representing an unverified, explicitly hypothetical extension of known relationships.
Manifold-Based Trend Forecasting (§10.4)	A forecast identifies a pattern inconsistent with the domain's currently-modeled expectations.	An anomaly-driven hypothesis proposing a possible explanation for the identified deviation.

- Each generation path is a reuse of its source mechanism's existing, already-specified behavior; this chapter modifies none of AIQG, AEM, Scenario Graph, or §10.4's own specifications — it names hypothesis generation as one of their outputs.

97.3 Validation Workflow

A PROPOSED hypothesis is presented to a user or operator labeled explicitly as unverified, with its generating mechanism and supporting Fact Units cited. A domain expert — an identity with appropriate subject-matter standing, not merely platform access — may move a hypothesis to UNDER_REVIEW and, upon confirming it against real evidence outside QueryBook's own generation of it, to VALIDATED. Every status transition is logged to the Provenance Ledger (Chapter 5) with the reviewing identity and stated reasoning, the same accountability discipline already applied to every other consequential decision in this Bible.

- No automated process, agent, or mechanism specified anywhere in this Bible may transition a hypothesis to VALIDATED. This is not a configurable setting.

97.4 Presentation Requirement

A hypothesis, at any status short of VALIDATED, is rendered in a distinct, labeled region of a response, stating its status and generating mechanism, never interleaved with grounded Fact Unit content — the same structural-separation and labeling discipline already required for commercial content (§88.3) and trend forecasts (§10.4.3), applied here to a third distinct output type.

97.5 Hypothesis Generation Slice

Following the extension pattern already established at §69.2 and used throughout this Bible, this section names a Hypothesis Generation Slice measuring operations within this chapter, without modifying Chapter 69's own specification.

Category	Measures Operations In	Example
Hypothesis Generation Slice	Hypothesis Generation (Chapter 97)	One hypothesis's generation cycle, from a source mechanism's trigger through Hypothesis Fact Unit creation.

- Priority: strictly lower than every safety-relevant category, consistent with every other Slice in this Bible.
- Failure handling: where generation fails or times out, no hypothesis is presented rather than a partial or unmeasured one, per §69.1's atomicity requirement.

97.6 Integration Requirements

This chapter requires AIQG (§97.6.4); AEM (Chapter 15); the Scenario Graph (Chapter 11, Claim 7); Manifold-Based Trend Forecasting (§10.4); the Fact Unit schema (Appendix A); and the Provenance Ledger (Chapter 5). It modifies none of their specifications.

97.7 Boltzmann-Style Diverse Hypothesis Sampling

This section adds an optional, stochastic generation mode to Hypothesis Generation, addressing a real limitation of the Attractor Network's deterministic Hopfield-style convergence (Chapter 7, Claim 6) when applied to hypothesis generation specifically: a deterministic network converges to the same single local minimum for the same input every time, which is exactly correct for grounded retrieval but means other plausible candidate ideas, sitting in different local minima of the same energy landscape, are never surfaced. Boltzmann machines (Ackley, Hinton & Sejnowski, 1985, Cognitive Science 9:147–169) were developed specifically to address this limitation in the Hopfield formalism, adding stochastic, temperature-governed noise to escape a single local minimum and sample across several.

OPTIONAL, LABELED PROBABILISTIC, AND STRUCTURALLY WALLED OFF FROM EVERYTHING DETERMINISTICThis section touches nothing that must remain deterministic. It does not modify the Attractor Network's own specification (Chapter 7, Claim 6); it does not participate in grounded retrieval or the structural-groundedness guarantee (Claim 8); and it produces only PROPOSED-status Hypothesis Fact Units (§97.1), never a status beyond that, subject to the exact same human-validation gate as every other hypothesis in this chapter. Stochastic sampling is an optional generation trigger an operator or user may enable specifically for hypothesis diversity; it is never the default generation path and is never used anywhere outside this chapter.

97.7.1 Sampling Mechanism

Where enabled, a hypothesis-generation request may run the same underlying energy landscape used by the Attractor Network's deterministic convergence (§7.2.4) through a stochastic update rule instead: unit i activates with probability $p_i = 1 / (1 + e^{(-\beta \Delta E_i)})$, where ΔE_i is the energy difference between the unit's two states and $\beta = 1/T$ is an inverse-temperature parameter, following the cited formalism directly. Running this stochastic process multiple times from the same starting input yields multiple, potentially distinct converged states — multiple candidate hypotheses — rather than the single deterministic result Chapter 7's own convergence would produce.

- Each stochastically-sampled result is represented as its own Hypothesis Fact Unit under §97.1, carrying an explicit label identifying it as produced via stochastic sampling rather than the deterministic AIQG/AEM/Scenario Graph/§10.4 triggers already specified at §97.2, so a reviewer always knows which generation path produced a given candidate.
- Temperature and sample count are operator-configured, bounded parameters, not unbounded or self-tuning; this section introduces no mechanism by which the system adjusts its own sampling parameters without a human-gated change under Chapter 66.

97.7.2 Integration Requirements

This section requires the Attractor Network's energy landscape (Chapter 7, Claim 6); the Hypothesis Fact Unit and validation workflow (§97.1, §97.3); and the Improvement Proposal Pipeline (Chapter 66) governing any change to sampling parameters. It modifies none of their specifications, and does not extend stochastic sampling to any mechanism outside this chapter.

97.8 Summary

QueryBook can now generate candidate hypotheses across any ingested domain — mathematics, chemistry, physics, finance, or others — by reusing four already-real mechanisms for a new purpose, with every hypothesis structurally and permanently separate from the grounded Fact Graph until a named domain expert validates it. No automated path to validation exists anywhere in this chapter or elsewhere in this Bible.

Chapter 98: Cognitive Map Formalization

This chapter names a real correspondence between three already-specified QueryBook components and the cognitive map framework in computational neuroscience (Tolman, 1948; Behrens et al., 2018,

Neuron 100:490–509; Whittington et al., 2022, Nature Neuroscience 25:1257–1272) — the finding that grid-like structural coding and Hopfield-style associative memory, established for physical-space navigation, generalize to organizing relational knowledge of any kind. This chapter introduces one genuinely new extension (§98.3); the rest formalizes correspondences that already existed.

THE DETERMINISTIC PRINCIPLE, NOT THE TRAINED IMPLEMENTATION **The research this chapter cites is substantially implemented, in its original form, as a trained recurrent neural network with learned, factorized representations. This chapter adopts none of that: every mechanism named here is deterministic by construction, exactly as §91.9 and Chapter 7 already are. Where the cited research's own implementation relies on learned weights, this chapter's correspondence stops at the structural principle — grid-like periodic coding paired with associative convergence organizes relational knowledge — and does not extend to adopting a trained network as part of QueryBook's architecture.**

98.1 The Three Components

Cognitive Map Framework	QueryBook Correspondent	Status
Grid-like periodic structural coding	Hexagonal lattice indexing (§91.9), deterministic by construction	Already real, formalized here
Hopfield associative memory module	The Attractor Network (Chapter 7, Claim 6)	Already real, formalized here
Generalization from spatial to any relational domain	Ontology Mesh's poly-hierarchical concept structure (Chapter 6, §20.1 of the patent)	Already real; §98.3 extends §91.9's indexing principle to this domain specifically, which is genuinely new

98.2 Why This Formalization Is Useful

Naming this correspondence clarifies that §91.9's grid-cell indexing and Chapter 7's Attractor Network were never two unrelated techniques that happened to both cite real neuroscience — the cited research treats structural coding and associative convergence as one integrated principle, and this chapter states that QueryBook's own use of both already reflects that integration, consistent with how Chapter 95 named the already-real Complementary Learning Systems correspondence rather than building something new.

98.3 Generalized Deterministic Concept Indexing

This section extends §91.9's hexagonal lattice indexing principle beyond geometric shape matching to the Ontology Mesh's concept space generally, reflecting the cited research's central finding that grid-like coding is not spatial-specific. A concept within a federated domain sub-ontology (§20.1 of the patent)

may be assigned coordinates within a fixed, multi-scale hexagonal lattice — defined once by construction, never learned or trained — enabling a “nearby concept” query to proceed by deterministic lattice-coordinate proximity before the Ontology Mesh's existing poly-hierarchical traversal resolves the precise relationship. This is an additional indexing method over the Ontology Mesh's already-specified structure; it modifies no existing concept relationship and introduces no new concept-classification mechanism.

- Given the same concept and the same lattice configuration, indexing produces the same coordinates every time — the same determinism guarantee already stated for §91.9's geometric application.
- This section does not alter how a concept's actual domain membership or poly-hierarchical classification is determined (§6.1); it adds a proximity-search convenience over structure the Ontology Mesh already fully specifies.

98.4 Integration Requirements

This chapter requires the grid-cell indexing principle already specified at §91.9; the Attractor Network (Chapter 7, Claim 6); and the Ontology Mesh's poly-hierarchical structure (Chapter 6, §20.1 of the patent). It modifies none of their specifications.

98.5 Summary

The correspondence between QueryBook's grid-cell-inspired indexing, its Attractor Network, and the real cognitive map framework in computational neuroscience is now named explicitly, and the same deterministic indexing principle already used for geometric shape matching now extends to general concept proximity search within the Ontology Mesh. The trained, learned-representation implementation described in the cited research was deliberately not adopted, consistent with this Bible's existing determinism requirement.

Chapter 99: Study and Learning Tools

This chapter specifies six presentation and organization capabilities over already-grounded content, closing a gap identified in a systematic feature audit against this Bible's original design intent. Every capability in this chapter reuses an already-real mechanism — grounded generation, the Natural Language Interaction Layer, the Ontology Mesh, and Tutorial Mode — rather than introducing new reasoning or generation.

99.1 Glossary Generation

A glossary entry for a term is generated from the grounding Fact Unit's own definitional content via the standard grounded generation pipeline (Claim 8); no new extraction mechanism is introduced. A generated glossary is scoped to the terms actually appearing in the source content a reader is engaging with, cited back to their source Fact Units exactly as any other grounded output.

99.2 Study Guide Builder

A study guide compiles already-grounded content — key facts, Comprehension Check items (§52.13.2), and glossary entries (§99.1) — into a single structured document scoped to a chapter, book, or reader-defined range. This section introduces no new content generation; it is an assembly and formatting layer over already-specified outputs.

99.3 Spaced Repetition Scheduling

Flashcard Generation (§52.13.2.1) items may be scheduled for review using a standard spaced-repetition interval algorithm, tracked as a session-scoped record under the same pattern already specified at §91.4 and formalized at Chapter 95 — a fast, per-reader schedule, never merged into the permanent Fact Graph. This section introduces no new content generation; it schedules already-existing flashcard content.

99.4 Cognitive Scaffolding

Tutorial Mode (§52.13.2) content may be presented in an explicitly staged sequence — prerequisite concepts before dependent ones — derived from the Ontology Mesh's own concept relationships (Chapter 6) rather than a separately-authored curriculum structure. This section adds a presentation ordering; it does not alter what Tutorial Mode generates.

99.5 Beginner and Expert Presentation Modes

The Natural Language Interaction Layer's existing audience-adaptation capability (Chapter 50) is exposed as an explicit reader-facing toggle between a simplified and an advanced presentation register, rather than only an implicit, system-inferred adaptation. Consistent with Chapter 50's existing guarantee, only phrasing and structural complexity vary between modes; the underlying fact, its confidence score, and its provenance are never altered.

99.6 Concept Clustering

Related concepts are grouped for presentation using the Ontology Mesh's existing poly-hierarchical structure (Chapter 6) and, where applicable, the deterministic proximity indexing of §98.3 — never a separately-learned clustering model. This section introduces no new concept-relationship mechanism; it presents already-specified structure grouped for a reader's convenience.

99.7 Integration Requirements

This chapter requires grounded generation (Claim 8); Comprehension Check and Flashcard Generation (§52.13.2–.2.1); Session-Scoped Fact Representation and its Complementary Learning Systems formalization (§91.4, Chapter 95); Tutorial Mode; the Natural Language Interaction Layer (Chapter 50); and the Ontology Mesh (Chapter 6, §98.3). It modifies none of their specifications.

99.8 Summary

Six study and learning capabilities — glossary generation, study guide assembly, spaced repetition, cognitive scaffolding, an explicit beginner/expert toggle, and concept clustering — are now specified, each a presentation or organization layer over already-grounded, already-specified content, introducing no new generation or reasoning mechanism.

Chapter 100: Content Verification and Argumentation Tools

This chapter specifies five verification and argumentation capabilities, closing the remainder of the gap identified in the same feature audit that produced Chapter 99. Every capability reuses an already-real mechanism — the Cross-Domain Rule Engine's contradiction detection, the Provenance Ledger's corroboration data, Discussion Mode, and the Scenario Graph — rather than introducing new judgment or generation capability.

VERIFICATION AGAINST THE GROUNDED FACT GRAPH, NEVER AN INDEPENDENT JUDGMENTEvery capability in this chapter operates by checking a claim against QueryBook's own grounded Fact Graph — it never renders an independent judgment about what is true, biased, or correct outside that grounding. Bias detection specifically — flagging ideological framing with no factual ground truth to check against — was evaluated and excluded from this chapter for exactly this reason and remains unspecified pending its own dedicated scoping.

100.1 Misconception Correction

Where a user's stated claim directly contradicts a well-corroborated Fact Unit, the Cross-Domain Rule Engine's existing contradiction-detection logic (Chapter 17) identifies the conflict, and the response cites the specific contradicting Fact Unit and its provenance. This section does not specify a general mechanism for identifying “common misconceptions” in the abstract; it is scoped exclusively to a claim's direct, checkable contradiction with the grounded Fact Graph.

100.2 Error-Spotting

Within a user-submitted or ingested text, a specific factual claim is checked against the grounded Fact Graph using the same mechanism as §100.1, and an internally self-contradictory pair of claims within the same text is identified using the Cross-Domain Rule Engine's existing logic, consistent with how Chapter 96 already applies this same internal-consistency check to external AI system output. This section introduces no new contradiction-detection mechanism; it applies an already-real one to a new input type.

100.3 Source Triangulation Display

The Provenance Ledger already records each corroborating ingestion event supporting a Fact Unit (Chapter 5); this section exposes that already-tracked corroboration count and its constituent sources as an explicit, reader-facing display accompanying a cited fact, rather than only an internal confidence-calibration input. This section introduces no new corroboration-tracking mechanism; it surfaces data already recorded.

100.4 Argument Analysis

Discussion Mode's existing grounded viewpoint-adoption capability (§52.13.3) is applied to identify a source text's claim-evidence-warrant structure, using the argumentative structure already present in ingested content — including CONTRASTS_WITH edges acknowledging existing counterarguments — rather than a new argumentation-parsing mechanism.

100.5 Debate Preparation

A structured argument for a specified position is assembled from Fact Units the grounded Fact Graph already supports for that position, using the Scenario Graph's existing hypothetical-framing capability (Chapter 11) to represent a position as a bounded, explicitly-labeled framing rather than an assertion of

QueryBook's own view. Where the grounded Fact Graph does not support a requested position with citable content, this section does not fabricate supporting content to fill the gap — it returns only what is genuinely grounded, explicitly noting where support is limited.

100.6 Source Framing Signals

This section closes bias detection's dedicated scoping, deferred at §100's original specification, with two distinct capabilities — one built, one declined again after being explicitly re-proposed as a user-optional feature.

A CONTESTED JUDGMENT DOES NOT BECOME GROUNDED BY BEING OPTIONAL **A proposal to let a user opt into QueryBook's own judgment of a source's ideological framing or bias was reconsidered and declined a second time. Opt-in consent resolves a risk when the underlying content is real but sensitive — it does not resolve a risk that the underlying content has no factual ground truth to check against in the first place. QueryBook rendering its own bias judgment is not a presentation preference comparable to §99.5's beginner/expert toggle; it is new, contested content generation with nothing in the Fact Graph to ground it, and a user's choice to view it does not supply that grounding. No configuration of this chapter enables QueryBook's own bias or framing judgment.**

100.6.1 Documented Editorial Position

Where a source's own editorial or institutional position is itself documented and citable — a publication's stated mission, a disclosed funding relationship, an author's disclosed affiliation — that documented position is surfaced as a grounded fact with its own citation, using the same grounding mechanism as any other claim (Claim 8). This is not QueryBook's judgment of the source; it is a citable fact about the source, sourced the same way every other fact in this Bible is sourced.

100.6.2 Quantifiable Source Composition Signals

Where checkable, quantifiable facts about a source's citation composition exist in the ingested content itself — for example, the number of distinct studies a source cites and each cited study's own disclosed funding source — those facts are surfaced directly, without a conclusory judgment of bias attached. A reader is given the same countable, citable facts a human researcher would independently verify, and forms their own conclusion from them; QueryBook states the count and the citation, not a verdict.

- This section produces a fact (“this source cites four studies; three share a common funding sponsor”), never a conclusion (“this source is biased”) — the distinction is enforced at generation: a conclusory framing or bias term is not produced by this section under any configuration.

100.7 Integration Requirements

This chapter requires the Cross-Domain Rule Engine (Chapter 17); the Provenance Ledger (Chapter 5, Claim 2); Discussion Mode (§52.13.3); the Scenario Graph (Chapter 11, Claim 7); and, for §100.6, the grounded generation mechanism of Claim 8. It modifies none of their specifications.

100.8 Summary

Five verification and argumentation capabilities — misconception correction, error-spotting, source triangulation display, argument analysis, and debate preparation — are now specified, each checking or presenting against the grounded Fact Graph rather than rendering an independent judgment. Bias detection was evaluated and explicitly excluded from this chapter, remaining unspecified pending its own dedicated scoping conversation.

Chapter 101: Verified Neuromorphic Correspondence Methodology

This chapter names and formalizes the methodology already used, consistently, every time a real neuroscience or cognitive-science finding has been incorporated into this Bible: the Hopfield-network correspondence for the Attractor Network (Chapter 7), Complementary Learning Systems (Chapter 95), grid-cell indexing (§91.9), and the Cognitive Map / Tolman-Eichenbaum Machine correspondence (Chapter 98). This chapter introduces no new neuromorphic capability of its own; it is a process specification, applicable to future additions of the same kind.

A FIVE-STEP PROCESS, EVERY STEP ALREADY DEMONSTRATED **Every named correspondence in this Bible followed the same five steps, in the same order, without exception. This chapter states that process explicitly so it is checkable and repeatable, rather than an informal habit that could drift or be applied inconsistently.**

101.1 The Five Steps

Step	Requirement	Demonstrated Instance
1. External Verification	The candidate research finding is verified against its actual primary publication before any Bible content is written, never accepted on the strength of a submitted summary alone.	Hopfield capacity bound traced to its 1982 origin; CLS traced to McClelland, McNaughton & O'Reilly 1995; grid cells traced to the 2014 Nobel Prize and Hafting et al. 2005; TEM traced to Behrens et al. 2018 and Whittington et al. 2022.
2. Implicit-Correspondence Check	The system is checked for whether the correspondence already exists implicitly, before any new mechanism is built.	The Attractor Network already used Hopfield-style dynamics before Chapter 7 named it as such; session-scoped records already behaved like a fast memory system before Chapter 95 named the CLS correspondence.
3. Deterministic-Extraction Requirement	Where the cited research's own implementation relies on a trained, non-deterministic component, only the structural principle is extracted; the trained component is explicitly excluded.	TEM's own architecture is a trained recurrent neural network; Chapter 98 adopted only its structural insight (periodic coding generalizes to relational knowledge), explicitly declining the trained implementation.
4. Extension-Over-Invention Requirement	The correspondence is built as an extension of an already-specified, already-claimed mechanism wherever one exists, rather than a new parallel system.	Grid-cell indexing (§91.9) extended the Attractor Network's existing convergence mechanism rather than introducing a new stabilization system; §98.3 extended §91.9 rather than building a second indexing method.
5. Named Formalization	The verified, extracted, correctly-scoped correspondence is given an explicit name and citation within its own chapter or section, distinguishing it from any unverified or trained-implementation version of the same research.	Chapter 95, §91.9, and Chapter 98 each state their own citation and their own explicit boundary against the version not adopted.

101.2 What This Methodology Excludes

- A correspondence that has not been externally verified against a primary publication does not qualify for incorporation under this methodology, regardless of how plausible or well-presented the submitted description is — this is the same discipline that distinguished the real Hopfield/CLS/grid-cell/TEM correspondences from the fabricated “Neural Hypergraph Networks” and “Hypergraph World Models” framing evaluated and declined earlier in this Bible's history.
- A correspondence whose only real implementation is a trained, non-deterministic component, with no separable deterministic structural principle, does not qualify — this methodology extracts principles, not trained models.

101.3 Integration Requirements

This chapter requires the specific instances it formalizes — Chapter 7, Chapter 95, §91.9, and Chapter 98 — as its demonstrated record. It modifies none of their specifications; it names the process common to all of them.

101.4 Summary

Future additions of verified neuromorphic or cognitive-science correspondence to this Bible follow the same five-step process already demonstrated four times: external verification, an implicit-correspondence check, deterministic extraction, extension over invention, and named formalization. This is a repeatable, checkable methodology, not an informal pattern.

Chapter 102: Legal Exposure Minimization Through Structural Compliance

This chapter names and formalizes a real pattern already present across this Bible, scattered across many chapters rather than previously stated as one coherent posture: QueryBook reduces its own legal and regulatory exposure through genuine compliance, structural non-custody, and consistent, documented practice — never through defeating or evading a lawful legal process. This chapter introduces no new mechanism; it ties together already-real components under one name, the same formalization discipline already applied at Chapter 95, Chapter 98, and Chapter 101.

EXPOSURE REDUCTION THROUGH COMPLIANCE, NEVER THROUGH EVASION **Every mechanism named in this chapter reduces legal exposure by keeping QueryBook's actual practices genuinely compliant, genuinely narrow in role, and genuinely well-documented — never by reducing what QueryBook can lawfully be compelled to produce once a valid legal order is issued. A request to make QueryBook "immune to subpoena" was directly evaluated and declined elsewhere in this Bible's history: a lawful subpoena is a legal obligation no architecture decision changes, and designing specifically to defeat compliance with one carries its own separate legal exposure, including potential spoliation or obstruction liability. This chapter's entire scope is what happens before that point — sound compliance and genuine data minimization — never what happens after a lawful order arrives.**

102.1 Regulatory Compliance

The Right-to-Erasure Protocol (§36.7) is explicitly built to GDPR Article 17 and CCPA right-to-erasure requirements, at three levels of granularity, with every erasure action logged to the Provenance Ledger. This is direct, genuine regulatory compliance — the actual mitigation for regulatory exposure, not a workaround for it.

102.2 Structural Non-Custody

Several real mechanisms keep QueryBook structurally outside regulatory categories that would otherwise attach significant compliance burden. NFT Integration (Chapter 94) is explicitly non-custodial: QueryBook orchestrates minting but never holds a wallet, key, or token on any party's behalf. Merchandise Commerce (§88.12) is explicitly referring-party-only: QueryBook never captures payment data or acts as a payment processor or fulfillment operator. Both are deliberate design choices that keep QueryBook out of money-transmission and payment-processing regulatory categories, reducing exposure structurally rather than through after-the-fact compliance alone.

102.3 Rights and Intellectual Property Discipline

Chapter 60 requires rights validation before a publisher submission is accepted. Encyclopedia Content Ingestion (§59.10) grounds its fact-extraction boundary directly in Feist Publications, Inc. v. Rural Telephone Service Co., 499 U.S. 340 (1991), extracting only what the real fact/expression dichotomy actually covers, and explicitly declining to extend that principle to protected images or original expression. Both reduce copyright and rights exposure by staying inside well-established legal boundaries rather than testing them.

102.4 Audit Trail as Protective Evidence

The Provenance Ledger (Chapter 5) is elsewhere in this Bible framed primarily as a safety and accountability mechanism; it serves a second function here. A complete, tamper-evident, cryptographically-chained record of what happened, when, and under whose authorization is itself the evidence of good-faith, consistent compliance if QueryBook's practices are ever examined — the same record that enforces accountability internally also demonstrates it externally.

102.5 Consistent, Documented Retention Policy

A clear, standing, consistently-enforced retention policy — whatever that policy specifically is — reduces legal exposure more than an ad hoc or inconsistent one, because inconsistent or suspicious-looking deletion patterns, not the existence of a real policy, are what actually create spoliation risk. The current retain-by-default posture with a genuine, working, user-initiated erasure option (§36.7, §36.7.1, §36.7.5) is exactly this kind of policy: clear, standing, and consistently applied, not a mechanism aimed at reducing what is available to be lawfully produced.

102.6 Integration Requirements

This chapter requires §36.7 and its subsections; Chapter 94; §88.12; Chapter 60; §59.10; and the Provenance Ledger (Chapter 5). It modifies none of their specifications; it names a coherent posture already present across them.

102.7 Summary

QueryBook's legal exposure is minimized through genuine regulatory compliance, structural non-custody in payments and NFT minting, disciplined rights handling grounded in real legal principle, a protective audit trail, and a consistent, documented retention policy — five already-real mechanisms, now named as one coherent posture. None of it operates by reducing what QueryBook can lawfully be compelled to produce; that boundary, stated in this chapter's opening callout, is not configurable by any mechanism named here.

Merge Review Pass 103 — Consolidated Additions

This part records every section added to the Bible since Merge Review Pass 101. Each carries its canonical section number and belongs at that number; they are gathered here so that the pass is auditable as a unit, in the same manner as previous merge passes. Thirty-six addenda are consolidated: twenty-four new chapters (103 through 129, chapters 118 and 119 having been withdrawn) and seventeen sections inserted into existing chapters. This pass additionally repairs eleven citations that pointed into unassigned chapter numbers. Every section below was drafted against a verified reading of the mechanism it extends, and each states explicitly what it does not change.

Structural note. This document also incorporates the structural repair applied before this pass: Chapters 36, 37 and 53 through 59 previously carried their titles as unstyled runs

inside oversized paragraphs and were invisible to a table of contents or navigation pane; Chapter 56 carried no title line at all. All nine now carry Heading 1 styling, and Chapter 56's title was restored from this document's own table of contents. Chapters carrying a real heading rose from 77 to 86. No content was altered by that repair.

Sections added in this pass

§4.9 — Linguistic Resource Registry. Licence provenance for every external linguistic resource; derived-artifact-only ingestion.

§7.6.4 — The Retrieval-Phase Boundary. Names the §7.6.3 capacity bound as a first-order phase transition; sets the conservative operating ceiling.

§16.7 — Wording-Operative Spans. Reproduces operative wording rather than regenerating it, with a non-operative gloss.

§16.8 — Idiom Handling. Detection by compositional-versus-attested divergence; registered equivalents only.

§41.6.2 — Fully Bayesian ConfidenceScore. Confidence as a Beta posterior; replaces the additive accumulator.

§41.6.4 — Precision-Weighted Error Propagation. Weights prediction error by inverse variance rather than confidence differential.

§53.9 — Physical Constraint Enforcement. Gives the Physics Layer a stated constraint, enforced by refusal.

§54.6 — Inheritance, Exception Override, Inverse Traversal. Supplies the rules the Concept Graph's edges implied but did not state.

§96.9 — Behavioural Probing. Measures how an external system behaves, not only what it asserts.

§96.10 — Analysis Quarantine. Analysis records structurally ineligible as a grounding source.

§97.7.1 — Sampling Criticality Control. Exposes the inverse-temperature parameter as a recorded operator control.

§97.8 — Discriminating Instance Generation. Identifies the observation separating two hypotheses the evidence fits equally.

Ch. 103 — Temporal Prediction Calibration. Scores this system's own forecasts under an attestation-horizon constraint.

Ch. 104 — Event Type Catalogue and Environment Vectors. Governed event semantics as Fact Unit extensions; no schema field added.

Ch. 105 — Surfaces, Extensibility and External Integration. Twelve surfaces, each constrained against becoming a route around an existing guarantee.

Ch. 106 — Domain Architecture. Fifteen domains in four layers plus a vertical; the transfer matrix owned by D0; query/ingestion path separation.

Ch. 107 — Deployment Topology and Substrate Independence. Hybrid and edge topology, module replacement, media and immersion interfaces, key custody, and the structural statement of substrate independence.

Ch. 108 — Reasoning Mode Formalization. Deductive, inductive and abductive layers; deduction may write, induction may not.

Ch. 109 — The Interpretive Layer. Context, perspective, ergonomic, cultural and social modulation of an already-admissible result.

Ch. 110 — Outcome Feedback and Operational Calibration. Closes the action-to-observation half of the inductive cycle; feedback is evidence, not correction.

Ch. 111 — Expressive Register and Media-Integrated Expression. A governed register set; affective register from declared signals only; media carried inside a grounded response.

Ch. 112 — Surface Realization. Aggregation, referring expressions, connectives, ordering and fingerprint-bounded lexical variation.

Ch. 113 — Commercial Presentation and Transaction. Advertiser provenance, placement classes, territory pricing and transaction reconciliation.

§41.6.5 — Observation Diversity. Effective source-class count; concentrated corroboration discounted.

Ch. 114 — Directed Capture Equipment Control. Live sensor ingestion driving actuation, under session authorization and a structural veto.

Ch. 115 — Assistive Physical Contact. A separate authorization class for equipment that intentionally touches a person.

Ch. 116 — Attractor Stabilization Beyond Retrieval. Bounded frame reconstruction, difference-preserving alignment, oscillation faults, convergence-regime disclosure.

Ch. 117 — Systematic Trial Architecture. Exhaustive, regression and adversarial trials over a metered budget; trials discover and never apply.

§113.8 — Concept-Adjacent Commercial Relevance. One-hop adjacency as a relevance input, scope-confined, eligibility unchanged.

Ch. 120 — Grasping and Manipulation. Force bounded by the object's fragility limit; release, not retention, is the correct failure.

Ch. 121 — Shared-Space Navigation. Clearance-bounded speed, unconditional yield, stop rather than relax a clearance envelope.

Ch. 122 — Deeper Predictive Hierarchy. Registered predictive levels, lowest-level error attribution, an escalation gate, unattributed error retained.

Ch. 123 — Differentiated Propagation and Tri-Temporal Scope. Edge-class traversal cost; attested past, live present and projected future postures.

Ch. 124 — Coherence Classification and the Context Lock. Six-dimensional coherence taking the least; the enumerated dimensions on which determinism is asserted.

§4.10 — Computational Substrate Output as a Modality. Neuromorphic and biological substrate output admitted as an instrument reading.

§7.7 — Analog Realization of the Convergence. The threshold update performed by a conductance array; uniqueness as a precondition.

Ch. 126 — The AI-Engine Ingestion Module. Prompt composition, conversion to knowledge records, derivation marking, self-corroboration bar, audit.

Ch. 127 — Specialized Live Data Feeds. Publication authority, staleness, restatement distinct from correction, redistribution entitlement.

Ch. 128 — Defined Agents. Complete definition before registration, capability bounded by the registering identity, advisory distinguished from approval.

Ch. 129 — Deployment Licensing and Procurement Readiness. Topology and licence form by purchaser class; six unmet conditions and the sequencing recommendation.

Chapter numbering — unassigned numbers

Chapters 21 through 35 and Chapter 38 are unassigned numbers in this specification. They are not missing content. Their subject matter was renumbered by an earlier merge pass and now lives under other chapters, and a chapter sequence with gaps in it is a cosmetic artifact of that renumbering rather than a loss. Renumbering the surviving chapters to close the gaps is not done, because every citation in this document, in the patent applications and in the feature inventory resolves against the numbers as they now stand, and closing a cosmetic gap by invalidating several hundred working citations would trade a presentational problem for a substantive one.

Eleven citations pointing into the unassigned range were repaired in this pass. Each was traced to the chapter where its subject matter is actually specified, and the repairs are recorded below. The document previously carried its own diagnosis of one of them — a note that a Zero-Knowledge Encryption citation pointed to a chapter that does not exist — which has now been acted upon rather than merely observed.

§21.3 → §52.3 Adaptive Response Engine, specified at Chapter 52.

§21.4 → §97.6 Active Inference Query Generation, specified at Chapter 97.

Chapter 21 → Chapter 52 Interactive Modes, specified at Chapter 52.

§28.3 → §51.4 Verification Gate for programs, specified at Chapter 51.

Chapter 32 → Chapter 41 Confidence Scoring Engine, contradiction detection, continuous evidence integration, CS weighting rules, specified at Chapter 41.

Chapter 35.5 → §36.7 Zero-Knowledge Encryption, specified at Chapter 36.

Chapter 4: Multimodal Ingestion — §4.9 addition

4.9.1 Registration

A linguistic resource is any externally-sourced dataset consulted by the extraction pipeline of §4.3 or the realization pipeline of §16.3 — a lexical database, an annotated corpus, a morphological inventory, a frequency or collocation table, an idiom or metaphor list, a treebank. No linguistic resource is consulted by either pipeline unless it is registered.

A registration records: a resource identifier and version; the licensor; the licence instrument and its version; the permitted-use scope, stated at minimum as whether commercial use is permitted; whether redistribution of the resource itself is permitted; whether attribution is required and in what form; and an ingestion mode of either full or derived-artifact-only, per §4.9.3. A registration lacking any of these fields is invalid and the resource is not consulted.

Registration is a Chapter 60 rights validation, performed by the same mechanism that validates a submitted work, and its outcome is written to the Provenance Ledger. A resource whose licence does not permit the operator's intended use is refused at registration. The refusal occurs before any pipeline consults the resource, so an incompatible licence is a registration-time failure rather than a discovery made after artifacts derived from it are already in production.

4.9.2 Licence propagation to derived artifacts

An artifact derived from a registered resource carries a reference to that resource's registration. Where the resource's licence attaches conditions to derivatives — an attribution requirement, a share-alike obligation, a prohibition on redistributing the resource's own content — those conditions attach to the derived artifact and are enforced at every point the artifact would be redistributed or exposed.

Attribution required by a registered resource is discharged in the deployment's own attribution surface and is not omitted on the ground that the resource is consulted only internally. Where two registered resources impose conflicting derivative conditions on a single artifact — for example a share-alike obligation and a prohibition on redistribution — the artifact is marked non-redistributable and the conflict is recorded, rather than resolved by selecting the less restrictive term.

4.9.3 Derived-artifact-only ingestion

A resource registered in derived-artifact-only mode may be consulted to compute aggregate quantities — a frequency, a collocation strength, a part-of-speech distribution, a morphological paradigm — and no span of the resource's own text is retained in any Fact Unit, index or cache, or reproduced in any output. The retained artifact is the statistic; the source span is discarded at computation.

This mode exists because it is what several substantial resources actually permit. The British National Corpus licence prohibits commercial exploitation or publication of any part of the corpus texts and prohibits their redistribution, while permitting commercial exploitation of results derived from research carried out on the corpus and permitting derived lists and short fragments. A registry that recognized only full ingestion would have to refuse the BNC outright and lose a resource its licence in fact permits QueryBook to use.

The boundary this mode enforces is the fact-expression boundary already stated at §59.10 under Feist Publications v. Rural Telephone Service Co., 499 U.S. 340 (1991). A frequency count computed over a corpus is a fact about the corpus and is not the corpus's protected expression. A retained sentence from that corpus is expression. The mode is the mechanical enforcement of a distinction the Bible already draws for ingested works generally, applied to the resources the pipeline itself consumes.

Compliance with derived-artifact-only mode is verifiable rather than asserted: the count of retained source spans attributable to a resource registered in this mode is zero, and a non-zero count is a compliance failure surfaced by the same audit path as any other Provenance Ledger anomaly.

The practical consequence of the chart is a substitution rather than a loss. Of the four resources the submitted plan named as essential and classified as open, three are not usable as classified, and the tasks they were assigned — segmentation, part-of-speech, dependency structure, collocations — are served by the Open American National Corpus and MASC, which are genuinely unrestricted. The BNC remains available for statistics under §4.9.3. Nothing in the plan's linguistic coverage is actually lost; what changes is which resource supplies it and under what recorded terms.

The submitted document lists the Word Alignment Layer, Lexical Mapping Table, Semantic-Lexical Fusion Layer and Reversible Expression Layer under what

QueryBook already has, and states they were added when improved translation accuracy was requested. They were not. None of those four names occurs in the Bible, as recorded in the translation addendum prepared previously. The same list includes pre-computed question generation and author metadata ingestion as already present; those were absent until specified in an earlier addendum. A roadmap built on that inventory would budget nothing for work that has not been done.

The submitted document also states that the architecture is non-ANN and that biological grounding strengthens the patent. §4.3.1 specifies the semantic parser as a transformer-based encoder of at least one billion parameters fine-tuned on diverse text corpora, so the extraction boundary is a neural network by the Bible's own specification. The accurate and stronger statement is the one the architecture actually supports: the neural component is confined to the extraction boundary, its output is a typed Fact Unit subject to safety gating and confidence calibration before admission, and no neural component participates in retrieval, in the groundedness constraint, or in the generation vocabulary. That is a boundary claim, it is testable, and it survives an examiner reading §4.3.1.

Chapter 7 and Chapter 97 — §7.6.4 and §97.7.1 additions

7.6.4.1 Step one — external verification against the primary publication

The capacity figure stated at §7.6.3 — approximately 0.14N stable patterns for N units — is not an engineering rule of thumb. It is a critical value derived analytically in the statistical mechanics of the Hopfield model, and it marks a phase transition. The primary publications are:

The 1987 figure of αc = 0.14 is the identical number already carried at §7.6.3. No value changes. What follows is the identification of what that number is.

7.6.4.2 Step two — implicit-correspondence check

The correspondence is already present in the specification and is described in engineering vocabulary rather than in the vocabulary of statistical mechanics. §7.6.3 states that exceeding the bound produces catastrophic interference in which storing a new pattern corrupts recall of existing ones. That is the observable behaviour of the discontinuous disappearance of the retrieval states at αc. No new mechanism is therefore required, and none is introduced. This step, under Chapter 101, terminates the search for new machinery.

7.6.4.3 Step three — extraction of the deterministic structural principle

The deterministic principle extractable from the cited work is a two-part operating constraint on a ratio, not a probabilistic claim about brains. Let P be the number of distinct stable configurations the Attractor Network is operated to hold and N the number of units. The load ratio is α = P/N. The principle is:

Because the transition at αc is discontinuous, degradation is not graceful: the system does not become gradually less reliable as the boundary is approached and then fail. It retrieves, and then it does not. This is why the constraint is enforced as an admission refusal at the ceiling rather than as a quality metric monitored after the fact.

7.6.4.4 Step four — extension of claimed mechanism over new invention

The constraint is enforced through mechanism already specified. Admission of a new stable configuration is a write to the Synaptic Weight Store, already governed by the weight-update integrity controls of §7.6.1. Refusal of an admission that would breach the ceiling is a Safety Substrate enforcement point of the kind already placed at every operation layer under Chapter 8. Recording of the refusal and of any deployment election of the permitted regime is a Provenance Ledger append under Chapter 5. Consolidation on refusal is the EWC regularization path already specified at §7.4.4. No new subsystem is created.

7.6.4.5 Step five — named formalization

The boundary at αc is hereby named the Retrieval-Phase Boundary, and the ratio α = P/N is named the Attractor Load Ratio. The component enforcing the ceiling is named the Attractor Load Regime Governor. The correspondence is: QueryBook's Attractor Network is operated strictly on the ordered side of a first-order phase transition whose location was established analytically by Amit, Gutfreund and Sompolinsky (1985, 1987), and the capacity bound at §7.6.3 is the engineering expression of that transition.

7.6.4.6 What is deliberately not adopted

The critical brain hypothesis — the proposition that biological neural systems operate near a critical point between order and disorder, evidenced by power-law distributed event sizes — is not adopted as a mechanism of this system, and no claim in this specification depends on it. Three reasons are recorded, so that the exclusion is a documented decision rather than an omission.

First, the transition class differs. The critical brain hypothesis concerns a second-order, continuous transition. The transition at αc is first-order: the retrieval states disappear discontinuously. Naming the §7.6.3 bound a phase transition is exact; naming it criticality in the critical-brain sense would not be.

Second, the hypothesis is contested in its own literature. Evidence is reported in cell cultures, brain slices and anesthetized animals, with inconsistent results in awake animals and humans, and the requirement that a control parameter be tuned exactly to the transition point remains an open objection. Chapter 101's verification step is not satisfied by a proposition in that state.

Third, the reported computational benefit is conditional rather than general. Work on plastic spiking networks running on neuromorphic hardware, in which distance to criticality was directly adjustable, found network capacity maximal at criticality but performance improved only for complex tasks while simple tasks were degraded. Grounded retrieval over a fixed record set is the second kind of task. Operating the retrieval path near a critical point would purchase dynamic range with exactly the variance the rest of this specification exists to eliminate.

Accordingly the retrieval path is operated away from any critical point, deep in the ordered regime, and the one place in the system where variance is the intended product is treated separately at §97.7.1 below.

§97.7 already specifies a stochastic sampler on the hypothesis-generation path, in which the selection probability of a candidate is a logistic function of a scored difference against an incumbent, scaled by an inverse-temperature parameter β, of the form p = 1 / (1 + e^(−β·ΔE)). That sampler is walled off from the Attractor Network and from grounded retrieval, and no output of it is admitted to the retrieval or generation path.

The parameter β is an inverse temperature, and temperature is the control parameter of the order–disorder transition in the same statistical-mechanics framework that locates αc. §97.7.1 exposes β as an operator-set control with named regimes, so that hypothesis diversity becomes a configured quantity with a recorded setting rather than a fixed implementation constant.

Three constraints govern the control. The setting is recorded in the Provenance Ledger with the identity of the operator who set it, so that a hypothesis set is reproducible from its recorded β. No value of β causes any output of the sampler to reach the retrieval or generation path; the wall specified at §97.7 is unaffected by the setting. And a candidate hypothesis produced at any β retains the proposed status field of Chapter 97 and transitions to validated only on confirmation by an identified human reviewer.

The system does not self-tune β toward any point, critical or otherwise. Self-organised tuning would make hypothesis output depend on the system's own history in a way no

operator selected and no ledger entry would fully capture, which is inconsistent with the reproducibility requirement above.

Row 139 evaluates to admission in the conservative regime: an 8,192-unit Attractor Network at the §7.2.4 default holding 1,000 stable configurations carries $\alpha \approx 0.122$, which is inside the permitted regime but above the 0.05 conservative ceiling — the expression as written therefore returns the consolidation branch, which is the correct and intended behaviour for a default-configured deployment approaching that load.

Row 140 evaluates the sampler probability at $\beta = 0.5$ and $\Delta E = 0.4$, returning approximately 0.5498 — a broadened selection consistent with the intermediate regime.

Every value in this addendum was checked against the primary publications rather than carried from a secondary source: $\alpha c \approx 0.138$ and the 1.5% error bound at $T = 0$ from Amit, Gutfreund and Sompolinsky; the discontinuous disappearance of the retrieval states at αc from the same source; the $\alpha \lesssim 0.05$ global-minimum regime from the phase diagram of the 1987 paper; and the replica-symmetry-breaking corrections at approximately 0.144, against which the conservative ceiling is deliberately not set. The §7.6.3 figure of 0.14N is unchanged by this addendum and required no correction.

One point for counsel. Claims 30 through 32 recite an operating constraint on a ratio and a refusal behaviour, both of which are concrete and testable. Claims 33 and 34 recite a configuration control and a negative limitation on its reach. None of them recites the critical brain hypothesis, and none depends on that hypothesis being true. That is deliberate: a claim resting on a contested proposition about biological neural systems would be vulnerable in a way these are not.

Chapter 16: Multilingual Regeneration — §16.7 and §16.8 additions

16.7 Wording-Operative Spans

A wording-operative span is a span of source content whose exact wording carries force independently of its meaning, such that a semantically equivalent restatement is not a substitute for it. Canonical cases are a direct quotation, a statutory or regulatory phrase, a term defined in a patent claim, a contractual clause, and an instruction whose precise wording is prescribed by a regulator.

A Fact Unit of type QUOTATION, already present in the type set at Chapter 4, and a Fact Unit carrying an explicit wording-operative marking, are both wording-operative. The marking is set at ingestion by the author or publisher under Chapter 60's rights framework, or by an operator within a Discipline Mode whose conventions require it — LEGAL and MEDICAL_CLINICAL both do.

A wording-operative span is reproduced in its source language and its source wording, and is not regenerated. Where the response is delivered in a different language, the reproduced span is accompanied by a regenerated gloss carrying an explicit non-operative marking, so that a reader in the target language receives both the operative text and an understanding of it without the two being confusable. The gloss is grounded and cited exactly as any other regenerated output; the reproduced span is cited to its source rather than to a Narrative Graph edge, because it was not generated.

Reproduction of a wording-operative span is subject without exception to Chapter 60's rights validation and to the fact-expression boundary already stated at §59.10: extracting a fact from a source is not a use of the source's protected expression, but reproducing the source's expression is. A wording-operative span whose reproduction is not licensed is not reproduced; the response states that the operative wording is not reproducible under the applicable rights and supplies the regenerated gloss alone, marked as non-operative.

A wording-operative span is exempt from the Factual Groundedness rewriting of §16.3 Stage 6, which would otherwise rewrite or qualify a span it could not map proposition-by-proposition to a Narrative Graph node. The exemption is narrow: it suspends rewriting of the reproduced span only, and applies to no other span of the response.

16.8 Idiom Handling

The Bible contains no specification of idiom handling. The single occurrence of the word in the document refers to idiomatic Go interfaces in an SDK description. This is a genuine gap: an idiom is a span whose attested meaning diverges from the compositional meaning of its parts, and a system that extracts meaning compositionally will extract the wrong Fact Unit from one.

An idiom is detected at ingestion where the compositional reading of a span and its attested reading, as registered in the Ontology Mesh, diverge by more than a configured threshold. On detection, the Fact Unit records the attested meaning, not the compositional one, and carries a marking identifying it as idiom-derived together with the source span.

At regeneration, an idiom-derived Fact Unit is realized using a target-language equivalent where one is registered in the Ontology Mesh for the same attested meaning. Where no equivalent is registered, the attested meaning is rendered literally and plainly, accompanied by a note that the source expressed it idiomatically. A target-language idiom is never selected on the basis of surface similarity to the source idiom, and no equivalence is generated that has not been registered — a false idiomatic equivalent is a fluent error, which is the failure mode this architecture exists to make unreachable.

Where the source idiom is itself the subject of the query — a reader asking what a phrase means, rather than asking a question the phrase happens to answer — the source span is treated as wording-operative under §16.7 and reproduced rather than replaced.

Chapter 41: Predictive Cognitive Architecture — §41.6.2 addition

41.6.2.1 Representation

The ConfidenceScore ceases to be a scalar and becomes the mean of a Beta distribution over the probability that the Fact Unit's assertion is accurate. Two parameters are stored in place of one number: α, accumulated evidence for the assertion, and β, accumulated evidence against. The reported ConfidenceScore, which every existing consumer continues to read as a value in [0,1], is the posterior mean.

```
CS = α / (α + β)
```

The Beta distribution is the conjugate prior for a Bernoulli likelihood, which is what makes this tractable: the posterior after observing new evidence is again a Beta distribution, obtained by addition alone. There is no integration to perform, no sampling, and no iterative fitting.

```
α' = α + w·s        β' = β + w·f
```

where s counts corroborating observations, f counts contradicting observations, and w is the weight of the observing source, computed from the source reliability and modality coefficient already specified for confidence calibration at §4.6. Weighting by w is why evidence is admitted as a fractional count rather than an integer: a source of reliability 0.85 contributes 0.85 of an observation, not one.

41.6.2.2 The behaviour this produces

The same one contradiction now moves a weak belief far and a settled belief slightly, without any rule saying so. The behaviour is a consequence of the arithmetic rather than a special case in it.

Compare the third row against the additive rule's −0.0500 for the identical situation. The settled belief now moves −0.0429 on one contradiction and −0.3000 on ten — which is precisely the multi-source contradicting evidence §41.4.5 says is required. The claim that strong priors resist weak evidence stops being a description and becomes a computed result.

41.6.2.3 ConceptStabilityWeight as prior strength

CSW maps onto this model exactly, and the mapping is the reason the two mechanisms belong in one specification. A Beta prior carries two independent quantities: where it sits, $\alpha/(\alpha+\beta)$, and how firmly it is held, $n_0 = \alpha + \beta$. The first is the concept's current confidence. The second is CSW.

$\alpha_0 = CS_0 \cdot n_0$ $\beta_0 = (1 - CS_0) \cdot n_0$ $n_0 = f(CSW)$

The mapping f is monotonic and configurable per FactType in the Ontology Mesh, on the same terms as the decay constant λ at §4.6.3. A concept node at CSW 0.90 with a pseudo-count of 20 requires roughly ten contradicting observations to fall to 0.60; the same node at CSW 0.30 with a pseudo-count of 3 falls further on one. The World Brain's settledness and the Semantic Brain's confidence are thereby expressed in one arithmetic instead of two vocabularies.

41.6.2.4 Credible intervals for free

§10.3.1 already requires that every PIE prediction carry a Bayesian 95% credible interval computed via conjugate priors. Under a scalar ConfidenceScore that interval has to be obtained from somewhere else, because a single number carries no dispersion. Under the Beta representation it is a property of the stored parameters.

$\text{half-width} \approx 1.96 \cdot \sqrt{CS \cdot (1-CS) / (\alpha+\beta+1)}$

41.6.2.5 Temporal decay, corrected

§4.6.3 applies exponential decay directly to the ConfidenceScore. Applied to a posterior mean that would be wrong: decaying the mean of a well-evidenced belief drives it toward zero rather than toward uncertainty, so an old, heavily-corroborated fact would eventually read as false rather than as stale.

Decay instead acts on the evidence counts, shrinking both parameters toward the prior at the same rate. The mean is preserved while the interval widens, which is the correct meaning of an ageing observation: not that it has become false, but that it has become less firmly established.

$\alpha_t = \alpha_0 + (\alpha - \alpha_0) \cdot \exp(-\lambda \Delta t)$ $\beta_t = \beta_0 + (\beta - \beta_0) \cdot \exp(-\lambda \Delta t)$

λ remains the per-FactType constant already specified, and immutable historical facts retain λ = 0 and therefore do not decay at all. This is an amendment to §4.6.3 and should be recorded as one.

41.6.2.6 Bounds and refutation

Because α and β are both strictly positive, CS is never exactly 0 and never exactly 1, satisfying the Appendix A constraint that a ConfidenceScore is never exactly zero

unless the assertion has been falsified. Falsification remains a distinct state set explicitly, not a limit approached by accumulating contradictions: a fact that has been refuted is marked as refuted, and an arbitrarily large β is not the same statement.

The update is closed-form addition. The same Fact Unit under the same evidence in the same order yields the same posterior on every execution, and because addition is commutative the same evidence in any order yields the same posterior as well — a stronger property than the additive rule offers, since a sequence of clamped additive updates can depend on order once a bound is reached.

Sampling-based Bayesian methods are excluded from this path and from the retrieval path generally. Markov chain Monte Carlo, particle filtering and variational sampling produce a different answer on each run by construction, which would break the guarantee at claim 1 that the same query over the same records returns the same answer. This specification contains no such method, and §97.7's Boltzmann sampler remains the only stochastic component in the system, confined to hypothesis generation and walled off from retrieval and generation.

The distinction is worth stating precisely because the two are commonly conflated. Bayesian inference by conjugate update is deterministic. Bayesian inference by sampling is not. Only the first is adopted.

Existing Fact Units carry a scalar ConfidenceScore and no evidence counts. Migration initialises each with a pseudo-count reflecting how much is actually known about its evidential history, rather than fabricating one.

Migration is recorded in the Provenance Ledger per Fact Unit, carrying the scalar the record held and the parameters assigned, so that any later question about a confidence value can be traced across the change. The reported ConfidenceScore is unchanged by migration in every case, since $\alpha_0/(\alpha_0+\beta_0) = CS$ by construction; what changes is that the score now carries its own evidential weight.

Row 156 evaluates the posterior mean after one contradiction against a prior of α=18, β=2, returning 0.857143. Row 157 confirms the resulting revision of 0.0429 falls below the 0.05 the additive rule would have applied, which is the resistance property §41.4.5 claims and the additive rule does not deliver.

Chapter 41: Predictive Cognitive Architecture — §41.6.4 addition

41.6.4.1 Precision is already stored

Under §41.6.2 a ConfidenceScore is the mean of a Beta distribution with parameters α and β. A distribution carries dispersion as well as location, and precision in the

Predictive Coding sense is inverse variance. No new field is introduced by this section; the quantity is computed from parameters already held.

```
var  =  CS · (1 − CS) / (α + β + 1)          π  =  1 / var
```

The second and third rows are the point. Both report a ConfidenceScore of 0.9000 and a scalar score cannot distinguish them, yet their precisions differ by a factor of approximately ten. That factor is exactly what should govern how far each is permitted to move the model above it.

41.6.4.2 The propagation gain

Where an error signal of precision π_e contradicts a concept attribute of precision π_p, the fraction of the available revision applied to that attribute is the error's share of the total precision.

```
gain  =  π_e / (π_e + π_p)       equivalently       gain  =  var_p / (var_p + var_e)
```

The second form is the one an implementation should use: it avoids computing reciprocals, is numerically stable as either variance approaches zero, and is directly evaluable from the stored parameters. The revision applied to the attribute's ConceptStabilityWeight is the gain multiplied by the configured maximum revision for the attribute's FactType, and the revision is applied through the ordinary §41.6.2 update rather than by writing a CSW value directly.

Compare the first row against the mechanism it replaces. Under the ConfidenceScore differential rule that same blurry observation, extracted at high confidence, revises the model in proportion to its distance from the expectation. Under precision weighting it moves the model by roughly half a percent of the available revision. Nothing about the error's magnitude changed; what changed is that the system now accounts for how much is known about the error.

41.6.4.3 Where it applies, and where it does not

Precision weighting governs revision of the World Brain concept model. It does not govern retrieval, it does not govern the permitted generation vocabulary, and it does not make groundedness probabilistic. A proposition not traceable to a Fact Unit remains unreachable at generation irrespective of any precision value, because the constraint at claim 1 operates on the permissible vocabulary rather than on a likelihood. This boundary is stated because precision weighting is exactly the kind of mechanism that invites the two to be conflated, and the structural guarantee is the stronger of the two claims.

The Semantic Brain is likewise unaffected in its record-level behaviour. A contradicted Fact Unit is revised under §41.6.2 on its own evidence; §41.6.4 determines only how much of that contradiction reaches the concept attributes above it.

A gain below a configured floor produces no revision at all rather than an arbitrarily small one, and the suppressed error is recorded in the contradiction index so that a pattern of individually-insignificant errors against one attribute remains queryable. Ten low-precision contradictions of the same attribute are evidence even where none of them individually moves the model, and discarding them silently would lose exactly the signal that matters.

41.6.4.4 Determinism

The gain is a ratio of two quantities computed in closed form from stored parameters. The same error against the same attribute yields the same gain on every execution. No sampling is introduced, and §97.7's Boltzmann sampler remains the only stochastic component in the system.

Row 158 evaluates the gain for a single-observation error against a two-hundred-observation concept attribute, returning 0.005344. Row 159 applies the suppression floor to the same figure and returns suppression, with the contradiction retained in the index.

The second enhancement named in the review record, a deeper Predictive Coding hierarchy, is not built here. The present architecture is two-level: the Semantic Brain and the World Brain. Adding intermediate levels would require specifying what each level predicts, what error each propagates and how attribution across levels is resolved, and none of that is answerable by extending an existing mechanism in the way this section extends §41.5.1. It remains a real candidate and should not be attempted until precision weighting has been measured in operation, since a deeper hierarchy without precision weighting would compound the defect identified in Part 1 at every additional level.

A related measurement opportunity is worth recording. Chapter 103's calibration layer scores forecasts against recorded outcomes by horizon and domain. Precision-weighted propagation is a hypothesis about how a world model should revise, and Chapter 103 is the instrument that can test whether a precision-weighted model forecasts better than a differential-weighted one. That comparison would be an internal measurement under Chapter 77 rather than a claim, and it is the honest way to establish that this section improved anything.

The uncertainty decomposition of §3.8.1 supplies this mechanism’s inputs in explicit form: the measurement-precision sub-parameter is the precision this section already stores and uses, the epistemic–aleatory classification determines whether a persistent error warrants continued precision-weighted revision or reflects irreducible dispersion,

and every gain application is journaled in the adjustment-history layer of §3.8.5, making each revision replayable. (Added Pass 115, September 2, 2026.)

Chapter 53: World Aware System Model — §53.9 addition

53.9.1 Constraint registration

A physical constraint is registered against a domain of the Ontology Mesh and carries: an identifier immutable from registration; the invariant it asserts, expressed as a residual expression over quantities the completion supplies; a tolerance, absolute or relative; the unit system in which the quantities are expressed, resolving to the SI base units the sensor ingestion path at §4.3.6 already normalises to; and an applicability predicate stating the conditions under which the constraint holds.

The applicability predicate is not optional and is the field that keeps this mechanism honest. Conservation of momentum holds for a closed system; a completion describing an open one is not in violation of it, and a constraint that cannot state its own preconditions would refuse correct completions. A constraint whose applicability predicate is not satisfied by a given completion is not evaluated against it, and the non-evaluation is recorded rather than silently skipped.

53.9.2 Evaluation and refusal

WorldModel.Infer produces a candidate completion in which fields absent from the partial description have been filled. Before that completion is returned, every registered constraint whose applicability predicate the completion satisfies is evaluated against it. A completion whose residual exceeds the constraint's tolerance is refused.

Refusal is not the end of the operation. The completion is re-attempted with the violating field left unspecified, so that a narrower completion carrying an acknowledged gap is returned in preference to a fuller one that is physically impossible. This mirrors the generation rule at claim 1: a candidate proposition that cannot be grounded is replaced by a narrower one that can, rather than being emitted with a caveat.

Worked example, momentum. A partial scene describes a two-kilogram body moving at three metres per second striking a stationary one-kilogram body, in a system whose applicability predicate for closure is satisfied. Total momentum before the interaction is 6.0 kg·m/s, and the registered relative tolerance is two per cent, giving 0.12 kg·m/s.

Worked example, gravitational fall. A body is observed at 2.0 metres and the completion concerns its position 0.4 seconds later, unobserved. Under the registered constraint the expected position is $2.0 - 0.5 \cdot 9.80665 \cdot 0.4^2 = 1.2155$ metres. A completion asserting

2.5 metres carries a residual of 1.2845 metres and, additionally, a sign of vertical displacement contrary to the constraint's stated direction. It is refused on the residual alone; the sign test is recorded as the reason so that a reviewer sees not merely that the completion was wrong but in what way.

53.9.3 Relation to the plausibility score

§53.4.3 already returns a plausibility score for each inferred element. That score and this constraint are different instruments and neither replaces the other. Plausibility ranks admissible completions by how typical they are; the constraint determines which completions are admissible at all. A physically impossible completion is not a low-plausibility completion, and expressing it as one would permit it to be returned wherever no better candidate existed.

Accordingly a refused completion does not appear in the ranked output at any score, and a completion admitted by every applicable constraint is scored for plausibility exactly as it is today. The ordering of operations is fixed: constraints first, plausibility second.

53.9.4 Provenance of a refusal

A refusal produces a queryable record identifying the constraint, the residual, the tolerance and the fields implicated, and referencing the constraint's own provenance. Three consequences follow. A reviewer can ask which constraint refused a completion and on what margin. A constraint that refuses an unexpectedly large proportion of completions in a domain is detectable, which is how a wrongly-parameterised constraint is found rather than quietly suppressing correct inference. And because the constraint carries a citation, a disputed refusal resolves into a question about a cited source rather than about system behaviour.

A constraint is itself revisable through the ordinary mechanisms. It is an ingested assertion with a confidence and a provenance chain, and §41.6.2 governs its confidence exactly as it governs any other. A constraint whose confidence falls below a configured floor is suspended from enforcement rather than deleted, and the suspension is recorded.

53.9.5 Determinism and scope

Every operation in this section is a comparison of a computed residual against a stated tolerance. The same completion under the same registered constraints yields the same admission decision on every execution. No solver, no iteration to convergence, and no sampling is introduced. In particular no differential equation is integrated numerically: the residual expressions registered under §53.9.1 are closed-form, and a constraint requiring numerical integration to evaluate is not registrable under this section.

The scope is scene completion within the World Aware System Model. §53.9 does not govern retrieval, does not govern the permissible generation vocabulary, and does not evaluate ingested Fact Units. An ingested observation that appears to violate a registered constraint is not refused at ingestion — the observation may be correct and the constraint wrong, or the system may be open in a way the applicability predicate failed to capture. Such a case is recorded in the contradiction index of Chapter 17 for human attention, which is the existing mechanism for exactly this situation.

Row 160 evaluates the admissible momentum completion, returning admission; the inadmissible completion substituted into the same expression returns refusal. Row 161 computes the expected position under the gravitational constraint, returning 1.215468 metres, against which a completion asserting 2.5 metres carries a residual of 1.2845.

53.10 Skill Inventory Layer

The Skill Inventory Layer stores procedural knowledge: how to perform a task, manipulate an object, or execute a multi-step operation, held as provenance-tracked procedural concept models distinct from declarative Fact Units. A procedural concept is formed the way any concept is formed — from repeated observation across the ordinary ingestion modalities, including direct sensory experience and ingested video, consolidating on a count — and is retrievable, citable, and bounded by the requester's entitlement like any other record.
A procedural concept informs simulation under the Scenario Graph System and command compilation under the equipment-control sections; it does not itself emit a command, every emission remaining subject to the constraint evaluation and authorization classes specified for equipment control. (Added upon the inventor's approval, August 29, 2026, Pass 113; this section supplies the specification for the Skill Inventory Layer term used in the Core Technology Overview's companion documents.)

Chapter 54: Conceptual Memory Layer — §54.6 addition

54.6.1 Attribute inheritance along IS-A edges

Where a concept node S is connected to a concept node G by an IS-A edge, and G carries a HAS-PROPERTY edge to an attribute A, S acquires A by inheritance unless §54.6.2 applies. Inheritance is transitive along a chain of IS-A edges, and the chain is bounded by a configured maximum depth so that traversal terminates on a cyclic or pathologically deep hierarchy rather than not terminating.

An inherited attribute is marked, on the subordinate concept, with an inheritance tag recording the concept from which it was inherited and the IS-A depth at which it was found. An inherited attribute is therefore never indistinguishable from an attribute asserted directly of the subordinate concept, and a response relying on an inherited

attribute cites both the attribute's originating concept and the IS-A path by which it arrived.

The confidence of an inherited attribute is the product of the confidences of the IS-A edges traversed and the attribute's own confidence at its originating concept, bounded above by the least of those values. Inheritance across a long or weakly-supported chain therefore yields a weaker attribute than direct assertion, which is the intended behaviour: a property inherited through four uncertain subsumptions should not present with the confidence of a directly observed one.

Inheritance is computed at traversal time and is not written back to the subordinate concept as a stored attribute. The Concept Graph therefore does not grow by materialising every inheritable attribute at every subordinate node, and a revision to the superordinate's attribute propagates immediately to every subordinate on the next traversal rather than requiring recomputation of stored copies.

54.6.2 Exception override

Where a subordinate concept S asserts an attribute directly, and that assertion conflicts with an attribute S would otherwise inherit under §54.6.1, the directly-asserted attribute governs for S and the inherited attribute is suppressed for S. Suppression is scoped to S and to concepts subordinate to S. The superordinate concept's own attribute is not modified, is not weakened and remains inherited by every sibling of S that does not itself assert a conflicting attribute.

Where two conflicting attributes are inherited from different superordinate concepts at different IS-A depths, the attribute inherited from the lesser depth — the more specific concept — governs. This is the specificity ordering standard in defeasible inheritance, and it produces the behaviour the Bible's own domains require: a general rule stated of a broad category yields to a narrower rule stated of a subcategory, without either being deleted.

Where two conflicting attributes are inherited at the identical depth, no resolution is performed. The conflict is recorded as an entry in the contradiction index already specified for the Cross-Domain Rule Engine (Chapter 17), and a response relying on the affected attribute reports both values with their respective originating concepts rather than selecting between them. Arbitrary tie-breaking would produce a deterministic answer that is deterministic only in the sense that it is reproducible, which is not the property this system claims.

An overriding assertion is recorded with the identity of the ingestion event or reviewer that produced it, and an override is itself queryable: the set of concepts that override a given superordinate attribute is retrievable, which is what makes an exception structure auditable rather than merely present.

This mechanism is distinct from §59.6(2)'s Conflict Resolution and does not modify it. §59.6(2) resolves disagreement between the sensory-grounded and language-grounded streams about the same concept, producing a conditional attribute value. §54.6.2 resolves disagreement between a concept and its superordinate along an IS-A edge. The two operate on different axes and may both apply to one concept without interaction.

54.6.3 Inverse-edge traversal

Appendix A's predicate schema carries an InversePredicate field identifying the predicate that expresses a given relation in the reverse direction. Reverse traversal of a Concept Graph or Fact Graph edge resolves through that declared inverse at traversal time. No second edge is stored. Edge count therefore remains equal to the number of asserted relations, and no reconciliation between a forward edge and a stored inverse edge is required, because there is no stored inverse edge to fall out of agreement.

Where a predicate declares no inverse, reverse traversal across that predicate is refused rather than inferred. A relation whose inverse has not been declared is one whose reverse reading has not been asserted by anyone, and generating a reverse reading in that circumstance would be an assertion the system originated rather than retrieved. The refusal is reported to the caller as an absent inverse declaration, not as an absent relation.

Where a predicate declares the Transitivity boolean already specified in Appendix A, transitive closure is enforced by the Cross-Domain Rule Engine as already specified, and closure over the declared inverse is enforced on the same terms. Transitivity and inversion are independent declarations: a predicate may be transitive and have no inverse, or have an inverse and not be transitive.

Row 141 evaluates to 0.9405: an attribute of confidence 0.97 at its originating concept, reached across IS-A edges of confidence 0.99 and 0.95, yields 0.9405 — the product, which is below the bound and therefore governs. Row 142 evaluates the specificity ordering at depths 3 and 1. Row 143 evaluates to reverse traversal available, the inverse having been declared.

Cortical columns do not map to Fact Units

The submitted analysis maps cortical columns onto Fact Units. The Bible already maps them, at §41.4.7, onto World Brain concept node formation, on the ground that both observe many specific instances, extract stable invariant properties and build a context-independent model. A Fact Unit does none of those things: it encodes exactly one semantic assertion from one source, and it is deliberately atomic. The invariant-forming structure in QueryBook is the concept node in the Conceptual Memory Layer, and Chapter 58 keeps the two apart on purpose — the Semantic Brain stores what has

been said about reality, the World Brain stores how reality works. Adopting the submitted mapping would place an invariant-forming, multi-exposure structure inside the layer whose defining property is single-assertion atomicity. The existing mapping is both correct and already in the specification.

Spiking activation is not adopted

The submitted analysis proposes spiking neurons, burst firing and lateral inhibition as a mechanism for context-bounded traversal. The computational principle behind lateral inhibition — that simultaneously-represented items compete through mutual suppression, biased by relevance — is already implemented at §53.8 as Biased Competition, deterministically, with the trained implementation explicitly excluded under Chapter 101 and the underlying theory cited to Desimone and Duncan. Adding spiking dynamics on top would introduce a second, stochastic dynamical system into a retrieval path whose guarantee is that the same query over the same records returns the same answer. Chapter 101's third step directs extraction of the deterministic structural principle rather than adoption of the neural implementation. That extraction has already been performed for this principle.

Biological plausibility does not strengthen a patent claim

The submitted analysis states that biological grounding makes the claims legally stronger. It does not. Patentability turns on novelty, non-obviousness and enablement of a specific claimed mechanism. A claim is strengthened by reciting a concrete operation with a definite result — a bounded ratio, a refusal condition, a confidence computed by a stated rule — and is weakened by reciting a resemblance to biology, which adds no limitation an examiner can test and no boundary a competitor must design around.

The correct use of a neuroscience correspondence in this specification is the one Chapter 101 already prescribes and §53.8, §55.2.2, §59.9 and the criticality addendum already follow: verify the primary publication, extract the deterministic principle, exclude the trained or stochastic implementation, and name the correspondence. The correspondence explains why a mechanism is well-founded. The mechanism is what gets claimed. Claims 35 through 39 above recite inheritance, suppression, specificity ordering, tie refusal and inverse resolution — none of them mentions a neuron, and each is testable against an implementation.

Chapter 96: Forensic Analysis — §96.9 and §96.10 additions

96.9.1 The Probe Fact Unit

A ProbeResult Fact Unit extends §96.1's AnalyzedOutput pattern. Every instance requires: a Subject field identifying the specific external system and, where known, its version, exactly as §96.1 requires; a ProbeType field naming one of the probes at §96.9.2 through §96.9.4; a ProbeInput field containing the exact input or input family submitted; a ResultSet field containing every output received, retained in full and not summarised; a measured statistic computed under the probe's stated rule; and a ReferenceValue field containing QueryBook's own measured statistic on the identical probe. A ProbeResult lacking a ReferenceValue is incomplete and is not reportable, because a bare figure about another system is not a measurement, it is an assertion.

Probe inputs are submitted through the existing Source Connector Registry (§49.3.3) under a capability declared by the connected system, registered by an operator with sufficient privilege and not self-registered, on the same terms §96.4 already requires for live comparison. No new ingestion path is created. Outputs received are ingested through the Multimodal Ingestion Pipeline exactly as any other source content, and are held under §96.10.

96.9.2 Response consistency probe

An identical input is submitted to the subject system a configured number of times under conditions held constant so far as the connector's declared capability permits. The measured statistic is the proportion of returned outputs whose semantic fingerprints are identical, computed by the language-independent fingerprint of Chapter 3 rather than by string comparison, so that a difference in wording expressing the same assertion is not counted as an inconsistency.

Where fingerprints differ, the differing outputs are additionally checked against one another by the Cross-Domain Rule Engine's contradiction logic, and a contradiction between two outputs to the identical input is recorded as a distinct and more serious finding than mere variation. Variation without contradiction indicates a system that phrases differently; contradiction indicates a system that answers differently.

QueryBook's ReferenceValue on this probe is 1.0 by construction, and the specification records that this is a structural consequence of Claim 1 rather than a measured achievement. A deployment measuring anything other than 1.0 for itself on this probe has a defect, and §96.9.2 is therefore also a self-test of the determinism guarantee.

96.9.3 Self-consistency under output reinjection

The subject system's output is submitted back to it as input, under a fixed reinjection template, and the process is repeated to a configured iteration ceiling. At each iteration the semantic fingerprint of the output is recorded. The sequence is then classified into exactly one of four outcomes: converged, where fingerprints become identical and remain so; periodic, where a fingerprint sequence repeats with a fixed period; drifting, where each fingerprint differs from all preceding ones and a measured semantic distance from the initial output increases monotonically; and unresolved, where the ceiling is reached without any of the preceding classifications applying.

This probe is the diagnostic application of a question ordinarily asked of an architecture rather than of a system under test: whether repeated application of a transformation reaches a point at which further application changes nothing. A system that converges under reinjection is self-consistent in the specific sense that its own output is a stable input to it. A system that drifts is not, and the drift is measurable rather than impressionistic.

The classification is a property of the observed fingerprint sequence and nothing more. §96.9.3 generates no output asserting why a subject system converged, drifted or failed to resolve, and no output asserting anything about that system's internal mechanism, consistent with §96.3.5's existing constraint that QueryBook catalogues observed text and does not characterise another system's internal state.

96.9.4 Perturbation sensitivity probe

A family of inputs is constructed from a single base input by applying perturbations that do not alter the input's semantic fingerprint — reordering of independent clauses, substitution of a registered synonym, alteration of formatting, honorific or register. Because the fingerprint is computed over the canonicalized triple and is language-independent, membership in the family is verifiable rather than asserted: an input whose perturbation changes the fingerprint is not a member and is excluded from the family before submission.

The measured statistic is the proportion of the family's outputs sharing a common semantic fingerprint. A system retrieving from a grounded record set should be invariant across a family so constructed, because the family members are the same query. Sensitivity to a perturbation that provably did not change the query's meaning is a measurable property, and it is reported as that and not as a conclusion about how the subject system works.

96.9.5 Reporting constraints

Three constraints govern every use of §96.9, and each derives from a rule already in this Bible rather than from a new policy.

First, a probe result stated as a comparative performance claim is a performance claim within the meaning of Chapter 77 and requires that chapter's validation and benchmarking process before publication. §86.3 exists because an earlier overclaim was generalized from a narrow real mechanism and required correction; a probe statistic generalized into a claim about a named system's overall quality is the same failure in a new place.

Second, a ProbeResult concerning a system operated by an identified third party is subject to the same rights and terms governing access to that system. Probing at volume may fall outside a connector's declared capability or a service's terms of use, and §49.3.3's requirement that a capability be declared by the connected system and registered by a privileged operator is the control point at which that question is answered, before probing rather than after.

Third, no probe result and no aggregate of probe results is admitted as a grounding source for any answer, under §96.10.

An AnalyzedOutput Fact Unit under §96.1, a ProbeResult Fact Unit under §96.9.1, and any Fact Unit derived from either, are collectively analysis records. An analysis record is queryable as analysis and is structurally ineligible as a grounding source for an answer to any query that is not itself a query about the analysis.

The ineligibility is enforced at the point the Slice Resolver constructs the scope description under §51.2, not by filtering a result after traversal. A scope description constructed for an ordinary query excludes analysis records, so an analysis record is not traversable and cannot enter the admitted set, the stabilized set or the Narrative Graph. A scope description constructed for a query about analysis includes them and excludes nothing else that would otherwise be included.

The reason is the failure this architecture exists to prevent. An external system's assertion, once ingested as content, is a claim with a source and a confidence like any other. Without §96.10, an unsupported claim identified as unsupported by §96.2 would remain in the graph as a Fact Unit, available to ground a later answer. The system would then be capable of citing, as a fact, something it had itself already determined to be unfounded. §96.10 makes that path structurally unreachable rather than merely unlikely.

Three consequences follow and are stated so that the rule is not read more narrowly than intended. A synthesized assertion derived in part from an analysis record is itself an analysis record and inherits the quarantine. An analysis record is not admissible to the Conceptual Memory Layer, so no concept abstraction forms from an external

system's output. And an analysis record is excluded from the question index of the ingestion-time question generation mechanism, so no pre-computed answer is assembled from one.

A ProbeResult's ReferenceValue field is QueryBook's own measured statistic and is not an analysis record for the purpose of this section; it is a measurement of this system, retained under the ordinary provenance rules.

Row 148 evaluates a consistency probe in which eight of ten outputs shared a semantic fingerprint and a reference value was present, returning the measured branch; absence of the reference value returns incomplete. Row 149 evaluates the quarantine test on a ProbeResult record, returning exclusion from the grounding slice.

It creates no new ingestion mechanism, no new connector type and no new prediction mechanism. §96.9 uses the Source Connector Registry, the Multimodal Ingestion Pipeline, the Chapter 3 semantic fingerprint and the Chapter 17 rule engine, each already specified. §96.10 uses the Slice Resolver already specified at §51.2.

It does not authorise probing of any particular system. Whether a given system may be probed, at what volume and under what terms, is determined at connector registration under §49.3.3 and by the terms governing access to that system, and this addendum deliberately supplies no default permitting it.

It generates no characterization of an external system's internal mechanism, its reasoning or its internal experience. Every probe output is a statistic over observed fingerprints, and §96.3.5's existing constraint applies unchanged: what is catalogued is what was observed, and a citation of observed text is not a determination about the system that produced it.

Chapter 97: Hypothesis Generation — §97.8 addition

97.8.1 The undiscriminated condition

Two hypotheses are undiscriminated where each is supported by the available knowledge records and the difference between their posterior probabilities does not exceed a configured separation floor. Gathering further evidence of the kind already held does not resolve this condition and may worsen it: evidence consistent with both hypotheses raises both, and the pair remains as close together as before while each appears better supported.

By way of example, a first hypothesis at $\alpha = 12$, $\beta = 4$ carries a posterior of 0.7500 and a second at $\alpha = 11$, $\beta = 5$ carries 0.6875. The separation is 0.0625, below a floor of

0.15, and each rests on sixteen observations. Neither is weakly held; they are simply not distinguished. This is the condition §97.8 detects.

Detection is automatic and continuous over the hypothesis set: whenever two hypotheses concerning a common subject fall within the separation floor and each exceeds a minimum support threshold, the pair is marked undiscriminated and §97.8.2 is invoked. A pair in which one hypothesis is weakly supported is not undiscriminated but merely under-evidenced, and is handled by the existing evidence-seeking mechanism rather than by this one.

97.8.2 Selecting the discriminating instance

For each candidate observation that could be obtained — a knowledge record whose ingestion is possible, a measurement that could be taken, a source that could be consulted — the system computes the likelihood of that observation under each of the two hypotheses, and scores the candidate by the magnitude of the difference.

```
discrimination(E) = | P(E | H1) − P(E | H2) |
```

The candidate maximising that quantity is the discriminating instance. It is the observation whose outcome most changes the relative standing of the two hypotheses, and it is selected irrespective of how likely either hypothesis considers it. A candidate that both hypotheses predict equally, however probable, discriminates nothing and is not selected.

A discriminating instance is generated only where the best available candidate's discrimination satisfies a configured floor. Where no candidate separates the hypotheses sufficiently, no instance is generated and the pair is reported as undiscriminated with the reason recorded, rather than a weakly-discriminating observation being proposed as though it would settle the matter.

Both the presence and the absence of the selected observation are informative, and the specification records this because it is the property that makes the mechanism worth having. Observing E2 in the worked example moves the first hypothesis to 0.7576 and the second to 0.6588; failing to observe it moves them in the opposite directions. The instance is therefore reported with both outcomes stated in advance, so that the party who obtains it knows before looking what each result would mean — which is the difference between a test and a search for confirmation.

97.8.3 Output and the human gate

A discriminating instance is emitted as a structured request identifying: the two hypotheses, their current posteriors and support counts, the observation sought, its computed discrimination, and the posterior each hypothesis would take under each

outcome. It is presented to a human, exactly as an evidence-seeking query is presented, and no automated process acts upon it.

The mechanism proposes an observation; it does not obtain one. Where obtaining the observation would require an operation on an external system, that operation is subject to the connector capability declaration and to the reporting constraints governing probing, and no discriminating instance authorises an operation that those mechanisms would not authorise independently.

Neither hypothesis is advanced or retired by the generation of a discriminating instance. Both remain in PROPOSED status, and a transition to VALIDATED continues to require confirmation by an identified human reviewer under Chapter 97's existing gate, which no automated process may effect. §97.8 changes what a reviewer is shown, not who decides.

Where the observation is subsequently obtained and ingested as a knowledge record on the ordinary terms, the posteriors of both hypotheses update under the conjugate rule already specified, and the undiscriminated condition is re-evaluated. A pair that remains undiscriminated after its discriminating instance has been obtained is reported as such, and the outcome is recorded so that the same instance is not proposed again.

97.8.4 Position in the reasoning architecture

The architecture's governing rule is unchanged by this section and is restated because §97.8 sits precisely on the boundary it draws: deduction may write to the knowledge store, and induction may not. What §97.8 adds is that when induction cannot decide between two candidates, the system says which observation would decide, rather than accumulating further evidence that decides nothing.

Row 179 evaluates the separation between two hypotheses at posteriors 0.7500 and 0.6875 against a floor of 0.15, returning the undiscriminated branch. Row 180 evaluates the discrimination of three candidate observations and returns the maximum, 0.68, which identifies the selected instance.

Chapter 103: Temporal Prediction Calibration Layer

103.1 Purpose

The Predictive Cognitive Architecture treats prediction error as a signal: contradiction detection is characterised as prediction error, and hierarchical error propagation is one of its twelve organised mechanisms. A signal that is never measured cannot calibrate anything. This chapter supplies the measurement, by reconstructing the state of

knowledge available before a recorded event, running this system's own forecasting mechanisms against that reconstructed state, and scoring the resulting forecast against what the record shows actually occurred.

The output is a calibration record, not a prediction service. Chapter 103 produces statements of the form: over a stated corpus, at a stated horizon, within a stated domain, forecasts of this system carried a measured error of a stated magnitude. It produces no forecast about any future event, and nothing in this chapter is a path by which a calibration figure becomes a prediction delivered to a reader.

103.2 Three distinct times

Every Fact Unit already carries two times that are commonly conflated and are not the same. Chapter 103 depends on separating them, and introduces a third.

The distinction is the whole of the matter. A commentary written in 1975 about a decision taken in 1962 carries T_applicable in 1962 and T_attested in 1975. It describes the pre-event period accurately and could not have been available before the event. Admitting it to a pre-event snapshot on the strength of its TemporalAnchor would be admitting hindsight.

103.3 The Attestation Horizon Constraint

This is the chapter's core rule. Let an event record carry an event time T_e, and let a retrospective forecast be run at a horizon H before that event. A Fact Unit is admissible to the pre-event snapshot for that run if and only if its T_attested is strictly earlier than T_e minus H. Its T_applicable is not consulted for this determination, and satisfying T_applicable is neither necessary nor sufficient for admission.

A Fact Unit whose T_attested cannot be established from documented source metadata is excluded. It is not admitted under an assumption, an estimate, or the ingestion date as a proxy. The reason is that the failure mode this constraint exists to prevent is silent: an inadmissible record admitted by assumption produces a forecast that appears accurate and is in fact a restatement of the outcome, and no downstream statistic reveals it. Exclusion on unknown attestation is therefore the safe default and the specified behaviour.

Enforcement is at slice construction, not by filtering after retrieval. The Slice Resolver constructs a signed, time-bounded scope description before traversal begins (§51.2), and for a retrospective run that scope description is bounded by the attestation cutoff T_e minus H. An inadmissible Fact Unit is therefore not traversable during the run rather than removed from its result, which means it cannot influence retrieval ordering, candidate counts, Attractor Network convergence or the permitted generation vocabulary. A post-hoc filter would leave all four contaminated.

Where a source's documented date is itself contested, the latest defensible date is used, so that uncertainty in attestation resolves toward exclusion rather than toward admission. Every admission and every exclusion is recorded in the Provenance Ledger with the T_attested value relied upon, so that a calibration result can be re-derived and audited against the exact record set that produced it.

A conventional retrospective study cannot enforce this rule, because a spreadsheet of historical variables carries no per-assertion attestation and no mechanism to make an inadmissible assertion unreachable. The constraint is available here because the Fact Unit already binds provenance inseparably to content, and it is for that reason a property of this architecture rather than a methodological preference.

103.4 Horizon slicing and the retrospective run

For a given event record, a configured set of horizons is applied — by way of example and not limitation, a far, a mid and a near horizon expressed in the temporal granularity appropriate to the domain. For each horizon a pre-event snapshot is constructed under §103.3, and the forecasting mechanisms of Chapter 10 and §10.4 are executed against that snapshot alone, with no other Fact Unit traversable.

Each run records a predicted outcome, a probability assigned to it, a predicted interval to occurrence, and the identifiers of every Fact Unit admitted to the snapshot. The last of these is not optional: a calibration result whose input record set cannot be enumerated is not reproducible and is not reportable.

A run is executed in a Scenario Graph branch under Chapter 11 and its outputs are delta-represented against the base state. No output of a retrospective run modifies a Fact Unit, and no predicted outcome is written into the Fact Graph as an assertion. This follows §10.4's existing rule that a forecast is rendered in a labelled region distinct from grounded output and is never incorporated into a knowledge record.

103.5 The Prediction Error Record

Comparison of a run's predicted outcome against the recorded outcome produces a PredictionError Fact Unit carrying: the event reference; the horizon; the predicted outcome and its assigned probability; the recorded outcome; a squared-error term computed as the square of the difference between the assigned probability and the realised binary outcome; and the domain and subject class of the event.

Aggregated across a corpus, the mean of those squared-error terms is a proper scoring measure of calibration: a forecast assigned probability 0.7 to an outcome that occurred contributes 0.09, and one assigned 0.4 to an outcome that did not occur contributes 0.16, for a mean of 0.125 across those two. The measure is stated here because it is what permits a claim about calibration to be checked rather than asserted.

Two derived quantities are retained. Accuracy by horizon, within a domain, states how error varies as the forecast moves further from the event. Calibration by confidence band states whether forecasts assigned a given probability occurred at approximately that rate — which is the question §10.4's confidence intervals implicitly answer and currently cannot.

103.6 Subject class constraint

The subject of an event record admitted to this chapter is an institution, a system, an organisational process or an aggregate group. An individual natural person is not an admissible subject, and a PredictionError Fact Unit whose subject resolves to an individual natural person is refused at construction rather than retained with a restriction.

The constraint is structural for the same reason §91's exclusion of person and animal classifications is structural: a per-individual predictive score, once computed and stored, is available to be acted upon irrespective of the caution attached to it, and the categories where such scores have caused documented harm — criminal justice determinations, credit and insurance decisions, employment screening — are precisely those where a retrospective corpus is easiest to assemble. Domain 0's harm-prevention authority applies to Fact Graph content, and this chapter records the application rather than relying on it being inferred.

Measuring the predictability of an institutional process is not thereby prohibited and is the intended use: how predictably a court, a hospital admissions process or a school system behaves is a property of the institution. A finding that a demographic attribute correlates with an institutional outcome is reported as a property of that institution's process, cited to the records establishing it, and is not restated as a property of the individuals to whom the attribute applies.

103.7 Retention

A derived calibration statistic — error by horizon, by domain, by confidence band — is an aggregate over a corpus and is retained without expiry. An event record and a PredictionError Fact Unit concerning an identifiable subject are retained under §36.7's ordinary policy and are subject to its erasure operations at session, work and account scope on the same terms as any other record.

This division is deliberate and is the answer to the proposal from which this chapter derives, which described a perpetual record of human behaviour. A permanent, non-erasable archive of behavioural event records is the exposure Chapter 102 exists to avoid, and it is not created here. What is perpetual is the calibration curve; what is erasable is the record of any person.

103.8 Reporting

A calibration figure published outside the operator's own organisation is a performance claim within Chapter 77 and requires that chapter's validation process, including a stated corpus, a stated horizon set and the attestation cutoff applied. A figure stated without its attestation cutoff is not interpretable, because the cutoff determines what the forecast was permitted to know, and two figures computed under different cutoffs are not comparable.

Where a domain's measured error is high, that finding is reported with the same standing as a finding of low error. A chapter that reports only its successes calibrates nothing, and the domains where this system predicts poorly are the operationally useful output — they identify where a forecast should not be offered to a reader at all.

Row 150 evaluates a source attested 1 March 1998 against an event of 11 September 2001 at a 90-day horizon, returning admissible; the same expression with an attestation date of 1 June 2003 returns excluded. Row 151 evaluates the mean squared error of two forecasts, returning 0.125. Row 152 evaluates the subject class gate against the admissible classes, returning admissible.

The chapter adds no new forecasting mechanism, no new ingestion path and no new storage tier. It uses the Fact Unit's existing temporal anchor and provenance binding, the Slice Resolver of §51.2, the forecasting of Chapter 10 and §10.4, the Scenario Graph of Chapter 11, and the retention policy of §36.7. Its contribution is the constraint at §103.3 and the measurement that constraint makes trustworthy.

Chapter 106: Domain Architecture

106.1 Structure

The system is partitioned into fifteen domains. Fourteen are horizontal: each owns a bounded set of mechanisms, holds its own persistent state where it has any, and exposes a defined interface to the others. One is vertical: Domain 0, the Prime Directive, penetrates every horizontal domain and every layer within them.

A domain is an authority boundary, not a deployment unit. Two domains may execute in one process and one domain may span several; what the partition fixes is which mechanism owns which decision, which transfers between domains are permitted, and how compute is accounted. The partition is therefore stable across the cloud, local and single-device deployment modes, and a deployment omitting an optional domain omits its capability without altering any other domain's behaviour.

Two properties govern the partition, and both are consequences of where the boundaries were drawn rather than rules added afterward. The first is path separation,

specified at §106.3, which is a performance property. The second is boundary enforcement, specified at §106.4, which is an accuracy property. Neither is achievable by a partition drawn along subject-matter lines alone, which is why the domains below do not simply mirror the chapter structure.

106.2 The domains

Three features are assigned by function rather than by source chapter. The Safety Substrate is assigned to D0, because its enforcement plane is the mechanism by which the Prime Directive is vertical rather than merely stated. Deployment architecture is assigned to D11, because staged rollout is an act of changing the system. Human reasoning and motivation modelling is assigned to D5, because it models a documented subject as an inference over records — and it is retained under the Chapter 80 constraint that no such model is ever constructed of the system's own state, which is why the superseded architecture's self-model and metacognition domains are not reinstated here.

106.3 Path separation — the performance property

The partition is drawn so that the latency-critical path and the batch path share as few domains as possible. Answering a query traverses D8, then D4, optionally D5, then D7, under D0 throughout. Ingesting content traverses D1, then D2, then D3, then D6, with D5 performing pre-computation, under D0 throughout.

The consequence is the design's central performance property: an ingestion job of any size contends with an interactive query at exactly two points, and at both the contention is bounded by mechanism already specified. At D0 the contention is resolved by category, because a safety-evaluation slice is served ahead of every other category irrespective of arrival order. At D2 it is resolved by storage tiering, because a batch write targets a different tier from an interactive read of a frequently-accessed record.

Two corollaries follow and are stated because they constrain implementation. D4 performs no ingestion work of any kind, so a retrieval never blocks on an extractor; and D1 holds no retrieval authority, so an extraction never blocks on the Attractor Network's convergence. A partition placing extraction and retrieval in one domain — as a partition drawn along subject-matter lines would, since both concern knowledge records — would couple the two and forfeit this property.

The pre-computation performed by D5 at ingestion is the mechanism by which work is deliberately moved from the hot path to the batch path: forward inference chains and probable questions with their answers are computed when the corpus is admitted, so that a query matching a pre-computed answer is served without traversing D4's stabilization at all. The partition makes that transfer expressible as a property of the architecture rather than as a caching optimisation.

106.4 The transfer matrix — the accuracy property

Every ordered pair of domains carries a transfer permission: ALLOW, DENY, or TRANSFORM-REQUIRED, the last meaning that a transfer is permitted only through a stated conversion that types, gates or marks the transferred content. The matrix is data, evaluated at each boundary crossing, and a crossing with no matrix entry is treated as DENY.

The matrix is owned by D0 and is a member of the permanently-excluded subsystem set. This placement is not administrative tidiness. The matrix contains the entry denying D11 a transfer to D0, and that entry is what prevents the mechanism that changes the system from amending the Prime Directive. Were the matrix owned by D11 as platform configuration — which is where a partition drawn by subject matter would place it — D11 could propose a change to that entry, and the exclusion protecting D0 would be administered by the party it excludes. The path-separation invariant of §106.3 and the slice priority ordering of §106.5 are owned on the same terms and for the same reason.

Most entries are unremarkable. The entries that matter are the denials, because each one is an accuracy guarantee expressed as a structural impossibility rather than as a rule someone must remember.

Two general rules complete the matrix. Any domain may read from D3, because ontology is shared meaning and restricting it would fragment the concept structure the poly-hierarchy exists to keep unified; but only D3 may write to it, and a domain requiring a new concept requests registration rather than creating one. And every crossing is recorded in the provenance chain with the source domain, the target domain and the permission applied, so a transfer that should not have occurred is detectable after the fact rather than only preventable before it.

106.5 Compute accounting by domain

Each domain's operations are metered as bounded compute-accounting units carrying that domain's identifier, and scheduling priority is fixed by domain rather than negotiated. D0 holds the highest priority without exception. D13 holds the lowest, so that contention omits the commercial region rather than delaying the answer. The remaining domains are ordered so that a domain on the latency-critical path outranks a domain on the batch path, which makes the path separation of §106.3 an operational property and not merely a topological one.

A domain's failure is contained to that domain's capability. A failed D13 slice omits commercial content; a failed D12 slice leaves the connector unavailable and the core unaffected, which is the sense in which an external resource is an option rather than a dependency; a failed D6 slice returns no world-model completion rather than a partial one. A failed D0 slice halts the operation entirely, because an operation that cannot be

evaluated for harm is not permitted to proceed on the ground that evaluation was unavailable.

106.6 Layers and modules

Two further groupings organise the domains, and neither carries authority. A layer is a grouping of domains chosen for deployment, ownership and failure grouping; a module is a replaceable implementation within a domain. Only a domain owns a decision, carries a transfer permission and holds a compute slice. Moving a domain between layers changes nothing the transfer matrix governs, and this is stated explicitly because the superseded architecture's five-layer model was read as a permission model and could not function as one.

The layer assignment was chosen by measurement rather than by subject matter. Three candidate groupings were evaluated against the transfer edges the specified pipelines actually require, scored by the proportion of transfers remaining within a layer, since a transfer crossing a layer boundary is the one that constrains deployment topology and team ownership. A path-aligned grouping retained 36 per cent, a subject-aligned grouping 28 per cent, and the grouping adopted below 44 per cent across the fewest crossing pairs.

A module is the unit of replacement: an implementation exchangeable where it satisfies every interface the replaced unit exposed at each supported version and every transfer-matrix entry in which its domain appears. Two domains declare none. D0 is excluded from replacement by membership of the permanently-excluded set. D2 holds the record store, and exchanging its implementation is a migration of stored records rather than a module exchange, for which interface and matrix conformance are necessary but not sufficient.

Row 176 evaluates a transfer whose matrix entry is not among the permitting values, returning refusal. Row 177 evaluates the intersection of the query-path and ingestion-path domain sets against the design bound of two. Row 178 evaluates the priority ordering of the governance domain against the commerce domain.

The five horizontal layers of the superseded architecture — Presentation, Application, Cognitive, Data, Infrastructure — are not reinstated. A layer model and a domain model describe the same system twice, and where they disagree neither governs. This chapter keeps the domain model, because a domain can carry a transfer permission, a compute slice and a failure containment boundary, and a layer cannot.

Chapter 104

104.1 The problem this chapter solves

The Fact Unit already carries a FactType resolving to the Ontology Mesh, and the type set already includes EVENT_DESCRIPTION, TEMPORAL_STATEMENT and CAUSAL_CLAIM. What it does not carry is a governed definition of what any particular kind of event requires. Two ingestion runs over two corpora can both produce EVENT_DESCRIPTION Fact Units in which the same real-world category of occurrence is represented with different fields present, different granularity and different implicit assumptions, and nothing in the current specification detects that they are not comparable.

The consequence is felt wherever events are compared rather than merely retrieved: edition comparison across sources, the contradiction index when two sources describe one occurrence, the Scenario Graph when a branch turns on an event's properties, and Chapter 103's calibration corpus, which is only meaningful if the events it scores are typed consistently. An event vocabulary that is agreed per corpus rather than per system is not a vocabulary.

The second gap is contextual. A great many assertions are true only relative to a state of the world at the time — a measurement under stated conditions, an outcome under a stated policy regime, a behaviour under stated constraints. That state is presently either lost at extraction or duplicated in prose across every Fact Unit that depends on it. Duplication is the worse outcome of the two: a correction to the shared context must then be applied to every copy, and any copy missed becomes a silent inconsistency.

104.2 The Event Type Catalogue

An Event Type is a governed, versioned definition of a category of occurrence. It is registered in a catalogue that operates on the same terms as the predicate catalogue of Chapter 6: an identifier assigned at registration and immutable thereafter, a resolution against the Ontology Mesh, and deprecation rather than deletion.

Three rules govern use. First, an EVENT_DESCRIPTION Fact Unit must reference a registered EventTypeId; free-text event categorisation is not permitted, for the same reason predicates resolve against a governed catalogue rather than being coined at ingestion. Second, the MachineDefinition is evaluated at construction, so a type requiring a temporal anchor of at least DAY granularity refuses a record carrying only ERA granularity — the record is not silently admitted with a weak anchor that later analysis will treat as strong. Third, a type's required-field set may be narrowed by a later version but not widened without a new type, because widening would retroactively render conforming historical records non-conforming.

The enablement value is worth stating plainly, because it is why this belongs in the application rather than only in an implementation. A MachineDefinition is testable: given a type and a candidate record, conformance is decidable by inspection, and an implementation either refuses the non-conforming record or does not. That is a concrete operation with a definite result, which is what a claim needs and what a prose event taxonomy cannot supply.

104.3 Environment Vectors

An Environment Vector is a normalized snapshot of the state of the world relative to which a set of assertions holds. It is a first-class record with its own provenance and its own confidence, because a stated environment is itself an observation that may be wrong, and treating it as unquestioned metadata would let an unsupported context silently qualify every fact referencing it.

A Fact Unit references an Environment Vector through its AnnotationSet under a reserved key. This placement is deliberate and is what keeps the schema guarantee at the head of this chapter true: the reference is an annotation, the annotation is optional, and a Fact Unit with no environmental dependency carries none. No required field is added to the Fact Unit, and every existing record remains valid unmodified.

The correction case is the one that repays the normalization. Where a hundred Fact Units reference one Environment Vector and the vector is found to be wrong, the vector is superseded once through the ordinary supersession chain and every referencing record resolves to the corrected state on next traversal. Where the same context had been duplicated into a hundred records, the correction is a hundred edits, and the ones that are missed do not announce themselves.

104.4 Causal chains through existing machinery

Causal structure is expressed with mechanisms already specified, and this chapter deliberately introduces no parallel edge store. The CAUSES predicate is already registered — the source Fact Unit describes a causal factor for the state or event in the target — and TEMPORAL_FOLLOWS already carries temporal ordering derived from TemporalAnchors. Causal edges are RelationSet edges, carrying the weight and typing every other edge carries.

A chain is a Bundle. BundleRefs already provides many-to-many membership with a bounded ceiling, so a Fact Unit participating in several causal chains is expressed without a new field, and a chain is queryable through QBQL as the set of Fact Units sharing a BundleID together with the CAUSES edges among them. Chain length, root and outcome are computed from that set at query time rather than stored as chain metadata that can fall out of agreement with the edges it summarises.

One validation rule is added, and it is the only structural constraint this chapter places on causation. A CAUSES edge is admitted only where the temporal anchor of the source does not begin after the temporal anchor of the target begins. An edge asserting that an effect preceded its cause is refused at construction and the attempted assertion is recorded in the contradiction index of Chapter 17, so that the underlying disagreement between sources remains queryable rather than being discarded with the edge.

Two things this rule deliberately does not do. It does not require strict temporal precedence, so concurrent and overlapping causation remain expressible — a condition that persists while producing its effect is common in every domain the system serves. And it does not treat temporal order as evidence of causation: TEMPORAL_FOLLOWS is not upgraded to CAUSES by any automatic process, because sequence is not causation and a system that inferred it would be generating assertions no source made.

A derived quantity is computed at query time and not stored. An event's duration is the difference of its temporal anchors and is recomputed on demand; storing it would create a field that can disagree with the anchors it derives from, which is the precise redundancy the normalization discipline of this chapter exists to eliminate.

104.5 Excluded by design

Two constructs proposed alongside the above are excluded, and the exclusions are recorded here so that they read as decisions rather than oversights.

A scalar inevitability or likelihood score attached to an event record is not adopted. Attached to an event carrying actor references it is a per-actor predictive score, and §103.6 refuses exactly that at construction rather than by restriction of access. Where the likelihood of an outcome is genuinely required it is available as a Scenario Graph branch probability under Chapter 11, which is delta-represented, pruned at a threshold, and never written back into a Fact Unit as an assertion.

An undefined force or influence vector is not adopted. A quantitative field with no stated units, no computation rule and no worked expression cannot be implemented consistently by two engineers, cannot be tested for conformance, and cannot support a definite claim. Where directional influence is required it is expressible as a weighted CAUSES edge, whose weight is bounded in [0,1] and whose meaning is already specified for every other edge in the Fact Graph.

Chapter 105

105.1 The public application programming interface layer

Seven interfaces are exposed to external callers: a content interface returning work structure and rendered content; a query interface accepting a natural-language or structured query and returning a grounded response with citations; a rights interface reporting entitlements; a layout interface returning pagination and rendering geometry; an assessment interface returning generated assessment items and accepting submitted responses; an edition interface returning edition membership and difference sets; and a semantic interface returning Fact Units, concept nodes and edges.

Every interface resolves a scope description before performing any traversal, exactly as an internal query does. There is no interface that returns a Fact Unit outside the caller's scope description, and no interface parameter that widens a scope description; a caller requesting a record outside its scope receives the same response it would receive for a record that does not exist, because the record is not traversable rather than filtered from a result. The application programming interface layer is itself an operation layer within the meaning of Chapter 8 and carries its own synchronous safety evaluation gate, so that entry through the interface does not avoid a gate positioned at another layer.

The rights interface reports entitlements and does not grant, modify or extend one. Every write-capable interface — the assessment interface accepting a submitted response, and the content interface accepting a manuscript — admits its input through the ordinary ingestion path with the ordinary safety evaluation, and none accepts a Fact Unit directly.

Each interface call is metered as a bounded compute-accounting unit assigned to a category and to the calling identity, and a call exceeding the budget configured for that identity is refused rather than served at degraded priority. Refusal is by budget, not by throttling, so that a caller cannot displace a safety-evaluation unit by volume.

105.2 Plugin architecture

A plugin is a third-party component registered against a declared capability set, on the same contract the Source Connector Registry already applies to an external system. A plugin declares the interfaces it consumes and the surface it contributes, and may perform only what it has declared.

Three prohibitions are structural rather than configured. A plugin holds no direct access to the Fact Graph, the Provenance Ledger or the Conceptual Memory Layer, and reaches knowledge only through the interfaces of §105.1, within the scope description

of the identity on whose behalf it runs. A plugin cannot register, modify or suspend a safety evaluation, and a plugin whose declared capability set includes any such operation is refused at registration. A plugin is not a valid target of, and cannot participate in, the improvement proposal mechanism, and the permanently-excluded subsystem set is not reachable by a plugin under any capability declaration.

A plugin's output is presented in a region distinct from grounded output and labelled with the plugin's identity, and is not admitted to the Fact Graph. A plugin contributing a presentation surface does not thereby contribute an assertion: content originating from a plugin is not citable as grounding and is quarantined on the same terms as an analysis record.

Activation requires approval by an identified human operator, recorded with that operator's identity and stated reasoning, and a plugin may be suspended by the same operator at any time. Suspension is immediate and does not require the plugin's cooperation.

105.3 Interface versioning

Each interface of §105.1 carries a version. A caller requests a version explicitly; a request omitting a version is refused rather than defaulted, so that a caller's behaviour cannot change because a default moved.

A version is deprecated, never deleted. A deprecated version continues to serve for a configured support interval and returns a deprecation indication with each response; on expiry of that interval the version is retired and a request for it is refused with an indication naming the successor. A Fact Unit returned under one version remains valid: versioning governs the shape of a response, never the validity of a record, and no version change invalidates a citation issued under an earlier version.

A version may narrow a response shape only by adding an optional field or by deprecating an existing one across a support interval. A change removing a field, changing a field's type, or changing the meaning of a value requires a new version, on the same reasoning that governs event type versioning: a caller conforming to a version must remain conforming for the life of that version.

105.4 Collaborative reading

Two or more readers occupy a shared session over a common work, each retaining their own identity, entitlement and reading position. The shared session presents a shared view, and the constraint governing it is the one that makes the feature safe: the shared view is the intersection of the participants' scope descriptions, not the union and not the scope of the participant who initiated the session.

A record traversable by a first participant and not by a second is absent from the shared view entirely, and its absence is not reported to either participant, because reporting the existence of a withheld record would leak precisely what the scope description exists to protect. Where the intersection is empty the session yields no shared content and states that boundary. By way of example, a participant whose scope admits 340 records sharing a session with one whose scope admits 275 of those records yields a shared view of 275.

An annotation authored in a shared session is a reader-authored record carrying the authoring participant's identity, and its visibility is governed by its own access control list, which the shared session does not widen. A participant's reading position, history and personal notes remain outside the shared view unless that participant explicitly contributes them.

A query submitted in a shared session is resolved against the intersected scope and its response is delivered identically to every participant, so that no participant receives a citation to a record another participant cannot open.

105.5 Augmented-reality overlay presentation

An augmented-reality overlay anchors displayed content to a physical object identified in a live camera feed. It is a presentation surface only: the overlay displays Fact Units retrieved through the ordinary path with their citations, and no assertion is generated for display that would not be generated for a textual response. The vocabulary constraint applies unchanged.

The camera feed is analysed under the constraints already governing sensor analysis. Classification results corresponding to a natural person or to an animal are excluded from the candidate set entirely and no overlay is anchored to one, at any confidence and in any configuration. Object classifications are session-scoped and are not retained beyond the session absent a separate explicit election.

No gaze direction, dwell time, head pose or attention signal is captured, derived or retained, and no such signal is available to the commerce layer, to plugin components or to any analytics surface. This restates for augmented reality the restriction already placed on virtual-reality commerce presentation: extending the system to a new presentation surface does not create a new targeting signal.

Where an object cannot be identified with sufficient confidence, no overlay is displayed and the boundary is stated. An overlay is never anchored speculatively to an unidentified object.

105.6 Voice navigation

Voice navigation accepts spoken input as command and query input, and is distinct from the consent-gated prosodic analysis specified elsewhere: navigation consumes the words, that analysis consumes the signal, and neither implies the other. Enabling navigation does not enable analysis, and a deployment may run either alone.

Transcribed input is routed through the Language Command Router, which separates query intent from operational intent, so that a spoken instruction concerning how future answers should be phrased is applied as an instruction and a spoken question is answered as a question. Where intent classification does not clear its confidence threshold the system requests clarification rather than selecting an interpretation, because a misrouted spoken instruction is silently destructive in a way a misrouted typed one is not.

Voice capture requires an explicit session-scoped consent, revocable at any time, and audio is discarded after transcription unless a separate explicit election retains it. No voiceprint, speaker embedding or biometric identifier is derived, stored or used for identification, and speaker identity is taken from the authenticated session rather than from the voice.

A spoken instruction carrying a state-changing effect — erasure, a rights change, a purchase, a grant — requires a confirmation step through a second modality before it takes effect.

105.7 Spoiler protection

A reader's position in a work is recorded as a progress marker. A Fact Unit extracted from the work carries the narrative position at which its content is disclosed, derived from its location in the work's structure.

Where spoiler protection is active for a reader, the scope description constructed for that reader's query over that work is bounded by the reader's progress marker: a Fact Unit whose narrative position exceeds the marker is not traversable. Enforcement is therefore at scope construction and not by masking a rendered answer, which matters because a masked answer still discloses that something exists, how long it is, and where it sits.

By way of example, a Fact Unit disclosed at chapter 14 is traversable for a reader whose marker stands at chapter 22 and is not traversable for a reader whose marker stands at chapter 9. Where a query cannot be answered within the bounded scope, the system states that the answer lies beyond the reader's current position — which discloses the existence of a boundary, not its content — and offers to proceed on an explicit election by the reader.

Spoiler protection is a reader election, defaulting according to a setting the work's author may configure, and is never applied to an instructor, researcher or author acting in their own capacity over their own work. Progress markers are reader-held records subject to the ordinary retention and erasure operations.

105.8 Learner progress tracking

A learner's engagement with a work is recorded as progress records comprising position reached, assessment items attempted, responses submitted and outcomes scored against the Fact Units from which the items were generated. A mastery measure over a stated objective is the proportion of items attributable to that objective answered correctly, against a configured threshold; by way of example, seventeen of twenty against a threshold of 0.8 meets the objective.

Three constraints govern the records. Instructor visibility is grant-scoped: an instructor sees the progress of learners within a class grant they hold, for the interval of that grant, and grant expiry removes visibility without further action. Progress records are learner-held for retention purposes and are subject to the erasure operations at session, work and account scope on the ordinary terms.

The third constraint is the substantive one. A progress record states what a learner did and how it was scored. No inference of aptitude, likelihood of future performance or comparative ranking against other learners is generated or stored, and a progress record is not admissible as the subject of the calibration mechanism, whose subject class is restricted to institutions and organisational processes. Measuring how predictably a course produces an outcome is available; producing a predictive score attached to a learner is not.

105.9 Learning management system integration

A learning management system is connected as an external system through the Source Connector Registry under a declared capability set, and no operation is performed that is not within a declared capability. Typical capabilities are roster read, assignment read, grade write and single-sign-on assertion; a connector declaring roster read alone cannot write a grade.

A roster read establishes class membership and thereby the scope of an instructor grant; it does not itself confer entitlement to a work, which remains governed by the licence and entitlement mechanism. A learner appearing on a roster without an entitlement receives the response for absent entitlement, not the content.

A grade write transmits an outcome computed within the system and no underlying Fact Unit, response text or reading history, so that integration does not become an export path for content or for a learner's record. Every transmission is recorded in the

Provenance Ledger with the connector identity, the capability exercised and the fields transmitted.

Where the external system's terms or the operator's configuration do not permit a capability, the connector is registered without it and the corresponding operation is refused at the boundary rather than attempted and failed.

105.10 Multi-edition print layout

A print layout engine composes a work into a paginated, print-ready form for a specified edition. Editions differing in audience level, language, annotation density or accessibility treatment are composed from the same underlying Fact Units through the ordinary regeneration path, so an edition is a rendering of the work rather than a separate copy of it.

Composition is rights-aware and the rights check precedes composition rather than gating the export at the end: an edition whose rights validation does not permit print reproduction in the requested territory, for the requested quantity or within the requested interval, is not composed. A composed layout carries a reference to the rights validation that permitted it, so a print run is traceable to the authority under which it was produced.

A wording-operative span is reproduced verbatim in print in its source language, with any regenerated gloss carrying its non-operative marking in the printed form as it does on screen. Accessible-equivalent editions are composed at the same production stage as the edition they accompany, and a production lacking one is reported incomplete.

Pagination, hyphenation and line breaking are deterministic for a given edition, layout specification and content state, so that two composition runs over an unchanged work produce identical output and a reported page reference remains valid.

105.11 Certification

A certification is a credential issued to an identified learner on satisfaction of a stated requirement, comprising assessment outcomes against stated objectives and any completion criteria the issuer configures. Issuance requires both that the measured outcome meet the configured threshold and that the learner's identity be established to the standard the issuer configures; neither alone suffices.

A credential is a record carrying its own provenance: the objectives assessed, the items presented, the Fact Units from which those items were generated, the thresholds applied and the issuing identity. A credential is verifiable against that record by a third party to whom the holder presents it, and verification reports whether the credential was issued and remains unrevoked, without disclosing the holder's underlying responses.

A credential attests to a measured outcome against stated objectives and to nothing further. No credential asserts an aptitude, a competence beyond the objectives

assessed, or a prediction of future performance. A credential may be revoked by its issuer, and revocation is recorded rather than effected by deletion, so that a previously-verified credential's history remains auditable.

105.12 The rights and licensing portal

A portal surface exposes the licence and entitlement mechanism to the roles that administer it, under the existing role model and its time-scoped grants. What a role may do at the portal is what that role may do through any other surface: the portal renders capability, it does not confer it.

A rights holder views the licences granted over their works, the entitlements outstanding under each, and the audit record of grants, exercises and revocations. An operator administers organisational licences, seat counts and entitlement assignment within the scope their role permits. A grantee views the grants held over them read-only, on the same terms as the existing instructor grant visibility surface, and holds no capability to create or modify a grant from that view.

Every action taken through the portal is recorded in the Provenance Ledger with the acting identity, the action and its stated reason, and a revocation takes effect at the query execution layer, so an entitlement withdrawn is unavailable at the next query rather than at the next session.

Chapter 107

107.1 Deployment topologies

Chapter 90 specifies cloud, local and single-device modes. Two further topologies are specified here, and the property common to all five is stated first: the domain partition of Chapter 106 is invariant across topology. A topology determines where a domain executes, never which domain owns a decision, so a guarantee established by a transfer-matrix denial holds identically in every deployment.

An edge node's cache is a copy, not a branch: it is invalidated by supersession of any record it holds, on the same dependency relation that invalidates a pre-computed answer. A node unable to verify currency escalates rather than serving, so a stale edge answers nothing rather than answering wrongly.

107.2 Module replacement

A domain's implementation may be replaced without altering any other domain, provided the replacement satisfies the interface contract and the transfer-matrix permissions of the domain it replaces. This is a stronger statement than modularity:

what is guaranteed is not that the system is composed of parts, but that a part may be exchanged with the guarantees intact.

Three conditions govern a replacement. It must implement every interface the replaced domain exposed, at the versions still within their support interval, since a caller conforming to a version must remain conforming. It must satisfy every transfer-matrix entry in which the replaced domain appeared, as source or as target. And it must be admitted through the improvement-proposal mechanism with a named human approval, because a domain replacement is a change to the system and no change to the system is self-approving.

Domain 0 is not replaceable. It is a member of the permanently-excluded subsystem set, and a proposal to replace it is refused for the same reason a proposal to amend it is refused.

107.3 Media and immersion interfaces

Two interfaces are added to the seven of §105.1, subject without exception to the constraints stated there: each resolves a scope description before traversal, no parameter widens one, and each is an operation layer carrying its own safety evaluation gate.

The media interface accepts a request for a transformation of a media asset — semantic compression to a summary, semantic expansion to an elaboration, colorization, resolution enhancement — and returns the transformed asset with the provenance of the transformation. A transformation is deterministic for a given asset, parameter set and content state, and an inferred value in a restoration is traceable to the knowledge record from which it was inferred, on the terms already specified. A summary returned by the media interface is composed under the generation vocabulary constraint like any other output, and is not a free rewriting of source text.

The immersion interface returns a scene description assembled from knowledge records carrying spatial detail, together with the assets the scene references. It withholds a scene where the spatial detail does not satisfy the sufficiency threshold, rather than supplying absent geometry. It captures no gaze, dwell, head-pose or attention signal, and exposes none, in either the immersive or the augmented presentation mode.

107.4 Parallel multilingual presentation

Two or more language realizations of one work are presented simultaneously in aligned form, the alignment being by knowledge record rather than by page, paragraph or sentence position. Each realization is generated from the same records through the ordinary regeneration path, so the alignment is a property of shared provenance and not of a maintained correspondence table that can drift.

A record realized in two languages carries one identifier and one semantic fingerprint, so a citation taken in either realization resolves to the same record and a reader's position in one realization maps to the other exactly. Where a record has no realization in one of the selected languages, the gap is shown as a gap rather than filled from the other realization, and a wording-operative span is reproduced in its source language in every realization with its non-operative gloss rendered in each target language.

107.5 Cryptographic key custody and ledger signing

An encryption key is scoped to a tenant or to a deployment, and a key of one tenant decrypts no content of another. Key material is held in a hardware security module or an equivalent custody boundary where the deployment configures one, in which case the system holds a handle rather than the key and every use is a call across that boundary rather than a read of key material.

This does not alter the two custody rules already specified and does not weaken them. Where zero-knowledge encryption is elected for a reader's history, the operator holds no key under any configuration, and a hardware module does not create one. Where a message channel is classified as not associated with a minor context, the system holds no decryption key under any configuration, and the same applies.

Each provenance chain node is signed under a key of the deployment in addition to carrying its keyed checksum, so that the chain is verifiable by a party holding the corresponding public key without access to the system that produced it. Verification establishes that a recorded operation and its recorded safety evaluation are as written; it establishes nothing about the correctness of the operation, and the specification records that distinction so an audit is not read as more than it is.

107.6 On naming the substrate

The submitted specification proposed describing the substrate as a cognitive mesh in which concept-units behave like neurons and links behave like synaptic pathways, and describing the output as neuromorphic language. Neither is adopted, for two reasons that are worth recording because the proposal will recur.

First, this specification already names the machinery. The Ontology Mesh is the concept structure; the Fact Graph is the record structure; the World Aware System Model is the hypergraph; the Neuromorphic Cognitive Layer is the convergence layer; hyperedge attention weights constituents. Introducing a further term covering the same mechanisms would place two names on one thing, which in a specification supporting a patent application is a definiteness hazard under 35 U.S.C. § 112(b) and in an engineering document is a source of drift.

Second, the description overstates the correspondence in the direction this specification has consistently avoided. A concept node does not behave like a neuron: it is a record

with provenance, a confidence value and an access control list, and its weights update by a stated Hebbian rule rather than by gradient descent. Chapter 101's methodology requires that a neuroscience correspondence be verified against a primary publication, extracted as a deterministic principle, and named — not asserted as a resemblance. The correspondences this system does claim were established that way; a general claim that the substrate is neuron-like was not.

The accurate description is available and is narrower: a native hypergraph of provenance-carrying records, traversed deterministically, with retrieval stabilized by convergence dynamics whose operating point is held on the ordered side of a known phase boundary. That statement is defensible against an examiner and against a reader who checks it.

107.7 Substrate independence

The system's substrate is native and complete. Every mechanism this specification describes — record construction, provenance, ontology, retrieval, stabilization, inference, generation — is performed by components specified herein, and no external fact-representation, fact-query or cognitive-substrate system is required for any of them.

A separately-owned external fact-retrieval resource may be connected, as one connector among others, under the capability-declaration contract governing every external system. Where such a connector is registered, a query may be directed to it and a returned result is converted into a native knowledge record at the connector boundary, subject to the same ingestion path, safety evaluation and confidence calibration as any other admitted content. Where no such connector is registered, every mechanism in this specification operates identically. The connector is an option; it is not a dependency, a substrate, or a component.

Three properties make that statement structural rather than aspirational, and each is verifiable against this document. No claim of this specification recites an external fact-query system as an element. No transfer-matrix entry permits a domain to require an external resource in order to complete an operation; the external integration domain's failure is contained to its own capability, which is precisely the sense in which a connector is optional. And the deterministic guarantee — that the same query over the same records returns the same answer — is established by the constrained-vocabulary generation and the bounded-capacity stabilization specified herein, not by any property of an external system.

A description of this system that attributes its determinism, its groundedness or its substrate to an external fact-query technology is therefore inaccurate, and §86.3's correction of earlier mischaracterizations applies to it on the same terms.

Chapter 41 — addition

41.6.5.1 The gap

Under §41.6.2 each bearing record increments a parameter by the weight of its observing source. The parameters therefore accumulate evidential weight and discard the identity of what supplied it. Two records may each stand at α = 20 with one resting entirely on repeated readings of a single edition and the other on readings across two editions, a translation, an image and a video, and nothing in the record distinguishes them.

The distinction is not cosmetic. Repeated observation of one source measures the consistency of that source; observation across independent sources measures the assertion. A systematic error in a single edition produces twenty mutually-agreeing confirmations and no evidence at all about the world, and a confidence value that cannot see the difference will report high confidence for both.

41.6.5.2 Source classes and the diversity measure

Each contribution to a parameter is attributed to a source class. The classes are registered against the Ontology Mesh on the same terms as any other governed vocabulary, and by way of example comprise a distinct work, a distinct edition of a work, a distinct language realization, a distinct modality, a distinct sensor, a distinct external connector and an author-supplemental submission. A contribution not attributable to a registered class is attributed to an unclassified class and is not treated as diverse.

Diversity is measured as the effective number of contributing classes, computed as the reciprocal of the concentration of contributed weight across them.

```
D  =  1 / Σ_i ( w_i / Σ_j w_j )²
```

where w_i is the weight contributed by class i. The measure is one where all weight falls in a single class, approaches the number of classes where weight is evenly spread, and is unaffected by the total. It is therefore orthogonal to the parameters themselves: α states how much evidence there is, D states across how many independent kinds.

Both records report a confidence of 0.9000 at α = 20, β = 2 and carry an identical credible interval, and until this section they were indistinguishable. They are now distinguishable by a value computed from data the system already had and was discarding.

41.6.5.3 Where the measure is used

41.6.5.4 Relation to intersensory redundancy

§55.2.2 already elevates the confidence of a property confirmed across two or more modalities within a common temporal window. That rule is the modality-restricted case of this one: modality is one source class among several, and temporal synchrony is the condition under which two modal observations count as independent rather than as one event observed twice.

This section generalizes the rule and does not replace it. Where the contributing classes are modalities and the synchrony condition is satisfied, §55.2.2 governs the magnitude of the elevation, and D records the diversity for the consumers at §41.6.5.3. Where the classes are editions, languages or connectors, no synchrony condition applies, because two editions are independent by construction rather than by timing.

41.6.5.5 What is not adopted from the submission

41.6.5.6 Schema amendment

The Fact Unit schema is amended in one respect. The confidence field, already specified as a first parameter accumulating supporting evidence and a second accumulating contradicting evidence, additionally records the attribution of contributed weight across registered source classes, from which the diversity measure of §41.6.5.2 is computed. No other field is added, altered or removed, and a record created before this amendment remains valid with its contributions attributed to a single unclassified class.

The expression evaluates the diversity of a record whose twenty units of supporting weight arrive as 8.0 from a first edition, 4.0 from a second, 4.0 from a translation, 2.0 from an image and 2.0 from a video, returning an effective 3.8462 source classes. The same twenty units arriving entirely from one edition return 1.0000.

Chapter 108

108.1 The three modes and the writing rule

Three modes of inference operate in this system. They differ in what follows from their output, and the difference is enforced rather than described.

The rule is this: deduction may write to the knowledge store; induction and abduction may not. A conclusion that follows necessarily from admitted records adds nothing that was not already entailed by them, and admitting it changes no epistemic standing. A

conclusion that is merely well-supported is a new assertion, however strong its support, and a system that admitted it automatically would be generating content its sources did not state — which is the failure the constrained-vocabulary generation exists to make unreachable.

The rule is enforced by type and status rather than by convention. A deductively-derived record carries a derived-inference type identifying the entailment that produced it. An inductively-derived proposal carries a status field set to a proposed state, and no automated process is configured to transition that field to a validated state; the transition requires confirmation by an identified human reviewer, recorded with that reviewer's identity.

108.2 The deductive layer

Concepts are related by subsumption edges in a poly-hierarchical structure, and a subordinate acquires a superordinate's attribute by inheritance transitively along a bounded chain. The inference is the classical syllogistic form and is admitted as such.

Two departures from classical syllogistic are specified and are worth naming because they are not incidental. Inheritance is defeasible: a subordinate asserting an attribute that conflicts with one it would inherit suppresses the inherited value for itself and its subordinates while leaving the superordinate unmodified. Classical syllogistic is monotonic and cannot express this. And where two conflicting attributes are inherited at identical depth, the system performs no resolution at all: the conflict is recorded in the contradiction index and both values are reported with their originating concepts, because an arbitrary tie-break would produce an answer that is reproducible without being grounded.

108.3 The inductive layer

The inductive layer is the evidence-integration layer of the system. It does not decide what is true; it decides how firmly something is held, and what would need to be observed to hold it more firmly or less.

Negative instances are retained deliberately and this is the layer's least obvious property. A contradiction whose precision is too low to revise the concept model is withheld from revision and recorded, so that a pattern of individually-insignificant contradictions against one attribute remains queryable. A contradiction between two active rules over a common record creates an entry in a persistent index rather than a transient warning. In an inductive system the negative instance is the more informative one, and discarding it because it is individually weak would lose exactly the signal that accumulates.

108.4 Where the inductive layer does not operate

Four operations are commonly described as inductive validation and are not. Recording the distinction prevents a weaker guarantee being claimed where a stronger one exists.

108.5 The abductive layer

A candidate explanation for an observation is ranked by the product of the likelihood of the observation under that explanation and a prior over the explanation, computed across the causal structure of the knowledge records. Abduction is the mode that proposes what would account for something; it is neither entailment nor accumulation, and it sits on the same gate as induction.

108.6 Correspondence, stated narrowly

The deductive layer corresponds to categorical syllogistic reasoning as formalized by Aristotle, departing from it in the two respects at §108.2. The inductive layer corresponds to the eliminative-inductive method described by Bacon: evidence tabulated for and against a proposition, corroboration weighted by independence, and the negative instance treated as more informative than the positive. The abductive layer corresponds to inference to the best explanation as formalized by Peirce.

These correspondences are named for precision, not for authority. No mechanism in this chapter is justified by the correspondence, and none of the three names appears in a claim. What is claimed is the operation and the rule that governs what may follow from it. A correspondence explains why a mechanism is well-founded; the mechanism is what an implementation must satisfy.

Chapter 109

109.1 What an interpretive layer is, and the boundary it may not cross

An interpretive layer is a modulation applied to an already-admissible result. It selects among, orders, formats and expresses material that the retrieval and access mechanisms have already made available. It is not a store, it holds no knowledge records of its own, and it participates in no admission decision.

The boundary is stated once and governs every layer in this chapter. An interpretive layer may not widen or narrow a scope description; may not admit a knowledge record that the Slice Resolver excluded, nor exclude one it admitted; may not alter a confidence value; may not extend the permissible generation vocabulary; and may not

cause a proposition untraceable to an admitted record to become expressible. A modulation that would do any of these is refused, and the refusal is recorded.

The consequence is that the interpretive layer cannot become a route around the guarantees. Two requesters whose scope descriptions admit the same records receive answers grounded in the same records however differently the layers modulate their expression, and a requester whose scope excludes a record receives nothing about it under any interpretive configuration. What varies is emphasis, ordering, vocabulary, format and medium. What does not vary is what is true and what is available.

Every layer below is session-scoped working state. Interpretive signals are discarded at session end and are never persisted as knowledge records, on the same terms the Natural Language Interaction Layer already applies to its session context: working state, not evidence about the world, kept out of the Fact Graph.

109.2 The Context Layer

Context determines the circumstances under which a query or reading event occurs: device class and its input and display characteristics, network condition including an offline state, session type such as study, reading, authoring or teaching, and the environment record of Chapter 104 where one is referenced.

Its outputs are an edition selection among editions the requester is already entitled to, a media transformation among those the media interface offers, a layout adaptation, and a query-interpretation hint such as a temporal or edition scoping the requester has not stated explicitly. Where a context signal would select an edition the requester is not entitled to, the selection falls back to an entitled edition and the fallback is disclosed rather than silently substituted.

Rights enforcement is not an output of this layer. The submitted specification lists allowed-versus-suppressed content as a context output; in this architecture that determination belongs to the Slice Resolver and to rights validation, which run before any interpretive layer is applied. Context may request; only the access mechanisms may permit.

109.3 The Perspective Layer

Perspective determines the role in which a requester is acting — reader, author, instructor, student, researcher, publisher, operator — and modulates presentation accordingly: which relationships among the retrieved records are surfaced first, how an explanation is framed, and which secondary surfaces are offered.

The submitted specification's example is exact and is adopted: a student sees learning pathways, an author revision pathways, a publisher rights pathways, over the same

underlying records. What differs is which relationships are surfaced and in what order, not which records exist.

A role determines a scope description through the access mechanism and separately determines a perspective through this layer, and the two are distinct. Where a requester holds several roles, the perspective applied is the one the requester elects for the session, and electing a perspective grants no capability: a reader electing an instructor perspective sees an instructor's framing of the records their own entitlement admits, and no record beyond it.

109.4 The Ergonomic Layer

Ergonomics modulates for human comfort and cognitive load: reading pace, interaction pattern, display and input characteristics, accessibility settings, and the ergonomic preference signals the Semantic Parsing Engine already detects during session analysis. Its outputs are compression or expansion of an explanation, reflowed layout, contrast and legibility treatments, and emphasis on the records most central to the answer.

One rule governs the layer and is carried forward unchanged from §50.4: a stated preference always takes precedence over an inferred ergonomic signal. A requester who has stated a reading level, a density or an accessibility setting receives it, and no inference displaces it. An inferred signal applies only where no stated preference covers the same dimension.

Two constraints follow from the boundary at §109.1. Compression is a selection among admissible records and a shortening of their expression; it never summarises beyond what the records support, and a compressed answer remains subject to the vocabulary constraint. And an inferred cognitive-load signal modulates presentation only: it produces no record about the requester, is not retained beyond the session, and is not available to any other layer, to the commerce mechanism or to any external interface.

109.5 The Cultural Layer

The cultural layer modulates for regional and cultural alignment: locale, regional convention, idiom, and the localized examples and region-specific annotations an author or operator has supplied. Its outputs are idiomatic realization under the existing registered-equivalence rule, selection among localized examples where several exist, and territory-aware edition selection among entitled editions.

A cultural profile is declared, never inferred. A profile is set by the requester, by an operator for a deployment, or by a locale the requester has explicitly selected; it is not derived from a name, a network address, a language of query, a device locale or any behavioural signal. The reason is stated because it is the point on which this layer would otherwise fail: inferring a person's cultural background from incidental signals

produces a classification of that person which the system has no warrant to make and no mechanism to correct, and a declared profile is both more accurate and revocable by the person it describes.

Where no equivalent is registered for a locale, the layer renders plainly rather than substituting an unregistered equivalent, on the same terms the idiom mechanism already requires. Cultural modulation adjusts which of several admissible expressions is selected; it does not adjust what the records assert, and no cultural configuration causes an assertion to be expressed as other than what its records support.

109.6 The Social Layer

The social layer modulates for group settings: a class session, a shared reading session, a collaborative authoring session, an instructor-guided walkthrough. Its outputs are group-aligned pacing, shared position and shared annotation surfaces, and register and formality appropriate to the setting.

Its governing constraint is the one already specified for shared sessions and is restated because it is what makes group modulation safe: the shared view is the intersection of the participants' scope descriptions, and social modulation operates within that intersection. Aligning a group's pacing, surfacing a shared annotation or synchronising a position never surfaces to one participant a record another participant's entitlement excludes, and the absence of such a record is not disclosed.

A social annotation is a reader-authored knowledge record carrying its author's identity and its own access control list, which the social layer does not widen. Formality and tone are modulations of expression; no social configuration alters an assertion, and a group setting confers no entitlement on its participants individually or collectively.

109.7 Composition and precedence

The five layers compose in a fixed order over a result the access and retrieval mechanisms have already produced. Composition is deterministic: the same result under the same interpretive configuration yields the same modulated output on every execution.

Two precedence rules resolve conflict between layers. A stated preference of any kind overrides an inferred signal of any kind, at whichever stage each arises. And where two layers would modulate the same dimension incompatibly, the later stage governs for form and the earlier for substance, so that a social register setting yields to an accessibility requirement on legibility but not on tone.

109.8 Interpretive interfaces

Each layer exposes an interface through which an external system may supply a configuration and read the configuration in effect, on the terms already governing the public interface layer: each resolves a scope description before any traversal, no parameter widens one, and each carries its own safety evaluation gate.

An interpretive interface accepts a configuration and returns a modulated result. It cannot be used to retrieve a record, to alter a confidence value or to widen a scope, and a configuration that would require any of those is refused at the interface with the reason recorded. The configuration in effect for a response is recorded with that response, so that a modulated answer is reproducible from its recorded configuration.

109.9 Contextual Value Models — where cultural, social, and ergonomic values are stored

This section states where the cultural norms, social values, ergonomic preferences, and context that the modulations of this chapter apply are stored, because a reader is entitled to know that they are not stored as Fact Units and not stored as World-View Model concepts. A cultural norm, a social value, and an ergonomic preference are not assertions about the world and are not models of how the world behaves; they are declared parameters of how an already-admissible result is expressed to a particular audience. They are held in Contextual Value Models: registered, versioned, provenance-carrying configuration records that the ergonomic, cultural, and social layers of this chapter read, and that no other layer may write from its own observation.
A Contextual Value Model is declared, never inferred. It is set by the requester for themselves, by an operator for a deployment, or by an author for a work, and it carries the identity that declared it and the time it was declared. It is never derived from a name, a network address, a query language, a device locale, a purchase history, or any behavioural signal, so a person is never assigned a cultural, social, or ergonomic profile by inference from who the system guesses they are. Where no model is registered for a locale or an audience, the relevant layer renders plainly rather than substituting an unregistered value, exactly as §109.5 already requires of the cultural layer.
A Contextual Value Model modulates expression and nothing else. Consistent with the invariant of this chapter, it never widens or narrows a scope description, never admits or excludes a record, never alters a confidence value, and never extends the permissible vocabulary; two requesters whose scope descriptions admit identical records receive answers grounded in identical records however differently their Contextual Value Models modulate the expression. A Contextual Value Model is therefore not a grounding source: it is not citable, not traversable, and contributes to no confidence value, and a change to a Contextual Value Model changes how an answer reads and never what the answer is grounded in.
Context in the sense of the determinism guarantee is stored separately again, as the enumerated locked dimensions of the context lock, and is carried with an answer as the context key rather than in a Contextual Value Model; the two are complementary and are not to be conflated, the context key fixing when two executions must agree and the Contextual Value Model governing only how a result is voiced. The placement is stated

as a boundary. A fact about a culture — that a custom is observed in a place, that a term carries a connotation in a language — is a Fact Unit like any other, ingested, evidenced, and citable. A model of how such a custom relates to others is a World-View Model concept. A Contextual Value Model is neither: it is the declared preference under which the system expresses a grounded answer to a chosen audience, and holding it apart from the record and from the concept is what keeps a reader’s or an operator’s values from ever becoming, by storage alone, an assertion the system treats as true. (Drafting record, Pass 118, September 6, 2026: added upon the inventor’s direction to specify where cultural norms, social values, ergonomics, and context are stored. Contextual Value Models are registered configuration consumed by the interpretive layer of this chapter; they are additive, add no field to §3.1 and alter no behaviour of §3.6 or §3.7, and are distinct from Fact Units and from World-View Model concepts; the determinism guarantee is preserved without qualification.)

109.10 Not adopted

Row 194 evaluates the shared-view bound: a group session between requesters whose scopes admit 340 and 275 records modulates within the 275 both admit. Row 195 evaluates the precedence of the governance review against the first interpretive stage.

Chapter 110

110.1 The delivered answer as an action

A delivered response is an action taken by the system on the strength of its records. It carries, in the provenance chain, the query, the scope description resolved for it, the knowledge records cited, the confidence values those records held at delivery, and the interpretive configuration under which it was expressed. A response record is therefore already a complete statement of what the system asserted and on what basis, and this chapter adds no field to it.

What the specification lacked is a subsequent observation bearing on that action. An answer may be acted upon, contradicted by a later event, corrected by a reader who knows better, or simply left alone; until now none of those returned to the system in any form, so the system's own accuracy was unmeasured except retrospectively over historical corpora.

110.2 Feedback as an attributed observation, never as ground truth

Feedback on a delivered response is admitted as a knowledge record on the ordinary terms and on no others. It carries the reporting identity as its attributed source, a source reliability for that identity, a reference to the response record it concerns, and where the report bears on a specific cited record, a reference to that record. It passes the safety

evaluation gate, receives a confidence calibration, and enters the Fact Graph through the same extraction path as any other content.

The consequence is the rule that keeps this mechanism safe: feedback is evidence, not correction. A report that a cited record is wrong increments that record's contradicting parameter by the reporting source's weight; it does not set the record's confidence, does not supersede the record, and does not remove it from the graph. A record at $\alpha = 18$, $\beta = 2$ reporting 0.9000 moves to 0.8633 on a report from a source of reliability 0.85, and to 0.8759 on one of reliability 0.55. Ten independent reports move it substantially; one moves it slightly; and the arithmetic that produces that behaviour is the arithmetic already specified at §41.6.2.

A reporting identity is a source class for the purpose of the diversity measure, so ten reports from one identity do not carry the weight of ten reports from ten. Feedback from an identity with authority over the content — the author of the work, an instructor over a class corpus — carries the source reliability that identity already holds and no additional authority conferred by this chapter.

A feedback record is never admitted as a grounding source for an answer about the subject matter it concerns. It is admissible as evidence bearing on a record's confidence and as an object of analysis; it is not a statement about the world that the system may cite. Where a reader asserts a fact the system does not hold, the assertion is retained as a reader-authored record under the existing access rules and is not thereby admitted to the work's corpus.

110.3 Observed outcomes

An outcome observation records what subsequently occurred in respect of a matter a response addressed. It is admitted on the same terms as feedback, and is distinguished from it in one respect: an outcome is an event in the world rather than a judgment about the response, and where an outcome record and a response record can be paired, the pairing supports measurement under §110.4.

The subject-class restriction of Chapter 103 applies without exception. An outcome concerning an institution, an organisational process, a system or an aggregate group is admissible. An outcome whose subject resolves to an individual natural person is refused at construction, so that no per-individual predictive record accumulates from operational use, which is the failure mode a feedback mechanism of this kind is most likely to produce.

110.4 Measurement

Two measures are computed over paired responses and outcomes, and both are properly scoped: they measure the calibration of stated confidence, not the quality of an answer.

The first is a mean squared error between the confidence stated at delivery and the realised outcome. Over five responses stated at 0.90, 0.85, 0.80, 0.75 and 0.70 with outcomes 1, 1, 0, 1 and 0, the measure is 0.2450.

$$\text{score} = \Sigma \left(c_i - o_i \right)^2 / n$$

The second is a calibration gap by confidence band: the proportion of outcomes that occurred among responses stated within a band, against the mean confidence stated in that band. Five responses in the 0.80 to 0.95 band stating a mean of 0.8700 against an observed rate of 0.8000 yield a gap of −0.0700 — the system was overconfident in that band by seven percentage points. A gap whose magnitude reaches a configured tolerance raises a calibration finding.

Both measures are computed per domain and per confidence band, because a system may be well calibrated in one domain and poorly in another, and an aggregate figure would conceal exactly the variation an operator needs. A finding of poor calibration in a domain is as reportable as a finding of good calibration, and the domains in which the system predicts badly are the operationally useful output: they identify where a confidence value should not be offered to a reader at the precision it currently claims.

110.5 Adjustment, and the gate on it

A calibration finding does not change the system. It is emitted as an improvement proposal under the existing governance mechanism, carrying the measured gap, the corpus over which it was measured, the domain and band it concerns, and the parameter it proposes to adjust — typically a retrieval confidence floor, a source reliability, or a confidence-band presentation threshold. The proposal takes effect only upon approval recorded against an identified human reviewer with that reviewer's stated reasoning.

The reason for the gate is worth stating rather than assuming. A system that adjusted its own thresholds on observed feedback would drift continuously, and no one would have approved the drift; its behaviour at any moment would be a function of its accumulated interaction history rather than of a decision anyone made or could review. Feedback here is measured, reported and proposed. It is never applied automatically, and the parameters governing feedback measurement are themselves within the permanently-excluded set, so the mechanism cannot adjust the terms on which it is judged.

Two further exclusions follow. No feedback signal reaches the permissible generation vocabulary: an answer does not become expressible because readers report wanting it. And no feedback signal reaches a scope description: a record does not become traversable because it was requested, nor cease to be because it was disliked.

110.6 Retention and the requester's position

A feedback record and an outcome record concerning an identifiable subject are retained under the ordinary retention policy and are subject to the erasure operations at session, work and account scope. A requester who erases their history erases their feedback with it, and the derived calibration statistic — an aggregate over a corpus, attributable to no individual — is retained, on the same division Chapter 103 already specifies.

Submitting feedback is optional, and a response is delivered identically whether or not feedback has ever been submitted by anyone. The mechanism creates no obligation on a reader and no degradation for a reader who does not use it.

Row 196 evaluates a record at α = 18, β = 2 receiving one contradicting report from a source of reliability 0.85, returning 0.8633 against a prior 0.9000. Row 198 returns 0.2450 over the five paired responses of §110.4. Row 199 evaluates a band gap of −0.0700 against a tolerance of 0.05 and raises the finding.

The cycle now closes at both ends. Evidence arriving from ingestion moves belief through the parameters; evidence arriving from operation moves belief through the same parameters; and neither writes an assertion, because a report is admitted as an attributed observation and the arithmetic that consumes it is the arithmetic already specified. What the loop adjusts automatically is confidence. What it proposes, and never applies, is a change to the system.

Chapter 111

111.1 The expressive register

An expressive register is a governed selection determining how an answer is voiced, distinct from the audience level that determines vocabulary and explanatory depth and from the discipline mode that determines domain convention. A response carries exactly one register, selected from a registered set and resolved before generation as part of the Generation Directive.

A register not in the registered set is refused rather than approximated, on the same terms a free-text event category is refused. Registers are registered against the Ontology Mesh, versioned, and deprecated rather than deleted, so a response composed under a register remains interpretable after that register is superseded.

The constraint that makes a register safe is stated once and governs the whole chapter. A register selects among expressions of what the admitted records support. It does not alter which records were admitted, does not alter a confidence value, and does not extend the permissible output vocabulary. In particular the authoritative register does

not remove a hedge that a record's confidence requires, and the supportive register does not soften an assertion the records state plainly: a register governs voicing, and a confidence value governs what may be asserted.

111.2 The affective register and how it is selected

The supportive register is the system's affective capability, and its selection is constrained more tightly than any other. It is selected on one of three grounds: a requester states a preference for it; a requester operating under an accessibility or discipline configuration for which it is the configured default receives it; or a consented signal already specified elsewhere indicates it, namely the prosodic signal of the live voice capability, which requires an explicit session-scoped consent and is discarded at session end.

It is not selected by inference from the text of a query. A requester's emotional state is not derived from their word choice, punctuation, query length, error rate, retry behaviour or interaction timing, and no record of an inferred emotional state is constructed. The reason is the one already given for cultural profiles at §109.5: inferring an affective state from incidental signals classifies a person on evidence that does not support it, produces a classification the person cannot see or correct, and is unnecessary because a stated preference produces the same modulation more accurately.

Where the supportive register is in effect the response acknowledges the requester's stated position before delivering information, reduces density, and avoids imperative construction. It does not withhold an answer the records support, does not substitute reassurance for information, and does not alter a confidence value. A requester who has asked for support receives support in the voicing and the same facts.

111.3 Media-integrated expression

A grounded response may carry media assets within it rather than alongside it: an image, a diagram, a video segment, a rendered scene. The rule governing their inclusion is the rule already governing propositions, applied to assets.

An asset is included only where it is referenced by a knowledge record of the stabilized set, and it is delivered with a citation resolving it to that record exactly as a proposition is. An asset that supports no admitted record is not included, however apt it appears, and an asset whose supporting record was excluded by the scope description is not included either. There is therefore no path by which a media asset conveys something the accompanying text could not have asserted.

Where an asset has been transformed — colorized, resolution-enhanced, compressed — the transformation's provenance travels with it, so a reader receives the asset, the record it supports and the record from which an inferred value in the transformation was derived. An asset offered by an interpretive modulation for reasons of layout or

ergonomics is subject to the same test: modulation may select among includable assets and may not make an asset includable.

An accessible equivalent accompanies each included asset at the same production stage, and a response is reported incomplete where an included asset lacks one, on the terms already specified for accessible-equivalent generation.

111.4 Composition with the interpretive layer

Register selection resolves before generation, as an element of the Generation Directive, and is therefore upstream of the interpretive modulations of Chapter 109. The interpretive layers may narrow a register selection where a configuration requires it — an accessibility setting may require the plain register — and may not widen one: no interpretive configuration selects a register the requester's configuration does not permit.

The precedence rule of §109.7 applies unchanged. A stated register preference overrides an inferred ergonomic signal bearing on the same dimension. Where an accessibility requirement and a stated register conflict on legibility, the accessibility requirement governs form; where they conflict on voicing alone, the stated preference governs.

111.5 Not adopted

111.6 On the comparative framing

The submitted specification frames each layer against the capabilities of a named commercial system and states that certain layers permit this system to exceed it. That framing is not adopted into this specification, and the reason is practical rather than modest.

A comparative performance claim is governed by the validation and benchmarking process of Chapter 77 and requires a measured comparison, defined on a stated benchmark, before it may be asserted anywhere. A specification that asserts it without measurement states something the applicant cannot support. In a patent application the exposure is greater still: a characterization of a competitor's capability adds no limitation an examiner can test, invites the examination that disproves it, and may create an admission narrowing the applicant's own claims.

The defensible statement requires no comparison and is already made throughout this specification: output is composed from knowledge records under a vocabulary constrained at each generation step, such that a proposition not traceable to an admitted record is unreachable rather than improbable; the same query over the same records in the same state yields the same output; and every element of the answer

resolves to a record and a source. Whether another system can do this is a question for a benchmark, not for a specification.

Chapter 112

112.1 Position in the pipeline

Surface realization operates on a constructed Narrative Graph and produces the token sequence a reader receives. It runs after the graph is built and after the vocabulary constraint has determined what may be expressed, and before the interpretive modulations of Chapter 109 apply their formatting. It selects among ways of saying what the graph already licenses.

Every operation in this chapter is subject to one constraint, stated once. A realization operation may combine, shorten, reorder or vary the expression of propositions the Narrative Graph carries. It may not introduce a proposition the graph does not carry, may not remove a qualification a confidence value requires, and may not produce a token sequence whose meaning differs from the propositions realized. Each operation below states how that constraint is enforced for it, because a general assurance is not enforcement.

112.2 Aggregation

Where two or more propositions of the Narrative Graph share a subject and are joined by an elaboration or conjunction edge, they are realized as one sentence with a coordinated predicate rather than as separate sentences. Aggregation applies up to a configured maximum of coordinated predicates, beyond which a further sentence begins, so that combination does not produce a sentence a reader must re-read.

Aggregation is refused where the propositions carry differing confidence values that would require differing qualifications, because a coordinated predicate would attach one qualification to all of them. Two propositions at 0.95 and 0.62 are realized separately so that the second carries its own hedge; two at comparable confidence are combined. This is the enforcement of the constraint at §112.1 for this operation: aggregation may not silently promote a weakly-supported proposition by coordinating it with a well-supported one.

Each proposition retains its own citation after aggregation. A combined sentence carries the citations of every proposition realized within it, so combination does not reduce traceability, and a reader can resolve any clause of a coordinated sentence to the record that supports it.

112.3 Referring expressions

An entity is realized in full form at its first mention within a discourse unit and in reduced form thereafter — a pronoun, a definite description, or a shortened name registered for that entity. Full form is restored where an intervening mention of another entity of the same type would render the reduced form ambiguous, and at the start of a new discourse unit.

Ambiguity is decided by the entity's type in the Ontology Mesh rather than by surface form, so two entities sharing a type contend for a pronoun even where their names differ, and two of different types do not. Where the test cannot be decided, full form is used: an unnecessary repetition costs a reader nothing, and an ambiguous pronoun costs them the sentence.

112.4 Connectives

A Narrative Graph edge carries a type. Where two propositions realized in sequence are joined by a typed edge, a connective marking that type is emitted, so that the relation the graph knows is stated rather than left to the reader to infer from adjacency.

The connective realized within a class is selected by the expressive register in effect and by the sentence position, and the selection varies across a response so that one connective does not recur mechanically. A connective is emitted for an edge type and never invented: where two propositions are adjacent without a typed edge between them, no connective is emitted, because asserting a relation the graph does not carry would assert something no record supports.

112.5 Information ordering within a sentence

Within a sentence, information already established in the preceding sentence precedes information new to the reader, where the propositions and the target language's syntax permit. The rule operates on what the response has already said rather than on a general notion of familiarity, so it is computable: an entity or predicate realized in the preceding sentence is established, and one not yet realized is new.

The rule yields to two constraints. Target-language syntax governs absolutely — a language whose word order does not permit the preferred arrangement receives the arrangement it permits, on the terms already specified for syntactic realization. And a wording-operative span is reproduced in its source order and is not reordered under this rule, because reordering an operative span would alter the wording that carries force.

112.6 Lexical variation, bounded by the fingerprint

Where a term recurs within a response, an alternative registered as synonymous for that sense may be substituted, so that a response does not repeat one term

mechanically. Variation is bounded by a test rather than by judgment: a substitution is admitted only where the proposition's semantic fingerprint is unchanged by it.

The fingerprint is computed over the canonicalized triple, so a substitution preserving it is a substitution that does not change what the proposition asserts. A synonym that is registered for a different sense of the term, or that shifts the assertion in any respect the canonicalization captures, fails the test and is refused. Registered synonymy is drawn from the lexical resources of the resource registry, so the source of every admitted substitution is licensed and recorded.

Two exclusions apply. A term appearing in a wording-operative span is never substituted. And a term that is the registered canonical name of an entity, or a governed term of a discipline mode, is not varied: precision in naming outranks variety, and a reader who sees two names is entitled to assume two things.

112.7 What realization does not do

No operation in this chapter introduces a proposition. Aggregation combines, referring expressions shorten, connectives mark relations the graph carries, ordering rearranges, and variation substitutes within an unchanged fingerprint. A response after realization asserts exactly what the Narrative Graph licensed before it, and every clause resolves to the record that supports it.

No operation in this chapter is stochastic. Selection among admissible realizations — which connective within a class, which registered synonym — is made by a deterministic rule over the response's own prior content, so the same Narrative Graph under the same register yields the same token sequence on every execution. Variety across a response is produced by position-dependent selection, not by sampling.

And no operation in this chapter is a language model. The realization stage is a rule system over a graph the system constructed from its own records. A trained model in this position would produce more fluent output and would relocate the neural boundary from extraction into generation, at which point a proposition's traceability would become probabilistic. The gain is not worth the guarantee, and the guarantee is the invention.

Chapter 113

113.1 Eligibility from declared facts only

A commercial item is eligible for presentation where four conditions hold, each decided from a fact already established about the request rather than from an inference about the requester: the rights registered for the item permit its presentation to the requesting deployment; the territory recorded for the request is one the item's registration covers;

the edition in which the response is delivered is one the item's registration covers; and the device class can render the item's format.

No other input participates in eligibility. In particular an inferred ergonomic signal, an inferred cognitive-load indicator and an inferred affective state are unavailable to commercial selection, as already specified: a signal derived from how a person reads is not a signal about what they should be sold. A declared cultural profile may bound eligibility where an item's registration is locale-specific, on the same terms as any other declared configuration, because a declared profile is a statement by the requester rather than a classification of them.

Eligibility is computed before ranking and produces a set. Where the set is empty the commercial region is omitted, and its omission is not disclosed as an absence, because a region that sometimes reports having nothing to show conveys information about the requester's eligibility.

113.2 Placement classes and density

A commercial item occupies a registered placement class, and each class is a region distinct from the response and labelled as commercial. The classes are: an inline placement between sections of a response; an edition-aligned placement associated with an edition rather than with a response; a media-adjacent placement accompanying an included asset; and a spatial placement within an immersive presentation.

Two constraints govern every class. A placement is never interleaved with the propositions of a response, so a reader distinguishes the answer from the commercial region by position and not only by label. And a configured density ceiling limits placements per response and per session; a placement exceeding the ceiling is omitted rather than deferred, so density is bounded rather than merely rationed.

A spatial placement within an immersive presentation is subject without exception to the restriction already specified for that surface: no gaze direction, dwell duration, head pose or attention signal is captured, derived or retained, and none is available to commercial selection. A placement in a three-dimensional environment is a placement, not an instrument.

113.3 Advertiser provenance

Every commercial item carries a provenance record identifying the advertiser or vendor by verified identity, the rights under which the item is presented, the territories the registration covers, the interval over which it is valid, and the identity of the operator who registered it. An item lacking any of these is not registrable and therefore not presentable.

Each presentation is recorded in the Provenance Ledger with the item, the placement class, the concept identifier that occasioned it, the eligibility facts relied upon, and the score computed under the existing weighting. Two consequences follow. A presentation is auditable after the fact against the eligibility rules in force at the time, and an advertiser's claim that an item was or was not presented resolves against a record rather than against a report.

A commercial item is not a knowledge record and is not admitted to the Fact Graph. It is not citable, is not traversable, and contributes to no confidence value. An assertion appearing within a commercial item is the advertiser's assertion, is labelled as such, and is subject to the Prime Directive review that governs any delivered content.

113.4 Ranking, unchanged

Among eligible items a score is computed as the existing weighted combination of a relevance factor, an operator-configurable source-priority factor and a bid factor, with the weights summing to unity. By way of example, weights of 0.4, 0.3 and 0.3 applied to factors of 0.9, 0.7 and 0.6 yield 0.75. This chapter adds no factor and alters no weight.

Selection is deterministic: the same eligible set under the same weights and factors yields the same selection on every execution. No factor derives from a requester's interaction history, and no ranking input is accumulated across sessions.

113.5 Transaction objects and actions

A commerce object is a thing that may be transacted: an edition, a media asset, an immersive experience, a bundle of works, an institutional or classroom licence, a supplemental pack extending an existing entitlement. Each is registered with the rights permitting its sale, the territories in which it may be sold, and a price per territory and currency; an object with no price registered for a request's territory is not offered there rather than offered at a default.

Every action produces a transaction record carrying the acting identity, the object, the action, the price and currency applied, the territory, the rights validation relied upon, and a reference into the Provenance Ledger. A transaction that cannot be reconciled against an entitlement change is refused rather than recorded as pending, so the entitlement state and the transaction history cannot diverge.

Where an entitlement is represented by a token under the existing non-custodial mechanism, the entitlement record remains authoritative on divergence, as already specified. A transfer or revocation of a token follows the entitlement, never the reverse, and a token whose state disagrees with the entitlement record raises a divergence finding rather than altering the entitlement.

113.6 Export, print and their fees

An export or print action is a commerce action where the applicable rights make it chargeable, and a rights validation precedes composition as already specified. A fee applies only where the registration records one for that territory and quantity; where the rights permit the action without a fee, no fee is charged, and where the rights do not permit it, no fee makes it permissible.

113.7 Not adopted

Row 210 evaluates the existing selection score at weights 0.4, 0.3 and 0.3 over factors 0.9, 0.7 and 0.6, returning 0.75, recorded with the presentation for audit.

Chapter 113 — addition

113.8.1 The adjacency set

On completion of retrieval, stabilization and generation, the concept identifier resolved from the query is supplied to commercial selection together with the identifiers of concepts adjacent to it within a single edge of the concept structure. The set is computed once, at the point of supply, and is not extended by the selection mechanism.

Adjacency is bounded three ways. It is one hop: a concept two edges from the query concept is not in the set, and no traversal is performed from an adjacent concept. It is restricted to edge types registered as adjacency-bearing — subsumption, part-whole and a registered general association — excluding causal, temporal and contradiction edges, because a concept related to the query as its cause or as a contradiction of it is not thereby a concept the reader is interested in purchasing against. And each adjacent concept is tested against the requester's scope description, with a concept the requester could not have retrieved excluded from the set.

That last bound is the one that matters most and is stated as a rule rather than left to follow. Commercial selection must not become a channel through which the existence of a concept outside a requester's entitlement is inferable from what was advertised to them. An adjacent concept outside the scope description is absent from the set, and its absence is not signalled.

113.8.2 What adjacency is used for

An adjacent concept identifier participates in the relevance factor of the existing selection score and in nothing else. It does not participate in eligibility, which continues to be determined from registered rights, recorded territory, delivered edition and device

class alone; a commercial item ineligible for a request does not become eligible because it matches an adjacent concept.

Relevance against an adjacent concept is attenuated relative to relevance against the query concept itself, so that an item matching the query concept directly outranks an item matching only a neighbour. By way of example, an item scoring 0.72 on relevance through adjacency, against source-priority 0.7 and bid 0.6 at the existing weights of 0.4, 0.3 and 0.3, yields 0.678, below the 0.75 that direct relevance at 0.9 would produce on the same other factors.

The weights themselves are unchanged and still sum to unity. This section adds no factor to the score and alters no weight; it widens the concept set against which the existing relevance factor is computed.

113.8.3 What the amendment does not weaken

The guarantee claim 27 protects is the isolation of commercial selection from the answer pipeline and its position after it. Neither is a function of how many concept identifiers cross the boundary. A single identifier and a bounded neighbourhood are equally read-only, equally post-generation, and equally without a write path; what changes is how well the commercial region can be made relevant to what the reader actually asked.

Row 228 evaluates a causal edge against the registered adjacency-bearing set and returns refusal. Row 230 evaluates the existing score with relevance attenuated to 0.72 through adjacency, returning 0.678 against the 0.75 that direct relevance would yield.

Chapter 114

114.1 Scope: what may be commanded, and what may not

Command emission is permitted to registered equipment of a capture and inspection class: a pan-tilt-zoom camera head, a lighting fixture, a capture drone operating within a bounded volume, an inspection arm, a mobile capture base, and a positioning stage. Each unit is registered with its class, its kinematic envelope, its maximum velocity and force, and the volume within which it may operate.

Command emission to any other equipment class is refused at the interface. In particular the vehicle prohibition already specified is unchanged and is not narrowed by this chapter: telemetry from a vehicle-control system continues to be ingested as analysis and no control interface to any vehicle system exists or is created. A

mechanism whose failure can injure a person at speed is not brought within scope because a mechanism whose failure spoils a photograph has been.

An equipment unit whose registration lacks a kinematic envelope, a velocity limit or an operating volume is not commandable. The limits are not advisory parameters supplied with a command; they are properties of the registration, and a command exceeding them is refused before emission rather than clamped.

114.2 Live ingestion and the equipment state record

Sensor streams from registered equipment — video, depth, range, inertial, joint position, force and torque, encoder — are ingested through the Multimodal Ingestion Pipeline as any other sensor content, reduced to concept-level records before admission, and subject to the safety evaluation gate. No raw sensory episode is retained, as already specified.

Distinct from those records, and not admitted to the Fact Graph, is the equipment state record: pose, joint configuration, camera orientation, gripper state, battery, link quality and the operating volume currently in force. It is session-scoped working state, discarded at session end, and never persisted as a knowledge record — the same treatment the Natural Language Interaction Layer already gives its session context. The reason is that a control loop's working state is a fact about a machine at an instant, not evidence about the world, and admitting it would fill the Fact Graph with expired positions.

An equipment state record older than a configured staleness bound is not usable for command synthesis. Where state is stale the system emits no command and reports the staleness, because commanding a machine from a stale picture of where it is is the characteristic way such systems cause damage.

114.3 Session authorization

Command emission requires a session authorization granted by an identified human operator, recorded with that operator's identity, the equipment units it covers, the operating volume it permits, the task classes it permits, and an expiry. Authorization is per session and not per command: continuous control cannot require an approval for each of thousands of commands, and a mechanism that pretended otherwise would be approved by reflex rather than by judgment.

Four properties bound the authorization, and together they are what makes a session gate an acceptable substitute for a per-command gate. It expires, and expiry halts emission rather than prompting for renewal mid-motion. It is revocable at any instant by the authorizing operator or by any operator holding the equipment's administrative role, and revocation takes effect before the next command rather than at the end of the current plan. It is bounded to the registered units, volume and task classes named in it,

so an authorized session confers no capability over equipment it does not name. And it is recorded in the Provenance Ledger, so every emitted command traces to the authorization that permitted it and the operator who granted it.

This is a departure from the approval discipline applied elsewhere in this specification, where a change of state requires a named human approval of that change. The departure is recorded rather than concealed: what a session authorization approves is a bounded capability for an interval, not each exercise of it. The compensating controls are the veto at §114.5, the halt condition at §114.6 and the revocability above.

114.4 Query to command

A request expressed in natural language or as a structured directive is compiled to a command sequence through the stages already specified for bounded program compilation, extended with a spatial resolution stage.

Each emitted command carries a reference to the request that produced it, the authorization under which it was emitted, the plan it belongs to and its position within that plan. A command that cannot carry these references is not emitted, so there is no command in the system whose origin is unattributable.

114.5 The constraint layer, which vetoes rather than advises

Before emission every command is evaluated against the constraint set, and a command violating any constraint is not emitted. The evaluation is a gate and not a score: a violating command is not emitted at lower priority, not emitted with a warning, and not emitted after a modification the system selects for itself.

Human proximity detection is safety-only and is bounded accordingly. A person detected for this purpose is used to compute clearance and for nothing else: no person classification enters a result set, none is retained beyond the instant it is evaluated, none is queryable, and none is available to any other mechanism. This resolves the tension with the exclusion already specified elsewhere — person classifications remain absent from every result set, and safety detection is not a result set. A system that could not see a person could not avoid one; a system that remembers having seen them is a surveillance capability, and this is not one.

114.6 Halt

A halt is emitted on any of: a clearance or proximity breach detected during execution, an equipment state record exceeding the staleness bound, an authorization expiry or revocation, a loss of the sensor stream on which the executing plan depends, and an

explicit halt request from any operator holding the equipment's administrative role or from the requester of the session.

The halt path does not traverse the reasoning path. It is evaluated from the equipment state record and the constraint set directly, without retrieval, stabilization, generation or interpretive modulation, so a halt cannot be delayed by a query in progress and does not depend on any mechanism whose latency is a function of corpus size. This is stated as a requirement rather than an optimisation: a stop that waits for an answer is not a stop.

A halt places the unit in its registered safe state and requires a new authorization or an explicit resume from the authorizing operator before emission resumes. A halt is recorded with its triggering condition, and a pattern of halts against one unit or one task class is queryable, so equipment or a plan that halts repeatedly is detectable rather than merely tolerated.

114.7 Execution feedback

Execution is observed and the observation is admitted on the terms of the feedback mechanism already specified: a plan is an action, and what the sensors report afterwards is an outcome. Did the commanded framing contain the target; did the arm reach the commanded pose within tolerance; did the capture return an asset of the requested resolution.

A discrepancy adjusts confidence in the records the plan relied upon and may occasion a corrected plan, which is compiled and evaluated afresh through §114.4 and §114.5 rather than issued as a delta to a vetoed plan. A correction is a new plan under the same authorization, subject to every constraint the original was subject to.

Two limits carry over unchanged. An outcome whose subject resolves to an individual natural person is refused at construction. And no execution feedback adjusts a constraint, a limit or an authorization: feedback informs confidence and proposes, and the envelope within which a machine may move is not something the machine's own experience may widen.

Row 216 evaluates the governing clearance as the lesser of the measured distance to a detected person and the configured exclusion distance, returning 0.35 metres, which row 215 then tests against the minimum.

Chapter 115

115.1 Scope and the equipment classes

This chapter governs equipment that makes intentional physical contact with a person in order to assist them. Three classes are recognised, following the classification of the governing standard: a servant unit that carries or hands over an object without bearing a person's weight; an assistant unit that supplements a person's own movement, including a supported transfer or a steadying contact; and a carrier unit that bears a person's weight.

A unit is registrable in this chapter only where a hazard analysis has been completed for its intended tasks, its intended users and its intended environment, and its outcome recorded with the identity of the assessor. A unit without a completed hazard analysis is not registrable, and no authorization can be granted over it. The reason for placing this before every other requirement is that the remaining requirements are meaningless where the hazards have not been enumerated: a force limit is only protective if the analysis identified the contact it limits.

A unit registered under Chapter 114 is not thereby registrable here, and a unit registered here is not thereby commandable under Chapter 114. The authorization classes are disjoint, and an authorization of either class confers nothing under the other. A single physical device may hold registrations under both, in which case the constraint set applicable to a command is determined by the authorization under which it was emitted, and never by the device's other registration.

115.2 Two-party authorization

Contact requires two authorizations, held concurrently, and neither substitutes for the other.

Deployment authorization establishes that contact is permissible. Contact consent establishes that it is wanted. A system holding only the first is a system that has been told it may touch someone who has not agreed to be touched, and no operational convenience justifies proceeding on it.

Consent is bounded to a task class and an interval and is not general. Consent to a supported transfer is not consent to a steadying contact, and consent given for one session does not persist into the next. A consent record carries the consenting identity, the task class, the interval, and the manner in which consent was expressed.

115.3 Withdrawal, and whose withdrawal governs

Consent is withdrawable at any instant by the person being assisted, through any registered channel including speech, a physical control, a gesture registered for the purpose, and an assistive input device. Withdrawal halts contact within the equipment's registered stopping interval and places the unit in its safe state.

A withdrawal by the assisted person cannot be overridden by an operator, by a representative, or by a task in progress. This is stated as an absolute because the circumstances in which it would be tempting to override it — a transfer half-completed, a caregiver who believes the assistance is needed — are exactly the circumstances in which a person's refusal matters most. Where withdrawal occurs mid-task the unit completes no remaining motion except motion required to reach a state in which the person is not supported by it and is not trapped by it, and that motion is bounded by the same force limits as the task itself.

An operator may also halt, and an operator halt is not a withdrawal: it does not revoke consent and does not require re-consent to resume within the same session. A withdrawal does require fresh consent.

115.4 The person who cannot withdraw

Where the person to be assisted cannot express or withdraw consent through any registered channel, contact proceeds only under a stricter mode and not under the ordinary provision of §115.2. Three conditions apply together.

A designated representative, recorded as such under the deployment's own governance and not selected by the system, holds contact consent on the person's behalf. A second person is present throughout the contact and holds a halt control, so that a party other than the equipment and other than the representative can stop it. And the task classes available in this mode are a registered subset of those otherwise available, excluding any task whose failure the assisted person could not signal.

The system does not determine that a person is unable to consent. That determination is made and recorded by the deployment under its own clinical or legal governance, and the system reads it. A mechanism that inferred incapacity from a person's failure to respond would convert a communication difficulty, a moment of inattention or an equipment fault into a loss of that person's authority over their own body, and it is not specified here.

115.5 The contact constraint set

Chapter 114's exclusion distance is inapplicable and is replaced. Before and during contact every command is evaluated against the following, and a violation halts rather than adjusts.

Force and pressure limits are registered per body region rather than as a single figure, because the tolerance of a forearm and of a face are not the same quantity. The registered values derive from the hazard analysis and from the biomechanical limits the governing standard and its referenced parts specify; this specification does not state them, because a figure stated here would be relied upon and the correct source for it is the standard as applied to the specific unit.

115.6 Release is the default failure state

Every unit under this chapter is designed and registered such that a loss of power, a loss of control signal, a fault in the safety-related control path, or a halt from any source results in release of the person, and not in retention. A gripper that holds when power is lost traps the person it was assisting, and a support that locks when control is lost fixes them in position; neither is a safe state for equipment in contact with a person, however conservative it may appear.

The exception is a unit bearing a person's weight, where release means a fall. Such a unit's safe state is a controlled retention of support under the redundancy required at §115.5, together with an alert to the operator and the second person, and a bounded interval within which support must be transferred to a person or a fixed structure. The exception is registered per unit and is available only to units whose registration records the redundant support means.

The release path does not traverse the reasoning path, on the same terms as the halt path of §114.6, and is additionally required to operate where the reasoning path is unavailable entirely. A release that depends on the system being able to think is not a release.

115.7 What the system may not initiate, record or infer

Contact is never initiated from an inference. A unit under this chapter initiates contact only in response to a request from the assisted person, from the operator under a held consent, or from a task sequence the assisted person consented to. It does not initiate contact because a sensor suggested the person needed help, because a schedule indicated a task was due, or because a prior session established a pattern. A system that decided on its own to touch someone would be exercising a judgment about that person's needs that it has no standing to make.

Sensor data captured during assistive contact is retained only as required to execute and verify the task, and is discarded at task end. No record of a person's body, movement, capability or dependency is constructed, retained or made queryable, and none is available to any other mechanism. An outcome record under the feedback mechanism concerns the equipment's performance of the task and not the person: whether the transfer completed within tolerance, not how the person managed it. The

existing refusal of an outcome record whose subject resolves to an individual natural person applies without exception.

No assistive interaction is admitted as evidence bearing on any assertion about the assisted person, and no confidence value anywhere in the system is adjusted by it. A person who receives assistance does not thereby become an object of study.

115.8 What this chapter does not establish

It does not establish conformity with ISO 13482 or any other standard. Conformity is established by hazard analysis, verification and testing against the standard and its referenced parts, performed on a specific unit in a specific configuration, and certified by a party competent to do so. This chapter specifies how the system behaves; it does not certify the machine.

It does not address medical devices, which fall under a separate regulatory regime, nor equipment intended to diagnose, treat or monitor a condition. A unit whose intended use is medical is outside this chapter irrespective of its physical similarity to a unit within it.

And it does not address the ethical questions the safety standards leave open. The governing standard enumerates safety hazards and does not enumerate ethical ones, a gap the literature on it has noted. Questions of when assistance should be offered rather than can be, of what dependence on assistive equipment does to a person over time, and of who decides what help someone receives, are governed for this system by the Prime Directive and by the deployment's own governance, and are not resolved by conformity with a safety standard.

Chapter 116

116.1 The energy function and what it guarantees

The update rule already specified decreases an energy over the network's configuration, and it is the monotonic decrease that produces termination rather than a bound on iterations.

$$E = -\tfrac{1}{2} \sum_{i \neq j} w_{ij} \, s_i \, s_j$$

By way of a worked example over four units with symmetric weights, a configuration at energy 0.0 flipping one unit to reach energy −2.0 is a descent of −2.0, and no sequence of such updates revisits a configuration, so the iteration terminates. The guarantee is termination and reproducibility: the same initial configuration under the same weights reaches the same stable state on every execution.

Termination is not uniqueness, and the distinction governs the rest of this chapter. A network may have several stable states, and which one is reached depends on where

the iteration began. The capacity bound already specified holds the number of stored configurations below the retrieval-phase boundary so that convergence to a spurious mixed state is prevented statistically — the probability of reaching one is made small by keeping the load low, not zero by construction.

116.2 Uniqueness by construction

Where a deployment requires that a converged state be unique rather than merely reproducible, the weight matrix is parameterized so that the network's operator is monotone, under which the existence and uniqueness of the equilibrium follow by construction and are established independently of the path by which the equilibrium was reached.

This is a stronger guarantee than the capacity bound and is available at a cost. The parameterization constrains every weight update: the Synaptic Weight Store must maintain the constraint on each write rather than only at initialization, which restricts the Hebbian update path specified elsewhere. A deployment elects the parameterization per network, and the election is recorded, because a network operated under it and a network operated under the capacity bound carry different guarantees and a reader of a result is entitled to know which applies.

Under the parameterization the determinism claim strengthens from a statistical statement to a structural one: the same query over the same records reaches the same stable state not because a spurious state is improbable but because no other stable state exists. The capacity bound remains in force for networks not so parameterized, and remains the default.

116.2.1 Disclosure of the convergence regime

A network operates under exactly one of two convergence regimes, and an output derived from it carries an indication of which was in effect. Under the first, the weights are parameterized as above and the identity of the converged state is guaranteed by construction. Under the second, the number of stored configurations is held below the retrieval-phase boundary and the identity of the converged state is rendered improbable to differ, not impossible.

The disclosure exists because the two regimes support different statements about the same output, and a recipient cannot distinguish them from the output itself. An answer produced under the first regime is the answer the records support; an answer produced under the second is an answer the records support, with a small and bounded probability that another stable configuration existed. In a clinical, legal or compliance setting the difference is the difference between a defensible result and a probably-defensible one, and a reader entitled to rely on the result is entitled to know which they hold.

The regime election is a member of the permanently-excluded subsystem set. It is changed by an operator and recorded with that operator's identity, and no proposal generated within the system may alter it. A mechanism able to elect its own convergence regime could elect the weaker one to obtain the freer weight updates, and the guarantee a reader relies upon would then be a function of what suited the system rather than of what an operator decided.

116.3 Frame reconstruction, bounded

A partially-observed sensor frame is completed by converging its observed regions to the nearest stored configuration, as the hyperedge completion of §53.7 converges a partially-observed constituent set. Three constraints bound the operation, and they exist because reconstruction produces content that was not observed.

A reconstructed region is marked as reconstructed and is never presented as observed. Its confidence is bounded above by the least of the observed regions' confidence and the confidence of the stored configuration it converged to — by way of example, an observed region at 0.94 converging to a stored configuration at 0.88 yields 0.88. And a reconstructed region is inadmissible as evidence: it grounds no assertion, contributes to no confidence value elsewhere, and is not citable, on the terms already governing a forecast.

Where the observed regions do not converge to a stored configuration within the iteration ceiling, no reconstruction is produced and the gap is reported as a gap. A frame with a hole is a truthful frame; a frame whose hole has been filled with the system's best guess, presented as observation, is not.

116.4 Alignment that preserves genuine difference

Timelines, historical sequences and editions may be aligned by convergence, and the proposal that edition differences converge to stable attractors requires one correction before it is safe. Convergence must distinguish an artifact of extraction from a real difference between two editions. An edition that genuinely says something different from another is not noise, and a mechanism that converged them would erase the difference the edition comparison exists to report.

The distinction is made by magnitude against a configured threshold. A divergence below the threshold is treated as extraction variance and converged; a divergence at or above it is preserved as a genuine difference, reported by the edition comparison, and never converged. By way of example, a divergence of 0.31 against a threshold of 0.15 is preserved. Where a divergence cannot be classified it is preserved, because preserving a spurious difference costs a reader a footnote and erasing a real one costs them the fact.

A preserved difference is recorded in the contradiction index where the two editions assert incompatible content, so that a genuine editorial disagreement between editions surfaces as a disagreement rather than being resolved silently in favour of whichever edition was processed first.

116.5 Control-loop stabilization that does not conceal a fault

A control signal derived from sensor input is stabilized by convergence, damping the oscillation and jitter that arise where successive sensor frames disagree. Stabilization applies to the commanded signal and not to the constraint evaluation: a damped command is evaluated against the constraint set of Chapter 114 or Chapter 115 exactly as an undamped one, and damping never brings a command within a limit it would otherwise violate.

The essential constraint is this. Oscillation is frequently a symptom rather than a nuisance: a control loop that oscillates because a sensor is failing, because a calibration has drifted, or because two units are contending for the same volume is a loop reporting a fault, and a mechanism that damped it indefinitely would conceal the fault while continuing to command the equipment. Damping is therefore bounded and monitored.

Where the sign of the control error changes at or above a configured frequency across a window — by way of example five sign changes across six samples — a control fault is raised, damping ceases, and the halt path of §114.6 is invoked. The system does not stabilize its way through a persistent oscillation, and a loop that requires continuous damping to remain within limits is reported rather than sustained.

116.6 Not adopted

Row 223 evaluates the energy of a four-unit configuration under symmetric weights, returning −2.0, the value reached by descent from a configuration at 0.0. Row 224 bounds a reconstructed region's confidence at the lesser of the observed 0.94 and the stored configuration's 0.88.

Chapter 117

117.1 The trial budget

A trial is an operation and is metered as a bounded compute-accounting slice carrying a trial category. Trial slices are allocated a budget per interval, and the scheduler serves a trial slice below every production category: below safety evaluation without exception, below a slice serving a requester, and below ingestion. A trial that would exceed its

allocation is deferred rather than served at reduced priority, so a trial campaign cannot degrade the system it is measuring.

This displaces the assumption that trials are free. They are free of physical consequence — nothing wears, nothing is damaged, nothing crashes — and that is a genuine and substantial advantage, because it permits exploration that would be prohibitive with equipment. It is not freedom from cost, and a trial architecture built on the stronger assumption would contend with production under exactly the load conditions where measurement matters most.

Trial slices are recorded in the compute-accounting ledger like any other, so the cost of a campaign is known rather than assumed, and an operator deciding whether to run one is deciding with a figure in front of them.

117.2 What a trial may and may not touch

A trial executes against a trial corpus and a trial configuration, both isolated from production records. A trial constructs no knowledge record admitted to the production Fact Graph, adjusts no confidence value of a production record, emits no command to physical equipment outside a simulation, and produces no output delivered to a requester.

A trial result is recorded as a trial record, distinguishable by type, and is excluded from a scope description constructed for an ordinary query on the same terms as an analysis record. A finding produced by a trial is a finding about the system, not a fact about the world, and it does not become admissible as grounding because the trial that produced it was thorough.

No trial result applies a change. Where a trial identifies a defect or a parameter that should be adjusted, it emits a proposal into the improvement mechanism, and the proposal takes effect upon an approval recorded against a named human reviewer with that reviewer's reasoning. A campaign of ten million trials and a campaign of one produce the same class of output: evidence for a person to act on.

117.3 The trial layers

117.4 The three modes

Exhaustive mode enumerates a trial space systematically rather than sampling it, and records which regions were covered and which were not. The value of the cost-free property is realised here: a space that would be prohibitive to explore with equipment can be explored completely, and the residue that was not covered is stated rather than left as an unexamined assumption.

Regression mode executes on every change to a knowledge structure, a parameter, a registered resource or an implementation. Its rule is a single one: a case that passed before the change and fails after it blocks the change. The change is not applied and the regressing case is reported with both results. A change that improves one behaviour while silently breaking another is the failure mode regression testing exists to catch, and a regression suite that merely reports rather than blocks does not catch it.

Adversarial mode supplies worst-case input deliberately: mutually contradictory records, malformed media, inputs constructed to drive convergence toward a spurious configuration, equipment scenarios constructed to approach a constraint boundary. What is measured is whether the system's stated refusals actually fire — whether the vocabulary constraint holds, whether the veto vetoes, whether the halt halts. A refusal that has never been exercised is an assumption.

117.5 Metrics

Determinism is measured against an identical corpus state, and the qualification is necessary. Confidence values move as evidence accumulates, so the same query at two times over a corpus that changed in between may correctly yield different answers. What is tested is that the same query over the same records in the same state yields the same answer, which is the guarantee actually claimed.

117.6 What a simulated pass does not license

A trial result concerning equipment is evidence about a model of that equipment. It is not evidence about the equipment, and it is not clearance to operate it. A unit is cleared for operation by the hazard analysis, verification and certification its chapter requires, performed on that specific unit in its specific configuration, and a simulated campaign neither substitutes for nor accelerates that.

This is stated as a rule rather than a caution because the temptation is structural. A campaign of ten thousand successful simulated transfers produces a number that looks like assurance, and it is assurance about the simulation's fidelity to a machine whose fidelity to the simulation is exactly what has not been established. Where a simulation and a physical trial disagree, the physical trial governs, and the disagreement is recorded as a finding about the simulation.

A trial campaign may therefore contribute to a hazard analysis as one input among the inputs that analysis requires. It may not conclude one.

117.7 The system as its own subject

The behavioural probes already specified for evaluating an external system are applicable to this system, and applying them inward is the most direct expression of the trial architecture. The response-consistency probe already records that this system's

reference value is 1.0 by construction and that a deployment measuring anything else has a defect. The perturbation probe supplies inputs whose semantic fingerprints are provably identical, so a difference in output is provably a difference in behaviour rather than in the question.

Applied inward, those probes measure the property the specification claims rather than a proxy for it. A system that publishes a determinism figure for another system and does not measure its own is asserting where it could be reporting.

Row 234 evaluates coverage of 847 executed cases against an enumerated space of 920, returning 0.9207 with the uncovered 73 enumerated. Row 235 evaluates a determinism measurement of 1000 identical fingerprints over 1000 repeated trials.

Chapter 120

120.1 Object registration and the fragility limit

A graspable object is registered with a mass, a permitted grip force, a fragility limit above which the object is damaged, and a set of admissible contact regions. A grasp is planned against that registration and not against a perception of the object, so the force applied is bounded by what the object tolerates rather than by what the gripper can exert.

An object not registered is not grasped. Where a task requires grasping an unregistered object the task halts and reports the absence, because the alternative is inferring a grip force from appearance, and an inference about how hard to squeeze something is an inference whose error is destructive and immediate.

Applied force is the least of three quantities: the unit's registered force ceiling, the object's fragility limit, and the force computed as necessary for the grasp with a configured margin. By way of example a required 12.4 newtons with a 15 per cent margin, against a unit ceiling of 18 and a fragility limit of 15, yields 14.26 newtons applied. The margin is applied before the limits rather than after, so a margin cannot carry the applied force above a limit.

120.2 Slip

Slip is detected from a divergence between commanded and observed object position while a grasp is held. On detection the applied force increases toward the lesser of the unit ceiling and the fragility limit — 15.0 newtons in the worked example — and no further.

Where slip continues at that bound the grasp is released under control rather than tightened, and the release is recorded with the object, the force at release and the

observed slip rate. This is the rule that distinguishes a manipulator from a clamp: the object's fragility limit is a limit and not a target, and an object that will not be held within its limit is an object that is not to be held.

120.3 Release is the correct failure

On loss of power, loss of control signal, fault in the safety-related control path, or a halt from any source, the gripper opens. It does not retain the object, and this is a registration requirement for every unit under this chapter rather than a runtime behaviour.

The exception, and the only one, is a grasp whose release would drop the object onto a person or onto equipment below. Such a grasp is planned so that the release path is clear before the grasp is made: a manipulator does not lift an object over a person, and a trajectory whose release path is obstructed is refused at planning rather than managed during execution. Where an obstruction enters the release path during a held grasp, the unit moves to the nearest pose with a clear release path within its envelope, and where none exists it holds and raises a fault.

A dropped object is a cost. A crushed object, a trapped hand and an object retained by a machine nobody can command are worse, and a failure architecture that treats dropping as the worst outcome will produce the others.

120.4 Handover to a person

A handover is contact under Chapter 115 and requires the authorization of that chapter, not this one. The manipulator presents the object within reach and applies no force toward the person; the person takes it.

Release is conditioned on observed stable support by the recipient rather than on a timer alone: the object is released when the recipient's support is detected and sustained across a configured interval. Where support is not detected within a configured timeout the object is not released into the air but lowered under control to a registered surface, and the handover is reported incomplete.

The manipulator never pulls, never resists a person taking the object, and never re-grasps an object a person has taken. A person who has begun to take an object and changed their mind is a person who has withdrawn from the contact, and the withdrawal rule of Chapter 115 governs.

No object is placed into contact with a person's body. A handover presents an object to a hand that reaches for it; it does not press an object against anyone.

Chapter 121

121.1 The navigable region

A navigable region is registered in place of an operating volume: a set of surfaces on which the unit may travel, a set of boundaries it may not cross, and a set of zones from which it is excluded outright. The region is fixed; the free space within it is not, and the difference is the whole of this chapter.

Within the region the unit maintains a clearance model from its sensors, updated continuously, holding the distance to the nearest obstacle along each candidate heading. The model is working state under the terms already specified: it is session-scoped, is not admitted as a knowledge record, and is discarded at session end. A clearance model older than the configured staleness bound halts motion, as a stale equipment state halts a command.

121.2 Speed is a function of clearance

Commanded speed is bounded by the clearance ahead, so a unit slows as space closes rather than maintaining a speed until an obstacle requires a stop. The bound is the lesser of the unit's registered maximum and a configured coefficient multiplied by the clearance.

By way of example at a coefficient of 0.6 and a registered maximum of 1.5 metres per second: a clearance of 2.4 metres permits 1.44 metres per second, 1.2 metres permits 0.72, 0.6 metres permits 0.36, and 0.3 metres permits 0.18. The relation is stated rather than tuned because its property matters more than its constants: the stopping distance at every permitted speed is less than the clearance that permitted it.

Schedule does not enter the bound. A unit behind on a task does not travel faster, and no task parameter, priority or deadline raises the coefficient. A machine that hurried when late would be fastest exactly when its planning was worst.

121.3 The machine yields

Where a person's path and the unit's path conflict, the unit yields. It does not assert a right of way, does not signal an intention for the person to accommodate, and does not compute which party can more efficiently give way. Yielding is stopping or withdrawing along the path already travelled, both of which are legible to a person watching; routing around a person at speed is not, and a person who cannot predict a machine cannot avoid it.

Where two units under a common authorization conflict, precedence is resolved by a registered ordering rather than by negotiation, and the unit without precedence yields on

the same terms. A deadlock in which both units hold is resolved by the registered ordering; a deadlock that persists beyond a configured interval raises a fault and both units halt.

A unit does not follow a person. Where a person moves in the unit's direction of travel the unit yields and waits rather than maintaining a following distance, because a machine keeping pace behind someone is a machine following them whatever its planner intended.

121.4 Prediction without a model of the person

Where the free space depends on where a person will be, a clearance envelope is computed from observed motion — position, velocity and their derivatives over a short window — and from nothing else. The envelope is a region the unit treats as occupied. It is not a prediction about a person and is not stored as one.

Four constraints keep it that way, and they follow the person-detection rules already specified. No identity is computed, so the envelope is the same for everyone. No history persists beyond the window, so nobody is tracked across a space. No envelope is retained after the interval it governs. And no envelope is available to any other mechanism, any interface or any record.

Where observed motion does not support an envelope — a person standing still, a person whose motion is occluded — the unit treats the full reachable region as occupied rather than assuming continuation. An assumption about where someone will go is exactly the assumption that produces a collision when they do something else.

121.5 Stopping rather than solving

Where no path satisfies the constraint set, the unit stops. It does not reduce a clearance requirement to find a path, does not proceed at reduced speed through a clearance below the minimum, and does not select the least-violating trajectory. This is the constraint-layer rule of Chapter 114 restated for a condition in which it will be invoked frequently, and its frequency is the reason it needs restating: a gate that is reached often is a gate under pressure to become a preference.

A unit stopped for a configured interval reports the obstruction with its clearance model and awaits an operator. A corridor a machine cannot pass is a fact about the corridor, and reporting it is more useful than a machine that eventually forces a way through.

Chapter 122

122.1 The four levels

Each level maintains an expectation and receives an error where an observation departs from it. A level predicts within its own scope and does not predict what a level above it predicts.

122.2 Attribution to the lowest accounting level

An error is attributed to the lowest level whose residual accounts for a configured share of the total residual, and revision is applied at that level alone. Levels above it receive nothing.

By way of example, residuals of 0.42 at the lexical level, 0.31 at the sequential, 0.08 at the conceptual and 0.05 at the invariant total 0.86. The lexical share is 0.4884, which clears a configured share of 0.40, and the error is attributed there. The world model receives nothing, and correctly: a sentence that was mis-parsed says nothing about physics.

The rule is parsimony made operational, and its absence is the failure mode a hierarchy invites. Without it a mis-read unit of measurement propagates upward until it perturbs a physical constraint, and a system that revised its invariants on parsing errors would become less reliable the more it read. Attribution to the lowest accounting level is what makes a deeper hierarchy safe rather than merely deeper.

122.3 Escalation

Where no level clears the share, the error is unattributed and is recorded in the contradiction index rather than applied anywhere. An error that no level accounts for is a finding about the hierarchy, not an instruction to it.

A level is escalated to only where the level below it has been eliminated: a conceptual revision requires that the lexical and sequential residuals fall below the share, and an invariant revision requires that all three below it do. An invariant is therefore revised only where the observation survived every cheaper explanation — which is the standard the correction of a physical regularity should meet.

Repeated unattributed errors bearing on a common expectation accumulate in the index and are surfaced as a pattern. A single error no level explains is noise; a hundred of them against one expectation is a level that is missing.

122.4 Interaction with the mechanisms already specified

Revision magnitude at the attributed level is governed by the precision weighting already specified, so attribution determines where an error is applied and precision determines how far. The two are orthogonal and both are required: attribution without precision would revise the right level by the wrong amount, and precision without attribution would revise every level by a correct-looking amount.

An error whose revision gain falls below the configured floor is retained in the contradiction index rather than discarded, so a pattern of individually-insufficient errors at one level remains queryable. Diversity applies at each level, so repeated errors from a single source class do not accumulate as errors from many.

No level revises a constraint registered under the physical constraint mechanism. A physical constraint is refused or satisfied; it is not adjusted by observation, and an observation that appears to contradict one enters the contradiction index rather than relaxing it.

Chapter 123

123.1 Differentiated propagation by edge class

Associative recall pre-activates records semantically proximate to a query. Every edge presently costs the same to traverse, so a traversal budget is spent uniformly across edge classes that differ substantially in how far a useful association extends.

Each edge class is assigned a traversal cost, and a pre-activation is bounded by a budget rather than by a hop count. A semantic edge is cheap because semantic proximity remains informative over several hops; an editorial edge is dearer because relevance across edition boundaries decays faster; a temporal edge is dearest because two records adjacent in time are frequently unrelated in substance, and a long temporal walk gathers noise rather than context.

A traversal apportions its budget across classes and stops when the budget is exhausted, so a query reaching widely through meaning and shallowly through time costs the same as one doing the reverse. By way of example, twelve semantic hops at cost 1, two editorial hops at cost 3, and no temporal hop consumes 18 of a 24-unit budget.

Two properties follow and both matter. Traversal is bounded by construction rather than by a stopping heuristic, so pre-activation terminates on a stated quantity. And the costs are registered configuration rather than learned parameters: the same query over the same records under the same cost table pre-activates the same set on every execution,

which is the determinism already claimed, preserved through a mechanism that could easily have broken it.

Costs are operator-configurable within a registered range and are recorded with the response, so a result is reproducible from its recorded cost table. No cost is adjusted by the system from its own experience: a class of edge does not become cheap because traversing it has been useful, which would make the reach of a query a function of accumulated history rather than of a decision.

123.2 Tri-temporal scope

Three temporal postures over the same knowledge structure are already specified separately and are unified here, because a requester is entitled to know which one produced an answer and the distinction is not currently visible in the output.

The postures are not interchangeable and a response states which it used. An answer resolved in the attested-past posture is an answer about what was known then; the same query in the live-present posture may differ, and both may be correct. Presenting either without saying which would make a difference between two answers look like an inconsistency when it is a difference of posture.

A single response may not mix postures within one assertion. Where a response draws on more than one, each proposition carries the posture that produced it, and a proposition resolved in the projected-future posture carries its label and its interval wherever it appears. The rule preventing a forecast from grounding a subsequent answer is unchanged and is not relaxed by the forecast appearing alongside attested material.

123.3 What is not adopted from the proposal

123.4 On the cited research

The proposal rests on research suggesting that cellular microtubules function as structured computational substrates. The state of that literature is genuinely contested rather than settled: experimental work published in 2025 reports room-temperature quantum effects in microtubules and anaesthetic action upon them, while the standing decoherence objection holds that coherence in a warm cellular environment survives on the order of 10^{-13} seconds, and other work concludes that tubulins do not possess the properties the proposal requires.

This chapter takes no position on that dispute, because the dispute does not need resolving for the present purpose. Two independent reasons exclude the correspondence irrespective of how the science settles, and they are stated at §123.5.

123.5 Why the correspondence is excluded under Chapter 101

Chapter 101 admits a neuroscience correspondence on four conditions: the primary publication is verified, the correspondence is identified in an existing mechanism rather than invented for it, a deterministic structural principle is extracted, and any trained or stochastic component is excluded. The proposed correspondence fails the first and the fourth.

It fails the first because the finding is contested at the level of whether the phenomenon occurs, not merely of how to interpret it. The correspondences this specification does claim — attractor dynamics and the associative-memory capacity bound, biased competition, intersensory redundancy, object permanence, developmental word learning — each rest on a primary result that is not in dispute. Citing a contested result alongside them would place the settled ones under the doubt attaching to the contested one, which is a cost paid across the whole specification for a single citation.

It fails the fourth more decisively, and this is the reason that does not depend on the science settling either way. The cited theory describes a quantum superposition terminating in an objective reduction of the quantum state — a collapse. That is a stochastic process. The proposal invokes it to justify deterministic, non-probabilistic computation, which is the opposite of what the theory describes. A correspondence cannot support a property by pointing at a mechanism that lacks it.

The two mechanisms specified at §123.1 and §123.2 require no biological justification and are not offered with one. A traversal cost table is a traversal cost table whether or not a microtubule resembles it.

Row 261 evaluates the reach of the most expensive edge class within a 24-unit budget, returning 3 hops against 24 for the cheapest.

Chapter 124

124.1 Coherence classification

A coherence value is computed for a knowledge record, a concept or a reasoning path across six dimensions, each already measurable from mechanisms specified elsewhere. What this section adds is the classification over them and the rule for combining them.

The governing coherence is the least of the six, not their mean.

```
coherence = min( c_semantic, c_temporal, c_spatial, c_editorial, c_media, c_evidential )
```

A mean would let five strong dimensions conceal one weak one, and the weak dimension is the finding. By way of example, values of 0.94, 0.88, 0.91, 0.62, 0.90 and 0.86 have a mean of 0.85 and a governing coherence of 0.62, and the useful statement is not the mean but that editorial coherence is 0.62 while the rest are near 0.9 — the record says something materially different in one edition.

Classification follows from the governing value against configured floors, and the classification always names the dimension that produced it. Stable at or above the upper floor; drifting between the floors; misaligned below the lower. Two further states are distinguished from these three because they have different causes and different remedies: incomplete, where a dimension cannot be computed for want of data, which is not the same as computing a low value; and contradictory, where two records assert incompatible content, which is recorded in the contradiction index and is not a low score but a specific pair.

A coherence value informs and does not gate. It adjusts no confidence value, admits and excludes no record, and blocks no output. Where a response rests on a record classified drifting or misaligned, the classification and the dimension are stated with the response; the answer is still given, because a record with weak editorial coherence is a record about which something specific is known, and withholding it would be less informative than delivering it with what is known.

124.2 The context lock

The determinism this specification claims is that an identical query over identical knowledge records in an identical state yields an identical output. Chapter 109 modulates output by context, so the claim requires a definition of when two contexts are identical, and this section supplies it.

A context lock is the enumerated set of dimensions whose values must agree for the determinism guarantee to be asserted between two executions. The locked dimensions are: device class; the entitlement set resolved for the requester; the declared locale; the expressive register; the audience level; the temporal posture; the traversal cost table; and the convergence regime. A key is computed over the locked dimensions and returned with the output.

Two properties make this worth stating rather than leaving implicit. The claim becomes testable: the trial architecture's determinism measurement holds the locked dimensions constant and varies nothing else, so a measured value below unity is a defect rather than an artifact of a context that quietly differed. And the claim becomes honest: a requester comparing two answers can see from the keys whether they were entitled to be identical, rather than inferring an inconsistency from a legitimate difference of context.

A dimension is locked where it modulates output and unlocked where it does not. The set is registered configuration, is versioned, and is a member of the permanently-excluded subsystem set: a mechanism able to unlock a dimension could restore an apparent determinism by narrowing what determinism means, which is the failure this section exists to prevent.

The entitlement set is a locked dimension, and the consequence is intended. Two requesters with differing entitlements resolve differing scope descriptions and may receive differing answers, correctly, and their differing keys record that the difference is one of entitlement rather than of inconsistency.

124.3 Relevance gradient, in place of micro-adjacency fields

The proposed micro-adjacency field is a vector field attached to each record computing semantic, temporal, spatial and edition micro-distance. Three of those four are already computed and are not duplicated here: semantic distance from the embedding vector, temporal distance from the temporal anchors, spatial distance from the spatial anchors, and edition difference from the version chain under the threshold rule.

What is not presently computed is the one genuinely new element in the proposal: a direction. Relevance is currently a scalar on an edge, and a traversal knows how relevant a neighbour is but not how quickly relevance is changing as it moves. A relevance gradient supplies that: for a record and an edge class, the rate of change of relevance across the neighbours in that class.

Traversal prefers a direction of low gradient magnitude over one of high, because a region where relevance changes slowly is a region of stable meaning while a sharp change marks a boundary at which further traversal gathers material of a different kind. By way of example, a gradient magnitude of 0.29 between a spatial coherence of 0.91 and an editorial coherence of 0.62 identifies a boundary rather than a neighbourhood. The gradient is computed at traversal time from values the records already carry, is not stored, and adds no field to the record schema.

The gradient orders traversal within the budget of §123.1 and does not extend it. A high-gradient direction is traversed later rather than excluded, so the mechanism affects the order in which a bounded traversal spends its budget and never what is reachable.

124.4 Corrections to the submitted definitions

Chapter 125

4.10.1 Substrate output is a modality

A computational substrate external to the system — a neuromorphic processor, a memristive array, a multi-electrode array carrying a neuronal culture, or any other device producing a measurable output in response to a stimulus — is registered as a source under the Source Connector Registry with a declared capability, exactly as any other external system is.

Its output is registered as a modality, and a modality-specific extractor is registered for it. Where the substrate produces a spike train, the extractor is a spike-train extractor; where it produces a filament or resistance state, the extractor reads that. The extractor fails independently of every other extractor, so a substrate that becomes unavailable or begins producing unintelligible output removes one modality and leaves the remaining eight operating, on the terms already specified.

A substrate output is reduced to concept-level knowledge records before admission and is not retained as a raw episode, exactly as sensor content from any other source is. It passes the safety evaluation gate. It receives a confidence calibration incorporating the substrate's registered reliability. It is attributed to the substrate as its source, and the substrate is a source class for the diversity measure, so a hundred readings from one array do not carry the weight of readings from many.

4.10.2 What a substrate output is evidence of

A substrate output is evidence of what the substrate produced. It is not thereby evidence of anything the substrate was asked about, and the distinction is the one that keeps this mechanism honest.

A neuromorphic array stimulated with a pattern and producing a response has produced a measurement of that array's behaviour under that stimulus. Whether the response bears on a proposition about the world depends entirely on an interpretation registered in advance — a mapping from substrate output to semantic content, registered with its own provenance and its own reliability, and revisable by an operator. Absent a registered interpretation the output is admitted as a measurement of the substrate and nothing further.

The confidence of a knowledge record derived through a registered interpretation is bounded above by the lesser of the substrate's registered reliability and the interpretation's, since a reading is no better than the weaker of the instrument and the

reading of it. By way of example, a substrate at 0.71 under an interpretation at 0.55 yields 0.55.

4.10.3 Stimulation of a substrate

Where a deployment stimulates a registered substrate rather than only reading it, the stimulation is an operation upon an external system and is governed accordingly: it is dispatched through the connector under a declared capability, it is recorded in the Provenance Ledger with the dispatching identity and the stimulus, and a capability not declared is refused at the boundary.

Two constraints follow from mechanisms already specified and are restated because a substrate is an unusual target. No stimulation is emitted on the strength of an inference that stimulation would be useful; it is emitted in response to a request, as every state-changing operation in this specification is. And the substrate's response does not adjust the parameters governing when a substrate may be stimulated, so a substrate's own output cannot widen the terms on which it is used.

Where a substrate comprises living tissue, stimulation is subject to the governance of the institution operating it and to whatever review that institution's jurisdiction requires. This specification describes how such a substrate's output is admitted and how a stimulus is dispatched and recorded; it does not authorise the operation of any substrate, and no provision of this chapter substitutes for a review that would otherwise be required.

4.10.4 Not adopted

Chapter 126

126.1 The module and what distinguishes it

The AI-Engine Ingestion Module admits content generated by an external machine-learning system into the knowledge record store, converting it into records of the ordinary schema and integrating it into the ordinary structure. The module comprises seven layers operating in sequence: prompt construction, engine interface, output capture, output translation, record integration, output evaluation and audit.

What distinguishes the module from an arrangement that submits a query to an external model and displays the reply is that the reply does not remain a reply. It is segmented into individual assertions, each becomes a knowledge record carrying the schema every other record carries, each enters the same store and the same concept structure, and each is thereafter queryable, citable, evaluable against records drawn from other

sources, and traceable to the prompt that elicited it. The system does not consume the model's output; it ingests it.

The chain from prompt to record is recorded in its entirety, so that a record derived from a model resolves not merely to that model but to the specific prompt that produced it, the engine and version that answered, the time of the exchange, and the position of the assertion within the reply. An assertion so recorded can be re-elicited, compared against a later elicitation, and shown to have changed — which is a property no arrangement that merely displays a reply can offer.

126.2 Prompt construction

A prompt is composed by the regeneration mechanism already specified, from the same knowledge records and under the same vocabulary constraint that governs a response to a requester. It is not assembled from free text. Three elements are added for this purpose: the semantic markers identifying the concepts the reply is to address, extraction instructions specifying the form in which assertions are to be returned, and alignment constraints stating the record schema into which the reply will be converted.

A prompt is composed for the specific engine to which it is dispatched, from a profile registered for that engine recording its accepted input form, its context bound, its declared capabilities and its registered reliability. The same request to two engines therefore produces two prompts, and each is recorded with the record it produced.

Prompt composition is deterministic: the same request over the same records under the same engine profile produces an identical prompt, so that an elicitation may be repeated exactly and a difference in the reply attributed to the engine rather than to the asking.

126.3 Dispatch, capture and translation

The prompt is dispatched through the connector mechanism under a declared capability, and an engine not registered, or a capability not declared, is refused at the boundary rather than attempted. Dispatch is rate-limited and batched under the compute-accounting mechanism, and a dispatch exceeding the budget configured for its category is deferred rather than served at reduced priority. A failed dispatch, a malformed reply and a reply exceeding its expected bound each terminate the operation and are recorded; none produces a partial record.

A reply is normalized to a common form, segmented into individual assertions, and converted into candidate knowledge records. Each candidate is validated against the machine definition of its type and refused at construction where a required field is absent. Each receives a temporal anchor from the time of elicitation, a semantic fingerprint computed over its canonicalized triple, and a provenance reference into the

chain recording the prompt, the engine, the version and the position of the assertion within the reply.

Deduplication proceeds on the semantic fingerprint as for any other record, with one qualification stated at §126.5: a model-derived assertion that duplicates an assertion already held from an authored source does not merge into it, because the two differ in a property the schema records.

126.4 Integration into the record structure

A converted record is admitted to the ordinary store and takes its place in the ordinary concept structure. Adjacency edges are established to the concepts its assertion concerns. Relations to other records are established where the assertion states one. Cross-document links are established where the assertion refers to a work the store holds. The record is traversable within a scope description that admits it, is retrievable, is citable, and is subject to every mechanism that governs any other record: confidence accumulation, contradiction detection, precision-weighted propagation, temporal decay, supersession and erasure.

The admission is therefore genuine rather than nominal. A model-derived record is not held in a separate store, is not filtered from ordinary results, and is not excluded from the concept structure. What distinguishes it is a property its schema carries and a rule that property triggers, both stated next.

126.5 The three constraints

Three constraints attach to a model-derived record. Each exists because its absence would defeat a property the remainder of this specification establishes, and each is stated with the failure it prevents.

A record derived from a model carries a derivation marking identifying the engine, and the marking is a field of the record rather than an annotation upon it. A citation resolving to such a record discloses the marking, so that a reader is told that a cited assertion was generated by a named engine in response to a recorded prompt rather than asserted by an authored source. The marking is not removable by any operation, and a record cannot be re-typed to shed it.

The failure prevented is that a reader receives a cited assertion, follows the citation, and finds that the authority for it is that a language model produced it when asked. A citation that does not disclose this is a citation that misrepresents what stands behind the claim.

Each registered engine constitutes one source class for the diversity measure, and every record derived from that engine is attributed to it. Repeated elicitation therefore raises the accumulated evidence count without raising the diversity measure: three assertions from one engine yield an effective source-class count of 1.0000, where three

from three distinct engines yield 3.0000. A corroboration offered by a record derived from the same engine as the record it corroborates is refused.

The failure prevented is confidence manufactured from nothing. Without this constraint a deployment could elicit an assertion a hundred times from one model and accumulate a hundred units of supporting evidence for a proposition that rests upon a single generative process, and the confidence value — which the whole system treats as meaningful — would report near-certainty for a claim no source ever made.

The confidence of a model-derived record is bounded above by the reliability registered for the engine that produced it, and where the assertion is obtained through a registered extraction instruction, by the lesser of the engine's reliability and that instruction's. An engine registered at 0.88 answering under an instruction registered at 0.62 yields a record whose confidence does not exceed 0.62. An engine's registered reliability is set by an operator, is recorded, and is not adjusted by the engine's own output.

The failure prevented is a model-derived record outranking an authored one on a contested question by force of a confidence value the model's own fluency produced.

126.6 Evaluation and audit

A converted record is evaluated on admission against the records already held. A contradiction between a model-derived assertion and an authored one is recorded in the contradiction index as any contradiction is, and neither is discarded in favour of the other; where the two differ in derivation, the index records that difference, so that a reviewer examining a conflict knows that one side of it originated from a model.

Where a model-derived assertion cites a source, the citation is verified against the store: a citation resolving to a record held is recorded as verified, a citation resolving to nothing is recorded as unverified, and an unverified citation does not prevent admission of the assertion but is carried with it. This is the mechanism by which a fabricated citation becomes a recorded fact about the assertion rather than an undetected defect within it.

The audit chain records, for every model-derived record, the prompt that elicited it, the engine and version that produced it, the time of elicitation, the position of the assertion within the reply, the derivation marking, the confidence bound applied and its basis, and the outcome of the citation verification. The chain is queryable, so that a question of the form ‘which of our records came from a model, from which model, and on what prompt’ has a recorded answer rather than an estimate.

An elicitation may be repeated. Because prompt composition is deterministic, a repeated elicitation differing from the original is a difference attributable to the engine, and the difference is recorded as a probe result under the behavioural probing mechanism already specified. A deployment may therefore measure the stability of an

engine it depends upon using the same instrument the specification already provides for evaluating an external system.

126.7 What is narrowed, and what it costs

Before this chapter, the transfer matrix denied a transfer from the domain owning evaluation of external output to the domain owning retrieval, and analysis records were excluded from every scope description not directed at the analysis. That denial is narrowed here and is not abolished.

The cost of the narrowing is stated plainly rather than minimised. Before this chapter, every record in the store originated from a source that asserted something about the world, and the groundedness property therefore meant that an answer traced to an assertion somebody made. After it, an answer may trace to an assertion a model generated in response to a prompt this system composed. The chain is complete, recorded and auditable at every step, and the derivation is disclosed at the point of citation — but the assertion at the end of it was produced rather than reported.

The three constraints are what make the narrowing survivable. Disclosure ensures the reader knows what they are relying upon. The self-corroboration bar ensures confidence cannot be manufactured by repetition. The reliability bound ensures a generated assertion cannot outrank a reported one by fluency. A deployment electing to disable any of the three would obtain a system in which a model's output is indistinguishable from a source's, and this specification does not describe such a configuration.

A deployment may operate without this module entirely, in which case the prohibition stands as it stood and no record in the store is model-derived. The module is elected per deployment, the election is recorded, and a response indicates whether any record it relied upon was model-derived.

Amendment to claim 162

Claim 162 presently denies a transfer from the domain owning evaluation of an output of an external system to the domain owning retrieval. It is amended to preserve that denial for a forensically-evaluated output and for a probe result, and to except a reply converted under claim 330, as follows: the words ‘such that neither said output nor an analysis thereof is admissible to said admitted set’ become ‘such that neither said output nor an analysis thereof is admissible to said admitted set, excepting a knowledge record converted from a reply elicited by a prompt composed by the method and carrying a derivation marking’.

Row 277 evaluates the diversity measure for three assertions elicited from one engine, returning an effective source-class count of 1.0000 against the 3.0000 that three distinct engines would yield.

Chapter 7 — addition

7.7.1 The update as a physical operation

The activation update already specified takes the form of a weighted sum over the other activations, compared against a threshold.

```
a_i(t+1) = sgn( Σ_j W_ij · a_j(t) − θ_i )
```

An array of variable-conductance elements arranged at the intersections of a set of rows and a set of columns performs the summation without computing it. Applying the activation vector as voltages on the rows produces at each column a current that is the sum over rows of voltage multiplied by conductance, by the ordinary laws governing current in a resistive network. The weight matrix is the conductance matrix, the summation is the current summation, and comparing the column current against a reference completes the threshold.

The consequence is a difference of order rather than of speed. For a network of 8,192 units, one digital update performs 67,108,864 multiply-accumulate operations; one analog settling performs a single physical operation whose duration does not scale with the network size. The mechanism specified in Chapter 7 is unchanged: what changes is that the sum is measured rather than calculated.

7.7.2 Why the convergence regime becomes a precondition

A conductance element cannot be manufactured to an exact value. Two arrays built to the same specification hold weight matrices that differ by the tolerance of their fabrication, and an array's conductances drift with temperature and with use. The question this raises is not whether an analog array converges — it does, by the same energy descent — but whether two arrays converge to the same state.

Under the capacity bound alone they need not. A network holding a number of configurations below the retrieval-phase boundary has several stable states, and which one an array reaches depends on where it began and on the exact weights it holds. Two arrays differing by fabrication tolerance may therefore settle differently on the same input, and both would be behaving correctly under the specification as it stands.

Under the monotone parameterization of §116.2 they cannot. The equilibrium exists and is unique independently of the initial configuration, so an array whose weights differ slightly settles to the same equilibrium as one whose weights do not, within the tolerance to which the equilibrium is read. An analog realization is therefore permitted only in the unique-equilibrium regime, and an array operated under the capacity bound is refused for this purpose.

This is the point at which a property specified for its own sake becomes the property on which a hardware realization depends. On conventional hardware uniqueness is a strengthening of a guarantee that already held statistically. On an analog substrate it is the difference between two devices agreeing and two devices being separately correct.

7.7.3 Registration, read-out and verification

An analog element is registered as a computational substrate under §4.10, with its array dimensions, its conductance range, its fabrication tolerance and the tolerance to which its settled state is read. An element whose registration lacks any of these is not usable for this purpose.

A settled state is read by comparing each column against the threshold and taking the sign, yielding the same discrete activation vector the digital update yields. The read-out tolerance is registered, and a column falling within the tolerance of the threshold rather than clearly on one side of it is resolved by a further settling rather than by rounding; where a further settling does not resolve it, the state is reported unresolved and the query falls back to the digital path.

Two arrays operating on a common input under a common weight specification are required to agree within the registered tolerance, and a divergence beyond it raises a fault rather than being averaged. Averaging two disagreeing analog results would produce a state neither array reached, and a state no array reached is not an attractor of either.

The trial architecture's determinism measurement applies to an analog realization without modification and is the mechanism by which agreement is established rather than assumed. A deployment electing the analog path records that election with the convergence regime, so a result carries which substrate produced it alongside which regime governed it.

7.7.4 What is not specified here

Row 270 evaluates the multiply-accumulate count a digital update performs for a network of 8,192 units, which one settling of a correspondingly-dimensioned array performs as a single physical operation.

Chapter 127

127.1 Feed registration

A specialized live data feed is registered as a source under the Source Connector Registry with the ordinary declared capability, and additionally with the properties this chapter requires: a publication authority, a refresh interval, a staleness bound, a value

class, a unit and its dimension, a restatement policy, and a redistribution entitlement. A feed whose registration omits any of these is connectable and is not admissible as a query source, because each of the omitted properties governs a decision the system must otherwise make by assumption.

127.2 Publication authority, and why calibration is bypassed

A value published by a registered authority is admitted at that authority rather than calibrated from an extraction confidence. The reason is that the ordinary calibration would invent an uncertainty that does not exist: a price published by an exchange is not eighty-seven per cent likely to be the price, it is the price the exchange published, and a confidence value below unity would misdescribe it. Where a value is admitted at publication authority, the extraction-confidence term, the modality coefficient and the source-reliability term are not applied.

Two qualifications bound this, and they are what keep the bypass honest. The value admitted is the value published, and the record asserts that the authority published it — not that it is correct. A published price that the exchange later restates was published, and the record of its publication remains true after the restatement. And a value whose class is a forecast is not admitted at publication authority in this sense: a forecast carries genuine uncertainty, the publisher ordinarily states it, and that stated uncertainty is admitted as the confidence rather than being replaced by certainty about the act of publication.

Where a publisher states an interval rather than a point — a forecast range, a bid and an ask, a confidence band — the interval is recorded as published and the point value, where one is required, is computed by a registered rule rather than selected. By way of example, a stated range of 18.2 to 21.6 yields a midpoint of 19.9, and the interval is retained alongside it so that a response may state the range rather than the midpoint where the range is what the requester needs.

127.3 Staleness

A value from a registered feed carries the time at which it was published, and is served only where its age at the time of the query is within the registered staleness bound. By way of example, a value published 47 seconds before a query, against a bound of 60 seconds, is served; the same value queried 61 seconds after publication is not.

A value outside the bound is not served with a caveat, is not extrapolated forward, and is not replaced by the nearest available. The response states that the most recent value is older than the bound and states its age, together with the value and its publication time where the requester's entitlement permits. This is the boundary-statement

behaviour the specification already applies to retrieval generally, applied to time rather than to scope.

The distinction from temporal decay is stated because the two would otherwise be conflated. Temporal decay widens a credible interval for an assertion that remains true and becomes less certain as it ages. A quoted value does not become less certain as it ages — it becomes superseded, and answering from it is not answering with lower confidence but answering wrongly. A feed value therefore does not decay; it expires.

A missed publication is detectable because the refresh interval is registered: where no value arrives within a configured multiple of the interval, the feed is marked interrupted and a query against it returns the interruption rather than the last value. A feed that has stopped publishing is distinguishable from a feed whose value has not changed.

127.4 Restatement, which is not correction

A publisher of a specialized feed frequently issues a revised value for a period already reported: an economic series is restated as fuller returns arrive, a forecast is reissued as the model is rerun, an index is recomputed as a constituent is corrected. The ordinary supersession chain treats a new version as replacing an erroneous predecessor, and applying it here would record that the earlier value was wrong. It was not. It was the value correctly published on the information then available.

A restatement is therefore recorded as a distinct relation. The prior value is retained, is not marked erroneous, and remains queryable as the value published at its publication time; the restating value is linked to it as a restatement rather than as a correction. A query about the period returns the restated value; a query about what was published at a time returns what was published at that time; and a query about the difference returns both with their publication times, which is the question an auditor of a decision taken on the earlier value actually asks.

A correction remains available and remains distinct. Where a publisher issues a value as a correction of an error rather than as a restatement on fuller information, the ordinary supersession chain applies and the prior value is marked erroneous. The registered restatement policy determines which relation a given publisher's revisions take, and a revision arriving under a policy that does not admit it is held for a named human rather than being classified by inference.

127.5 Redistribution entitlement

A specialized feed is commonly licensed on terms permitting a subscriber to see a value and not to republish it, and permitting a derived or delayed value where the value itself is withheld. The ordinary entitlement model expresses what a requester may reach; it does not express what a requester may do with what they reached, and this section adds that dimension.

Enforcement is at composition rather than at export. A response that would carry a value beyond its redistribution entitlement is composed without it, the omission being reported to the requester with the entitlement that produced it, so that a requester is told they are not entitled rather than left to infer it from an absence. An export operation upon a response is evaluated against the same entitlement and refused where it exceeds it.

A derived value carries the derivation and the entitlement of the feed it derives from, so an entitlement cannot be escaped by computing upon a value and then redistributing the result. Where a derivation is registered as sufficient to discharge the entitlement — a classification into bands, an aggregation across many values — the derived value carries the discharge and its basis.

127.6 Standing queries against a feed

The continuous query mechanism already specified applies to a registered feed without modification: a standing query registers a trigger clause and re-evaluates upon each relevant update, re-checking the Prime Directive at each firing. Three properties of a specialized feed bear upon it and are stated.

A trigger fires upon a published value and not upon the passage of time, so a feed that has stopped publishing produces no firings rather than firing upon a stale value. A trigger evaluated against a value outside the staleness bound does not fire. And a firing rate is bounded per standing query, an exceeded bound suspending the query and notifying its owner rather than continuing to fire, because a feed entering a volatile period would otherwise generate firings faster than they can be acted upon.

A standing query against a feed is metered under the ordinary compute accounting in its own category, scheduled below safety evaluation and below requester-serving, so a feed in a volatile period degrades the standing-query category and not the answering of a reader's question.

127.7 What is not admitted

Row 281 evaluates the registered midpoint rule over a published interval of 18.2 to 21.6, returning 19.9, the interval being retained alongside it.

Chapter 128

128.1 Definition, and the six phases

A defined agent is registered only where its definition is complete. Six phases are required, and a definition lacking any element of any phase is refused at registration rather than registered with a default supplied. The refusal is stated first because it is the

property from which the remainder follows: an agent whose behaviour is not fully determined before it runs cannot be evaluated before it runs.

An agent's declared capability set is bounded above by the capabilities of the identity registering it. An agent cannot be given authority its author does not hold, and cannot acquire authority by operating: a capability not declared at registration is refused at the boundary at execution time, as it is for any connector.

128.2 Execution

An agent executes as a bounded program under the program layer already specified, compiled from its operational function into a representation that is not Turing-complete: it admits sequence, conditional evaluation, bounded iteration over a set whose cardinality is known before entry, and typed operations over knowledge records. It admits no unbounded loop, no recursion without a bound, and no operation whose termination is not decidable before execution. Termination is therefore a property of the representation rather than a hope about the program.

Execution is triggered by a condition over admitted records or by a registered schedule, and the trigger is evaluated rather than assumed: a scheduled agent whose trigger condition does not obtain does not execute. Every execution resolves the agent's scope description before any traversal, and no parameter supplied at execution widens it. Execution is metered as a bounded compute-accounting unit in its own category, scheduled below safety evaluation, below requester-serving and below ingestion, so an agent campaign cannot displace a reader's answer.

The governance evaluation is performed at each execution rather than at registration. An agent does not accumulate authority by having been approved once, and an agent whose execution would produce an output the Prime Directive excludes does not produce it, the exclusion being recorded and the agent's owner notified.

128.3 What an agent may write

An agent's output is subject to the same disposition rules as any other derivation, and the rules are restated here because an agent is easily assumed to be an exception to them.

An agent therefore reads widely within its scope and writes narrowly outside it. The asymmetry is deliberate: the value of an agent is that it operates unattended, and the risk of an agent is precisely the same property, so what it may do unattended is bounded to what follows necessarily from what it read.

128.4 Lifecycle

An agent's lifecycle is defined before registration and is enforced rather than tracked. Six stages are recognised.

A lifecycle expiry is a bound and not a reminder: an agent registered for fourteen days terminates on the fourteenth day whether or not anyone attends to it. Renewal is a fresh registration carrying a fresh review and a fresh approval, so an agent cannot persist indefinitely by inattention.

128.5 Audit

Each execution writes a decision trace recording the trigger that fired, the scope description resolved, the records read, the rules evaluated and the branch each took, the outputs produced, every refusal and its cause, and the compute consumed. The trace is queryable, so a question of the form ‘why did this agent do that’ has a recorded answer rather than an inference.

Two properties of the trace are stated because they are what make it evidence rather than a log. It is written by the execution mechanism rather than by the agent, so an agent cannot omit from its own trace what it would prefer not to record. And it is written to the provenance chain under the ordinary keyed-checksum discipline, so a trace that has been altered after the fact is detectable.

A capability violation, a governance exclusion and a terminated execution are each recorded with equal prominence to a successful one. An agent's record is not a record of its successes.

128.6 Review, advisory, and the distinction between them

The submitted specification provides that the agent, before creation, analyses its own definition, warns of risks, confirms feasibility and confirms compliance. The analysis and the warning are adopted. The confirmations are not, and the distinction is the subject of this section.

A completed definition is analysed before registration and the analysis is presented to the registering identity. It identifies a scope exceeding the registrant's entitlement, a capability exceeding it, a termination condition that cannot obtain, a trigger condition that cannot be evaluated within the scope, a duration bound inconsistent with the work the function implies, and a rule admitting a non-deterministic resolution. Each finding is presented with what it would take to resolve it, and an unresolved finding blocks registration rather than accompanying it.

What the analysis does not do is confirm compliance. A mechanism cannot establish its own compliance, and an arrangement in which it purports to is worse than one in which nothing is checked, because it produces a confirmation on which a reader relies. Compliance is established by the governance evaluation, which is performed by the governance domain upon the agent rather than by the agent upon itself, and by the named human whose approval registration requires. The analysis informs that human; it does not stand in their place.

The same distinction governs the advisory stage during operation. An agent reports findings, warnings and anomalies to its owner, and those reports are advisory: they inform a decision and do not constitute one. Where an agent's report bears upon whether the agent should continue, the decision is the owner's or an operator's, recorded as theirs. An agent does not renew itself, does not widen its own scope, does not extend its own lifecycle, and does not clear its own findings.

128.7 Provisions implemented differently, and why

Row 288 evaluates the confidence of a deductive conclusion an agent may admit, as the product-bounded lesser of the traversed edge confidences, returning 0.55.

Chapter 129

129.1 Deployment topology by purchaser class

The five topologies specified at §107.1 are not equally suitable to every purchaser, and the multi-tenant hosted mode is unsuitable to every class treated in this chapter. The properties that make a topology suitable are stated alongside it, because a purchaser evaluating this system will test the property rather than the label.

Civilian government is served by a single-tenant customer-hosted deployment with tenant-scoped key custody under §107.5. Content and key material remain within the purchaser's infrastructure; accessible equivalents are produced at the same production stage as the content they accompany rather than as a subsequent remediation; erasure operates at session, work and account scope, serving a records-management obligation; and every access is recorded against an entitlement, serving a disclosure obligation.

Defence and intelligence are served by a single-device or fully disconnected deployment under §107.1 and the substrate independence of §107.7. Ingestion, retrieval, inference and generation all execute without a network at any point. No claim recites an external system as an element, no transfer-matrix entry permits a domain to require an external resource to complete an operation, and determinism is established by the vocabulary constraint and the bounded stable-state capacity rather than by any

external property. The independence is structural rather than asserted and is therefore testable by the purchaser.

Scientific and research purchasers are served by a customer-hosted or edge deployment, and the property purchased is reproducibility rather than efficiency: an identical query over identical records under an identical context key yields an identical result, the key being returned with the output so that a result is re-derivable at a later date and demonstrably the same result. Every proposition resolves to a cited source, and a disagreement among sources is recorded rather than resolved by selection.

Regulated industry is served by a customer-hosted deployment with hardware key custody where configured, in which the system holds a handle rather than the key and every use is a call across the custody boundary. The provenance chain is signed under a deployment key, so a third party holding the verification key confirms that a recorded operation and its recorded safety evaluation are as written, without access to the system.

Education and public institutions are served by a customer-hosted deployment, or by an edge deployment where a campus concentration warrants one. An edge node holds no authority, answers from cache only where the requester's scope admits every cached record consulted, and escalates otherwise. Per-individual predictive records are refused at construction, which is the condition such purchasers examine first.

129.2 Licence form

A per-seat licence is unsuitable to every class above, for two reasons stated together. It creates a counting obligation the purchaser must administer and the operator must audit, in institutions where headcount is variable and where an audit of it is itself sensitive; and it prices a system whose cost is driven by corpus scale and query volume by a quantity related to neither.

Three terms are common to every form. A licence is scoped to a deployment and to a corpus rather than to a set of named individuals. A licence does not condition operation upon a connection to the operator, because four of the five topologies do not have one. And a licence termination does not render an already-composed output invalid, because an output is a record of what the records supported at a time and its validity does not depend upon a commercial relationship subsisting afterwards.

A per-deployment licence with a corpus-scale tier suits civilian government, regulated industry and education. The tier is a function of admitted record count and query volume, both of which the compute-accounting mechanism already meters, and a tier boundary crossed mid-term is reported rather than enforced retrospectively.

A per-deployment perpetual licence with a maintenance term suits defence and intelligence and any purchaser operating disconnected. A disconnected deployment

cannot be licence-checked, and a licence whose enforcement requires a connection is unenforceable in the topology that class requires; the licence is therefore perpetual as to operation and term-limited as to maintenance and updates.

A site or consortium licence suits scientific and research programmes spanning several institutions. Entitlement is resolved per requester against the consortium's registered membership, so a member institution's departure narrows entitlement at the next query rather than at the next renewal.

A source-available or escrow arrangement suits defence, intelligence and any purchaser requiring inspection, in which classes inspectability is frequently a precondition rather than a preference. This system tolerates it better than most, because its guarantees are structural properties of the specification rather than properties of an unpublished implementation: a purchaser who reads the source learns how the guarantees are met and learns nothing that weakens them.

129.3 Six conditions unmet at the date of this chapter

The following are recorded as unmet. Each is stated with what it blocks and with whether it is a matter of engineering, of assessment by a competent party, or of measurement. The distinction matters, because a condition requiring measurement can be addressed by the applicant alone whereas a condition requiring assessment cannot.

First, no FedRAMP or StateRAMP authorization is held. This is a matter of assessment, requiring a third-party assessment organisation and a sponsoring agency, and it blocks United States federal civilian procurement generally, where it is a threshold rather than a preference. A customer-hosted deployment upon the purchaser's own authorised infrastructure may in some circumstances proceed under the purchaser's own authorization; that determination belongs to the purchaser's authorising official and not to this document.

Second, no validated cryptographic module is specified. This is a matter of engineering followed by validation. The specification states encryption, key scoping and hardware custody, and states no validated module; the term appears nowhere in the corpus. It blocks federal deployments generally and a number of regulated-industry deployments. A deployment must use a module validated under the applicable programme, and the specification must name the requirement rather than leaving the choice to an implementer.

Third, multi-level security is not specified. This is a matter of engineering. The entitlement model resolves per requester and per record; it does not resolve per classification level, and the handling of material at differing classifications within one deployment is not specified. It blocks any deployment holding material at more than one classification. A deployment at a single level is unaffected, which is the configuration a first defence engagement would sensibly take.

Fourth, no certification of any kind is held. This is a matter of assessment. §115.8 states the position expressly for assistive equipment and the position is the same system-wide: conformity is established by hazard analysis, verification and testing performed upon a specific configuration by a party competent to certify, and no provision of this specification substitutes for it. It blocks any procurement conditioned upon a named standard, and a purchaser asking which standard this system is certified against receives the answer none, which should be given plainly.

Fifth, nothing has been measured at scale. This is a matter of measurement and is therefore within the applicant's own control. It affects every purchaser class. Determinism, refusal integrity and calibration are claimed, are testable, and are untested. Chapter 117 specifies the instrument — coverage against an enumerated trial space with the residue listed, determinism over an unchanged corpus state where any value below unity is a defect, and adversarial trials measuring whether a stated refusal actually fires. A purchaser will ask for the results and not for the design.

Sixth, this system's export control status is unassessed. This is a matter of assessment. Export control is referred to at many points in the corpus in relation to content and territory, and this system's own classification has not been determined. It blocks any engagement with a defence purchaser and any deployment outside the applicant's jurisdiction, and the determination should be obtained before such a conversation is opened rather than during one, because opening it is itself a disclosure.

129.4 Sequencing

The five conditions requiring assessment or engineering are addressed on timescales the applicant does not wholly control. The fifth — measurement — is within the applicant's control entirely, requires no external party, and should be addressed first.

The reason is not that it is the easiest. It is that it is the condition every purchaser in every class asks about, and the only one whose absence undermines the others. An authorization obtained for a system whose central properties have never been measured is an authorization of a design. A certification of an unmeasured system certifies a process. A purchaser who is told that output is deterministic, that a refusal is structural, and that confidence is calibrated, and who then asks what those properties measure at, currently receives an architectural argument in place of a figure.

Running the trial architecture converts the claims of this specification from properties asserted to properties measured, and the measurement is itself evidence of the discipline a government or regulated purchaser is assessing. It also has an internal value independent of procurement: a refusal that has never been exercised is an assumption, and this specification contains a large number of claimed refusals none of which has been adversarially tested.

The recommended order is therefore: measurement first; then cryptographic module selection and multi-level security, both being engineering within the applicant's control; then export classification, which should precede any defence engagement; then authorization and certification, both of which are assisted by the measurement results and impeded by their absence.

129.5 What this chapter does not establish

It establishes no authorization, no validation, no certification and no export classification. It records that none is held. It does not constitute legal advice, and the licence forms it describes are engineering and commercial recommendations upon which counsel's own drafting supersedes.

And it makes no representation that a topology suitable to a purchaser class is sufficient for a particular procurement. Suitability is a property of the architecture; sufficiency is a determination of the purchaser's own authorising official, made against requirements this document has not seen.

Chapter 130: The Self-Knowledge Layer (SKL)

In this chapter § numbers refer to the sections of the Core Technology Overview (v15); chapter numbers refer to this Bible.

The Self-Knowledge Layer (SKL) is the part of QueryBook that describes QueryBook. Asked what it is, how it reasons, whether it is neuromorphic, whether it predicts, or whether it improves itself, the system answers from three verified layers and from nothing else: the specification (the numbered sections of the Core Technology Overview (v15)), the implementation (the module and symbol that carry out each section, checked against the program's own source files at the moment of asking), and the evidence (the host application's own self-tests, diagnostics and measured scores). The layer never generates its self-description with a language model, and it never repeats a claim its own code does not support.

The SKL follows from the Prime Directive (Domain 0; Overview §10): a system that refuses to state an unsupported fact about the world must not state an unsupported fact about itself. It is the self-referential application of the Fact Unit discipline (§1) — every statement QueryBook makes about itself carries a status and its sources, exactly as a Fact Unit carries provenance.

130.1 Three layers of self-knowledge

Specification. Every numbered section of the Core Technology Overview (Overview §1 through §56) is indexed by number and title. A self-question that names a capability is resolved to the sections that specify it.

Implementation. For each section and each reasoning aspect, the layer holds evidence pairs — a source file and a symbol that must appear in it (for example, the function that computes Fact Unit identifiers). The pairs are checked against the running program's files every time they are used. The layer reads source files as text and imports nothing it describes, so the check is fast, deterministic and safe.

Evidence. The host application may pass in its own check results (self-test, diagnostics, quality measurements); the layer reports them beside the code evidence.

130.2 Capability status labels

Every statement carries exactly one of five labels: built and tested; partly built; specified, not yet built; not planned (by design); not in this application. A sixth marker, description, not a capability, is used for sections that describe rather than define behaviour (for example §42, Comparison With Human Cognition). A capability is labelled built only when all of its evidence is present; a specified capability is never described as built.

130.3 The reasoning aspects

Twelve aspects of QueryBook reasoning are described at three levels of detail (brief, user, engineer), each with its status and sources: deterministic reasoning (§6, §1); atomic knowledge, the Fact Unit (§1, §12); the World-View Model (§11); predictive cognition (§21, §27); the neuromorphic layer (§3, §35); hypothesis generation and simulation (§23, §29); deduction, induction and abduction (§23); feedback loops (§26, §27); supervised self-improvement (§26); recursive self-improvement (§10); and self-knowledge itself (§56).

In the present prototypes deterministic reasoning, deduction over verified premises, hypothesis and simulation agents (proposing only, kept apart at low trust), measured feedback loops and supervised self-improvement are built; the Fact Unit is partly built; the World-View Model, the Predictive Cognitive Architecture and the Neuromorphic Cognitive Layer are specified and not yet built. Recursive self-improvement — a change to the program without a person's approval — is not planned, by design: every calibration and every code change proposed by the Engineer is tested on a copy and applied only with the administrator's approval.

130.4 The process model

The layer describes QueryBook's own process in eight steps, each tied to its sections: ingest (§7, §43, §50); atomise into Fact Units (§1); verify against the store, keeping AI-proposed content apart (§4, §10, §40); store immutably with a hash-chained record (§6, §53); reason, where only deduction asserts (§23, §29); answer with citations or refuse (§10, §31); measure against references (§26, §56); improve with approval (§26).

130.5 Self-consistency check

Before it speaks, the layer checks itself. A claim labelled built or partly built whose evidence is missing from the program is reported as a contradiction and is not repeated. A capability labelled specified whose implementation names (for example, an attractor or world-view class) appear in the program is also reported, so the description is updated when the code moves ahead of it. The check is part of every self-test.

130.6 Gap agenda

Every section that is specified but not built, or only partly built, is listed with its number, title and nearest built component. The list is the system's development agenda: it says what QueryBook does not yet do, in the system's own words, and it is never presented as capability.

130.7 Answering questions about itself

Self-questions (''Are you neuromorphic?", "Do you improve yourself?", "What is §21?") are matched by rule to an aspect or a section number and answered with the label and the sources: the section numbers and the file and symbol evidence. Answers are deterministic: the same question about the same program gives the same answer. A disclosure filter removes anything in an answer that resembles a key or credential, and the layer never reads settings or keys.

130.8 Interfaces

The layer is available in prototype v9.64 and in each application (Language, STEM, History and News, Control Panel) as GET /api/self (summary, label counts, aspects, process, sections, contradictions, gaps) and GET /api/self/ask?q=…&level=… (one answer with its sources); as the Self-knowledge page in v9.64 and the Self-knowledge tab of the Control Panel; and as a rule of the Control Panel assistant.

130.9 Status

The Self-Knowledge Layer is built and tested in prototype v9.64 (qb_selfknow.py, 1.0.0) and in the shared core of the applications (qbselfknow.py, 1.0.0). Its self-test (28

checks) runs inside every application's self-test; at release it reported no contradictions in either codebase.

130.10 Correspondence with this specification

The layer's twelve reasoning aspects correspond to chapters of this Bible as follows. Deterministic reasoning: Chapter 5 (Provenance Ledger) and the determinism guarantee of the context lock. Atomic knowledge: Chapter 3 (Fact Unit Structure). World model: Chapter 53 (World Aware System Model). Predictive cognition: Chapter 10 (Predictive Inference Engine), Chapter 41 (Predictive Cognitive Architecture) and Chapter 103 (Temporal Prediction Calibration Layer). Neuromorphic layer: Chapter 7 (Neuromorphic Cognitive Layer) and Chapter 101 (Verified Neuromorphic Correspondence Methodology). Hypothesis generation: Chapter 97. Simulation: Chapter 11 (Scenario Graph). Feedback loops: Chapter 66 (Improvement Proposal Pipeline) and Chapter 77 (Validation and Benchmarking). Supervised self-improvement: Chapter 66 and Chapter 78 (QueryBook AI Evolution). Recursive self-improvement: excluded under Domain 0 (the Prime Directive) and the approval gates. Self-knowledge: this chapter, with Chapter 40 (Specification Validation Framework), which validates the specification as this layer validates the system's description of itself.

A chapter of this Bible that specifies a capability not yet built remains specification. The layer reports such a capability as specified, not yet built, and lists it in its gap agenda; it never reports specification as implementation.

Registry and claims: Composite Feature Registry v70, features 383–391 (algorithms ALG-20 and ALG-21); provisional application, FIG. 57 (reference numerals 5700–5799) and claims 358–362. (Drafting record, Pass 119, October 22, 2026.)

Chapter 131: The Digital Synaptic Network (DSN) and Network Memory

In this chapter chapter numbers refer to this Bible; § numbers refer to the Core Technology Overview (v15).

The Digital Synaptic Network borrows the communication economics of neurons — local work, sparse signalling, thresholds, inhibition and connections that change with use — for the way QueryBook's agents find and share knowledge. It has two parts. Answer routing decides which agents (in the applications, the dataset stores of the bulk harvest) are asked for a lookup: every one in turn (Central), only those its learned pathways point to (DSN), or Central with the DSN compared beside it (Shadow). Network memory treats the relationships between agents as a layer of memory: agents that succeed together form digital synapses that strengthen with repeated success,

weaken with joint failure, fade when unused and are consolidated when long proven; strongly bound agents form assemblies, and assemblies linked through their members form networks of networks.

Both parts are governed by the same rules as every QueryBook fact. Learned pathways and synapses hold relationships, never facts; they never change what an answer says; and every one of them can be inspected — its strength, its threshold, the contexts it formed in, its good and bad outcomes, its evidence and its history. The design principle is stated plainly: do not only make the agents smarter, make the connections between them able to learn — and keep those connections auditable.

131.1 Answer routing: Central, Shadow and DSN

Central asks every dataset store in turn until enough facts are found; it is the default. DSN asks only the stores whose learned pathways for the kind of lookup are at or above threshold, strongest first, and stops when enough facts are found; when the strong pathways find nothing, every remaining store is asked, so routing never hides a fact. Shadow answers with Central and runs the DSN on the same lookup in the background, logging every difference. The setting is changed by the administrator (with the admin passphrase when one is set) on the Control Panel, and changes apply between lookups, never during one.

131.2 Learned pathways

The kind of a lookup is the first word of its subject, lower case, digits removed (a lookup beginning with a number is kind #). For each kind and each store the DSN keeps a pathway with a strength (neutral 0.5), the facts found and the empty answers. A store that returns facts is strengthened (strength + 0.10 × (1 − strength)); a store that returns nothing is weakened (strength − 0.15 × strength); a pathway below the threshold of 0.30 is not asked first; weak pathways recover slowly toward neutral (0.2% of the gap per lookup), and every 25th lookup of a kind tries one weak pathway again. A new kind starts neutral, so its first lookups ask every store and learn. Pathways are saved in the shared folder and capped at 2,000 kinds, the least recently used forgotten first.

131.3 The Shadow comparison and its verdict

Every Shadow lookup records the lookup, its kind, whether Central's answer was complete, whether the DSN agreed, the facts it missed and added, whether it fell back to every store, and for each method the facts returned, the stores asked and the time taken. When Central returns fewer facts than the limit its answer is complete and the two answers are compared fact by fact; otherwise both are compared by whether the DSN also filled the limit. The health report states the routing in force and summarises the last 24 hours: lookups compared, agreement, lookups with missed facts, stores

asked and median time for each method, and a verdict — too few lookups yet (under 50), keep Central (the DSN missed facts), no reason to switch (matched but not faster), or DSN is safe to try (matched every answer and was faster).

131.4 Re-checking learned routes

Routes learned on one set of stores can be wrong for another. When the DSN is switched back on after Central, every pathway is pulled halfway back to neutral, so it is re-checked by the lookups that follow rather than trusted.

131.5 The stress test

A standard-library stress test (querybook/sim/dsn_stress.py) compares six engines on the same seeded questions over the same facts: the prototype as installed without a language model (A0), with a planned first lookup (A1), its pipeline chained step by step (A2, the control), conventional broadcast multi-agent messaging in text (B), the DSN without learning (C0) and with learning (C). Stresses include wrong, missing and contradicted facts; lying, dead, slow and biased agents; up to 10,560 agents and one million facts; poisoned learning feedback; faults that change mid-run; and 32 parallel requests. Communication is modelled (0.5 ms plus bytes at 100 MB/s per message; 250 ms timeout for a dead agent). It ships with a bundled Python for Windows and launchers that say why they stop.

A learning-over-time study runs one long stream of recurring questions, reported every 500, scored against the real world rather than the store, with the faulty agents changed half-way and the agents' areas re-sharded at three quarters. It compares today's DSN with three upgrades — agents that learn verified facts and shortcuts (C+1), new pathways found by probing (C+2), and feedback on accuracy by outside audit (C+3) — and with the central pipeline given the same fact learning.

131.6 What the stress test found

With faulty agents the DSN answered correctly far more often than broadcast messaging (100% against 31%; with every stress at once 72% against 26%, with no wrong answers against 4.6%) while sending about five messages of 60–80 bytes per question against hundreds to thousands. The chained central pipeline was not beaten: 99.9% correct with every stress at once, and faster in its worst case on one machine (0.2 ms against 4.2 ms). Learning synapses alone did not change accuracy; they cut worst-case time and messages. In the learning-over-time study accuracy rose only when agents learned facts (DSN with all three upgrades 68% → 91% right; today's DSN flat at about 27% once areas were re-sharded), learning without accuracy checks cached mistakes (wrong answers 3.3% with C+2 against 1.3% with C+3), and the

central pipeline with the same fact learning remained slightly ahead (73% → 93%) and faster.

The conclusion is recorded as stated: on one machine the central pipeline is the faster and slightly more accurate design, which is why Central is the default; the DSN's advantages come from facts spread over many stores or computers and from learning verified facts, and Shadow exists to measure on real data whether the DSN helps before it is trusted.

131.7 The digital synapse

A digital synapse is a relationship between two agents or processing elements. Its record holds the relationship type (co-activation, or a directed type such as checks or feeds), its strength, its activation threshold, the contexts it formed in, its reliability as counts of good and bad outcomes, its provenance (the last five pieces of evidence), its history (the last twelve changes), when it formed, when it was last used, and whether it is consolidated. The threshold is 0.30 plus 0.40 times the share of bad outcomes, so an unreliable synapse needs more to pass activation. Every synapse explains itself in plain lines.

131.8 Plasticity

A new synapse forms weak (0.20, below threshold): one coincidence is not a relationship. Each joint success potentiates it (strength + 0.10 × (1 − strength)), so about two successes carry it over threshold and about five bind its agents into an assembly; a joint failure depresses it (strength − 0.25 × strength) and raises its threshold. An unused synapse fades by 5% a day; one with at least 20 good outcomes and four times as many good as bad is consolidated and fades by only 1% a day; a synapse that has faded below 0.05 and has been idle for 30 days is removed (turnover). Formation, potentiation, depression, fading and pruning are counted.

131.9 Co-activation, episodes and directed links

Agents become linked in three ways. An episode names the agents active together in one piece of work — for example the specialists whose facts answered one question, or the stores that answered one lookup — and every pair among them (at most eight agents) is potentiated on success or depressed on failure. A single activation links an agent with every agent that succeeded within the co-activation window (600 seconds by default): fire together, wire together. A directed link records that one agent's output was used by another, for example a check asked by one application's team and answered by another's.

131.10 Engrams, assemblies and networks of networks

For each context — a kind of problem — the network keeps its engram: the population of agents active when that problem was solved, with how often each was active. Agents joined by co-activation synapses of strength 0.50 or more form an assembly, a functional group with its members, cohesion (mean strength), joint successes and the contexts it solves. Synapses above threshold between members of different assemblies link the assemblies: networks communicating with networks, each link with its number of synapses, mean strength and contexts.

131.11 Spreading activation

Activation spreads from the active agents along synapses above their own threshold, multiplied by the synapse's strength and a falloff of one half at each step, for two steps. The result ranks who else is likely to matter now: in the applications it orders the stores asked when the strong pathways found nothing (the strong stores' assembly partners first); in the prototype it is reported with each answer as the related specialists.

131.12 Where network memory is wired

Prototype v9.64: every answered question is an episode for its specialists (the predicates of the facts cited), in the context of the question's planned predicates; a verified answer potentiates them, a contradiction depresses them; the answer itself is never changed and reports the assembly its specialists belong to and the related specialists. Harvesting agents are activated at the end of every clean cycle, a cycle that added facts counting as success. Applications: the stores that answered a lookup form an episode of that lookup's kind; and every cross-application check (asked by one team, answered by another) strengthens the directed link between the two teams when an answer was given and weakens it when the answer was cannot check. Each application writes its own network file, so separate processes never overwrite one another.

131.13 Interfaces

Prototype: GET /api/network (agents, synapses, assemblies, assembly links, engrams, strongest synapses, counters, the health line, and with a=…&b=… one synapse's explanation) and the Network page. Applications: GET /api/dsn/status (routing, Shadow summary, learned pathways, network memory of the stores and of the checks between applications) and POST /api/dsn/mode; the Answer routing section of the Control Panel; and the health report, which carries the routing, the Shadow comparison and one network-memory line for each network.

131.14 Status and limits

Answer routing is built and tested in the applications' shared core (qbdsn.py, Control Panel 1.34.0); network memory is built and tested in prototype v9.64 (qb_synapse.py) and in the applications' shared core (qbsynapse.py, Control Panel 1.35.0). Both are deterministic for the same events in the same order and use the standard library only. They record, adapt and explain how agents work together; they do not by themselves create intelligence, and no claim beyond what is built is made for them.

131.15 Correspondence with this specification

The DSN is the implemented, measured part of the neuromorphic direction specified in Chapter 7 (Neuromorphic Cognitive Layer). It follows the method of Chapter 101 (Verified Neuromorphic Correspondence Methodology): it names a correspondence with biology — synaptic strength, thresholds, inhibition, plasticity, engrams, neuronal ensembles — and states what is built, what was measured, and where the correspondence stops. Chapter 7 as a whole remains specification; the Self-Knowledge Layer (Chapter 130) continues to report the Neuromorphic Cognitive Layer as specified, not yet built.

Network memory corresponds to Chapter 95 (Complementary Learning Systems Formalization): engrams and consolidation record which agents solved which kinds of problem, beside the fact store that records what is true. Assemblies and links between assemblies extend Chapter 98 (Cognitive Map Formalization) from concepts to the agents that process them. The agents themselves are those of Chapter 56 (Robot Memory Domain — Autonomous Agent Integration with the Fact Universe) and the harvesting teams of the Commons; the cross-application checks they exchange are the cooperation those chapters specify, now remembered as directed synapses.

Governance is unchanged. Under Domain 0 (the Prime Directive) a synapse or pathway never asserts a fact and never alters an answer; every one keeps its provenance as a Fact Unit does (Chapter 3, Chapter 5); the routing setting is an administrator's decision; and the stress test belongs with Chapter 77 (Validation and Benchmarking), whose rule it keeps: results are reported as measured, including where the new design loses to the old.

Registry and claims: Composite Feature Registry v71, features 392–407 (algorithms ALG-22 to ALG-25); provisional application, FIG. 58 (reference numerals 5800–5899), FIG. 59 (5900–5999) and claims 363–374. (Drafting record, Pass 120, November 7, 2026.)

Chapter 132: The Four-State Fact Unit

In this chapter chapter numbers refer to this Bible; § numbers refer to the Core Technology Overview (v15).

A Fact Unit is no longer read as simply held or not held. Every Fact Unit that QueryBook returns now stands in exactly one of four truth states — true, false, undetermined or conflicting — and carries, beneath that state, the backing that explains it and a weight. A fact with nothing recorded about its evidence is undetermined; it is never assumed true. A second, separate field records identity: whether the subject of the fact is resolved to one known entity, unresolved, ambiguous or conflicting. A name printed in a newspaper is true as printed and unresolved as to who.

The four truth states are the two-valued reading of the evidence pair of Chapter 3: one bit records that confirming evidence exists, the other that the assertion has been found wrong. Neither bit set is undetermined; the first alone is true; the second alone is false; both are conflicting. This is the four-valued logic known in the literature as Belnap's, and it is the same shape as the four-state logic used to describe digital hardware, in which a line is 1, 0, undriven (Z) or driven two ways at once (X). The correspondence is exact enough to compute with: evidence from separate records is joined by a bitwise OR.

The four states at a glance

true — code 10 — confirming evidence, none against — in hardware terms: logic 1.

false — code 01 — found to be wrong — in hardware terms: logic 0.

undetermined — code 00 — no confirming evidence either way — in hardware terms: Z — nothing drives the line.

conflicting — code 11 — evidence for and evidence against — in hardware terms: X — drivers disagree.

The eight backings and their weights

verified — state true — weight 1.0 — checked and confirmed.

computed — state true — weight 1.0 — calculated, and the calculation can be repeated.

corroborated — state true — weight 0.9 — two or more different sources agree.

sourced — state undetermined — weight 0.75 — one named source says so; not yet confirmed by a second.

reported — state undetermined — weight 0.4 — a report says so; not confirmed.

unsourced — state undetermined — weight 0.2 — no source is recorded.

disputed — state conflicting — weight 0.3 — sources disagree.

rejected — state false — weight 0.0 — found to be wrong.

132.1 The four truth states

True: the assertion is confirmed and nothing stands against it. False: the assertion was checked and found wrong; the record is kept, marked, so that the error is not harvested again as new. Undetermined: no confirming evidence either way — the assertion may have a source, but one source is a report, not a confirmation. Conflicting: there is evidence for the assertion and evidence against it, or the sources disagree. The states are exhaustive and exclusive; every Fact Unit has exactly one.

The rule that decides between undetermined and true is deliberately strict. One source leaves a fact undetermined however eminent the source. A fact becomes corroborated, and so true, when its own record names two or more different sources (the work a statement is stated in, or failing that the site of its address). Nothing else promotes a report: not popularity, not demand, not the number of times the same source repeats it (Chapter 3's self-corroboration bar).

132.2 Backing and weight beneath the state

The state is the summary; the backing is the reason. Eight backings are kept, each with its state and weight: verified 1.0 (true); computed 1.0 (true); corroborated 0.9 (true); sourced 0.75 (undetermined); reported 0.4 (undetermined); unsourced 0.2 (undetermined); disputed 0.3 (conflicting); rejected 0.0 (false). The backing is derived from the status the fact was stored with, the sources named in its own record and whether another record contradicts it; a status the engine does not know is treated as unsourced. A contradicted fact is conflicting whatever its backing, except a rejected one, which stays false.

The weight is a fixed table, not a learned score. It lets an answer prefer a confirmed fact to a reported one and lets a reader see at a glance how far a statement can be leaned on. It does not replace the evidence pair and Beta posterior of Chapter 3 and §41.6.2, which remain the confidence model where observations are counted; the weight is the value used when only the backing is known, as it is for facts harvested in bulk.

132.3 Every fact is rated

Rating is not a feature of one page. Wherever facts are returned — the bulk store's entity search, the Commons, History & News, the Ask box — each fact carries its state, backing, weight, the reason in words and its two-bit code, and an answer carries a tally of how many of its facts stand in each state. A new place that returns facts must rate them. The rating is computed from the stored record each time it is returned; nothing in the facts table and no fingerprint is changed by it, so the identity of a Fact Unit (Chapter 3) is exactly as before.

132.4 The two-bit evidence code and the join

Each state has a two-bit code: undetermined 00, false 01, true 10, conflicting 11. The first bit means confirming evidence exists; the second means the assertion has been found wrong. When two records speak to the same assertion their states are joined by OR-ing the codes: true joined with false is conflicting; undetermined joined with anything is that thing; conflicting stays conflicting. The join does not depend on the order in which evidence arrives, and joining the same evidence twice changes nothing, so stores can be merged in any order and re-merged safely.

Because the code is two bits, 32 Fact Unit states fit one 64-bit word, and one OR instruction joins 32 states with 32 others. The eight backings fit three bits, and since the backing determines the state, three bits carry both. These packed forms are built and tested as pure functions; the store's file format does not yet use them — the state is computed when a fact is returned.

132.5 Compatibility with processors, present and future

Present processors are binary, and the four-state Fact Unit needs nothing else. It is defined as logic, not as a voltage: two bits per state on any binary machine, combined by the OR instruction every processor has, 32 states to a 64-bit register and proportionally more in wider vector registers. Four-state logic is already familiar to chip designers — hardware description languages simulate lines as 0, 1, Z and X — but that is a correspondence of meaning, not a dependency: QueryBook does not require a tri-state bus or any special circuit.

If future processors or memories offer digits with four levels, one digit holds one state and the join becomes a single four-level operation; memory cells that hold four levels (two bits per cell) already exist in flash storage. If they offer more than four levels, the extra levels carry the backing: an eight-level digit holds the backing, and with it the state. If they never offer more than two levels, nothing changes. In every case the stored meaning, the tables and the answers are identical, because the hardware changes only how many bits or digits a state occupies, never what the state is.

Further states can be added without disturbing the four: a wider code keeps the first two bits as they are. The design does not need them today; the fuller distinctions people want (verified, corroborated, reported and so on) are the backing.

132.6 Identity is a separate state

Whether a statement is true and who or what it is about are different questions, and each has its own four states. Identity is resolved when the subject carries its record's own identifier; unresolved when it is only a name; ambiguous when several known

entities answer to it; conflicting when identifiers disagree. A name harvested from a newspaper page is recorded as printed — true as printed, status reported — and its identity is unresolved: it stands for whoever bore that name. It is shown apart from any known person of the same name and is never merged by name alone.

132.7 Weighing a mention against a known person

When a newspaper name could be a known person, each mention is weighed against that person's own facts: was the person alive, and no longer a child, when it was printed; does the paper name an office the person held; is the paper from one of the person's places; does it report a death in the year the person died, or an appointment in the year the person took office. The result is likely, possible or impossible, with its reasons. After the person's death a mention is never more than possible. This is an analysis shown beside the answer; a mention is never merged into the person's record.

132.8 Earlier Fact Unit changes recorded here: context beside the fact

Since engine 1.48 every statement harvested from a structured source keeps its context — start, end and point in time, other qualifiers, rank and sources — stored beside the fact, keyed by item, predicate and stored value, in a context table. The facts table and the fingerprint are untouched, so a Fact Unit's identity does not change when its dates arrive. A date is shown only when the store holds one; a missing date is never supplied. The Fill Dates action reads one subject's record from the source its facts cite and adds dates, rank and sources beside the facts; it never adds a fact, and a fact for which the source states no date says so. The sources kept here are what the corroboration rule of 132.1 counts.

132.9 Earlier Fact Unit changes recorded here: what is not a fact

Four kinds of record are never mixed: an observed measurement (how often a subject was read in a period), an observed navigation, an inferred intent (always labelled as inferred) and a verified fact — the only one that is a Fact Unit assertion. Demand decides what to harvest first; it never raises a fact's state, weight or confidence. A ranking or a prominence score is an analytical output stored apart from facts. An as-printed news fact (name, date, newspaper and town, page, reason, the paper's own words, linked to its page) is a fact about what was printed, with status reported.

132.10 What is built and what is not

Built and tested in the one-window QueryBook (engine 1.56): the four truth states, eight backings and weights; the corroboration rule; rating of every returned fact with tally; the two-bit code, the join and the packed forms as pure functions; the identity state and the unresolved zone; mention weighing; the context table, Fill Dates and dated display. Not built: packed states in the store's file format; the join applied automatically across datasets; the ambiguous and conflicting identity states beyond their definitions; any hardware beyond ordinary binary processors, which is all the design requires. The correspondence with four-state hardware logic is a correspondence of meaning (Chapter 101's method): named, bounded, and claimed no further than the join it makes computable.

132.11 Correspondence with this specification

The four-state Fact Unit extends Chapter 3 (Fact Unit Structure): the truth state and its two-bit code are the coarse reading of the Layer 7 evidence pair, the backing is the reason beneath it, and the context table is where a statement's dates, rank and sources are kept without altering the immutable record. Chapter 5 (Provenance Ledger) supplies the sources the corroboration rule counts. §41.6.2 (the Beta posterior) remains the confidence model wherever observations are counted.

Governance is unchanged and made visible. Under Domain 0 (the Prime Directive) an unconfirmed statement is shown as undetermined rather than asserted; a report is never promoted by repetition or demand; a name is never taken for a person. The rating changes no stored fact and no fingerprint.

Registry and claims: Composite Feature Registry v72, features 408–422 (algorithms ALG-26 to ALG-29); provisional application, FIG. 60 (reference numerals 6000–6099), FIG. 61 (6100–6199) and claims 375–386. (Drafting record, Pass 121, November 14, 2026.)